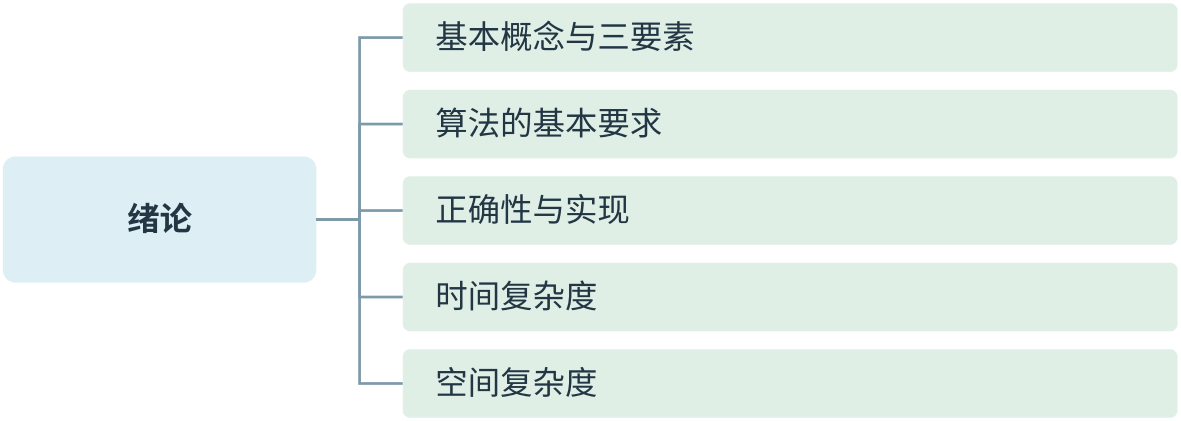
1 绪论

章节脉络



本章中，n 表示输入规模。代码中的数组下标和逻辑位序分别说明；后续各类结构按教材给出的存储表示确定有效范围。指针变量保存结点地址，`p->next` 是 `p` 所指结点的后继指针域；`p=NULL` 表示 `p` 当前不指向任何有效结点。伪代码中的赋值号 `=` 不表示数学等式，循环边界和执行顺序同样是算法的一部分。

课程连接·逻辑与物理

同一逻辑关系可以用不同存储结构实现。顺序或链式描述物理安排，线性或非线性描述元素之间的关系。

1.1 逻辑关系怎样落实到存储

数据是信息的载体，是能输入计算机并被程序识别和处理的符号集合。**数据元素**是数据的基本单位，一个元素可由若干数据项组成。例如一条学生记录是一个元素，学号和姓名是它的数据项。**数据对象**是性质相同的数据元素的集合。讨论数据结构时，还必须说明元素之间的关系：是排成先后序列，形成父子层次，还是由任意边相连。

逻辑结构描述元素之间的逻辑关系，不涉及具体地址。集合结构没有规定元素间的先后关系；线性结构中，除首尾元素外，每个元素都有一个直接前驱和一个直接后继；树形结构表达层次；图结构表达更一般的关联。栈和队列都以线性关系为基础，但允许的操作位置不同，因此属于不同的抽象数据类型。

存储结构是逻辑结构在计算机中的表示。顺序存储用相邻位置体现关系，链式存储用指针联系结点，索引存储另建索引以定位元素，散列存储由关键字计算位置。四者都只是表示方法，不能把“线性结构”直接等同于“连续地址”。线性表既能顺序存储，也能链式存储。

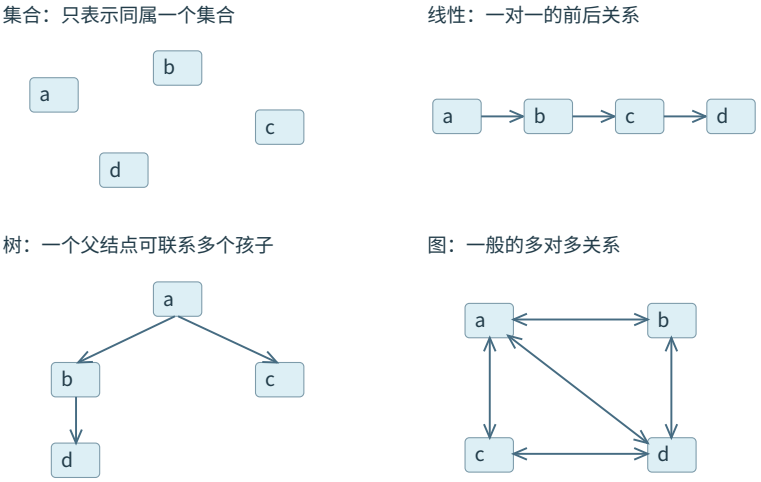


Figure 1.1: 集合、线性、树和图的关系结构对照

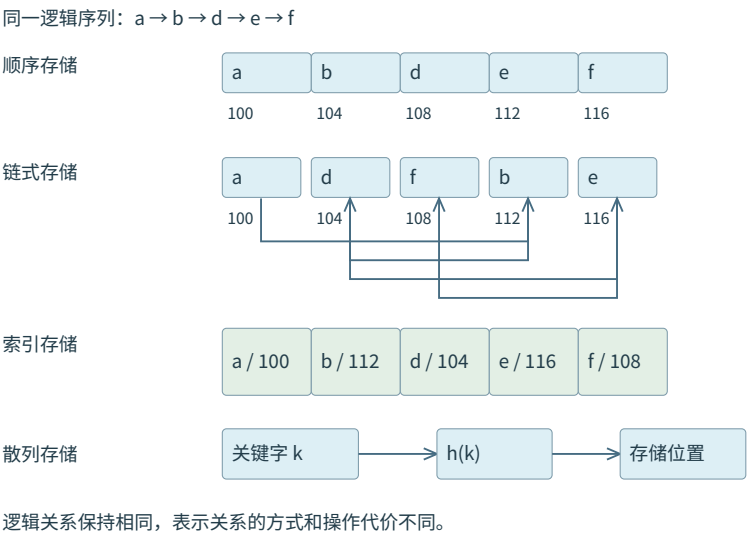


Figure 1.2: 同一逻辑序列的顺序、链式、索引和散列表示

1.1.1 例 1-1　值以外还要存什么

例 1.1.1 (配套例题)

〖1.1-005〗在存储数据时，通常不仅要存储各数据元素的值，还要存储（　）。

A. 数据的操作方法　B. 数据元素的类型　C. 数据元素之间的关系　D. 数据的存取方法

解析。若只保存若干值，就无法区分这些值原本属于一个序列、一棵树还是一张图。存储结构必须表达元素间的关系，答案为 **C**。关系未必另占一个专门字段：数组的相邻位置已隐含先后，链表则用指针显式记录后继。

运算的定义与实现也要分开。线性表的 “按位序取值” 是一种抽象运算；在数组上用下标访问、在链表上顺指针查找，则是两种实现。选

课程连接·逻辑与物理

同一逻辑关系可以用不同存储结构实现。顺序或链式描述物理安排，线性或非线性描述元素之间的关系。

择存储结构时，要同时考虑需要保存的关系和需要执行的运算，不能只看对象包含多少个元素。

抽象数据类型（ADT）把数据对象、关系和允许的操作合在一起描述，而不限定具体的机器实现。例如，栈规定 `push`、`pop` 及后进先出语义；顺序栈和链栈都能实现这一约定。

错项排除·有穷不等于很快

执行 2 的 n 次方步，只要对每个合法有限 n 都能结束，仍可满足有穷性。可行性也不是“考试规模下跑得快”；先判基本特性，再评价实际效率，不要混用两个层次。

1.2 算法怎样保证结束并得到正确结果

算法是求解一类问题的有限操作序列，通常要求有穷性、确定性、可行性、输入和输出：对合法输入，执行有限步后结束；每一步含义明确，并能在有限时间内完成；可以有零个或多个输入，至少有一个输出。输出不限于屏幕打印，也可以是返回值或修改后的数据状态。

算法的有穷性针对**每一次合法输入**。运行时间很长，仍可能满足有穷性；没有终止条件的服务程序可以是有用的程序，却不属于这里所说的有限求解算法。可行性要求操作能够执行，并不要求它在现实规模下足够快。正确、可读、健壮以及时空效率是进一步评价设计的维度；“必须具备的特性”与“希望达到的设计质量”属于不同层面的问题。

证明循环正确性，可以找一个每轮前后都保持的关系，称为**循环不变式**。例如，依次扫描数组求最大值：处理前 i 个元素后，`max` 等于这 i 个元素的最大值。证明分三步：初始化时说明该关系为何成立；每轮用下一个元素更新后说明它仍成立；终止时 $i = n$，由不变式可知结果覆盖了全体元素。此外还要说明循环变量不断接近边界，以排除关系成立却永不终止的程序。

设计和检验算法时，要考虑空表、单元素、重复元素和查找失败等情况。若把初始化写成 `max=A[0]`，输入规模就必须满足 $n \geq 1$；若允许空数组，应另行约定失败时的返回值，不能继续访问 `A[0]`。

1.2.1 一组记录如何经过有限步骤得到结果

以年龄序列 56、49、40、49、18 为例，每一轮扫描尚未确定的后缀，取出最小记录并插入当前前缀的末尾。出现同龄记录时，本例先取输入中靠前的一条。

状态	年龄序列	已确定的前缀长度
初始	56、49、40、49、18	0
第 1 轮	18、56、49、40、49	1
第 2 轮	18、40、56、49、49	2
第 3 轮	18、40、49、56、49	3
第 4 轮	18、40、49、49、56	4；末元素随之确定

比较的是记录中的年龄项，但搬动的是整条记录，姓名、财富等其他数据项仍须跟随对应记录。描述算法时应明确“取出再插入”还是“交换两个位置”；二者在元素相等时可能有不同的稳定性。这里只用这一过程说明输入、确定步骤与输出，第 8 章将进一步比较具体排序算法。

1.3 怎样由执行次数得到时间复杂度

实际运行时间受机器、编译器和输入分布影响。比较算法的增长规律时，先选择一种基本操作，统计它的执行次数 $T(n)$，再研究 n 增大时的数量级。本书采用单位成本模型：定长整数的比较、赋值及一次数组

访问都视为常数时间。若整数位数随输入增长，或者需要访问外存，这个假设就要重新检查。

$T(n) = O(f(n))$ 给出渐近上界；$T(n) = \Omega(f(n))$ 给出渐近下界；上下界同阶时记为 $\Theta(f(n))$。选择题通常用 O 表示所要求的紧数量级。$3n^2 + 7n + 9 = \Theta(n^2)$：当规模足够大时，低次项和常数因子都不会改变增长阶。把 $O(n)$ 写成 $O(2n)$ 并没有更精确地给出渐近阶。

不变量·累计值与轮数不同

第 k 轮结束时 sum=k(k+1)/2，循环次数是满足停止不等式的最小 k。sum 约等于 n 不代表循环 n 次；每轮增量逐步变大，正是平方根阶出现的原因。

1.3.1 先看循环变量如何变化

`i++` 经过 n 次增长到 n；`i*=2` 则约经 $\log_2 n$ 次到达循环边界。分析嵌套循环时，不能只数层数。若内层范围依赖外层变量，应先写出每轮次数再求和。例如：

```
for i = 1 to n:
    for j = 1 to i:
        执行一次基本操作
```

总次数为 $1+2+\cdots+n = n(n+1)/2$，所以是 $\Theta(n^2)$。外层和内层各有一个变量，并不意味着次数总是 $n \times n$；本例只是恰好达到平方阶。

1.3.2 例 1-2　循环由累计和决定何时停止

例 1.3.1 (配套例题)

【2017】下列函数的时间复杂度是（　）。

```
int func(int n) {
    int i=0, sum=0;
    while (sum<n) sum += ++i;
    return i;
}
```

A. $O(\log n)$　B. $O(n^{1/2})$　C. $O(n)$　D. $O(n \log n)$

解析。条件检查的是 `sum`，应先把它写成循环次数的函数。执行 k 轮后，`i=k`，$\text{sum} = 1 + 2 + \cdots + k = k(k+1)/2$。终止时，$k$ 是满足 $k(k+1)/2 \geq n$ 的最小非负整数，即

$$k = \left\lceil \frac{\sqrt{1+8n} - 1}{2} \right\rceil .$$

每轮只做常数次操作，故 $k = \Theta(\sqrt{n})$，答案为 **B**。`sum` 最终约为 n，不能据此推出执行了 n 轮——每轮的增量本身也在增长。分析按题目给定的整数模型进行；实际实现还应防止固定字长溢出。

1.3.3 例 1-3　两层循环为什么仍是线性时间

不变量·先列每轮工作量

外层 i 取 1、2、4 等值，内层工作量恰与 i 相同。用等比和可知所有小轮之和仍受末轮的常数倍控制；“两层循环” 没有提供平方阶的充分条件。

例 1.3.2 (配套例题)

【2022】下列程序段的时间复杂度是（　）。

```
int sum=0;
for (int i=1; i<n; i*=2)
    for (int j=0; j<i; j++)
        sum++;
```

A. $O(\log n)$　B. $O(n)$　C. $O(n\log n)$　D. $O(n^2)$

解析。外层的取值为 $1,2,4,\dots,2^k$，其中 $2^k < n \le 2^{k+1}$。各轮内层次数正好为这些值，总计

$$1+2+4+\cdots+2^k = 2^{k+1}-1.$$

该和小于 $2n$，而当 $n>1$ 时又不小于 n 的某个常数倍，故为 $\Theta(n)$，答案为 **B**。若把每一轮都估成最后一轮的 $O(n)$，再乘以外层轮数，得到的只是不紧的 $O(n\log n)$ 上界，忽略了前几轮内层规模很小这一事实。

1.3.4 最好、最坏与平均情况在比较什么

线性查找长度为 n 的表：目标在首位时比较 1 次；在末位或不存在时，通常比较 n 次。这是相同规模下不同输入造成的差异。**最坏时间**取规模为 n 的合法输入中的最大执行次数；**最好时间**取最小；**平均时间**还须说明输入的概率分布。不能把 “平均” 直接理解为最好与最坏的算术平均。

若 n 个元素被查找的概率相同，并且保证查找成功，则顺序查找的平均比较次数为 $(n+1)/2$。若还可能失败，就要把失败概率及其代价计入；第 7 章将按查找模型分别计算。

递归算法先写出递推关系。例如，若每次只递归求解规模为 $n-1$ 的一个子问题，并做常数工作，则 $T(n)=T(n-1)+\Theta(1)=\Theta(n)$。若产生两个规模为 $n/2$ 的子问题，并在本层做线性合并，则 $T(n)=2T(n/2)+\Theta(n)$：每层总工作为 $\Theta(n)$，约有 $\log_2 n$ 层，总时间为 $\Theta(n\log n)$。不能只看到规模减半就写成对数时间；每层有多少个子问题同样决定总量。

1.4 辅助空间中为何要计入递归栈

辅助空间复杂度衡量算法运行时所需的额外空间随输入规模增长的情况，一般不计输入本身占用的空间；题目对输出区是否计入另有约定时，按题意处理。本书比较排序时把归并辅助数组计入额外空间，不重复计算原数组。

几个循环变量只占 $\Theta(1)$ 空间，循环执行一万次也不需要同时保存一万个变量。相反，递归调用尚未返回时，调用记录必须同时保留参数、返回位置和局部状态。递归求和每层只用常数空间，但同一时刻最多存在 n 层调用，因此辅助空间为 $\Theta(n)$；时间与空间同阶只是这个例子的特点，并非所有递归都如此。

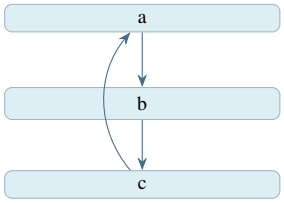
若一次调用先处理左半、返回后再处理右半，左右子树的递归调用就不会全程同时展开。二叉树遍历的调用总数为 $\Theta(n)$，最大栈深为树

高 h，故递归辅助空间是 $O(h)$。完全二叉树中 $h=\Theta(\log n)$；单支链中 $h=\Theta(n)$。这里不能用“访问了 n 个结点”代替“同时保留了多少层状态”。

原地算法通常允许常数个辅助单元。只数递归代码中有几个显式变量而忽略调用栈，就可能误判为原地算法。快速排序即使在原数组上划分，递归栈仍要计入：普通实现平均 $O(\log n)$，最坏 $O(n)$；第 8 章会解释两者分别对应怎样的划分。

最小反例·链式不等于树

三个结点 a→b→c→a 可以构成循环单链表的物理链接，链接图含环，不能据此认定为树。采用指针联系只是存储表示；逻辑对象究竟是线性表、树还是图，还需查看关系与操作约定。



循环链接含环，链式存储不能确定树关系

1.5 辨清对象、结构、类型与运算

定义 1.5.1 (数据结构的三要素)

数据结构的三要素是逻辑结构、存储结构和数据的运算。逻辑结构规定元素间的关系，存储结构在计算机中表示元素及关系，运算包括抽象功能的定义以及在具体表示上的实现。数据项是构成元素的不可分割单位；记录由多个数据项构成，却仍可作为一个元素整体处理。

数据类型由一个值的集合及定义在该集合上的操作组成。原子类型的值不再分解，结构类型的值可分解为成分；抽象数据类型进一步强调对使用者可见的对象、关系和操作语义，将实现细节藏在接口之后。逻辑结构与存储结构不是一一对应：同一逻辑结构可以有多种表示，同一种链式表示也能用于线性表、树、图。

判断维度	需要说明什么	例子	不能直接推出什么
逻辑结构	元素间关系	序列、层次、任意关联	是否使用连续地址
存储结构	地址与链接的表示	顺序、链式、索引、散列	ADT 允许的操作一定相同
运算定义	输入、输出与前后条件	取第 i 个元素、插入	具体执行多少次访问
运算实现	在已定表示上怎样执行	数组寻址、逐结点扫描	另一种表示下代价也相同

1.6 用一个完整算法连接特性、正确性与代价

先写接口契约，再写循环

以求数组最大值为例：合法输入为长度 $n\geq 1$ 的有效数组；返回值应等于数组中某个元素，且不小于任意元素。允许空输入时，必须把失败作为接口的一部分；不能凭空取一个数充当所有整数数组的最大值。

```
bool maximum(int a[], int n, int &answer) {
    if (n <= 0) return false;
    int best = a[0];
    for (int i = 1; i < n; ++i)
        if (a[i] > best) best = a[i];
    answer = best;
    return true;
}
```

接口采用教材式数组参数和引用输出参数，失败时不修改 answer。合法数组必须实际包含至少 n 个有效元素；调用者负责保证数组对象

和元素可访问。数据规模 n 为 0 时没有最大值，布尔返回值把这种情况与 “最大值恰好是 0” 区分。

扫描最大值的循环证明

进入第 i 轮前，不变量为 `best=max(a[0..i-1])`。初始化 $i=1$ 时，唯一已处理元素为 $a[0]$；比较 $a[i]$ 后取较大者，不变量扩展到前 $i+1$ 个元素。i 严格递增且上界为 n，故循环终止；终止时处理全部元素，返回正确最大值。非空输入恰有 $n-1$ 次元素大小比较，所以时间为 $\Theta(n)$，额外存储为 $\Theta(1)$。

□

最小反例·重复申请不等于同时占用

循环两次，每次申请一个单元并在下一轮前释放，峰值仍只占一个单元。空间分析要沿时间轴看尚未释放的状态；不能照搬 “轮数 × 每轮空间” 的时间计数方法。

1.7 渐近分析的运算规则及适用条件

定理 1.7.1 (加法取主阶，嵌套先计实际次数)

若两段代码依次执行，时间为 $T_1(n)+T_2(n)$，可按最大数量级归并；若一段工作被重复 $k(n)$ 次且每次代价有统一紧界，才可直接相乘。内层范围随外层变化时，先写求和。常数底的对数只相差常数因子。

常见增长次序（$c>1$ 为固定常数）为

$$1 \prec \log n \prec n \prec n\log n \prec n^2 \prec n^3 \prec c^n \prec n! \prec n^n.$$

O 上界并不唯一：线性算法当然也属于 $O(n^2)$，但题目通常要求最紧的常见阶。最好/最坏描述同规模输入的选择，$O/\Omega/\Theta$ 描述函数之间的界，不能将两组概念机械配对为 “O 只表示最坏”。

空间必须数**同一时刻仍存活的状态**。两段代码顺序使用并释放临时数组时，峰值可取各段最大值；两个数组同时保留时才相加。循环内每轮复用一个长度 n 的临时数组，即使循环 n 轮，峰值仍为 $\Theta(n)$ 而非 $\Theta(n^2)$。递归每层若分配不同规模的数组，必须沿一条同时活跃的调用链求和，而非只用层数乘最外层空间。

1.8 从计数到增长阶的完整过程

选定基本操作后，先保留一个能说明执行次数的式子，再化简数量级。循环的初始化、条件检查、循环体和更新并非同一执行次数：执行 n 轮的普通 for 循环，条件通常检查 n+1 次，循环体和更新各 n 次。例如按单位操作计数可能得到 $T(n)=3n+3$，最后才写成 $O(n)$。不同计数粒度会改变常数，但不改变这一增长阶。

循环或工作形式	先列出的次数关系	紧数量级
`i=1; i<=n; ++i`	n 次	$\Theta(n)$
`i=1; i<=n; i*=2`	$2^k \le n$	$\Theta(\log n)$
`i=n; i>0; i/=2`	整数反复折半直至 0	$\Theta(\log n)$
外层 i=1 至 n，内层 j=1 至 i	$\sum_{i=1}^{n} i$	$\Theta(n^2)$
外层 i=1 至 n，内层 j=1 至 n	$n \cdot n$	$\Theta(n^2)$
固定次数执行长度 n 的循环	cn，c 与 n 无关	$\Theta(n)$

乘法增长中，底数是大于 1 的固定常数，换底只改常数因子；若底数随规模变化就需重新分析。分支取决于输入数据时，最坏情况选择代

价最大的合法分支；平均情况按输入概率加权，而不是随意把每条分支平均。

错项排除·效率不是唯一质量

一个输出错误的程序即使时间和空间都很少，也没有解决目标问题。遇到用效率 “全面衡量优劣” 的说法，应先检查正确性等前提；正确性、可读性、健壮性不能被遗漏。

1.8.1 同时活跃的空间沿调用链相加

课件用局部数组展示了 “递归深度” 之外的另一项因素。若每层只有常数个局部变量，递归从 n 依次到 1，峰值为 $\Theta(n)$。若第 k 层还保留长度为 k 的局部数组，递归进入下一层时旧数组仍未释放，峰值为

$$n + (n-1) + \cdots + 1 = \frac{n(n+1)}{2} = \Theta(n^2).$$

若每次把规模减半且保留对应规模的局部数组，同时活跃的数组空间为 $n + n/2 + n/4 + \cdots < 2n$，故是 $\Theta(n)$，而不是简单将 “最外层 n” 乘 “深度 log n”。深度乘每层空间只适用于每层空间有同一紧界的情况。

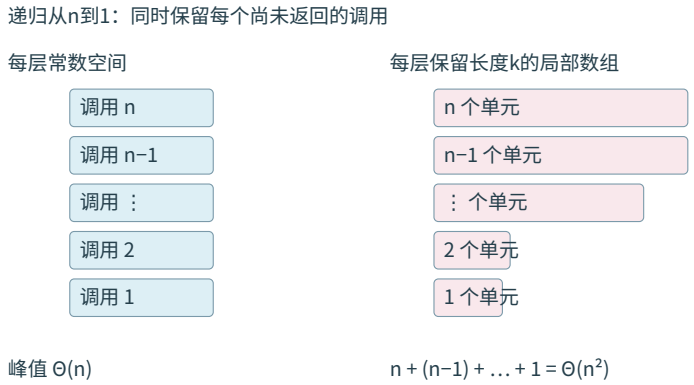


Figure 1.3: 相同递归深度下，每层保留空间不同导致峰值不同

1.9 历史错题指向的辨析

算法必须具备五项基本特性；正确性、可读性、健壮性以及时空效率则是进一步评价设计的维度。CPU 实际耗时受机器与实现影响，并非只由抽象算法决定；也允许以更多空间换取更少时间。配套单选对 “衡量优劣” 的概括存在措辞歧义，复习时应掌握这几个层次的区别，不把时空效率当作全部质量条件。做复杂度题则先找真正的停止量，列累计和或等比和，避免将变量最终值直接当作循环次数。

1.10 主动回忆自测

第 1 题为配套题，其余为自拟补充。先遮住答案，说明停止条件、存活空间与表示关系。

合上正文，沿导图复述

1. 【2019】设 n 是描述问题规模的非负整数，程序段 `x=0; while( n>=(x+1)*(x+1)) x=x+1;` 的时间复杂度是（　）。A. $O(\log n)$　B. $O(n^{1/2})$　C. $O(n)$　D. $O(n^2)$
2. 一段算法循环调用一个无递归函数 n 次，每次函数使用 10 个

整数局部变量。仅考虑这些局部变量，辅助空间是 $O(n)$ 还是 $O(1)$?
3. 一个对象采用链式存储，能否据此断定它的逻辑结构是一棵树?
4. 数据项、数据元素、数据对象之间是什么关系？ADT 是否规定必须用数组实现?
5. 算法的五项特性与正确性、健壮性有什么区别？“输入” 必须从键盘读入吗?
6. 单个循环变量连续翻倍，和外层翻倍但内层执行 i 次，两者为什么不是同一个阶?
7. 顺序执行两段各需 n 个临时单元的代码，峰值空间一定是 2n 吗?
8. 递归每次将规模减半、每层只用常数空间，时间是否一定是对数阶？还缺什么信息?

自查提示·推导与结果一起核对

核对复杂度答案时，同时复查每轮计数、递归终止条件及同时活跃的调用层数。

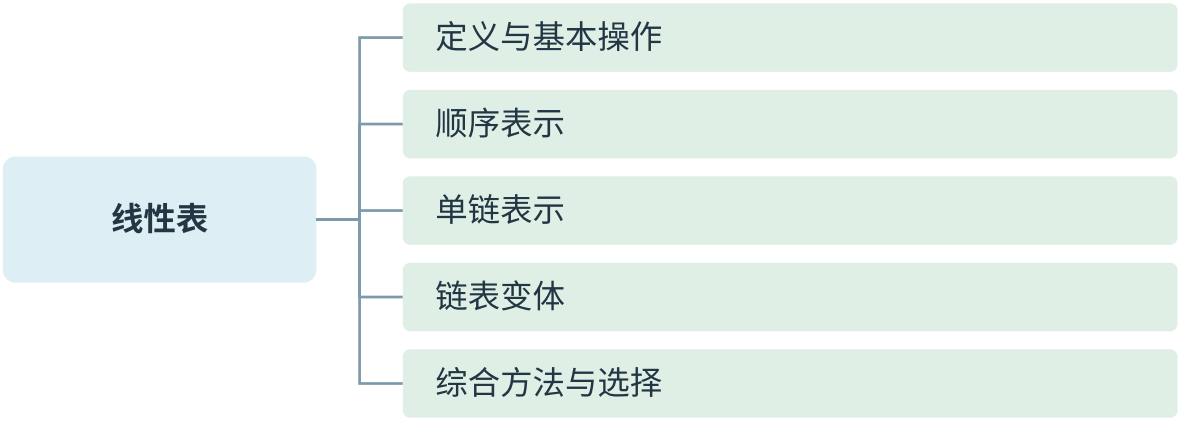
1.10.1 核对答案

先核条件，再核结论

1. **B**。次数恰为 $\lfloor\sqrt{n}\rfloor$：终止时 $x^2 \le n < (x+1)^2$，仍为平方根数量级，但停止方程与例 1-2 的累计和不同。2. $O(1)$：前一次调用返回后，局部空间可以复用。3. 不能。链式存储只是一种表示方法，线性表、树和图都可以采用。
2. 元素是处理的基本单位，由数据项组成；对象是同性质元素的集合。ADT 规定抽象语义而不限定表示。
3. 五项特性描述作为有限求解算法的要求；正确性等描述设计质量。输入可由参数、已存数据等提供，并不限定交互方式。
4. 前者轮数为对数；后者基本操作数为 1+2+4+$\cdots$，是线性量级。
5. 不一定。若前段的临时空间结束后可复用，峰值只需 n 个量级；若两段结果必须同时保留才相加。两者渐近阶均为 Θ(n)，但实际峰值不同。
6. 还要知道每层产生几个子问题、每个调用做多少非递归工作。两个半规模递归和一个半规模递归具有不同调用总量。

线性表 2

章节脉络



同一个线性表可以放在连续数组中，也可以由分散的结点连接起来。前者依靠数组下标定位，插入和删除常要搬移元素；后者把先后关系放在链接中，定位常要逐结点进行。分析操作复杂度时，必须把“找到位置”和“修改结构”分别计算。

最小反例·容量不等于表长

容量为 100 的数组可以表示空表，此时 length=0；也可只保存一个元素。仅给数组上界不能推断有效元素数。插入判断必须同时检查位序是否合法与是否尚有空位。

2.1 位序、下标和基本操作

线性表是具有相同数据类型的 n 个元素的有限序列。$n=0$ 时为空表。非空表的第一个元素没有直接前驱，最后一个没有直接后继，其余元素各有一个直接前驱和一个直接后继。这里的“相同类型”不要求元素的值互不相同。

长度为 n 的表，已有元素的位序是 $1\sim n$；在第 i 个位置插入新元素，合法范围是 $1\le i\le n+1$。最后一个位置表示追加。若用 `A[0..n-1]` 保存，位序 i 对应下标 $i-1$。查找失败的信号不能与下标 0 混淆，例如按位序返回 0 表示失败，或按下标返回 -1 表示失败。

本章约定单链表有头结点 `L`，首元结点为 `L->next`。头结点用于统一第一个位置的插入与删除，不保存有效表元素；**头指针**是指向它的指针变量，二者不是同一对象。无头结点的版本也可以实现相同运算，但修改第一个结点时必须能够修改头指针。

2.2 顺序表的寻址、插入和删除

若每个元素占 w 个地址单位，首元素地址为 b，则第 i 个元素地址为 $b+(i-1)w$。计算地址只需常数次算术运算，故可随机存取。“随机”指任意位置可以直接定位，与随机数无关。按值查找仍要比较元素，未排序时通常需要 $O(n)$ 时间。

在下标 k 处插入，要把 `A[k..n-1]` 各后移一位；必须**从后向前**移动，否则后一个元素可能已被前一个覆盖。删除下标 k 的元素，则把 `A[k+1..n-1]` 各前移一位，按从前向后的顺序执行。

```
insert(A, n, capacity, k, x):
    if k<0 or k>n or n==capacity: return false
    for j=n down to k+1: A[j]=A[j-1]
    A[k]=x
    n=n+1
    return true
```

不变量·搬移保护未读区

插入向后搬移时，从后往前处理能确保源元素尚未被覆盖；删除向前搬移则从前往后读。记住需要保护的未读区，就能由目标下标推回正确循环方向。

插入移动 $n-k$ 个元素，追加时为 0；删除移动 $n-k-1$ 个元素，删除末元素时为 0。等概率在 $n+1$ 个插入位置选择时，平均移动 $n/2$ 个；等概率删除 n 个已有元素时，平均移动 $(n-1)/2$ 个。平均数的分母不同，来自合法位置数不同。

2.2.1 例 2-1　没有给存储方式时能否确定移动数

例 2.2.1 (配套例题)

〖2.3-012〗在线性表 $a_0, a_1, \ldots, a_{100}$ 中，删除元素 a_{50} 需要移动（　）个元素。

A. 0　B. 50　C. 51　D. 0 或 50

解析。题目没有限定存储方式。顺序表中，后面的 $a_{51} \sim a_{100}$ 共 50 个元素前移；链表中改变链接即可，无需搬移其他结点的数据，故答案为 **D**。删除前若还要从头寻找该结点，链表访问时间仍可能是 $O(n)$；“不移动数据” 不等于 “全过程常数时间”。

静态数组的容量固定，不意味着表长固定。动态数组可以在空间不足时申请更大的连续区并搬移旧元素，但单次扩容可能需要 $O(n)$ 时间；分析某一次插入时不能忽略这次复制。

2.3 顺序表的完整状态与核心实现

顺序表保存数据元素以及当前表长；动态分配版本还需记录容量。静态容量与表长是不同概念：容量固定时，表长仍可在 0 至容量之间变化。代码沿用教材的类型名称，`ElemType` 表示元素类型，本章示例取整型。

```
#define MaxSize 100
typedef int ElemType;
typedef struct {
    ElemType data[MaxSize];
    int length;
} SqList;
```

静态类型使用宏 `MaxSize` 作数组容量，后文动态类型以成员 `MaxSize` 记容量。静态与动态两组类型示例独立使用；编译验证时分别组织为独立单元。

2.3.1 初始化、查找与随机访问

静态顺序表初始化令 `length=0`。`InitList(SqList &L)`中的引用参数使函数能修改调用者的表长。动态顺序表还需申请连续空间，申请成功后设定表长和容量；销毁时释放该空间。

```cpp
void InitList(SqList &L) { L.length = 0; }
ElemType GetElem(SqList L, int i) {
    return L.data[i - 1];  // 前提: 1 <= i <= L.length
}
int LocateElem(SqList L, ElemType e) {
    for (int i = 0; i < L.length; ++i)
        if (L.data[i] == e) return i + 1;
    return 0;
}
```

辨析·失败不能改状态

先检查位序和容量，再搬移、赋值和改表长。删除结果 e 只在成功时有新值。

按位查找先把位序换成数组下标，时间 $O(1)$；调用前须保证位序合法。按值查找从前向后扫描，成功返回首次相等元素的位序，失败返回 0，最坏 $O(n)$。本章 ElemType 取整型，可直接用 `==`；若元素为结构类型，实际代码应定义逐字段或按关键字的判等函数。若成功位置等概率，平均比较 $(n+1)/2$ 次；有序顺序表可采用折半查找，具体算法在查找章展开。

2.3.2 插入与删除的循环方向

插入合法范围是 $1 \le i \le n+1$，还要保证有剩余容量。将从位序 i 开始的后缀从后向前搬移；删除合法范围是 $1 \le i \le n$，先保存被删元素，再将后缀从前向后搬移。

```cpp
bool ListInsert(SqList &L, int i, ElemType e) {
    if (i < 1 || i > L.length + 1) return false;
    if (L.length >= MaxSize) return false;
    for (int j = L.length; j >= i; --j)
        L.data[j] = L.data[j - 1];
    L.data[i - 1] = e;
    ++L.length;
    return true;
}
bool ListDelete(SqList &L, int i, ElemType &e) {
    if (i < 1 || i > L.length) return false;
    e = L.data[i - 1];
    for (int j = i; j < L.length; ++j)
        L.data[j - 1] = L.data[j];
    --L.length;
    return true;
}
```

插入时先搬后面的元素，才能保存尚未读取的旧后缀；若从前向后执行 `data[j+1]=data[j]`，会反复复制同一个被覆盖的值。删除时源位置比目标位置大，从前向后搬移恰好保护未读元素。失败操作不修改表长，删除输出参数只在成功时有效。

2.3.3 动态分配与扩容

动态顺序表仍要求整个数组连续，只是容量可以重建。扩容不是在原地址后方一定能再接一段；需要取得新连续区，再搬移已有元素。下面保留教材的 `SeqList`、`data`、`MaxSize`、`length` 命名。`InitSize=100` 是王道 2027 教材中的示例初始容量；课件演示取 10，两者采用相同的存储结构和接口，实际值可以按需求设定。

```c
#define InitSize 100
typedef struct {
    ElemType *data;
    int MaxSize;
    int length;
} SeqList;
void InitList(SeqList &L) {
    L.data = (ElemType *)malloc(InitSize * sizeof(ElemType));
    L.length = 0;
    L.MaxSize = InitSize;
}
void IncreaseSize(SeqList &L, int len) {
    ElemType *p = L.data;
    L.data = (ElemType *)malloc(
        (L.MaxSize + len) * sizeof(ElemType));
    for (int i = 0; i < L.length; ++i)
        L.data[i] = p[i];
    L.MaxSize += len;
    free(p);
}
```

辨析·动态仍然连续

SeqList 动态区仍是一个连续数组。扩容先保住新旧地址，复制成功后再释放旧区，旧元素地址会变化。

这两段保留教材的 void 接口，以申请成功为前提；扩容还要求 len 为正，申请长度能由机器整数和地址空间表示。实际工程应先把 malloc 结果存入临时指针 q，检查非空后再替换 data；否则直接覆盖 data 可能在申请失败时丢失原地址。每次扩容搬移 n 个元素，时间 $O(n)$，新旧区同时存在时峰值临时空间也为 $O(n)$。`malloc/free` 成对使用；清空令长度为 0，销毁则释放数组并撤销有效指针、长度和容量。动态数组支持随机存取，动态分配本身不等于链式存储。

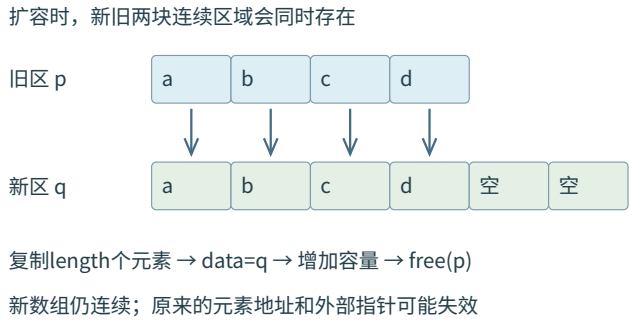


Figure 2.1: 动态顺序表扩容时复制有效元素并释放旧区

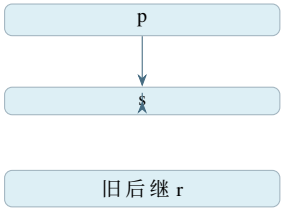
2.4 单链表：先保留哪条链接

本章采用教材的结点与头指针类型，申请与释放使用 `malloc/free`；对应代码包含 `<stdlib.h>`。

```c
typedef struct LNode {
    ElemType data;
    struct LNode *next;
} LNode, *LinkList;
bool InitList(LinkList &L) {
    L = (LNode *)malloc(sizeof(LNode));
    if (L == NULL) return false;
    L->next = NULL;
    return true;
}
```

最小反例·两句交换会丢链

p 原后继为 r。若先写 p->next=s，再写 s->next=p->next，第二句读到的是 s，形成自环，r 不再可从 p 到达。先暂存旧链接，是所有原地指针改写的基本纪律。



错误顺序使 s 自环，旧后继 r 不可达

结点由数据域与后继指针域构成。不同结点不要求物理相邻；同一个结构体结点所含的各域作为一个整体分配，因此不能把“结点之间地址分散”误说成“一个结点内部各域随意分散”。为了读取第 i 个元素，须从链头沿 `next` 走 $i-1$ 步，所以链表不支持按位序的常数时间随机访问。

已知有效结点 `p`，在其后插入新结点 `s`：先令 `s->next=p->next` 保存旧后继，再令 `p->next=s`。若先把 `p->next` 改成 `s`，随后从 `p->next` 取得的已不是旧后继，可能产生自环并丢失后半链。

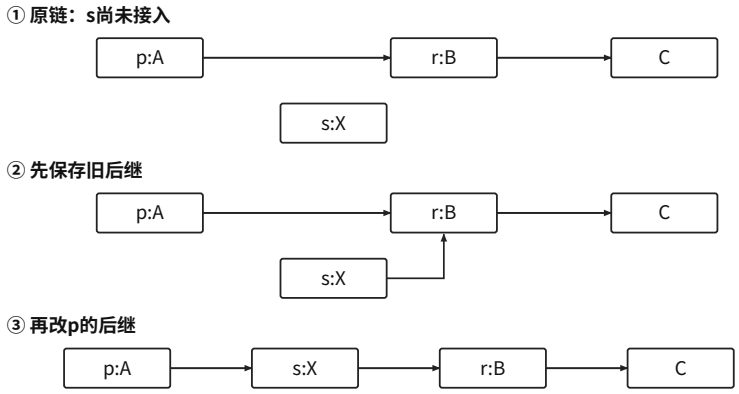


Figure 2.2: 插入前、保存旧后继、接入新结点；虚线表示待改链接

删除 `p` 的后继时，先检查 `p->next!=NULL`，再用 `q=p->next` 暂存待删结点，执行 `p->next=q->next`，最后释放 `q`。释放后不能继续读取 `q->next`。若只给待删结点的指针而不给其前驱，通常还要从头寻找前驱；用“复制后继数据再删后继”的办法不适用于尾结点，还可能改变题目要求保留的结点身份。

头插法不断把新结点接到头结点之后，输入次序会逆转；尾插法保存尾指针，每次接到尾部，能保留输入次序。两者若都维护所需指针，建立 n 个结点的链表均为 $O(n)$；若每插入一个元素都重新从头找尾部，才会累积到 $O(n^2)$。

2.4.1 原地逆置：未处理部分不能丢

维护 prev 为已逆置部分的新首结点，cur 为未处理部分的首结点。每次先暂存 nxt=cur->next，再把 cur->next 指向 prev，随后推进两个工作指针。

不变量·固定间隔保持到尾

fast 先走 k 步后，两指针始终相隔 k 条后继步。当 fast 到空位置时，slow 距空位置也恰为 k 步，故指向倒数第 k 个结点。先走阶段遇空可直接识别 k 超过表长。

```
void reverseList(LNode *L) {       // 带头结点
    LNode *prev=NULL, *cur=L->next;
    while (cur!=NULL) {
        LNode *nxt=cur->next;
        cur->next=prev;
        prev=cur;
        cur=nxt;
    }
    L->next=prev;
}
```

每轮只移动一个结点到逆置区。空表时循环不执行，头结点仍指向空；单结点时其 next 仍为空。时间 $\Theta(n)$，辅助空间 $\Theta(1)$。递归逆置虽然也能线性运行，却要保留递归栈，不能与迭代实现的空间混算。

2.4.2 例 2-2　如何一次扫描找到倒数第 k 个结点

例 2.4.1 (配套例题)

配套例题：已知一个带有表头结点的单链表，结点结构为 data | next，只给出了头指针 list。在不改变链表的前提下，设计尽可能高效的算法，查找倒数第 k 个位置上的结点（k 为正整数）。若成功，输出该结点的 data 值并返回 1；否则只返回 0。要求说明设计思想、实现步骤，并用 C、C++ 或 Java 描述算法，在关键处加注释。

解析。倒数位置由尾部决定，但链表不能直接向前访问。让两个指针保持固定间隔：fast 先从首元结点前进 k 步，再让 fast 与 slow 同步前进。若表长为 n，fast 到达空位置时共走了 n 步，slow 只走了 $n-k$ 步，正好指向第 $n-k+1$ 个结点。

```
int searchK(LNode *list, int k) {
    if (k<=0) return 0;
    LNode *fast=list->next, *slow=list->next;
    for (int j=0; j<k; ++j) {
        if (fast==NULL) return 0;   // 表长不足 k
        fast=fast->next;
    }
    while (fast!=NULL) {
        fast=fast->next;
        slow=slow->next;
    }
    printf("%d", slow->data);
    return 1;
}
```

以长度 5、$k=2$ 为例，先让 fast 指向第 3 个结点，slow 留在第 1 个；同步经过 “4 与 2—5 与 3—空与 4”，第 4 个即倒数第 2 个。$k=n$ 时第一阶段恰好走到空位置，第二阶段不执行，返回首元结点；$k>n$

时在第一阶段报失败。两个指针合计走 $O(n)$ 步，辅助空间 $O(1)$，没有修改任何链接。

最小反例·旧前驱必须还能取得

只有 r、p 两个相邻结点时，在 p 前插 q 的最终关系是 r↔q↔p。若第一句覆盖 p->prior，之后再从它读取就失去 r；四条目标链接全部成立才算插入完成。

2.5 双链表、循环链表与静态链表

双链表为每个结点增加 `prior`，以便从当前结点访问直接前驱。已知 `p` 后，删除 `p` 可在 $O(1)$ 时间内同时接好两侧；但这不消除按位序寻找 `p` 的成本。

2.5.1 例 2-3　把结点插到双链表结点之前

例 2.5.1 (配套例题)

〖2.3-015〗在双链表中向 `p` 所指的结点之前插入结点 `q`，正确操作是（　）。

```
A. p->prior=q; q->next=p;
   p->prior->next=q; q->prior=p->prior;
B. q->prior=p->prior; p->prior->next=q;
   q->next=p; p->prior=q->next;
C. q->next=p; p->next=q;
   q->prior->next=q; q->next=p;
D. p->prior->next=q; q->next=p;
   q->prior=p->prior; p->prior=q;
```

解析。假设 `p` 有有效前驱（可以是头结点）。记旧前驱为 `r`，结果必须同时满足 `r->next=q`、`q->prior=r`、`q->next=p`、`p->prior=q`。D 在最后才修改 `p->prior`，因此前三句还可以取得旧前驱，答案为 **D**。A 的第一句过早覆盖旧前驱；B 最后把 `p->prior` 设成了 `p` 自身；C 既改错方向，又使用尚未设置的 `q->prior`。

写程序时也可先 `q->prior=p->prior; q->next=p;`，再改两侧结点的链接。若 `p` 是不带头结点链表的首元结点，前驱为空，须单独更新头指针，不能直接解引用这个空前驱。

循环单链表把尾结点的 `next` 指回链头。带头结点时，空表满足 `L->next==L`；`L==NULL` 只说明没有有效头结点，并非这个模型的空表条件。遍历应在重新遇到头结点时结束，而不是等待 `NULL`。

只给尾指针 `rear` 的无头循环单链表，可以用 `rear->next` 取得首元结点；两个非空循环单链表头尾相接时，交换各尾部的后继链接即可完成 $O(1)$ 拼接。若只给头指针且没有尾链接，要先找到尾部，通常需要线性时间。若存在头结点，拼接时须处理多余头结点，不能把它当作数据留在两表之间。

循环双链表的头结点同时连接首尾，空表时 `next` 与 `prior` 都指回头结点。**静态链表**用数组保存结点，用整数下标代替机器指针；逻辑后继由游标决定，不要求结点下标相邻。还须维护空闲结点链或其他管理空闲位置的结构，数组开得大不等于每个位置都是有效数据。

2.6 双指针、归并与原地处理

错项排除·字符相同不等于共享

两个结点都存字符 i，也可能位于不同地址。公共后缀要求从某处起共享同一串结点，应比较指针而不是 data；共享链不能再按两条独占链分别释放。

这些方法把题目的约束变成指针之间的不变关系。使用前先确认能否修改原数据、是否有序、是否共享结点，以及是否需要保留重复元素。

2.6.1 例 2-4　数组循环左移怎样避免反复搬移

例 2.6.1 (配套例题)

配套例题：将 $n\ (n>1)$ 个整数存放在一维数组 R 中。设计一个时间、空间都尽可能高效的算法，将序列循环左移 $p\ (0<p<n)$ 个位置，即把 $(X_0,\dots,X_{n-1})$ 变为 $(X_p,\dots,X_{n-1},X_0,\dots,X_{p-1})$。给出思想、C/C++ 或 Java 实现，并说明时间与空间复杂度。

解析。题目要求两个连续区段交换位置，同时保持各段内部次序。设前 p 个元素为 A，其余为 B。分别逆置得 A^RB^R，再整体逆置，即得 BA。两个区段各被逆置两次，内部方向恢复，而相对位置交换。

```
void reverse(int R[], int l, int r) {
    while (l<r) {
        int t=R[l]; R[l]=R[r]; R[r]=t;
        ++l; --r;
    }
}
void rotateLeft(int R[], int n, int p) {
    reverse(R,0,p-1);
    reverse(R,p,n-1);
    reverse(R,0,n-1);
}
```

例如 `abcdefgh` 左移 3 位，依次得到 `cbadefgh`、`cbahgfed`、`defghabc`。总交换次数不超过 n，时间 $O(n)$；只用常数临时变量，空间 $O(1)$。若将首元素移到末尾重复 p 次，时间为 $O(np)$，当 p 与 n 同阶时会退化。三次逆置可处理 $p=1$ 和 $p=n-1$，无需另写搬移循环。

2.6.2 有序归并为什么可以一次扫描

两个递增表的当前首元素中，较小者一定是剩余全部元素中最小的。输出它并推进对应指针，重复直到某表为空，再接上另一个表的剩余部分。合并长度为 m,n 的两个表需要 $O(m+n)$ 时间；数组版本通常另开输出区，链表版本在允许破坏原表时可直接重接结点。

比较相等元素时，若希望保持原相对次序，先取左表元素。若题目要求集合并集而不是多重集合合并，则相等时只输出一次并推进两个指针；这项条件会改变算法，不能只改题目的答案。

2.6.3 例 2-5　相同后缀是相同字符还是相同结点

例 2.6.2 (配套例题)

配套例题：采用带头结点的单链表保存单词，两个单词可共享相同后缀的存储空间。例如 `loading` 与 `being` 共享从字符 `i` 开始的结点。`str1`、`str2` 指向两个头结点，结点为 `data | next`。设计时间上尽可能高效的算法，找到共同后缀的起始结点，给出思想、

程序及时间复杂度。

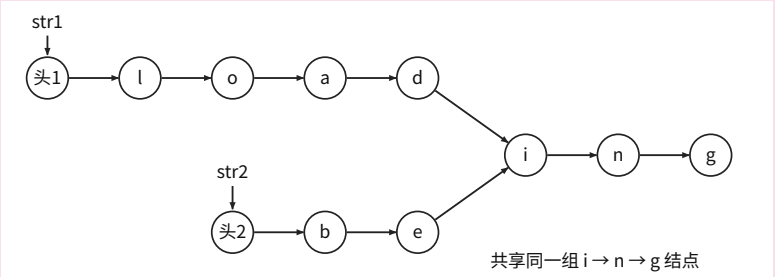


Figure 2.3: 两个前缀分别连接到同一个 i 结点，随后共用 n、g 结点

最小反例·剩下候选仍需复核

序列 1、2、3 经抵消可留下 3，但 3 只出现一次，未超过一半。抵消给出的是存在多数时的必要候选，并不证明多数存在；第二次计数补上充分性检验。

解析。共享指的是地址相同，不能以 `p->data==q->data` 判断。单链表一旦在某结点汇合，其后继也相同，所以两表从公共起点到末尾的长度相等。先各扫描一次求长 m, n，让较长表先走 $|m-n|$ 步，两指针便处于距表尾相同的位置；此后同步前进，第一次地址相等的位置就是答案。

```
LNode *commonSuffix(LNode *str1, LNode *str2) {
    int m=0,n=0;
    for (LNode *p=str1->next; p; p=p->next) ++m;
    for (LNode *q=str2->next; q; q=q->next) ++n;
    LNode *p=str1->next, *q=str2->next;
    while (m>n) { p=p->next; --m; }
    while (n>m) { q=q->next; --n; }
    while (p!=q) { p=p->next; q=q->next; }
    return p;                    // 无公共结点时二者同时为 NULL
}
```

图中两表长度分别为 7 和 5，先让长表跨过 `l,o`，再比较 `a` 与 `b`、`d` 与 `e`，最后到达同一个 `i`。若两表内容相同却各自分配结点，则直到空位置才相等，返回空。总时间 $O(m+n)$，辅助空间 $O(1)$，适用于有限、无环单链表。

2.6.4 例 2-6　主元素为什么还要再计数

例 2.6.3 (配套例题)

配套例题：整数序列 $A=(a_0,\dots,a_{n-1})$ 满足 $0\le a_i<n$。某值出现次数超过 $n/2$ 时，称它为主元素。例如 $(0,5,5,3,5,7,5,5)$ 的主元素为 5，而 $(0,5,5,3,5,1,5,7)$ 没有主元素。设计尽可能高效的算法，存在主元素时输出该值，否则输出 -1；给出思想、程序及时间、空间复杂度。

解析。每次删除两个值不同的元素，不会消灭一个原本占严格多数的值：若删去一个多数元素和一个其他元素，多数仍超过剩余的一半；若删去两个其他元素，多数优势更大。用一个候选值和计数模拟这种抵消。计数为 0 时选新候选；相同则加 1，不同则减 1。

```cpp
int majority(int A[], int n) {
    int candidate=0, count=0;
    for (int i=0; i<n; ++i) {
        if (count==0) { candidate=A[i]; count=1; }
        else if (A[i]==candidate) ++count;
        else --count;
    }
    int actual=0;
    for (int i=0; i<n; ++i)
        if (A[i]==candidate) ++actual;
    return actual>n/2 ? candidate : -1;
}
```

不变量·头结点不随表空而消失

有效头结点始终存在，数据结点从 L->next 开始。空表并不意味着 L 为空；统一前驱定位时 pos=0 对应头结点，才能让第一个位置的插入与其他位置共用同样两步链接。

抵消只保证“若有主元素，最后候选一定是它”，没有保证主元素存在。例如 `1,2,3` 最后仍可能留下候选 3，它只出现一次，所以必须第二次扫描核实。两个线性扫描合计 $O(n)$ 时间，额外空间 $O(1)$。题目给出的值域也允许用计数数组，但需 $O(n)$ 辅助空间；此处没有必要为省下一次扫描而付出这部分空间。

2.7 单链表的类型、定位与修改

教材用 `LNode` 表示结点类型，用 `LinkList` 表示指向结点的指针类型。`LNode *`与 `LinkList` 在这里是同一种指针类型，名称强调的语义不同：前者常强调某个结点，后者常强调一张表的头指针。

```cpp
LNode *GetElem(LinkList L, int i) {
    if (i < 0) return NULL;
    int j = 0;
    LNode *p = L;
    while (p != NULL && j < i) {
        p = p->next;
        ++j;
    }
    return p;
}
LNode *LocateElem(LinkList L, ElemType e) {
    LNode *p = L->next;
    while (p != NULL && p->data != e) p = p->next;
    return p;
}
```

以上采用有效头结点，`GetElem(L,0)`返回头结点，`GetElem(L,i)`在 i 超过表长时返回空。按值查找从首元结点开始。求表长可令计数初值 0，沿 `L->next` 逐结点递增，不能把头结点算作有效元素。

2.7.1 后插、按位插入与删除后继

后插的前提是 p 指向有效结点。申请成功后，先保存原后继再接入；两次指针改写的次序决定原链是否保留。

```cpp
bool InsertNextNode(LNode *p, ElemType e) {
    if (p == NULL) return false;
    LNode *s = (LNode *)malloc(sizeof(LNode));
    if (s == NULL) return false;
    s->data = e;
    s->next = p->next;
    p->next = s;
    return true;
}
bool ListInsert(LinkList &L, int i, ElemType e) {
    if (i < 1) return false;
    LNode *p = GetElem(L, i - 1);
    return InsertNextNode(p, e);
}
bool ListDelete(LinkList &L, int i, ElemType &e) {
    if (i < 1) return false;
    LNode *p = GetElem(L, i - 1);
    if (p == NULL || p->next == NULL) return false;
    LNode *q = p->next;
    e = q->data;
    p->next = q->next;
    free(q);
    return true;
}
```

辨析·第一位改谁

不带头结点时，第 1 位插入与删除要修改 L 本身；引用参数把新首元地址写回调用者。

`ListInsert` 还要定位第 i−1 个结点，因此按位序操作最坏 $O(n)$；已知前驱后的局部操作才是 $O(1)$。删除 q 前保存其后继并接回链，释放后不再读取 q。独占链销毁时依次保存后继、释放当前结点，最后释放头结点；两个共享后缀的表不能分别完整释放同一批共享结点。

只给结点 p、希望在 p 前插入新值时，普通方法先找前驱。若题目允许改变 p 的数据，可将新结点插在 p 后并复制旧数据，再把新值写入 p，常数时间得到相同的值序列；这种方法不保留“哪个值属于哪个结点”的身份。只给 p 删除自身时也可复制后继数据再删后继，但 p 为尾结点时失效。

2.7.2 不带头结点时，第一位需要单独处理

不带头结点的空表满足 `L==NULL`。在位置 1 插入要改头指针，因此接口用引用参数；其余位置先找前驱。

```cpp
bool ListInsert_NoHead(LinkList &L, int i, ElemType e) {
    if (i < 1) return false;
    if (i == 1) {
        LNode *s = (LNode *)malloc(sizeof(LNode));
        if (s == NULL) return false;
        s->data = e;
        s->next = L;
        L = s;
        return true;
    }
    LNode *p = L;
    int j = 1;
    while (p != NULL && j < i - 1) {
        p = p->next; ++j;
    }
    return InsertNextNode(p, e);
}
```

不带头结点删除首元时同样要令 `L=L->next` 并释放旧首元；带头结点则统一通过头结点的后继处理。头指针是一个变量，头结点是一个哨兵对象，首元结点是第一个实际数据对象，三者不能互换。

辨析·尾插不能反复找尾

r 始终指最后一个结点，每轮直接接入，总时间线性；每轮从头找尾才会累计到平方阶。

2.8 建表：头插逆序，尾插保持输入次序

头插法每次把新元素接到头结点之后，因此输入 a、b、c 后，数据次序为 c、b、a。尾插法维护尾指针 r，始终指向当前最后一个结点，输入同样数据后次序为 a、b、c。

```
LinkList List_HeadInsert(LinkList &L) {
    LNode *s;
    int x;
    L = (LinkList)malloc(sizeof(LNode));
    L->next = NULL;
    scanf("%d", &x);
    while (x != 9999) {
        s = (LNode *)malloc(sizeof(LNode));
        s->data = x;
        s->next = L->next;
        L->next = s;
        scanf("%d", &x);
    }
    return L;
}
LinkList List_TailInsert(LinkList &L) {
    int x;
    L = (LinkList)malloc(sizeof(LNode));
    LNode *s, *r = L;
    scanf("%d", &x);
    while (x != 9999) {
        s = (LNode *)malloc(sizeof(LNode));
        s->data = x;
        r->next = s;
        r = s;
        scanf("%d", &x);
    }
    r->next = NULL;
    return L;
}
```

两段采用教材的返回头指针接口，以输入读取和 malloc 均成功为前提，输入 9999 结束，因此 9999 不能同时作为这一输入协议中的普通数据。若首次输入就是 9999，头插法通过 L->next=NULL 得到空表，尾插法通过循环后的 r->next=NULL 得到空表。实际程序还应检查 scanf 返回值和申请结果；失败时须保存并释放已建前缀。两种建表均为 $O(n)$；尾插每次若重新从头找尾部，才会变成 $O(n^2)$。

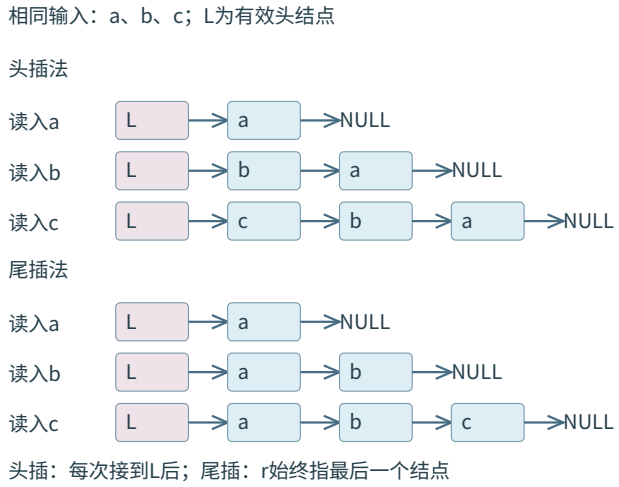


Figure 2.4: 同样输入 a、b、c，头插与尾插的中间状态和最终次序

辨析·循环终止条件

普通单链表等待 NULL，循环表等待回到头结点。带头循环表空时头结点的 next 指向自身。

2.9 双链表的局部操作与循环边界

双链结点同时保存前驱和后继。以下 p 有有效后继时采用统一接入；非循环链尾的后继为空，需对后继的 prior 赋值另作检查。

```
typedef struct DNode {
    ElemType data;
    struct DNode *prior, *next;
} DNode, *DLinklist;
bool InsertNextDNode(DNode *p, DNode *s) {
    if (p == NULL || s == NULL) return false;
    s->next = p->next;
    if (p->next != NULL) p->next->prior = s;
    s->prior = p;
    p->next = s;
    return true;
}
bool DeleteNextDNode(DNode *p) {
    if (p == NULL || p->next == NULL) return false;
    DNode *q = p->next;
    p->next = q->next;
    if (q->next != NULL) q->next->prior = p;
    free(q);
    return true;
}
```

带头循环双链表空时 `L->next==L` 且 `L->prior==L`；数据结点的前后方向均可遍历，但遇头结点时要停止，不能把头结点当作普通数据删除。循环单链表查尾条件是 `p->next==L`，与普通单链表的 `p->next==NULL` 不同。只用头指针时找尾仍需遍历；保存尾指针后可通过 `rear->next` 找到头部。

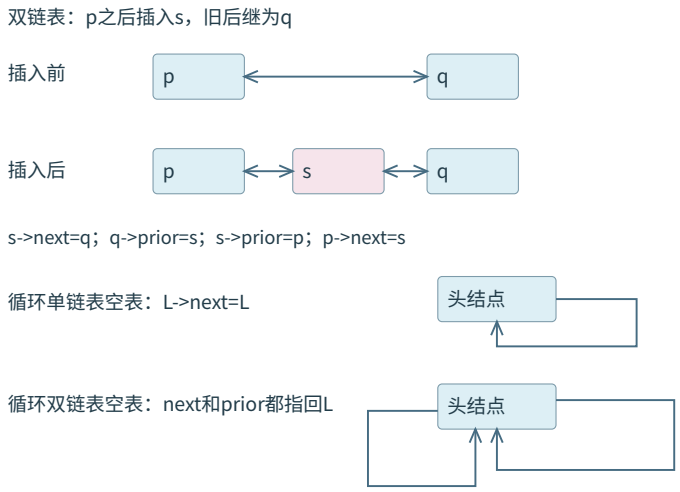


Figure 2.5: 双链表插入前后及两种循环链表的空状态

课程连接·表示影响访问局部性

连续数组通常能利用缓存中的相邻数据，分散结点可能增加缓存未命中。渐近复杂度相同也不保证实际用时相同；比较表示时同时检查数据规模、访问方式和存储开销。

静态链表以数组下标充当游标。例如 `struct { ElemType data; int next; } SLinkList[MaxSize];`中，next 记录的是下一个结点占用的数组位置，并非机器地址。还需要管理空闲位置；课件的一种具体表示令 0 号结点为头结点，next=-1 表示链尾，next=-2 表示该数组位置空闲：初始化令 a[0].next=-1，其余位置 next=-2；插入先找空位再接游标，删除先接回前驱游标，再将被删位置标为空闲。结束和空闲是两个不同状态。静态链表仍按链接顺序定位，不因结点集中在数组就获得按逻辑位序随机访问。

课件的静态链表把同一个逻辑序列 e1、e2、e3、e4 分散在下列数组位置：

数组下标	data	next
0	头结点	2
1	e2	6
2	e1	1
3	e4	-1
4、5	空闲	-2
6	e3	3
7、8、9	空闲	-2

遍历游标顺序为 $0\to2\to1\to6\to3\to-1$。若每结点数据域 4B、游标 4B，数组首地址为 addr，则 e1 位于 addr+2×8；它是逻辑第 1 个元素，物理上却占数组第 2 号结点。

2.10 结构选择与操作代价对照

操作条件	顺序表	单链表（有效头结点）	选择时还需检查
按位序取值	$O(1)$	$O(n)$	是否要求随机访问
无序按值查找	$O(n)$	$O(n)$	值是否有序、是否另有索引
按位序插入/删除	最坏 $O(n)$	定位最坏 $O(n)$	是否已给有效前驱
已知前驱后的局部修改	仍可能搬移后缀	$O(1)$	申请/释放与结点身份
存储开销	连续容量可能有空闲	每结点含链接域	局部性、碎片和最大规模

顺序表逻辑相邻通常也物理相邻，有利于连续访问；链表用额外链接换取接入灵活性，不能由此得出“链表一定更省空间”。静态链表虽然使用数组分配结点，逻辑后继仍由游标指定，所以不具备按逻辑位序直接寻址的能力。

不变量·删后检查新后继

若当前前驱后连续两个结点都等于 x，删掉第一个后马上前进，就可能跳过第二个。前驱只在保留结点时前进，才能让每个新后继都得到检查。

2.11 连续删除与有序去重的线性写法

> **删除后留在前驱，保留后才前进**
>
> 删除单链表所有值为 x 的结点时，用 p 指向已处理部分的最后一个保留结点。若 p 后继应删除，就摘除后继续检查 p 的新后继；否则 p 向前走。相邻多个 x 正是不能在每次删除后都移动 p 的原因。

```
void eraseAll(LNode* L, int x) {
    LNode* p = L;
    while (p->next != NULL) {
        if (p->next->data == x) {
            LNode* q = p->next;
            p->next = q->next;
            free(q);
        } else {
            p = p->next;
        }
    }
}
```

顺序表有序去重可让读指针从左到右扫描，写指针只扩展“已保留的互异前缀”。若当前值不同于最后一个保留值才写入；每个元素只处理常数次，总时间 $O(n)$，额外空间 $O(1)$。未排序数组中，相邻不重复不能保证全局不重复，不能悄悄去掉有序条件。

2.12 主动回忆自测

以下为原创补充，检查边界和不变式。

1. 允许修改单链表，要求删除所有值为 x 的结点。删除一个结点后，指向前驱的指针是否应立即后移?
2. 两个无环单链表长度相同，尾结点地址相同，能否断定它们一定有公共后缀?
3. 顺序表去除相邻重复元素时，为什么可用“已保留长度”作为写指针，而不必每删一个元素都搬移后半表?

答案。1. 不应立即后移；新的后继还可能等于 x，应继续检查当前前驱的后继。只有保留一个结点时前驱才推进。2. 能；至少尾结点本身就是公共后缀，公共起点还可能更早。3. 扫描指针只读未处理区，写指针只向前形成已保留区；每个元素最多被检查和写入常数次，总时间 $O(n)$。若原表无序，这只能消除相邻重复，不能保证全表唯一。

2.12.1 补充自拟回忆题

从边界恢复算法

4. 长度 n 的顺序表允许在哪些位序插入，在哪些位序删除？各自等概率模型的分母是什么？
5. 有效头结点、头指针、首元结点如何区分？带头结点空表中哪一个为空？
6. 只给单链表尾结点指针，能否总在 O(1) 时间内删除该结点并保留其他结点身份？
7. 主元素候选的计数归零意味着此前完全没有主元素吗？为什么最后还要核计数？
8. 两个内容相同却分别分配的单链表，公共后缀算法返回什么？适用前提是什么？

补充题核对答案

4. 插入位序 1 到 n+1，删除 1 到 n；平均插入移动数除以 n+1，平均删除移动数除以 n。
5. 头指针是地址变量，头结点是哨兵对象，首元结点是第一个有效元素。有效头结点存在时，空表是 L->next 为空。
6. 不能。单链表尾结点无后继可复制，且没有直接前驱链接，通常要从头找前驱。
7. 不意味着不存在严格多数；抵消保证 “若存在，最后候选是它”，不能反推候选一定占多数，必须再次扫描。
8. 返回 NULL。地址相等才表示共享，前提是有限无环单链表且头结点和链接有效。

自查提示·区分定位与修改

已知结点时的局部操作与从头查找位置的总代价应分别计算。

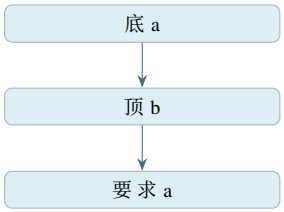
栈、队列和数组 3

章节脉络

栈与队列都保存线性序列，差别在于允许从哪一端加入和取出。判断一个操作序列是否合法，必须保留操作过程中的状态；只检查输入、输出是否含有相同元素，不能排除违反栈顶或队头约束的情况。

最小反例·数量相同仍不合法

顺序输入 a、b、c，要先输出 c 就必须把三者全部入栈；弹出 c 后栈顶为 b，接着要求 a 便失败。输出 c、a、b 与输入含相同元素，却违反栈顶约束。



c 已弹出，b 挡在 a 上方

3.1 栈的后进先出与合法序列

栈只允许在同一端插入和删除，这一端称为栈顶，另一端称为栈底。入栈称 `push`，出栈称 `pop`。栈为空时不能出栈；固定容量栈满时不能继续入栈。若中途不出栈，一批元素的出栈次序是入栈次序的逆序；若允许交替入栈和出栈，输出就可能有多种。

检验给定输出的方法是按目标逐个模拟：若当前栈顶不是所需元素，继续把尚未入栈的元素依次压入；一旦所需元素到顶，就弹出。若输入已用尽而栈顶仍不符，输出不合法。这个过程也给出实现该输出所需的最小容量，即模拟中出现的最大栈深。

3.1.1 例 3-1　队列不会改变栈的输出次序

例 3.1.1 (配套例题)

配套例题：栈 S 和队列 Q 初始均为空，元素 `a,b,c,d,e,f,g` 依次入栈 S。每个元素出栈后立即进入队列 Q，7 个元素出队顺序为 `b,d,c,f,e,a,g`，则栈容量至少是（　）。

A. 1　　B. 2　　C. 3　　D. 4

解析。队列保持进入它的先后次序，因此它的输出就是栈的输出。按目标模拟，栈内状态按栈底到栈顶写：

需要输出	必须先执行的入栈	弹出后栈内剩余
b	a、b	a
d	c、d	a、c
c	无	a
f	e、f	a、e
e	无	a
a	无	空
g	g	空

错项排除·共享栈只在相邻时满

两栈从数组两端向中间增长，未用单元恰位于两栈顶之间。两栈顶相邻已经没有空位；等到相等才判满，就已让两个栈占用同一个位置。

压入 d 或 f 后，栈深达到 3；表中过程也说明 3 个单元足够，答案为 **C**。不能把队列当作另一处可以随意调整次序的缓冲区。

若只用入栈 `I`、出栈 `O` 表示操作，一段从空栈出发并回到空栈的合法序列须满足：两类操作的总数相等，且每个前缀中 `O` 的个数不超过 `I`。对 n 个不同元素，合法出栈序列数为卡特兰数 $C_n = \frac{1}{n+1}\binom{2n}{n}$；这一公式用于计数，不代替具体容量与给定序列的模拟。

输入为 `a,b,c,d` 时，若输出以 `c,d` 开头，弹出 c 后栈中为 a、b，随后必须压入并弹出 d，剩余只能 b、a，因此仅有 `c,d,b,a` 一种。若题目指定的是栈的输出而让输入未知，也要反推中间栈顶。例如第 3 个输入是 1，输出从 1 递增，1 弹出时第 2 个输入压在第 1 个输入上方；若第 1 个是 2，就要先弹出上面的另一个值才能到 2，与“接下来输出 2”矛盾，所以第 1 个不可能是 2。

3.2 队列与双端队列

队列只在队尾加入、在队头删除，遵循先进先出。它适合保存等待依次处理的对象，如按发现顺序等待扩展的图顶点。队头与队尾是逻辑角色，不能凭数组下标的大小判断先后：循环队列绕回后，队尾下标可以小于队头下标。

双端队列允许在两端插入和删除。输入受限双端队列只允许一端输入，两端均可输出；输出受限双端队列允许两端输入，只允许一端输出。判断输出时应明确属于哪一种，不能直接套普通队列或普通栈的限制。双端队列能够模拟栈与队列，但这不表示它能按任意次序输出一组顺序输入的元素。

以 1、2、3、4 依次输入为例，普通栈有 14 种合法输出。输入受限双端队列有 22 种，不能输出 `4,2,1,3` 和 `4,2,3,1`：先输出 4 意味着 1、2、3 已按次序留在队列中，2 在中间，下一次无法取出。输出受限双端队列也有 22 种，不能输出 `4,1,3,2` 和 `4,2,3,1`：4 第一次输出前，前三个元素只能通过两端依次接入，其剩余次序只能为 123、213、312 或 321，不能为 132 或 231。两种受限队列的非法序列集合并不相同。

3.3 顺序栈、共享栈与循环队列

顺序栈存于 `A[0..M-1]`。本章令 `top` 指向当前栈顶，初值 -1：空条件 `top==-1`，满条件 `top==M-1`；入栈为 `A[++top]=x`，出栈为 `x=A[top--]`。若题目改用“`top` 指向下一个空位”，空条件就为 0，入栈与出栈的增减位置也随之改变。先画一个元素的状态，再写公式，通常比背四个表达式可靠。

两个栈共享一个数组时，可让一端从左向右长、另一端从右向左长。初值 `top1=-1, top2=M`，栈满当且仅当 `top2-top1==1`。未用空间是两栈顶之间的单元，某个栈已用多少并不单独决定全局是否栈满。

入栈前必须先检查两栈顶是否相邻，不能等两个栈顶已经重合才报告满。

辨析·循环队列先定约定

牺牲一个单元时，空为 front=rear，满为 (rear+1)%MaxSize=front；其他设计的条件需另算。

3.3.1 循环队列先立约定再计算

采用长度为 M 的数组，令 `front` 指向队头元素，`rear` 指向下一次入队位置。出队后 `front=(front+1)%M`，入队后 `rear=(rear+1)%M`。若只有这两个下标，空与完全占满都会出现 `front==rear`，无法区分。

牺牲一个单元时，约定空条件为 `front==rear`，满条件为 `(rear+1)%M==front`，可保存的最大元素数是 $M-1$。当前长度为

$$\text{len} = (rear - front + M) \bmod M.$$

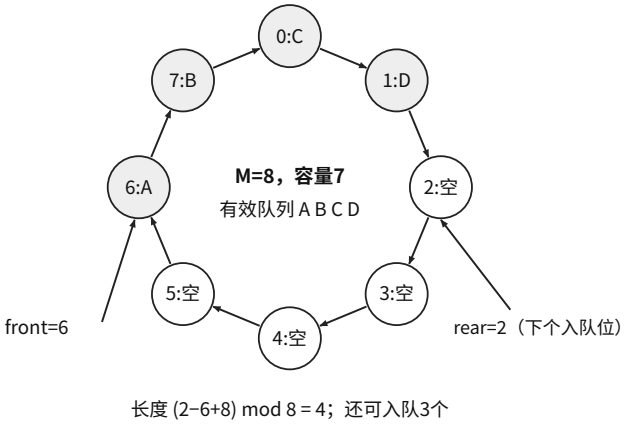


Figure 3.1: 数组环绕时，下标大小不代表队列先后；本图采用牺牲一个单元

若要求 M 个单元全部可用，可以增加 `size` 计数，或增加标志，区分最近一次使二者相等的是入队还是出队。此时 “空” 和 “满” 的判断依赖新增成员，不能仍照搬牺牲单元的满条件。

3.3.2 例 3-2　换了 front 含义，长度公式是否改变

例 3.3.1 (配套例题)

〖3.2-006〗循环队列存于数组 `A[21]`，`front` 指向队首元素的前一个位置，`rear` 指向队尾元素。当前 `front=8, rear=3`，队列长度为（　）。

A. 5　B. 6　C. 16　D. 17

解析。有效元素从下标 9 开始，经过 9 至 20，再经过 0 至 3，共 $12+4=16$ 个，答案为 **C**。公式仍可写成 $(3-8+21) \bmod 21 = 16$，是因为与前一约定相比，两个指针的含义都错开了一个位置。若只改变一个指针的含义，公式就未必不变。数组是 `A[0..n]` 时实际有 $n+1$ 个单元，取模也必须使用 $n+1$。

另一种常见表示令 front 仍指队头，而 rear 指当前队尾。可令初始 front=0、rear=M−1；入队先推进 rear 再写入，出队先读取 front 再推进。牺牲一个单元时，空条件是 `(rear+1)%M==front`，满条件是 `(rear+2)%M==front`，长度为 `(rear-front+1+M)%M`。例如 M=5、front=0、rear=3 时有 4 个元素，已经满；front=0、rear=4 则为空。它与本章正式

SqQueue 接口是不同表示，应连同初始化、动作次序和判空判满一起使用。

不变量·尾指针不能悬空

带头结点链队列始终让 rear 指向最后一个数据结点，空时指向头结点。删去唯一数据结点后，rear 必须回到 front；否则下次入队会经已释放地址写内存。

3.4 链栈与链队列

链栈通常把链头作为栈顶，头插与头删都是 $O(1)$。动态分配可随数据量申请结点，因而通常不受预先给定的小容量数组限制；但入栈仍可能因可用内存不足而失败，出栈时也仍可能遇到空栈。

单链队列把链头作为队头，链尾作为队尾，并保存两个指针。若反过来从单链表尾部删除，必须找到尾结点的前驱，出队会变为 $O(n)$。带头结点的链队列空时 `front==rear`，二者都指向头结点。入队时接到 `rear->next` 并更新 `rear`；出队时删除 `front->next`。

```
dequeue(Q):
    if Q.front==Q.rear: return 失败
    p=Q.front->next
    x=p->data
    Q.front->next=p->next
    if Q.rear==p: Q.rear=Q.front
    释放 p
    return x
```

最后一个数据结点出队时，必须让尾指针退回头结点；否则它指向已释放的空间。非空且不止一个元素时，无需改变尾指针，所以对“删除是否修改头尾指针”这类问题，答案取决于被删除的是否为最后一个元素。

只带队首指针的非循环双链表，要在另一端入队仍需找尾部；循环双链表则可通过 `front->prior` 直接取得尾部。比较结构时，应检查有没有可到达另一端的短链接，而不是看到“双链”就认定所有操作都为常数时间。

3.5 多维数组的地址怎样计算

数组同一维度的下标取值范围固定，每个元素大小相同。设二维数组的行下标为 $l_1 \sim u_1$，列下标为 $l_2 \sim u_2$，列数 $c = u_2 - l_2 + 1$，行数 $r = u_1 - l_1 + 1$，每元素占 w 字节，首元素地址为 b。

按行优先时，在 (i, j) 前有 $i - l_1$ 个完整行及本行的 $j - l_2$ 个元素，故

$$\mathrm{Addr}(A[i,j]) = b + [(i - l_1)c + (j - l_2)]w.$$

按列优先时，先数完整列：

$$\mathrm{Addr}(A[i,j]) = b + [(j - l_2)r + (i - l_1)]w.$$

例如 `A[2..4][-1..2]` 按行优先存放，每元素 4 字节，`A[2][-1]` 的地址为 1000。`A[4][1]` 前有 $2 \times 4 + 2 = 10$ 个元素，字节地址为 1040。下标差先得到“元素数”，最后乘元素宽度才得到字节偏移；按字编址的题则要统一成字地址再计算。

三维数组按行优先时，最右下标变化最快：`A[i][j][k]` 的相对元素位置为 $((i-l_1)n_2+(j-l_2))n_3+(k-l_3)$。可以逐维理解，不必另背一个没有下标原点的公式。

错项排除·三对角首尾行少一个

n 至少为 2 时，首行和末行各只保存 2 个位置，中间行才各 3 个。若把第 i 行之前全部按 3 个计，会从第一行就多算；先列 2+3(i−2) 再加本行偏移。

3.6 对称、三角、三对角与稀疏矩阵

压缩存储利用元素间的确定关系，避免重复保存。**对称矩阵**满足 $a_{ij}=a_{ji}$，保存一侧三角即可；访问另一侧时交换 i,j。**三角矩阵**的三角区之外，元素全部相同，通常还要额外保存这个公共常量，它未必为 0。**三对角矩阵**只在主对角线及相邻两条对角线上可能非零，共有 $3n-2$ 个需要保存的位置。

下三角按行保存、原矩阵与数组下标均从 0 开始时，位置 (i,j)（$i\ge j$）之前有 $1+2+\cdots+i=i(i+1)/2$ 个元素，故数组下标为 $i(i+1)/2+j$。若原矩阵从 1 开始而数组从 0 开始，公式相应变成 $i(i-1)/2+j-1$。推导的顺序始终是“前面完整行或列数＋本行或列内偏移”。

3.6.1 例 3-3　三对角矩阵的起止行不能一律算 3 个

例 3.6.1 (配套例题)

配套例题：100 阶三对角矩阵 M 的元素 $m_{i,j}$ $(1\le i,j\le 100)$ 按行优先依次压缩存入下标从 0 开始的一维数组 N。$m_{30,30}$ 在 N 中的下标为（　）。

A. 86　B. 87　C. 88　D. 89

解析。第一行有 2 个元素，第 2 至 29 行各有 3 个，共 $2+28\times3=86$ 个在第 30 行之前。第 30 行依次保存 $m_{30,29},m_{30,30},m_{30,31}$，目标是本行第 2 个，因此下标为 $86+1=87$，答案为 **B**。这里 87 是从 0 开始的下标，目标是第 88 个被保存的元素。

3.6.2 例 3-4　先用对称性换到实际保存的半边

例 3.6.2 (配套例题)

配套例题：10×10 对称矩阵 M 的上三角元素 $m_{i,j}$ $(1\le i\le j\le 10)$ 按列优先存入 C 语言一维数组 N。$m_{7,2}$ 在 N 中的下标为（　）。

A. 15　B. 16　C. 22　D. 23

解析。$m_{7,2}=m_{2,7}$，实际寻找第 7 列第 2 行。前 6 列各存 $1,2,\ldots,6$ 个，共 21 个；本列目标前还有第 1 行 1 个元素，所以从 0 开始的下标为 $21+1=22$，答案为 **C**。不能先把下三角坐标直接代入上三角公式，再试图以加减 1 修正。

上三角若改为按行优先，各行长度为 $n,n-1,\ldots,1$；第 i 行之前有 $(i-1)(2n-i+2)/2$ 个保存位置。下三角若按列优先，各列长度同样递减，但列内偏移为 $i-j$。公式外形相近，决定它的是保存方向与下标原点。

稀疏矩阵的非零位置没有固定三角规律，需要连同位置一起保存。三元组顺序表记录 `(行,列,值)`，还须保存矩阵行列数与非零元数；十字链表为同一个非零元同时设置行链接和列链接。压缩后不能再按普通二维数组地址公式随机访问任意元素，查找与修改成本随表示方法

而变。

最小反例·括号计数看不到嵌套

([)] 中每一类左右括号数都相等，但第三个字符要求圆括号配对，栈顶却是方括号。配对必须遵守最近未完成对象先结束，不能在扫描后任意重排。

3.7 表达式、递归与层次处理

3.7.1 括号与表达式中的“最近一个尚未完成对象”

左括号入栈；遇右括号时，栈顶必须是对应种类的左括号，匹配后弹出；扫描完后栈必须为空。`([)]` 中括号数量相等，但遇到第一个右圆括号时栈顶为左方括号，已经失败。这个过程正好需要最后遇到且尚未配对的对象，所以选择栈。

后缀表达式中，操作数先入栈，遇二元运算符时先弹出的数是**右操作数**，后弹出的是左操作数，计算后将结果入栈。减法、除法不能颠倒。合法的二元后缀表达式在遇到每个运算符时，栈中至少有两个可用值，最终恰好留下一个结果。

3.7.2 例 3-5　弹出次序怎样影响减法

例 3.7.1 (配套例题)

配套例题：栈 S_1 保存整数，S_2 保存运算符。函数 F() 依次：（1）从 S_1 弹出操作数 a、b；（2）从 S_2 弹出运算符 op；（3）执行 b op a；（4）把结果压入 S_1。初始 S_1 自底向顶为 5、8、3、2，S_2 自底向顶为 *、−、+。调用 3 次 F() 后，S_1 栈顶值是（　）。

A. -15　B. 15　C. -20　D. 20

解析。第一次弹出 2、3，用加法得到 $3+2=5$，整数栈变为 5、8、5。第二次弹出 5、8，用减法得到 $8-5=3$，栈变为 5、3。第三次得到 $5\times 3=15$，答案为 **B**。若第二次按弹出先后计算 $5-8$，就会得到错误的负数。

中缀转后缀可用运算符栈：操作数直接输出；左括号入栈；右括号触发输出直到匹配左括号；遇运算符时，先输出栈顶中优先级更高的运算符。对通常左结合的 +、−、*、/，同级也先弹出；若加入右结合运算符，同优先级的处理规则需要调整。

例如 $x+y(z-u)/v$（其中相邻表示乘法）先把括号内写成 `zu-`，乘除按左结合形成 `yzu-*v/`，最后与 x 相加，得 `xyzu-*v/+`。这里的输出顺序表达运算依赖，而不是把原式的符号按种类重新排列。

3.7.3 递归返回前保存了什么

递归调用栈保存尚未完成的调用。调用树的一个结点表示一次调用，不能把函数返回值当作调用次数。对于

```
int F(int n) {
    if (n<=3) return 1;
    return F(n-2)+F(n-4)+1;
}
```

令 $C(n)$ 表示含当前调用在内的调用次数，则 $n\le 3$ 时 $C(n)=1$，否则 $C(n)=1+C(n-2)+C(n-4)$。因此 $C(4)=C(2)+C(0)+1=3$，$C(6)=1+3+1=5$，$C(8)=1+5+3=9$。式中返回值的“+ 1”不

是另一次函数调用；次数递推中的 “+ 1” 是当前调用本身。此过程对应〖3.3-006〗的递归调用模型。

C/C++ 对加号两侧函数调用的求值先后并不一概保证从左到右。若题目问 “第几个被调用”，应检查答案是否依赖这个先后，不能把一种习惯顺序当作语言保证。调用总数与最终结果在本例不受这个先后影响。另如〖3.3-007〗把递归式改为 `func(x-2)+func(x-4)`、基例返回 2，并计算 `func(func(5))`：必须先完成内层 func(5)，它和 func(3)、func(1) 共 3 次调用，结果为 4，所以第 4 次是外层 func(4)；内层两个基例谁先执行不影响这个结论。

层次处理需要先进先出的队列：先发现的一层结点必须先扩展，后发现的下一层排在后面。第 5 章层序遍历与第 6 章 BFS 都用这条约束，但图还需 `visited` 防止同一顶点反复入队。

辨析·调用次数与同时深度

Fibonacci 调用树的结点总数反映总调用次数；栈空间取同时尚未返回的调用链长度。

3.7.4 阶乘、Fibonacci 与重复子问题

下面两段保留课件的递归定义。输入 n 为非负整数，计算结果须在 int 可表示范围内。

```
int factorial(int n) {
    if (n == 0 || n == 1) return 1;
    else return n * factorial(n - 1);
}
int Fib(int n) {
    if (n == 0) return 0;
    else if (n == 1) return 1;
    else return Fib(n - 1) + Fib(n - 2);
}
```

factorial(10) 沿 10、9、…、1 依次深入，最多同时保留 10 层调用，随后按相反次序返回并完成乘法。Fib(4) 的调用树有 9 个结点，返回值为 3，最长未返回调用链却只有 4 层；Fib(2) 重复出现两次。朴素 Fib 递归的时间满足 $T(n) = T(n-1) + T(n-2) + O(1)$，呈指数增长，递归工作栈为 $O(n)$。每层传递两个递归分支，不意味着空间也按整棵调用树相加；一个分支返回后，其空间可供另一个使用。保存已算子问题或改用迭代可减少重复计算，改变算法后须重新分析代价。

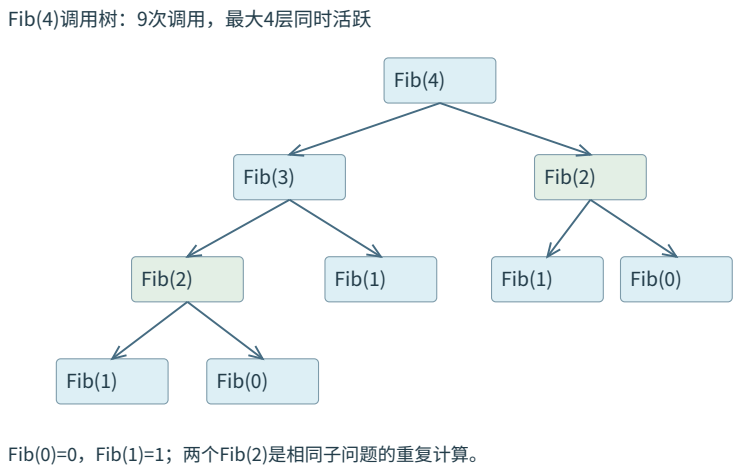


Figure 3.2: Fib(4) 的完整调用树：重复子问题与最大同时活跃调用链

3.8 栈与队列的教材接口

定义 3.8.1 (先约定指针指向什么，再写边界)

顺序栈中的 top 可以指栈顶元素，也可以指下一空位；循环队列中的 front/rear 也有不同语义。表示约定决定空满条件、增减时机和长度公式。同一个变量名并不保证同一种表示，改题时先画空状态和单元素状态。

不变量·top 含义决定增减位置

top 指栈顶时，空状态为 -1，入栈先加再写；top 指下一空位时，空状态为 0，入栈先写再加。用空栈和单元素两幅状态检查，比只背 push 语句更能应对改约定题。

下面采用教材的 `SqStack`，`top` 指当前栈顶；下标从 0 开始，空栈 `top=-1`。`ElemType` 按具体应用定义为元素类型，下面可取整型。

```
#define MaxSize 100
typedef int ElemType;
typedef struct {
    ElemType data[MaxSize];
    int top;
} SqStack;
void InitStack(SqStack &S) { S.top = -1; }
bool StackEmpty(SqStack S) { return S.top == -1; }
bool Push(SqStack &S, ElemType x) {
    if (S.top == MaxSize - 1) return false;
    S.data[++S.top] = x;
    return true;
}
bool Pop(SqStack &S, ElemType &x) {
    if (S.top == -1) return false;
    x = S.data[S.top--];
    return true;
}
bool GetTop(SqStack S, ElemType &x) {
    if (S.top == -1) return false;
    x = S.data[S.top];
    return true;
}
```

读栈顶不改变 top，出栈才将 top 减 1。若栈顶指向下一空位，压入顺序应为 `data[top++]=x`，弹出则为 `x=data[--top]`；这是一套独立表示，不能只改初始化而保留其余操作。

循环队列使用教材的 `SqQueue`，front 指队头元素、rear 指下一入队位置，牺牲一个单元。

```c
typedef struct {
    ElemType data[MaxSize];
    int front, rear;
} SqQueue;
void InitQueue(SqQueue &Q) { Q.front = Q.rear = 0; }
bool QueueEmpty(SqQueue Q) { return Q.front == Q.rear; }
bool EnQueue(SqQueue &Q, ElemType x) {
    if ((Q.rear + 1) % MaxSize == Q.front) return false;
    Q.data[Q.rear] = x;
    Q.rear = (Q.rear + 1) % MaxSize;
    return true;
}
bool DeQueue(SqQueue &Q, ElemType &x) {
    if (Q.front == Q.rear) return false;
    x = Q.data[Q.front];
    Q.front = (Q.front + 1) % MaxSize;
    return true;
}
bool GetHead(SqQueue Q, ElemType &x) {
    if (Q.front == Q.rear) return false;
    x = Q.data[Q.front];
    return true;
}
```

不变量·先换到已保存半边

对称矩阵访问未存的半边时，先交换 i、j，再按保存方向计数。坐标转换解决的是表示对应；数组从 0 还是 1 起始解决的是编号，两步不能互相替代。

入队先写 rear 位置再推进 rear，出队先取 front 位置再推进 front。空满检查均在修改之前进行，失败操作保持原状态，输出参数也不赋新值。读队头只读取，不出队。

环形长度公式来自有向距离

从 front 沿数组下标递增方向走到 rear，步数就是有效元素数。未绕回时为 rear−front，绕回时为 M−front+rear，合并为 $(rear - front + M) \bmod M$。牺牲一个单元将步数限制在 0 至 M−1，从而只让 0 表示空；若允许 M 个元素，模运算会把长度 M 也变成 0，必须增加 size 或标志保存这一个区别。

□

环形队列策略	空条件	满条件	可用容量
牺牲一个单元	front=rear	(rear+1) mod M=front	M−1
增加 size 计数	size=0	size=M	M
增加最近操作 tag	front=rear 且 tag 为出队	front=rear 且 tag 为入队	M

tag 方案初始设为 “出队” 态，初始化即表示空；成功入队/出队后更新 tag。计数方案每次成功修改长度，失败操作不能改 size。链队列没有预设数组容量，但申请新结点仍可能失败，不能将 “不判数组满” 理解为内存无限。

3.9 矩阵压缩：由坐标到下标，再从下标恢复坐标

完整行数加行内偏移

每次先写清：矩阵坐标从 0 还是 1 开始、压缩数组从 0 还是 1 开始、上/下三角、行/列优先。若是对称矩阵，先把目标换到实际保存的半边。最后数前面完整行或列，再加本行/列内偏移；不要用最后加减 1 修补错误的保存方向。

对于坐标从 0 开始、按行保存下三角的矩阵，令 $T_i = i(i+1)/2$，则 $k = T_i + j$。反向给定有效数组下标 k 时，先找到唯一满足 $T_i \le k < T_{i+1}$ 的 i，再令 $j = k - T_i$。数学上

$$i = \left\lfloor \frac{\sqrt{8k+1}-1}{2} \right\rfloor, \qquad j = k - \frac{i(i+1)}{2}.$$

例如 k=8，因 $T_3 = 6 \le 8 < T_4 = 10$，得 $i = 3, j = 2$。实现时若用浮点平方根，应以整数三角数边界复核，防止舍入把边界分到相邻行。

坐标从 1 开始、压缩数组从 0 开始时，上三角行优先下标为

$$k = \frac{(i-1)(2n-i+2)}{2} + (j-i), \quad 1 \le i \le j \le n.$$

下三角列优先则为

$$k = \frac{(j-1)(2n-j+2)}{2} + (i-j), \quad 1 \le j \le i \le n.$$

三角矩阵的公共常量区可另存一个值，所需空间为 $n(n+1)/2+1$；对称矩阵只保存一侧即可，不需要这个公共常量单元。三对角矩阵 $n \ge 2$ 时按行存 3n−2 个有效位置，坐标从 1、数组从 0 的统一公式为 $k = 2i + j - 3$，前提是 $|i-j| \le 1$ 且坐标不越界。n=1 时只有一个元素，应按单元素矩阵处理，不再套首尾行各 2 个的解释。

稀疏矩阵中零元位置不具有固定三角规律。三元组的 “下标” 表示第几个非零记录，不能直接由坐标用普通二维地址公式得到；修改原来的零元为非零元时，可能需要插入记录并调整次序或链接。

错项排除·FIFO 只是具体策略

队列 ADT 规定先进先出，操作系统的 “就绪队列” 名称却不保证所有调度算法都按 FIFO 选择。先来先服务可借 FIFO 说明，优先级等策略需要额外排序或选择规则。

3.10 典型应用的失败条件

中缀表达式将运算符写在左右操作数之间；前缀写在两者之前，后缀写在两者之后。前缀求值从右向左扫描，遇运算符先弹出的值是左操作数、后弹出的值是右操作数；后缀从左向右扫描则先弹出右操作数。方向与角色要一起记。

例如 $x + y(z-u)/v$ 按通常左结合规则，前缀为 `+ x / * y - z u v`，后缀为 `x y z u - * v / +`。两种形式都用位置表达运算依赖而省去括号；不是简单地把所有运算符搬到字符串开头或末尾。

按尚未完成对象的次序选结构

最近进入且尚未结束的对象要先处理，用栈：括号配对、函数调用、表达式中等待右操作数的运算。最早进入且尚未服务的对象先处理，用队列：层序遍历、BFS、先来先服务。操作系统的就绪队列只是管理等待对象，实际调度策略未必总是 FIFO。

后缀表达式求值必须同时检查：遇二元运算符时至少有两个操作数、除数合法、扫描末尾恰好留下一个结果。仅仅 “最终留下一个数” 不足以排除中途栈下溢。例如 `1 + 2` 按后缀语法扫描，在加号处就没有两个数；`1 2 3 +`则扫描后还剩两个值。

例 3.10.1 (自拟：括号数量正确仍可失败)

输入 `( [ ) ]`中左右圆括号和左右方括号数量分别相等。用栈检查时，在哪一个字符首次失败?

前两个字符依次入栈，栈顶为 `[`；读到第三个字符`)`时，它要求栈顶为 `(`，因此立即失败。只统计每类括号数量看不到嵌套关系，错误发生在第三个字符，不能等扫描完再随意重新配对。

辨析·栈顶选在链头

单链表头插头删只改常数条链接。若从单链表尾部弹出，还要先寻找前驱，代价会增加。

3.11 链栈、链队列与共享栈的实现边界

链栈通常不带头结点，Ltop 指向当前栈顶，空栈为 NULL。以下采用教材的链栈结点定义，申请与释放使用 malloc/free。

```
typedef struct Linknode {
    ElemType data;
    struct Linknode *next;
} *LiStack;
bool Push(LiStack &S, ElemType x) {
    LiStack p = (LiStack)malloc(sizeof(struct Linknode));
    if (p == NULL) return false;
    p->data = x;
    p->next = S;
    S = p;
    return true;
}
bool Pop(LiStack &S, ElemType &x) {
    if (S == NULL) return false;
    LiStack p = S;
    x = p->data;
    S = p->next;
    free(p);
    return true;
}
```

链栈从链头入栈、出栈，均为 $O(1)$；若改在普通单链表尾部出栈，定位尾结点前驱会变为线性时间。初始化令 S 为空即可；销毁时重复弹出或逐结点释放。

3.11.1 带头结点链队列

头结点使空队列和一般出队的前驱统一。front 始终指头结点，rear 指最后一个结点；空时二者指向同一个头结点。

```c
typedef struct LinkNode {
    ElemType data;
    struct LinkNode *next;
} LinkNode;
typedef struct {
    LinkNode *front, *rear;
} LinkQueue;
void InitQueue(LinkQueue &Q) {
    Q.front = Q.rear = (LinkNode *)malloc(sizeof(LinkNode));
    Q.front->next = NULL;
}
bool IsEmpty(LinkQueue Q) { return Q.front == Q.rear; }
void EnQueue(LinkQueue &Q, ElemType x) {
    LinkNode *s = (LinkNode *)malloc(sizeof(LinkNode));
    s->data = x;
    s->next = NULL;
    Q.rear->next = s;
    Q.rear = s;
}
bool DeQueue(LinkQueue &Q, ElemType &x) {
    if (Q.front == Q.rear) return false;
    LinkNode *p = Q.front->next;
    x = p->data;
    Q.front->next = p->next;
    if (Q.rear == p) Q.rear = Q.front;
    free(p);
    return true;
}
```

辨析·最后一删要回尾

删除唯一数据结点后，rear 必须退回 front；二者共同指向仍然有效的头结点。

初始化和入队保留王道课件的 void 接口，以内存申请成功为前提。实际程序须检查 malloc 结果，并在申请失败时保持原队列。最后一个数据结点出队后，必须把 rear 退回 front；随后再次入队才有有效尾指针。销毁链队列还需释放头结点。

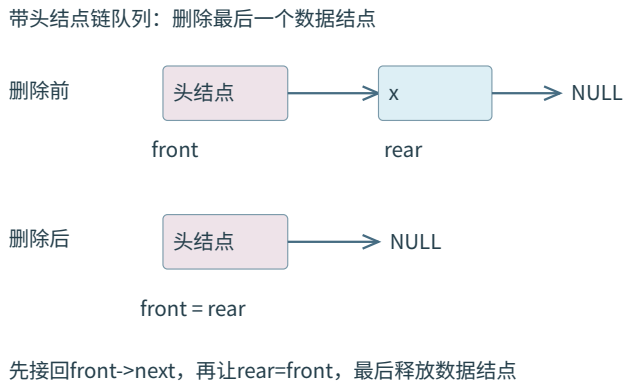


Figure 3.3: 最后一个数据结点出队后，front 与 rear 共同指向有效头结点

若采用不带头结点的版本，首次入队须同时建立 front 和 rear，最后一次出队须同时置空二者。

以下是另一套独立实现，仍使用 LinkQueue 类型，但 front 指首个数据结点。不能与前面的带头结点接口混合调用。

```
void InitQueue(LinkQueue &Q) {
    Q.front = Q.rear = NULL;
}
bool IsEmpty(LinkQueue Q) { return Q.front == NULL; }
void EnQueue(LinkQueue &Q, ElemType x) {
    LinkNode *s = (LinkNode *)malloc(sizeof(LinkNode));
    s->data = x;
    s->next = NULL;
    if (Q.front == NULL)
        Q.front = Q.rear = s;
    else {
        Q.rear->next = s;
        Q.rear = s;
    }
}
bool DeQueue(LinkQueue &Q, ElemType &x) {
    if (Q.front == NULL) return false;
    LinkNode *p = Q.front;
    x = p->data;
    Q.front = p->next;
    if (Q.rear == p) Q.rear = NULL;
    free(p);
    return true;
}
```

辨析·共享不改变次序

共享栈只共享空闲容量，两栈仍各自后进先出。top1-top0=1 表示中间已无空格。

3.11.2 共享栈的容量如何协同

```
typedef struct {
    ElemType data[MaxSize];
    int top0, top1;
} ShStack;
void InitStack(ShStack &S) {
    S.top0 = -1;
    S.top1 = MaxSize;
}
```

教材 `ShStack` 的两端栈顶初值为 `top0=-1,top1=MaxSize`，共享空闲区在两栈顶之间，满条件 `top1-top0==1`。若入 0 号栈，先检查满，再令 `data[++top0]=x`；入 1 号栈则令 `data[--top1]=x`。出 0 号栈取 `data[top0--]`，出 1 号栈取 `data[top1++]`，各自先判空。

共享只提高同一块空间的利用率；两个栈仍各自满足后进先出。若一栈已占满全部单元，另一栈即使为空也不能继续入栈。

3.12 括号匹配的完整状态检查

用字符作为 ElemType，复用顺序栈接口。新左括号意味着最近一个尚未完成对象；右括号必须关闭栈顶的同种左括号。

```
bool bracketCheck(char str[], int length) {
    SqStack S;
    InitStack(S);
    for (int i = 0; i < length; ++i) {
        char c = str[i];
        if (c == '(' || c == '[' || c == '{') {
            if (!Push(S, c)) return false;
        } else if (c == ')' || c == ']' || c == '}') {
            ElemType x;
            if (!Pop(S, x)) return false;
            if ((c == ')' && x != '(') ||
                (c == ']' && x != '[') ||
                (c == '}' && x != '{')) return false;
        }
    }
    return StackEmpty(S);
}
```

辨析·相同优先级看结合性

乘除和加减通常左结合，中缀转后缀遇同级运算符时先弹栈顶。新运算符不能抢先。

遇右括号但栈为空，属于右括号过早；栈顶类型不符，属于交叉嵌套；扫描结束栈非空，属于左括号尚未关闭。每字符只处理常数次，时间 $O(n)$，最坏保留 n 个左括号，辅助空间 $O(n)$；定长栈实现还要求实际所需深度不超过容量。

3.13 按课件表达式保留完整转换过程

中缀表达式 `A+B-C*D/E+F` 的运算次序为：先 A+B，再 C*D，再除以 E，再从 A+B 减去该商，最后加 F。按左结合规则得到后缀：

```
A B + C D * E / - F +
```

当前符号	运算符栈（底到顶）	已输出后缀
A	空	A
+、B	+	A B
−	−	A B +
C、*、D	−、*	A B + C D
/	−、/	A B + C D *
E	−、/	A B + C D * E
+	+	A B + C D * E / −

当前符号	运算符栈（底到顶）	已输出后缀
F、扫描结束	空	A B + C D * E / − F +

遇到/时先弹同级 *，对应乘除左结合；遇到最后一个 +时，先弹除号再弹减号。对于 `A+B*(C-D)-E/F`，括号内的减法先完成，得到 `A B C D - * + E F / -`。左括号只作为界限保存在栈中，不进入后缀输出。

课件的数值表达式

$$((15/(7-(1+1)))\times 3)-(2+(1+1))$$

对应后缀为 `15 7 1 1 + - / 3 * 2 1 1 + + -`。依次计算 $1+1=2$、$7-2=5$、$15/5=3$、$3\times 3=9$，右部分为 $2+(1+1)=4$，结果为 5。数字 15 是一个操作数，不能按字符 1 和 5 分开入栈。模拟可知操作数栈峰值为 4 个值，这和表达式的字符总数不同。

前缀形式用 “运算符、左操作数、右操作数” 表达同一依赖。从右向左求前缀值时，先弹出的数为左操作数；从左向右求后缀值时，先弹出的数为右操作数。把两种扫描方向混用，会在减除法中得到错误结果。

辨析·三角区与常量区

对称矩阵只存半边；三角矩阵还需存另一半的公共常量。两者存储单元数相差 1。

3.13.1 直接用两个栈计算中缀表达式

也可以将 “中缀转后缀” 与 “后缀求值” 融合：操作数立即压入操作数栈；运算符和括号按转换算法管理。每当转换规则本应输出一个运算符时，直接从操作数栈先弹右操作数、再弹左操作数，计算后将结果压回。左括号仍只作边界，不参与运算；表达式结束时清理运算符栈。

以 `A+B-C*D/E+F` 为例，在读到减号时，先弹出加号并把 A+B 压回；读到除号时，先完成 *CD*；读到最后一个加号时，依次完成 *(C*D)/E 和 (A+B)−(C*D)/E；最后再加 F。这与后缀 `A B + C D * E / - F +`表达同一依赖。两个栈分别保存 “尚未归约的操作数” 和 “尚未确定何时执行的运算符”，不可把运算符与值混放在同一种栈元素模型中。

除括号配对外，还应检查操作数数量、运算符合法性、除零及最终结果数；若题目包含一元负号或多位数，应先按词法单元识别，再应用相应操作数个数的规则。

3.14 数组与特殊矩阵的定位总表

下表统一规定：原矩阵坐标从 1 开始，压缩数组 B 下标从 0 开始；n 为阶数。先依据对称性换到实际保存的三角区，再计算 k。

保存区域与次序	合法坐标	压缩下标 k
下三角行优先	$i \ge j$	$i(i-1)/2 + j - 1$
上三角列优先	$i \le j$	$j(j-1)/2 + i - 1$
上三角行优先	$i \le j$	$(i-1)(2n-i+2)/2 + j - i$
下三角列优先	$i \ge j$	$(j-1)(2n-j+2)/2 + i - j$
三对角行优先	$\lvert i-j \rvert \le 1$	$2i + j - 3$

对称矩阵保存 $n(n+1)/2$ 个元素即可。三角矩阵另一个区域是同一个公共常量，需再保存一个常量单元，共 $n(n+1)/2+1$ 个；公共常量不必为 0。三对角矩阵 n≥2 时保存 $3n-2$ 个元素，首尾行各 2 个，中间每行 3 个。n=1 时只有一个有效位置，应直接按单元素处理。

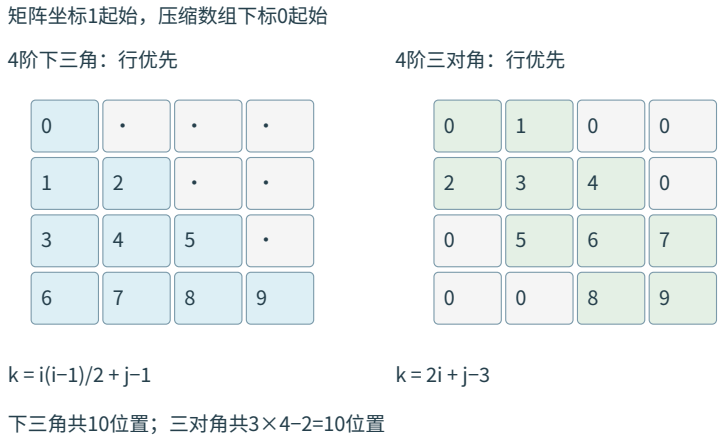


Figure 3.4: 4 阶下三角与三对角矩阵的压缩位置，矩阵 1 起始、数组 0 起始

反向给定三对角压缩下标 k 时，第 i 行之前有 $3(i-1)-1$ 个保存位置（首行单独核对），第 i 行结束前共 $3i-1$ 个。因此可以取

$$i=\left\lceil \frac{k+2}{3} \right\rceil, \qquad j=k-2i+3.$$

行号也可写为 $i=\lfloor (k+1)/3 \rfloor+1$，与上述向上取整形式等价；应先确定 i 再求 j。

例如 k=87，$i=\lceil 89/3 \rceil=30$，$j=87-60+3=30$，与例 3-3 一致。前提是 $0\le k<3n-2$ 且 n≥2；得到坐标后还可复核 $|i-j|\le 1$。

课件的 5 行 6 列稀疏矩阵有六个非零项，按行次序的三元组为：

行 i	列 j	值 v
1	3	4
1	6	5
2	2	3
2	4	9
3	5	7
4	2	2

第 5 行全零，仍属于矩阵；只看非零记录无法恢复总行数，所以三元组表还要记录矩阵总行数、总列数和非零元数。十字链表中同一个非零结点同时进入行链和列链，修改时须维护两套联系。稀疏仅说明非零元少，不能据此使用三角矩阵的固定位置公式。

自查提示·队列的满空条件

写清采用牺牲一个单元、计数器还是标志位，才能计算容量并判断队满。

3.15 主动回忆自测

第 2 题引自王道书，其余为原创补充，检查表示约定。

1. 数组有 8 个单元、牺牲一个单元的循环队列，`front=6,rear=2`，长度是多少？还能再入队几个元素？
2. 〖3.1-025〗顺序共享栈 `Share[0..n-1]`，第一个栈顶 top1 初值 −1，第二个栈顶 top2 初值 n，栈满条件是（　）。A. `top2-top1==1`　B. `top1-top2==1`　C. `top1==top2`　D. 都不对
3. 下标从 1 开始的三对角矩阵，若压缩数组也从 1 开始，非边界行元素 $a_{i,j}$ 的位置如何计算？

答案。1. 长度 $(2-6+8) \bmod 8 = 4$，最大容量 7，还能入队 3 个。2. **A**，两栈顶相邻时已无可用单元，不能等栈顶重合才判满。3. 前面有 $2+3(i-2) = 3i-4$ 个元素，本行从 $j = i-1$ 开始，故从 1 开始的位置为 $3i-4+[j-(i-1)]+1 = 2i+j-2$；前提为 $2 \le i \le n-1$ 且 $|i-j| \le 1$，首尾行须按实际保存位置解释。

自查提示·队列的满空条件

写清采用牺牲一个单元、计数器还是标志位，才能计算容量并判断队满。

3.15.1 补充自拟回忆题

先画状态，再作判断

4. 入栈顺序为 a、b、c，输出 c、a、b 是否合法？第一次失败时栈顶是谁？
5. 带头结点链队列删除最后一个数据结点时，front 和 rear 应指向哪里？
6. 同一个 n 阶对称矩阵，为什么下三角行优先和上三角列优先有相同的三角数结构？坐标如何转换？
7. 后缀表达式 `8 3 - 2 /`的值是多少？若按整数除法计算，先弹出的数是什么角色？
8. 递归访问 n 个结点，最大同时活跃调用数一定为 n 吗？应由哪个结构参数决定？

补充题核对答案

4. 不合法。弹出 c 后栈中为 a、b，想输出 a 时栈顶是 b，而输入已用尽。
5. front 仍指有效头结点，rear 必须退回 front；头结点的 next 为空。rear 不能留在已释放的数据结点上。
6. 一边各行保存 1、2、⋯ 个元素，另一边各列也保存 1、2、⋯ 个。先用 aij=aji 交换坐标，再按对应行或列求偏移。
7. 先计算 8−3=5，再作 5/2，整数结果为 2；每次先弹出右操作数，除法的除数是先弹出的 2。
8. 不一定。由最长未返回调用链决定；树遍历通常取树高 h，最坏链形树才可达到 n。

4 串

章节脉络

串把字符序列作为一个整体处理。学习本章要同时分清三个问题：逻辑上哪些字符属于串，存储时怎样记录字符与长度，匹配时怎样利用已经比较过的信息。本章代码采用王道的顺序串表示：字符从位置 1 开始，`ch[0]`弃用，`length` 另存串长；位置和匹配结果均按这一约定解释。

辨析·空串也有定义

空串长度为 0；空格串的空格都是字符。子串必须连续，子序列可以不连续。

4.1 字符串、子串与基本操作

串是由零个或多个字符组成的有限序列，记为 $S=' a_1a_2\cdots a'_n$；n 为串长。$n=0$ 的是空串。一个或多个空格构成空格串，它含有字符，长度大于 0。串中的字符可以是字母、数字、标点或其他编码字符；具体程序必须约定一个存储单元表示什么，本章算法以一个 `char` 为一个比较单位。

串中任意**连续**字符组成的子序列称为子串，包含该子串的串称为主串。子串在主串中的位置用其第一个字符的位置表示；同一值可能多次出现，例如 `ababa` 中的 `aba` 从位置 1 和 3 开始。模式串是要查找的串，它可能不存在于主串中；不能先把“模式串”当成已找到的子串。

两个串相等，当且仅当长度相同且每个对应字符都相同。串的大小比较采用字典序：从前向后找第一对不同字符，按约定的字符编码次序决定大小；若一方先结束且已比较部分全相同，较短串较小。例如 `wang` 小于 `wangdao`；比较串的大小不是比较串长，也不是将整个串当成十进制数。

串属于线性结构，但串的基本操作通常以一个串或子串作为处理对象，而普通线性表常以单个数据元素为对象。赋值、比较、连接、取子串、替换与定位是不同运算，输入输出和修改对象应写清。

操作	语义与结果	需要检查的条件
`StrAssign(T,chars)`	将字符序列赋给 T	目标容量、字符数
`StrCopy(T,S)`	用 S 的值复制得到 T	独立复制还是共享存储
`StrEmpty(S)`、`StrLength(S)`	判空、取得串长	空串的 length 为 0
`ClearString(S)`	使 S 成为空串	顺序串清空不必释放固定数组
`DestroyString(S)`	销毁并回收动态存储	堆串与定长串的资源行为不同
`Concat(T,S1,S2)`	按先后连接两串得到 T	长度之和能否容纳
`SubString(Sub,S,pos,len)`	取位置 pos 起、长度 len 的子串	起点、长度及末位置合法

操作	语义与结果	需要检查的条件
`StrCompare(S,T)`	返回正、零或负值表示大小关系	字符编码比较规则
`Index(S,T)`	返回 T 在 S 中第一次出现的位置	本章失败返回 0

辨析·先定接口边界

允许空子串时，pos 可以等于 n+1，但 len 只能为 0；越界请求不能读取字符。

4.1.1 子串的起点和长度怎样约束

设 S 长 n，取非空子串时须满足 $1 \le pos \le n$、$len > 0$ 及 $pos + len - 1 \le n$。若允许空子串，可将条件扩展为 $1 \le pos \le n+1$、$len \ge 0$ 及 $pos + len - 1 \le n$；这时 $n+1$ 只允许取长度 0 的串。接口采用哪种边界应明确约定。本章给出的 `SubString` 允许后一种情况；模式匹配的主体分析采用非空模式，空模式在调用入口约定返回位置 1。

4.2 顺序串、堆串与链串

定长顺序存储用连续字符数组保存串，串长由独立成员表示。教材结构以 `MAXLEN` 指定数组单元数。结合本章位置 0 弃用的约定，可直接访问的字符位置是 `1..MAXLEN-1`，所以实现时维持 `0<=length<MAXLEN`。若软件接口要求可保存整整 MAXLEN 个字符，则需要额外分配一个单元，并同步声明容量约定。

```c
#define MAXLEN 255
typedef struct {
    char ch[MAXLEN];
    int length;
} SString;
```

固定数组的容量有限。清空只需令 `length=0`，旧字符即使仍留在数组中，也不再属于有效串。显式记录长度时无需用`'\0'` 作结束标志；若把这种串传给要求 C 字符串的库函数，必须额外转换表示，不能直接把 `ch` 作为同一种接口使用。

教材式顺序串存在不同布局：用 `ch[0]`记录长度、从位置 1 保存字符；另设长度而从位置 0 保存字符；以`'\0'` 结束；另设长度且字符从 1 开始。本章统一采用最后一种。第一种还受长度单元可表示范围限制；结束符方式需要扫描才能求长，并将结束符与有效字符区分。

堆分配存储用指针指向动态申请的连续区，常见定义为：

```c
typedef struct {
    char *ch;
    int length;
} HString;
```

动态分配能按实际需要调整空间，代价是要管理申请、复制、释放和失败情形。赋值一个指针只复制地址，不能自动得到独立字符副本。长度改变且容量不足时，应先申请新空间，成功后搬移并释放旧区，避免申请失败后丢失原串。

辨析·返回符号即可

串比较的正、零、负分别表示大于、相等、小于；正值不要求恰好是 1。

链式存储可以每结点存一个字符，也可以每结点保存多个字符形成串块。若每字符 1 字节而指针 4 字节，单字符结点只计数据与链接时，存储密度为 $1/(1+4)=20\%$；一个结点存 4 字符时可提升至 $4/(4+4)=50\%$。实际平台指针宽度和结构体对齐还会改变字节数。块结点末尾的填充符与真实字符应靠长度或其他约定区分。

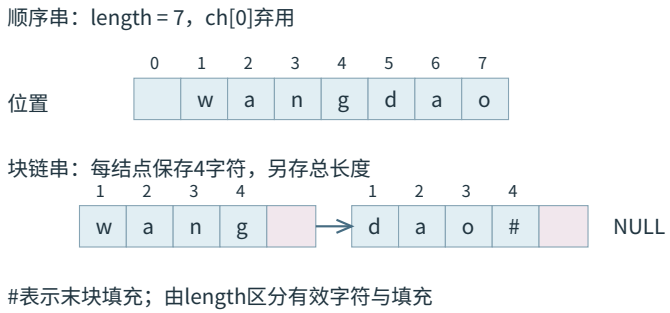


Figure 4.1: 顺序串的 1 起始字符位置与链块串的链接关系

4.3 基本操作怎样落实到代码

`SubString` 先检查范围，再从主串复制；失败时保留输出串原值。按固定数组模型，最多复制 `len` 个字符，时间 $O(len)$，除返回结果外只用常数个工作变量。

```
bool SubString(SString &Sub, SString S, int pos, int len) {
    if (pos < 1 || pos > S.length + 1 || len < 0 ||
        len > S.length - pos + 1)
        return false;
    for (int i = pos; i < pos + len; ++i)
        Sub.ch[i - pos + 1] = S.ch[i];
    Sub.length = len;
    return true;
}
int StrCompare(SString S, SString T) {
    for (int i = 1; i <= S.length && i <= T.length; ++i)
        if (S.ch[i] != T.ch[i])
            return (unsigned char)S.ch[i] -
                   (unsigned char)T.ch[i];
    return S.length - T.length;
}
```

比较的时间由第一次不同的位置决定，最坏 $O(\min(n,m))$。返回值只要求正、零、负正确；不能把“返回正值”误背成“必须返回 1”。转为 `unsigned char` 后比较，可以避免某些实现中 `char` 有符号导致高位编码被当作负数。

连接先检查总长，再分别复制两段。若接口允许输出与输入是同一个对象，需要按实现检查覆盖问题；下例按值接收定长顺序串，输入副本使 `Concat(S,S,T)`仍能读取原来的 S。

```
bool Concat(SString &T, SString S1, SString S2) {
    if (S1.length + S2.length >= MAXLEN) return false;
    for (int i = 1; i <= S1.length; ++i)
        T.ch[i] = S1.ch[i];
    for (int i = 1; i <= S2.length; ++i)
        T.ch[S1.length + i] = S2.ch[i];
    T.length = S1.length + S2.length;
    return true;
}
```

辨析·回溯到哪里

本轮起点是 i-j+1，失败后换下一起点 i-j+2，模式 j 恢复为 1。

若用字符串基本操作描述朴素定位，可以依次取长度 m 的子串，再用 `StrCompare` 比较。非空模式最多有 $n-m+1$ 个候选起点；模式长于主串时没有候选。下面改用字符位置直接实现，能更清楚地看到回溯和字符比较次数。

4.4 朴素匹配：失败后移到下一个起点

令 `i` 指主串当前比较字符，`j` 指模式当前比较字符，二者都从 1 开始。相等时同时递增；不等时，已经比较的模式前缀长为 $j-1$，本轮起点是 $i-j+1$，下一个起点是 $i-j+2$，因此执行 `i=i-j+2; j=1;`。

```
int Index(SString S, SString T) {
    if (T.length == 0) return 1;
    int i = 1, j = 1;
    while (i <= S.length && j <= T.length) {
        if (S.ch[i] == T.ch[j]) {
            ++i; ++j;
        } else {
            i = i - j + 2;
            j = 1;
        }
    }
    if (j > T.length) return i - T.length;
    return 0;
}
```

成功时，`j=m+1`，`i` 已越过匹配段，起点为 $i-m$。主串先耗尽而模式仍未完成时失败，返回 0。失败值 0 与合法位置 1 区分，不应改用另一个接口的 `-1`。

4.4.1 例：按课件主串逐次尝试

主串 `S=abaabaabcabaabc`，模式 `T=abaabc`。起点 1 处前 5 字符相同，第 6 字符 `a` 与 `c` 失配；起点 2 的 `b` 与模式首字符 `a` 失配；起点 3 先比较一个 `a`，下一字符 `a` 与 `b` 失配；起点 4 的 6 个字符全部相等，第一次成功位置为 4。

尝试起点	本轮字符比较次数	结果
1	6	第 6 对失配
2	1	第 1 对失配
3	2	第 2 对失配
4	6	全部匹配

合计 15 次字符比较。这个总数属于这一组数据，不能当作算法的一般复杂度。

最坏情形如主串前部有大量 `a`、模式为 `aaaab`：每个起点都在接近末尾处失败，时间为 $O((n-m+1)m)$，通常简记 $O(nm)$。若首起点就完全匹配，则只比较 m 次；讨论最好情况时须说明是否保证成功及是否把建表等准备工作计入。

辨析·位置与长度差 1

next[j] 是下一比较位置。j 前的最长边界长 k 时，next[j]=k+1。next[1]=0 另行定义。

4.5 KMP：利用已相等部分决定回退

设在位置 i, j 失配。此前有

$$S[i-j+1..i-1] = T[1..j-1].$$

这段主串已经被证明等于模式前缀。模式右移后仍能保留的部分，必须同时是 $T[1..j-1]$ 的真后缀和模式的真前缀。设最长相等真前后缀长为 k，那么前 k 字符无需再比较，下一次应将模式位置 $k+1$ 与同一个主串字符比较。

因此，对 $j > 1$ 定义

$$next[j] = 1 + \text{串} T[1..j-1] \text{ 的最长相等真前后缀长度},$$

并定义 $next[1] = 0$。`next[j]`保存的是**下次比较的位置**，不是直接保存前后缀长度。值 0 是首字符失配后的特殊状态：下一轮同时令 `i` 和 `j` 递增，主串进入下一字符，模式恢复到位置 1。

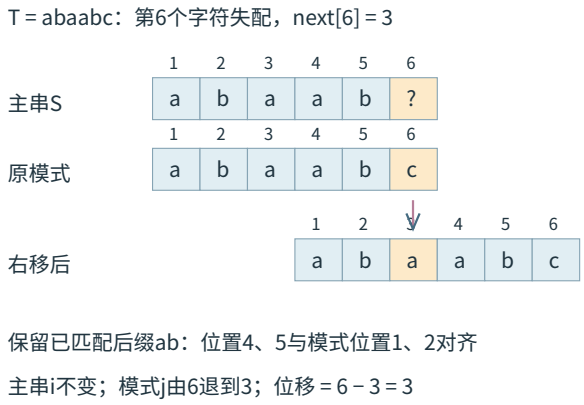


Figure 4.2: abaabc 在第 6 位置失配后，已匹配段的后缀 ab 与模式前缀 ab 对齐，j 由 6 退到 3

模式 `abaabc` 的普通 next 为：

位置 j	1	2	3	4	5	6
T.ch[j]	a	b	a	a	b	c
j 之前的串	空	a	ab	aba	abaa	abaab
最长相等真前后缀长	特殊	0	0	1	1	2
next[j]	0	1	1	2	2	3

例如 $j=6$ 前的串是 abaab，最长相等真前后缀为 ab，长 2，所以 next[6]=3。当第 6 字符失配时，只改变 j 到 3，i 不变；不是把模式移到主串位置 3。

辨析·主串没有回溯

失配时 i 保持不变，先改 j；只有字符相等或 j 退至 0 时才推进 i。

```
int Index_KMP(SString S, SString T, int next[]) {
    if (T.length == 0) return 1;
    int i = 1, j = 1;
    while (i <= S.length && j <= T.length) {
        if (j == 0 || S.ch[i] == T.ch[j]) {
            ++i;
            ++j;
        } else {
            j = next[j];
        }
    }
    if (j > T.length) return i - T.length;
    return 0;
}
```

短路求值使 j==0 时不访问 T.ch[0]。模式在逻辑上右移，主串指针单调递增；同一个主串字符仍可能与多个模式位置比较，所以不能将 KMP 解释成“每个主串字符恰好比较一次”。

4.5.1 例：同一主串上的 KMP 轨迹

仍取 S=abaabaabcabaabc、T=abaabc。先比较位置 $(1,1)$ 至 $(5,5)$ 成功，在 $(6,6)$ 失配。由 next[6]=3，下一次比较 $(6,3)$，主串位置 6 的 a 等于模式位置 3 的 a；随后比较 $(7,4)$、$(8,5)$、$(9,6)$ 都成功。此时 i=10,j=7，返回 $10-6=4$。

阶段	主串位置 i	模式位置 j	操作
已有 5 字符相等	6	6	a 与 c 失配
查 next	6	3	主串位置不变
继续比较	6、7、8、9	3、4、5、6	连续 4 对相等
完成	10	7	返回位置 4

本例普通 KMP 共比较 10 次字符，少于朴素的 15 次；判断 j==0 不算一对字符比较。另一课件主串 abaacaabcabaabc 会产生连续回退：在 $i=5,j=5$ 失配后 $j=2$，仍失配再退 $j=1$，再失配退 $j=0$；此后推进主串到 $i=6,j=1$。连续回退只查表，不让主串后退。

4.6 怎样在线性时间内求 next

求表也可看成模式串与自身前缀的匹配。设正在求 next[i+1]，已知 next[i]=j。若 j==0 或 T.ch[i]==T.ch[j]，当前相等前后缀可扩展一字符，故 next[i+1]=j+1；否则先尝试更短的候选边界，令 j=next[j]。

```
void get_next(SString T, int next[]) {
    if (T.length == 0) return;
    int i = 1, j = 0;
    next[1] = 0;
    while (i < T.length) {
        if (j == 0 || T.ch[i] == T.ch[j]) {
            ++i; ++j;
            next[i] = j;
        } else {
            j = next[j];
        }
    }
}
```

辨析·先切出 j 前的串

求 next[j] 只看 T[1..j-1]，当前位置字符不加入普通边界长度的计算。

这里的 `i` 是模式串内部的工作位置，与匹配函数的主串工作位置作用不同。`while(i<T.length)`保证递增后最多写到 `next[T.length]`。若使用另一种定义而把 `next` 记成前缀函数，表值和匹配代码就必须一起换；本章始终使用教材的下一比较位置定义。

4.6.1 三组课件手算练习

模式串	位置	普通 next
google	1、2、3、4、5、6	0、1、1、1、2、1
ababaa	1、2、3、4、5、6	0、1、1、2、3、4
aaaab	1、2、3、4、5	0、1、2、3、4

`google` 在位置 5 前的 `goog` 有长度 1 的相等真前后缀 `g`，所以 `next[5]=2`；位置 6 前的 `googl` 没有非空相等真前后缀，因此 `next[6]=1`。`ababaa` 在位置 6 前的 `ababa` 有最长边界 `aba`，所以 `next[6]=4`。`aaaab` 每个待考察前缀都是一串 `a`，其最长真边界只比它本身短 1。

课件主串 `googlogoogoogle`、模式 `google` 的匹配过程会反复利用这些边界，第一次成功位置为 10。手算时先列主串和模式的位置行，再记录每一次失配后的 `j`，比仅画模式移动的外形更容易核对。

4.7 nextval 怎样跳过必然失败的比较

若当前 `S.ch[i]`与 `T.ch[j]`已失配,而 `T.ch[next[j]]==T.ch[j]`，同一个主串字符与回退位置的字符也一定失配。可以继续沿失败链接，直接寻找字符不同的候选位置；把最终位置记为 `nextval[j]`。

由普通 next 求优化表时，先令 $nextval[1]=0$。为简化公式，记 $T[j]=T.ch[j]$，对 $j\ge 2$ 有：

$$nextval[j]=\begin{cases} next[j], & T[j]\neq T[next[j]],\\ nextval[next[j]], & T[j]=T[next[j]]. \end{cases}$$

对 $j\ge 2$，普通 `next[j]`至少为 1，所以该式不访问 `T.ch[0]`。`nextval` 使用已确认的失配信息，省去必定失败的比较，不改变匹配结果。

模式串	普通 next	优化 nextval
abaabc	0、1、1、2、2、3	0、1、0、2、1、3
ababaa	0、1、1、2、3、4	0、1、0、1、0、4
aaaab	0、1、2、3、4	0、0、0、0、4

辨析·两种表两种次数

aaaab 在 aaacaaaabdaaaab 中首次匹配位于 5；普通 next 比较 12 次，nextval 比较 9 次。

例如 `abaabc` 的 $j=3$ 字符为 `a`，而 `next[3]=1` 也指向 `a`，故 `nextval[3]=nextval[1]=0`；$j=5$ 是 `b`，普通回退到 2 仍为 `b`，故 `nextval[5]=nextval[2]=1`。

```
void get_nextval(SString T, int nextval[]) {
    if (T.length == 0) return;
    int i = 1, j = 0;
    nextval[1] = 0;
    while (i < T.length) {
        if (j == 0 || T.ch[i] == T.ch[j]) {
            ++i; ++j;
            if (T.ch[i] != T.ch[j])
                nextval[i] = j;
            else
                nextval[i] = nextval[j];
        } else {
            j = nextval[j];
        }
    }
}
```

将 `nextval` 作为 `Index_KMP` 的第三个实参即可使用优化表，匹配循环不变。普通 next 与优化 nextval 的具体比较次数可以不同，算题前先确认题目采用哪一张表。

4.7.1 例：重复字符使优化效果更明显

主串 `aaacaaaabdaaaab`，模式 `aaaab`。普通 next 在主串位置 4 的 `c` 处，会依次与模式位置 4、3、2、1 的 `a` 失配；优化表在位置 4 第一次失配后直接退到 0，跳过另外三次注定失败的比较。之后从主串位置 5 开始匹配，位置 5 至 9 恰为 `aaaab`，故返回 5。普通 next 共比较 12 次，nextval 共比较 9 次。

4.8 正确性、代价和容易混淆的边界

KMP 正确性的关键是：每一次跳过的对齐位置都被已经相等的前缀与后缀关系排除。模式右移后还能与旧匹配段重合的部分必须是一个边界；先尝试最长边界，失败后沿 next 链尝试更短边界，因此不遗漏仍可能成功的起点。

建表过程中，`i` 最多递增 $m-1$ 次；回退使 `j` 减小，而 `j` 的总增量受 `i` 的推进限制，故总工作 $O(m)$。匹配中，主串指针 `i` 最多递增 n 次，每次成功比较使 `j` 增加，每次失败回退使 `j` 减少；累积回退量仍受此前的推进约束，因此匹配扫描为 $O(n)$。总时间 $O(n+m)$，next 数组辅助空间 $O(m)$。只有“已给 next”的匹配阶段才可单独记为 $O(n)$。

问题	本章约定或结论	易混点
字符起始位置	1	与 C 数组常见 0 起始分别说明
匹配失败	返回 0	0 不是合法非空匹配起点
`next[j]`	下一次比较的模式位置	等于边界长度加 1，而非直接等于边界长度
`j==0`	下轮推进主串并恢复 j=1	不是读取第 0 个字符
字符比较次数	实际执行的字符相等判断	`j==0`、循环条件不是字符对比较
匹配成功后继续查找	取决于是否查全部、是否允许重叠	只找首次出现的接口可立即返回

辨析·失败值和合法位置

正文所有非空串位置从 1 起，失败返回 0；不要将另一种下标接口的返回值混入。

固定数组按值传参会复制整个结构体；教材复杂度通常关注字符扫描和串长变化，容量视作预设上限。若要在可增长规模的软件实现中精确计算拷贝成本，应把接口实现成本一并计入。本章算法推导仍采用教材的字符比较模型。

4.9 主动回忆与自查

以下为自拟检查题，串的位置和 next 均采用本章约定。

1. 空串和只含三个空格的串长度分别是多少？`abc` 是不是 `ac` 的主串？
2. 长度 7 的串取 `pos=6,len=2` 是否合法？`pos=8,len=0` 呢？
3. 字符位置为 1 起始时，朴素匹配在 $(i, j) = (8, 4)$ 失配，下一个起点和新的指针分别是什么？
4. 写出 `abaabc`、`ababaa`、`aaaab` 的 next 与 nextval，解释 `next[6]=3` 中 3 的含义。
5. KMP 在同一个主串字符上连续失配，会改变哪个指针？何时才推进主串？
6. 对 `S=abaabaabcabaabc,T=abaabc`，朴素与普通 KMP 第一次成功位置及字符比较次数分别是多少？
7. 对 `S=aaacaaaabdaaaab,T=aaaab`，普通 next 和优化 nextval 各比较多少次？
8. 空模式由接口约定返回 1 时，是否需要访问 next[1]？若模式长于主串，返回什么？
9. 只给匹配扫描的 $O(n)$，为何不足以说明整个 KMP 的准备与扫描代价？

4.9.1 核对答案

1. 分别为 0 和 3；`ac` 不是 `abc` 中的连续子串。2. 均合法；第二种取得串尾后的空串。3. 本轮起点为 5，下一个为 6，执行 `i=6,j=1`。4. next 和 nextval 分别见上表；3 表示下一比较位置，保留的是长 2 的边界 `ab`。5. 令 `j=next[j]`或 `j=nextval[j]`；字符相等或 j 退到 0 后推进主串。6. 都返回 4；朴素 15 次，普通 KMP10 次。7. 分别 12 次、9 次。8. 不需要建表或访问表；非空模式长于主串时返回 0。9. 还要把 $O(m)$ 建表时间和 $O(m)$ 表空间计入，总时间 $O(n+m)$。

树与二叉树 5

章节脉络

树用父子关系组织层次。二叉树在此基础上区分左、右位置，遍历把结构转成一个处理次序；哈夫曼树、并查集和堆则各自增加约束，分别处理编码、集合合并和最值访问。它们共享树的表示工具，但各自的性质不能互相替代。

课程连接·目录树和进程树

操作系统的目录层次、进程派生关系可用树表达。硬链接若允许同一对象有多个上级入口，目录对象关系就不能直接套用“除根外唯一双亲”；先核对对象与链接的含义。

5.1 本章知识结构

- **树与森林的概念**：结点与父子兄弟；有序树与无序树；结点度与树度；层次高度与路径；边数及叶数计数。
- **二叉树性质与表示**：满与完全二叉树；叶数与双孩子结点；顺序编号及空位；二叉链表空域。
- **遍历与线索化**：先中后序及层序；非递归栈状态；遍历序列还原；中序线索与标志；前驱后继及边界。
- **树森林的存储与转换**：双亲表示；孩子表示；孩子兄弟表示；左孩子右兄弟转换；先根后根与二叉遍历。
- **哈夫曼树与编码**：带权路径长度；合并两个最小权根；最优性与编码非唯一；前缀编码与剩余容量；多路归并的零权补叶。
- **并查集与堆的连接**：双亲森林与代表元；按规模合并；路径压缩不改集合；完全性与堆序；插入上滤及删根下滤。

树的概念和计数性质是后续推导的基础。读遍历时同时追踪程序、栈或队列与输出序列；读编码和并查集时，再考察它们各自增加的约束。

5.2 树的结点、度、层次与路径

定义 5.2.1 (树及结点的度)

有序树规定同一结点各子树的先后，交换两个孩子可能改变树；**无序树**不区分这种先后。森林是零棵或多棵互不相交的树，删去一棵非空树的根后，各子树构成森林；在一片森林上加新根、把各棵树根接为孩子，又得到一棵树。普通树的有序性与“关键字从小到大”无关。

非空树有且仅有一个根。除根外每个结点恰有一个双亲，根到任意结点只有一条简单路径；树的各子树互不相交。一个结点的孩子数称为该结点的**度**，树的度是所有结点度的最大值。度为 0 的结点称为叶结点。树中“度”通常指数孩子的个数，与无向图把关联边条数称为度的约定不同。

最小反例·度为 2 未必是二叉树

最小情形只有根与一个孩子：普通无序树不区分这个孩子在左还是在右，二叉树却区分两种位置。因此“每结点至多两个孩子”尚未表达二叉树的全部结构语义。

本章按王道课件约定，根为第 1 层，结点的层次（深度）从根向下按层计，树的高度（深度）为最大层号；结点高度从该结点向下到最深叶子按层计，叶子高度为 1，空树高度为 0。根到结点的路径长度仍按边数计，等于该结点层次减 1。题目若另行按边数定义高度或深度，须先换算全部相关公式。

路径长度以边数计，不以沿途结点数计。树的路径长度是根到各结点路径长度的总和；树高由最长的一条路径决定，二者不同。孩子、子孙、兄弟也要区分：兄弟共有同一个双亲，与堂兄弟不是同一个概念。

5.2.1 例 5-1　怎样从非叶结点数求叶结点数

例 5.2.1 (配套案例)

【配套例题】一棵度为 4 的树，含 20 个度为 4 的结点、10 个度为 3 的结点、1 个度为 2 的结点、10 个度为 1 的结点。叶结点数为（　）。

A. 41　　B. 82　　C. 113　　D. 122

解析。设叶数为 n_0。一方面总结点数为 $n = n_0 + 20 + 10 + 1 + 10 = n_0 + 41$；另一方面每条边从一个双亲指向一个孩子，边数是度数总和 $4 \times 20 + 3 \times 10 + 2 + 10 = 122$。树有 $n - 1$ 条边，所以 $n_0 + 40 = 122$，得 $n_0 = 82$，答案为 **B**。

一般地，令 n_i 为度为 i 的结点数，由 $\sum_i n_i - 1 = \sum_i i n_i$ 得

$$n_0 = 1 + \sum_{i \ge 2} (i-1) n_i .$$

度为 1 的结点不出现在右端，是因为在一条边上再串入一个单孩子结点，结点数、边数各增加 1，叶数不变。因此只给定 n_2, n_3 和叶数，未必能确定全树结点总数；还可能插入任意多个度为 1 的结点。

每结点至多有 m 个孩子的树，第 i 层至多有 m^{i-1} 个结点，高度 h 至多容纳 $(m^h - 1)/(m-1)$ 个结点（$m > 1$）。求最小高度时，找使容量至少达到 n 的最小整数 h。例如至多 3 叉、50 个结点，高 4 时最多 40 个，高 5 时最多 121 个，所以最小高度 5。这里必须把“度为 3”理解为全树最大孩子数 3，不能要求每个非叶结点都恰有 3 个孩子。

5.2.2 度为 m 的树与 m 叉树

“度为 m” 要求全树至少有一个结点恰有 m 个孩子；“m 叉树” 只要求每个结点至多有 m 个孩子。二者都满足每层的容量上界，最少结点数却不同。对非空树，按根为第 1 层：

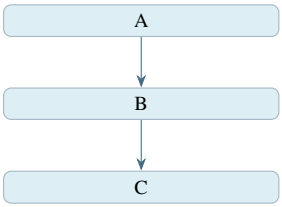
条件	最少结点数	最多结点数
高度 h 的 m 叉树	h，取一条单支链	$(m^h-1)/(m-1)$，$m>1$
高度 h 且度恰为 m	$h+m-1$，需存在 m 个孩子的分支	同上
有 n 个结点的 m 叉树	最小高度 $\lceil \log_m(n(m-1)+1) \rceil$	最大高度 n

第二行须有能容纳 m 个孩子的高度：$m>0$ 时 $h\geq 2$；单结点树度为 0。$m=1$ 时是一条链，容量公式改为 $n=h$，避免分母为 0。公式中的 “至多” 允许空缺，不能据容量上界推断某一棵给定树的实际形状。

例如高度 4、度为 3 的树至少有 $4+3-1=6$ 个结点：在一条 4 结点链的某个非叶结点处再增加 2 个叶即可；只说 “高度 4 的 3 叉树”，最少则只有 4 个结点。

错项排除·叶公式只用了二叉条件

证明 $n_0=n_2+1$ 时没有用到 “完全” 或 “满”。若选项声称任意二叉树还必有至多一个度为 1 结点，就偷偷增加了完全性；一条三结点单支链即可排除。



三结点单支链有 2 个度 1 结点；叶数公式仍成立。

5.3 二叉树、满二叉树与完全二叉树

二叉树的每个结点至多有两个孩子，而且左右有别。只有一个孩子时，左孩子与右孩子形成不同结构；二叉树也不是简单地把 “度不超过 2 的普通树” 换个名字。普通树的单孩子若不规定位置，就不区分左、右这两种形态。

高度为 h、每一层都达到最大结点数的二叉树称为**满二叉树**，共有 2^h-1 个结点。**完全二叉树**按满二叉树从上到下、从左到右编号，已有结点恰占前 n 个编号。最后一层可以不满，但只能缺右侧连续的一段；其余各层则必须排满。

“每个非叶结点恰有两个孩子” 只说明不存在度为 1 的结点，未必是这里定义的满二叉树。叶结点可能分布在不同层。使用外文资料中的 full、perfect 等名称时，也要先看定义。

二叉树第 i 层至多 2^{i-1} 个结点。非空二叉树有 $n_0=n_2+1$，由上一节公式直接得到。高度为 h 时，结点数满足 $h\leq n\leq 2^h-1$；给定 n，最小高度为 $\lceil \log_2(n+1) \rceil$，最大为 n。例如 50 个结点，最小高度 6。

完全二叉树有 $\lfloor n/2 \rfloor$ 个非叶结点、$\lceil n/2 \rceil$ 个叶结点，最多只有一个度为 1 的结点，且它只能有左孩子。普通二叉树不能沿用这些完全性结论。

定理 5.3.1 (二叉树的叶与双孩子结点)

任意非空二叉树中，若 n_i 表示度为 i 的结点数，则 $n_0=n_2+1$。

按结点与按孩子数边

结点总数 $n=n_0+n_1+n_2$，边数一方面为 $n-1$，另一方面为 n_1+2n_2。令两式相等并消去 n_1，即得 $n_0=n_2+1$。证明只需二叉树，不要求完全、满或平衡。

□

5.4 顺序编号与链式表示

不变量·编号表达位置而非秩

顺序存储中下标表达满二叉树的位置，删除缺失位置后不能把剩余元素紧凑前移而继续沿用 $2i, 2i+1$。链表遍历得到的次序也不是这种位置编号。

完全二叉树从 1 开始编号，结点 i 的双亲为 $\lfloor i/2 \rfloor$，左、右孩子为 $2i, 2i+1$，但只有编号不超过 n 的位置才存在。若下标从 0 开始，孩子公式改为 $2i+1, 2i+2$，非根结点的双亲为 $\lfloor (i-1)/2 \rfloor$。

任意二叉树也可以按满二叉树位置编号，但缺失的位置必须留空。因此实际结点少，顺序存储占用的空间未必小。高度 h 的一条全右链，从 1 开始编号时最后编号为 $2^h - 1$，只存 h 个结点却可能需要指数级下标范围。

5.4.1 例 5-2　任意形态的顺序存储需要预留多少

例 5.4.1 (配套案例)

【配套例题】对于任意一棵高度为 5、有 10 个结点的二叉树，采用顺序存储结构保存，每个结点占 1 个存储单元，只存放结点数据信息。存放该二叉树需要的存储单元数量至少为（　）。

A. 31　B. 16　C. 15　D. 10

解析。本题要求容量能够保存这种约束下的任意形态。高度 5 的满二叉树位置编号到 31；合法的 10 结点形态完全可能包含第 31 号位置（保留通往该位置的祖先，再安排其余结点）。因此为保证任意形态可保存，至少预留 31 个位置，答案为 **A**。并非每一棵给定的 10 结点树都实际用到下标 31；不能把 “保证任意形态” 读成 “在所有形态中挑一个最省空间的”。

二叉链表为每个结点保存左、右孩子指针。n 个非空结点共有 $2n$ 个孩子指针域，真正指向孩子的有 $n-1$ 个，所以空域为 $n+1$ 个。若再增加双亲指针，根的双亲域为空，其他 $n-1$ 个双亲域非空；总空域为 $3n - 2(n-1) = n+2$。某个结点被多少指针指向，不只由它自己有几个孩子决定：孩子的双亲域也会指回它。

5.4.2 二叉链表的教材定义

```c
typedef struct BiTNode {
    ElemType data;
    struct BiTNode *lchild, *rchild;
} BiTNode, *BiTree;
```

`BiTree T` 本质上是根结点指针，空树为 `T==NULL`。`T->lchild`、`T->rchild` 是孩子指针，不能把一个 `BiTNode` 对象和指向它的 `BiTree` 混用。下文 `visit(T)`表示处理该结点的数据；具体打印、累计或复制操作由应用决定。

顺序存储中，合法孩子编号还必须对应一个非空位置；对于有空洞的普通二叉树，`2*i<=数组容量`只说明下标未越界，并不保证该孩子实际存在。

5.5 遍历、还原与树形递归

最小反例·先序加后序的歧义

最小反例为 A 与其唯一孩子 B：B 在左或在右，先序都是 AB，后序都是 BA。缺少中序时，序列没有给出这个单孩子的左右位置；重复值还会带来身份歧义。

先序、中序、后序分别把“访问根”的位置放在左、右子树递归的前、中、后；三个次序都先处理左子树再处理右子树。对下图：

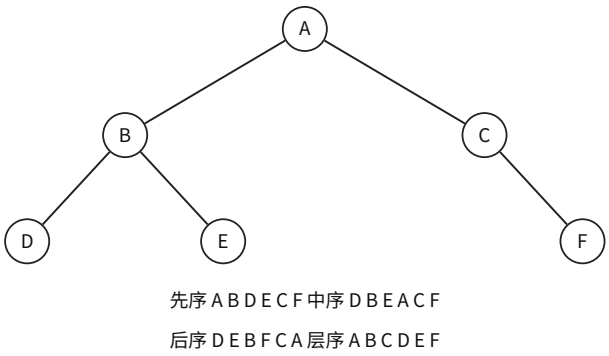


Figure 5.1: 同一棵二叉树的先中后序及层序；空的左孩子不应被当作另一个结点

先序为 `A B D E C F`；中序为 `D B E A C F`；后序为 `D E B F C A`；层序为 `A B C D E F`。先序在深入子树之前处理结点，适合复制或输出结构；后序在左右结果齐备后处理根，适合求子树大小、高度、释放结点。访问位置应服从依赖，不能只因为某种遍历最常见就选它。

```
int TreeDepth(BiTree T) {
    if (T==NULL) return 0;
    int hl=TreeDepth(T->lchild), hr=TreeDepth(T->rchild);
    return 1+(hl>hr ? hl : hr);
}
```

这段代码每个结点只处理一次，时间 $O(n)$；调用栈深为 $O(h)$。求高度时，叶结点返回 1，与本章高度约定一致。交换所有左右子树可以递归地先交换两个子树内部，再交换两个孩子指针，从而把中序序列整体反向；交换不改变根的身份，也不改变哪些结点是叶结点。

课件中的另一棵树增加了 D 的右孩子 G、C 的左孩子 F，可同时检查左右位置、访问时机与输出次序：

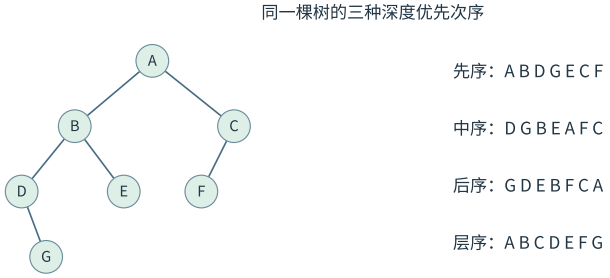


Figure 5.2: 原树与先序、中序、后序、层序的对应序列

5.5.1 递归遍历的三种访问位置

条件辨析·访问位置

先中后序共用左右递归骨架，根的访问位置分别在左递归前、两次递归间、右递归后。依赖子树结果的操作宜在返回后处理。

```
void PreOrder(BiTree T) {
    if (T!=NULL) {
        visit(T);
        PreOrder(T->lchild);
        PreOrder(T->rchild);
    }
}
void InOrder(BiTree T) {
    if (T!=NULL) {
        InOrder(T->lchild);
        visit(T);
        InOrder(T->rchild);
    }
}
void PostOrder(BiTree T) {
    if (T!=NULL) {
        PostOrder(T->lchild);
        PostOrder(T->rchild);
        visit(T);
    }
}
```

把每个递归结点看成有三个停留点：进入左子树前、左子树返回后、右子树返回后。三种遍历仅改变 `visit` 所在停留点，递归下降和返回的骨架相同。时间都是 $\Theta(n)$，最大同时存活的调用数由树高决定，辅助栈为 $O(h)$；单支树可达 $O(n)$，平衡形态为 $O(\log n)$。表达式树的先序、后序分别产生前缀、后缀表达式；中序输出中缀表达式时要根据结构添加括号，才能保留原运算顺序。

5.5.2 先序加中序为什么能还原

假设结点标识互不相同。先序第一个元素为根，在中序中找到根后，左侧全属于左子树，右侧全属于右子树；由左子树的长度，能把先序余段分成左右两段，再递归。上图根为 A，中序左段 `D B E` 长 3，故先序左段是 `B D E`，根 B 又把它分成 D 和 E。

中序加后序也能唯一还原，根取后序最后一个；先序加后序一般不能区分一个单孩子在左还是在右。只给含 n 个不同标识的先序序列，形态数仍为卡特兰数 C_n，例如 4 个元素为 14 种。结点值重复时，“在中序中找到根的位置” 可能不唯一，前述唯一性需要额外身份信息。

5.5.3 由层序和中序还原

结点标识互异时，层序首项是根；中序在根处分开左右子树。把其余层序项按 “是否属于左侧中序集合” 稳定筛分，得到左右子树各自的层序，再递归。

课件例：层序 `A B C D E`、中序 `A C B E D`。根为 A，中序左侧为空，所以 A 没有左子树；右子树层序为 `B C D E`，中序为 `C B E D`。该子树根 B，左子树只有 C；右子树根 D 且左孩子为 E。最终为 `A(空,B(C,D(E,空)))`。由此得到先序 `A B C D E`、后序 `C E D B A`。

课件另一组先序 `D A E F B C H G I`、中序 `E A F D H C B G I`，根 D 将中序分为 `E A F` 与 `H C B G I`；左子树根 A，孩子 E、F；右子树根 B，左子树为 C 且左孩子 H，右子树根 G 且右孩子 I。逐

层分段比仅凭序列外观猜树形可靠。

条件辨析·层序先分根再筛子树

层序中同一子树的相对次序保留，中序负责划分身份集合。不能只按层序剩余部分的长度直接切左右两段。

已知序列，标识互异	一般能否唯一还原	决定子树范围的依据
先序＋中序	能	先序首项定根，中序分左右
后序＋中序	能	后序末项定根，中序分左右
层序＋中序	能	层序首项定根，中序分左右并筛层序
先序＋后序	一般不能	单孩子的左右位置无法区分
先序＋层序、后序＋层序	一般不能	两结点的单左、单右树即为反例

“需要中序” 是一般二叉树、仅按这些遍历组合还原时的结论；增加满二叉树、显式空指针标记等约束后，要按新增信息重新分析。

非递归中序遍历用显式栈保存未完成的祖先：沿左链入栈，到空位置时弹栈访问，再转到其右子树。上图先依次压入 A、B、D，弹出 D 后没有右孩子，接着弹出 B 并转到 E；E 处理完才回到仍在栈中的 A。

```
inorder(root):
    p=root; 建空栈 S
    while p 非空 or S 非空:
        while p 非空:
            push(S,p); p=p->lchild
        p=pop(S); visit(p)
        p=p->rchild
```

非递归后序还需分辨右子树是否已经处理，可保存最近访问结点 `last`。从左子树返回时先看栈顶；它有尚未处理的右子树，就进入右子树，不能立即弹根。

```
postorder(root):
    p=root; last=NULL; 建空栈 S
    while p 非空 or S 非空:
        if p 非空:
            push(S,p); p=p->lchild
        else:
            q=top(S)
            if q->rchild 非空 and last!=q->rchild:
                p=q->rchild
            else:
                visit(q); last=pop(S)
```

`last` 只在一次完整子树处理后更新，因此它等于 `q->rchild`，表示右子树根已经最后访问过。每个结点各入栈、出栈一次，时间仍为 $O(n)$，栈最多同时保存一条祖先路径，空间为 $O(h)$。

层序遍历把根入队，反复出队访问并依次入队左右孩子。判定完全二叉树可在层序遍历中设置 “已经遇到空孩子” 标志：一旦出现空位置，此后的孩子位置不得再出现非空结点。只排除 “有右孩子而无左孩子” 还不够，还要排除遇到首个空孩子位置后又出现非空孩子的情况；一个结点只有左孩子，本身未必违反完全性。

5.5.4 例 5-3　中序前驱和后继在哪一侧

例 5.5.1 (配套案例)

【配套例题】p、q、v 都是二叉树 T 中的结点，v 有两个孩子，T 的中序遍历序列形如 …,p,v,q,…。正确的是（　）。

A. p 没有右孩子，q 没有左孩子　B. p 没有右孩子，q 有左孩子

C. p 有右孩子，q 没有左孩子　D. p 有右孩子，q 有左孩子

解析。v 有左子树，紧邻它之前的 p 必在左子树的最右端；若 p 还有右孩子，应先访问那棵右子树，p 就不可能紧邻 v。对称地，q 是右子树的最左端，不可能有左孩子。答案为 **A**。这里的“最右端”指沿右孩子指针一直走到不能再走的位置，不能按图上哪个结点画得最靠右来判断。

> **遍历问题先固定三个条件**
>
> 先写清访问根的位置、左右孩子顺序、空树处理。还原问题再检查结点身份是否唯一；若给先序与中序，根由先序首项决定，子树边界由中序决定，不能按数值大小猜左右。

层序遍历的队列表示“已发现但尚未处理”的结点。以本章图为例，A 出队后 B、C 入队，接着 B 出队才把 D、E 加入队尾，所以 C 在 D 之前处理。

```
levelOrder(root):
    if root为空: return
    Q入队root
    while Q非空:
        p=Q出队; visit(p)
        if p.lchild非空: Q入队p.lchild
        if p.rchild非空: Q入队p.rchild
```

时间 $O(n)$；队列峰值由树的宽度决定，最坏 $O(n)$。不能把递归遍历的 $O(h)$ 调用栈结论搬到层序队列。

5.6 线索二叉树：区分孩子链接与遍历线索

二叉链表有许多空孩子指针。把空左域改记某遍历次序下的前驱，把空右域改记后继，便得到相应的**线索二叉树**。新增 `ltag`、`rtag` 区分该域是孩子还是线索；本书约定 0 表示孩子，1 表示线索。寻找子树时必须先看标志，不能沿一条线索继续递归，否则可能回到祖先形成循环。

中序线索树中，若结点右域为线索，可直接到中序后继；若右域为孩子，后继是右子树中沿真实左孩子链到达的第一个结点。前驱同理：左域为线索则直接取，否则到左子树最右端。图中每个结点的 L、R 分别表示左、右指针域，其后数字为对应的 `ltag`、`rtag`。E 的 R1 指向 A，C 的 L1 也指向 A；两条线索各有独立的起点和箭头落点。D 的 L1 和 F 的 R1 则分别接空前驱、空后继端点，本图采用无头结点、首尾取空的约定。

错项排除·先看 tag 再沿指针

线索填写后，一条非空指针可能指向祖先或兄弟方向的前后继。若忽略 tag 把它当孩子继续递归，就可能绕回已访问位置；“指针非空”不等价于“存在子树”。

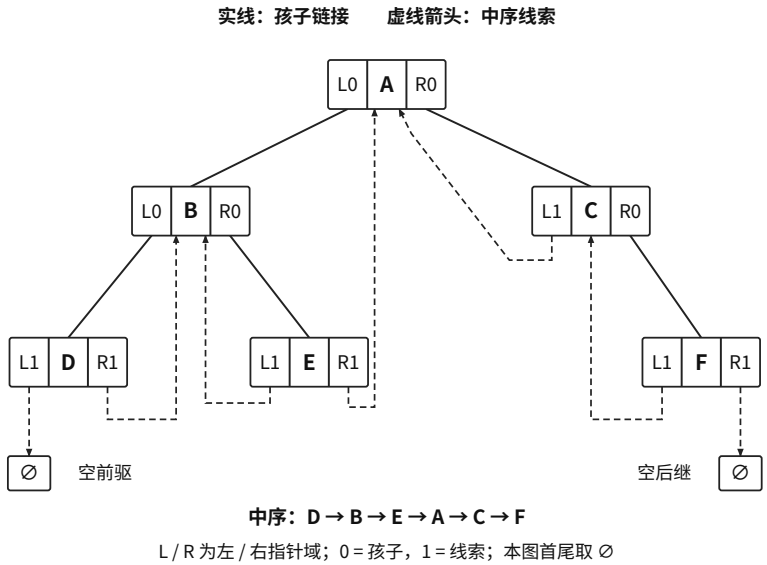


Figure 5.3: 虚线为中序线索，实线为孩子；首尾边界单独标出

$n+1$ 个原空孩子域均可改为线索域，其中首结点的前驱和末结点的后继需约定为空或指向专设头结点。题目说 “线索数为 $n+1$” 时通常统计这些线索域，包含首尾边界；若只统计实际指向普通结点的线索，则边界两项另算。加头结点后，其额外指针也不应混入原树的 $2n$ 域计数。

构造中序线索时，用 `pre` 保存刚访问过的结点，初值为空。处理当前结点 p 时，若其左域为空，填入 pre 并设置左线索标志；若 pre 的右域为空，填入 p 并设置右线索标志，随后令 pre=p。递归只沿真实孩子执行，避免把刚填的线索又当作子树。

错项排除·先看 tag 再沿指针

线索填写后，一条非空指针可能指向祖先或兄弟方向的前后继。若忽略 tag 把它当孩子继续递归，就可能绕回已访问位置；“指针非空” 不等价于 “存在子树”。

```c
typedef struct ThreadNode {
    ElemType data;
    struct ThreadNode *lchild, *rchild;
    int ltag, rtag;  // 0：孩子；1：线索
} ThreadNode, *ThreadTree;

void InThread(ThreadTree p, ThreadTree &pre) {
    if (p!=NULL) {
        InThread(p->lchild, pre);
        if (p->lchild==NULL) {
            p->lchild=pre;
            p->ltag=1;
        }
        if (pre!=NULL && pre->rchild==NULL) {
            pre->rchild=p;
            pre->rtag=1;
        }
        pre=p;
        InThread(p->rchild, pre);
    }
}
void CreateInThread(ThreadTree T) {
    ThreadTree pre=NULL;
    if (T!=NULL) {
        InThread(T, pre);
        pre->rchild=NULL;
        pre->rtag=1;
    }
}
```

条件辨析·中序末结点的右域

中序最后访问的是最右结点，它的右孩子必为空；后序最后访问的是根，根却可能有右孩子。收尾语句适用的遍历不能混用。

调用前所有原孩子标志为 0。图中先访问 D，其空左域指向空前驱；随后访问 B，D 的空右域才补为指向 B 的线索。访问 E 时填左线索指向 B，访问 A 时再把 E 的空右域补为 A。遍历结束后，pre 为最后结点 F，将其空右域标为后继线索，线索目标按约定为空或为头结点。若使用头结点，首结点空前驱也应统一改指头结点。

先序、后序线索的前驱后继依照各自次序，不可沿用中序规则。后序中，若叶结点 X 存在左兄弟 Y，则 X 是双亲的右孩子；左子树和右子树都处理完后才访问双亲，因此 X 的后序后继是双亲，而不是左兄弟。一般后序线索树若无双亲信息，不一定能像中序那样方便地只凭孩子和线索完成全部后继访问。

5.6.1 线索树的访问算法与六种前驱后继

`pre` 以引用传入，使左子树递归更新后的 “刚访问结点” 能够传回当前层；若按值传指针，调用返回后当前层仍拿着旧值。教材中序线索化只对原始、标志均为 0 的二叉链表调用。右递归开始前，当前结点原本非空的右孩子未被改写；因此该版本可直接递归右孩子。再次对已线索化的树调用时，必须按标志辨别真实孩子。

```
ThreadNode *Firstnode(ThreadNode *p) {
    while (p->ltag==0) p=p->lchild;
    return p;
}
ThreadNode *Nextnode(ThreadNode *p) {
    if (p->rtag==0) return Firstnode(p->rchild);
    return p->rchild;
}
void InOrderThread(ThreadNode *T) {
    if (T==NULL) return;
    for (ThreadNode *p=Firstnode(T); p!=NULL; p=Nextnode(p))
        visit(p);
}
```

条件辨析·中序末结点的右域

中序最后访问的是最右结点，它的右孩子必为空；后序最后访问的是根，根却可能有右孩子。收尾语句适用的遍历不能混用。

`Firstnode` 的前提是参数非空、所有空左域均已正确标成线索；`Nextnode` 接收一个实际结点。这样无需递归栈即可中序遍历。找一次后继最坏可能沿一条左链，整趟遍历每条孩子链接只走常数次，故总时间 $O(n)$、额外空间 $O(1)$。

线索类型	找前驱	找后继
中序	左域为线索则直接取；否则左子树最右结点	右域为线索则直接取；否则右子树最左结点
先序	左域为线索则直接取；有左孩子时通常还需双亲信息	有左孩子先取左孩子，否则取右域（孩子或线索）
后序	有右孩子先取右孩子，否则取左域（孩子或线索）	右域为线索则直接取；有右孩子时通常还需双亲信息

先序有左孩子的结点，其前驱在子树之外；后序有右孩子的结点，其后继在子树之外，因此普通二叉线索链表未必能直接取得。若有双亲域：先序中，左孩子的前驱是双亲；右孩子无左兄弟时前驱是双亲，有左兄弟时为左兄弟子树最后一个先序结点。后序对称：右孩子的后继是双亲，左孩子无右兄弟时后继是双亲，有右兄弟时为右兄弟子树第一个后序结点。

先序线索化在访问根后才递归，刚建立的左线索可能指回祖先，故递归左侧必须检查 `ltag==0`。后序先完成两棵原子树再建线索，注意结束后末结点是根，根未必原本没有右孩子；处理收尾时应判断空右域，不能照搬中序 “最后结点右孩子必空” 的条件。

5.7 普通树的三种存储

双亲表示法为每个结点保存双亲的下标，根用 -1 等特殊值标识。找双亲 $O(1)$，找全部孩子通常需扫描整个数组。它适合从结点反复向根追踪，却不便于列举孩子。

孩子表示法为每个结点保存孩子链表，便于依次访问所有孩子；反查双亲时若没有另外保存双亲信息，仍需搜索。**孩子兄弟表示法**只保存两个链接：`firstChild` 指向第一个孩子，`nextSibling` 指向下一个兄弟。它能用两个指针表示孩子数不定的树，却不能把 `nextSibling` 理解成普通树的孩子。

选择表示时，应先看主要方向：查双亲、枚举孩子或遍历同层兄弟，分别依赖不同的链接。多保存一个双亲域会增加空间，但上溯不必

再搜索，代价降低。

不变量·转换不改变数据结点

孩子兄弟转换保留全部数据结点，但改变链接解释。原树根无兄弟，所以转换后二叉根无右孩子；森林的多个根由右链串起，不能把右链误认成原树的后代。

5.7.1 三种树存储的教材结构

```c
#define MAX_TREE_SIZE 100
typedef struct {
    ElemType data;
    int parent;
} PTNode;
typedef struct {
    PTNode nodes[MAX_TREE_SIZE];
    int n;
} PTree;

typedef struct CSNode {
    ElemType data;
    struct CSNode *firstchild, *nextsibling;
} CSNode, *CSTree;
```

双亲数组中根的 `parent=-1`，其余值为双亲在数组中的下标，不能把它当作元素数据。孩子表示法则在顺序顶点表的每个元素中保存孩子链首指针，链结点保存孩子下标和下一孩子指针；它是顺序与链式存储的结合。森林的双亲表示可有多个 `parent=-1`，孩子表示则须保存各根的位置。孩子兄弟表示中，森林各根沿 `nextsibling` 相连。

5.8 树、森林怎样转换为二叉树

采用“左孩子、右兄弟”约定：普通树第一个孩子成为二叉树左孩子，后续兄弟依次以右孩子相连。原树的根没有兄弟，故其对应二叉树的根没有右孩子。森林的各棵树根互为兄弟，再沿右链连接，第一棵树的根成为二叉树的根。

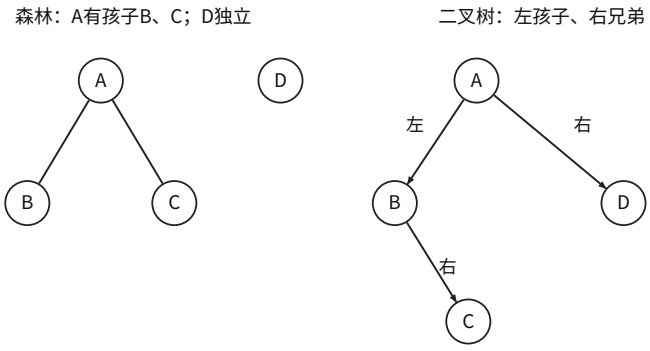


Figure 5.4: 普通树的孩子关系转成左孩子，兄弟关系转成右孩子

逆转换时，从某结点的左孩子出发沿右链走，沿途结点就是它的全部孩子；根及其右链上的各结点分别作为森林各棵树的根。转换改变了“左右链接的解释”，没有增加或删除数据结点。森林有 4 棵树、第一棵含 a 个结点时，二叉树根的左子树只含第一棵树除根外的 $a-1$ 个结点，其他树由根的右子树表示。

5.8.1 例 5-4　二叉树隔两条链接，在森林中是什么关系

例 5.8.1 (配套案例)

【配套例题】将森林转换成对应二叉树。若二叉树中 u 是 v 的父结点的父结点，则原森林中 u 与 v 可能的关系是（　）。

I. 父子关系　II. 兄弟关系　III. u 的父结点与 v 的父结点是兄弟关系

A. 只有 II　B. I 和 II　C. I 和 III　D. I、II 和 III

解析。从 u 到 v 恰经过两条二叉树链接。左—右表示 “先到 u 的第一个孩子，再到其下一个兄弟”，v 仍是 u 的孩子，I 可能；右—右表示连续两个兄弟链接，u、v 互为兄弟，II 可能。左—左表示祖孙，右—左表示 u 的兄弟的孩子，它们都不能产生 III 所说的父结点关系。因此答案为 **B**。应按每条链接的语义推回森林，不能把二叉树的双亲直接当作原树的双亲。

课程连接·编码和外排使用同一棵树

哈夫曼叶权可表示字符频数，也可表示外部归并段长。每次参加合并使该叶再负担一次权值，故总合并长度等于 WPL；第 8 章的最佳归并由此得到。

5.9 树与森林的遍历怎样对应

普通树的先根遍历先访问根，再依次先根遍历各子树；后根遍历先按顺序遍历各子树，再访问根。采用孩子兄弟转换后，树的先根次序对应二叉树先序，树的后根次序对应二叉树中序。原因是二叉树左链接深入第一个孩子，而右链接串联后续兄弟。

森林先序依次先根遍历各棵树，对应二叉树先序；森林中序先按中序处理第一棵树的子树森林，再访问该树根，最后处理其余树，对应二叉树中序。森林遍历要按定义递归到各棵树，并非在整片森林开头统一访问一次根。

层次遍历普通树时，出队一个结点后，按孩子链的次序把它的所有孩子入队；兄弟链接只用于枚举孩子，不能把整条兄弟链当成下一层的结点序列。

5.10 哈夫曼树、WPL 与前缀编码

给每个叶结点一个权值 w_i，根到它的路径长度（边数）为 d_i，**带权路径长度**为 $\mathrm{WPL}=\sum_i w_i d_i$。权值通常为频数或概率，内部结点不直接加入这项 “叶权乘深度” 的求和。求给定叶权下最小 WPL 的二叉树，可每次选当前两个最小权值合并为新结点，再把权值和放回候选集合，直到仅剩根。

为什么合并两个最小值：在一棵最优树中，可以让最深的一对兄弟叶承载两个最小权值而不增大 WPL；把它们收缩成一个权值为和的叶后，剩余结构仍应最优，否则替换后能得到 WPL 更小的树。每次合并都使这两棵子树中所有叶的深度增加 1，因此合并权值之和也等于最终 WPL。

5.10.1 例 5-5　加权平均码长怎样从合并过程得到

例 5.10.1 (配套案例)

【配套例题】6 个字符出现频次分别为 3、4、5、6、8、10，为它们构造哈夫曼编码，加权平均长度为（　）。

A. 2.4　　B. 2.5　　C. 2.67　　D. 2.75

错项排除·最优不意味着唯一

左右子树交换不会改变叶深和 WPL；同权候选的选择也可产生不同树。应排除 “不同编码必有不同平均码长”，同时避免把哈夫曼最优性扩大成完全或平衡。

解析。每轮重新从当前候选中取最小的两个，不能把原始序列分成相邻对后一次配完。

本轮合并	新权值	合并后候选权值
3 与 4	7	5、6、7、8、10
5 与 6	11	7、8、10、11
7 与 8	15	10、11、15
10 与 11	21	15、21
15 与 21	36	36

WPL 为 $7+11+15+21+36=90$，频次总和 36，所以平均码长为 $90/36=2.5$ 位/字符，答案为 **B**。不能把总码长除以 6，因为各字符出现频次不同。

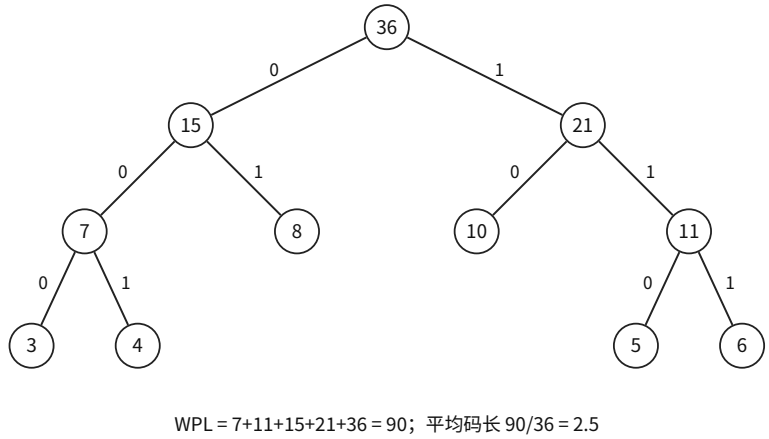


Figure 5.5: 同一合并过程构造的哈夫曼树；左右分支的 0、1 可对换

沿根到叶的 0、1 路径形成码字。字符只放在叶上，所以任何字符码都不是另一个字符码的前缀，这称为**前缀编码**。编码可因相等权值的选取、左右分支交换而不唯一，WPL 仍可相同。哈夫曼树没有度为 1 的内部结点，若有 n 个叶，内部结点 $n-1$、总结点 $2n-1$；它未必完全、未必平衡，n 个叶的最大高度可达 n。

5.10.2 王道课件中的合并与压缩实例

权值为 `1,3,2,2,7` 时，取相等权值的一种合法顺序：先合并 1 与 2 得到 3，再将另一个 2 与新 3 合为 5，原 3 与 5 合为 8，最后 7 与 8 合为 15。WPL 等于全部合并权值之和：$3+5+8+15=31$。权相等时可以换选，形态和码字可能变化，最优 WPL 保持 31。

再看课件的 100 题答案：C、A、B、D 分别出现 80、10、8、2 次。固定 2 位编码共需 200 bit；哈夫曼合并为 $2+8=10$、$10+10=20$、$20+80=100$，WPL 为 $10+20+100=130$ bit。按图中左 0 右 1，可取

C 为 0、A 为 10、D 为 110、B 为 111，平均码长 $130/100 = 1.3$ bit/题。与固定 2 位相比，编码数据长度为原来的 65%，节省 35%；这里暂未计入传递码表等附加信息。

课程连接·编码和外排使用同一棵树

哈夫曼叶权可表示字符频数，也可表示外部归并段长。每次参加合并使该叶再负担一次权值，故总合并长度等于 WPL；第 8 章的最佳归并由此得到。

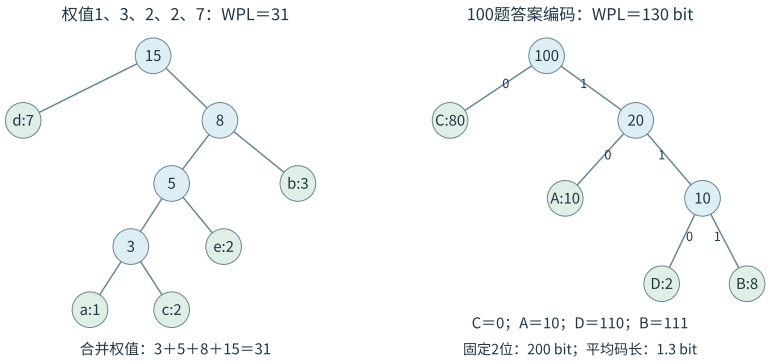


Figure 5.6: 两组王道课件数据的哈夫曼树、合并权值与码字

例如 `CAAABD` 编码为 `0|10|10|10|111|110`。沿树由根读取比特，到叶即输出一个字符再回根，就能解出原串；每个字符在叶上，因此完整码字不会是另一完整码字的前缀。

5.10.3 例 5-6　已占用的码字怎样限制剩余容量

例 5.10.2 (配套案例)

〖5.5-004〗哈夫曼编码长度不超过 4，已有两个字符编码为 1 和 01，还最多能对几个字符编码?

A. 2　B. 3　C. 4　D. 5

解析。以 1 开头的整棵子树已被码字 1 占用，以 01 开头的子树也被占用。剩余码字只能位于 00 开头的子树。长度至多 4，00 之后最多还放 2 位，最多形成 `0000`、`0001`、`0010`、`0011` 四个叶，答案为 **C**。使用较短的剩余码会占用其后代位置，不会增加字符总数。

推广到 m 叉最优归并/编码时，每次合并最小的 m 个权值。若叶数不满足满 m 叉树的条件 $(n-1) \bmod (m-1) = 0$，可补最少的零权虚叶。补齐后的叶数记 n'，内部结点数为 $(n'-1)/(m-1) = \lceil (n-1)/(m-1) \rceil$；不能直接把二叉的 “内部结点 $n-1$” 照搬。这里内部结点都按 m 叉合并计算，零权虚叶不代表额外字符。第 8 章最佳归并树会再次使用这项结构关系。

把哈夫曼的贪心选择写成可重复步骤

候选森林初始为各带权叶；重复取两个最小权根，创建权值为两者之和的新根，并将它放回候选集合。每轮候选数减少 1，故 n 个符号需 $n-1$ 轮。优先队列实现总时间 $O(n \log n)$，树与候选存储 $O(n)$；单符号编码的码长需另作约定。

```
huffman(weights):
    Q=以权值为键的小根优先队列
    while size(Q)>1:
        a=extractMin(Q); b=extractMin(Q)
        z=new Node(weight=a.weight+b.weight,left=a,right=b)
        insert(Q,z)
    return Q唯一根（若原输入为空则返回空）
```

不变量·压缩路径不压缩集合

路径压缩把沿途结点直接挂根，保留每个元素最终代表及集合划分。根的负规模只在合并两个不同根时累加；重复合并同一集合不能再增加规模。

5.11 并查集：用根结点代表一个集合

并查集维护若干互不相交的集合，主要支持两个操作：`find(x)` 查 x 所在集合的代表元，`union(x,y)` 合并两个集合。它不要求维护集合内部元素的顺序。用数组 `parent` 表示一片双亲森林，根存负数表示集合规模，非根存双亲下标。

例如初始 6 个单元素集合：`parent=[-1,-1,-1,-1,-1,-1]`。合并 0 与 1 后可记 `parent[0]=-2,parent[1]=0`；再合并 2 与 3 得 `parent[2]=-2,parent[3]=2`；把这两集合合并时，把根 2 接到根 0，根 0 规模记 -4。查找 3 时沿 $3\to2\to0$，不能把直接双亲 2 误当作最终代表。

按规模合并让较小树根接到较大树根，避免不断形成长链；**路径压缩**在查找后把沿途结点直接连到根。路径压缩不改变集合划分，只改变表示。两项同时使用时，连续操作的摊还复杂度极低，可用反阿克曼函数表示；单次查找仍须按当时实际路径执行，不能声称每次严格只访问一个结点。

```
void Initial(int S[], int n) {
    for (int i=0; i<n; ++i) S[i]=-1;
}
int Find(int S[], int x) {
    while (S[x]>=0) x=S[x];
    return x;
}
void Union(int S[], int Root1, int Root2) {
    if (Root1==Root2) return;
    S[Root2]=Root1;
}
```

这里的 `Union` 参数是两个**根下标**，调用前应先 `Find`。基础版本中根的负值只作根标志，合并时不维护规模。若使用按规模优化，则根负值必须始终等于负的结点总数，两种实现不能交叉调用。

```cpp
void UnionBySize(int S[], int Root1, int Root2) {
    if (Root1==Root2) return;
    if (S[Root2]>S[Root1]) {
        S[Root1]+=S[Root2];
        S[Root2]=Root1;
    } else {
        S[Root2]+=S[Root1];
        S[Root1]=Root2;
    }
}
int FindCompress(int S[], int x) {
    int root=x;
    while (S[root]>=0) root=S[root];
    while (x!=root) {
        int t=S[x];
        S[x]=root;
        x=t;
    }
    return root;
}
```

不变量·压缩路径不压缩集合

路径压缩把沿途结点直接挂根，保留每个元素最终代表及集合划分。根的负规模只在合并两个不同根时累加；重复合并同一集合不能再增加规模。

按规模合并使较小树结点每上升一层，其所属集合规模至少加倍，树高至多 $\lfloor \log_2 n \rfloor + 1$。路径压缩和按规模合并同时使用时，$m$ 次操作的总开销可写为 $O(m\alpha(n))$，这是摊还界；不能把它改说成任意一棵当前树高度严格等于 $\alpha(n)$。

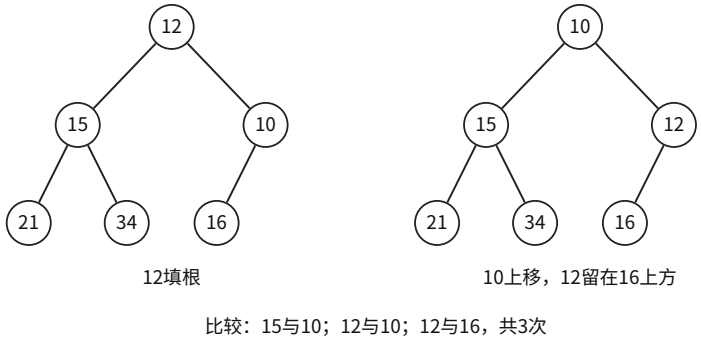
Kruskal 选边时，若两个端点代表元相同，加边会在现有森林中形成环；代表元不同才合并。这里代表的是当前已选边连成的分量，不是原图预先固定的一个编号。

5.12 堆的结构、调整与优先队列

二叉堆同时满足完全二叉树的形状和堆序。大根堆中每个双亲关键字不小于孩子，小根堆中不大于孩子。于是根保存全局最大值或最小值；兄弟间没有规定，整个数组也不必有序。堆中沿根到叶的关键字单调，二叉搜索树则按左右子树区间有序，二者不是同一个约束。

本书堆用下标从 1 开始的数组。小根堆的最大值一定可以在叶中找到，叶下标为 $\lfloor n/2 \rfloor + 1 \sim n$；不能确定是哪一个叶。非根结点若比所有后代都小，仍不保证它比别的子树中的元素小。

插入时先放到数组末尾，保证完全性，再与双亲比较并向上调整。删除根时，用末元素填根、缩小堆范围，再从根向下调整：选择较优孩子，与待调整的值比较，必要时交换并继续。每次最多跨越树高，时间 $O(\log n)$。原有左右子树必须已是堆，下滤才只需沿一条路径。



错项排除·堆序没有兄弟顺序

小根堆的根最小，两个孩子之间却没有规定。删除根时必须先选更优孩子，再和候选比较；只因左孩子更早出现在数组，就总往左走，会破坏右侧堆序。

Figure 5.7: 小根堆删根后，用末元素 12 填根，再与更小孩子 10 比较并下滤

建堆从最后一个非叶结点 $\lfloor n/2 \rfloor$ 开始逆序下滤。叶子本身已是堆，靠下的内部结点移动距离短；按各高度结点数求和，建堆总时间为 $O(n)$，不是把每个结点都按最坏树高估成 $O(n \log n)$。逐个插入也能建堆，但对应另一种过程和复杂度上界。第 8 章将用同一调整规则执行完整堆排序。

5.12.1 堆下滤的代码与不变式

以下大根堆采用 1 起始数组。调用前，左右子树各自满足堆序，只可能根位置过小；每轮把较大孩子提升，尚未解决的缺口沿一条链向下移动。

```
void HeadAdjust(int A[], int k, int len) {
    A[0]=A[k];
    for (int i=2*k; i<=len; i*=2) {
        if (i<len && A[i]<A[i+1]) ++i;
        if (A[0]>=A[i]) break;
        A[k]=A[i]; k=i;
    }
    A[k]=A[0];
}
```

向下每轮至少下降一层，最坏 $O(\log n)$，辅助 $O(1)$。只比较候选与左孩子会漏掉更大的右孩子；`i<len` 是检查右孩子确实存在。空堆不应调用此函数，外部循环从 $n/2$ 开始可自然跳过 $n = 0, 1$ 的情况。

结构	约束保存什么	操作与判断边界
普通二叉树	左右位置有别	遍历还原需序列与身份条件
完全二叉树	编号占据前 n 个位置	适合数组；只排除右孩子孤立还不够
二叉堆	完全形状及双亲孩子间堆序	根为最值；兄弟不保证有序
BST（第 7 章）	全部左右子树的关键字区间	中序有序；不保证完全或平衡
哈夫曼树	叶权下的 WPL 最小	通常不是完全树；编码可不唯一

5.13 主动回忆自测

以下为自拟补充，针对表示与前置条件。

1. 一棵二叉树有 8 个度为 2 的结点、3 个度为 1 的结点，共多少结点？它是否一定为完全二叉树？
2. 一棵二叉树的先序和中序完全相同，结点值互异，所有非叶结点的孩子位置有什么限制？
3. 并查集中 `parent[4]=2,parent[2]=0,parent[0]=-5`，执行带路径压缩的 `find(4)` 后，哪些关系改变？
4. 关键字互不相同的大根堆中，第二大元素一定在哪些可能位置？
5. 为什么一棵 n 结点二叉链表有 $n+1$ 个空孩子域？加上双亲域后怎样变化？
6. 普通树的后根遍历转换后对应二叉树哪一种遍历？为什么不是后序？
7. 权值 3、4、5 合并构造哈夫曼树，WPL 是多少？编码一定唯一吗？
8. 层序遍历与递归深度优先遍历分别由什么结构保存未处理状态？最坏辅助空间是否一定相同？

答案。

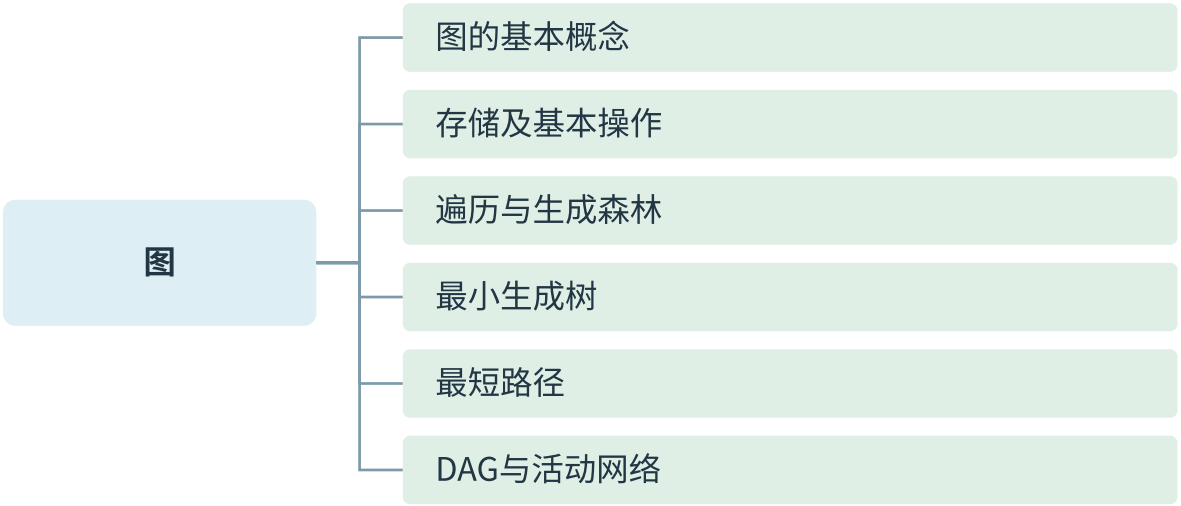
1. 由二叉树的性质 $n_0 = n_2 + 1$，得叶结点数 $n_0 = 8 + 1 = 9$，所以总结点数为 $9 + 3 + 8 = 20$。这棵树**不是完全二叉树**：完全二叉树至多有 1 个度为 1 的结点，而题中有 3 个，不满足这一必要条件。
2. 只能有右孩子；若存在左子树，中序会先于根访问它，与先序矛盾。
3. 返回 0，`parent[4]` 改为 0，2 仍指向 0，根规模不变。
4. 根的孩子中；若某更深结点是第二大，其双亲严格大于它，除根外还会出现一个更大的结点，产生矛盾。存在相同值时，须先明确第二大是否按不同值计数，不能原样沿用结论。
5. $2n$ 个孩子域中 $n-1$ 个非空，余 $n+1$；再加 n 个双亲域，除根外 $n-1$ 个非空，总空域 $n+2$。
6. 对应中序：左链递入孩子森林、右链枚举兄弟，全部孩子完成才访问本根；右孩子在转换语义中不是原树子女，不能按原二叉后序套用。
7. 先 3+4=7，再 5+7=12，WPL 为 7+12=19；左右 0/1 交换可改变编码而不改变 WPL。
8. 前者用队列、后者用调用栈；队列与宽度有关，调用栈与高度有关，形态不同可相差很大。

错项排除·堆序没有兄弟顺序

小根堆的根最小，两个孩子之间却没有规定。删除根时必须先选更优孩子，再和候选比较；只因左孩子更早出现在数组，就总往左走，会破坏右侧堆序。

6 图

章节脉络



图的题目先辨认边的含义：一条边可以表示可通行的道路、需要铺设的线路，也可以表示任务之间的先后约束。最短路径、最小生成树、拓扑排序和关键路径分别优化或检查不同对象；选择算法前先把题目要求写成图上的条件。

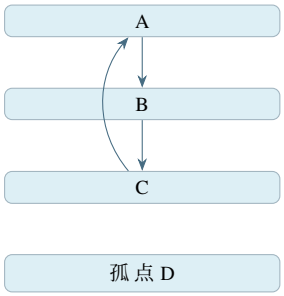
6.1 本章知识结构

- **图的基本概念**：有向无向与带权；度入度出度；路径回路与距离；连通和强连通分量；生成树森林与完全图。
- **存储及基本操作**：邻接矩阵；邻接表及逆邻接表；十字链表；邻接多重表；增删顶点与关联边。
- **遍历与生成森林**：DFS 的回退条件；BFS 的发现与队列；访问标记设置时机；遍历时间与辅助空间。
- **最小生成树**：总边权目标与适用图；Prim 的跨割状态；Kruskal 的分量合并；同权边与唯一性。
- **最短路径**：BFS 同权最短路；Dijkstra 非负权与松弛；Floyd 中转阶段；负边负环及不可达。
- **DAG 与活动网络**：共享子表达式；拓扑排序及唯一性；AOV 和 AOE 语义；事件最早最迟时间；活动余量与多条关键路径。

先明确图的方向、边权和存储表示，再跟踪访问标志、候选集合和路径信息。相同的图可以承载不同任务，判断算法时应始终回到它维护的状态与目标。

最小反例·n 减 1 条边不够判树

三点成环再加一个孤立点，共 4 顶点 3 边，恰有 $n-1$ 条边却不连通且有环。判树还要加连通或无环条件，不能只由数量推出结构。



4 顶点 3 边：三角形加孤点，满足 m=n−1 却不是树。

6.2 顶点、边、连通性与路径

课程连接·网络拓扑只是图的一种模型

计算机网络可把路由器视为顶点、链路视为边，但边权可能表示时延、费用或带宽的变换。先明确权的含义再选择算法；同一个物理网络可以对应不同优化问题。

定义 6.2.1 (图与本章模型)

图记为 $G=(V,E)$，V 为顶点集合，E 为边集合。无向边用无序顶点对表示，有向边（弧）用有序对表示。除另行说明，本章讨论无自环、无平行边的简单图，顶点数 $n=|V|>0$。无向简单图最多有 $n(n-1)/2$ 条边；有向简单图最多有 $n(n-1)$ 条弧。带权图的边还关联权值，权可以表示距离、费用或时间。

无向图顶点的度为关联边数，所有顶点度数和为 $2|E|$；有向图分别计入度、出度，各自总和都为 $|E|$。无向图度数为奇数的顶点个数必为偶数。路径长度若未加权，通常计边条数；带权长度计沿途权值和。简单路径不重复顶点，简单回路除首尾相同外不重复其他顶点。

无向图任意两顶点都有路径时称为连通图，其极大连通子图为连通分量；有向图中每对顶点相互可达时为强连通，极大强连通子图为强连通分量。单向可达不等于强连通。“极大” 指不能再加入原图其他顶点而保持相应性质，不是要求分量含全图最多的顶点。

连通无向图的**生成树**含全部顶点，连通且无环，因此有 $n-1$ 条边。它是极小连通子图：再删一边即不连通。生成树一般不是原图的连通分量，因为连通分量要保留该连通块原有的全部边，不能删到只剩一棵树。非连通图的每个分量各取一棵生成树，得到生成森林。

6.2.1 例 6-1 “有可能连通” 与 “保证连通” 不同

例 6.2.1 (配套案例)

【配套例题】无向图有 7 个顶点，要保证图在任何情况下都连通，需要的边数最少是（ ）。

A. 6　B. 15　C. 16　D. 21

解析。6 条边可以组成一棵生成树，但也可以都集中在部分顶点上，不能保证连通。若图不连通，边数最多的分法是 6 个顶点组成完全图，剩 1 个孤立顶点，共 $\binom{6}{2}=15$ 条边。超过 15 条就不可能仍不连通，故最少 16 条，答案为 **C**。

一般的 n 顶点简单无向图，边数大于 $(n-1)(n-2)/2$ 可保证连通；边数至少 $n-1$ 只是连通的必要条件。若图有 c 个连通分量，则每个分量至少要 “顶点数减 1” 条边，所以 $|E|\geq n-c$。若 $|E|<n-1$，必不连通；若 $|E|=n-1$，还需检查是否无环或连通，才能认定为树。

一张 n 顶点、n 条边的无向图必有环，因为无环森林最多有 $n-1$ 条边。但图有环并不保证连通，也不保证只有一个环。求给定边数下最多的连通分量数时，应让边尽量集中：例如 51 个顶点、21 条边，可在 7 个顶点上组成完全图，其余 44 个孤立，共 45 个分量；少于 7 个顶点最多 15 条边，无法容纳 21 条。

“完全图” 是边达到上限的图；“连通” 只要求路径存在。稠密与稀疏描述边数相对规模，具体选结构还要结合访问方式。对于 $n>1$ 的强连通有向图，每个顶点入度、出度至少 1，所以至少 n 条弧，有向环可达到下界；$n=1$ 时零弧图也可按零长度路径视为强连通，不能忽略这个边界。

有向树常指恰有一个入度为 0 的根、其余顶点入度为 1、根沿弧可达全部顶点的有向无环图。仅仅有 $n-1$ 条弧还不够。图中两点的距离是最短路径长；不可达时记为无穷，不把 “没有路径” 误记为距离 0。

6.3 邻接矩阵、邻接表与基本操作

错项排除·零权边不是没有边

带权邻接矩阵用无穷表示无边时，零是合法边权。若仍以 if(A[u][v]) 判断有边，就会漏掉零权边；AOV 的 0/1 邻接矩阵与这种表示的语义不同。

邻接矩阵 `A[u][v]` 表示是否存在 $u \to v$；带权时元素存放权值，用 $+\infty$ 表示不存在，不能把 0 既用作 “无边” 的标记，又用作零权边的权值。无向图矩阵对称，有向图不必。矩阵占 $\Theta(n^2)$ 空间，检查一条给定边为 $O(1)$，枚举某顶点邻边通常要扫整行 $O(n)$。

邻接表为每个顶点保存其邻接点链。无向图每条边通常出现两个表结点，有向图每条弧在出边表出现一次，空间均为 $\Theta(n+|E|)$。枚举邻边只访问实际表项，判断一条边是否存在则可能扫描相应链。逆邻接表以终点组织入边，方向与普通出边表相反。

下图同时用于说明存储表示与遍历。无向边为 `ab`、`ac`、`ae`、`be`、`cf`、`df`、`de`，各邻接表按字母升序排列。

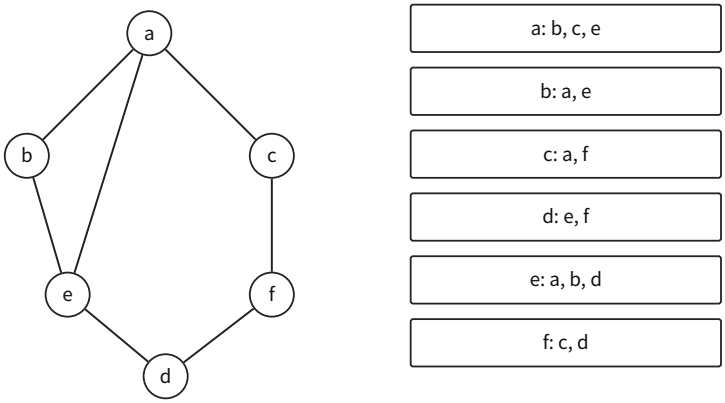


Figure 6.1: 本章遍历图；实线表示无向边，左侧表保存每个顶点的邻接点

对该图，a 行有 b、c、e 三个非零项，b 行有 a、e 两个。无向图中，顶点的度等于该行非零项的个数；有向图中，行的非零项数给出出度，列的非零项数给出入度。求有向图所有顶点的入度时，矩阵要扫描 n^2 个位置，邻接表只需扫描所有表项，成本不同。

删除顶点会同时删除其所有关联边。无向邻接表不能只删除该顶点自己的链，还要清除其他顶点中指向它的表项；有向图则须同时考虑入边和出边。题目若给出 “只按出度计删除成本” 的实现，应检查它是否另有入边索引。

6.3.1 顶点表、边表与基本操作接口

下面保留教材的矩阵和邻接表命名。`VertexType` 为顶点信息类型，`EdgeType` 为边权类型；实际使用前要根据问题定义它们。算法中的顶点编号是数组下标 `0..vexnum-1`，图上显示的 V_1, V_2 等名称可以另存于顶点信息中。

```c
#define MaxVertexNum 100
typedef struct {
    VertexType Vex[MaxVertexNum];
    EdgeType Edge[MaxVertexNum][MaxVertexNum];
    int vexnum, arcnum;
} MGraph;

typedef struct ArcNode {
    int adjvex;
    struct ArcNode *next;
    // 带权图在此补充权值域
} ArcNode;
typedef struct VNode {
    VertexType data;
    ArcNode *first;
} VNode, AdjList[MaxVertexNum];
typedef struct {
    AdjList vertices;
    int vexnum, arcnum;
} ALGraph;
```

条件辨析·编号与顶点信息

算法使用的 0 起始编号是数组下标。图上 V1 可以对应下标 0，也可以另存信息；转换一次后整章保持一致。

`arcnum` 记录图中的实际边或弧条数；无向邻接表即使有 $2|E|$ 个边表结点，也仍记录 $|E|$，不能按表结点总数填入。矩阵的 0/1 表示与带权表示应分别初始化：带权对角线在最短路径初始化中通常取 0，无边的非对角位置取无穷。

教材用下列接口屏蔽具体存储：

- `Adjacent(G,x,y)`判断边存在，`Neighbors(G,x)`枚举邻边。
- `InsertVertex`、`DeleteVertex` 增删顶点；`AddEdge`、`RemoveEdge` 增删边。
- `Get_edge_value`、`Set_edge_value` 读写指定边的权值。
- `FirstNeighbor(G,x)`求第一个邻点；`NextNeighbor(G,x,y)`求邻点 y 之后的下一个邻点，无对应邻点时返回 -1。

以下对这些接口采用同一方向约定：有向图的邻点指当前顶点的出弧所到达的顶点。

操作	邻接矩阵	普通出边邻接表
判断 $x \to y$ 是否存在	$O(1)$	扫描 x 的出边链
枚举 x 的全部出邻点	$O(\|V\|)$	$O(OD(x))$
求某点出度	扫一行，$O(\|V\|)$	数该链结点，$O(OD(x))$
求某点入度	扫一列，$O(\|V\|)$	扫全体出边表，$O(\|V\|+\|E\|)$
删除顶点	删除关联行列并处理编号	清除出链与所有指向它的入弧

有向图若另建逆邻接表或十字链表，求入度和删除入弧的代价可随之改变。无向邻接表查度数直接数对应链即可。

无向图矩阵对称，可只保存上三角或下三角，再按对称关系恢复另半。若连同对角线保存，需要 $n(n+1)/2$ 个单元；简单图的对角线已知全为 0、另作约定时，可只存严格三角的 $n(n-1)/2$ 项。压缩节省的是常数比例，空间阶仍为 $O(n^2)$；定位公式须与保存方向和 0 ／ 1 起始下标一致。

6.3.2 矩阵幂数的是游走

设 A 是非加权图的 0/1 邻接矩阵，按普通整数矩阵乘法计算，则 $(A^k)_{ij}$ 等于从 i 到 j、恰好经过 k 条边的**游走数**，允许重复顶点和边。这来自乘法的逐中间点求和：$(A^2)_{ij}=\sum_t A_{it}A_{tj}$，每个非零乘积对应一条 $i\to t\to j$。

例如只有无向边 $1-2$ 时，$(A^2)_{11}=1$，对应 $1\to 2\to 1$；它不是从 1 回到 1 的简单路径。题目若把这里的“路径”用作可重复的广义路径，应按这一含义理解。布尔矩阵乘法得到是否存在，普通算术乘法得到条数，二者也应区别。

不变量·一条边结点参与两条链

邻接多重表让同一个无向边结点同时属于两个端点链。沿当前端点前进时选择其对应链接；这解释了为何可统一标记边，也解释了不能始终沿 ilink。

6.4 十字链表与邻接多重表

十字链表面向有向图，每条弧只建立一个弧结点，记录弧尾、弧头以及两个链接：一个沿相同弧尾串联出弧，一个沿相同弧头串联入弧。每个顶点分别保存第一条出弧、第一条入弧。这样同一条弧可从源顶点的出边链或终点的入边链到达，无需存两份弧数据。

邻接多重表面向无向图，每条边只建一个边结点，记录两个端点 `ivex,jvex`，并有 `ilink,jlink`，分别串入两个端点的关联边链。沿顶点 u 的链前进时，须先判断 u 是这条边的哪个端点，再选择对应链接；不能始终只走 `ilink`。这种表示便于给一条边设置统一访问标记、执行删除等边操作。

普通邻接表为每条无向边保存两个表项；邻接多重表中的一个边结点同时挂在两条链上。数边时若直接数链表项，两种结构得到的数目会相差常数倍。稀疏矩阵的十字链表也使用“两种方向共享一个数据结点”的思想，但它的两维是行与列，不是有向图的弧尾与弧头。

6.5 深度优先搜索与搜索森林

DFS 从一个顶点开始，标记并访问，再递归访问尚未访问的邻接点；当前顶点已没有未访问邻点时才回退。标记用于防止沿环重复访问。遍历整张图时，还须依次检查所有顶点，对仍未访问的顶点重新启动搜索；一趟 DFS 未必到达全图。

```
bool visited[MaxVertexNum];
void DFS(Graph G, int v) {
    visit(v);
    visited[v]=true;
    for (int w=FirstNeighbor(G,v); w>=0;
         w=NextNeighbor(G,v,w))
        if (!visited[w]) DFS(G,w);
}
void DFSTraverse(Graph G) {
    for (int v=0; v<G.vexnum; ++v) visited[v]=false;
    for (int v=0; v<G.vexnum; ++v)
        if (!visited[v]) DFS(G,v);
}
```

上图按字母顺序枚举邻点，从 a 开始，得到 `a,b,e,d,f,c`。首次到达新顶点的边形成 DFS 生成树；无向图的搜索树棵数等于连通分量数，有向图不能用树棵数直接认定强连通分量数。

6.5.1 例 6-2　什么时候允许回退去另一个分支

例 6.5.1 (配套案例)

〖6.3-011〗无向图顶点为 `{a,b,c,d,e,f}`，边为 `{ab,ae,ac,be,cf,fd,ed}`。进行深度优先遍历，不能得到的序列为（　）。

A. acfdeb　B. aebdfc　C. aedfcb　D. abecdf

解析。D 访问到 `a,b,e` 时，e 的邻点 d 尚未访问，必须深入 d，不能先回退到 a 转去 c，因此 D 不可能。A 可沿 a→c→f→d→e→b；B 可先走 a→e→b，b 无未访问邻点后回到 e，再走 d→f→c；C 可先走 a→e→d→f→c，回退后再处理 b。答案为 **D**。输出序列中相邻的两个顶点不一定直接相连，因为中间可能发生回退；判断时真正要看的是，回退发生时当前顶点是否还有未探索的邻点。

采用邻接表时，每个顶点和表项只处理常数次，时间 $O(n+|E|)$；矩阵要对每个访问顶点扫一整行，时间 $O(n^2)$。递归栈最坏 $O(n)$，另有访问标志数组 $O(n)$。如果改为在递归返回前才输出顶点，得到的是完成次序；对 DAG，该完成次序的逆序就是拓扑序列，第 8 节会说明原因。

最小反例·边最少而权更大

三点 A、B、C，AB 与 BC 权 1，AC 权 3。BFS 从 A 一跳发现 C，少一条边却总权 3；经 B 两跳总权 2。因此 BFS 的分层证明不能覆盖不等权路径目标。

6.5.2 遍历序列、存储次序与生成森林

`Graph` 在遍历代码中表示已经实现上述邻点接口的图类型；选用矩阵或邻接表后，由对应接口完成枚举。只有**起始点、外层顶点扫描顺序、邻接点枚举顺序**都固定，遍历序列才固定。对编号固定的邻接矩阵，常规逐列递增扫描会给出一种确定顺序；若改变扫描方向，同一矩阵也可得到另一序列。

每个新顶点第一次被发现时，记录其前驱，便得到搜索生成树。无向图中一棵树覆盖一个连通分量，若有 c 个分量，森林共 $|V|-c$ 条树边。非树边仍属于原图，不能因它未出现在搜索树中就从图里删除。有向图的搜索树覆盖的是这次沿出弧可达且尚未访问的部分，外层启动次数受起点次序影响，不能拿来计算强连通分量。

DFS 的递归深度、BFS 的队列长度都可能达到 $O(|V|)$；把访问标志数组算入辅助空间时，完整遍历至少还需 $O(|V|)$ 标志空间。即便某个 DFS 实例的调用栈只有常数深度，也不能把整个实现的辅助空间一并写为 $O(1)$。

6.6 BFS 为什么得到无权最短路

BFS 用队列保存已发现、尚未扩展的顶点。起点先标记并入队；每次取出队头，将其尚未发现的邻点标记、记录距离并入队。标记应在**入队时**设置，若等到出队才标记，同一顶点可能被不同前驱重复入队。

```
void BFS(Graph G, int v) {
    SqQueue Q;
    InitQueue(Q);
    visit(v);
    visited[v]=true;
    EnQueue(Q,v);
    while (!QueueEmpty(Q)) {
        DeQueue(Q,v);
        for (int w=FirstNeighbor(G,v); w>=0;
             w=NextNeighbor(G,v,w)) {
            if (!visited[w]) {
                visit(w);
                visited[w]=true;
                EnQueue(Q,w);
            }
        }
    }
}
void BFSTraverse(Graph G) {
    for (int v=0; v<G.vexnum; ++v) visited[v]=false;
    for (int v=0; v<G.vexnum; ++v)
        if (!visited[v]) BFS(G,v);
}
```

条件辨析·三个顺序共同确定遍历

起始顶点、外层扫描次序、邻点枚举次序都固定，DFS 或 BFS 才有固定输出。矩阵存储本身没有禁止逆序扫描。

上面的队列元素为顶点编号，容量要能容纳全部可能同时入队的顶点。若采用第 3 章牺牲一个单元的循环队列，物理数组须比允许队列长度多 1。

求从源点 u 出发的无权最短路径时,初始化全部 `d[i]=INF`、`path[i]=-1`，令 `d[u]=0`。在发现边 $v \to w$ 且 `visited[w]==false` 的分支中增加:

```
d[w]=d[v]+1;
path[w]=v;
```

每个顶点只在首次发现时赋距离。`path[w]`是其路径前驱，可从目标反向追到源点；最后把这条逆序链倒过来。源点 `path[u]=-1` 是终止标志，不表示源点不可达。

BFS 首次发现即最短

队列保证按距离分层出队：距离 0 的顶点先于距离 1 的，距离 1 的又先于距离 2 的。首次发现 v 时，若还存在更短的路径，这条路径上 v 的前驱应位于更早的一层，早已被扩展，v 也就应更早被发现，矛盾。因此 `dist` 记录的是最少边数。

□

上图从 a 出发、邻点按字母枚举，出队访问序列为 `a,b,c,e,f,d`；b、c、e 距离 1，f、d 距离 2。该序列与 DFS 顺序不同，但也不唯一：同层邻接次序可改变，最少边数不变。带权不等长边时，少经过一条边未必总权更小，不能把这条证明直接推广为带权最短路。

邻接表与矩阵的时间同 DFS，队列和标记等辅助空间为 $O(n)$。有向图中，BFS 只沿弧的方向扩展；“无权最短路” 并不要求图无向，但要求每条边的代价相同，只按边数计。

6.6.1 王道课件的八顶点最短路径实例

下图中的顶点名称为 1～8，以 2 为源点，邻点按名称递增枚举。源点先发现 1 和 6；随后 1 发现 5，6 发现 3 和 7；再由 3 发现 4、由 7 发现 8。BFS 生成树保留每个非源点的首次发现前驱，其根到各点的路径具有最少边数。

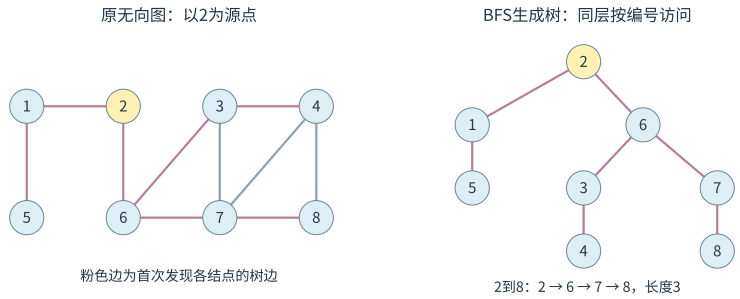


Figure 6.2: 八顶点原图与以 2 为根的广度优先生成树

顶点名称	1	2	3	4	5	6	7	8
距离 d	1	0	2	3	2	1	2	3
前驱 path	2	−1	6	3	1	2	6	7

从目标 8 倒推前驱，得到 $8 \leftarrow 7 \leftarrow 6 \leftarrow 2$，反转即为最短路径 $2o6o7o8$，长度 3。表中使用图上的 1 起始名称，写入前述 0 起始图接口时，应把名称映射为数组下标；源点的前驱 −1 保留为终止标记。

把题意翻译成图算法的目标

先确定有向还是无向、边权是否同一常数，再问目标是“全部顶点以最低总价接通”“某个起点到各点最短”“先后约束合法”还是“工程完成时间”。BFS 只保证同权边最短，MST 只控制整棵树总权；它们不能互相替代。

6.7 Prim 与 Kruskal 在挑什么边

最小生成树（MST）要求连通全部顶点、无环，并使所选 $n-1$ 条边权和最小。它不要求从某个起点到每个顶点的路径最短。若图不连通，只能得到每个分量的最小生成森林。

Prim 维护一个已连接顶点集合 U，每次从“一端在 U、一端在外”的边中选最小边，加入新顶点。Kruskal 则把全部边按权非降序考虑，端点在不同当前分量时选入并合并，同一分量则跳过以避免成环。二者都利用割上的轻边性质，但状态不同：Prim 始终扩展一棵树，Kruskal 可能同时有多棵。

6.7.1 例 6-3　同权边怎样影响第二次选边

例 6.7.1 (配套案例)

【配套例题】求下图最小生成树，可能为 Kruskal 第 2 次选中、但不可能为从 V_4 开始的 Prim 第 2 次选中的边是（　）。

错项排除·同权不等于 MST 必不唯一

若所有边权互异，MST 一定唯一；逆命题不成立。一个本来就是树的连通图，即使边权相同，其生成树仍只有它本身，不能根据同权直接判不唯一。

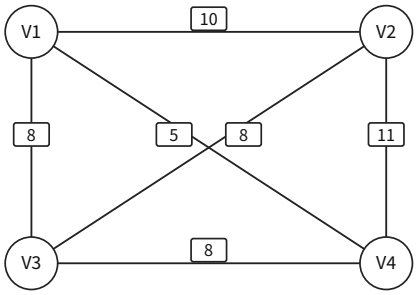


Figure 6.3: 四顶点带权无向图；三条权为 8 的边形成选择分支

A. (V_1, V_3)　B. (V_1, V_4)　C. (V_2, V_3)　D. (V_3, V_4)

解析。两个算法首先都选权 5 的 (V_1, V_4)。Kruskal 此后可选任意一条不成环的权 8 边，包括 (V_2, V_3)。Prim 此时 U 为 $\{V_1, V_4\}$，第 2 条边必须跨越这个集合的边界，只能在 (V_1, V_3) 与 (V_4, V_3) 中选权 8 的边；(V_2, V_3) 两端都在集合外，不能选。答案为 **C**。

继续执行时，Kruskal 若选 (V_2, V_3)，最后还要用一条权 8 边把两个分量连接，总权 21；Prim 选入 V3 后，再用 (V_3, V_2) 权 8 连 V2，也是 21。同权边可能导致不同的 MST；全部边权互异足以保证 MST 唯一，但存在同权边时 MST 仍可能唯一。

朴素 Prim 配矩阵为 $O(n^2)$；邻接表配专门维护候选最小值的结构可降低稀疏图中选最小边的代价。Kruskal 主要花在边排序，常写 $O(|E| \log |E|)$，加并查集维护分量。具体选哪一个，还要看图的表示与选边的实现，不能只比较算法名字。

6.7.2 最小生成树的核心伪码

```
Prim(s):
    U={s}; lowCost[v]=w(s,v)，无边记无穷
    重复直到选入全部顶点:
        u=U外lowCost最小者
        若lowCost[u]=无穷: 原图不连通，停止
        选入parent[u]到u的边; U加入u
        对每个U外邻点v:
            若w(u,v)<lowCost[v]: 更新lowCost[v]及parent[v]
```

这里 `lowCost[v]`始终是 U 到 v 的最轻跨割边，**不是**从 s 到 v 的路径长度，所以更新式没有 `lowCost[u]+w(u,v)`。

```
Kruskal(G):
    将边按权非降排序；每个顶点初始化为单独集合
    对每条边(u,v,w):
        若find(u)!=find(v):
            选入此边; unite(u,v)
    若选入边数<n-1: 得到最小生成森林而非生成树
```

为何跨割最轻边可以安全加入

不变量·Prim 的 key 不是距离

Prim 只问一条跨割边接新点的代价，key 更新取边权；Dijkstra 问源点到新点的完整路径，需加已确定前缀。把两者公式混写可能算出另一种对象。

设已有选择能扩展到某棵 MST，考虑与现有分量相容的割及其一条轻边 e。若某 MST 不含 e，加入 e 形成环，环上必有另一条跨该割的边 f，且 $w(e) \le w(f)$。删 f 保连通且不增权，得到含 e 的 MST；这说明选择可扩展，不表示所有 MST 都必须含 e。

□

错项排除·有负边与一定算错不同

负边意味着标准 Dijkstra 失去通用正确性保证，不代表每个含负边实例都会得错值。选项若用“只要有一条负边就必算错”的绝对语气，可用简单单路径反驳。

6.8 Dijkstra 与 Floyd 的条件和中间状态

Dijkstra 求单源最短路径。维护已确定集合 S 与暂定距离 `dist[v]`；每次选择 S 外距离最小的顶点 u，确定它的最短距离，再用 `dist[u]+w(u,v)` 松弛其邻点。尚未确定的 `dist` 是已知可达路径的最好上界，不一定已是真正最短距离。

通用正确性要求边权非负：若能先经过一个当前距离更远的顶点、再经负边绕回 u，u 的真实距离就可能小于已确定的值，于是 u 被过早确定。出现负边不表示每个实例都必然算错，但标准算法不能保证正确。不可达顶点的距离保持无穷，所有剩余距离都为无穷时可停止，不能再把无穷当作普通数参与加法。

6.8.1 例 6-4　求出第二条最短路径后哪些值还可变

例 6.8.1 (配套案例)

【配套例题】使用 Dijkstra 求下图从顶点 1 到其余顶点的最短路径，`dist` 依次保存到 2、3、4、5 的距离。求出第二条最短路径后，`dist` 内容为（　）。

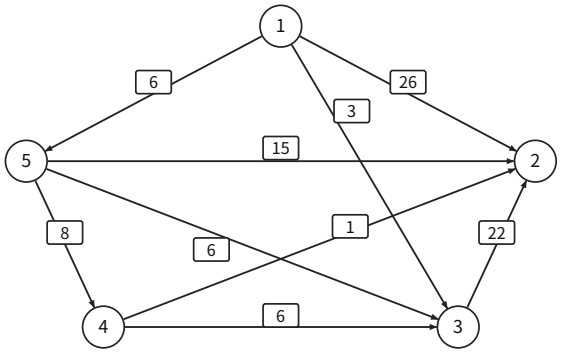


Figure 6.4: 原题有向边与权值；箭头方向参与松弛条件

A. 26、3、14、6　B. 25、3、14、6　C. 21、3、14、6　D. 15、3、14、6

解析。先把起点 1 确定，初始 `dist=(26,3,∞,6)`。第 1 条到其他顶点的确定路径是 1→3，长 3；用 3→2 的 22 更新到 2 距离为 25。随后选顶点 5，距离 6，第 2 条路径确定；由 5→2 的 15 把到 2 更新为 21，由 5→4 的 8 把到 4 更新为 14，得到 **(21,3,14,6)，答案 C**。

已确定的新顶点	暂定距离向量 (dist[2],dist[3],dist[4],dist[5])
1（初始化）	(26,3,∞,6)
3	(25,3,∞,6)
5	(21,3,14,6)
4	(15,3,14,6)
2	(15,3,14,6)

不变量·已确定与暂定要分开

Dijkstra 中暂定 dist 只是一条已知路径的最好上界。选出最小暂定点后才永久确定；过程题若提前使用未来松弛的值，就把最终答案错误放入了中间状态。

答案要求的是第 2 条路径确定后，不是算法最终结果。顶点 4 随后才用其到 2 的权 1 把 21 改为 15，不能把这个将来更新提前放进答案。

Floyd 求任意顶点对之间的最短路。令 $D^{(k)}[i,j]$ 表示只允许编号不超过 k 的顶点作为内部中转时的最短距离，则

$$D^{(k)}[i,j]=\min\{D^{(k-1)}[i,j],D^{(k-1)}[i,k]+D^{(k-1)}[k,j]\}.$$

两种候选分别是不经过 k，或在 k 处分成两段。实现时中转顶点 k 必须放在最外层循环，才能对应这个阶段的含义。时间 $O(n^3)$，距离矩阵 $O(n^2)$；允许负边，但要求不存在影响所求路径的负权回路，否则不存在有限最短值。无向图中一条负权边就能往返构成负权闭合游走，不能直接套用通常的有限最短路模型。

矩阵版 Dijkstra 单源时间 $O(n^2)$；对每个源点重跑得到全源需 $O(n^3)$，不能仍写成 $O(n^2)$。BFS 求的是同权边下的最少边数，MST 最小化整棵树的边权总和，Floyd/Dijkstra 最小化指定端点之间的路径权和；三个目标要分别辨认。

6.8.2 最短路径的阶段循环

```
Dijkstra(s):
    dist[s]=0; 其余为无穷; S为空
    重复选择S外dist最小的u:
        若dist[u]=无穷: 停止
        S加入u
        对每条u到v，若v未确定且dist[u]+w(u,v)<dist[v]:
            更新dist[v]; parent[v]=u
```

最外层每次永久确定一个顶点，这是非负权前提的用处；已经确定的点不能靠未来负边回头改写。实现中先检查 `dist[u]`有限，避免无穷哨兵加权溢出。

```
Floyd(D):
    D[i][i]=0; 其他位置按直接边初始化
    for k=0..n-1:
        for i=0..n-1:
            for j=0..n-1:
                若D[i][k]与D[k][j]均有限:
                    D[i][j]=min(D[i][j],D[i][k]+D[k][j])
```

k 在外层时，“已允许的中间顶点集合” 才依次扩大。若结束后某 `D[v][v]<0`，说明存在负环；受该负环影响的点对没有有限最短距离，不能继续解释该数值为答案。

6.8.3 Floyd 三阶段计算实例

条件辨析·path 有两种语义

Floyd 的 path[i][j] 记录中转点；BFS 或 Dijkstra 的一维 path[v] 通常记录直接前驱。重建路径时分别递归拆分或反向追溯。

课件的三顶点有向图，按 V_0, V_1, V_2 排列，直接边为 $V_0 \to V_1 = 6$、$V_0 \to V_2 = 13$、$V_1 \to V_0 = 10$、$V_1 \to V_2 = 4$、$V_2 \to V_0 = 5$。无边记 ∞，对角线为 0：

$$A^{(-1)} = \begin{pmatrix} 0 & 6 & 13 \\ 10 & 0 & 4 \\ 5 & \infty & 0 \end{pmatrix}.$$

允许 V_0 中转后，$2 \to 1$ 由 ∞ 更新为 $5 + 6 = 11$；允许 V_1 中转后，$0 \to 2$ 由 13 更新为 $6 + 4 = 10$；允许 V_2 中转后，$1 \to 0$ 由 10 更新为 $4 + 5 = 9$：

$$A^{(0)} = \begin{pmatrix} 0 & 6 & 13 \\ 10 & 0 & 4 \\ 5 & 11 & 0 \end{pmatrix},$$

$$A^{(1)} = \begin{pmatrix} 0 & 6 & 10 \\ 10 & 0 & 4 \\ 5 & 11 & 0 \end{pmatrix},$$

$$A^{(2)} = \begin{pmatrix} 0 & 6 & 10 \\ 9 & 0 & 4 \\ 5 & 11 & 0 \end{pmatrix}.$$

中转记录初值 `path[i][j]=-1`。每次因 k 而更新距离，就令 `path[i][j]=k`；例如最终 `path[0][2]=1`，说明 0 到 2 应拆为 0 到 1、1 到 2。若子段还有中转点，再递归拆分。这个矩阵保存**中间顶点**，与 Dijkstra 或 BFS 保存 “直接前驱” 的一维 `path` 含义不同。

6.8.4 DAG 怎样共享子表达式

表达式树为每次运算保留一份子树；DAG 可以让多个父运算共享相同子表达式。对 `(a*b)*(a*b)*(a*b)*c`，先为操作数 a, b, c 各建一个顶点，建立一次 $t_1 = a * b$，再按左结合次序建立 $t_2 = t_1 * t_1$、$t_3 = t_2 * t_1$、$t_4 = t_3 * c$。两个操作数相同的乘法也保留左右两个输入位置，即便它们指向同一个 t_1。

构造时先标运算依赖，再合并相同操作数和相同运算的相同输入子图；只有操作符和有序输入都相同，才可直接共享。`a-b` 和 `b-a` 不能合并；`a/(b*c)`与 `(a/b)*c` 也不能按视觉相似合并。含赋值、函数调用等副作用时，还须遵守表达式求值语义。

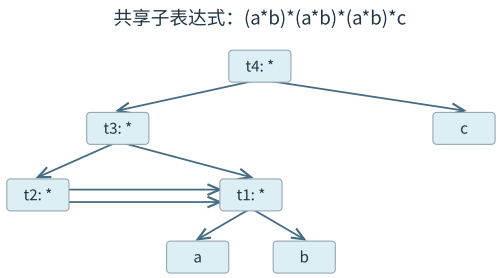


Figure 6.5: 同一个 a*b 结点分别被平方和后续乘法引用，边指向输入子表达式

6.9 拓扑排序：由入度判断哪些任务可以先做

不变量·入度为零才可先执行

入度表示尚未完成的直接前驱数；输出顶点并移除其出弧后，后继入度随之递减。队列为空而顶点尚未全部输出时，图中存在有向环。

有向无环图（DAG）可以用顶点表示活动、弧表示先后约束，称为 AOV 网。拓扑序列要求对每条弧 $u \to v$，u 都排在 v 之前。算法先把所有入度为 0 的顶点作为候选，每次删除一个候选及其出弧，更新后继入度；若处理完的顶点数不足 n，则存在有向环。

有多个候选时可以选择不同的顺序，因此拓扑序列一般不唯一。对 DAG 做 DFS 并在退出时输出，每条弧的终点必先完成，得到的是逆拓扑序列；把该完成序列反向才是拓扑序列。若存在有向环，仅取得一个 DFS 完成次序并不能证明拓扑合法。

6.9.1 例 6-5　怎样判定唯一拓扑序列

例 6.9.1 (配套案例)

【配套例题】载人航天工程包含众多子工程，为保证有序开展，需要明确前导子工程。这可抽象为有向图的拓扑序列问题。已知图 G 采用邻接矩阵存储：

```
typedef struct {
    int numVertices,numEdges;
    char VerticesList[MAXV];
    int Edge[MAXV][MAXV];
} MGraph;
```

设计 `int uniquely(MGraph G)` 判定 G 是否存在唯一拓扑序列，是则返回 1，否则返回 0。给出设计思想及带关键注释的 C 或 C++ 实现。

解析。每一轮的候选零入度顶点必须恰有一个。若图无环且某轮至少有两个，二者都可作为下一项，交换其先后可产生至少两个拓扑序列；若某轮没有候选而尚未处理完，则余图有环，无法完成。这两种情况都不满足唯一，均可返回 0。只有始终唯一且处理完全体顶点，才能返回 1。

```
int uniquely(MGraph G) {
    int n=G.numVertices, indeg[MAXV]={0};
    bool used[MAXV]={false};
    for (int u=0; u<n; ++u)
        for (int v=0; v<n; ++v)
            if (G.Edge[u][v]) ++indeg[v];
    for (int step=0; step<n; ++step) {
        int pick=-1;
        for (int v=0; v<n; ++v)
            if (!used[v] && indeg[v]==0) {
                if (pick!=-1) return 0; // 至少两个候选
                pick=v;
            }
        if (pick==-1) return 0;          // 剩余部分有环
        used[pick]=true;
        for (int v=0; v<n; ++v)
            if (G.Edge[pick][v]) --indeg[v];
    }
    return 1;
}
```

课程连接·同步前驱约束就是偏序

操作系统中多个事件的前驱关系可用 DAG 表示，零入度事件代表当前所有前驱已完成。拓扑排序只给合法先后，不计算耗时；带持续时间的工期还需 AOE 时间传播。

这里 AOV 邻接矩阵用 0/1 分别表示无边与有弧，不能直接把带权矩阵的 “无边为无穷” 搬进这个判断。总时间 $O(n^2)$，工作数组额外占 $O(n)$；题给的按值传参若实际复制整个矩阵，形参副本还占 $O(n^2)$，可按接口许可改为只读引用。若使用邻接表并维护零入度候选数，处理时间可为 $O(n+|E|)$。顶点已经按某个拓扑次序编号时，所有弧才保证由小编号指向大编号，邻接矩阵才呈严格上三角；任意编号的 DAG 不必原样为三角矩阵。

DAG 还可表达共享子表达式：相同子式只存一份，不同上层结点可指向同一子式。它表达计算依赖而非普通树的唯一双亲关系；不能再套用 “除根外每结点恰有一个双亲”。

6.9.2 一般拓扑排序的队列实现

```
topological(G):
    统计所有顶点的入度；把入度0者入队Q
    count=0
    while Q非空:
        u=出队；输出u；count++
        对每条u到v: indeg[v]--
            若indeg[v]==0: v入队
    返回count==n
```

入度表示余图中尚未删除的前驱数，只有从 1 降到 0 时加入候选。若只关心任意拓扑序列，可以任选候选；要判断唯一，则每一步候选数都必须恰为 1，原例的代码正是在增加这个约束。

6.9.3 逆拓扑排序与 DFS 环检测

从出度为 0 的顶点开始，删除顶点及其入弧，得到逆拓扑序列。普通出边邻接表查找入弧不便，可使用逆邻接表维护出度变化；若直接每轮扫描全图，复杂度会改变。DFS 在返回前输出顶点，也能在 DAG 上得到逆拓扑序列。

判断有向环时，把顶点分为未访问、正在递归栈中、已完成三种状态。沿弧到达“正在递归栈中”的顶点，说明该弧与栈上的路径形成有向环；到达已经完成的顶点本身并不表示有环。单一 `visited` 只能防止重复访问，不能替代三色状态判断。无向图的回边判断还需排除通往当前双亲的那条树边。

条件辨析·visited 不足以判有向环

到达已完成结点不说明有环；回到当前递归栈中的结点才形成回边。三色状态区分发现与完成。

6.10 关键路径：事件与活动的时间不同

AOE 网用顶点表示事件、带权有向边表示活动，边权为持续时间。某事件要等全部进入活动都完成才能发生；它发生之后，其出边活动才能开始。以下假设该网无环、持续时间非负，且有统一的源点和汇点；若源点或汇点不止一个，可添加零时长的虚边统一。

事件最早发生时间 `ve[v]` 按拓扑序计算：从所有前驱中取 `ve[u]+w(u,v)` 的**最大值**，因为全部前导活动都得完成。令汇点的最迟发生时间等于总工期，事件的最迟发生时间 `vl[u]` 则按逆拓扑序计算，取所有后继的 `vl[v]-w(u,v)` 的**最小值**，以保证每条后续活动仍赶得上。

活动 $u \to v$ 的最早开始时间为 $e = ve[u]$，最迟开始时间为 $l = vl[v] - w(u,v)$。$l - e$ 是该活动的时间余量；余量 0 的活动为关键活动。关键路径是源到汇的一条最长带权路径，决定最短可完成工期；它不是边最多的路径。

6.10.1 例 6-6　多条关键路径时怎样缩短工期

例 6.10.1 (配套案例)

【配套例题】下图 AOE 网包含 8 个活动。通过同时加快若干活动，可缩短整个工程工期。下列选项中，加快所列活动进度就可缩短工期的是（　）。

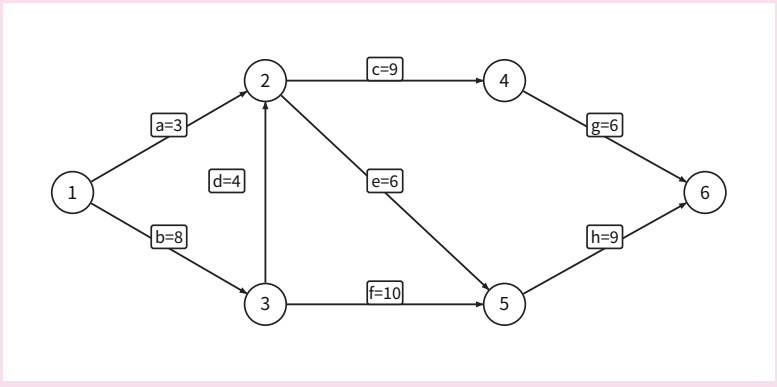


Figure 6.6: 箭头旁给出活动名和持续时间；d 的方向为 3→2

A. c 和 e　　B. d 和 c　　C. f 和 d　　D. f 和 h

解析。先按拓扑次序 1、3、2、4、5、6 算事件时间。

事件	ve	vl
1	0	0
2	12	12
3	8	8
4	21	21
5	18	18
6	27	27

例如 `ve[2]=max(0+3,8+4)=12`，不能仅沿 1→2 算成 3；`ve[5]=max(12+6,8+10)=18`。从汇点反推，`vl[2]=min(21−9,18−6)=12`，同时约束它通往 4 和 5 的两条后续活动。

活动 a 余量为 9，其余 b、c、d、e、f、g、h 均为关键活动。关键路径有三条：`b-d-c-g`、`b-d-e-h`、`b-f-h`，长度均 27。若只缩短其中一条路径上的活动，其他路径仍可能保持 27。f 与 d 合起来覆盖三条关键路径：d 影响前两条，f 影响第三条；两者同时适当缩短，所有原关键路径都会变短。答案为 **C**。A、B 遗漏路径 b-f-h，D 遗漏路径 b-d-c-g。

增加关键活动时长会使包含它的路径变长，进而延长工期；缩短某一关键活动却未必缩短工期，因为还可能有不包含它的其他关键路径。持续缩短还可能使原非关键路径成为新的关键路径，必须重新计算，不能无限按原余量推断。

错项排除·缩短一条关键活动未必够

存在两条等长关键路径时，只缩短其中一条独有活动，另一条仍保持原工期。要检查所有当前关键路径是否都被影响，并在缩短后重新计算新关键路径。

6.10.2 关键路径的计算表怎样落到程序

```
criticalPath(G):
    取得拓扑序T；若不存在则模型非法
    ve[*]=0
    按T正序，对每条u到v：ve[v]=max(ve[v],ve[u]+w(u,v))
    D=ve[统一汇点]；vl[*]=D
    按T逆序，对每条u到v：vl[u]=min(vl[u],vl[v]-w(u,v))
    对每条活动u到v:
        e=ve[u]；l=vl[v]-w(u,v)
        若e==l：该边为关键活动
```

按“全部前驱都完成”取最大最早时间，按“不拖延任何后继”取最小最迟时间，方向不能交换。不能将 `ve[v]`直接当作边 $u \to v$ 的活动最早开始时间，活动开始由弧尾事件 u 决定。

任务	前提与优化对象	核心状态	典型边界
BFS 最短路	同权边；最少边数	分层队列和距离	非等权不能直接用
MST	带权无向图；总树权最小	Prim 跨割值或 Kruskal 分量	不保证源点路径最短
Dijkstra	非负权；单源路径和最小	已确定集合与暂定上界	负边破坏通用保证
Floyd	无相关负环；点对最短	允许的中转点集合	中转点循环在最外层
拓扑	DAG；满足先后约束	余图入度	无候选而未结束表示有环
关键路径	AOE DAG；总工期	事件 ve/vl 及活动 e/l	多条关键路径需同时考虑

6.11 主动回忆自测

以下为自拟补充，检查算法选择的边界。

1. 图有 8 个顶点、7 条边，能否直接认定为树？若再给“连通”呢？

2. 有向图 A→B、A→C、B→D、C→D 有几个拓扑序列？若添加 B→C，怎样变化？
3. 一条边权变大，MST 总权与某一对顶点最短距离是否一定同时改变？
4. 一条关键活动缩短 1，工期是否必缩短 1？
5. 无向图三点 A、B、C，边 AB=1、AC=3、BC=1。从 A 作 BFS 得到的到 C 路径是否必为最短权路径？
6. Prim 的 `key[v]` 与 Dijkstra 的 `dist[v]` 分别保存什么？为什么松弛公式不同？
7. Floyd 把 `k` 放内层，仅执行一次三重循环还能直接沿用原证明吗？
8. DAG 中某轮有两个零入度顶点，能否仍有唯一拓扑序列？

答案。1. 不能；若无向且连通，7 条边恰够生成树，不可能再有环，故为树。2. 原有 A-B-C-D 与 A-C-B-D 两个；添加后仅 A-B-C-D。3. 不一定；该边可能不在任何当前采用的最优解中，两种优化目标也不同。4. 不一定；有另一条不含该活动的同长关键路径时工期不变。

5. 不能；BFS 可一条边直接到 C，权为 3，实际 A-B-C 权为 2。它最少的是边数，不是总权。
6. 前者是连接现有树集合到 v 的最轻边；后者是从固定源点到 v 的已知路径最小上界，后者需累加已确定前缀距离。
7. 不能，阶段已不再对应统一的中转集合；必须重新证明算法，而非只因循环次数相同就认定等价。
8. 不能；在无环余图中这两个候选可分别作为下一项，并可扩展为合法序列，至少产生两种拓扑序。

错项排除·缩短一条关键活动未必够

存在两条等长关键路径时，只缩短其中一条独有活动，另一条仍保持原工期。要检查所有当前关键路径是否都被影响，并在缩短后重新计算新关键路径。

查找 7

章节脉络

查找结构要回答两个问题：怎样缩小候选范围，插入或删除后怎样保持这种缩小范围的能力。顺序查找只依赖逐项比较；折半查找依赖顺序存储且有序；搜索树把有序关系放进分支；散列表则用关键字计算位置。外存索引还要考虑一次读入能取得多少个关键字。

错项排除·先决定 ASL 的事件空间

成功概率分配给已有记录，失败概率可能分配给关键字间隙或初始散列地址。先写清每项概率再计平均；不能看到有效记录有 n 个，就把所有失败长度都除以 n。

7.1 本章知识结构

- **查找对象与代价**：主关键字与次关键字；静态动态查找表；成功失败事件；概率与平均查找长度；比较次数及外存页数。
- **线性表上的查找**：无序及有序顺序查找；哨兵与失败边界；分块索引与块长；折半查找的闭区间；判定树与失败外部位置。
- **二叉搜索与平衡**：BST 全部子树区间；查找插入删除；AVL 平衡因子；LLRRLRRL 旋转；红黑性质及黑高；红黑插入删除修复。
- **多路外存索引**：m 阶 B 树容量；分裂借位合并；根升降与叶等深；B 树高度界；B+ 树索引及叶链；点查范围与读盘次数。
- **散列查找**：散列函数构造；冲突及同义词；链地址法；开放定址探查序列；删除标记及重复判断；装填因子与 ASL。
- **结构选择的条件**：有序与随机定位；动态更新及维护；平衡性与最坏保证；外存页分支数；等值与范围检索。

按“候选范围怎样缩小、失败怎样判定、更新怎样维护结构”三条线阅读。ASL 计算始终先写清事件集合、概率和计数口径，再执行实际查找路径。

7.2 关键字、成功失败与平均查找长度

定义 7.2.1 (查找表、关键字与动态性)

查找表是同一类型记录的集合，**关键字**是用于识别记录的数据项。主关键字唯一标识记录，次关键字可能对应多条记录。只进行查找、不改变记录集合的表称为静态查找表；还需要插入、删除记录的表为动态查找表，不能仅凭 “采用数组” 判断是静态还是动态。

查找长度通常指一次查找中关键字的比较次数，平均查找长度为 $\mathrm{ASL}=\sum_i p_i C_i$，其中 p_i 为某一查找事件的概率，C_i 为该事件的比较次数。成功 ASL 在表内各条记录之间分配概率；失败 ASL 在不存在的关键字可能落入的失败区间或初始散列地址之间分配概率，分母未必是表长。

树形查找中，访问一个含多关键字的 B 树结点可能只触发一次外存读，却在内存中比较多个关键字。因此 “读盘次数”“访问结点数”“关键字比较次数” 不是同一个量，题目要求哪个就统计哪个。

条件辨析·同结构名也要看有效范围

哨兵顺序查找的 SSTable 用 1 至 n，折半查找课件用 0 至 n-1。elem 和 TableLen 含义相同，有效下标范围由具体算法规定。

7.3 顺序查找、分块查找与提前结束

无序表从一端逐项比较，成功时停止，扫描完仍未找到则失败。等概率成功时，平均比较 $(n+1)/2$ 次；失败通常比较 n 次。哨兵可以统一循环终止条件、减少边界判断，但不会改变比较次数的数量级。若计入哨兵自身的关键字比较，次数还须按代码约定加上相应一项。

递增有序表从头查找时，当前值等于目标时应报告成功，当前值**大于**目标才可提前失败；把 “不小于” 一概判为失败会把相等的目标漏掉。若仍逐个向后比较，等概率成功 ASL 与无序表相同，只是失败可能提前结束；有序并不自动意味着每种查找都使用折半。

7.3.1 顺序查找的哨兵实现

```c
typedef struct {
    ElemType *elem;
    int TableLen;
} SSTable;
int Search_Seq(SSTable ST, ElemType key) {
    ST.elem[0]=key;
    int i;
    for (i=ST.TableLen; ST.elem[i]!=key; --i) ;
    return i;
}
```

本段将有效记录放在 `elem[1..TableLen]`，另有可写的 0 号哨兵单元。从后向前查找，命中返回该下标，失败必停在 0 并返回 0。函数虽按值传入结构体，其指针成员仍指向同一数组，所以写哨兵会修改原数组的 0 号单元。空表也必须存在可写哨兵位置。

等概率成功时，比较次数为 $1,2,\ldots,n$，平均 $(n+1)/2$；失败比较 n 个实际记录再比较哨兵，共 $n+1$ 次。若题目只数与实际记录的比较，则最后一次哨兵另算。哨兵节省每轮边界检测，并没有省去线性扫描。

课件的非等概率例，六条记录依次命中概率为 0.15、0.05、0.10、

0.40、0.28、0.02，从前向后查的成功 ASL 为 3.67。按命中概率非增序排列后，为 0.40、0.28、0.15、0.10、0.05、0.02，ASL 降为 2.18。交换相邻两条逆概率记录可使代价降低，故这种排序适合无须维持关键字顺序的静态顺序查找表。

分块查找把表分成若干块，块内可无序，块间有序：前一块的最大关键字小于后一块所有关键字。索引保存每块最大关键字和块起始位置，先定位块，再在块内顺序找。索引查找若分别采用顺序查找与折半查找，效率并不相同。

索引保存块的最大值时，应定位**第一个最大关键字不小于目标**的块。若对索引折半后没有命中相等值，循环结束的 `low` 仍可指向目标所属块；只有 `low` 超过最后一个索引项，才能直接判断目标不在表内。例如块最大值为 `10,20,30`，查 17 应进入最大值 20 对应的块，不是因索引无 17 而失败。

块内有序并非分块查找的必要条件。动态表可以把每块组织成链表以方便更新，但索引最大值和块间范围仍须维护。

不变量·mid 比较后必须排除

闭区间中，相等已处理后，目标只能在 mid 左边或右边。若更新 high=mid 且区间仅两个元素，可能反复取得同一 mid 而不缩小；循环条件与更新式必须配套。

7.3.2 例 7-1　两级顺序查找怎样选择块长

例 7.3.1 (配套案例)

【配套例题】查找表有 400 个元素，查找每个元素的概率相同，采用均匀分块。先顺序查找确定所在块，块内也顺序查找。为使查找效率最高，每块包含元素个数应为（　）。

A. 8　B. 10　C. 20　D. 25

解析。设每块 s 个，共 $b = 400/s$ 块。查成功时，索引平均比较 $(b+1)/2$ 次，块内平均比较 $(s+1)/2$ 次，总计

$$\mathrm{ASL} = \frac{400/s + s}{2} + 1.$$

两项乘积固定，$400/s + s$ 在 $s = \sqrt{400} = 20$ 时最小，答案为 **C**。四个选项均能整除 400，无需处理尾块。若索引改为折半，这个 “平方根块长” 的推导不再对应实际代价。

7.4 折半查找与判定树

折半查找要求按关键字有序且能快速定位中点。本章教材代码在闭区间 `[low,high]` 中查找，取 `mid=(low+high)/2`，向下取整。它按 `elem[0..TableLen-1]`存放有效记录，与上一节哨兵顺序查找的有效范围不同。目标比中点小时令 `high=mid-1`，比中点大时令 `low=mid+1`；中点已经比较过，更新区间时不能把它又留在候选范围内。

```c
int Binary_Search(SSTable L, ElemType key) {
    int low=0, high=L.TableLen-1, mid;
    while (low<=high) {
        mid=(low+high)/2;
        if (L.elem[mid]==key) return mid;
        else if (L.elem[mid]>key) high=mid-1;
        else low=mid+1;
    }
    return -1;
}
```

课程连接·比较少不代表访存少

折半查找在数组上可常数时间定位中点；链表上虽能设计相同的比较决策，走到中点还要访问中间结点。算法分析要把抽象比较与实际存储访问分开。

成功查找的比较次数等于对应内部结点所在层次。失败发生在候选区间为空时，空位置本身通常不做关键字比较，因此沿途比较次数等于经过的内部结点数。取上中点或下中点会改变具体路径，数量级仍是 $O(\log n)$。

对长度 11 的递增表 `7,10,13,16,19,29,32,33,37,41,43`，第一次比较中间项 29，查 33 依次比较 29、37、32、33；查 12 依次比较 29、13、7、10，然后 `low>high` 失败。

判定树以每次比较的中点为内部结点。向下取中点时，偶数长度区间分出的右子树比左子树多 1 项，奇数长度时两侧一样多；因此判定树高度平衡，却未必是完全二叉树。上述 11 项表，成功结点分布为第 1 层 1 个、第 2 层 2 个、第 3 层 4 个、第 4 层 4 个，成功 ASL 为 3。12 个失败区间中 4 个需 3 次比较、8 个需 4 次，失败 ASL 为 $11/3$。失败外部结点所在层次比它实际比较的内部结点数多 1。

若把中点改为向上取整，左右规模关系相反，具体 ASL 和搜索轨迹须按新判定树计算。处理可能很大的整型下标时，`low+(high-low)/2` 可避免求和溢出；本章教学表长取可安全相加的范围。

7.4.1 例 7-2　失败查找的最大比较次数

例 7.4.1 (配套案例)

【配套例题】长度为 16 的顺序表 L 按关键字有序，采用折半查找，查找一个不存在的元素，关键字比较次数最多为（　）。

A. 4　B. 5　C. 6　D. 7

解析。用从 1 开始的位序、向下取中点，查找一个大于全部元素的值：区间依次为 `[1,16]→[9,16]→[13,16]→[15,16]→[16,16]→ 空`，比较位置为 8、12、14、15、16，共 5 次，答案为 **B**。$\log_2 16 = 4$ 不是这里的最大比较次数；还需要最后一次区分第 16 个元素与它右侧的失败区间。

同一约定下，长度 11 时查第 11 个元素，比较位序为 6、9、10、11；长度 10、关键字依次 `b,c,d,e,f,g,q,r,s,t` 时查 b，比较 f、c、b。不能在不同题中改用向上取中点。

有序链表即使能逐步寻找中点，也没有数组的常数时间随机定位能力；普通折半查找的对数比较次数并不自动带来对数总访问时间。无序数组不满足排除半区的条件；静态链表虽然用数组存结点，逻辑顺序也不保证按下标连续。

> **先写查找区间不变式再写边界**
>
> 闭区间折半查找循环开始时，若目标存在，它必在 `[low,`

> `high]`。比较过 `mid` 并排除相等后，下一区间必须去掉 `mid`；每轮区间严格缩小，`low>high` 才判失败。改为半开区间时，终止条件与更新式要一起改，不能从两份模板各抄一半。

7.5 二叉搜索树的区间不变式

本书二叉搜索树（BST）默认关键字互异，左子树所有值小于根，右子树所有值大于根，左右子树分别满足同样性质。只检查根与两个直接孩子还不够；一个较深后代也可能越过祖先设定的区间。中序遍历严格递增是等价的整体判断。

查找从根出发，每次比较后进入相应子树；插入沿相同路径，到空位置新建叶结点。比较次数最多为树高，树可能退化成链，因此普通 BST 最坏为 $O(n)$，不能无条件写 $O(\log n)$。

删除叶结点直接断开双亲链接；只有一个孩子时让孩子接替位置；有两个孩子时，可把中序前驱（左子树最大值）或后继（右子树最小值）移到当前位置，再删除那个原结点。被选的前驱不会有右孩子，后继不会有左孩子，最后归结到前两种情况。若题目要求保留结点地址而非只保留关键字，那么复制数据与整体移接就是两种不同的实现，指针的改法也随之不同。

验证 BST 可以中序扫描并比较相邻值，也可以给每个结点传递允许区间 `(lower,upper)`。左递归把上界收紧到当前值，右递归把下界提高到当前值；这样不会漏掉“孩子合格但孙子越过祖先”的错误。当边界值可能等于整数类型极值时，用更宽类型或显式“有无边界”标记，不要随意把一个合法值当作无穷。

定理 7.5.1 (BST 的中序判据)

关键字互异时，二叉树是递增 BST，当且仅当其中序遍历严格递增。

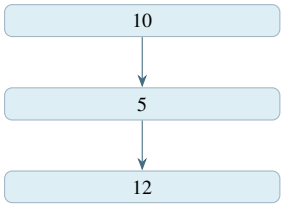
把全子树区间与中序顺序连接

若为 BST，左子树各值小于根、右子树各值大于根，分别递归中序，整体严格递增。反之，中序严格递增使每个结点左子树的全部访问项均小于它，右子树全部项均大于它；递归对子树也成立，故满足 BST 的全局定义。证明涉及全部后代，不能简化为只比直接孩子。

□

最小反例·孩子都合格，孙子越界

根 10，左孩子 5，5 的右孩子 12。每个直接孩子比较都合格，但 12 位于 10 左子树而大于 10，违反全局区间；检验 BST 需中序或上下界递归。



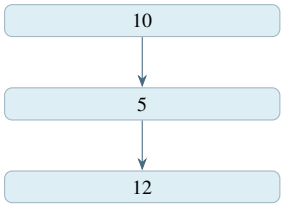
10 的左子树里出现 12：直接父子比较仍可能全对。

7.5.1 BST 查找、插入和删除的基本路径

```c
typedef struct BSTNode {
    int key;
    struct BSTNode *lchild, *rchild;
} BSTNode, *BSTree;
BSTNode *BST_Search(BSTree T, int key) {
    while (T!=NULL && key!=T->key) {
        if (key<T->key) T=T->lchild;
        else T=T->rchild;
    }
    return T;
}
int BST_Insert(BSTree &T, int k) {
    if (T==NULL) {
        T=(BSTree)malloc(sizeof(BSTNode));
        T->key=k;
        T->lchild=T->rchild=NULL;
        return 1;
    } else if (k==T->key) return 0;
    else if (k<T->key) return BST_Insert(T->lchild,k);
    else return BST_Insert(T->rchild,k);
}
void Create_BST(BSTree &T, int str[], int n) {
    T=NULL;
    for (int i=0; i<n; ++i) BST_Insert(T,str[i]);
}
```

最小反例·孩子都合格，孙子越界

根 10，左孩子 5，5 的右孩子 12。每个直接孩子比较都合格，但 12 位于 10 左子树而大于 10，违反全局区间；检验 BST 需中序或上下界递归。



10 的左子树里出现 12：直接父子比较仍可能全对。

删除的递归返回值表示 “删除后这棵子树的新根”，从而统一处理删除整树根与内部结点。

```
erase(t,key):
    if t为空: return 空
    if key<t.key: t.lchild=erase(t.lchild,key)
    else if key>t.key: t.rchild=erase(t.rchild,key)
    else:
        if t.lchild为空: 保存t.rchild，释放t，返回保存值
        if t.rchild为空: 保存t.lchild，释放t，返回保存值
        q=t.rchild；沿q.lchild找到右子树最小结点
        t.key=q.key（记录有附加字段时一并复制）
        t.rchild=erase(t.rchild,q.key)
    return t
```

沿路每层常数工作，查找/插入/删除为 $O(h)$；递归删除额外栈 $O(h)$。这里复制记录后释放后继结点，不保留所有原结点地址；若题目约束结点身份，应改用移接方案。

`BST_Search` 返回实际结点指针或 `NULL`，迭代实现额外空间 $O(1)$；`BST_Insert` 返回 1 表示插入新关键字，返回 0 表示重复而未插入，引用 `T` 使空子树能够接回新结点。这里沿用教材默认内存申请成功的教学前提，实际程序还需处理分配失败。

课件按 `50,66,60,26,21,30,70,68` 构造 BST，根为 50，左子树 26 的孩子为 21、30，右子树 66 的孩子为 60、70，70 的左孩子为 68。按 `50,26,21,30,66,60,70,68` 插入也得到相同树，因为各子树内相对插入次序保持不变。把递增关键字依次插入则可能形成单右链，体现普通 BST 对插入次序的依赖。

7.6 AVL：先定位最小失衡子树

错项排除·删除传播与插入不同

插入修复最小失衡子树后一般恢复原高度，更高祖先不再受本次增高影响。删除后局部修复可能继续降低高度，仍须向上；一处调整并非所有 AVL 更新的共同规则。

AVL 是每个结点左右子树高度差绝对值不超过 1 的 BST。令平衡因子为“左高减右高”，合法值为 −1、0、1。插入后沿查找路径向上更新高度，找到离插入位置最近的失衡祖先 A，再根据新结点相对 A 及其较高孩子的位置分类。

LL（左孩子的左侧增高）对 A 右旋；RR 对 A 左旋；LR 先对 A 的左孩子左旋，再对 A 右旋；RL 先对 A 的右孩子右旋，再对 A 左旋。名称描述失衡路径，不能把 “LL” 误当作 “向左旋”。旋转必须保持中序次序：右旋时原左孩子 B 升为局部根，B 原来的右子树移成 A 的左子树，因为它的值介于 B 与 A 之间。

以下数值树用于观察旋转机制。插入前各树高度均为 3；灰底新结点插入后，最小失衡子树高度升为 4。单旋图同时保留需要转接的中间子树：LL 中 40 由 30 的右子树改接为 50 的左子树，RR 中 40 的转接方向对称。

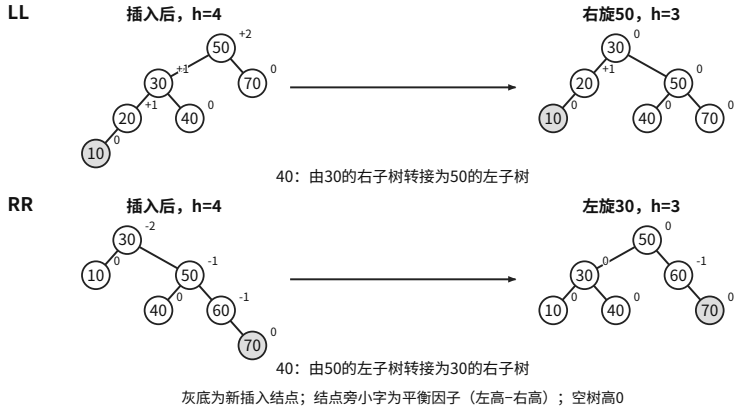


Figure 7.1: LL/RR 各一次旋转；结点旁为平衡因子，非空中间子树随旋转转接

LR 图在 70 的左孩子 30 的右侧插入 40，先左旋 30，再右旋 70；第一步把 40 转接为 30 的右孩子。RL 图在 50 的右孩子 90 的左侧插入 80，先右旋 90，再左旋 50；第一步把 80 转接为 90 的左孩子。**第一步后的树仍可能失衡**，双旋应检查完整两步，不能在中间态停下。四例都保持同一组关键字的升序中序，最终高度恢复为 3，所有平衡因子回到 −1、0、1。

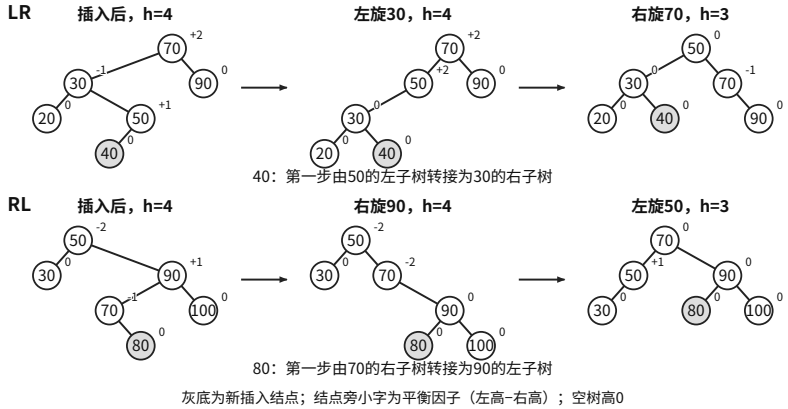


Figure 7.2: LR/RL 依次给出插入后、第一旋后和第二旋后三个状态

7.6.1 例 7-3　连续插入时在哪里旋转

例 7.6.1 (配套案例)

〖7.3-003〗把 34、23、15、98、115、28、107 依次插入初始为空的平衡二叉搜索树，要求每次插入后保持 AVL 性质，给出每步结果。

条件辨析·孩子平衡因子 0

这个分支在删除中常出现，可用一次单旋修复，且局部高度可能保持不变。插入口诀不能代替删除的高度传播判断。

解析。以下用 **根（左子树,右子树）** 表示局部结构，⌀ 表示空。

插入	失衡处与调整	调整后整树
34	无	34
23	无	34(23,∅)
15	34 处 LL，右旋	23(15,34)
98	无	23(15,34(∅,98))
115	34 处 RR，左旋	23(15,98(34,115))
28	23 处 RL，先右旋 98，再左旋 23	34(23(15,28),98(∅,115))
107	98 处 RL，先右旋 115，再左旋 98	34(23(15,28),107(98,115))

插入 28 时，不能只旋转它的双亲 34：34 仍平衡，最先失衡的是 23，新增路径是 “23 的右—98 的左”，故为 RL。每次调整后复查中序始终为已插入关键字的升序，避免只把树形画平衡而破坏搜索关系。

最小失衡子树调整后通常恢复到插入前的高度，更高的祖先不会再因这次插入而失衡。删除则可能使子树高度继续缩短，需要一路向上检查，不能把插入时 “一处修复即可” 的结论直接用于删除。AVL 最小结点数满足 $N(h)=1+N(h-1)+N(h-2)$，由此树高是 $O(\log n)$，查找与更新路径才有对数保证。

固定高度下的形态数则是另一个量。按本书的规定，空树高 0、单根高 1，令 $C(h)$ 为恰高 h 的 AVL 形态数，$C(0)=C(1)=1$；恰高 2 的左右高度只能是 `(1,1)`、`(1,0)`、`(0,1)`，所以 $C(2)=3$。恰高 3 时，左右高度为 `(2,2)`、`(2,1)`、`(1,2)`，故 $C(3)=3^2+2\times 3\times 1=15$，对应〖7.3-013〗。这不等于固定结点数的卡特兰计数，也不等于最小结点数递推。

7.6.2 AVL 删除怎样向上传递

删除先按 BST 规则处理实际被移除的结点，再沿其原祖先路径向上更新高度。失衡结点 A 若左高，考察左孩子 B 的平衡因子；右高情形完全对称。

A 的平衡因子	较高孩子的平衡因子	调整
$+2$	B 为 $+1$ 或 0	对 A 右单旋
$+2$	B 为 -1	先左旋 B，再右旋 A
-2	B 为 -1 或 0	对 A 左单旋
-2	B 为 $+1$	先右旋 B，再左旋 A

孩子平衡因子为 0 是删除中需要单独识别的情形：旋转后局部树高可能保持为删除前的高度，于是上层高度不再改变；其他情形旋转后仍可能比原先矮 1 层，需要继续向上。只看 “LL/RR” 口诀而漏掉孩子为 0，容易选错旋转或错误停止。

最小示例：`4(2(1,3),5)`删除 5 后，根 4 左高 2，左孩子 2 的平衡因子 0。右旋得到 `2(1,4(3,空))`，树高仍为 3，所有结点已平衡。若删除使下层子树真正矮 1 层并影响更高祖先，则应在更高处继续修复。整个查找、删除和沿祖先路径调整的时间为 $O(\log n)$。

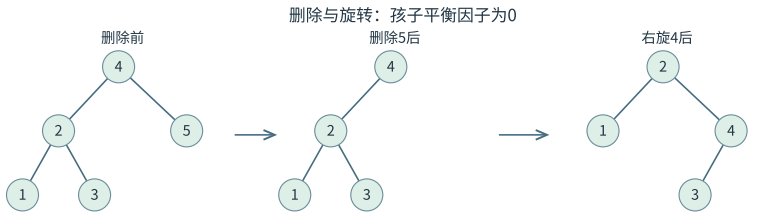


Figure 7.3: 删除 5 造成左高，孩子 2 平衡因子为 0；右旋后高度保持 3

不变量·旋转保留中序次序

右旋时 B 原右子树 T 的关键字介于 B 与 A 之间，转接为 A 左子树仍符合区间。若忘记保存 T，会丢失子树；若只交换数值，结点身份和其他链接语义也可能改变。

7.6.3 例 7-4　中序为降序时搜索方向怎样变

例 7.6.2 (配套案例)

【配套例题】一棵无重复关键字的 AVL 树，中序遍历得到降序序列。正确叙述是（　）。

A. 根的度一定为 2　　B. 最小元素一定是叶结点

C. 最后插入元素一定是叶结点　　D. 最大元素一定没有左子树

解析。本题的搜索方向与通常递增 BST 相反：左子树全大于根，右子树全小于根。最大值就是一直向左走到的那个结点，它没有左子树，答案为 **D**。AVL 约束不保证根一定有两个孩子；最小值结点可以有左孩子；新结点虽然最初作为叶插入，旋转后可能成为内部结点。不要用平衡性代替关键字的方向条件。

7.6.4 单旋代码为何必须保存中间子树

```
void R_Rotate(BSTree &p) {
    BSTNode *lc=p->lchild;
    p->lchild=lc->rchild;
    lc->rchild=p;
    p=lc;
}
void L_Rotate(BSTree &p) {
    BSTNode *rc=p->rchild;
    p->rchild=rc->lchild;
    rc->lchild=p;
    p=rc;
}
```

这里以引用 `p` 接回旋转后局部根，调用时将原双亲的相应孩子域传入；旋转整树时传入整树根指针。若结构还保存平衡因子或高度，须在旋转后依实际子树重新维护，先处理下降结点，再处理提升结点。右旋中 $B < T < A$ 的区间关系保证 T 接到 A 左侧仍合法；只交换两个关键字不等同于链接旋转。

AVL 插入可在 BST 递归返回时逐层更新高度，若平衡因子超界，则按新增关键字落在重侧孩子哪一边选择 LL/RR/LR/RL。最少结点递推的边界为 $N(0)=0, N(1)=1$；删除时子树变矮可能继续向上传播，应沿整条祖先路径复查。

7.7 红黑树：颜色约束怎样限制高度

错项排除·红叔要先变色

红双亲配红叔时，旋转不是第一选择：把双亲与叔变黑、祖父变红，将冲突上移，再保证根黑。黑叔的内外侧旋转情形不能覆盖这个状态。

红黑树仍是 BST，另为结点标红或黑，采用以下约定：根为黑；所有空孩子视为黑色 NIL 叶；红结点的孩子均为黑；从任一结点到其所有后代 NIL 叶的路径，黑结点数相同。NIL 不是普通数据结点，画图时可以省略，但计算黑高时必须把它计入。

相邻红结点不允许，所以一条路径中红结点数不会超过相应黑结点数。本书把黑高 b 定义为：从当前结点向下到任一 NIL 的路径上，不计当前结点，但把终点 NIL 计入，统计所含的黑结点数；NIL 自身的黑高记 0。黑高为 b 的子树至少含 2^b-1 个内部结点。于是最长路径的长度至多约为最短路径的两倍，整体高度为 $O(\log n)$。它不要求每个结点左右高度差至多 1，比 AVL 的局部高度约束宽。

按上述黑高 b 约定，根为黑的非空红黑树内部结点数满足 $2^b-1 \le n \le 2^{2b}-1$。下界可由 b 层全黑满树达到；上界由黑、红两色交替并填满共 $2b$ 个内部层达到。例如根黑高 2 时，内部结点数最少 3、最多 15。NIL 不计入内部结点数，不能把其数量也加到这两个界中。

插入先按 BST 加入，通常把新结点染红，以免立即改变各路径黑结点数。双亲黑则完成；双亲红时，看祖父另一侧的叔结点：

- 叔为红：双亲与叔变黑、祖父变红，把问题上移到祖父，最后保证根黑。
- 叔为黑（含 NIL）：若形成内侧折线，先在双亲处旋转为外侧；再在祖父处反向旋转，并交换提升结点与原祖父的颜色，消除红红相邻。

例如黑根 10、红左孩子 5，插入红 1，叔为空黑，形成 LL：右旋 10 后 5 成为黑根，1 与 10 为红，黑路径数保持一致。若原来还存在红右孩子 15，则插入 1 先使 5、15 变黑，祖父 10 暂红，最后根 10 恢复黑；这个情境应变色，不应直接套前一个旋转。

7.7.1 按课件序列观察红黑插入

从空树依次插入 `20,10,5,30,40`：20 作为根染黑；10 作为红左孩子接入；插入 5 使红 10 与红 5 相邻，叔为黑 NIL，属于 LL，右旋 20 并换色，得到黑根 10、红孩子 5 和 20。插入 30 时父 20 与叔 5 均红，把 5、20 染黑，祖父 10 暂红后恢复根黑；插入 40 时父 30 红、叔为空黑，发生 RR，对 20 左旋、把 30 染黑 20 染红。

最终为黑根 10，左孩子 5 黑，右孩子 30 黑且有红孩子 20 和 40。每步既保持中序递增，又保持所有通往 NIL 的黑高相等。课件后续的 `57,3,2,4,35,25,18,22,23,24,19,18` 用于继续观察向上传播、内侧双旋及重复值处理；若本章按互异关键字的 BST 接口实现，末尾重复 18 应返回未插入。题目允许重复时，先说明重复项安放或计数规则。

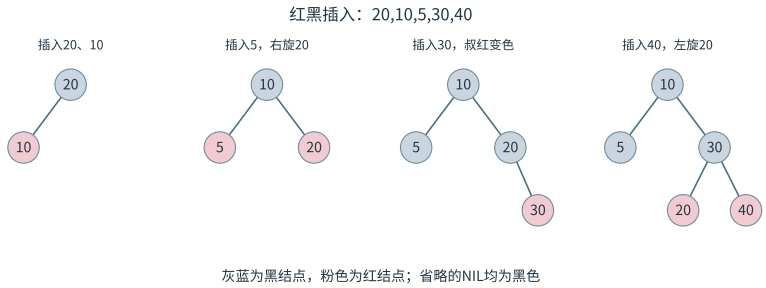


Figure 7.4: 从 20、10、5 到 30、40：黑叔旋转和红叔变色分别处理

修复的旋转次数和沿祖先链变色的次数应分别统计。红黑插入至多作常数次旋转，颜色调整可能一路传播到根；整个插入仍需先搜索且最坏 $O(\log n)$。红黑树不意味着一次插入的所有工作都为 $O(1)$。

删除时先按 BST 把待删结点化为至多只有一个非空孩子的情形。删红不减少路径黑高；删黑后，若有红孩子接替，可把接替孩子染黑。若替代位置是黑或 NIL，则沿该分支缺少一个黑结点，需要根据兄弟与其孩子颜色修复。

设缺黑位置 x 是双亲 p 的左孩子，兄弟 w 为 p 的右孩子。w 的左孩子为近侧、右孩子为远侧，均相对 x 而言；NIL 按黑处理。按表中的条件逐步转换，不能把 “红兄弟” 当作已经完成修复。

不变量·B 树合法性有五项

每次操作完成后同时检查结点内有序、孩子区间、键数上下限、孩子数、叶层深度。只画出平衡外形会漏掉关键字重复或孩子接错；临时溢出框不是最终合法状态。

当前条件	操作与后续
w 红	w 染黑、p 染红，对 p 左旋；重新取 x 的兄弟，再处理下列黑兄弟情况
w 黑、两个孩子都黑	w 染红；若 p 红则 p 染黑并结束，否则将缺黑位置上移到 p 继续，移到根即可结束
w 黑、远侧黑、近侧红	近侧染黑、w 染红，对 w 右旋；重新取兄弟，转成下一行
w 黑、远侧红	w 继承 p 原颜色，p 与远侧染黑，对 p 左旋，结束缺黑修复

缺黑在右侧时，左右及旋转方向对称互换。最后一行的最小例子可写成 `10 黑 (缺黑 NIL,20 黑 (NIL,30 红))`：左旋 10 并换色，得到 `20 黑 (10 黑,30 黑)`，两侧向下的黑结点数相同。旋转前后中序仍为 10、20、30；修复既要补黑高，也要保持搜索次序。

这里颜色不表示访问频率，也不影响 BST 比较方向。做性质判断时，先检查中序有序，再检查根与 NIL、红红相邻、各分支黑高；仅数红黑结点总数不能判定一棵红黑树是否合法。

7.8 B 树：一个结点保存多个分隔关键字

本书 m 阶 B 树规定每结点最多 m 个孩子、最多 m−1 个关键字。除根外，每个数据结点至少有 t−1 个关键字，其中 $t = \lceil m/2 \rceil$；有非空孩子的非底层数据结点至少 t 个孩子，非叶根至少 2 个孩子、1 个关键字。最底层数据结点没有非空孩子，并且全部在同一层。若另外画查找失败的空外部叶，它们在下一层且不存关键字；以下高度不计这层空叶。

以下容量与高度公式采用 $m \geq 3$，高度为数据结点层数；“叶” 若按教材专指最下层的空失败结点，就不把它混入该高度。阶数 m、最小孩子数 $t = \lceil m/2 \rceil$ 和关键字数必须区分，不能把别处 “最小度数 t” 的算法原样按 m 替换。

一个结点有 k 个升序关键字，就有 k+1 个孩子区间。先在结点内找到目标或确定区间，再读相应孩子。多路分支降低外存层数；结点内可以顺序或折半比较，读盘次数不直接等于关键字比较次数。

7.8.1 插入从叶开始，分裂向上传播

以下是 **5 阶 B 树的教学机制例**，用紧凑关键字显示分裂与合并，不对应后面原题的阶数。5 阶结点最多 4 项，非根至少 2 项；非底层非根因此有 3 至 5 个孩子，非叶根仍可只有 2 个孩子。

初态根为 `[5,10,15,20]`，从左到右的 5 个叶分别为 `[1,3]`、`[6,8]`、`[11,13]`、`[16,18]`、`[21,23,25,27]`，全树共 16 个关键字。沿

搜索路径插入 29 后，最右叶临时有 5 项。提升第 3 项 25，叶分成 [21,23] 与 [27,29]，提升项从叶中移走。根因而临时成为 [5,10,15,20,25]，也有 5 项，不能到此结束。

再将根的第 3 项 15 提升为新根，左右内部结点分别为 [5,10]、[20,25]。原来的 6 个孩子按区间各接 3 个：左侧接 [1,3]、[6,8]、[11,13]，右侧接 [16,18]、[21,23]、[27,29]。两次分裂均为“5 项 → 左 2 项、提升 1 项、右 2 项”，内部结点分裂时还须同步分配孩子。根分裂使数据层数由 2 增至 3；局部叶分裂本身不增加全树高度。

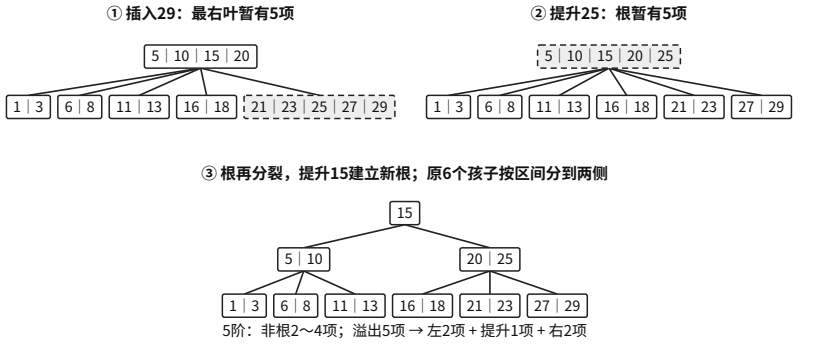


Figure 7.5: 5 阶 B 树插入 29，先分裂叶，再处理根溢出；虚框是待修复的 5 项结点

7.8.2 删除后的借位、合并与向上传播

删除内部关键字时，先用相邻前驱或后继关键字替换，再到最底层删除对应项。删除后若该结点的关键字少于最小数，先看相邻兄弟能否借：父分隔关键字下移，兄弟中最靠近当前结点的一项上移到父；若在内部层借位，右兄弟的最左孩子或左兄弟的最右孩子也要转接，保持孩子数等于关键字数加 1。兄弟也只有最小数时，合并当前结点、父分隔关键字与相邻兄弟，父可能继续下溢；空根仅剩一个孩子时，由孩子接替，高度减 1。

先看 5 阶借位小例：根 [10]，两个叶 [2,5] 与 [12,15,18]。删除 2 后左叶只剩 [5]，低于 2 项下限；右兄弟有 3 项，能够出借。将父 10 下移到左叶、右兄弟最小项 12 上移到父，得到根 [12]、叶 [5,10] 与 [15,18]。不能把 12 直接塞给左叶而让父仍为 10，否则分隔关系就被破坏。

再从插入图的最终树删除 1：左叶剩 [3]，相邻叶 [6,8] 已只有 2 项，不能借；合并它们与父分隔项 5，成为 [3,5,6,8]。其双亲原为 [5,10]，此时只剩 [10] 和 2 个孩子，也低于非根下限。右内部兄弟 [20,25] 同样不能借，故将左侧 [10]、根 15、右侧 [20,25] 合为 [10,15,20,25]，按顺序接 5 个原孩子。旧根为空，删除旧根后数据层数从 3 降为 2。

最小反例·5 阶不能提前随意合并

5 阶非根至少 2 键、最多 4 键。两个合法最小兄弟各 2 键，加父分隔项共 5 键，已经溢出；删除后某侧只剩 1 键，才能按 1+1+2=4 合并。阶数约定决定合并时机。

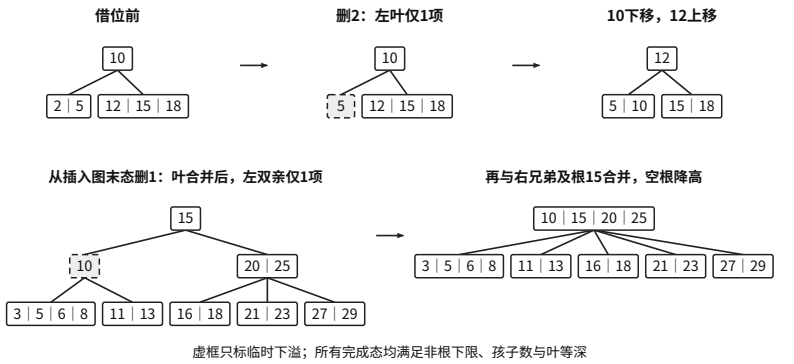


Figure 7.6: 上排为经父结点借位；下排为叶合并使双亲下溢，再合并并降低树高

5 阶 B 树的合并要特别注意时机：**两个各有 2 项的合法兄弟加父分隔项会有 5 项，超过上限**；本例是在删除造成一侧仅 1 项后，按 $1+1+2=4$ 项合并。不能把其他阶数或最小度参数下的提前合并规则直接套来。每个完成态都要复查结点内有序、孩子区间、关键字上下限、孩子数和叶等深；虚框中的临时下溢或溢出不算一个已完成的合法 B 树状态。

若高度 h 按数据结点层数计，非空 m 阶 B 树关键字总数满足 $2t^{h-1}-1 \le n \le m^h-1$，其中 $t=\lceil m/2 \rceil$。左端让根取 1 项、其余结点取最少分支，右端让各结点取满；求最大高度用左端，求最小高度用右端。

7.8.3 例 7-5　关键字总数怎样限制高度

例 7.8.1 (配套案例)

〖7.4-009〗一棵 3 阶 B 树共有 2047 个关键字，最大高度为（　），最小高度为（　）。

A. 11　B. 10　C. 8　D. 7

解析。高度按数据结点层数计。要使树尽量高，各层尽量稀疏：根 1 项、2 个孩子；其他结点至少 1 项、至少 2 个孩子。高度 h 时最少关键字 $1+2+\cdots+2^{h-1}=2^h-1$。因此 $2^h-1 \le 2047=2^{11}-1$，最大 11，第一空 **A**。

要使树尽量矮，各结点取满，每层关键字数分别 2、6、18……，高度 h 时最多 3^h-1。$3^6-1=728<2047\le 3^7-1=2186$，所以最小 7，第二空 **D**。这两端分别使用 “最少容量” 和 “最大容量”，不能将不等式方向颠倒。

7.8.4 例 7-6　小规模 B 树的形态怎样完整计数

例 7.8.2 (配套案例)

【配套例题】给定 7 个不同关键字，能构造的不同 4 阶 B 树的个数最多为（　）。

A. 7　B. 8　C. 9　D. 10

解析。每结点最多 3 项，非根至少 1 项；关键字的升序分配由形态与各结点项数确定，不再任意排列。

高度 1 最多 3 项，不够。高度 2 时，设根有 r 项，则有 r+1 个叶，叶

不变量·B 树合法性有五项

每次操作完成后同时检查结点内有序、孩子区间、键数上下限、孩子数、叶层深度。只画出平衡外形会漏掉关键字重复或孩子接错；临时溢出框不是最终合法状态。

项数和为 7−r，各叶 1 至 3 项：r=1 时只有 `(3,3)`，1 种；r=2 时是正整数三元组和 5，各项至多 3，包括 `(3,1,1)` 的 3 个排列与 `(2,2,1)` 的 3 个排列，共 6 种；r=3 时只有 `(1,1,1,1)`，1 种。因此高度 2 共 8 种。

高度 3 时，最少项数为 $1+2+4=7$，只能每结点 1 项且每个内部结点 2 个孩子，1 种。高度 4 最少 15 项，已不可能。合计 $8+1=9$，答案为 **C**。只考虑根的取值而不枚举叶的分配，会漏掉不同的结点分组。

课程连接·索引层服务外存页访问

B+ 内部省去整条记录，通常能在同页放更多索引项，降低层数；叶链便于范围扫描。外存优势取决于页大小、缓存和记录存放，不能改写成任何输入均比内存 BST 快。

7.9 B+ 树：索引层与叶层分工

B+ 树把记录或记录指针集中在叶结点，内部结点保存索引关键字及孩子指针；叶结点按关键字有序，并用链接便于连续访问。一次点查找沿索引降到叶；范围查找先定位起点叶，再沿叶链扫描，避免对每个值重新从根下降。

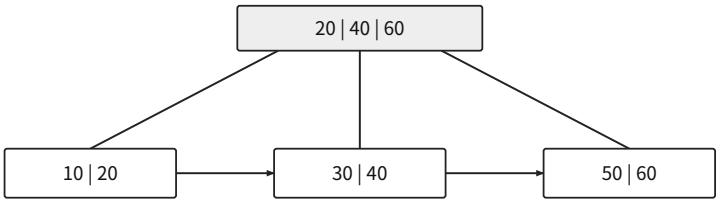


Figure 7.7: 内部层只导航，叶层保存全部关键字并按序相连

本章按王道的 “内部结点每个关键字对应一棵子树最大关键字” 约定。非叶根至少有 2 棵子树，其余分支结点至少有 $\lceil m/2 \rceil$ 棵子树，最多均为 m 棵；分支结点的关键字数与子树数相同。本图中，索引 `[20|40|60]` 指向最大值分别为 20、40、60 的三片子树。做题应先按给定图或定义确定，不把 B 树的 “k 键必有 k+1 孩子” 无条件套给 B+ 树。

B 树的查找可以在内部结点命中时结束；B+ 树的记录查询通常仍要走到叶。叶链让 B+ 适合区间访问，并不意味着每次点查都顺着整条叶链查找。内部不携带整条记录，通常能提高同样页大小下的分支数，减少外存层数；优势来自页组织与访问方式，不能据此声称任何规模都一定比内存 BST 更快。

7.10 散列表：冲突、删除与 ASL

散列函数 $H(key)$ 把关键字映射到表地址；不同关键字有相同函数值称为同义词，发生地址冲突。函数应对同一关键字给出确定地址，并尽量均匀分散。随机改变同一关键字的地址会使后续查找无法复现位置，不能直接作为稳定散列函数。

7.10.1 散列函数的构造条件

直接定址可用 $H(key)=a\,key+b$，适合关键字分布较连续且范围可控；它避免了所选模型内的碰撞，但地址范围大时空间可能浪费。除留余数法用 $H(key)=key \bmod p$，通常选不大于表长且有利于分布的素数 p；取模常数与开放定址继续探查的表长 m 是两个参数。

数字分析法利用已有关键字分布选择较均匀的数位；平方取中法先平方再取中间若干位，希望让多位信息影响结果。没有一种构造能对所有未知输入自动保证无冲突或均匀，应依据关键字类型、分布和地址范围选择；同一关键字必须能稳定复现相同初始地址。

开放定址在表内继续探查空位置。线性探查地址为 $(H_0+k) \bmod m$；二次探查可能规定为 $(H_0+k^2) \bmod m$，也可能按正负平方交替，须按题目公式执行。探查时的模数取表长 m；散列函数内部的除数 p 可能小于 m，两者不能混用。线性探查能够在一整轮内访问全表，二次探查不一定遍历所有位置，即使表未满也可能找不到可达空位。

链地址法把同义词放在一个桶的链表中，成功时沿链比较，失败时须比较到链尾。开放定址的聚集不仅来自同义词，也可能由不同初始地址的探查链交错引起。装填因子 $\alpha = n/m$ 越大，一般可用位置越少；散列并不保证任何输入下都严格常数时间。

条件辨析·互质条件检查实际步长

素数表长也不能挽救模表长为 0 的步长。步长须在 1 至 m-1 范围且与 m 互质，才能覆盖全表。

7.10.2 探测公式与覆盖范围

开放定址统一写作 $H_i = (H(key) + d_i) \bmod m$。增量每次加在**初始地址**上，各方法的规则如下：

方法	增量或步长	完整探测的条件
线性探测	$d_i = i$	连续 m 次能遍历全表
平方探测	0、1、−1、4、−4、9、−9、⋯	依表长与增量范围；有空位未必能到达
双散列	$d_i = i \cdot hash_2(key)$	步长与 m 互质时遍历全表
伪随机探测	题目规定的可复现增量序列	需实际检查增量模 m 覆盖的剩余类

当 m 为形如 $4j+3$ 的素数并采用足够的正负平方增量时，可覆盖全部位置。若只用正平方序列，则是另一套覆盖条件，不能把这条结论直接移用。

双散列需保证步长模 m 非零且与 m 互质。例如 $m=8$、步长 2 只能访问 4 个位置，步长 3 能访问全部 8 个位置。即使 m 为素数，若取 `hash2(key)=m-(key%m)`，在 `key%m==0` 时步长等于 m，仍会反复停在原地址；可在整数关键字场景采用 `1+(key%(m-1))`产生 $1..m-1$ 的步长，再明确负数取模规则。

7.10.3 链地址法的完整例

课件取表长 13、$H(key) = key \bmod 13$，依次插入 `19,14,23,1,68,20,84,27,55,11,10,79`。若采用头插，非空桶为：

桶号	从首结点到尾结点的关键字
1	79→27→1→14
3	55→68
6	84→19
7	20
10	10→23
11	11

查 27 比较 79、27，共 2 次；查 66 落桶 1，比较 4 个关键字后失败；查 21 落空桶 8，关键字比较次数为 0。若题目统计空指针判断，则另计。等概率成功 ASL 为 $(10+3+3+1+3+1)/12 = 21/12$；13 个桶等概

率失败时 ASL 为 12/13，因为桶长之和为 12。采用尾插会改变某一具体记录的成功路径，等概率成功 ASL 对每桶长度固定的全体记录平均仍相同。

课件还讨论了桶内保持关键字有序的链表：递增链中，查找值小于当前关键字时即可提前失败，有机会减少失败比较；代价是插入时也要维护有序位置。具体 ASL 须按新链序和目标值分布重新计算，不能继续沿用 “所有失败都扫描到链尾” 的口径。

条件辨析·分母由失败事件决定

表长 16 且 H 取模 13 时，初始地址只有 13 种。探测经过的地址和初始地址集合不同，平均时不能见表长就除 16。

7.10.4 表长与散列值域不同的 ASL

同一关键字序列放入长度 16 的开放定址表，$H(key) = key \bmod 13$，冲突时以表长 16 作线性探测。最终 0 号空，1～12 号依次为 `14,1,68,27,55,19,20,84,79,23,11,10`，13～15 号空。成功探测次数依插入次序为 `1,1,1,2,1,1,3,4,3,1,3,9`，成功 ASL 为 $30/12 = 2.5$。

失败事件若按**初始地址 0～12 等概率**，对应探测长度为 `1,13,12,11,10,9,8,7,6,5,4,3,2`，故失败 ASL 为 $91/13 = 7$。13～15 号仍可在探测过程中经过，但散列函数不会把它们作为初始地址；分母应为 13 而不是 16。删除 20 并保留删除标记后，成功记录变 11 项，成功 ASL 为 29/11，失败路径仍不被截断，失败 ASL 保持 7。

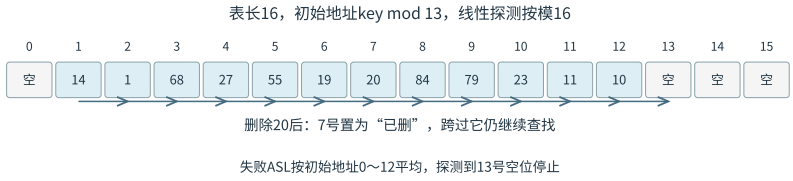


Figure 7.8: 同一探测链跨过删除标记；初始散列地址只来自 0 至 12

7.10.5 例 7-7　删除标记为什么继续计入失败路径

例 7.10.1 (配套案例)

【配套例题】长度为 5、初始为空的散列表 HT，$H(k) = (k + 4) \bmod 5$，线性探查处理冲突。依次插入 2022、12、25，再删除 25，查找失败的平均查找长度为（　）。

A. 1　B. 1.6　C. 1.8　D. 2.2

解析。 2022 与 12 初始地址都为 1，分别存 1 与 2；25 存 4。删除 4 处关键字后应留 “已删除” 标记，而不是把它当作从未使用的空位置。得到：

地址	槽状态	从此地址起失败探查次数
0	从未用空	1
1	2022	3
2	12	2
3	从未用空	1
4	已删除	2

失败查找遇到从未使用的空位才结束；越过删除标记还要继续。按 5 个初始地址等概率，ASL 为 $(1 + 3 + 2 + 1 + 2)/5 = 1.8$，答案为 **C**。探查到空单元也计一次表访问，这是本题失败 ASL 的计数口径；删除标记处没有有效关键字可比，却不能在这里终止。

若把一个探查链的中间位置直接清空，后面的元素虽然还在，查找却会在这里提前失败。插入时删除位置可以复用；若不允许重复关键字，要继续查到能排除已有同关键字的位置，再决定使用哪个已删除槽。

不变量·墓碑保住后续探查路径

已删除槽说明这里曾发生占用，后面可能仍有沿同一序列插入的元素。查找只能在从未用空槽或探查耗尽处失败；插入复用墓碑前还要排除后方已有同关键字。

7.10.6 例 7-8　二次探查必须回到原始地址计算

例 7.10.2 (配套案例)

【配套例题】将 20、3、11、18、9、14、7 依次插入初始为空、长度 11 的 HT。$H(key) = (key \times 3) \bmod 11$，初始地址为 H_0，冲突时 $H_k = (H_0 + k^2) \bmod 11$ $(k = 1, 2, \ldots)$。

（1）画出 HT 并计算装填因子。（2）给出查找 14 时的关键字比较序列。（3）查找 8，确认失败时的散列地址是多少?

解析。 20、3、11、18 直接存于 5、9、0、10。9 从 5 冲突后到 6；14 从 9 依次探查 10、2，存 2；7 从 10 依次探查 0、3，存 3。

地址	关键字
0	11
1	空
2	14
3	7
4	空
5	20
6	9

地址	关键字
7	空
8	空
9	3
10	18

装填因子为 7/11。查 14 的探查地址为 9、10、2，比较关键字为 3、18、14。查 8 时，$H_0 = 24 \bmod 11 = 2$，探查地址依次为 2、3、6、0、7，第 7 号为空，所以在**地址 7** 确认失败。每个新地址从同一个 H_0 加平方增量计算，不能在上一次地址上再加 k^2。

7.10.7 开放定址的三种槽状态

```
hashFind(key):
    按约定的探查序列访问地址
    若槽为有效且关键字相同：成功
    若槽从未使用：失败
    若槽已删除或关键字不同：继续探查
    若可探查位置已耗尽：失败
```

删除先按同一路径找到关键字，再把槽改为 “已删除”。插入可记住第一个已删除槽，但不能立即停在那里：若后面已有同关键字且不允许重复，提前写入会制造重复。探查总步数须有界，表满或二次探查可达集合已走完时不能无限循环。

链地址法的插入在相应桶链完成，删除是真实断链并释放目标结点；它不需要开放定址的墓碑标记。链长才是比较量来源，不能拿开放定址中 “遇从未用空槽终止” 的规则解释链地址失败。

不变量·墓碑保住后续探查路径

已删除槽说明这里曾发生占用，后面可能仍有沿同一序列插入的元素。查找只能在从未用空槽或探查耗尽处失败；插入复用墓碑前还要排除后方已有同关键字。

7.11 怎样按操作约束选择方法

下表把主要成立条件与查找代价分开。记录数为 n，散列表长为 m，树高记为 h；B/B+ 树的 h 按数据层数计，假定每个结点占一个外存页，表中的读盘上界指索引结点路径上的页访问，还未扣除已缓存页；若叶中只存记录指针，随后取叶外记录的 I/O 另计。

方法或结构	成立条件	查找代价	更新、范围或外存特点
顺序查找	可无序；数组或链表	最坏 $O(n)$	不需维护有序；范围筛选仍须扫描
折半查找	有序且能随机定位的顺序表	最坏 $O(\log n)$	数组插删最坏移动 $O(n)$；可定位区间端点
普通 BST	满足全部子树的区间关系	$O(h)$，最坏 $O(n)$	支持前驱后继；无平衡维护时可退化成链
AVL	BST 且任一结点两侧高差至多 1	最坏 $O(\log n)$	动态插删需维护高度与旋转
红黑树	BST 及红黑性质均成立	最坏 $O(\log n)$	动态插删维护颜色、黑高与旋转
B 树	多关键字有序、容量约束、叶等深	至多 h 次结点读盘	内部命中可停；读入结点后的键比较另计
B+ 树	按给定索引约定，叶层存全体关键字及记录或记录指针	点查沿根到叶，至多 h 次索引页读盘	定位起点后沿叶链作范围访问

方法或结构	成立条件	查找代价	更新、范围或外存特点
散列表	散列函数、装填因子和冲突策略匹配	合适散列与受控负载下平均 $O(1)$；最坏链扫描 $O(n)$，开放定址探查 $O(m)$	扩容有维护成本；原生不保序，不直接支持有序范围定位

一个表可能有多种索引。不能只为某一次查找更快，就忽略频繁更新带来的维护成本；也不能用散列的平均等值查找优势替代区间检索所需的有序性。题目给出 “有序、随机访问、频繁插删、外存、范围” 时，分别把它们转成结构成立条件与访问成本，再作选择。

7.12 主动回忆自测

以下为自拟补充，检查维护过程与失败口径。

1. BST 中每个结点都满足 “左孩子小于自身、右孩子大于自身”，是否足够？给反例。
2. 4 阶 B 树一个非根结点最多、最少有多少关键字？根为空时能否直接认定整树空？
3. 一条散列探查路径经过 “有效关键字—删除标记—有效关键字—从未用空位”，目标均不等，失败探查几次？
4. 红黑树根 10 黑、孩子 5 红和 15 红，插入 1 后，首先应旋转还是变色？

5. AVL 删除后只修复最深一处失衡是否总够？与插入有何不同？
6. B 树一个内部结点有 3 个关键字，按本章约定有几个孩子区间？B+ 树能无条件沿用吗？
7. 在有序链表上执行折半式比较，为何不能直接承诺总时间 $O(\log n)$？
8. 为什么散列表失败 ASL 的分母未必是有效记录数？若表长 5、只存 2 项，按初始地址等概率应除以几？

答案。1. 不够；根 10，左孩子 5，5 的右孩子 12，所有直接比较都满足，但 12 在 10 左子树却大于 10。2. 最多 3 项、最少 1 项；根空但还有唯一孩子时应降层，由孩子接替，树不空。3. 4 次，删除标记不能截断路径。4. 先变色，叔 15 红，把 5、15 变黑、10 暂红，最后根恢复黑，无需该处旋转。

5. 不总够；删除可能让修复后子树高度继续减 1，导致更高祖先失衡。插入修复最小失衡子树后通常恢复原高度，可阻断继续传播。
6. 有 4 个孩子区间；B+ 索引约定各异，若每项表示一棵子树最大关键字，键数和孩子数可以相等，需按给定定义。
7. 比较次数可能为对数，但链表不能常数时间随机定位中点，节点访问成本没有消失。
8. 失败事件分布可以是初始散列地址或失败区间，而非已有记录；该例应按 5 个初始地址求平均，除以 5。

不变量·墓碑保住后续探查路径

已删除槽说明这里曾发生占用，后面可能仍有沿同一序列插入的元素。查找只能在从未用空槽或探查耗尽处失败；插入复用墓碑前还要排除后方已有同关键字。

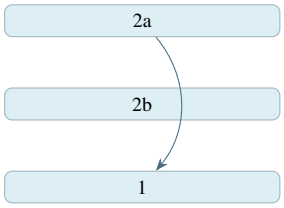
8 排序

章节脉络

排序把记录按关键字排列，同时可能改变相等关键字记录原有的相对次序。做过程题时，先确定这一趟处理的是一个元素、一个增量分组，还是所有当前子区间；做选择算法的题时，再比较规模、初始状态、稳定性、空间和外存访问。算法名字相同、划分约定不同，产生的中间序列也可能不同。

最小反例·相同数字隐藏记录身份

最小反例给相等关键字附身份：$2_a, 2_b, 1$ 经简单选择首趟成为 $1, 2_b, 2_a$。只写数字 2、2 会看不出顺序改变；稳定性讨论的是整条记录的身份。



交换首项与 1 跨过 2b，结果为 1、2b、2a。

8.1 本章知识结构

- **排序的基本概念**：关键字与记录身份；稳定与不稳定；内部外部排序；趟数比较移动空间。
- **插入类排序**：直接插入有序前缀；折半插入定位及移动；希尔同余类与增量。
- **交换与选择类**：起泡相邻逆序与早停；快速划分及枢轴就位；简单选择的比较次数；堆排序建堆及下滤。
- **归并与分配类**：二路归并及稳定合并；LSD 低到高与同桶保序；MSD 递归桶任务；计数范围及前缀和。
- **算法性能与条件**：最好平均最坏时间；递归栈及输出数组；稳定性条件与反例；比较排序下界。
- **外部排序与优化**：初始归并段；多路归并及输入输出缓冲；I/O 计数与趟数；败者树冠军路径；置换选择与冻结；最佳归并树及补零权叶。

每种排序先固定待处理区、有序区和一趟的含义，再逐步执行移动或交换。通过同一组记录追踪身份、趟数、比较次数和空间需求，可以把算法差异落实到具体操作。

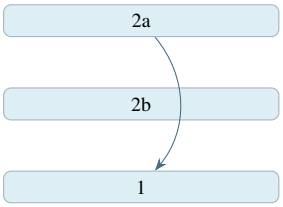
8.2 稳定性、内外排序与比较口径

定义 8.2.1 (排序稳定性)

若两条记录关键字相等，排序后它们的相对次序与原来一致，称该排序算法**稳定**。给记录加身份下标，如 $2_a, 2_b$，就能检验相等值是否被交换；只看两个相同数字无法观察稳定性。稳定性是算法对所有合法输入的保证，某个不稳定算法在一份特定数据上也可能恰好没有打乱相等记录。

最小反例·相同数字隐藏记录身份

最小反例给相等关键字附身份：$2_a, 2_b, 1$ 经简单选择首趟成为 $1, 2_b, 2_a$。只写数字 2、2 会看不出顺序改变；稳定性讨论的是整条记录的身份。



交换首项与 1 跨过 2b，结果为 1、2b、2a。

内部排序指排序时待处理记录能全部放入内存；外部排序需要在内外存间分批交换。二者的区别在于处理期间是否需要外存参与，不能只按一个固定的数据量阈值划分。外部排序通常优先减少块 I/O，内部排序则常比较关键字比较次数、移动次数和辅助空间。

“一趟” 指什么，须按具体算法的说明确定。直接插入一趟放好一个新记录；归并一趟合并所有相邻段；本章快速排序的过程题，把同一深度上尚未确定最终位置的所有子区间各划分一次，算作一趟。若程序逐子区间调用一次划分就计一趟，应按那份程序重新解释。

8.3 直接插入与折半插入

直接插入维护左侧已排好序的前缀。取下一个记录 x，从前缀末尾向左比较，把大于 x 的记录右移，直到找到插入位置。每次循环结束，前缀扩大 1 且仍有序。

```
void InsertSort(int A[], int n) {
    int i,j;
    for (i=2; i<=n; ++i)
        if (A[i]<A[i-1]) {
            A[0]=A[i];
            for (j=i-1; A[0]<A[j]; --j)
                A[j+1]=A[j];
            A[j+1]=A[0];
        }
}
```

教材此版将有效元素放在 `A[1..n]`，`A[0]`同时暂存待插入值并充当哨兵。若一直退到 0，`A[0]<A[0]`为假，循环必停，故内层无需再判断 `j>=1`。数组必须有 $n+1$ 个单元；`n=0,1` 时外层自然跳过。和下面使用 0 起始数组的起泡、选择、快排相比，存储范围须按各段代码分别说明。

相等时不再左移，把新记录放在已有相等记录后面，因此稳定。若条件改为 `A[j]>=x`，会把新记录放到旧相等记录之前，稳定性随之改变。初始有序时每趟只比较一次就停止，时间 $\Theta(n)$；逆序时要移动 $1+2+\cdots+(n-1)$ 个记录，最坏 $\Theta(n^2)$；仅用一个暂存记录和几个下标，空间 $O(1)$。

折半插入在有序前缀中用折半查找决定插入点，减少了关键字比较；但数组中腾出位置仍要移动元素，平均和最坏移动量仍为 $\Theta(n^2)$，因此平均和最坏总时间也为 $\Theta(n^2)$。要保持稳定，应寻找 “第一个大于 x” 的位置，把 x 插到所有旧相等项之后。

本章比较表采用**每趟都完整执行折半定位**的实现，不先用前缀末项

判断是否已经就位。因此即使初始有序，比较总量仍为 $\Theta(\sum_{i=1}^{n-1}\log(i+1))=\Theta(n\log n)$，最好总时间也是 $\Theta(n\log n)$；最好时只是移动少，比较代价并不随之减少。若另加“已不小于前缀末项则直接跳过折半”的判断，有序输入可变成线性时间，那是增加了提前判断的实现。

不变量·相等时继续向右定位

折半插入寻找首个大于 x 的位置，旧相等项都留在左边。定位快只减少比较，数组腾位置仍要移动；看到折半二字就写整体对数时间，漏掉了主要移动量。

8.3.1 折半定位怎样保持稳定

```
void BinaryInsertSort(int A[], int n) {
    int i,j,low,high,mid;
    for (i=2; i<=n; ++i) {
        A[0]=A[i];
        low=1; high=i-1;
        while (low<=high) {
            mid=(low+high)/2;
            if (A[mid]>A[0]) high=mid-1;
            else low=mid+1;
        }
        for (j=i-1; j>=high+1; --j) A[j+1]=A[j];
        A[high+1]=A[0];
    }
}
```

本段仍采用 `A[1..n]`，0 号仅暂存。相等时向右找，结束时 `low==high+1`，正是有序前缀中第一个大于新值的位置。寻找位置的比较减少，腾出位置的移动仍可能为二次；“折半”描述定位方式，不是整体复杂度结论。

8.4 起泡：一次相邻交换消除一个逆序

教材版本从后向前比较相邻元素，逆序则交换，一趟后最小记录到达未排序区最前端；下趟只处理右侧尚未确定的范围。若某一趟没有发生交换，整段已无相邻逆序，可提前结束。没有这项标志的实现即使输入有序，也仍会执行全部固定比较。

例如 `4,2,3,1` 第一趟从后向前依次变为 `4,2,1,3`→`4,1,2,3`→`1,4,2,3`，只保证开头 1 就位，不保证其余已序。每次交换仅在严格大于时进行，不会跨过相等记录，故稳定。带提前终止时最好 $\Theta(n)$，平均、最坏 $\Theta(n^2)$，空间 $O(1)$。

8.4.1 带提前停止的起泡代码

错项排除·最好线性依赖早停标志

起泡有序输入一趟无交换即可停，所以线性；若程序固定执行全部趟数，即使每次都不交换，比较仍是二次。复杂度表需要与实际程序条件配套。

```
void BubbleSort(int A[], int n) {
    for (int i=0; i<n-1; ++i) {
        bool flag=false;
        for (int j=n-1; j>i; --j)
            if (A[j-1]>A[j]) {
                int temp=A[j-1];
                A[j-1]=A[j]; A[j]=temp;
                flag=true;
            }
        if (flag==false) return;
    }
}
```

此段使用 C++ 的 `bool`；改用 C 时应包含 `stdbool.h` 或使用整型标志。每趟后前部已确定的最小记录不再参与下一趟，未交换则剩余后缀也已序。

8.5 简单选择：比较与移动的代价不同

简单选择每趟从未排序区找到最小记录，与该区首记录交换，使前部有序区扩大。无论初始是否有序，找最小值都要扫描当前未排序区，关键字比较总数固定为 $(n-1)+(n-2)+\cdots+1=n(n-1)/2$。至多交换 n−1 次，比较次数却始终为 $\Theta(n^2)$。

它通常不稳定。例如 $2_a, 2_b, 1$，第一趟把 1 与 2_a 交换，得到 $1, 2_b, 2_a$，相等关键字次序逆转。这不是比较时相等就交换造成的，而是与远处最小记录交换时跨越了 2_b；仅把找最小值的条件改成严格小于，不能保证稳定。

```
void SelectSort(int A[], int n) {
    for (int i=0; i<n-1; ++i) {
        int min=i;
        for (int j=i+1; j<n; ++j)
            if (A[j]<A[min]) min=j;
        if (min!=i) {
            int temp=A[i]; A[i]=A[min]; A[min]=temp;
        }
    }
}
```

省掉 `min==i` 时的自交换可以减少移动，却不减少寻找最小值的比较。这说明“比较固定”不等于“每份输入交换数也固定”。

8.6 希尔：一趟只保证各增量子序列有序

希尔排序选择递减增量 d，对下标同余的记录分别作直接插入排序，称为 d 间隔有序。较大增量使远处记录较快移动，最后必须有 d=1 这一趟，才能保证整体有序。某个 d 间隔有序，不代表相邻记录有序；改用更小的增量后，也仍需再排。

8.6.1 例 8-1　两趟希尔排序怎样按下标分组

不变量·d 间隔有序不是相邻有序

一趟希尔处理全部同余类，得到每个间隔 d 子序列有序。不同余数类之间仍未比较；只有最后 d=1，才把全数组约束为同一有序序列。

例 8.6.1 (配套案例)

〖8.2-012〗输入序列为 `13,24,7,1,8,9,11,56,34,51,2,77,5`，增量依次为 5、3、1。两趟排序后为（　）。

A. 1,7,8,9,13,24,11,34,51,2,5,56,77

B. 1,7,5,2,8,9,24,11,34,51,13,77,56

C. 2,11,5,1,8,9,24,7,34,51,13,77,56

D. 2,5,11,1,8,9,7,24,34,13,51,77,56

解析。d=5 按下标余数分组，例如下标 0、5、10 的 `13,9,2` 排为 `2,9,13`；下标 2、7、12 的 `7,56,5` 排为 `5,7,56`。第一趟整体为 `2,11,5,1,8,9,24,7,34,51,13,77,56`。

d=3 重新分组：下标 0、3、6、9、12 的 `2,1,24,51,56` 排为 `1,2,24,51,56`；下标 1、4、7、10 的 `11,8,7,13` 排为 `7,8,11,13`；下标 2、5、8、11 的 `5,9,34,77` 已有序。填回原下标得 `1,7,5,2,8,9,24,11,34,51,13,77,56`，答案为 **B**。

“一趟” 包含全部余数分组，不是只排第一组。最后 d=1 这一趟才对全序列作插入排序。希尔排序不稳定，复杂度与增量序列有关：不附增量条件，便给不出通用的精确平均阶；空间通常为 $O(1)$。

8.6.2 希尔排序的同余类循环

```
void ShellSort(int A[], int n) {
    int d,i,j;
    for (d=n/2; d>=1; d=d/2)
        for (i=d+1; i<=n; ++i)
            if (A[i]<A[i-d]) {
                A[0]=A[i];
                for (j=i-d; j>0 && A[0]<A[j]; j-=d)
                    A[j+d]=A[j];
                A[j+d]=A[0];
            }
}
```

有效元素为 `A[1..n]`，内层以 d 为步长在同一同余类中移动，因此一次 i 循环只处理一个新记录；完整一趟要遍历全部 i。`A[0]`在此仅为暂存单元，**不是所有增量子表都能遇到的哨兵**，所以必须保留 `j>0`。所有增量必须为正，否则无法保证进展；末增量 1 将各分组约束最终汇成整体有序。

8.7 快速排序：先划分，再递归

一趟划分选择枢轴，使左区记录不大于枢轴、右区记录不小于枢轴，并确定一个枢轴的最终位置；两侧内部仍可能无序。以下采用首记录作枢轴、双向扫描的 “挖坑” 方式，便于明确指针停留与填回位置。

```cpp
int Partition(int A[], int low, int high) {
    int pivot=A[low];
    while (low<high) {
        while (low<high && A[high]>=pivot) --high;
        A[low]=A[high];
        while (low<high && A[low]<=pivot) ++low;
        A[high]=A[low];
    }
    A[low]=pivot;
    return low;
}
```

条件辨析·相同输入，不同状态

插入前缀有序不代表枢轴就位；快速划分左右分隔不代表内部有序。过程题先按算法检查它真正保证的关系。

```cpp
void QuickSort(int A[], int low, int high) {
    if (low<high) {
        int pivotpos=Partition(A,low,high);
        QuickSort(A,low,pivotpos-1);
        QuickSort(A,pivotpos+1,high);
    }
}
```

每次划分扫过当前区间，代价与区间长度同阶。两侧规模较均衡时，每层合计 $O(n)$、约 $\log n$ 层，时间 $O(n\log n)$；若首枢轴总是最小或最大，规模为 $n-1$ 与 0，累积为 $\Theta(n^2)$。原序列已经有序时，这个首枢轴实现反而落到最坏情况，不能把插入排序“有序输入最快”的结论照搬过来。

递归栈平均 $O(\log n)$、最坏 $O(n)$；数组内划分的常数临时变量不能抵消调用栈。远距离移动会改变相等记录顺序，通常不稳定。等值很多时，具体左右扫描规则会影响划分平衡；三路划分是另一种处理方式，不能把它的状态与上述程序的状态混写。

8.7.1 同一组课件记录的分趟结果

初始序列为 `49,38,65,97,76,13,27,49`。为观察稳定性，把前后两个 49 分别记为 $49_a, 49_b$。直接插入各趟结果如下：

插入的记录	完成后序列
38	$38, 49_a, 65, 97, 76, 13, 27, 49_b$
65	$38, 49_a, 65, 97, 76, 13, 27, 49_b$
97	$38, 49_a, 65, 97, 76, 13, 27, 49_b$
76	$38, 49_a, 65, 76, 97, 13, 27, 49_b$
13	$13, 38, 49_a, 65, 76, 97, 27, 49_b$
27	$13, 27, 38, 49_a, 65, 76, 97, 49_b$
49_b	$13, 27, 38, 49_a, 49_b, 65, 76, 97$

采用教材首枢轴 `Partition`，同一输入第一次划分得到 `27,38,13,49_a,76,97,65,49_b`，枢轴位于 0 起始下标 3。先从右向左扫描，末尾 $49_b \geq 49_a$ 而跳过，遇 27 填左缺口；再从左扫描遇 65 填右缺口，继续完成。这不是直接插入的第一趟结果。后续递归还须分别处理左右两段。

Figure 8.1: 直接插入的有序前缀与快速划分的枢轴就位表达不同状态

不变量·尾部已序，前部只是堆

升序堆排序每轮把当前最大值固定到尾部，下一轮只调整缩小后的前部堆。前部兄弟不保证有序，不能把一次建堆的数组误判为一趟完整升序输出。

8.7.2 例 8-2　第二趟结束后应有几个枢轴就位

例 8.7.1 (配套案例)

【配套例题】对尚未确定最终位置的所有元素进行一遍处理称为一趟。下列序列中，不可能是快速排序第二趟结果的是（　）。

A. 5,2,16,12,28,60,32,72

B. 2,16,5,28,12,60,32,72

C. 2,12,16,5,28,32,72,60

D. 5,2,12,28,16,32,72,60

解析。先确定完全有序结果为 `2,5,12,16,28,32,60,72`。一个就位枢轴还必须满足左边全不大于它、右边全不小于它。若第一枢轴在端点，第二趟只划分一个非空子区间，可能只有两个枢轴就位；若第一枢轴不在端点，两侧均非空，第二趟后至少有三个就位位置（单元素区间自身已就位）。

选项 D 中只有 12 与 32 这两个位置满足分隔关系，而且两者都不是端点。无论谁作为第一枢轴，第二趟都应在它左右再各形成一个就位位置，这与只有两个就位位置相矛盾，所以答案为 **D**。

选项 A 可先把最大值 72 定在末尾，再在左区把 28 就位；选项 B 可先把最小值 2 定在首位，再把 72 就位；选项 C 可先把 28 就位，再让左区的 2、右区的 32 就位。判断时要检查每个枢轴对整个区间的分隔作用。

8.8 堆排序：堆范围与有序区分开

升序堆排序先建大根堆，根即未排序区的最大值；把根与当前末元素交换，这个最大值就留在末尾，堆范围减 1，再对新根下滤。重复到堆范围只剩一个元素。此时数组尾部有序，前部只是一个堆，整体并未递增。

建堆 $O(n)$，每次删除最大值并调整至多 $O(\log n)$，完整排序有 $O(n \log n)$ 时间上界；常规迭代下滤空间 $O(1)$。教材常按关键字互异的模型，把各种情况都列为对数线性阶，但不能据此断言任意重复输入都要做同样多的比较：若关键字全相等且下滤在相等时立即停止，具体实现可以只做线性数量次比较。末尾与根的远距离交换使堆排序通常不稳定。小根堆反复交换到末尾得到的是降序数组，不能只看 “每次取最小” 就认定数组最后为升序。

8.8.1 例 8-3　删除堆顶时怎样数关键字比较

课程连接·优先队列与选择小部分记录

只找最小 10 项时可维护 10 项大根堆，把更小新值替换堆顶；不必完整排序。流式读入适合这种状态维护，若可原地随机访问，还可比较选择算法的期望线性方案。

例 8.8.1 (配套案例)

【配套例题】小根堆为 `8,15,10,21,34,16,12`，删除关键字 8 之后重建堆，此过程中关键字之间的比较次数是（　）。

A. 1　B. 2　C. 3　D. 4

解析。用最后的 12 填根，堆范围由 7 缩为 6。先比较两个孩子 15 与 10，选 10（第 1 次）；再比较候选 12 与 10（第 2 次），10 上移、12 下到原 10 位置。该位置只有一个孩子 16，比较 12 与 16（第 3 次）后停止。结果为 `10,15,12,21,34,16`，答案为 **C**，过程如下图所示。

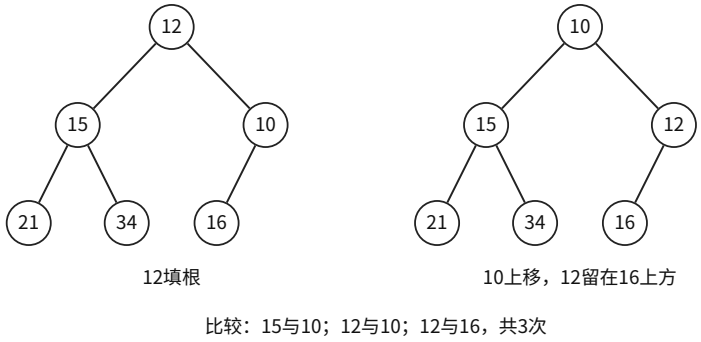


Figure 8.2: 删除堆顶后沿较优孩子路径下滤；只比较真实存在的孩子

无需再把 16 与不存在的右孩子比较，循环边界检查也不是关键字比较。若题目给了另一个下滤程序，例如始终下沉至叶再回升，应按那个程序计数，不能把本过程的 3 次当作所有实现的共同常数。

8.8.2 完整堆排序的两段工作

```
void HeadAdjust(int A[], int k, int len) {
    A[0]=A[k];
    for (int i=2*k; i<=len; i*=2) {
        if (i<len && A[i]<A[i+1]) ++i;
        if (A[0]>=A[i]) break;
        A[k]=A[i];
        k=i;
    }
    A[k]=A[0];
}
void BuildMaxHeap(int A[], int len) {
    for (int i=len/2; i>0; --i) HeadAdjust(A,i,len);
}
void HeapSort(int A[], int len) {
    BuildMaxHeap(A,len);
    for (int i=len; i>1; --i) {
        int temp=A[i]; A[i]=A[1]; A[1]=temp;
        HeadAdjust(A,1,i-1);
    }
}
```

有效元素存于 `A[1..len]`，0 号为暂存单元。`HeadAdjust` 的前提是左右子树已经成堆，只有根位置可能违反堆序；先选较大孩子，再

与暂存根比较。两个孩子相等时，代码保留左孩子，具体比较过程与此规则一致。

课件建堆数组 `53,17,78,9,45,65,87,32`，依次调整根 4、3、2、1。根 4 使 32 上升，根 3 使 87 上升，根 2 使 45 上升；根 1 使 87 上升，再使 78 上升，最终大根堆为 `87,45,78,32,17,65,53,9`。排序第一轮将 87 放到末尾，对前 7 项下滤，得到 `78,45,65,32,17,9,53 | 87`。竖线右侧为已确定的有序后缀，不能再参加下一轮的堆调整。

第一段自底向上建堆，第二段才把最大值逐个固定到末尾。不能把建堆 $O(n)$ 当成完整堆排序的最坏时间。

课程连接·优先队列与选择小部分记录

只找最小 10 项时可维护 10 项大根堆，把更小新值替换堆顶；不必完整排序。流式读入适合这种状态维护，若可原地随机访问，还可比较选择算法的期望线性方案。

8.9 二路归并：有序子段怎样变成长有序段

归并两个有序段时，只需比较当前未输出首记录，较小者先输出；一段耗尽后，另一段直接复制。相等时先取左段能保持稳定。归并长度分别为 a、b 的段，最坏作 a+b−1 次关键字比较：到只剩最后一个记录时，它属于哪个段已经确定，直接复制，不再作跨段比较。

自底向上先把每个记录视为长度 1 的段，一趟两两合并，段长最长变为 2，再到 4、8……；自顶向下先递归处理两半，再归并。每层总处理 $O(n)$ 个记录，层数 $O(\log n)$，故普通二路归并时间为 $O(n\log n)$，不因原序列已序就自动跳过整套过程。数组实现的归并辅助区通常为 $O(n)$，递归栈的 $O(\log n)$ 不改变总空间阶。

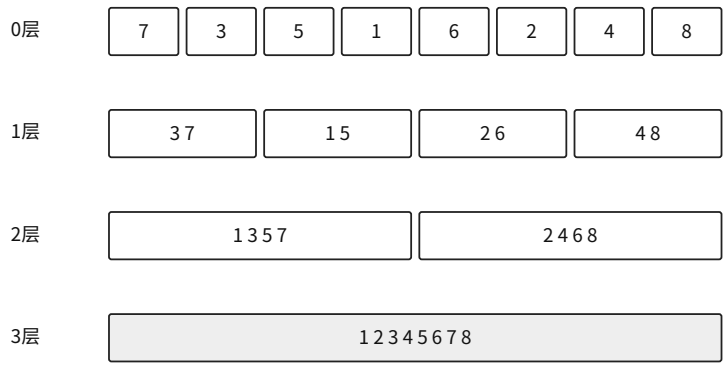


Figure 8.3: 每层归并读取上一层的有序段，最后一步仍需比较两个段首

8.9.1 例 8-4　比较次数不等于移动次数

例 8.9.1 (配套案例)

【配套例题】三个有序序列 `(3,5)`、`(7,9)`、`(6)`，按从左到右次序选择序列进行二路归并，关键字总比较次数为（　）。

A. 3　B. 4　C. 5　D. 6

解析。先归并前两段，比较 3 与 7、5 与 7，两次后左段耗尽，直接接 7、9，得到 `3,5,7,9`。再与单元素 6 归并，比较 3 与 6、5 与 6、7 与 6，共 3 次，之后接 7、9。总数 $2+3=5$，答案为 **C**。最坏上界 a+b−1 只有在具体过程能一直交错到仅剩一个元素时才取到，本题第一次只比较 2 次，不能机械按 4−1 计 3 次。

8.9.2 两段归并的稳定合并核心

错项排除·比较上界不等于具体次数

归并 a、b 项最坏比较 a+b−1 次，但一段提前耗尽就直接复制另一段。计算给定值的过程题应逐次比较段首，不能每次都机械代入最坏上界。

```
// B在排序开始前分配，容量至少等于A的有效数组长度。
int *B;
void Merge(int A[], int low, int mid, int high) {
    int i,j,k;
    for (k=low; k<=high; ++k) B[k]=A[k];
    for (i=low,j=mid+1,k=i; i<=mid && j<=high; ++k)
        if (B[i]<=B[j]) A[k]=B[i++];
        else A[k]=B[j++];
    while (i<=mid) A[k++]=B[i++];
    while (j<=high) A[k++]=B[j++];
}
void MergeSort(int A[], int low, int high) {
    if (low<high) {
        int mid=(low+high)/2;
        MergeSort(A,low,mid);
        MergeSort(A,mid+1,high);
        Merge(A,low,mid,high);
    }
}
```

相等时先取左段为何保序

假设两个输入子段内部已经稳定有序。来自同一段的记录依次输出，相对次序不变；来自不同段的相等记录，左段原本在前，比较相等时仍先输出左段。因此合并保留所有相等记录的原相对次序，递归或分层重复后全序列仍稳定。

□

8.10 基数排序：低位排好后怎样保留次序

基数排序按关键字的组成部分分配、收集。LSD 从最低有效位开始，逐位稳定排序；每轮收集必须保持同一桶内原顺序，才能保留先前已经排好的低位关系。处理更高位后，低位只在高位相同时才决定顺序，这也正是从低到高稳定处理的依据。

8.10.1 例 8-5　第一趟后为什么尚未整体递增

例 8.10.1 (配套案例)

〖8.5-012〗对 `05,46,13,55,94,17,42` 进行基数排序，一趟后的结果为（　）。

A. 05,46,13,55,94,17,42

B. 05,13,17,42,46,55,94

C. 42,13,94,05,55,46,17

D. 05,13,46,55,17,42,94

解析。本题采用通常的十进制 LSD，第一趟按个位分配：桶 2 放 42，桶 3 放 13，桶 4 放 94，桶 5 依次放 05、55，桶 6 放 46，桶 7 放 17。按桶号收集得 `42,13,94,05,55,46,17`，答案为 **C**。桶 5 不能把 55 放到 05 之前。第二趟按十位稳定处理，才得到 `05,13,17,42,46,55,94`。

若共 d 位、基数 r，每轮分配 n 项并收集 r 个桶，时间为 $O(d(n+r))$。空间要按实现：已有链式记录、链接域计入原记录时，额外的桶头、桶尾指针为 $O(r)$；数组分桶另开输出数组时为 $O(n+r)$。固定十进制位数与 r 时可近似线性，但不能把位数随规模增长的情形也无条件写为 $O(n)$。

MSD 从最高位开始，把记录分成桶后递归处理各桶；它的任务分解与 LSD 不同。负整数、可变长字符串、空缺位的次序都需要事先定义，不能把非负定长十进制示例原样用于所有关键字。

不变量·LSD 依赖每轮的稳定性

更高位相同的记录仍要靠已经排好的低位决定先后，故同桶必须保留上轮次序。改成插桶头或逆序收集可能破坏这个不变式，即使每轮单独看都按本位有序。

8.10.2 按取值计数：空间与稳定实现的条件

计数排序把关键字的值映射为计数数组的下标。若整数取值从 min 到 max，令 $k = \max - \min + 1$，以 `key-min` 为下标：先统计频次，再求前缀和，得到每个值应占据的输出位置范围。例如序列 $2_a, 4, 3_a, 0, 2_b, 3_b$ 的频次数组为 `[1,0,2,2,1]`，前缀和为 `[1,1,3,5,6]`。从后向前扫描原序列，每次把记录放到该值当前最后一个输出位置，再将位置减 1，得到 $0, 2_a, 2_b, 3_a, 3_b, 4$，相等记录仍保序。

按这一稳定实现，时间为 $\Theta(n+k)$，输出数组与计数数组合计辅助空间 $\Theta(n+k)$。它适合整数取值范围较小的情形；k 很大时，清零和累计计数本身就可能不合算。若只按次数把数值重新覆写回数组，没有保存相等记录的身份及附加数据，不能据此宣称对记录作了稳定排序。计数、基数利用关键字结构，不属于仅靠关键字两两比较来确定次序的模型，因而不直接受比较排序的 $\Omega(n \log n)$ 最坏下界约束。

8.10.3 LSD 和稳定计数的核心次序

```
LSD(records,d,r):
    从最低位到最高位执行d轮:
        清空r个桶
        按当前序列从前向后，把记录追加到对应桶尾
        按桶号递增、桶内先入先出收集
```

```
stableCounting(records,min,max):
    k=max-min+1; count[0..k-1]=0
    扫描记录并按key-min累计count
    对count求前缀和
    从后向前扫描原记录x:
        count[x.key-min]--
        B[count[x.key-min]]=x
    返回B
```

计数输出位置以 0 开始。输出的是整条记录而非只覆写关键字，才有条件讨论相等记录身份的稳定性；若取值范围很大，必须先衡量 k 所需空间与清零代价。

8.10.4 计数排序的课件代码

课程连接·外排瓶颈联系操作系统 I/O

外排优先按块读写，输出缓冲攒满再写可减少细粒度 I/O。多路需要更多输入缓冲，路数受实际内存约束；双缓冲与异步 I/O 还会改变可用块数和耗时模型。

```
void CountSort(int A[], int B[], int n, int k) {
    int i, C[k];
    for (i=0; i<k; ++i) C[i]=0;
    for (i=0; i<n; ++i) ++C[A[i]];
    for (i=1; i<k; ++i) C[i]+=C[i-1];
    for (i=n-1; i>=0; --i) {
        --C[A[i]];
        B[C[A[i]]]=A[i];
    }
}
```

本段采用 C99 变长数组，前提为 $k>0$、$n\ge 0$、$0\le A[i]<k$，输出数组 B 至少能容纳 n 项。扫描方向从后向前、每次减少该值的剩余位置，使同值记录的前后身份得以保持；这里用整数展示，排序完整记录时连同附加数据一起复制。调用结束后结果位于 B，A 仍是输入。

课件代码附件还给出支持负整数的版本：先求最小值 min、最大值 max，用 `A[i]-min` 映射到 `0..k-1`，$k=max-min+1$，动态分配计数和输出数组，完成后复制回 A 并释放。使用前还需检查空输入、取值范围计算是否溢出以及分配是否成功。数据 `4,2,2,-3,5,0,3,3,1` 排序为`-3,0,1,2,2,3,3,4,5`。

8.10.5 基数排序的链式队列分配

对已有 `next` 链接的记录，每个数位设置 r 个桶头 `front[]`和桶尾 `rear[]`。分配一个结点 p 时先保存其原后继，再把它摘下并置 `p->next=NULL`；桶空则同时令首尾为 p，桶非空则接在 `rear[digit]->next` 后并更新尾指针。收集时按桶号顺序把非空桶首尾相接，并清空本轮各桶指针。

因此同一桶中先出现的记录仍在前面，收集整个桶只需接首尾，遍历全部桶为 $O(r)$。若每次向桶头插入，桶内顺序会逆转，LSD 已排好的低位关系就被破坏。求递减结果时，分配仍按当前序列顺序进入桶尾，只把收集桶号改成 $r-1$ 到 0，桶内保持先入先出。

8.11 外部排序：归并段、缓冲与 I/O

外部归并先把可装入内存的一批记录排成有序初始归并段，再多路归并。内存可同时放 B 个块、每路至少 1 个输入缓冲块且留 1 个输出缓冲块时，最多支持 $B-1$ 路简单归并；双缓冲或额外选择结构还可能占更多空间。提高路数可以减少趟数，但不能忽略缓冲的实际容量。

若有 r 个初始段，固定 k 路平衡归并需要 $\lceil \log_k r \rceil$ 趟，最后一组可不足 k 路。每一趟把全部 N 个数据块读入并写出，通常为 2N 次块传输；生成初始段也读写全文件，若计入该阶段，总量为 $2N[1+\lceil \log_k r \rceil]$，这里不计寻道、缓存命中或最后输出直送等特殊节省。

例如 4500 条记录，每块 75 条，内存能容 450 条，则文件 60 块、内存 6 块。直接分批内排序可得到 10 个初始段，每段 450 条、6 块；若用 5 路归并，$10\to 2\to 1$，需要 2 趟归并。计初始段生成，总块传输 $2\times 60\times 3=360$ 次。记录数、块数、段数各起不同作用，不能把 4500 直接当作 I/O 次数。

8.11.1 课件中的 16 块外排实例

条件辨析·块与记录分开计

16 块文件每完整读写一趟是 32 次块传输。每块含多少条记录不应再次乘入已经按块统计的次数。

文件有 16 个磁盘块，内存用于生成初始段的工作区容纳 2 块，分 8 批内部排序，得到 8 个两块长初始段；这一阶段读 16 块、写 16 块。二路归并时，2 个输入缓冲块和 1 个输出缓冲块就能循环归并任意长的两个段：输入块读尽才补下一块，输出块写满才写盘。

二路平衡归并的段数为 $8 \to 4 \to 2 \to 1$，共 3 趟，每趟读写各 16 块，所以连同初始段生成总传输 $32 + 3 \times 32 = 128$ 块。改成 4 路，须 4 个输入块加 1 个输出块，段数 $8 \to 2 \to 1$，共 2 趟，总传输 $32 + 2 \times 32 = 96$ 块。节省来自全文件读写次数减少，并非每个磁盘块本身变小。

k 路平衡归并要求每趟把当前各段按至多 k 段一组合并，下一趟段数为 $\lceil r/k \rceil$；最后一组可以不足 k 路。严格 k 叉最佳归并树的补虚段规则服务于另一种按段长优化的树模型，不能据“有一组不足 k 路”否定它是 k 路平衡归并。

8.11.2 置换—选择为什么能产生长于内存的段

工作区保存当前仍可进入本段的记录。每次输出其中最小值 x，再读一个新记录 y：若 $y \geq x$，它仍可参加当前段；若 $y < x$，就把它冻结，留给下一段。可用记录耗尽时结束本段，把全部冻结记录解冻，重新开始。这里工作区只保持 m 条候选，输出段却可以随着不断换入而增长。

8.11.3 例 8-6　冻结记录在什么时候重新参加

例 8.11.1 (配套案例)

【配套例题】对含 $n > 0$ 个记录的文件外部排序，以置换—选择生成初始归并段，工作区能保存 m 个记录。

（1）文件 19 个关键字依次为 `51,94,37,92,14,63,15,99,48,56,23,60,31,17,43,8,90,166,100`，m=4 时生成几个初始段？各是什么？

（2）对任意 m（$n \gg m > 0$），第一个初始段长度最大、最小值分别是多少？

解析。先装入 51、94、37、92，输出 37 后读入 14，因为 14<37 而冻结；输出 51 后读入 63，63 仍可参加本段。前 6 次输出和换入如下：

输出	新读入	新记录归属
37	14	冻结
51	63	当前段
63	15	冻结
92	99	当前段
94	48	冻结
99	56	冻结

此时工作区 14、15、48、56 全被冻结，第一段结束。解冻后继续，同样逐次选最小；读入 17、43、8 和 100 时，它们分别小于当时最后输出的 31、48、56 和 166，留给下一段。最终得到 3 段：

1. `37,51,63,92,94,99`；
2. `14,15,23,31,48,56,60,90,166`；
3. `8,17,43,100`。

每个原记录恰好输出一次，三段长度 $6 + 9 + 4 = 19$，段内均递增。

第一个初始段最多可含全部 n 项，输入已经非降时即如此；最少为 m 项，例如输入严格递减，每个新读入都冻结，但最初 m 个仍要全部输出才能结束。答案是**最大 n、最小 m**，不是每段都恰好 m 或 2m。

条件辨析·块与记录分开计

16 块文件每完整读写一趟是 32 次块传输。每块含多少条记录不应再次乘入已经按块统计的次数。

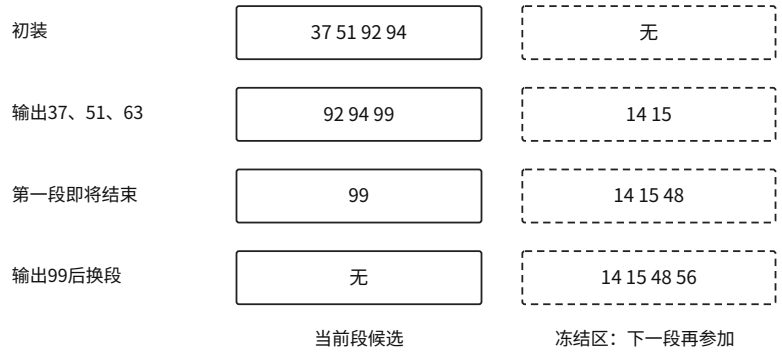


Figure 8.4: 实框为当前段候选，虚框为冻结记录；全部冻结时才换段

8.11.4 败者树怎样减少每次输出后的比较

k 路归并每次都要找各段当前首记录中的最小者。败者树把各路作为叶，内部结点记录一次比赛的败者，并额外保存总冠军的路号。冠军输出后，仅该路换入新首记录，它沿原比赛路径与保存的败者重新比较；每层只需一次关键字比较，更新约 $O(\log k)$。

例如 4 路首记录为 A:3、B:5、C:2、D:7，冠军是 C 的 2，内部保存 5、7 及最终败者 3。C 换入 6 后，先与 7 比较，再与 3 比较，冠军变为 A 的 3。这次更新不必重新比较未变化的 3 与 5。堆的下滤一般先比较两个孩子再与候选比较，败者树每层沿既定对手路径比较，过程不同；堆可以在中途已满足堆序时停止，败者树更新冠军路径则要传到总冠军位置。

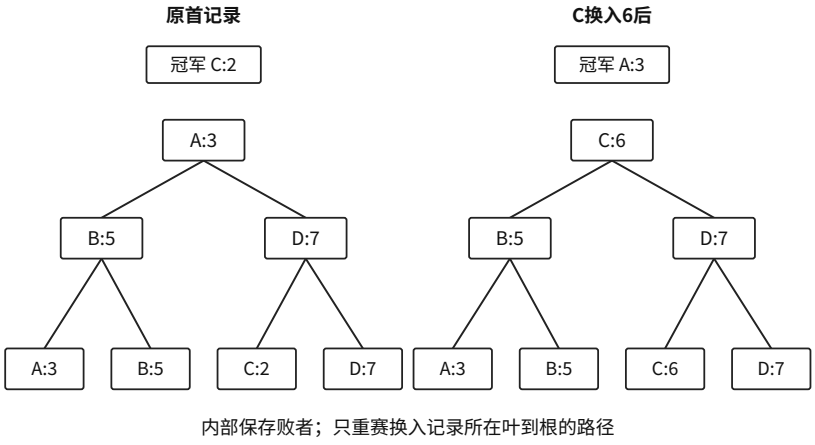


Figure 8.5: 两轮败者树；内部框存败者路号和值，冠军另外保存

8.11.5 例 8-7　不等长有序表怎样安排合并

例 8.11.2 (配套案例)

【配套例题】6 个升序表 A、B、C、D、E、F，分别含 10、35、40、50、60、200 个元素。通过 5 次两两合并成为一个升序表，要求最坏情况下比较总次数最小。

（1）给出完整合并过程及最坏比较总次数。（2）描述 N 个不等长升序表的合并策略并说明理由。

最小反例·输出段可以比内存更长

最小直观情形：工作区 1 条，输入 1、2、3。每输出一条再换入更大者，三条属于同一段，内存始终只存 1 条。不能把段长固定等同工作区容量。

解析。一张表每参加一次合并，它的全部元素就再次参与一层归并工作。让长表尽量晚参加、短表尽早参加，等价于最小化以表长为叶权的二叉归并树 WPL，即采用哈夫曼合并策略：

合并段长	新段长	本次最坏比较
10 与 35	45	44
40 与 45	85	84
50 与 60	110	109
85 与 110	195	194
195 与 200	395	394

最坏比较总数 $44+84+109+194+394=825$。合并长度之和即 WPL，为 830；每次归并比较至多“合并长度 -1”，共有 5 次，所以也可得 830−5=825。推广到 N 段，就是反复选当前两个最短段合并后放回候选集合，直到只剩一段，理由与哈夫曼最优树相同。

8.11.6 例 8-8　四路最佳归并为什么补零权段

例 8.11.3 (配套案例)

〖8.7-004〗12 个初始段长分别为 30、44、8、6、3、20、60、18、9、62、68、85。进行 4 路外部归并，画出最佳归并树并计算 WPL。

解析。满 4 叉归并树的叶数应满足 $(r-1) \bmod 3 = 0$。12 不满足，补 1 个零权虚段得到 13 个叶。每次取 4 个最小权值：

`0+3+6+8=17`，`9+17+18+20=64`，`30+44+60+62=196`，最后 `64+68+85+196=413`。

所以 WPL 为 $17+64+196+413=690$。虚段不含真实记录，补它是为了让第一次归并允许不足 4 个真实输入，不会平白增加文件数据量。若每次合并段都完整读写，且段长按块计，归并阶段总块 I/O 为 $2\times 690=1380$；这是按该模型计数，不包含此前生成初始段的读写。

8.11.7 外部归并和置换—选择的状态

```
mergeKRuns(runs):
    每路读入首缓冲块，另留输出块
    重复选择各非空路当前最小首记录，写入输出缓冲
    选中路的块用尽时读它的下一块
    输出块满时写外存；最后写出不足一块的剩余输出
```

```
replacementSelection(input,m):
    工作区读入至多m条，标记为当前段可用
    while 工作区非空:
        若无可用记录：结束当前段，全部冻结项解冻
        输出可用记录中最小的x
        若还有输入，读入y占用x释放的位置：
            若y<x则冻结，留下一段；否则本段可用
        否则删除该工作区位置
```

错项排除·下界没有否定计数排序

比较排序下界假设只凭两两比较区分互异关键字排列；计数索引利用整数取值范围，成本还含 k。若 k 很大，线性于 n 的口号忽略了计数数组和前缀和代价。

每个输入记录恰好换入并输出一次；冻结表示属于下一段，不能在当前段末尾直接按数值插回。工作区大小固定，输出段长度却可以继续增长，二者不是同一个量。

外部排序先统一单位再计 I/O

依次写记录数、每块记录数、内存块数、初始段数、路数与趟数。若题目包含初始段生成，另计一次全文件读写；若按归并树 WPL 计不等长段合并，则逐内部结点累计读写，不能再乘一个全文件的统一趟数。

8.12 怎样比较和选择排序算法

下表比较的是**完整排序的总时间**，同时计入比较、移动和循环工作，不把一次划分、一次下滤或只建堆当作完整排序。辅助空间不计输入记录本身，但计入递归栈和另开输出数组；Θ 表示同阶，O 只给上界。r 为基数，d 为固定执行的位数，k 为计数排序的取值范围大小。比较排序的平均列按互异关键字的等概率随机排列分析；随机枢轴快排也可对其随机选择取期望，这些都不等于每份输入的保证。

算法	最好时间	平均时间	最坏时间
直接插入	$\Theta(n)$	$\Theta(n^2)$	$\Theta(n^2)$
折半插入	$\Theta(n\log n)$	$\Theta(n^2)$	$\Theta(n^2)$
起泡	$\Theta(n)$	$\Theta(n^2)$	$\Theta(n^2)$
简单选择	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(n^2)$
希尔	依增量序列	依增量序列	依增量序列
快速	$\Theta(n\log n)$	$\Theta(n\log n)$	$\Theta(n^2)$
堆	$O(n\log n)$，见下文	$\Theta(n\log n)$	$\Theta(n\log n)$

算法	最好时间	平均时间	最坏时间
二路归并	$\Theta(n\log n)$	$\Theta(n\log n)$	$\Theta(n\log n)$
LSD 基数	$\Theta(d(n+r))$	$\Theta(d(n+r))$	$\Theta(d(n+r))$
计数	$\Theta(n+k)$	$\Theta(n+k)$	$\Theta(n+k)$

算法	辅助空间与稳定性	重要实现条件
直接插入	$O(1)$；稳定	只移动严格大于新项的记录
折半插入	$O(1)$；稳定	每趟都折半定位；插在旧相等项之后
起泡	$O(1)$；稳定	有无交换标志可提前停；仅逆序相邻项交换
简单选择	$O(1)$；不稳定	每趟选最小后与首位交换；比较固定
希尔	$O(1)$；不稳定	逐组插入，末增量为 1；未给增量不填统一时间阶
快速	栈平均 $O(\log n)$，最坏 $O(n)$；不稳定	本章单枢轴划分与双支递归；平均需随机模型
堆	$O(1)$；不稳定	自底向上建堆、迭代下滤；最好紧界有条件

算法	辅助空间与稳定性	重要实现条件
二路归并	$\Theta(n)$；稳定	数组常规归并，不跳过已有序段；相等先取左段
LSD 基数	链式额外 $O(r)$；数组 $O(n+r)$；稳定	固定执行 d 轮；同桶保序；链式原记录已有链接域
计数	$\Theta(n+k)$；稳定	计数、前缀和、逆向放入输出数组的记录实现

堆排序的教材汇总表在关键字互异的常规模型下，把最好、平均、最坏都列为对数线性阶；本表最好一栏保留总时间上界，并提醒具体停机条件：若允许所有关键字相等，下滤比较到相等即停止，建堆和全部输出可只做 $\Theta(n)$ 工作。不能把教材汇总表的最好紧界无条件套到这类重复输入上，也不能因这一特例删去堆排序的 $\Theta(n\log n)$ 最坏时间。起泡若无提前停止标志，则有序输入仍执行固定二次比较；归并若另加跳过有序段的优化，最好情况也应随实现重算。

规模小、接近有序时，直接插入移动少、控制简单；需要可靠的最坏对数线性时间且额外空间有限时，可考虑堆；要求稳定而可用线性辅助区时，可用归并；关键字可分成少量固定位，或整数取值范围较小时，可分别比较基数排序、计数排序的代价。外存排序则优先设计初始段和多路归并，不能只把内部快排换到磁盘上逐元素访问。

移动、比较和趟数不能互相替代。简单选择比较量不受初始次序影响，交换次数却依实现而定，某次交换可以在代码中省去；折半插入比较量变化小，移动量仍依赖初始排列；LSD 按固定 d 位执行时，分配收集的轮数固定；快速排序的划分深度取决于划分结果。并行能力也依实现：快排两子区间可独立，归并同层不同段可独立；LSD 的下一位依赖本位稳定结果，但每一位内部仍可设计并行分配。不能把某个串行教材步骤的依赖，扩大成算法在任何实现下“绝不可能并行”。

定理 8.12.1 (比较排序下界的适用模型)

仅通过两两关键字比较区分 n 个互异关键字的全部排列时，最坏比较次数为 $\Omega(n\log n)$。

决策树至少要容纳全部排列

互异关键字有 $n!$ 种相对次序，一棵二分比较决策树至少需要 $n!$ 个叶。高度为 h 的二叉树至多 2^h 个叶，所以 $h \geq \lceil \log_2(n!) \rceil = \Omega(n\log n)$。计数与基数利用取值或数位结构，操作不只在这棵两两比较树中分支，故不与其线性特殊模型矛盾。

错项排除·下界没有否定计数排序

比较排序下界假设只凭两两比较区分互异关键字排列；计数索引利用整数取值范围，成本还含 k。若 k 很大，线性于 n 的口号忽略了计数数组和前缀和代价。

□

错项排除·下界没有否定计数排序

比较排序下界假设只凭两两比较区分互异关键字排列；计数索引利用整数取值范围，成本还含 k。若 k 很大，线性于 n 的口号忽略了计数数组和前缀和代价。

8.13 主动回忆自测

第 2 题为配套例题，其余为自拟补充，检查模型变化。

1. 直接插入把大于条件改为大于等于，时间阶没变，哪项性质可能改变?
2. 【配套例题】n（n>100000）个数保存在一维数组 M 中，需要查找最小的 10 个数。（1）设计算法，要求平均情况下比较次数尽可能少，简述思想，不需要程序实现。（2）说明平均时间复杂度和空间复杂度。
3. 100 个初始段，最多 4 路归并，需要几趟？若每趟完整读写 500 块，归并阶段多少块传输?
4. 一个不稳定排序在没有相等关键字的输入上，能否观察到稳定性差异?
5. 直接选择用严格小于找最小值，为什么仍可能不稳定？给最小三记录反例。
6. 折半插入将相等时的分支改为向左找，会影响比较次数的阶还是稳定性?
7. 两个长度分别为 2、3 的有序段，归并一定需要 4 次关键字比较吗?
8. 置换—选择工作区 4 条，输入已递增的 100 条记录，第一段最多多长？内存是否因此装入 100 条?

答案。1. 可能破坏稳定性，新相等记录移到旧记录前。2. 用随机枢轴划分，把第 10 小作为选择目标，只继续处理包含这一秩的子区间；其余一侧无需继续排序。确定分界后，左侧保留 10 个最小数，边界有相等值时按还需个数选取。平均时间 $O(n)$，在原数组上迭代选择可用 $O(1)$ 辅助空间；题目不要求这 10 个数内部有序。也可维护大小 10 的大根堆：只用小于堆顶的数替换并下滤，适合流式读入，代价为 $O(n \log 10)$、辅助 $O(10)$，不能把它误写成必须完整排序。3. $\lceil \log_4 100 \rceil = 4$ 趟，传输 $4 \times 2 \times 500 = 4000$ 块；若包含初始段生成，再加 1000 块。4. 不能。输入中不存在相等记录，但算法的稳定性定义仍按所有合法输入判断。

5. $2_a, 2_b, 1$ 第一趟把 1 与 2_a 远距离交换，得到 $1, 2_b, 2_a$，破坏稳定性；找最小值时没有因相等更新 pick 也无法避免。
6. 比较阶通常不变，但把新相等记录插到旧相等记录之前，稳定性可能破坏。
7. 不一定，4 只是最坏上界；若较短段的所有值都小于另一段，只比较 2 次就直接复制余段。
8. 可长 100 条；输出一条就换入一条，工作区始终最多 4 条，段长不等于同时驻留的记录数。

计算机系统与程序执行的层次 1

章节脉络

计算机系统将硬件、软件和运行规则结合起来，使程序能够按确定的步骤处理数据。本章从功能部件出发，沿存储程序与指令执行理解各层接口，再用 CPU 时间、吞吐量和容量计算分析系统性能。

课程连接·功能部件接到数据通路

把 c=a+b 分成取 a、取 b、相加、存 c 后，第 5 章会逐步标出寄存器传送和控制信号。这里的功能划分仍成立；一块芯片可以同时实现多种功能，不能按芯片数倒推五部件。

1.1 从程序看计算机的功能部件

定义 1.1.1 (概念与适用对象)

计算机系统由硬件和软件共同构成。硬件提供保存信息、执行运算、控制步骤和输入输出的物理装置，软件用程序和数据规定装置怎样完成任务。仅凭处理器芯片无法判断一个应用能否运行；还要看指令集、操作系统、库及文件格式等条件。

计算机的五个基本功能部件是运算器、控制器、存储器、输入设备和输出设备。这个分类按**功能**划分，不是按芯片这一物理实体划分。CPU 通常包含运算器、控制器和寄存器，还可能集成若干级 Cache；一个存储控制器也可能集成在处理器芯片内。

部件	在执行 c=a+b 时做什么
运算器	对取来的两个数进行加法，形成结果及有关标志
控制器	解释指令，在合适时刻发出选择、读写和运算等控制信号
存储器	保存指令、a、b、c 及执行所需的其他信息
输入、输出设备	将系统外部的信息送入系统，或把系统内的结果送出

中央处理器 CPU 的数据通路包括寄存器、运算部件及其连接，控制器决定数据在通路上何时、向哪里流动。第 5 章将把一次加法展开为寄存器传送；这里先把 “处理数据” 和 “控制步骤” 分清。

主存储器能由 CPU 直接按地址访问，程序执行时所需的指令和数据通常要进入主存。磁盘、SSD 等外存提供较大且非易失的存储空间；文件保存在磁盘，并不意味着 CPU 每执行一条指令都直接向磁盘取一次。

最小反例·同 ISA 不保证文件通用

最小反例：两系统使用同一指令集，但系统调用号、动态库和可执行文件格式不同。一份应用文件仍可能无法直接运行。ISA 兼容解决指令语义，运行环境兼容还需操作系统与 ABI 条件。

1.1.1 ISA 规定什么，微体系结构决定什么

指令集体系结构 ISA 是软件可见的机器接口，规定机器指令的功能和编码、软件可见的寄存器、数据类型、寻址方式以及相关的存储访问规则。微体系结构则指在该接口约束下怎样组织与实现机器，例如采用几级流水线、怎样转发数据、Cache 怎样实现、加法器采用何种进位结构。

同一个整数加法的机器指令，可以由不同硬件通路实现。只要软件可见的效果符合 ISA，程序不必知道内部用了串行进位还是先行进位。不过，机器码能否执行还取决于运行环境；“同一 ISA” 不自动保证不同操作系统的可执行文件可直接互用。硬件与软件能实现相同逻辑功能，也不表示两者的时间、成本和可修改性相同。

1.2 存储程序怎样变成一次次指令执行

存储程序的基本思想是：执行前把程序指令和数据存入存储器，运行时自动取出指令、解释并执行。指令和数据都可以用二进制编码，机器根据当前操作和访问位置解释其含义。同一串位的 “指令身份” 或 “数据身份” 都离不开上下文。

冯·诺依曼型计算机按指令控制执行过程，常称控制流驱动。遇到转移、调用或异常中断时，下一步不再简单地取顺序地址上的指令，但仍由确定的控制机制改变执行位置。“存储程序” 并不要求程序所在的存储内容在指令执行期间保持不变。

程序计数器 PC 保存指令执行所需的地址信息。最小的顺序执行过程可以写成：

1. 根据 PC 访问指令所在存储单元，取得指令。
2. 解释操作码和有关字段，取得操作数。
3. 执行运算、访存或控制转移，保存结果。
4. 按顺序地址或转移目标更新执行位置，继续取指。

这里的顺序地址增量取决于指令长度和地址单位。按字节编址、指令长 4 B 时，顺序地址通常增加 4；不能把所有机器的 PC 更新都写成 “加 1”。流水线中取指、译码、执行还可能重叠，程序员看到的指令语义与内部时序应分开。

1.2.1 从 C 源文件到 CPU 能执行的内容

在常见的静态编译流程中，预处理展开头文件和宏等指令；编译把高级语言转换为汇编形式；汇编把汇编指令编码成可重定位目标文件；链接合并目标文件与所需库，处理符号引用和重定位，形成可执行文件。装入程序再按可执行文件的组织方式建立进程的地址空间，准备执行入口。

汇编语句 `add` 是便于人阅读的符号，CPU 取到的是其机器编码。宏、伪指令还可能展开为零条、一条或多条机器指令。一条 C 语句也可能展开为多条指令；反过来，编译优化可能把多条 C 语句合并或删除。

1.2.2 例 1-1　硬件直接执行哪一种程序

例 1.2.1 (配套例题)

【配套例题】计算机硬件能够直接执行的是（　）。

I. 机器语言程序　II. 汇编语言程序　III. 硬件描述语言程序

A. 仅 I　B. 仅 I、II　C. 仅 I、III　D. I、II、III

解析。执行普通指令的硬件按 ISA 识别机器编码，因此 I 成立。汇编语言必须经过汇编，硬件描述语言用于描述和设计硬件，不能作为普通机器指令直接送入 CPU 执行。答案为 **A**。题目问的是执行接口，而不是哪种语言更接近硬件。

解释执行是另一种程序执行方式：解释器读取源程序或中间表示，并在运行时完成相应操作。实际解释器本身仍由机器指令运行。现代语言实现也可能结合解释、预编译和即时编译，不能用 “高级语言都逐行解释” 概括。

不变量·先数动态执行次数

总周期数来自真正执行过的每一条指令。循环体写一份不等于只执行一次；分支未走到的代码不贡献本次动态指令数。比较性能时，要让任务、输入和计数区间保持相同。

1.3 性能计算：先确定谁的时间在改变

响应时间是从提出任务到得到结果经历的时间；吞吐量是单位时间内完成的任务量。增加处理器可以让多个独立任务同时进行，因而可能提高系统吞吐量；一个不能并行的单线程任务却不因此自动缩短执行时间。

程序的总运行时间还可能包含 I/O 等待、系统调度等。CPU 时间是 CPU 为该程序工作所花的时间。本节的基本性能式只计算指定口径的 CPU 时间：

$$T_{\text{CPU}} = \frac{N \times \overline{\text{CPI}}}{f} = N \times \overline{\text{CPI}} \times T_{\text{clk}}.$$

N 是实际执行的指令条数，不能用程序文件中的静态指令条数代替；f 是主频，单位 Hz；$T_{\text{clk}} = 1/f$ 是时钟周期；平均 CPI 是每条指令平均占用的时钟周期数。若第 i 类指令执行 n_i 条、平均 CPI 为 c_i，则

$$\overline{\text{CPI}} = \frac{\sum_i n_i c_i}{\sum_i n_i}.$$

不同类指令的 CPI 必须按条数加权平均。CPI 还受 Cache 缺失、流水线停顿等影响，并非仅由指令操作码决定。流水线稳态下 CPI 可以接近 1；多发射处理器的平均 CPI 也可以小于 1，这并不表示一条指令的全部处理延迟小于一个时钟周期。

1.3.1 例 1-2　主频相同，怎样由指令组成求时间

例 1.3.1 (配套例题)

【配套例题】某计算机主频为 1 GHz，程序 P 运行过程中共执行 10000 条指令，其中 80% 的指令执行平均需 1 个时钟周期，20% 的指令执行平均需 10 个时钟周期。程序 P 的平均 CPI 和 CPU 执行时间分别是（　）。

A. 2.8，28 μs　B. 28，28 μs　C. 2.8，28 ms　D. 28，28 ms

解析。条件给的是动态指令比例，先求加权平均：$0.8\times1+0.2\times10=2.8$。总周期数为 $10000\times2.8=28000$；1 GHz 对应每周期 1 ns，因此 CPU 时间为 28000 ns，即 28 μs。答案为 **A**。百分比必须乘各类周期数，GHz 与 μs 的换算不能略去。

1.3.2 例 1-3　CPU 加快后，I/O 时间怎样处理

例 1.3.2 (配套例题)

【配套例题】假定基准程序 A 在某计算机上的运行时间为 100 s，其中 90 s 为 CPU 时间，其余为 I/O 时间。若 CPU 速度提高 50%，I/O 速度不变，则运行基准程序 A 所耗费的时间是（　）。

A. 55 s　B. 60 s　C. 65 s　D. 70 s

解析。“速度提高 50%”是原速度的 1.5 倍，因此原来的 90 s 变为 $90/1.5=60$ s，而不是减为 45 s。I/O 仍占 10 s，总时间为 70 s，选 **D**。

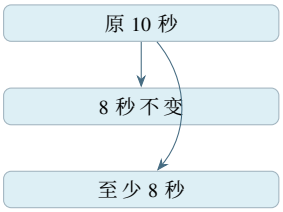
一般地，原运行时间中比例 p 的部分加快为原速度的 s 倍，其余不变，且两部分时间可以这样相加时，加速比为

$$S=\frac{1}{(1-p)+p/s}.$$

该式的上限是 $1/(1-p)$，但前提是加速部分划分和时间模型成立。若 I/O 与计算本来重叠，或并行化新增了通信与同步开销，应先画出实际时间关系，不能直接把各部分都相加。

最小反例·速度翻倍不等于总时减半

最小反例：10 秒任务中只有 2 秒可加速，即使这 2 秒缩短为 0，总时间仍至少 8 秒。只有整个统计对象都按相同倍率加速，总历时才按该倍率缩短；先画出未改变的部分。



可加速部分即使归零，总时间仍不小于 8 秒。

1.3.3 比较性能时，为什么不能只看主频或 MIPS

两台机器的时间之比同时取决于指令条数、平均 CPI 和主频。相同源程序在不同 ISA、不同编译器下可能有不同的动态指令条数。即使平均 CPI 相等，也不能仅凭主频断言谁完成同一任务更快。

性能变化也应放在同一任务与同一统计口径下比较：编译优化可能同时减少动态指令条数、改变指令组成；改善 Cache 通常减少访存停顿，从而改变平均 CPI；提高主频改变每周期时间，但若伴随更多流水级或等待，CPI 也可能变化；增加核心则先要问任务能并行到什么程度。只有题干明确其他量不变时，才把时间比约成某一个因子的比例。

这些变化还对应不同系统层次。ISA 规定一条指令对软件可见的效果，编译器决定怎样用指令表达任务，微体系结构决定这些指令怎样在资源和时序上实现，操作系统决定进程可使用的运行环境与资源。问机器码能否执行，要查 ISA 和运行环境；问同一机器码为何快慢不同，要沿动态指令数、CPI、主频和等待时间寻找变化。用同一个“机器更先进”概括，无法支持具体推算。

MIPS 表示每秒执行多少百万条指令，$\text{MIPS}=f/(\overline{\text{CPI}}\times10^6)$；IPC 是每周期平均完成的指令数，在统计区间一致时与平均 CPI 互为倒数。MFLOPS 等指标统计浮点运算次数，与机器指令条数不是同一量。跨机器比较应优先比较同一任务的实际时间，再看指标如何解释差异。

机器字长主要反映 CPU 一次处理整数数据的基本位宽。它关系到整数表示范围和相关运算能力，但不能单独推出主存容量、总线宽度或某程序的速度。“64 位机器”也不表示所有数据类型都占 64 位。

1.4 从层次图走到可核算的模型

性能题四步

1. 写清任务和统计范围：CPU 执行时间、总历时还是单位时间完成数。
2. 数动态指令，按条数加权求 CPI；把 GHz、ns、微秒统一。
3. 标出变化因子：指令数、CPI、主频或可并行比例；不变条件必须来自题设。
4. 用原方案回代，再检查加速后的总时间是否仍包含不可加速部分。

最小反例·字长不能推出地址容量

同样是 32 位整数处理器，可以配较小物理内存，也可以有更宽物理地址。最小反例已表明 “字长 32 位所以最多 4GiB” 缺少地址位数前提。容量必须由地址数与每地址对应的单位共同确定。

1.4.1 为什么 CPI 按条数加权

从总周期数出发

第 i 类指令执行 n_i 条，每条占 c_i 周期，故该类总周期为 $n_i c_i$；各类相加得到 $C = \sum_i n_i c_i$。平均每条周期必为 $C/(\sum_i n_i)$，因此权重是指令条数比例，而非各类指令所占时间比例。乘上共同的时钟周期，才得到 CPU 时间；若各阶段频率不同，应分阶段计算。

□

层次结构图中的 “机器” 是一种抽象视角。高级语言机器依靠编译或解释实现，汇编语言机器依靠汇编工具，操作系统提供资源与服务接口，传统机器执行 ISA 指令，微程序机器描述某类控制器内部实现。并非所有 CPU 都采用微程序控制；不可把五层图误读成每个程序必经五种独立硬件。

1.4.2 从地址位数到容量

若有 m 位完整地址、按字节编址且所有地址均可用，则最多可区分 2^m 个字节位置；若按 w 位字编址，容量应为 $2^m \times w$ 位。MDR 或数据总线宽度描述一次交换的数据位数，不能单独代替地址位数。以 20 位字节地址为例，可编址 1MiB；若同为 20 位地址却每地址对应 16 位字，则可编址 2MiB。

课程图中容量常按 2^{10} 递增，而频率、传输率通常按 10^3 递增。为避免含混，容量推导写成 KiB/MiB 或明确 “本题 K=2^{10}”；带宽则把每次字节数乘每秒次数，最后再按题设进制换单位。

1.4.3 发展历程的背景定位

电子管、晶体管、中小规模集成电路、大规模与超大规模集成电路依次推动器件集成、可靠性和成本变化。它们帮助理解实现技术的演变；性能判断仍要回到具体任务的执行时间。教材与课件把部分发展史标为参考内容，本章保留这一背景，不以年代细节替代核心性能模型。

层次	向上说明什么	不能直接推出什么
ISA	软件可见指令与状态	具体流水段数、Cache 组织
微体系结构	指令在部件和时序上的实现	应用跨操作系统直接兼容
操作系统	资源、进程与运行接口	所有应用可并行加速

1.5 五大部件怎样共同完成一次计算

课程连接·五部件的连接

存储器提供指令与数据，控制器协调传送与运算；同一条指令还需明确数据经过哪些寄存器。

冯·诺依曼计算机把指令和数据都存放在存储器中，用地址选择存储单元。指令与数据在存储形式上都表现为二进制位串；CPU 取出的内容被当作指令还是操作数，取决于当前执行阶段和指令语义。早期机器以运算器为中心，外设和主存之间的数据常经运算器；现代组织通常以存储器为中心，外设可借助接口、总线和 DMA 与主存交换数据。

功能部件	基本任务	与程序执行的关系
运算器	算术、逻辑运算以及移位	ALU 完成运算，寄存器保存操作数和中间结果
控制器	取指、分析指令、产生控制信号	决定何时传送数据、执行哪种运算以及下一步做什么
主存储器	保存正在使用的指令与数据	提供按地址读写的存储单元
输入设备	将外部信息转换成机器能处理的形式	经接口把程序和数据送入系统
输出设备	将处理结果转换成人能识别或外部需要的形式	经接口接收输出数据和控制信息

主机通常指 CPU 和主存；CPU 包括运算器与控制器。辅存、输入设备、输出设备合称外部设备。按功能划分的部件与按物理芯片划分的部件未必一一对应：一个 CPU 芯片可以包含多个核、多级 Cache 和其他控制逻辑。

1.5.1 存储字、机器字和指令字的区别

一个存储单元中保存的位串称为存储字，位数为存储字长。机器字长一般指 CPU 一次能处理的二进制数据位数，常与通用寄存器、运算部件的数据通路宽度相关。指令字长是某条指令所占的位数；一种 ISA 可以采用固定长度或可变长度的指令。

设按字编址的主存共有 2^{10} 个单元、每单元 16 位，则容量为 $2^{10}\times 16$ bit，即 2 KiB；地址至少需要 10 位。若同样的 2 KiB 容量改按字节编址，则共有 2^{11} 个可编址单元，地址至少 11 位。求容量时先写“单元数 $\times$ 每单元位数”，能避免把地址线数与数据线数互换。

运算器中的 ACC 可暂存操作数或结果，MQ 可辅助保存乘数或乘积低位，X 可暂存另一个输入；这些是课件示意机器的组织约定。现代机器未必使用完全相同的专用寄存器，分析具体数据通路仍应以题设为准。

1.5.2 例：从主存中的程序求出 $y=ab+c$

设主存按字编址，地址 0～4 依次保存取数、乘法、加法、存数和停机指令；地址 5～8 分别存放 $a=2$、$b=3$、$c=1$ 和 $y=0$。取指时，PC 给出的是指令地址；执行访存指令时，地址字段给出的是数据地址。

执行的指令	主要数据变化	完成后的状态
0：取地址 5 中的数	$M[5]\rightarrow$ ACC	ACC=2，PC 指向 1
1：乘地址 6 中的数	ACC $\times M[6]\rightarrow$ ACC	ACC=6，PC 指向 2
2：加地址 7 中的数	ACC $+M[7]\rightarrow$ ACC	ACC=7，PC 指向 3
3：将结果存入地址 8	ACC $\rightarrow M[8]$	$M[8]=7$，PC 指向 4
4：停机	停止继续取执行指令	计算结束

同一条访存指令至少要区分 “取指令” 和 “取操作数”：前者由 PC 形成地址，后者由 IR 的地址字段及寻址规则形成地址。MAR 在不同时刻可保存这两类地址，MDR 在不同时刻可保存指令或数据；寄存器名称并未限定其中只能保存一种内容。

课程连接·程序如何到机器上

编译得到目标代码，链接处理符号与所需模块，加载将程序置入内存；不要把这三个阶段混为一次操作。

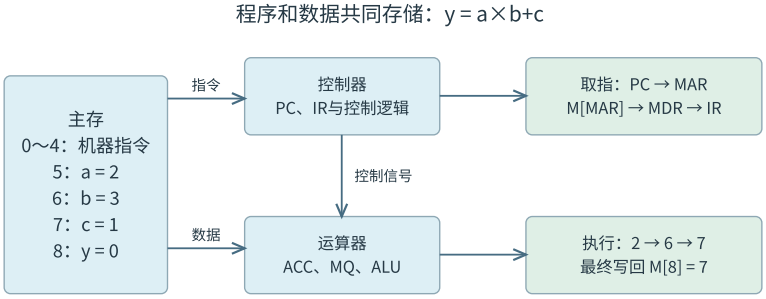


Figure 1.1: 主存按地址同时保存程序与数据，控制器组织取指与执行

1.6 系统软件、应用软件与程序翻译

系统软件负责管理资源、提供运行环境和支撑程序开发，包括操作系统、语言处理程序、数据库管理系统及有关工具；应用软件面向具体任务。编译程序把高级语言程序翻译成目标语言程序，解释程序则在解释过程中组织执行，高级语言语句与机器指令通常不是一一对应关系。

软件与硬件的逻辑功能等价，是指同一逻辑功能可以用软件算法实现，也可以增加专门硬件实现。选择哪种实现取决于性能、成本、功耗和灵活性。它不意味着软件无需硬件，也不意味着两种实现的时间、空间开销相同。

从下向上可以把系统理解为微程序机器级、传统机器语言级、操作系统级、汇编语言级和高级语言级。层次中的 “虚拟机器” 是下层向上层提供的一套使用规则；某一功能在下层实现时，对上层可以是透明的。“透明” 在体系结构语境中通常指使用者无需看见或处理其实现细节。

1.7 性能指标的统计口径与单位

主频 f 与时钟周期 T 互为倒数。对含有多类指令的程序，若第 i 类执行 n_i 条、每条平均需要 CPI_i 个周期，则

$$N=\sum_i n_i,\qquad C=\sum_i n_i\mathrm{CPI}_i,\qquad t_{\mathrm{CPU}}=\frac{C}{f}.$$

吞吐量衡量单位时间完成多少工作，响应时间衡量从请求开始到结果完成经历多久。提高并行度可能增加吞吐量，而单个请求的响应时间还受到等待、通信和 I/O 影响。比较两台机器运行同一任务的性能时，性能比等于执行时间的反比。

MFLOPS 表示每秒百万次浮点运算，GFLOPS、TFLOPS 分别采用 10^9、10^{12} 量级；它们侧重浮点工作负载。MIPS 为每秒百万条指令，依赖 ISA、编译结果和指令组成。容量中的 KiB 采用 2^{10} B，而频率中的 MHz 采用 10^6 Hz，计算时应保持量纲一致。

1.7.1 拓展：硬件技术的演进

电子管、晶体管、中小规模集成电路、大规模和超大规模集成电路是常用的发展阶段划分。集成度提高使存储容量、运算速度和可靠性提升，并推动微型化、多核、网络化以及高性能并行计算。摩尔定律描述历史上集成电路晶体管数量增长的经验趋势，不能直接据此把某个程序的运行时间按相同比例缩短。

课程连接·自查顺序

先写公式和单位，再代入数值；题目限定的工作负载决定可以比较的指标。

1.7.2 计算与辨析自查

1. 为什么同一个 MDR 既能接收指令又能接收数据?
2. 主频 2 GHz，运行 10^8 条指令，平均 CPI 为 1.5，CPU 时间是多少?
3. 4 KiB 主存、存储字长 32 位，分别按字和按字节编址需要几位地址?

答案。1. MDR 保存存储器本次交换的数据位串，内容的用途由执行阶段决定。2. $10^8 \times 1.5/(2 \times 10^9) = 0.075$ s。3. 按字编址有 1024 个单元，需 10 位；按字节编址有 4096 个单元，需 12 位。

1.8 主动回忆自测

1.8.1 配套自测

1. 【配套例题】程序 P 在机器 M 上的执行时间是 20 s。编译优化后，P 执行的指令数减少到原来的 70%，而 CPI 增加到原来的 1.2 倍，则 P 在 M 上的执行时间是（ ）。A. 8.4 s　B. 11.7 s　C. 14 s　D. 16.8 s。
2. 原创补充：两个任务都由同一单核 CPU 各执行 2 s。换成两核后，每个任务仍占用一个核执行 2 s，且可同时开始。两个任务的总完成时间和单个任务的响应时间分别怎样变化?

答案。1. D。主频不变，时间变为 $20 \times 0.7 \times 1.2 = 16.8$ s。2. 两个任务全部完成的时间从 4 s 降为 2 s；无排队时单个任务本身的执行时间仍为 2 s。该小例用于区分吞吐量改善与单任务加速。

1.8.2 遮住答案复述知识关系

主动回忆·自拟

1. 不看正文，画出五大功能部件，并圈出 CPU 与主机各自包含什么。
2. 同一 ISA 的两台机器为什么能执行同一类机器指令，却可能有不同 CPI?
3. 一段程序 CPU 时间占 80%，只把 CPU 部分加速为 2 倍，总加速比是多少?
4. 20 位地址能否直接推出主存为 1MiB? 还缺哪个条件?
5. 用一句话分别说明响应时间、吞吐量、IPC 和 FLOPS 的统计对象。

1.8.3 参考答案

1. 五部件为输入、输出、存储、运算、控制；常见模型 CPU 含运算与控制及寄存器，主机还含主存。

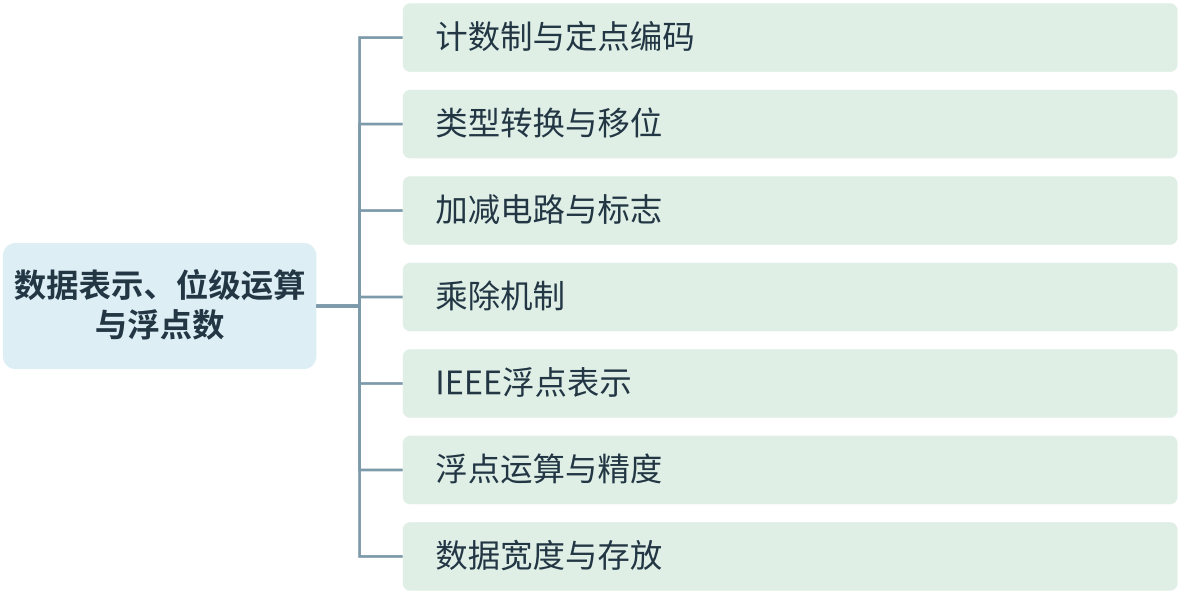
2. ISA 规定可见语义，流水线、Cache、功能部件与等待时序由具体微架构决定。
3. 原时间记 1，变为 $0.2 + 0.8/2 = 0.6$，加速比 $5/3$。
4. 须说明按字节编址、完整 20 位地址均可用；已安装物理容量还可能更小。
5. 响应时间按单任务历时，吞吐按单位时间完成任务数，IPC 按周期完成指令数，FLOPS 按秒完成浮点运算数。

课程连接·自查顺序

先写公式和单位，再代入数值；题目限定的工作负载决定可以比较的指标。

数据表示、位级运算与浮点数 2

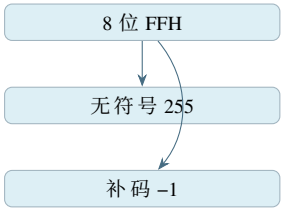
章节脉络



本章围绕“计数制与定点编码 → 类型转换与移位 → 加减电路与标志 → 乘除机制 → IEEE 浮点表示 → 浮点运算与精度 → 数据宽度与存放”展开。先判断对象与条件，再沿机制推导结果，最后用反例和数值检验结论。

最小反例·最高位 1 并不总是负数

最小反例：同一 8 位串 FFH 按无符号解释是 255，按补码解释是 −1。编码和位宽必须与位串一同给出。直接把最高位称作负号，会把无符号扩展、浮点阶码和地址都误读成有符号数。



同一位串依编码得到不同真值。

2.1 位串与数值之间还差一个解释规则

定义 2.1.1 (概念与适用对象)

位是取 0 或 1 的二进制位，字节 B 在本讲义中为 8 位，bit 写作 b。同一位串可以解释成无符号整数、补码整数、浮点数或字符编码。计算前必须同时确定**位宽、编码和小数点位置**，不能看见最高位为 1 就断言“这个数是负数”。

r 进制采用位置权，$(a_k \cdots a_0.a_{-1} \cdots)_r$ 的值是 $\sum_i a_i r^i$，其中 $0 \le a_i < r$。二进制与八进制、十六进制转换时，从小数点向两侧分组：每 3 位一组对应一位八进制，每 4 位一组对应一位十六进制；整数端左侧补 0，小数端右侧补 0。

2.1.1 例 2-1　整数与小数分别换基数

例 2.1.1 (配套例题)

【配套例题】若十进制数为 137.5，则其八进制数为（　）。

A. 89.8　　B. 211.4　　C. 211.5　　D. 1011111.101

解析。$137 = 2\times 8^2 + 1\times 8 + 1$，小数部分 $0.5\times 8 = 4$，因此为 $(211.4)_8$，选 **B**。八进制数字只能是 0 至 7，A 本身就不是合法八进制数。分组也可验证：$137.5 = (10001001.1)_2 = (010\ 001\ 001.100)_2$。

十进制整数转二进制可反复除 2 取余、倒序排列；小数则反复乘 2 取整数、顺序排列。有限十进制小数未必能写成有限二进制小数，例如 0.1 的二进制表示无限循环。保存有限位时会发生近似，这正是后面浮点舍入出现的原因之一。

不变量·范围来自位权

补码最高位权为负，其余位权为正；最小值只有最高位 1，最大值只有最高位 0。每次推范围先写位权，就能同时解释零唯一、负数多一个，以及最小负数同宽取负不可表示。

2.2 定点编码：范围先于运算规则

真值是按通常数学符号写出的数值；机器数是采用约定编码保存的位串。定点表示约定小数点位置固定，并不表示存储器真的另存了一个小数点。

对 n 位整数，常用编码如下。

编码	最高位和数值含义	表示范围
无符号	所有位均有正权	0 至 2^n-1
原码	最高位为符号，其余为绝对值	$-(2^{n-1}-1)$ 至 $2^{n-1}-1$，两个零
反码	正数同原码，负数的数值位取反	与原码相同，两个零
补码	最高位权为 -2^{n-1}，其余为正权	-2^{n-1} 至 $2^{n-1}-1$，一个零

8 位补码 `1111 1101` 的值为 $-128+64+32+16+8+4+1=-3$。也可先把它作为无符号数 253，再因最高位为 1 减去 256，得到 -3。后一方法适合十六进制机器数：对 n 位补码，若无符号解释为 u，则最高位为 0 时取 u，为 1 时取 $u-2^n$。

求一个负整数的补码时，可在给定位宽下把它的绝对值逐位取反再加 1；位宽不能省。8 位中 -128 编码为 `1000 0000`，它的数学相反数 128 不能用 8 位补码表示。“取反加 1” 得到的仍是同一位串，这说明运算按模 256 进行，并不表示已经得到可表示的正 128。

原码中的 `1000 0000` 表示 -0，补码中则表示 -128。反码的 -0 是 `1111 1111`。对于常见的偏置 2^{n-1} 的整数移码，有 $E = x + 2^{n-1}$，与对应补码相比只需翻转最高位；IEEE 754 阶码的偏置通常为 $2^{k-1}-1$，不可套用 “翻转最高位” 这一特殊关系。

若采用一位符号、$n-1$ 位小数的补码定点小数，最高位权为 -1，其余权从 2^{-1} 至 $2^{-(n-1)}$，范围为 $[-1, 1-2^{-(n-1)}]$。同样的位串改变小数点位置，真值就按 2 的幂缩放。

2.3 截断、扩展与 C 类型转换

位宽变小时，硬件截取低位；能否保留原数值，要检查被丢掉的位。位宽增大时，无符号数在高位补 0，有符号补码数复制符号位。原码、反码各有自己的扩展规则，补码规则并不适用于所有编码。

8 位补码 -3 扩为 16 位是 `1111 1111 1111 1101`，无符号 253 扩为 16 位则是 `0000 0000 1111 1101`。扩展后低 8 位相同，真值不同。**扩展方式首先取决于源数怎样解释，而不是目标类型名字中有没有 unsigned**。

2.3.1 例 2-2 无符号短整数扩展

错项排除·扩展看源数解释

FFFAH 若是 unsigned short 则为 65530，扩宽补 0；若是 signed short 补码则为 −6，扩宽复制符号。目标名字中出现 unsigned 不代表先补 0：数值转换还需按目标模数处理。

例 2.3.1 (配套例题)

【配套例题】假定编译器规定 int 型和 short 型长度分别为 32 位和 16 位，执行下列 C 语言语句：

```
unsigned short x = 65530;
unsigned int y = x;
```

得到 y 的机器数为（ ）。

A. 0000 7FFAH B. 0000 FFFAH C. FFFF 7FFAH D. FFFF FFFAH

解析。$65530 = 65536 - 6 =$ `FFFAH`，源类型是无符号 16 位数，扩展补 0，得到 `0000 FFFAH`，选 **B**。最高位为 1 在这里是普通数值位，不是负号。

2.3.2 例 2-3 负的 short 转成 unsigned int

例 2.3.2 (配套例题)

【配套例题】在 32 位计算机上执行下列 C 语言代码段后，ui 的值是（ ）。

```
short si = -32767;
unsigned int ui = si;
```

A. $2^{15}-1$ B. $2^{15}+1$ C. $2^{32}-2^{15}-1$ D. $2^{32}-2^{15}+1$

解析。按题目的 16 位 short、32 位 unsigned int 模型，-32767 的 16 位补码为 `8001H`。数值转换为 32 位无符号整数后，得到与 -32767 模 2^{32} 同余的数，即 $2^{32}-32767 = 2^{32}-2^{15}+1$，机器数为 `FFFF 8001H`，选 **D**。把 `8001H` 直接补 0 会把负的源值误当成无符号 32769。

C 中的 usual arithmetic conversions 也会改变比较方式。若 32 位 `int i=-1` 与 32 位 `unsigned u=1` 比较，常见模型下 i 转为无符号 $2^{32}-1$，所以 `i<u` 为假。需要先完成整数提升和通常算术转换，再决定按哪种值比较。

本讲义中 “截断取低位” 的位级推演按题设机器模型进行。C 语言本身不保证有符号溢出按补码绕回：有符号加法溢出属于未定义行为；按本讲义采用的 C11/C17 口径，超范围转换到有符号类型还涉及实现规定。考题若明确给机器位宽和截断模型，可据模型计算；编写可移植 C 程序时不能把这些题设当成语言的普遍保证。

2.3.3 移位先问逻辑还是算术

逻辑右移高位补 0，算术右移复制符号位；左移均在低位补 0，但如何解释结果及是否溢出仍取决于编码。8 位补码 -3 算术右移 1 位得到 -2，相当于向负无穷方向取整；整数除法 $-3/2$ 通常向零截断得到 -1，两者不能无条件互换。左移时被丢弃的高位无法恢复，“低位补 0” 也不保证

数值恰好乘 2。

不变量·减法中的一只加一

控制 Sub=1 时第二路逐位取反，最低进位再加 1，总体才是 A−B。若已经送入 B 的负数补码，又把最低进位设 1，就把那一个 1 重复加入。追踪输入端比只写 “取补码” 更精确。

2.4 加法器怎样同时支持加与减

一位全加器接收 A_i, B_i, C_i，产生和位 S_i 与向高位的进位 C_{i+1}：

$$S_i = A_i \oplus B_i \oplus C_i, \qquad C_{i+1} = A_i B_i + (A_i \oplus B_i) C_i.$$

定义进位生成 $G_i = A_i B_i$、进位传递 $P_i = A_i \oplus B_i$，即可逐层展开进位表达式。串行进位需等低位进位逐级传播；先行进位用更复杂的组合逻辑并行形成若干进位，缩短关键路径。更快的进位结构没有改变加法的数值规则。

加减运算器用控制位 Sub 将第二输入按位与 Sub 异或，同时令最低进位 $C_0 = \text{Sub}$。Sub=0 时执行 $A + B$；Sub=1 时执行 $A + \overline{B} + 1$，即模 2^n 的减法。算术逻辑部件 ALU 还通过选择控制实现与、或、异或等逻辑操作；移位器是否算在 ALU 内部，要看具体的数据通路图。

2.4.1 例 2-4　减法器的两路输入不直接送负数符号

例 2.4.1 (配套例题)

【配套例题】某计算机字长为 8 位，CPU 中有一个 8 位加法器。已知无符号数 $x = 69, y = 38$，若在该加法器中计算 $x - y$，则加法器的两个输入端信息和输入的低位进位信息分别为（　）。

A. 0100 0101、0010 0110、0

B. 0100 0101、1101 1001、1

C. 0100 0101、1101 1010、0

D. 0100 0101、1101 1010、1

解析。$69 = 0100\,0101$, $38 = 0010\,0110$。第二路取反为 `1101 1001`，另由最低进位补上 1，所以选 **B**。若第二路已取反加 1 又令最低进位为 1，就重复加了 1。运算实际为 $69 + 217 + 1 = 287$，低 8 位 31、最高进位 1，符合 $69 - 38 = 31$。

2.5 CF 与 OF 分别判断什么

同一次加法器运算可以同时按有符号和无符号两种规则解释，因此产生不同用途的标志。ZF 表示结果低 n 位是否全 0，SF 等于结果最高位；SF=1 并不自动表示数学真值为负，还要确认按有符号解释且未发生有符号溢出。

对于加法, CF 反映最高位向外进位; 对于本讲义采用的 “CF=1 表示借位” 约定, 减法的 CF 为最高进位的反值。可统一写为 $\text{CF} = C_n \oplus \text{Sub}$。某些 ISA 用 “无借位” 作为减法进位标志，遇到这类题设时，必须按该 ISA 的约定判断。

OF 判断有符号结果是否越出补码范围：加法时同号相加而结果异号会溢出；减法时两操作数异号且结果符号与被减数不同会溢出。也可用符号位的进位输入与进位输出异或：$\text{OF} = C_{n-1} \oplus C_n$。它不要求把符号位当成独立的符号去做原码运算。

2.5.1 同一个结果，按哪一种数判断

位串运算、数值解释和条件判断应分三步。先按位宽完成加减，保留低 n 位；再按无符号或补码规则解读；最后选择与这种解释对应的标志。不能为了得到希望的正负号，改写已经产生的机器结果。

8 位运算	保留结果	按无符号解释	按补码解释
`7FH+01H`	`80H`	$127+1=128$，可表示	$127+1$ 超过 127
`FFH+01H`	`00H`	$255+1$ 超过 255	$-1+1=0$，可表示
`80H-01H`	`7FH`	$128-1=127$，无借位	$-128-1$ 小于 -128
`00H-01H`	`FFH`	$0-1$ 需借位	$0-1=-1$，可表示

最小反例·结果负号不能代替 OF

最小反例：7FH 加 01H 得 80H，SF=1 而精确和 128 为正，只因结果已越界。相反 FFH 加 01H 得 00H，CF=1 却没有补码溢出。只看结果正负，会同时误判这两类题。

同一过程的其余状态（按首列对应）：

8 位运算	CF、OF
`7FH+01H`	0、1
`FFH+01H`	1、0
`80H-01H`	0、1
`00H-01H`	1、0

ZF 和 SF 只看保留下来的结果：上表第二行 ZF=1，第一行与第四行 SF=1。若要由减法结果判断补码数的有符号小于或大于关系，应结合 OF 修正结果符号；例如第三行 SF=0，却不能据此说原来的 -128 大于 1。是否相等只需看 ZF，不必额外使用 OF。编码范围决定“能否表示”，运算电路决定“产生哪些位”，转移条件决定“怎样使用标志”，三者要连起来判断。

2.5.2 例 2-5　十六进制补码相减

例 2.5.1 (配套例题)

【配套例题】假定有符号整数采用补码表示，若 int 型变量 x 和 y 的机器数分别是 FFFF FFDFH 和 0000 0041H，则 x、y 的值及 x−y 的机器数分别是（　）。

A. x=−65，y=41，x−y 的机器数溢出
B. x=−33，y=65，x−y 的机器数为 FFFF FF9DH
C. x=−33，y=65，x−y 的机器数为 FFFF FF9EH
D. x=−65，y=41，x−y 的机器数为 FFFF FF96H

解析。`FFFFFFDFH` 距 2^{32} 差 33，故 x=−33；`41H` 是十进制 65。数学结果为 -98，在 32 位补码范围内，机器数为 $2^{32}-98=$ `FFFFFF9EH`，选 **C**。十六进制的 41 不能按十进制 41 读。

2.5.3 例 2-6　有符号减法为何仍会产生借位

例 2.5.2 (配套例题)

【配套例题】已知 x、y 为 int 类型，当 x=100、y=200 时，执行“x 减 y”指令得到的溢出标志 OF 和借位标志 CF 分别为 0、1。那么当 x=10、y=−20 时，执行该指令得到的 OF 和 CF 分别是（　）。

A. OF=0, CF=0　B. OF=0, CF=1　C. OF=1, CF=0　D. OF=1, CF=1

解析。有符号真值为 $10-(-20)=30$，可表示，OF=0。若按无符号解释输入位串，-20 对应 $2^{32}-20$，10 小于它，减法需要借位，CF=1，选 **B**。题目已经用 100−200 说明了借位标志约定，不能把两个标志都按有符号真值判断。

条件转移利用这些标志完成比较。先计算 $a-b$：无符号 $a<b$ 对应 CF=1，无符号 $a>b$ 对应 CF=0 且 ZF=0；带符号 $a<b$ 对应 SF $\neq$ OF，带符号 $a>b$ 对应 ZF=0 且 SF=OF。用 SF 单独判断带符号大小，会在溢出时失败。

不变量·完整积保留双倍宽

两个 n 位输入的完整积最多 2n 位。判断截为 n 位是否有符号溢出，要看高半部是否恰为低半部符号扩展；只看高半部是否全 0，会把合法负积误判。

2.6 乘除运算必须保留的中间量

2.6.1 无符号乘法：为什么需要部分积和双倍位宽

两个 n 位无符号数的精确乘积最多需要 $2n$ 位。以王道书 §2.2 中的 13 乘 11 为例：$13=1101_2$，$11=1011_2$。乘数从低位到高位为 1、1、0、1，部分积依次是 13、26、0、104，总和 143，即 `1000 1111`。

顺序乘法器可以每轮检查乘数最低位，决定是否把被乘数加到部分积高半部，再将 “进位—部分积—乘数” 组合右移。部分积开始为 0，循环 n 次后得到 $2n$ 位积。相应的等价算法也可让被乘数左移、乘数右移，把选中的部分积加到双倍宽累加器；两种寄存器布局的逐轮数值不同，不应把一种布局的移位规则套用到另一种。

对 13×11，用双倍宽累加器的轨迹如下：

本轮乘数低位	本轮被乘数	累加器更新后	乘数右移后
1	13	13	5
1	26	39	2
0	52	39	1
1	104	143	0

带符号乘法不能把负数的补码直接当成同宽无符号数求完整积。Booth 法在乘数末端附一位 0，考察相邻两位 q_iq_{i-1}：01 时加被乘数，10 时减被乘数，00 或 11 不加减，再对组合寄存器算术右移。这里的前提是符号扩展到位、中间寄存器足够宽；对最小负数取负时，尤其需要保护位。

以 4 位补码的 -3×5 说明 Booth 法（原创机制示例）。乘数寄存器 Q 为 4 位，附加位 q_{-1} 为 1 位，A 采用 5 位补码以保留保护位，被乘数 M=−3 也扩成 5 位。初态 A=`00000`、Q=`0101`、$q_{-1}=0$。每轮先看 Q_0q_{-1}，对 A 加减 M，再把 A、Q、q_{-1} 连接成 10 位整体算术右移；A 的最低位移入 Q 最高位，Q 最低位移入 q_{-1}。

轮次	判断位	移位前操作及 A 真值	移位后的 A
1	10	A−M=3	00001
2	01	A+M=−2	11111
3	10	A−M=2	00001
4	01	A+M=−2	11111

同一过程的其余状态（按首列对应）：

轮次	移位后的 Q	q_{-1}
1	1010	1
2	0101	0
3	0010	1
4	0001	0

不变量·余数必须最终回验

无符号除法结束后，同时核 N=QD+R 及 0≤R<D。恢复法与不恢复法可以出现不同中间余数，但最终条件相同；不恢复法若结束时 R 为负，常需补做一次加 D 校正。

四轮后 AQ 为 9 位 `11111 0001`，去掉冗余符号保护位即 8 位 `1111 0001`，真值为 -15。移位时必须复制 A 的符号位，并在寄存器边界传位；把 A、Q 分别右移会丢掉跨界位，得到错误乘积。

阵列乘法器用组合电路完成部分积生成与压缩求和，可能在一个足够长的时钟周期内完成；顺序乘法器复用 ALU 和移位器，通常跨多个周期。“一个周期”不能脱离所用电路和时钟周期长度讨论。常数乘法可以在保持规定语义的前提下化为移位与加减，例如 $13x = (x \ll 3) + (x \ll 2) + x$。

2.6.2 除法：商来自每一步余数比较

以无符号 7÷3 为例，按被除数的有效位 `111` 从高到低处理，初始余数 R=0。每读入一位，先令 $R \leftarrow 2R + b$，若 R≥3 则减 3 并在商中记 1，否则记 0。

读入位	比较前余数	本位商	本步余数
1	1	0	1
1	3	1	0
1	1	0	1

商为 `010` 即 2，余数 1，满足 $7 = 2 \times 3 + 1$ 且 $0 \le R < 3$。硬件通常固定处理整个位宽，在前面补零位，不会因此改变结果。恢复余数法先试减，结果为负时再加回除数；不恢复余数法根据余数符号决定下一轮加还是减，用不同轨迹完成同一任务。

不恢复余数法也可用同一 7÷3 观察。令余数 R 为 4 位有符号寄存器、Q 为 3 位，除数 D=3，初态 R=0、Q=`111`。每轮先把 R、Q 连接左移一位，Q 最高位移入 R 最低位；若上一轮 R 非负，本轮减 D，若上一轮 R 为负则加 D；运算后 R 非负就在 Q 最低位置 1，否则置 0。

轮次	左移后 R 真值	加减操作	操作后 R
1	1	R−D	−2
2	−3	R+D	0
3	1	R−D	−2

同一过程的其余状态（按首列对应）：

轮次	记商后的 Q
1	110
2	101
3	010

最后 R 仍负，补做 R←R+D，得到余数 1，Q=`010` 为商 2。第二轮没有先把 -2 恢复成 1 再试减，而把恢复效果合并进下一轮加法，这正是不恢复余数法的含义；最终余数校正不能漏掉。硬件布局或商编码约定不同会改变表中轨迹，做题时应遵循同一套规则。

有符号除法还要确定商、余数符号和舍入方向。按 C 向零截断模型，-7÷3 的商为 -2、余数为 -1，仍满足被除数 = 商 × 除数 + 余数。

除数为 0 时需要异常处理；$-2^{n-1}/(-1)$ 的商超出 n 位补码范围。某种除法器可能检测到被除数绝对值小于除数，就提前输出商 0；另一种固定轮数除法器仍运行完整迭代。只有题干电路给出提前退出逻辑时，才能比较哪组数据用时最短。

错项排除·字段长度不等于精度

binary32 尾数字段 23 位，正规格化数隐含前导 1，因此有效精度 24 位；非规格化数隐含 0。把所有编码都当 1.F，会把最小正数、零和特殊值混在一起。先判断阶码是否全 0 或全 1。

2.7 IEEE 754：字段既规定精度，也规定特殊情况

IEEE 754 binary32 即通常的单精度格式，由 1 位符号 s、8 位阶码 E、23 位尾数字段 F 构成；binary64 分别为 1、11、52 位。这里 F 的整数值要除以 2^{23} 或 2^{52} 才是小数尾数部分。

项目	binary32（单精度）	binary64（双精度）
总位数	32 位	64 位
符号位 s	1 位	1 位
阶码 E	8 位	11 位
尾数字段 F	23 位	52 位
阶码偏置 B	127	1023
规格化数的真阶码 e	$-126 \le e \le 127$	$-1022 \le e \le 1023$
规格化数的有效精度	24 位，含隐含前导 1	53 位，含隐含前导 1

项目	binary32（单精度）	binary64（双精度）
最小正规格化数	2^{-126}	2^{-1022}
最小正非规格化数	2^{-149}	2^{-1074}
最大有限正数	$(2-2^{-23})2^{127}$	$(2-2^{-52})2^{1023}$

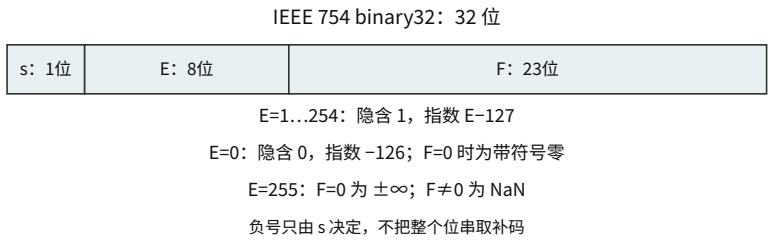


Figure 2.1: 单精度字段及三类指数编码

解码时先看阶码是否全 0 或全 1，再看尾数字段。下表中 p 是尾数字段位数，binary32 取 23、binary64 取 52；B 是对应的阶码偏置，F 按无符号整数解释。

类别	阶码 E	尾数字段 F	数值或含义
正零、负零	全 0	0	$(-1)^s 0$，符号位区分正负零
非规格化数	全 0	非 0	$(-1)^s(F/2^p)2^{1-B}$，隐含前导 0
规格化数	既非全 0，也非全 1	任意	$(-1)^s(1+F/2^p)2^{E-B}$，隐含前导 1
正无穷、负无穷	全 1	0	$(-1)^s\infty$
NaN	全 1	非 0	非数；不能当作无穷或普通实数

对于 binary32：

$$x=\begin{cases}(-1)^s(1+F/2^{23})2^{E-127}, & 1\le E\le 254;\\ (-1)^s(F/2^{23})2^{-126}, & E=0;\\ (-1)^s\infty, & E=255, F=0;\\ \text{NaN}, & E=255, F\neq 0.\end{cases}$$

不变量·阶码字段先减偏置

普通 IEEE 浮点数的指数是 E 减偏置，尾数字段另按二进制小数解释。整数移码常见的翻转符号位关系，不能直接套到偏置 127 的 IEEE 阶码。编码和解码各走一遍可验证字段。

规格化数有隐含的前导 1，有效精度 24 位。非规格化数前导为 0，最小指数同样取 -126，数值逐渐接近 0；当 F=0 时是带符号零。非规格化数不能再用指数 E$-$127 配隐含 1 计算。

最小正规格化数为 2^{-126}，最小正非规格化数为 2^{-149}，最大有限正数为 $(2-2^{-23})2^{127}$。这些是数值范围；“能连续精确表示的整数” 是另一问题。binary32 能精确表示 $[-2^{24},2^{24}]$ 内所有整数，更大的某些整数仍能精确表示，但相邻可表示数的间隔可能大于 1。

2.7.1 例 2-7 负小数怎样组成单精度机器数

例 2.7.1 (配套例题)

【配套例题】-0.4375 的 IEEE 754 单精度浮点数表示为（ ）。

A. BEE0 0000H B. BF60 0000H C. BF70 0000H D. C0E0 0000H

解析。$0.4375=0.0111_2=1.11_2\times 2^{-2}$。

s=1，E=$-$2+127=125=`01111101`，F 为 `110` 后补 20 个 0。

连接为 `1|01111101|11000000000000000000000`，十六进制为 `BEE00000H`，选 **A**。负数只改变符号位，不对浮点数全部位取补码。

2.7.2 例 2-8 同为负浮点数时怎样比较

例 2.7.2 (配套例题)

【配套例题】float 型数据常用 IEEE 754 单精度浮点格式表示。假设两个 float 型变量 x 和 y 分别存放在 32 位寄存器 f1 和 f2 中，若 `(f1)=CC90 0000H`、`(f2)=B0C0 0000H`，则 x 和 y 之间的关系为（ ）。

A. x<y 且符号相同 B. x<y 且符号不同 C. x>y 且符号相同 D. x>y 且符号不同

解析。两个最高位都为 1，均为负。x 的阶码为 153，实际指数 26；y 的阶码为 97，实际指数 -30。两者均为有限规格化数，x 的绝对值远大于 y，因此 x<y，选 **A**。此处不必把尾数展开为完整十进制数。对负数，绝对值更大的反而更小。

int 转 float 可能丢低位，32 位 int 转 binary64 则可精确表示，因为 binary64 有 53 位有效精度。float 转 double 在通常 IEEE 格式下能精确保留原来的有限数值，但不能恢复 float 形成时已经丢掉的信息。浮点转整数按语言规则截断，且须检查范围；NaN 与任何数（包括自身）的相等比较都为假，不能把 `x/x==1` 用在 x=0 或无穷等情形。

2.8 浮点加减：对阶丢位以后还要判断舍入

最小反例·不溢出仍可不精确

最小反例：binary32 能精确存 2 的 24 次方，却不能精确存它加 1。指数远未到上限，有效位已经不足。浮点溢出与舍入损失要分别检查，答案有限并不保证与数学精确值相等。

两个浮点数只有指数相同，才能直接把有效数相加。通常先把较小指数的有效数右移，使指数与较大者一致；这叫对阶。大阶向小阶对齐会要求更多高位和更大的寄存器，不符合固定精度运算的通常实现。

对阶之后进行尾数运算，再规格化和舍入。加法可能得到 $10.xxx_2$，须右移 1 位、阶码加 1；异号相加发生抵消，可能得到 $0.00\cdots1xxx_2$，须左移、阶码减少。舍入使尾数加 1 时还可能再次进位，必须再检查规格化和阶码范围。若得到精确的 0，也不能继续无止境地左移寻找前导 1。

用一个仅保留 4 位有效数的教学格式说明舍入。$x = 1.000_2 \times 2^0$，$y = 1.000_2 \times 2^{-4}$，精确和为 1.0001_2，恰好位于保留为 `1.000` 和 `1.001` 的中点。采用 “舍入到最近，正中取偶数”，保留最低位为 0 的 `1.000`。若 x 改为 `1.001`，精确和为 `1.0011`，则正中取偶数进为 `1.010`。偶数指保留有效数的最低位为 0，不是判断整个真值是奇整数还是偶整数。

实际实现常保留保护位、舍入位和粘滞位。粘滞位记录更低位中是否出现过 1，因此 `1000`…与 `1000`…`1` 的舍入结论可能不同，不能只保留移出的第一位。

2.8.1 例 2-9　哪些步骤可能造成指数越界

例 2.8.1 (配套例题)

【配套例题】下列有关浮点数加减运算的叙述中, 正确的是（　）。

I. 对阶操作不会引起阶码上溢或下溢
II. 右规和尾数舍入都可能引起阶码上溢
III. 左规时可能引起阶码下溢
IV. 尾数溢出时，结果不一定溢出

A. 仅 II、III　B. 仅 I、II、IV　C. 仅 I、III、IV　D. I、II、III、IV

解析。对阶把较小阶提高到较大阶，最终阶码仍是原有合法阶码之一；右移尾数可能损失精度，但不使该阶码越界，I 正确。右规和舍入后的再右规会使指数加 1，可能越过上界，II 正确。抵消后的左规要减小指数，可能进入非规格化区或触及题设的下溢界，III 正确。尾数暂时形成 `10.xxx` 还可以右规，只要指数容纳得下，结果仍可表示，IV 正确。选 **D**。尾数的暂时进位与最终浮点范围溢出是不同层次。

采用 IEEE 754 的渐进下溢时，指数过小并不立即把所有结果变成 0，还可能形成非规格化数。题目若采用 “阶码、尾数均为补码” 的教学格式，就应按其位宽、小数点位置、双符号位和舍入方法计算，不能把该格式的尾数补码接到 IEEE 字段中。

浮点加法不满足数学实数的结合律。例如在 binary32 中，$a = 2^{25}, b = -2^{25}, c = 1$：$(a + b) + c = 1$，而 $a + (b + c)$ 中 b+c 可能先舍入回 b，结果为 0。发生差异的原因是中间舍入，不是加法指令随意改变数学规则。

2.8.2 定点整数与浮点数的范围、精度怎样比较

32 位补码整数把 31 位用于正数的数值范围，范围内整数连续、没有空缺；binary32 把位分给符号、指数和有效数，扩大数值范围，同时只保留 24 位有效二进制数字。浮点数在某个数量级的间隔随指数增大，因

而 “32 位” 相同不表示能精确保存同一批整数。

例如 2^{24} 可被 binary32 精确表示，但 $2^{24}+1$ 正好落在相邻两个可表示数之间；按最近偶数舍入回 2^{24}。而 2^{25} 仍可精确表示。问 “所有不超过某界的整数都精确” 与问 “某个大整数是否精确”，必须分别作答。整数溢出检查编码范围；浮点溢出检查舍入、规格化后的指数；尚未溢出的浮点结果也可能已经不精确。

课程连接·编程中的循环边界

无符号 n=0 时 n−1 绕到最大值，会改变循环终止；有符号溢出又涉及语言规则。把循环、编码范围、浮点精度三步分开检查，可以说明同一算法在不同类型中为何产生不同结果。

2.8.3 例 2-10　同一求和程序中的循环边界、整数范围与浮点精度

例 2.8.2 (配套例题)

【配套例题】已知 $f(n)=\sum_{i=0}^{n}2^i=2^{n+1}-1=\underbrace{11\cdots1}_{n+1\text{位}}{}_2$，计算 $f(n)$ 的 C 语言函数 f1 如下。

```c
int f1(unsigned n) {
    int sum = 1, power = 1;
    for (unsigned i = 0; i <= n - 1; i++) {
        power *= 2;
        sum += power;
    }
    return sum;
}
```

将 f1 中的 int 都改为 float，可得到计算 $f(n)$ 的另一个函数 f2。假设 unsigned 和 int 型数据都占 32 位，float 采用 IEEE 754 单精度标准。请回答下列问题。

（1）当 n=0 时，f1 会出现死循环，为什么？若将 f1 中的变量 i 和 n 都定义为 int 型，则 f1 是否还会出现死循环？为什么？

（2）f1(23) 和 f2(23) 的返回值是否相等？机器数各是什么（用十六进制表示）？

（3）f1(24) 和 f2(24) 的返回值分别为 33 554 431 和 33 554 432.0，为什么不相等？

（4）$f(31)=2^{32}-1$，而 f1(31) 的返回值却为 −1，为什么？若使 f1(n) 的返回值与 f(n) 相等，则最大的 n 是多少？

（5）f2(127) 的机器数为 7F80 0000H，对应的值是什么？若使 f2(n) 的结果不溢出，则最大的 n 是多少？若使 f2(n) 的结果精确（无舍入），则最大的 n 是多少？

解析。本题把同一数列放进不同表示中：先查循环是否结束，再查运算范围，最后查浮点有效位。以下对 f1(31) 的既定结果和 n=0 时持续循环的轨迹，都按 32 位补码机器执行模型解释；标准 C 中有符号溢出属于未定义行为，n=0 时循环体也可能先发生这种溢出，不能把本题机器推演当成可移植 C 保证。

（1）i 和 n 均为 unsigned，n=0 时，`n-1` 按模 2^{32} 运算成为 $2^{32}-1$。任何 32 位无符号 i 都满足 `i<=n-1`，i 达到最大值后再加 1 又回到 0，因此循环条件一直为真。若两者都改为 int，n−1 为 −1，第一次测试 $0\le -1$ 就为假，循环一次也不执行，不会因这个边界形成死循环。

（2）$f(23)=2^{24}-1=16\,777\,215$，是 24 个连续的二进制 1。它在 32 位 int 范围内，且 binary32 恰有 24 位有效精度，故两者数值相等。

f1(23) 的补码机器数为 `00FF FFFFH`。浮点形式为 $1.\underbrace{11\cdots1}_{23位}\times2^{23}$，s=0、E=23+127=150、F 全 1，故 f2(23) 的机器数为 `4B7F FFFFH`。相同数值不要求机器编码相同。

（3）$f(24)=2^{25}-1$ 需要 25 位有效数字，int 仍能精确保存，binary32 却只能保留 24 位。最后一次加法的精确结果为 33 554 431，处于 33 554 430 与 33 554 432 之间的正中；最近偶数舍入选后者，得到 2^{25}，机器数 `4C00 0000H`。这里尚未浮点溢出，只丢失了低位精度。

（4）$2^{32}-1$ 超过 32 位补码 int 最大值 $2^{31}-1$。按题设保留的低 32 位全为 1，解释为补码 -1。要求所有返回值与数学数列一致，须 $2^{n+1}-1\le 2^{31}-1$，所以最大 n=30；不能因为无符号 32 位能表示 $2^{32}-1$，便认为 int 也能表示。

（5）`7F80 0000H` 的阶码全 1、尾数全 0、符号 0，表示正无穷。n=126 时，精确数 $2^{127}-1$ 被舍入为 2^{127}，仍是有限数；n=127 时，结果越过最大有限数并舍入为正无穷。因此不溢出的最大 n=126。要完全精确，数列中 n+1 个连续 1 必须全部容纳在 24 位有效数中，最大 n=23。两个上界分别来自指数范围与有效精度，不能互换。

不变量·字节序和对齐分两步

先依成员类型与对齐规则求对象布局，再把每个值拆成字节按大小端放入。字节序不改变结构体填充，也不反转数组元素次序。两步分别核对后，才能回答某地址保存的具体字节。

2.9 大端、小端和对齐解决不同问题

多字节数据的大端方式把最高有效字节放在最低地址，小端方式把最低有效字节放在最低地址。字节序只改变一个多字节对象内部的字节排列，不反转每个字节内的位，也不改变数组元素的顺序。

32 位值 `12345678H` 从低地址到高地址，大端为 12、34、56、78，小端为 78、56、34、12。讨论 “高位字节” 时指数字意义上的高位，讨论 “高地址” 时指存储位置，两个维度应分开。

对齐指对象的起始地址满足规定的倍数要求。常用的自然对齐模型中，2 B 对象的起始地址是 2 的倍数，4 B 对象是 4 的倍数。编译器还可能为结构体在成员之间和末尾加入填充，使各成员地址和相邻数组元素都满足对齐规则。具体要求属于 ABI/编译器约定；题目给出 “按边界对齐” 时，才按给定类型大小计算。

2.9.1 例 2-11　先确定结构布局，再看低字节

例 2.9.1 (配套例题)

【配套例题】某计算机存储器按字节编址，采用小端方式存放数据。假定编译器规定 int 型和 short 型长度分别为 32 位和 16 位，并且数据按边界对齐存储。某 C 语言程序段如下：

```
struct {
    int a;
    char b;
    short c;
} record;
record.a = 273;
```

若 record 变量的首地址为 0xC008，则地址 0xC008 中的内容及 record.c 的地址分别为（　）。

A. 0x00，0xC00D　　B. 0x00，0xC00E

C. 0x11，0xC00D　　D. 0x11，0xC00E

解析。a 占偏移 0 至 3，b 占偏移 4。下一地址偏移 5 不是 2 的倍数，须填充 1 B，c 占偏移 6、7，故 c 地址为 C00EH。$273 = \texttt{00000111H}$，小端最低地址保存低字节 11H，选 **D**。填充来自对齐，低字节 11 来自字节序，两步不能互相代替。

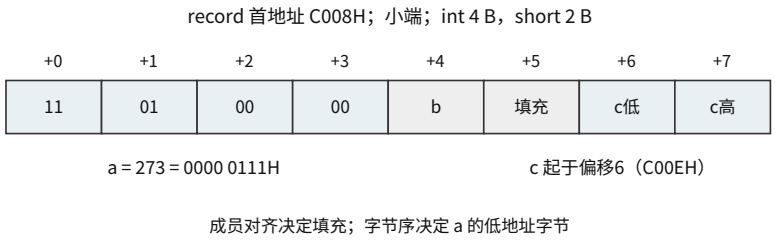


Figure 2.2: 结构体成员、填充与低地址字节分别标明

二维数组按行优先存放时，`a[i][j]` 的元素偏移为 $i \times$ 列数 $+ j$，字节偏移还要乘元素字节数。对于取负值的 short，先写出完整的 16 位补码，再按字节序拆开。例如 -6 为 `FFFAH`，小端低地址是 FA、下一字节是 FF，不是把真值“-6”拆成十进制字符。

不变量·相邻间隔随指数变大

正规格化 binary32 在同一指数区间内间隔是 2 的 e−23 次方。指数每增 1，相邻间隔也翻倍。先算间隔再判某整数是否可表示，比只记 “24 位精度” 更容易处理大整数与舍入中点。

2.10 把位串、真值和条件分开核算

表示与运算题的统一路径

先写位宽与编码，再求数学真值或完整精确结果；按题设完成截断、扩展或舍入，最后解释结果位串并检查标志。浮点解码先识别阶码的全 0/全 1，普通数公式不适用于特殊编码。涉及 C 代码时再核语言语义，不能把某种硬件结果提升为标准 C 保证。

2.10.1 补码为什么恰好多一个负数

由位权得到范围

n 位补码的值是 $-b_{n-1}2^{n-1} + \sum_{i=0}^{n-2} b_i 2^i$。最小时只有符号位为 1，值为 -2^{n-1}；最大时符号位 0 而其余全 1，值为 $\sum_{i=0}^{n-2} 2^i = 2^{n-1} - 1$。这既说明负数多一个，也说明最小负数的相反数无法放回同宽补码中。

□

2.10.2 精度与范围各问什么

binary32 在 $[2^e, 2^{e+1})$ 内，相邻正规格化数相差 2^{e-23}。原因是尾数字段每增加 1，有效数增加 2^{-23}，再乘指数倍率 2^e。在 $[2^{24}, 2^{25})$ 间距为 2，所以 $2^{24}+1$ 无法精确保存；2^{25} 本身却是精确的二进制幂。有限数是否溢出取决于指数边界，能否精确取决于有效位，两者不可互代。

最近偶数舍入先比较被舍弃部分与半个间隔：小于一半向较近值，大于一半向另一侧；恰好一半时选保留末位为 0 的候选。只写 “四舍五入” 会漏掉二进制等距点规则。

2.10.3 对齐和容量的单位

课程连接·单位衔接存储系统

数据宽度、地址单位和块大小会在下一章共同决定偏移位数与容量。按字节编址时 4B 整数跨 4 个地址，但同一字节内 8 个位并不产生 8 个独立字节地址。每个对数前都先写清以字节还是以字计数。

机器字长是 CPU 一次处理整数的基本宽度，存储字长是一个存储单元保存的位数，指令字长是某条指令的编码位数；三者可以不同。按字节编址时，32 位整数通常占 4 个连续地址；按 32 位字编址时，同一个字只占一个地址。先把地址单位写出来，再计算数组偏移、总容量和跨界访问次数。

自然对齐要求对象起点满足某个倍数，但具体倍数来自题设或 ABI。结构体末尾填充使数组中下一对象也满足对齐；删除末尾填充可能让第一个对象内部正确而第二个对象起点出错。字节序则只回答多字节值的高低有效字节如何落到地址，不能决定填充量。

2.11 数字逻辑：从真值表到运算部件

逻辑运算把 0、1 解释为假、真。与运算只有输入全为 1 才输出 1；或运算只要有输入为 1 就输出 1；非运算反转一个输入；异或在两个输入不同时输出 1，同或在两个输入相同时输出 1。逻辑式中的 “+” 表示或，“乘积” 表示与，不能直接当作整数加乘。

A, B	与 AB	或 $A+B$	异或 $A \oplus B$
0,0	0	0	0
0,1	0	1	1
1,0	0	1	1
1,1	1	1	0

非、与、或的通常优先级依次降低；括号和非号覆盖范围优先明确。德摩根律为 $\overline{A+B}=\bar{A}\bar{B}$、$\overline{AB}=\bar{A}+\bar{B}$。异或满足交换律和结合律，多个位异或的结果为 1，当且仅当其中 1 的个数为奇数；这既可生成一位二进制加法的本位和，也可构造奇偶校验。

2.11.1 选择器和三态门怎样控制通路

k 选 1 多路选择器 MUX 有 k 组数据输入、一组输出和选择信号，至少需要 $\lceil \log_2 k \rceil$ 位选择编码。2 选 1 且选择位为 s 时，输出可写为 $Y=\bar{s}D_0+sD_1$。若数据总线有 32 位，应把它理解为 32 路对应位一起被选择。

三态门除 0、1 外，还有高阻态。使能时输出跟随输入；关闭时输出端呈高阻，对公共总线不施加有效驱动。高阻态不是数值 0。一条内部总线可连接多个输出端，但同一时刻必须按控制约束选择驱动源；多个源同时输出相反电平会发生总线争用。

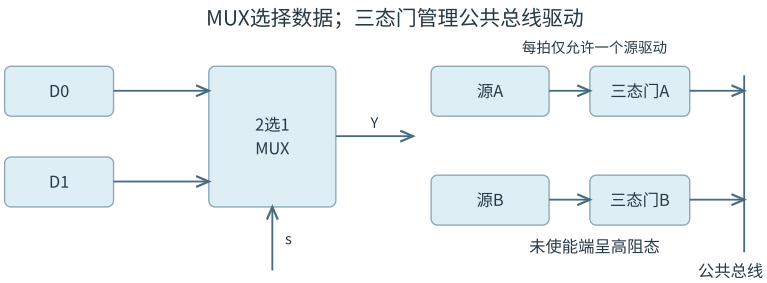


Figure 2.3: 选择器选择数据源，三态门决定输出端是否驱动公共线

2.11.2 全加器、进位产生与 ALU 选择

课程连接·乘法轮次

每轮先按寄存器低位或相邻位决定加减操作，再按约定整体移位，特别关注符号扩展。

一位全加器输入 A_i, B_i, C_{i-1}，输出本位和与进位：

$$S_i = A_i \oplus B_i \oplus C_{i-1}, \qquad C_i = A_iB_i + (A_i \oplus B_i)C_{i-1}.$$

令 $G_i = A_iB_i$ 为进位产生信号，$P_i = A_i \oplus B_i$ 为进位传播信号，则 $C_i = G_i + P_iC_{i-1}$。继续代入可以写出 $C_2 = G_2 + P_2G_1 + P_2P_1C_0$。串行进位加法器逐级等待前一级进位；先行进位电路通过组合逻辑提前形成进位，以增加电路复杂度换取速度。

“并行加法器” 可指多个输入位同时送入，并不保证进位也同时生成。ALU 则在加法器和逻辑电路基础上，由操作控制位选择加、减、与、或、移位等功能；数据位宽与操作选择编码位宽是不同的量。

2.12 乘法：按寄存器状态逐轮核算

2.12.1 无符号乘法：1101×1011

设 $n = 4$，被乘数寄存器 $X = 1101$，初始部分积 $P = 0000$、乘数 $Y = 1011$、进位 $C = 0$。每轮先根据 Y 最低位决定是否把 X 加到 P，保留最高进位，再把 (C, P, Y) 整体逻辑右移，重复 4 轮。

轮次	Y 最低位与加法结果 C, P	整体右移后的 P, Y
初始	—	0000,1011
1	1；0,1101	0110,1101
2	1；1,0011	1001,1110
3	0；0,1001	0100,1111
4	1；1,0001	1000,1111

最终 $(P, Y) = 10001111_2 = 143 = 13 \times 11$。完整乘积需要 $2n$ 位；若结果字段仅保留低 n 位，则高 n 位非零表示无符号乘积不能由 n 位完整表示。课件中的硬件标志行为应与具体指令定义对应；C 语言无符号类型运算按模 2^n 处理，而有符号溢出的语言语义另行规定。

2.12.2 Booth 补码一位乘法：-3×6

补码乘法让符号位参加运算。令 $X = 1101$ 表示 -3，初始 $P = 0000$、$Y = 0110$、辅助位 $q_{-1} = 0$。观察 (Y_0, q_{-1})：00、11 执行 $+0$，01 执行 $+X$，10 执行 $-X$。随后把 (P, Y, q_{-1}) 整体算术右移，保留最左符号位。

轮次	观察位与操作	算术右移后的 P, Y, q_{-1}
初始	—	0000,0110,0
1	00，$+0$	0000,0011,0
2	10，$-X$	0001,1001,1
3	11，$+0$	0000,1100,1
4	01，$+X$	1110,1110,0

乘积为 $11101110_2 = -18$。若只保留 4 位，则低 4 位 1110 表示 -2，与真实乘积不同。对 $2n$ 位补码乘积保留低 n 位，未溢出的条件是高 n 位均为低 n 位符号位的扩展，即最高 $n+1$ 位全部相同。处理最小负数等边界时，需要足够的内部符号扩展，不能在过窄的临时寄存器中直接对最小负数求正值。

2.12.3 硬件实现的三种组织

逐位乘法用同一加法器多次完成加法、移位，硬件少而周期多；阵列乘法器用更多组合电路并行形成部分积并累加，延迟取决于电路路径；流水线乘法器在组合逻辑中插入阶段寄存器，允许多组乘法交叠。流水线主要提高持续输入时的吞吐率，单次乘法仍要经历完整延迟。

课程连接·商余数约束

先写被除数、除数、余数和商的位宽与符号规则；逐轮试商后检验被除数等于商乘除数加余数。

2.13 除法：先明确被除数、商和余数的位宽

无符号双精度除法通常以 (R,Q) 保存 $2n$ 位被除数，除数寄存器保存 n 位除数，结束后 Q 保存商、R 保存余数。单精度被除数先零扩展，带符号被除数则按约定作符号扩展。除数为 0 属于除法异常；商还必须落在目标字段的表示范围。

恢复余数法可理解为：移入下一被除数位，试减除数；若结果非负，上商 1；若为负，加回除数恢复余数，上商 0。不恢复余数法则利用当前余数符号决定下一轮加或减，减少“立即恢复”动作，但最终须按算法规则校正余数。

以 $1111_2 \div 0010_2$，即 $15 \div 2$ 为例，下面用逐位长除法呈现同一数学过程：

处理的被除数位	移入后的中间值	本位商、余数
第 1 位 1	1	0，1
第 2 位 1	$2 \times 1 + 1 = 3$	1，1
第 3 位 1	3	1，1
第 4 位 1	3	1，1

得到商 0111、余数 0001，满足 $15 = 7 \times 2 + 1$ 且 $0 \le 1 < 2$。表中的长除法是数学核算模型，和硬件 (R,Q) 整体移位时的寄存器切分应分别标明。

2.13.1 双精度除法的商溢出与寄存器轨迹

无符号 $2n$ 位被除数先分装到 R,Q，各 n 位，Y 保存 n 位除数。$Y > 0$ 时，若初始高半部 $R \ge Y$，则商至少为 2^n，不能装入 n 位 Q；因此正式迭代前可先作商溢出检查。课件中的 $255/15 = 17$ 超出 4 位商范围，而 $200/15 = 13$ 余 5 可以表示。单精度无符号被除数先零扩展，高半部为 0；除数非零时，商不会超出同宽无符号范围。

对 $15/2$，初态为 $R = 0000, Q = 1111, Y = 0010$。每轮将 RQ 整体左移，使下一被除数位进入 R，再试算 $R - Y$：非负时保留差并把 Q 最低位置 1；负时恢复原 R 并置 0。循环 4 轮后，R 为余数、Q 为商。

Figure 2.4: 王道课件 15 除以 2 的四轮寄存器状态，包含试减、恢复与商位写入

课程连接·商余数约束

先写被除数、除数、余数和商的位宽与符号规则；逐轮试商后检验被除数等于商乘除数加余数。

“试减为负” 是临时结果，并不表示被除数变为带符号负数。电路须保存用于试算符号或借位判断的状态；把负的试算低位直接当作最终无符号余数，会破坏 $0 \le R < Y$。

2.13.2 补码除法中的符号扩展和商修正

王道课件以 4 位补码 $-7/3$ 说明有符号除法。被除数 1001 先符号扩展为 8 位，初态 $R = 1111, Q = 1001$；除数 $Y = 0011$。下面列出这一实例的状态，R 栏按 4 位补码解读。被除数与正除数异号，试算为 $R + Y$，本例试算结果若改变余数原来的负号就恢复，否则保留并生成商的绝对值位。

轮次	移位后的 R、Q	试加后的 R	轮末 R、Q
1	1111、0010	0010，符号改变	恢复为 1111、0010
2	1110、0100	0001，符号改变	恢复为 1110、0100
3	1100、1000	1111，保留	1111、1001
4	1111、0010	0010，符号改变	恢复为 1111、0010

四轮后 $Q = 0010$ 是商的绝对值 2。因原操作数异号，再对 Q 取补得到 1110，即商 -2；$R = 1111$ 即余数 -1，满足 $-7 = (-2) \times 3 + (-1)$。课件的另外两例为 $6/3 = 2$ 余 0，以及 $3/(-7) = 0$ 余 3。表中没有出现试算恰为 0 的情形；一般电路还需单独正确处理整除，不能把 “结果符号位变 0” 机械地判为试算失败。

也可先在足够宽的内部表示中求两数绝对值，按无符号规则求商余数，最后恢复商符号和余数符号；这是一种便于核算的等价模型。最小负数的绝对值需要保护位，不能直接在原来的 n 位补码寄存器中取正值。单精度补码除法还须预先排除除数 0 及最小负数除以 -1 两类异常。

带符号整数除法中，商符号由两操作数符号异或决定；按 C 语言向 0 截断的约定，非零余数与被除数同号，始终满足 $X = QY + R$。例如 $-15/2 = -7$、余数 -1；$15/(-2) = -7$、余数 1。对 n 位补码，最小负数除以 -1 的商超出可表示范围，应单独判断。

2.14 浮点数舍入的临界情形

课程连接·舍入看被舍部分

最近偶数舍入的正中间情形，要检查保留结果末位的奇偶；规格化溢出后还要重新检查阶码。

IEEE 754 单精度规格化数含 24 位有效二进制精度。在 $[2^{24}, 2^{25})$ 范围内，相邻可表示数的间距为 2；因此 $2^{24}+1$ 恰好处于两个可表示数的中点。

最近偶数舍入的步骤是：先比较被舍部分与半个最低有效位；小于半单位截断，大于半单位进 1；恰好半单位时，选择保留部分末位为 0 的结果。保留保护位、舍入位、粘滞位，可判断被舍部分的大小；粘滞位是更低所有被舍位的或。

加法	精确结果相对相邻浮点数的位置	单精度最近偶数结果
$2^{24}+0.5$	距 2^{24} 更近	16777216
$2^{24}+1$	正好居中，低端末位偶	16777216
$2^{24}+1.5$	距 $2^{24}+2$ 更近	16777218
$(2^{24}+2)+1$	正好居中，高端末位偶	16777220

对阶导致移位不等于立刻丢弃全部低位；舍入可能再次产生尾数进位，需要右规并使指数增加。判断上溢应在规格化、舍入后检查指数；接近 0 的结果还要根据表示格式区分次正规数和下溢处理。

2.14.1 计算与辨析自查

1. 高阻态为什么不能当成数据 0？ 2. n 位补码乘积保留低 n 位时怎样判断溢出？ 3. $-15 = Q \times 2 + R$ 按向 0 截断时 Q, R 分别是多少？ 4. 单精度数 16777218 加 1 为什么成为 16777220？

答案。1. 高阻表示输出端不主动驱动总线，而 0 是主动输出低电平。2. 检查完整 $2n$ 位结果是否为低 n 位的符号扩展。3. $Q=-7, R=-1$。4. 精确结果居中，最近偶数舍入选择有效尾数末位为 0 的高端数。

2.15 主动回忆自测

2.15.1 配套自测

1. 原创补充：8 位机器计算 `7FH+01H`，按本章标志约定求低 8 位结果、CF、OF、SF、ZF。
2. 原创补充：binary32 位串 `0000 0001H` 与 `0080 0000H` 各表示什么正数？两者是否都具有隐含前导 1？
3. 【配套例题】已知 32 位 int 变量 i=12345，float 变量 $f = 1.2345 \times 2^3$。下列结果为假的是：A. `i==(int)(double)i`；B. `f==(float)(double)f`；C. `i==(int)(float)i`；D. `f==(float)(int)f`。

答案。1. 结果 `80H`，CF=0、OF=1、SF=1、ZF=0；127+1 越出补码范围，但未越出 8 位无符号范围。2. 分别为 2^{-149} 与 2^{-126}，前者非规格化、隐含 0，后者规格化、隐含 1。3. D。12345 能被 binary32 精确表示；float 到 double 再回来也保留原 float 值，但 f 截断为整数会丢小数部分。

2.15.2 遮住答案复述知识关系

课程连接·从条件出发

回答转换题时先写位宽、编码及符号，再进行位操作和真值计算。

主动回忆·自拟

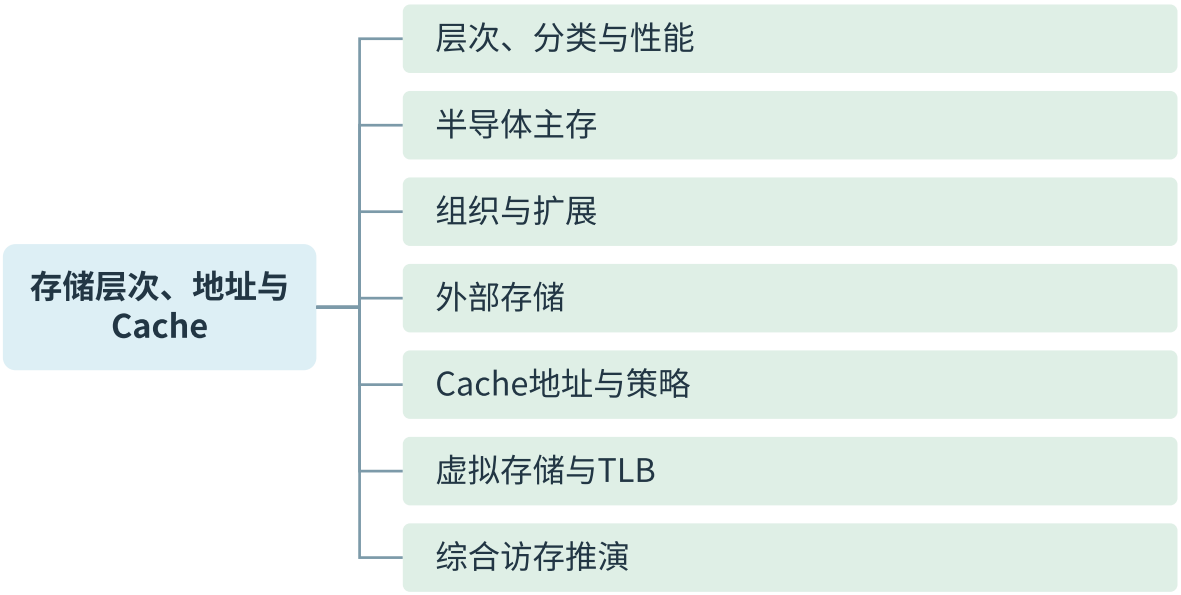
4. 配套结构体案例：某 32 位计算机按字节编址、小端、int 为 4B，按自然边界对齐。结构为 `struct record { int id; char name[10]; int salary; } employee[200];`，数组首址 A0B0H。若 employee[1].id 为 12345678H，其中 56H 的地址是多少?
5. 自拟：8 位补码最小负数取反加 1 得到自身，为什么不能据此说数学相反数等于自身?
6. 自拟：解释 8 位补码 -3 算术右移 1 位和 C 整数 $-3/2$ 为何可能不同。
7. 自拟：binary32 在 $[2^{25}, 2^{26})$ 的相邻间隔是多少？$2^{25}+1$ 按最近偶数舍入到哪里?
8. 自拟：列出恢复余数和不恢复余数法都必须满足的最终商余数关系。
9. 自拟：12345678H 采用大端和小端时，最低地址分别保存哪一字节？字节中的位需要反转吗?

2.15.3 参考答案

4. A0C5H。结构大小 20B，第二元素首址 A0C4H，小端 56H 位于其后 1B。
5. 同宽运算按模 256 保存位串；+128 不在 8 位补码范围，不能把位串绕回当作数学等式。
6. 算术右移复制符号位，-3 变 -2，相当于向负无穷取整；C 整数除法向零截断得 -1。还须核语言版本对负数右移的规定。
7. 间隔 4，$2^{25}+1$ 离 2^{25} 更近，舍入到 2^{25}。
8. 无符号被除数 $N = QD + R$，$D > 0$ 时 $0 \le R < D$；不恢复法可能需最终负余数校正。有符号还需给定商、余数符号及舍入约定。
9. 大端 12H、小端 78H；不反转字节内部位。

3 存储层次、地址与 Cache

章节脉络



本章围绕“层次、分类与性能 → 半导体主存 → 组织与扩展 → 外部存储 → Cache 地址与策略 → 虚拟存储与 TLB → 综合访存推演”展开。先判断对象与条件，再沿机制推导结果，最后用反例和数值检验结论。

课程连接·分层连接操作系统

Cache 利用短期访问局部性，虚拟存储还承担地址空间、隔离和换页。两者都能体现层次思想，但缓存对象与缺失处理不同：Cache 常缓存数据块，TLB 缓存映射，页表记录虚页到页框关系。

3.1 为什么一个程序同时使用几层存储器

定义 3.1.1 (概念与适用对象)

寄存器、Cache、主存和外存的速度、容量及单位成本不同。寄存器紧贴运算部件、容量小；Cache 保存近期可能使用的主存块；主存容纳正在运行的程序和数据；外存提供较大且通常非易失的空间。层次化存储利用程序访问的局部性，使大量访问落在较快的层中，同时保留较大容量。

随机存取指访问时间基本不随单元位置改变，不等于“随机挑一个位置”。顺序存取要按顺序经过前面的内容；直接存取先定位一个区域，再在区域内访问，例如磁盘。相联存储器按内容匹配，TLB 的相联查找就是常见用途。

存取时间是完成一次读或写所需的时间；存取周期是连续两次访问之间允许的最短间隔。DRAM 读后需要恢复等操作，存取周期可能大于单纯读出延迟。计算连续访问的总时间时，不能只拿一次存取时间乘访问次数。

容量要写成“单元数 × 每单元位数”，再结合编址单位换算。2^{20} 个 8 位单元为 1 MiB；本讲义在芯片容量、地址空间、Cache 中使用二进制单位，主频和传输率中的 M、G 按十进制。题源沿用 KB、MB 时，根据地址幂次说明实际口径。

3.2 SRAM、DRAM 与 Flash 保存信息的方式

最小反例·动态不等于访问变化

最小反例：一个程序只反复读同一 DRAM 单元，芯片仍需刷新。动态描述电荷保持机制，而非程序写入频度；随机描述按地址选取能力，也不意味着无需刷新。把这两条分类轴分开。

SRAM 用稳定电路状态保存位，只要供电就不需周期刷新，通常速度快、集成密度相对低，适合 Cache。DRAM 用电容电荷表示位，电荷会泄漏，需要刷新；读出还伴随感知、放大和恢复。它的高密度使其适合主存。

“动态” 说的是保存机制，不是程序是否经常改写数据；“随机” 说的是访问方式，不是是否需要刷新。SRAM、DRAM 都是易失性存储器，掉电后不能保持原信息。

Flash 是非易失性半导体存储器。读、编程和擦除的粒度与代价不同，通常不能像 SRAM 那样直接覆盖写入任意旧位；例如 NAND Flash 常按页读写、按块擦除。SSD 控制器借助映射、垃圾回收和磨损均衡，把复杂的物理操作组织成主机看到的逻辑块访问。掉电保持不等于写入次数无限，也不等于写延迟与读延迟相同。

比较存储器时，要固定同一维度。SRAM 与 DRAM 在“能否随机访问”上相同，在“怎样保持一位、是否刷新、速度与密度”上不同；DRAM 与 Flash 在“是否掉电保持”上不同；Cache 与主存则首先是系统中的层次角色，常分别由 SRAM 和 DRAM 实现，而角色的划分与器件技术的划分不是一回事。芯片题再把器件组织写成单元数 $\times$ 位宽：一片 $4\text{M}\times 8$ 的容量是 4 MiB，不能因它用于主存，就把 “地址线、数据线、存储单元数” 当成同一个数。

3.3 地址引脚不总等于地址位数

SRAM 的 $2^k \times w$ 表示有 2^k 个可选单元，每次选中输出 w 位，通常需 k 根地址线和 w 根数据线，另有片选和读写等控制线。DRAM 常将行地址、列地址分时送入同一组地址引脚，由行、列选通信号分别锁存，因此逻辑地址位数不能直接作为地址引脚数。

3.3.1 例 3-1　DRAM 的复用地址怎样计数

例 3.3.1 (配套例题)

【配套例题】某容量为 256 MB 的存储器由若干 $4\text{M}\times 8$ 位的 DRAM 芯片构成，该 DRAM 芯片的地址引脚和数据引脚总数是（　）。

A. 19　B. 22　C. 30　D. 36

解析。$4\text{M}=2^{22}$，芯片内部需要 22 位单元地址。按题目采用的行列各 11 位、地址复用模型，需 11 根地址引脚；数据宽 8 位，需要 8 根数据引脚，共 19 根，选 **A**。题目给出的整机 256 MB 并不改变单片引脚数。若问组成整机需要多少片，则另算 $256/4=64$ 片。

实际 DRAM 还可能含多个 Bank，地址、命令及引脚结构要按题目给出的组织判断。刷新按行进行，不是每保存一个位就分别发一次刷新。集中刷新把若干刷新操作连在一起，期间可能有一段较长的时间不能正常访问；分散刷新把一次正常存取和一次刷新组合到扩大的存取周期内，不形成集中刷新的长间隙，但会降低正常访存速率。异步刷新在保持时间内把各行刷新均匀分散，定期插入短的刷新周期，其间仍可进行正常访问。若有 R 行、保持时间为 T_r，把每行各刷新一次并均匀分布，平均间隔为 T_r/R；还须检查每次刷新占用多久。

3.3.2 MAR 的位数由地址空间决定

MAR 需容纳可由该机器给出的主存地址。它与 MDR 的位数没有 “必须相等” 的关系。MDR 反映某次数据传送的位宽，MAR 反映地址编码位宽。

最小反例·已安装容量不决定 MAR

最小反例：机器给出 64MiB 字节地址空间，只安装 32MiB 内存。MAR 仍需容纳 26 位地址，而非按已安装容量只取 25 位。芯片单元地址、可编址空间、已安装容量是三个问题。

3.3.3 例 3-2　已安装容量不是全部可编址空间

例 3.3.2 (配套例题)

【配套例题】某计算机存储器按字节编址，主存地址空间大小为 64 MB，现用 $4\text{M}\times 8$ 位的 RAM 芯片组成 32 MB 的主存储器，则存储器地址寄存器 MAR 的位数至少是（　）。

A. 22 位　B. 23 位　C. 25 位　D. 26 位

解析。题目问覆盖 64 MB 地址空间需要多少位，而不是按已安装的 32 MB 去算。$64\text{MB}=2^{26}\text{B}$，每个字节一个地址，所以 MAR 至少 26 位，选 **D**。芯片地址位数 22、已安装容量地址位数 25 都不能回答这一问题。

3.4 单体多字和多体交叉怎样提高带宽

单体多字存储器扩宽一次读写的数据量，一个存储单元包含多个机器字。顺序使用相邻字时，一次访问可以提供多个字；若频繁跳转，取来的其他字未必有用。它主要改善带宽，不能仅凭 “一次取多个字” 就当成解决地址空间不足的方案。

多体存储器则包含能相对独立工作的存储模块。高位交叉（顺序编址）用地址高位选模块，各模块保存连续的一段地址；低位交叉用低位选模块，连续字地址轮流落到不同模块。对 m 个模块且按字编址的常规模型：

$$\text{模块号}=\text{字地址 mod } m,\qquad \text{模块内地址}=\left\lfloor\frac{\text{字地址}}{m}\right\rfloor.$$

若给的是字节地址，每字 w B，且各模块每次存取一字，通常应先由 $\lfloor A/w\rfloor$ 求出字地址，再据此分模块；最低的字节偏移不参与模块编号。多体交叉与单体多字可以结合，但两者不是同一种机制。

设单模块存取周期为 T，连续启动相邻模块的间隔为 r，模块数 m。连续顺序访问不冲突的条件是 $mr\ge T$。若第一字在 T 后到达，后续每隔 r 得到一个字，读 n 个字的时间为 $T+(n-1)r$。第一字延迟仍为 T，提高的是连续吞吐率。

3.4.1 例 3-3　同一模块不等于立即发生冲突

例 3.4.1 (配套例题)

【配套例题】某计算机使用 4 体交叉编址存储器，假定在存储器总线上出现的主存地址（十进制）序列为 8005、8006、8007、8008、8001、8002、8003、8004、8000，则可能发生访存冲突的地址对是（　）。

A. 8004 和 8008　B. 8002 和 8007　C. 8001 和 8008　D. 8000

和 8004

解析。先求地址模 4：1、2、3、0、1、2、3、0、0。最后两次 8004、8000 连续访问模块 0，模块可能尚未结束前次存取，所以选 **D**。8008 与 8004 也在模块 0，却隔了四个访问位置，不能只凭同体就忽略时间间隔。这里沿题目的低位交叉约定解释 “可能冲突”，具体是否冲突仍由模块周期与请求间隔决定。

不变量·位扩展共享片选

同一地址上的多片芯片分担不同数据位，因此片选和片内地址应同步；不同地址组才由高位译码区分。先画组，再连数据线，能避免把位扩展误画成容量分区。

3.5 主存芯片怎样连到 CPU

位扩展把若干芯片并联在同一地址上，各提供数据的一部分；字扩展让若干组芯片承担不同地址范围，用片选区分。组成 $M \times W$ 的存储器，单片容量 $m \times w$，且两个方向都能整除时，位方向需要 W/w 片，字方向需要 M/m 组，总片数为两者乘积。

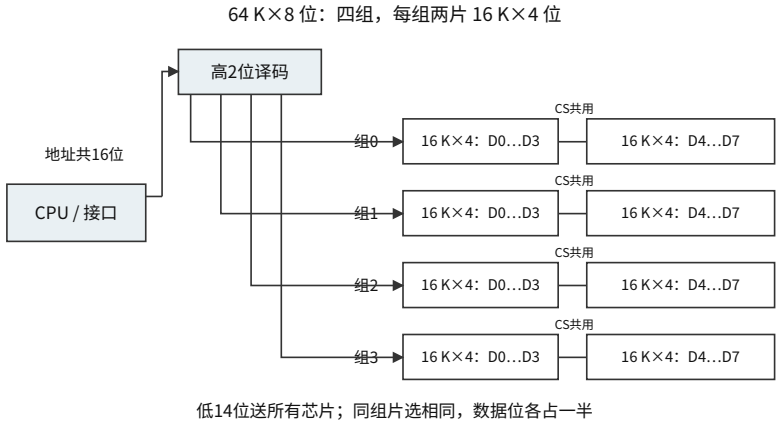


Figure 3.1: 两片并联扩位，四组译码扩字；同组选中信号相同

图中每片 16K × 4 位，同行两片并成 16K × 8 位，四组组成 64K × 8 位。低 14 位送给所有芯片选择片内单元，高 2 位经 2–4 译码器选一组。位扩展增加数据宽度，字扩展增加可选单元数，两者所用连线不同。

3.5.1 例 3-4　ROM 与 RAM 分区要分别计算

例 3.5.1 (配套例题)

【配套例题】某计算机主存容量为 64 KB，其中 ROM 区为 4 KB，其余为 RAM 区，按字节编址。现要用 2K × 8 位的 ROM 芯片和 4K × 4 位的 RAM 芯片来设计该存储器，则需要上述规格的 ROM 芯片数和 RAM 芯片数分别是（　）。

A. 1、15　　B. 2、15　　C. 1、30　　D. 2、30

解析。 ROM 单片 2 KB 且已有 8 位宽，需 $4/2 = 2$ 片。RAM 总量 60 KB；单片虽有 4 K 单元，每单元只有 4 位，要两片并成 4K × 8 位的一组，再用 $60/4 = 15$ 组，共 30 片，选 **D**。只算 60/4 会漏掉位扩展。

确定地址范围时，先确定编址单位，再拼接片选地址和片内地址。首地址由片选编码后接全 0 构成，末地址由片选编码后接全 1 构成。例如按字编址，32 K×16 位芯片的片内字地址为 15 位；若高 3 位片选为 111，则字地址范围为 `38000H` 至 `3FFFFH`。若改按字节编址，每个字占 2 B，片内字节范围需 16 位，范围随之改变，不能沿用同一答案。

地址线的编号方式同样要从题干读。通常本书令 A0 为最低位；有的题明确写 A0 为最高位，此时 4 K 片内地址使用最后 12 根低位地址线，片选应取其上方的线。下标数字不是判断高低位的可靠依据。

错项排除·平均旋转等半圈

转速 r 转每分意味着一圈 60/r 秒；均匀随机角位置的平均等待为半圈，传一扇区另按一圈中该扇区比例计算。SSD 没有这段机械旋转过程，不能给它机械套用半圈等待。

3.6 磁盘与 SSD 的一次访问花在哪些时间上

磁盘由盘面、磁道和扇区组织信息。同一半径处各盘面的磁道组成柱面；选择盘面由磁头完成，寻道把磁头移动到相应磁道。旋转等待使目标扇区转到磁头下方，随后才传输数据。磁盘地址的典型几何字段是柱面号、磁头号、扇区号；磁头号已经指定盘面，不必再把“盘面号”另算一个定位维度。

$$T_{\text{access}} = T_{\text{seek}} + T_{\text{rotation}} + T_{\text{transfer}}.$$

只有题设要求时再计控制器或排队开销。转速为 r 转/分时，一圈为 60/r 秒；随机到达且目标角位置均匀时，平均旋转等待为半圈。传输一个扇区的时间只占一圈的相应部分，不能把等待的半圈再算进传输时间。

3.6.1 例 3-5　7200 转每分不是 7200 转每秒

例 3.6.1 (配套例题)

【配套例题】若磁盘转速为 7200 转/分，平均寻道时间为 8 ms，每个磁道包含 1000 个扇区，则访问一个扇区的平均存取时间大约是（　）。

A. 8.1 ms　B. 12.2 ms　C. 16.3 ms　D. 20.5 ms

解析。一圈 $60/7200$ s ≈ 8.333 ms，平均旋转等待约 4.167 ms；一扇区传输约 $8.333/1000 = 0.008333$ ms。总时间约 12.175 ms，选 **B**。把一整圈当成平均等待，会得到另一数量级的选项。

SSD 没有机械寻道和旋转等待，但仍有接口传输、闪存访问、控制器与写放大等开销。因此不能照搬磁盘的平均旋转等待公式。RAID 把多个盘组织起来：条带化提升并行访问能力，镜像提供冗余，奇偶校验可支持某些盘故障后的恢复。RAID0 没有数据冗余；即便采用冗余，也不等于可以代替备份：误删的内容仍可能被同步写到冗余盘。

3.7 Cache 命中率怎样变成时间

时间局部性指近期访问过的信息可能很快再访问，例如循环累加器；空间局部性指相邻位置可能很快被访问，例如顺序扫描数组。Cache 按块调入，因此顺序数组中一次缺失可能为随后几个元素准备数据。

命中率 $h = N_{\text{hit}}/(N_{\text{hit}} + N_{\text{miss}})$。平均访问时间必须先明确“未命中”指的是哪一段时间。若先查 Cache 花 t_c，未命中后再用 t_m 完成取块，则

$$\text{AMAT} = t_c + (1 - h)t_m.$$

若题目给出的“未命中访问时间”已经包含查 Cache 的时间，记为 t_{miss}，则写为 $ht_c + (1-h)t_{\text{miss}}$。两式只是对同一事件的不同分解，第

二式中不能再额外加一次 t_c。多级 Cache 则应区别局部缺失率与相对 CPU 总访问的全局缺失率。

最小反例·循环轮数不是访问次数

最小反例：一条 a[k]=a[k]+32 每轮读一次又写一次，因此四元素八次数据访问。空 Cache 顺序扫描时一块首次读缺失，后续写命中。分母错用四轮，就把正确 12.5% 误算成 25%。

3.7.1 例 3-6　读后写的数组访问怎样计缺失率

例 3.7.1 (配套例题)

【配套例题】有如下 C 语言程序段：

```
for (k = 0; k < 1000; k++)
    a[k] = a[k] + 32;
```

若数组 a 和变量 k 均为 int 型，int 型数据占 4 B，数据 Cache 采用直接映射方式，数据区大小为 1 KB、块大小为 16 B，该程序段执行前 Cache 为空，则该程序段执行过程中访问数组 a 的 Cache 缺失率约为（　）。

A. 1.25%　　B. 2.5%　　C. 12.5%　　D. 25%

解析。每块容纳 4 个 int。按顺序扫描，每块第一次读缺失，其余 3 次读命中；第一次读已把块取入，随后对应的写也命中。每 4 个元素共 8 次数组访问、1 次缺失，因此约为 12.5%，选 **C**。若数组首地址未按块对齐，块数可能多 1，不影响本题 “约为” 的结论。不能把循环 1000 轮误当作只访问数组 1000 次。

3.7.2 一次缺失损失要不要反复付启动代价

若每次取 8 B 需发送地址 1 周期、准备 8 周期、传输 1 周期，64 B 块包含 8 次独立访问，总计 $8(1+8+1)=80$ 周期。若支持突发，只发一次首地址，等待首数据 8 周期后连续传 8 次，则共 $1+8+8=17$ 周期。是否支持突发会改变这里的计算结果，详见第 6 章；不能只看总线同为 64 位就给出同一个时间。

3.8 地址拆分决定块能放在哪里

先由数据容量 C 和块大小 B 求行数 $L=C/B$。若每组 r 行，则组数 $G=L/r$。按字节编址且 B、G 为 2 的幂时，块内偏移为 $b=\log_2 B$ 位，组号为 $g=\log_2 G$ 位，剩下高位为标记。若 Cache 总容量已经包含标记等附加位，这个总量就不能再当作 C。

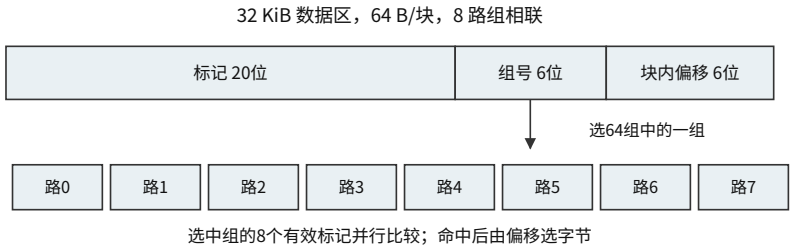


Figure 3.2: 先去块内偏移，再用块号确定组和标记

常规低位选组的规则为：主存块号 $q=\lfloor A/B \rfloor$，组号 $q \bmod G$，标

记 $\lfloor q/G \rfloor$。直接映射是 r=1，全相联是 G=1；r 路组相联中的 “路数” 是每组行数，不是组数。

在地址位数、数据容量 C 和块大小 B 都相同的条件下，三种映射可放在同一框架比较。表中 C 为数据区容量，与 B 采用相同单位；块大小和组数按 2 的幂组织，字段位数才可直接写成以 2 为底的对数。

映射	每块可放的位置	地址中的组选字段	一次查找的比较范围
直接映射	唯一一行	$\log_2(C/B)$ 位	选中行的一个标记
r 路组相联	唯一一组内的 r 行	$\log_2(C/(Br))$ 位	该组 r 个有效标记
全相联	全 Cache 任一行	0 位	全部行的有效标记

同一过程的其余状态（按首列对应）：

映射	满时怎样选牺牲行
直接映射	无行间选择
r 路组相联	仅在该组内替换
全相联	在全 Cache 内替换

最小反例·命中会改变 LRU

最小反例：两路一组访问 A、B、A、C。LRU 在第三次命中 A 后淘汰 B，FIFO 仍按装入次序淘汰 A。只记录装入顺序无法模拟 LRU，命中也必须更新最近使用状态。

提高相联度时，块内偏移不变，组选字段变短，标记相应变长；比较器和替换状态的成本也可能增加。这样做能减少某些冲突缺失，却不消除首次访问的强制缺失，也不能让有限容量容纳任意大的工作集。映射决定候选位置，替换决定满组时选谁，写策略决定何时更新下一层，应分别求解。

访问时用组号选一组，并行比较该组各行的有效标记。组号已经用于选组，不需要再保存到每行标记中；块内偏移只负责从已找到的块中选字节。

3.8.1 例 3-7　8 路组相联需要几个比较器

例 3.8.1 (配套例题)

【配套例题】若计算机主存地址为 32 位，按字节编址，某 Cache 的数据区容量为 32 KB，主存块大小为 64 B，采用 8 路组相联映射方式，该 Cache 中比较器的个数和位数分别为（　）。

A. 8，20　　B. 8，23　　C. 64，20　　D. 64，23

解析。行数 $2^{15}/2^6 = 2^9$，组数 $2^9/8 = 2^6$。偏移 6 位、组号 6 位，标记 $32-6-6 = 20$ 位。一次只在选中的一组内比较 8 个标记，因此 8 个 20 位比较器，选 **A**。组数为 64，并不表示同一次访问就要比较 64 个组。

3.9 LRU 更新的是访问次序，不只是装入次序

当目标组有空行时先用空行；满了才按替换算法决定牺牲哪一行。直接映射每个块只有一个位置，不需要在多行间选择。FIFO 记录装入次序，命中一般不改变队列；LRU 记录最近访问次序，命中也要更新；随机替换不跟踪精确最近次序。

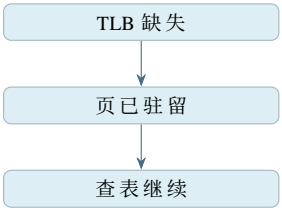
下面以两路的一个组解释 LRU 状态，顺序从最近使用到最久未使用。开始为空，访问 A 后为 [A]，访问 B 后为 [B,A]，再次访问 A 后为 [A,B]。此时访问 C，应淘汰 B，变为 [C,A]。若按 FIFO，则第三次访问 A 不改变装入队列，访问 C 会淘汰 A。这组状态足以说明两算法的分歧。

执行替换题的表格应至少记录本次地址、块号、组号、命中与否，以及该组更新后的状态。替换仅发生在对应组内，不能从其他组挑一个

“全 Cache 最久未用” 的块。若题干改用高位或特殊函数选组，先按题干给出的映射分组；LRU 本身不负责决定组号。

最小反例·映射没缓存不等于没驻留

最小反例：TLB 未命中，但页表存在位为 1。查表后即可得到页框号，无需从磁盘调页。TLB 缺失只说明映射未缓存，缺页才说明合法目标页当前不驻留。



TLB 未命中且存在位为 1，不发生缺页。

3.10 写命中与写缺失是两个独立问题

写命中时，直写同时更新 Cache 和下一层；回写只改 Cache 并置修改位，替换脏块时才写回。直写常配写缓冲，以减少等待，但缓冲满时仍可能阻塞。回写时主存内容暂时旧于 Cache，这是允许的受控状态，不能凭 “二者立即不同” 就断定设计有错。

写缺失时，写分配先调入块再更新 Cache，非写分配直接写下一层而不把该块装入 Cache。回写常配写分配、直写常配非写分配，这是常用组合，不是逻辑上仅有的两种组合。

3.10.1 例 3-8　Cache 总容量还包括哪些位

例 3.10.1 (配套例题)

【配套例题】假定主存地址为 32 位，按字节编址，主存和 Cache 之间采用直接映射方式，主存块大小为 4 个字，每字 32 位，采用回写方式，则能存放 4 K 字数据的 Cache 的总容量的位数至少是（　）。

A. 146 K　　B. 147 K　　C. 148 K　　D. 158 K

解析。一块 $4 \times 4 = 16$ B，块内偏移 4 位。4 K 字数据共 16 KB，Cache 有 $4096/4 = 1024 = 2^{10}$ 行，行号 10 位，标记 18 位。每行数据 128 位，还需有效位 1 和修改位 1，共 $128 + 18 + 1 + 1 = 148$ 位；总计 $1024 \times 148 = 148$ K 位，选 **C**。直接映射无需 LRU 替换位。本题 “至少” 按这些必要字段计算，不额外猜测 ECC 等未给出的实现。

多个 CPU 各有 Cache 时，一个核的写入还涉及其他副本怎样失效或更新；DMA 与 Cache 也可能有一致性处理。单核回写正确不等于多核副本会自动一致。协议细节超出本章基本模型，此处保留 “需要一致性机制” 的边界，不把全写法当作解决任意多副本问题的证明。

3.11 虚拟存储器怎样让程序看到地址空间

虚拟存储器把进程使用的虚拟地址映射到物理地址，允许部分内容暂存外存，也支持隔离与保护。它不是 “用硬盘代替所有内存”：CPU 最终仍需要可访问的物理存储，缺页时还要调入并等待。

操作系统建立、维护页表等映射信息，决定分配、换入换出及保护；运行时 MMU 根据这些信息实施地址转换并检查权限。把 “映射由 OS 管理” 与 “每次访存都调用 OS 软件转换” 混为一谈，会误判硬件路径。正常的 TLB 命中不需要逐次陷入内核。

3.12 页表与 TLB：地址偏移为何原样保留

页式系统把虚拟地址分为虚页号 VPN 和页内偏移 d；页表查得物理页框号 PPN 后，物理地址为 PPN $\times P + d$，P 是页大小。因为页和页框大小相同，页内位置不变。虚拟地址位数和物理地址位数可以不同，改变的是页号部分。

TLB 是页表项的高速缓存。TLB 未命中表示缓存里没有本次所需的映射，不表示页面一定不在主存；随后可以查页表并填入 TLB。对于地址合法且映射已建立的页面，存在位为 0 表示尚未驻留，按题设触发缺页异常。非法地址、无效映射或权限不足的情况须另行检查，不能一律按可调页修复处理；保护异常也可能发生在页面已经在主存时。

不变量·写回与写分配独立

直写或回写回答命中时怎样更新下一层，分配或不分配回答缺失时是否调入块。常见搭配不是逻辑上的唯一搭配。读清题设组合，再决定缺失数、主存写次数和脏块处理。

3.12.1 TLB、页表与 Cache 各自缺少什么

结构	查找所依据的信息	保存的主要对象	未命中或无效后继续做什么
TLB	虚页号；组相联时再拆组号和标记	虚页到物理页框的映射及权限等	查页表；此时还不能断定缺页
页表	虚页号，可能经过多级索引	页框号、存在位、权限及管理状态等	地址合法且未驻留时按题设缺页；非法映射与保护失败另行处理
物理地址访问的 Cache	物理块号；再拆组号和标记	主存数据块或指令块的副本	访问下一存储层，按写策略调块或写出

因此，TLB 命中仍可能 Cache 缺失；TLB 未命中也可能页面已在主存。不能把三个结构的 “有效” 都当成 “原数据已经在 Cache”。TLB 表项存的是地址映射，不是一页完整数据；Cache 行的标记也不等同于物理页框号。

按 “先转换、后查物理 Cache” 的串行教学模型，一次普通数据读可这样展开：先由虚拟地址分出 VPN 与页内偏移；用 VPN 查 TLB，未命中则走题设页表层数；取得有效页框号后拼出物理地址；再由物理地址分 Cache 标记、组号、块内偏移，决定从 Cache 还是下一层取得数据。若中间缺页，先处理并重试指令，不把尚未形成的物理地址继续送去查数据 Cache。计算次数或时间时，就沿这条路径逐项累计；页表访问是否又经过 Cache、TLB 与 Cache 能否重叠，均须服从题设。

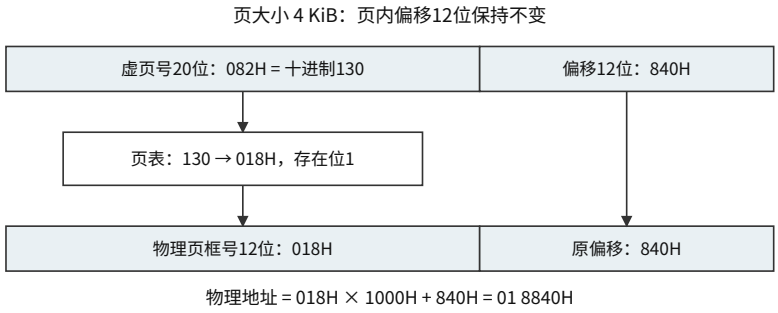


Figure 3.3: 虚拟地址先查映射，再用页框号与原偏移形成物理地址

3.12.2 例 3-9　页号的十六进制不能直接当表中十进制

例 3.12.1 (配套例题)

【配套例题】某计算机主存地址为 24 位，采用分页虚拟存储管理方式，虚拟地址空间大小为 4 GB，页大小为 4 KB，按字节编址。某进程的页表部分内容如下。

虚页号	实页号（页框号）	存在位
82	024H	0
…	…	…
129	180H	1
130	018H	1

当 CPU 访问虚拟地址 0008 2840H 时，虚—实地址转换的结果是（ ）。

A. 得到主存地址 02 4840H　B. 得到主存地址 18 0840H
C. 得到主存地址 01 8840H　D. 检测到缺页异常

解析。页大小 2^{12} B，低 12 位是偏移 `840H`，虚页号 `82H` 等于十进制 130，并非表中第一行的十进制 82。查到页框 `018H` 且存在位 1，物理地址为 `018H×1000H+840H=018840H`，选 **C**。计算前逐列辨明进制，可避免把本题误判为缺页。

3.12.3 例 3-10　TLB 标记不包含页内偏移

例 3.12.2 (配套例题)

【配套例题】某计算机按字节编址，采用页式虚拟存储管理方式，虚拟地址为 32 位，主存地址为 30 位，页大小为 1 KB。若 TLB 共有 32 个表项，采用 4 路组相联映射方式，则 TLB 表项中标记字段的位数至少是（ ）。

A. 17　B. 18　C. 19　D. 20

解析。偏移 10 位，VPN 为 22 位。TLB 共有 8 组，选组用 VPN 的 3 位，标记为 $22-3=19$ 位，选 **C**。物理地址 30 位决定页框号位数，不决定用来匹配虚页的标记位数；也不能把 Cache 块偏移的位数套用到 TLB 标记上。

计算含 TLB 的平均时间时，应画出题设路径。例如，设访问串行进行，TLB 命中率为 h，TLB 耗时 t_T，单级页表和主存访问各耗时 t_M，则不计缺页时的平均时间为 $h(t_T+t_M)+(1-h)(t_T+2t_M)$。多级页表、页表项缓存、TLB 与 Cache 并行查找都会改变路径。缺页还包括外存和操作系统处理，通常远大于一次普通主存访问。

3.13 段式与段页式地址为什么需要界限检查

不变量·段界限先约束偏移

段式地址包含段号与段内偏移，应先核段表和偏移是否在界限内，再形成物理地址或进入分页。扩大基址不能修复越界偏移；非法地址与合法页未驻留也不能混成同一种缺页。

段式系统按代码、数据等逻辑单位划分不等长段。逻辑地址含段号和段内偏移，段表记录段基址、段长及权限。若段长表示可用字节数 L，必须先验证 $0 \le d < L$，再计算物理地址 base $+ d$；当 d=L 时已经越界，不能把末地址写成 base+L。

段页式先按段号找到该段页表，再用段内页号查页框，最后拼接页内偏移。段表基址、页表基址和最终数据地址不是同一个量。若段本身有界限，页号合法仍不保证最后一页的所有偏移均合法。

例如某段基址 10000 B、段长 3000 B，偏移 2999 合法、物理地址 12999 B；偏移 3000 非法。若段页式采用 1 KB 页，该段的逻辑长度仍为 3000 B，最后一个页号 2 中的偏移 952 对应段内 3000，应被界限检查拒绝，不能因页框还有空间就允许访问。

3.14 综合访存：数组次序、TLB 替换与 Cache 查找

错项排除·缺页后停止本轮查数据

若页表判断目标合法但未驻留，本轮尚无可用页框号，不能拿旧字段继续查数据 Cache。先完成缺页处理、更新映射并重执行访问。题目未给主存字节时，也不能凭路径猜出数据值。

3.14.1 例 3-11　交换内外层循环，命中率一定降低吗

例 3.14.1 (配套例题)

【配套例题】改编（两个赋值式统一为常量 10）已知计算机 M 字长为 32 位，按字节编址，采用请求调页策略的虚拟存储管理方式，虚拟地址 32 位，页面大小为 4 KB；数据 Cache 采用 4 路组相联映射方式，数据区大小为 8 KB，主存块大小为 32 B。现有 C 语言程序段如下：

```
int a[24][64];
/* ... */
for (i = 0; i < 24; i++)
    for (j = 0; j < 64; j++) a[i][j] = 10;
```

已知二维数组 a 按行优先存放，在虚拟地址空间中分配的起始地址为 0042 2000H，`sizeof(int)=4`。假定在 M 上执行上述程序段之前数组 a 不在主存，且在该程序执行过程中不会发生页面置换。请回答下列问题。

（1）数组分布在几个页面中？对于数组 a 的访问，会发生几次缺页异常？页故障地址各是什么？

（2）不考虑变量 i 和 j，该程序段的数据访问是否具有时间局部性？为什么？

（3）计算机 M 的虚拟地址（A31～A0）中哪几位用作块内地址？哪几位用作 Cache 组号？a[1][0] 的虚拟地址是多少？其所在主存块对应的 Cache 组号是多少？

（4）数组 a 占用多少主存块？假设上述程序段执行过程中数组 a 的访问不会和其他数据发生 Cache 访问冲突，则数组 a 的 Cache 命中率是多少？若将循环中 i 和 j 的次序按如下方式调换，则数组 a 的 Cache 命中率又是多少？

```
for (j = 0; j < 64; j++)
    for (i = 0; i < 24; i++) a[i][j] = 10;
```

解析。先把数组元素下标化为字节地址，再分别按页和 Cache 块划分。按行优先，每行 64 个 int，共 256 B，因此

$$\mathrm{addr}(a[i][j]) = \texttt{00422000H} + (64i + j) \times 4.$$

本题参考解按写分配模型处理首次写缺失：写一个尚不在 Cache 中的块时先调入该块。后续命中率均采用这一约定；若明确改为非写分配，结论须重算。数组初始不在主存，因此不能把数组块预先算作有效命中。

（1）数组大小为 $24 \times 64 \times 4 = 6144$ B，即 6 KiB。起始地址低 12 位为 0，恰在 4 KiB 页边界；前 16 行占第一页 4096 B，后 8 行占第二页

2048 B，所以共 2 页。

第一次执行 a[0][0] 时缺页，故障地址为 0042 2000H；第一次执行 a[16][0] 时进入下一页，故障地址为 0042 3000H。无页面置换，每页首次调入后保留，共 2 次缺页。6 KiB 除以 4 KiB 后向上取整在这里成立，是因为首地址页对齐；若数组从页中部开始，必须把首地址页内偏移也计入。

错项排除·缺页后停止本轮查数据

若页表判断目标合法但未驻留，本轮尚无可用页框号，不能拿旧字段继续查数据 Cache。先完成缺页处理、更新映射并重执行访问。题目未给主存字节时，也不能凭路径猜出数据值。

（2）以数组元素为数据对象，各元素只被写一次，没有重复访问同一元素，因此没有时间局部性。按行连续访问相邻元素，具有空间局部性。同一 Cache 块中相邻元素的后续访问能够命中，正是按块调入利用空间局部性的结果；不应据此认为同一元素 “又访问了一次”。

（3）块为 2^5 B，块内地址为 A4～A0。Cache 行数 $8192/32 = 256$，4 路共有 64 组，组号 6 位，对应 A10～A5。虽然映射最终作用于物理块，但这些位都在低 12 位页内偏移内，地址转换不改变它们，因此可以直接从虚拟地址取出本题组号。

a[1][0] 在起点之后 256 B，即 0042 2100H。低 11 位中的组号为 `(2100H>>5) mod 64=8`，二进制 001000。不要把 “第 1 行” 当成只跨过一个 int。

（4）每块 32 B 可容纳 8 个 int，起点块对齐，总块数 $6144/32 = 192$。按行访问时，每块第一次写缺失，随后 7 次写命中，共 1536 次写、192 次缺失，命中率为

$$\frac{1536 - 192}{1536} = \frac{7}{8} = 87.5\%.$$

交换循环后，每个 j 先跨 24 行访问，同一列相邻两次地址相差 256 B，即相差 8 个块，组号也按模 64 增加 8。固定 j 时，24 个块轮流落在 8 个组中，每组恰好 3 块，**4 路容得下这 3 块**。因此不能仅凭 “按列走” 就断言块被赶走。

例如 j=0 时，行号 0、8、16 的块都映射到组 0，分别占据该组 3 路；第 4 路仍空。j=1 至 7 仍访问这 24 个已装入块，均命中。j=8 首次使用下一批 24 块，其组号在原来基础上加 1，与前批分属不同组；以后同理。全数组 192 块按映射可以同时驻留，仍只有 192 次首次缺失，命中率仍为 87.5%。

这个结论同时依赖数组大小、起始对齐、组数、路数和无其他数据冲突这些条件。不能只凭 “数组 6 KiB 小于 Cache8 KiB” 就下结论，因为总容量装得下，仍可能在少数组内发生冲突。

关键变式（原创）。若仅把 Cache 改成 4 KiB、2 路，保留 64 组和 32 B 块，则每个相关组仍轮流访问 3 块，却只有 2 路。按列遍历、LRU 替换时，三块循环会互相淘汰，下一列也难以复用，数组写访问全部缺失；按行遍历仍能在同块的 8 次访问中命中 7 次。决定解法的条件是每组能容纳的块数，而不是 “换循环必然降低命中率” 这类笼统说法。

3.14.2 例 3-12　TLB 的组选、标记与连续替换

例 3.14.2 (配套例题)

【配套例题】假设计算机 M 的主存地址为 24 位，按字节编址；采用分页存储管理方式，虚拟地址为 30 位，页大小为 4 KB；TLB 采用 2 路组相联方式和 LRU 替换策略，共 8 组。请回答下列问题。

（1）虚拟地址中哪几位表示虚页号？哪几位表示页内地址？

（2）已知访问 TLB 时虚页号高位部分用作 TLB 标记，低位部

分用作 TLB 组号，M 的虚拟地址中哪几位是 TLB 标记？哪几位是 TLB 组号？

（3）假设 TLB 初始时为空，访问的虚页号依次为 10、12、16、7、26、4、12 和 20，在此过程中，哪一个虚页号对应的 TLB 表项被替换？说明理由。

（4）若将 M 中的虚拟地址位数增加到 32 位，则 TLB 表项的位数增加几位？

错项排除·缺页后停止本轮查数据

若页表判断目标合法但未驻留，本轮尚无可用页框号，不能拿旧字段继续查数据 Cache。先完成缺页处理、更新映射并重执行访问。题目未给主存字节时，也不能凭路径猜出数据值。

解析。这里替换的是地址映射表项，先按 VPN 选 TLB 组，再在所选组内更新 LRU；不依据物理页框号选组，也不对全部 16 项作一次全局 LRU。

（1）4 KB=2^{12} B，故虚拟地址 A11～A0 为页内偏移，A29～A12 共 18 位为 VPN。物理地址 24 位只决定页框号为 12 位，不能据此把虚页号也截成 12 位。

（2）8 组需 3 位组号，取 VPN 最低 3 位，即虚拟地址 A14～A12；余下 A29～A15 共 15 位为 TLB 标记。把字段写成位数核对：$15+3+12=30$。

（3）题中虚页号均为十进制。组号为 VPN mod 8，标记为 $\lfloor \mathrm{VPN}/8 \rfloor$。下表的状态从最久未使用到最近使用，只列本次选中的组。

次序	VPN	组号	标记
1	10	2	1
2	12	4	1
3	16	0	2
4	7	7	0
5	26	2	3
6	4	4	0
7	12	4	1

次序	VPN	组号	标记
8	20	4	2

同一过程的其余状态（按首列对应）：

次序	命中与否	本次访问后该组的 VPN 次序
1	未命中，空行装入	[10]
2	未命中，空行装入	[12]
3	未命中，空行装入	[16]
4	未命中，空行装入	[7]
5	未命中，第二路装入	[10,26]
6	未命中，第二路装入	[12,4]
7	命中，更新最近次序	[4,12]

次序	命中与否	本次访问后该组的 VPN 次序
8	未命中，淘汰 4	[12,20]

只有组 4 遇到第三个不同虚页。第 7 次命中 12 后，最久未用的是 4，所以访问 20 时替换 VPN=4 的表项。若只按装入次序排队，会错误淘汰 12；若只看全部 16 项里已用了几项，还会误以为未满就无需替换。

（4）页大小、组数和主存地址位数都不变。新增的 2 位虚拟地址属于 VPN 高位，全部进入 TLB 标记；页框号仍 12 位，故每个 TLB 表项增加 2 位。增加虚址位宽扩大了可区分的虚页范围，不会自动增加实际主存容量或 TLB 项数。

错项排除·缺页后停止本轮查数据

若页表判断目标合法但未驻留，本轮尚无可用页框号，不能拿旧字段继续查数据 Cache。先完成缺页处理、更新映射并重执行访问。题目未给主存字节时，也不能凭路径猜出数据值。

3.14.3 例 3-13　TLB 未命中后，还要经过哪些判断

例 3.14.3 (配套例题)

典型示例（据王道《操作系统》§3.2.9 摘编、重排）。某系统按字节编址，虚拟地址为 14 位，物理地址为 12 位，页大小为 64 B。TLB 共 16 项，采用 4 路组相联；数据 Cache 使用物理地址，采用直接映射，共 16 行，每行 4 B。

按“先完成地址转换，再查数据 Cache”的串行模型分析。所有待访问的虚拟地址均合法，且允许读；页表存在位 $P=1$ 表示页面已在主存，$P=0$ 表示合法但尚未驻留，需要处理缺页。读 Cache 未命中时，从主存调入对应的 4 B 数据块。

以下给出本题所需的初始状态。表中页号、页框号、标记、Cache 行号和字节数据均用十六进制，后缀 H 表示十六进制；V 为有效位，“—”表示没有可用信息。TLB 的组号和路号均在 0 至 3 之间。

表 A　TLB 初始状态：每行对应一个表项

组号	路号	标记	有效位 V
0	0	03H	0
0	1	00H	0
0	2	09H	1
0	3	07H	1
3	0	07H	0
3	1	0AH	1
3	2	03H	1

组号	路号	标记	有效位 V
3	3	02H	0

同一过程的其余状态（按首列对应）：

组号	页框号
0	—
0	—
0	0DH
0	02H
3	—
3	34H
3	0DH

组号	页框号
3	—

表 B　页表中与本题有关的表项

虚页号	存在位 P	页框号
03H	1	02H
08H	0	—

表 C　Cache 初始状态：数据列按块内字节偏移排列

行号	标记	V	偏移 0
5H	0DH	1	36H
CH	02H	0	—

同一过程的其余状态（按首列对应）：

行号	偏移 1	偏移 2	偏移 3
5H	72H	F0H	1DH
CH	—	—	—

（1）写出查 TLB 时虚拟地址的字段划分，以及查 Cache 时物理地址的字段划分，并求各字段位数。

（2）分别读取虚拟地址 03D4H、00F1H、0229H 处的一个字节，依次说明 TLB 查找、必要的页表检查和 Cache 查找，指出最终从哪里取数，或在哪一步转入异常处理。能确定字节值时给出结果。

三次访问相互独立，均从上述初始状态开始。某次访问引起的回填不影响另两次；不推演缺页后的页框分配与替换，也不计算访问时间。

错项排除·缺页后停止本轮查数据

若页表判断目标合法但未驻留，本轮尚无可用页框号，不能拿旧字段继续查数据 Cache。先完成缺页处理、更新映射并重执行访问。题目未给主存字节时，也不能凭路径猜出数据值。

解析。先分别拆分虚拟地址和物理地址，再沿实际访问路径检查各表项。

（1）两种地址的字段划分。页大小为 $64 = 2^6$ B，页内偏移占 6 位，虚页号 VPN 占 $14 - 6 = 8$ 位。TLB 有 $16/4 = 4$ 组，用 VPN 的低 2 位选组，其余 6 位作为标记。因此，从高位到低位，虚拟地址分为 **TLB 标记 6 位、组号 2 位、页内偏移 6 位**。

Cache 每行 4 B，块内偏移占 2 位；共 16 行，行号占 4 位，其余 $12 - 4 - 2 = 6$ 位作为标记。因此，物理地址分为 **Cache 标记 6 位、行号 4 位、块内偏移 2 位**。页内偏移和 Cache 块内偏移的长度不同，不能混用。

（2）三次独立的读访问。判断顺序如下。

- **查 TLB：**在选中组内，只有 $V = 1$ 且标记相同的表项才算命中。命中可取得页框号；未命中则查页表，此时还不能断定缺页。
- **必要时查页表：**本题已保证地址合法且有读权限。若 $P = 1$，取得页框号，并可回填 TLB；若 $P = 0$，转入缺页处理，当前查找停止。
- **形成物理地址后查 Cache：**用物理地址选行，再同时检查有效位和标记。$V = 1$ 且标记相同才命中；否则从主存调块，按块内偏移取出目标字节。

读取 03D4H：TLB 命中，Cache 也命中。虚页号为 0FH，页内偏移为 14H。VPN 的低 2 位给出 TLB 组号 3，其余位给出标记 03H。表 A 的组 3、路 2 有效且标记为 03H，取得页框号 0DH，无需再查页表。物理地址为

$$0\mathrm{DH} \times 40\mathrm{H} + 14\mathrm{H} = 354\mathrm{H}.$$

拆分 354H，得到 Cache 标记 0DH、行号 5H、块内偏移 0。表 C 中第 5H 行的 $V=1$，标记也是 0DH，因此从该行偏移 0 的位置读得 **36H**。

读取 00F1H：TLB 未命中，页面已驻留，Cache 未命中。虚页号为 03H，页内偏移为 31H；TLB 组号为 3、标记为 00H。组 3 没有有效的匹配项，因此查表 B。虚页 03H 的 $P=1$、页框号为 02H，说明页面已在主存，**不发生缺页**。物理地址为

$$02\mathrm{H}\times 40\mathrm{H}+31\mathrm{H}=0\mathrm{B1H}.$$

拆分 0B1H，得到 Cache 标记 02H、行号 CH、块内偏移 1。第 CH 行虽保留着相同标记，但 $V=0$，所以仍未命中。须从主存调入地址 0B0H 至 0B3H 所在的 4 B 块，再取偏移 1 的字节。题中没有给出该主存字节的内容，**只能确定取数路径，不能确定字节值**。

读取 0229H：TLB 未命中，随后发生缺页。虚页号为 08H，页内偏移为 29H；TLB 组号为 0、标记为 02H。组 0 没有有效的匹配项，转查页表。表 B 中虚页 08H 的 $P=0$，因此触发缺页异常。此时尚无可用页框号，**不形成物理地址，也不继续查数据 Cache**。由操作系统完成缺页处理、更新映射并重新执行该访问后，才继续取数。

三次访问分别在 TLB、页表和 Cache 处作出不同判断。TLB 缓存的是映射，页表存在位说明页面是否驻留，Cache 保存的是数据块：**TLB 未命中不等于缺页，页面已驻留不等于 Cache 命中，标记相同也不能代替有效位检查**。真实系统还须区分非法地址和访问越权；本题通过 “地址合法、允许读” 的条件将这两类异常排除。

3.15 先画访问路径，再写时间与容量

存储系统综合题的顺序

1. 写明地址位数、按字节还是按字编址、块与页大小，把地址先拆成有意义的字段。
2. 按题设画 TLB、页表、Cache 和主存的实际查找路径；分清并行与串行，缺页与普通缺失。
3. 每一步同时检查标记、有效或存在位、权限；得到物理地址后再按指定 Cache 模型查数据。
4. 时间式以完整事件分支加权；容量式先分数据位、标记和有效/脏/替换等元数据。
5. 回代一个边界：空 Cache、组数 1、路数 1、页面不驻留或跨页，查看路径是否仍一致。

3.15.1 地址拆分为什么不遗漏信息

从整除和余数恢复地址

设字节地址 A，块大小 B，组数 G，各量为适用范围内的整数。先作 $A=qB+d$，其中 $0\le d<B$；再作 $q=tG+s$，其中 $0\le s<G$。因此 $A=(tG+s)B+d$，恰由标记 t、组号 s 和块内偏移 d 恢复。B、G 为 2 的幂时，这两次除法正对应二进制位段切分；不是 2 的幂时仍可整除取模，但不能硬套整数位数。

□

课程连接·从路径列事件

先列访问哪些层级、命中或缺失时发生哪些访问，再按模型计算平均时间；不要把不同路径的额外时间重复计入。

3.15.2 主存器件分类与刷新开销

不变量·相联程度不直接等于延迟

全相联能并行比较多行，组相联只比较一个组；实际延迟还受比较器、布线和编码影响。固定实现才可比较快慢。映射限制候选位置，替换选择牺牲行，电路时序决定访问时间。

只读存储器的“只读”描述其正常使用或修改方式，不表示所有 ROM 永远无法改写。掩膜 ROM 由制造过程固定，PROM 可一次编程，EPROM 可通过特定方式擦除重写，EEPROM 支持电擦除；Flash 也是电可擦写的非易失存储，但擦除粒度通常较大。判断题先区分是否掉电保持、能否改写、擦写粒度与速度，避免把这些维度混成同一个分类。

刷新保持 DRAM 电容状态，读后恢复则由一次读出带来的状态变化引起；二者目的相关，但不能据“读后要恢复”推断只有被访问的行才需刷新。若 4096 行必须在 64ms 内各刷新一次，平均每 15.625 微秒需安排一行刷新；若每次刷新占 100ns，则理想均匀安排下占用比例约 0.64%。这是自拟预算示例；具体实现可能一条命令刷新多行或由芯片内部完成，要服从题设。

3.15.3 相联查找的并行性与延迟条件

相联映射描述能放在哪里以及怎样匹配标记，不直接决定所有比较的实际电路延迟。全相联可以并行比较多行标记，组相联可以只比较目标组中的若干行；比较器数、扇入、布线和编码都会影响时间。比较“最快”“最慢”时须固定具体实现，不能仅用相联度高低给出普遍时延定理。

写策略同样分成两次决策：写命中采用直写或回写，写缺失采用分配或不分配。回写的英文为 write-back，写分配为 write-allocate；二者分别处理命中更新和缺失装入，名字不能互换。脏块替换时才需写回，并非每次替换都必然产生一次主存写。

3.16 存储器分类、刷新与双端口访问

按存取方式，随机存取 RAM 可按任意地址访问，通常不因地址位置不同而有明显不同的寻址步骤；顺序存取 SAM 必须沿记录次序定位，如磁带；直接存取 DAM 先选大区域再在局部定位，如磁盘；相联存取按内容或标记比较选择信息。随机存取描述访问方式，与“断电是否保持”是两个分类维度。

SRAM 利用双稳态电路保存位，读取通常非破坏性，无需周期刷新；DRAM 利用电容电荷保存位，读出后要再生，并需在刷新周期内刷新所有行。读后再生恢复刚读出的电荷，定时刷新防止未读单元因漏电丢失信息，二者目的不同。DRAM 按行刷新时，刷新地址计数与行数有关，而不取决于存储器容量中的每一个字。

集中刷新在固定时段连续刷新全部行，形成较长死时间；分散刷新把正常存取和刷新交替安排，增加每次存取周期；异步刷新将刷新动作均匀分散在保持时间内。设有 128 行、每行刷新一次需 0.5 μs、要求 2 ms 内全部刷新，则相邻行刷新启动间隔可按 $2\text{ms}/128 = 15.625\,\mu\text{s}$ 安排，总刷新时间为 64 μs，占比 3.2%。

双端口 RAM 具有两套独立地址、数据和控制线路，两端可并行访问不同单元。同一单元上的读写或写写冲突，需要通过忙信号、仲裁或芯片规定的访问规则处理；“双端口”并不自动保证任何两次访问都无冲突。

3.16.1 ROM 各类别的可改写能力

课程连接·页与擦除块

闪存按页读取或编程，按块擦除；逻辑覆盖常转为异地写入并更新映射，磨损均衡不能消除擦除次数上限。

器件	内容写入或修改方式	需要注意的条件
掩膜 ROM	制造时写入	使用阶段不能修改
PROM	用户一次编程	写入后通常不能重新编程
EPROM	紫外线擦除后重写	通常整片擦除，修改不便
EEPROM	电擦除、电改写	改写速度与次数受器件限制
Flash	按块擦除、按规定粒度写入	写前擦除，读写和擦除粒度不同

3.17 SSD 的页、块与磨损均衡

SSD 由闪存翻译层和多个闪存芯片组成，芯片内再分块，块内再分页。系统给出逻辑块号，闪存翻译层 FTL 把它映射为物理页地址。NAND Flash 通常按页读取或编程、按块擦除；页与块大小取决于具体器件，课件例示页约 512 B～4 KiB、块约 16～512 KiB，这些是示例范围，不能作为所有 SSD 的固定参数。

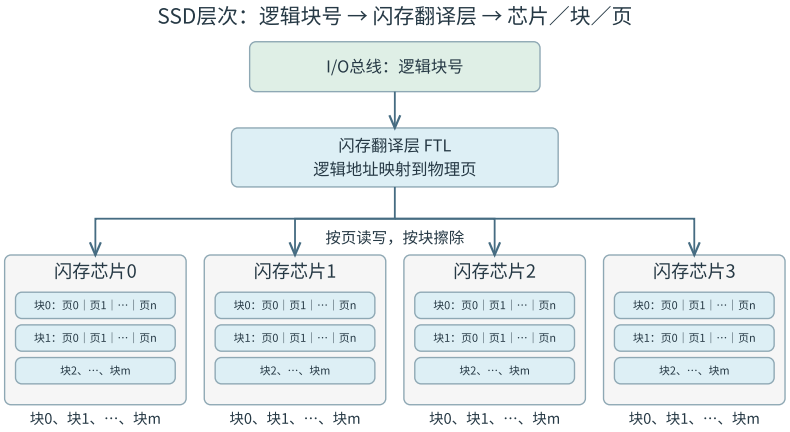


Figure 3.4: 系统逻辑块号经 FTL 映射到闪存芯片中的块与页

旧页中的数据需要更新时，控制器可把有效数据和新内容写入另一个已擦除块，更新映射，再把旧页标为失效。垃圾回收把仍有效的页集中搬走后擦除旧块，恢复空闲空间；额外搬移会产生写放大。读写快、无机械寻道、低噪声和抗震是相对机械磁盘的典型特点；擦写寿命、容量成本和写入管理是另一组比较维度。

磨损均衡通过分散擦写次数延长整体寿命。**动态磨损均衡**在发生写入时，优先使用擦除次数较少的空闲块；长期不改写的冷数据可能一直占着某些块。**静态磨损均衡**还会主动搬移冷数据，让擦写较少的块参加新写入，使各块使用更均匀。主动搬移增加了管理开销，不能把静态均衡理解为“完全不移动数据”。

3.18 位扩展、字扩展与片选译码

课程连接·位扩展与字扩展

位扩展让多片提供同一地址的不同数据位；字扩展选择不同芯片提供更多地址，两者可组合。

芯片标称 $N \times w$ 表示 N 个单元、每单元 w 位。用 $n \times b$ 芯片组成 $N \times W$ 存储器，若两维均为整数倍，片数为 $(N/n)(W/b)$。

位扩展让若干芯片同时接收相同地址，各自负责不同数据位，共同构成更宽的存储字；片选相同，数据线分组连接。**字扩展**让各组芯片分别负责不同地址区间，低位地址接到组内所有芯片，高位地址经译码产生互斥片选。字位同时扩展时，先确定同组并行芯片数，再确定地址组数。

3.18.1 例：用 16K×8 位芯片组成 32K×16 位

目标字数扩大 2 倍，字长扩大 2 倍，共需 4 片。每组 2 片分别接 $D_7 \sim D_0$ 和 $D_{15} \sim D_8$；$A_{13} \sim A_0$ 并联到四片，A_{14} 选择两组。按字编址时，组 0 范围 0000H～3FFFH，组 1 范围 4000H～7FFFH。

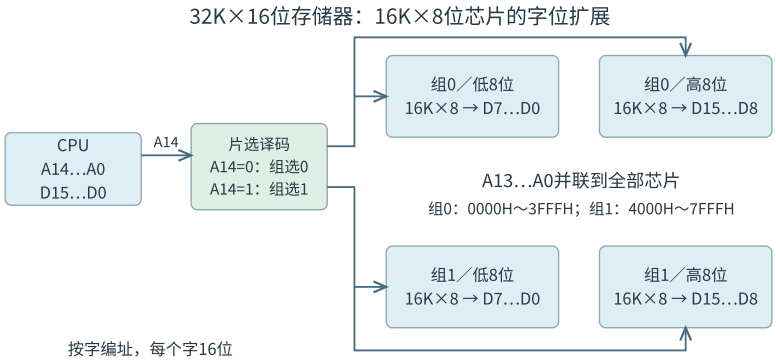


Figure 3.5: 同组两片作位扩展，A14 选择两组作字扩展

若按字节编址、每字 16 位，还需要说明最低地址位怎样区分字节以及字节使能信号如何连接，不能直接把上述按字地址范围照搬。片选有效电平由芯片和译码器图中的小圆圈、横线约定决定；逻辑表达式必须同时考虑访存使能和地址译码，避免 I/O 访问错误选中内存。

全译码使用全部需要判别的高位地址，地址对应关系唯一；部分译码省略某些高位，可能让多个地址选择同一物理单元，形成地址重叠；线选法直接用地址线选片，电路简单，但可用地址组合与地址连续性受限制。

3.19 交叉编址的连续访问时间

高位交叉用地址高位选择模块，同一连续区间常落在同一模块，主要扩容。低位交叉用低位选择模块，连续地址轮流分布到各模块，允许各模块启动时间交错。

设每个模块访问周期为 T，总线传送一个字需 τ，模块数 m 满足 $m\tau \geq T$，按连续地址访问且无其他冲突时，可每隔 τ 启动一个新字。读取连续 n 字的理想时间为 $T + (n-1)\tau$。若 $m\tau < T$，最先使用的模块尚未完成恢复便又被访问，须加入等待。

例如 $T = 200$ ns、$\tau = 50$ ns，至少 4 个模块才可连续交错启动；读取 8 字需 $200 + 7 \times 50 = 550$ ns。步长访问还要看模块号序列：模块数 4 且步长 4 时，总是访问同一模块，即使地址不同也不能获得连续逐字带宽。

3.20 Cache 替换：把命中后的状态也写出来

课程连接·命中也会改变 LRU

LRU 命中后需把命中块更新为最近使用；FIFO 命中一般不改变装入顺序。每次访问后写出状态，避免只数缺失。

直接映射的候选位置唯一，冲突时无需在多行中选择牺牲者；组相联只在所选组内替换，全相联在全部行内替换。有效位为 0 的空行优先使用。替换策略独立于写策略；被替换行若采用写回且脏位为 1，还要先回写。

FIFO 按装入时间淘汰最早装入行，命中不改变装入顺序；LRU 按最近使用情况淘汰最久未使用行，命中也要更新次序；LFU 按累计访问次数选择，需规定计数初值及并列时的规则；随机替换由硬件产生选择，成本较低。

4 行全相联 Cache 初始为空，访问主存块 $1,2,3,4,1,2,5,1,2,3,4,5$：

算法	命中位置（从 1 计数）	命中数／缺失数
FIFO	第 5、6 次	2 ／ 10
LRU	第 5、6、8、9 次	4 ／ 8

本序列中 LRU 命中 4 次、FIFO 命中 2 次，牺牲块和中间内容也不同。例如访问 5 前，FIFO 仍先淘汰 1，LRU 先淘汰 3；不能只核对最终命中率，就认为过程表正确。实现 LRU 可用访问次序栈或相应状态编码；题目采用计数器法时还须遵守其更新规则。

3.21 平均访问时间的两个模型

设 Cache 访问时间为 t_c、主存时间为 t_m、命中率为 h。若 Cache 和主存同时开始访问、命中时用 Cache 结果，则平均时间按 $ht_c+(1-h)t_m$ 计算。若先访问 Cache，确认未命中后再启动主存，则为 $t_c+(1-h)t_m$。还存在题目直接给出 “缺失损失” 的形式 $t_c+(1-h)p$，其中 p 须按定义判定是否含取块、回写和重新访问。

课件模型取 $t_m=5t_c$、$h=0.95$：同时访问时平均 $1.2t_c$，相对仅访问主存加速 $5/1.2\approx4.17$ 倍；先 Cache 后主存时平均 $1.25t_c$，加速 4 倍。这里的同时访问是计算模型，不是所有处理器的必然实现。

写直达在写命中时同时更新下层，写缓冲可以暂存待写内容；写回只更新 Cache 并置脏位，替换脏行时才写下层。写分配与非写分配处理的是写缺失：前者调入块后写，后者绕过当前 Cache 写入下层。常见组合属于工程选择，分析题目时按明确给出的策略推演。

3.22 磁盘记录与 RAID 的边界

磁盘由盘面、磁道、扇区组成，柱面是不同盘面上半径相同的磁道集合。一次访问通常包含寻道、旋转等待和数据传输；平均旋转等待在随机到达且均匀假设下为半圈。容量按有效盘面数 $\times$ 每面磁道数 $\times$ 每道扇区数 $\times$ 每扇区字节数计算，格式开销和保留区域需按题设处理。

RAID0 条带化提升并行带宽而无冗余；RAID1 镜像提高冗余但有效容量约减半；RAID3、RAID4 采用专用校验组织，RAID5 分布式校验避免固定校验盘成为所有写入的同一热点。冗余的容错范围与级别、故障数量有关，不能据 “有校验” 就断言任意多盘损坏都能恢复。写入更新还可能产生数据与校验的一致性开销。

3.22.1 计算与辨析自查

1. 四模块低位交叉中连续字和步长 4 访问的模块分布有何不同？2. 32K×16 位目标由 16K×8 位芯片组成需要几片？3. 为什么 FIFO 命中后不改队列而 LRU 要改？4. SSD 按页写、按块擦除会引出哪些管理动作？

答案。1. 连续字轮流访问 0、1、2、3 号模块，步长 4 始终访问同一模块。2. 4 片，两片一组、两组字扩展。3. FIFO 记录装入时间，LRU 记录使用时间。4. 异地更新、地址映射、失效页管理、垃圾回收和磨损均衡。

课程连接·地址划分顺序

依题意先确定块内偏移，再根据组数确定组号；比较标记前还要验证有效位。

3.23 主动回忆自测

3.23.1 配套自测

1. 原创补充：32 位字节地址、Cache 数据区 4 KB、块 16 B、2 路组相联，求偏移、组号、标记位数。假定另需有效位和修改位，先不计替换位，每行多少位？
2. 原创补充：两路某组按 LRU 访问 A、B、A、C、B，初始空，列每次命中情况与最近到最久次序。
3. 【配套例题】下列事件中，不是在 MMU 地址转换过程中检测的是（ ）。A. 访问越权　B. Cache 缺失　C. 页面缺失　D. TLB 缺失。

答案。1. 4、7、21 位；每行 $128+21+1+1=151$ 位。2. 缺失 [A]、缺失 [B,A]、命中 [A,B]、缺失 [C,A]、缺失 [B,C]。3. B。MMU 负责地址转换和相关保护；Cache 缺失属于 Cache 查找结果，不能与 TLB 缺失等同。

3.23.2 遮住答案复述知识关系

主动回忆·自拟

4. 自拟：块大小 64B、物理地址 32 位、Cache 共 16 行，采用 2 路组相联。组号多少位？与直接映射相比，标记位数怎样变化？
5. 自拟：一次查 Cache 为 2ns，命中率 95%，缺失后额外访问下一层 80ns，AMAT 是多少？如果“缺失时间 80ns”已经含查 Cache，公式又怎样写？
6. 自拟：4 体低位交叉、按字编址，字地址 0、4 连续访问为何可能冲突？是否任何两次同模块访问必冲突？
7. 自拟：TLB 未命中、页表存在位为 1、Cache 有效位为 0 时，分别发生哪两种缺失？是否缺页？
8. 自拟：回写 Cache 的一行被淘汰，是否必定写主存？还要看哪一位？
9. 自拟：DRAM 无需掉电也可能丢信息，Flash 掉电仍保留信息但为什么不能按 SRAM 方式任意覆盖写？

3.23.3 参考答案

自拟题核对答案

4. 8 组、组号 3 位，偏移 6 位，标记 23 位；直接映射 16 行，行号 4 位、标记 22 位，因此 2 路相联标记多 1 位。
5. 额外 80ns 时 $2 + 0.05 \times 80 = 6$ns；已含查 Cache 则 $0.95 \times 2 + 0.05 \times 80 = 5.9$ns。
6. 两者模 4 都为 0，若第二次请求到达时模块尚忙则冲突；若时间间隔足够则不冲突。
7. TLB 缺失和 Cache 缺失；页面已驻留，不发生缺页，仍需正常读取物理主存。
8. 不一定；只有脏位表明内容已修改且需要回写时才写回，干净副本可直接丢弃。
9. DRAM 电荷泄漏，需要周期刷新；Flash 编程和擦除粒度、允许的位变化及寿命条件不同，通常需先擦后写并由控制器管理。

课程连接·地址划分顺序

依题意先确定块内偏移，再根据组数确定组号；比较标记前还要验证有效位。

指令系统与 C 程序的机器级表示 4

章节脉络

本章围绕 “指令格式与编码 → 数据寻址 → 指令寻址与访问 → ISA 设计取舍 → 程序工具链 → 控制流与过程调用” 展开。先判断对象与条件，再沿机制推导结果，最后用反例和数值检验结论。

不变量·操作码与操作数字段分开数

固定总长下，地址字段占用越多，可留给操作码的位越少。扩展操作码逐层使用未占用前缀，不是直接从全部位串数里减去指令条数。地址字段组合也占编码空间，容量必须按同一层级计算。

4.1 指令字段怎样决定可表示的操作和地址

定义 4.1.1 (概念与适用对象)

一条机器指令包含操作码以及完成该操作所需的其他字段。操作码规定做什么，地址字段可能给出立即数、寄存器编号、形式地址、位移或寻址方式，并非所有字段都是主存绝对地址。指令字长是指令编码的位数，机器字长是整数运算的基本位宽，两者不必相等。

按显式操作数地址字段数，可以有零地址、一地址、二地址、三地址指令。零地址指令也可以操作数据：栈式机器的加法隐含从栈顶取两个数；一地址指令可隐含使用累加器；二地址形式常让一个地址同时承担源和目的，但具体语义仍要由题目给出。“两个字段” 并不对应固定的访存次数：字段还可能指定寄存器或立即数。

数据传送指令移动或复制值；算术逻辑指令形成新值和可能的标志；转移指令改变后继执行地址；调用在转移前保留返回位置，返回根据保存的位置继续；输入输出及系统控制指令还可能有权限限制。通常的赋值和算术指令不会把源操作数 “取走后清空”。

固定长度操作码用 k 位可区分至多 2^k 种编码。扩展操作码把部分短编码保留为扩展前缀，用更长的操作码表示操作数较少的指令。已作为短指令使用的编码不能同时作长指令前缀，否则译码有歧义。

4.1.1 例 4-1 扩展操作码应逐层扣除哪些前缀

例 4.1.1 (配套例题)

【配套例题】设计某指令系统时，假设采用 16 位定长指令字格式，操作码使用扩展编码方式，地址码为 6 位，包含零地址、一地址和二地址 3 种格式的指令。若二地址指令有 12 条，一地址指令有 254 条，则零地址指令的条数最多为（ ）。

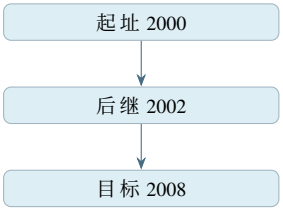
A. 0 B. 2 C. 64 D. 128

解析。二地址格式需要 12 位地址，短操作码为 4 位，共 16 种前缀，用掉 12 种，剩 4 种可扩展。每个前缀再接 6 位，共提供 $4 \times 64 = 256$ 种一地址操作码；用掉 254 种，剩 2 种再各扩展 6 位，得到 $2 \times 64 = 128$ 条零地址指令，选 **D**。

不能从 2^{16} 中直接减去 “12+254”，因为每条二地址指令还对应 2^{12} 种操作数字段组合，每条一地址指令对应 2^6 种组合。逐层保留前缀的算法直接反映译码过程，也方便处理其他字段长度。

最小反例·相对基准不一定是起点

最小反例：2B 指令从 2000H 取完，PC 已变 2002H，再加 6 应得 2008H。直接用指令起点加 6 得到 2006H，漏算了取指更新。负位移还须先按给定位宽作符号解释。



2B 指令取完后加 6，目标为 2008H。

4.2 从形式地址到有效地址

以下记指令地址字段为 A，寄存器 R 的内容为 (R)，主存地址 x 的内容为 M[x]；有效地址 EA 指最终操作数所在的主存地址。立即数不需要另求操作数的主存地址；寄存器寻址的操作数也在寄存器中，不必为凑访存次数而虚构一次主存访问。

寻址方式	操作数或有效地址	核对条件
立即	操作数在指令中	字段的符号扩展规则
直接	EA=A，操作数 =M[A]	A 的编码范围
一次间接	EA=M[A]	先取地址，再取操作数
寄存器直接	操作数 =(R)	字段是寄存器编号
寄存器间接	EA=(R)	寄存器保存的是地址
基址	EA=(BR)+ 位移	位移位宽及符号
变址	EA=A+(IX)	元素下标是否已乘元素宽度

寻址方式	操作数或有效地址	核对条件
相对	目标 =PC 基准 + 位移	PC 指当前还是后继地址

间接寻址得到的中间地址位宽可能大于指令地址字段，从而扩大可寻址范围，代价是多一次取地址的访存。基址可用于程序重定位，变址适合数组遍历；二者都可写成寄存器加位移的形式，但寄存器的用途不同，具体机器也可能用同一种编码机制实现。

4.2.1 例 4-2 相对转移用哪个 PC

例 4.2.1 (配套例题)

【配套例题】某机器字长为 16 位，主存按字节编址，转移指令采用相对寻址，由两个字节组成，第一个字节为操作码字段，第二个字节为相对位移量字段。假定取指令时，每取一个字节 PC 自动加 1。若某转移指令所在主存地址为 2000H，相对位移量字段的内容为 06H，则该转移指令成功转移后的目标地址是（ ）。

A. 2006H　B. 2007H　C. 2008H　D. 2009H

解析。指令从 2000H 开始，占 2 B，取完后 PC=2002H。相对位移为 +6，目标为 2008H，选 **C**。若位移字段改为 FAH 且按 8 位补码解释，则位移为 −6，目标为 1FFCH；不能把 FAH 当作正 250。PC 基准必须以本题规定的更新时机为准。

数组寻址还需区分元素下标和字节偏移。按字节编址、int 为 4 B 时，`a[i]` 地址为 base+4i。某 ISA 若有 “比例变址”，可由硬件把索引乘 4；若没有，则由移位或其他指令形成 4i。这不意味着数组元素之间多留了 3 个空洞，四个字节共同存放一个元素。

4.2.2 访存次数要按执行路径来数

若指令为一个字、无 Cache、无预取，直接寻址的读操作数指令通常至少取指一次、取操作数一次；一次主存间接寻址再多读一次地址。若还写回主存，需另计一次写。若指令跨两个存储字、发生缺页或含两个主存操作数，次数会改变。“寄存器间接” 中的 “间接” 不会额外读一次主存地址，因为地址已经在寄存器中。

不变量·只数真实读写事件

无 Cache 无预取时，取指、取有效地址、取操作数和写回主存各是独立事件。某字段在寄存器中就不应虚构一次主存读取；指令跨字或有两个主存源时也不能沿用一字一源的计数。

4.3 RISC 与 CISC：设计倾向不代替具体实现

RISC 通常采用较规整的指令格式、较少的寻址方式和 Load/Store 风格：算术逻辑操作主要在寄存器间进行，访存由装入和存储指令承担。这有利于规整译码和流水处理。CISC 往往支持更丰富的指令形式与寻址方式，单条指令可以表达较复杂操作。

下表按共同维度比较教材中的典型设计。表中的 “较多”“较少” 只是同一历史设计背景下的倾向，并不按指令条数、某条指令或单项性能为所有处理器划定边界。

比较维度	RISC 的典型设计	CISC 的典型设计	做题时的适用边界
指令功能与使用	选用常见的简单操作，复杂功能由指令组合实现	指令功能较丰富，一条指令可表达较复杂操作	不用 “小于 100 条／大于 200 条” 等历史概数代替具体 ISA 定义
编码与寻址	格式较规整，常用定长编码，寻址方式较少	格式和寻址方式较多，常见变长编码	定长、变长须按题给编码确认；不能只凭 RISC 名称确定每条指令字节数
数据操作数访存	典型 Load/Store 模型中，装入和存储访问数据存储器，算术逻辑在寄存器间进行	多类指令可含主存操作数	这里说的是数据访问，取指也要读存储器；具体指令能否访存仍查其语义
寄存器与调用	通用寄存器较多，多用寄存器传参，减少数据访存	传统设计的通用寄存器相对较少，操作数来源较丰富	寄存器数由 ISA 给定；传参位置还受 ABI 约束，不由 RISC/CISC 名称单独决定
执行时间与 CPI	简单操作便于规整流水；教材常用多数简单指令单周期的理想模型	不同指令的执行时间差异往往较大	单条指令延迟、流水吞吐和平均 CPI 分别计算；访存等待、乘除及发射宽度都会改变结果
控制方式	多采用硬布线控制	复杂指令常借助微程序控制	两者是控制器实现方式，可结合使用，不能断言某类 ISA 只能选一种
流水线	规整编码和操作有利于流水处理	也可通过译码、分解内部操作等方式实现流水	采用流水线不是 RISC 独占的能力；阶段数、旁路和阻塞规则均由具体微架构决定
编译优化	复杂功能较多依靠编译器选指令、分配寄存器和调度	丰富指令给出更多选择，选指令与优化可能更复杂	两类都需要编译优化；目标代码条数较少不直接等于执行时间较短

做题时要按题目中 “多采用”“通常” 等措辞判断典型特征；删去这些限定，就会把设计倾向误读成必然条件。

课程连接·ISA 不等于微架构

是否采用流水线、旁路和哪种控制器主要涉及微架构实现，ISA 规定软件可见指令语义。两类层次相互影响但不能互代。指令简单有利于规整实现，不意味着只可能采用一种硬件结构。

4.3.1 例 4-3　RISC 设计能否支持流水线

例 4.3.1 (配套例题)

【配套例题】下列关于 RISC 的叙述中，错误的是（　）。

A. 多采用硬连线方式实现控制器
B. 通常采用 Load/Store 型指令设计风格
C. 难以采用流水线数据通路实现微架构
D. 多采用寄存器传递过程调用时的参数

解析。格式规整、操作较简单，通常有利于流水线实现，所以 C 错。A、B、D 描述的是常见设计倾向，不应把 “多采用” 读成 “只能采用”。答案为 **C**。

4.4 编译、汇编、链接怎样确定程序中的地址

第 1 章已给出工具链顺序，这里进一步说明为什么目标文件未必能直接运行。编译和汇编时，一个模块可能引用另一个模块的函数或全局变

量，汇编器尚不知道它的最终地址，只能留下符号与重定位信息。链接器合并相应节，解析符号，再按选定地址修改相关引用。

可执行文件包含机器代码，也包含装入所需的组织信息；CPU 并不把文件的每一个字节都当作指令。目标文件还可能含符号表、调试信息和重定位记录。高级语言的变量名在运行时未必仍对应一个固定的主存地址；优化后变量本身可能在寄存器中、被折叠成常量，或已经消失。

错项排除·目标代码不是最终运行状态

汇编得到可重定位目标文件，还可能缺跨模块符号地址。链接并安排引用后形成可执行文件，运行还需装入和环境。把汇编与链接交换，既无法解析源语言，也无法解释地址何时确定。

4.4.1 例 4-4　工具链顺序

例 4.4.1 (配套例题)

【配套例题】将高级语言源程序转换为可执行目标文件的主要过程是（　）。

A. 预处理 → 编译 → 汇编 → 链接
B. 预处理 → 汇编 → 编译 → 链接
C. 预处理 → 编译 → 链接 → 汇编
D. 预处理 → 汇编 → 链接 → 编译

解析。先处理源文件中的预处理指令，再将高级语言翻译成汇编表示，继而编码为目标文件，最后解析跨模块引用，选 **A**。这是题目采用的典型静态编译流程；动态链接把部分工作留到装入或运行时，但汇编编码仍必须在程序开始执行前完成。

4.5 分支和循环怎样改变 PC

机器执行选择结构时，先形成条件，再决定是否改变后继地址。条件可由比较指令设置标志，也可由分支指令直接比较寄存器。翻译 C 条件时必须保留 signed/unsigned 的区别：两者都用减法硬件，但转移条件并不相同。

下列教学指令约定整数均为 32 位，R1 存放 x，R2 存放 y，`bge` 按有符号数比较，标签代表指令地址；并假定本例的差值可用 32 位有符号数表示：

```
    bge R1, R2, ELSE     // x>=y 时跳过 then
    sub R3, R2, R1      // then: z=y-x
    jump END
ELSE:
    sub R3, R1, R2      // else: z=x-y
END:
```

这里要实现 `if(x<y) z=y-x; else z=x-y;`，分支条件取原条件的否定，于是满足原条件时顺序落入 then。转移目标是代码位置，不是把标签的字符存进 PC。

分析循环时，先确定入口测试、循环体、更新和回边。对以下机器级教学轨迹，R0 为数组基址，R1 为元素下标，R2 为和，R3 为元素个数，int 占 4 B：

```
    R1 <- 0; R2 <- 0
TEST:
    if R1 >= R3 jump EXIT
    R4 <- R1 << 2
    R5 <- M[R0 + R4]
    R2 <- R2 + R5
    R1 <- R1 + 1
    jump TEST
EXIT:
```

不变量·返回地址属于调用点

调用保存的是调用之后继续执行的位置；函数入口只是被调目标。递归各层可以在同一代码位置调用，却有独立参数和栈帧。恢复现场时必须与当前返回层次配对，不能只追踪函数名。

R3=0 时直接退出，不读 `a[0]`；R3=3 时读取偏移 0、4、8 而非 0、1、2。这两个边界足以检查循环上下界和地址单位。若变换为指针遍历，每轮指针加 4，终止地址为 base+4R3；终止地址并不是最后一个有效元素的地址。

逆向读代码时，先标出各寄存器的初值和更新式，再识别分支条件。循环回边指令的执行次数与循环体次数可能不同：入口先测的 n 次循环，测试通常执行 n+1 次。算动态指令数和 Cache 访问次数时，这个差别必须计入。

4.6 过程调用为什么要保存返回位置

调用至少要解决四件事：把参数交给被调过程，记录返回位置，给局部变量和临时值安排空间，将返回值交回调用者。ISA 提供调用、返回或寄存器转移等基础机制，ABI 进一步规定参数寄存器、返回值位置、栈对齐，以及哪些寄存器由哪一方保存。

返回地址可以放到栈，也可以先放到链接寄存器。若采用链接寄存器，叶子函数可能不访存保存返回地址；会再次调用其他函数的过程，通常要在覆盖链接寄存器之前保存其中的返回地址。“返回地址一定写主存” 并不是所有指令系统的共同特征。

以下采用一个明确的栈模型说明调用：栈向低地址增长，每槽 4 B，SP 指向当前栈顶；push 先 SP 减 4 再写，pop 先读再 SP 加 4；调用压入后继指令地址，返回弹出该地址送 PC。调用前 SP=1000H，返回地址为 2004H，则调用后 SP=0FFCH、M[0FFCH]=2004H。

若被调过程又保存旧帧指针 FP、再分配 8 B 局部区，SP 依次变为 0FF8H 和 0FF0H。此时局部变量不能覆盖 0FFCH 的返回地址。退出时先撤销局部区，恢复旧 FP，再取返回地址；SP 最终回到 1000H。图中的地址单位为字节。

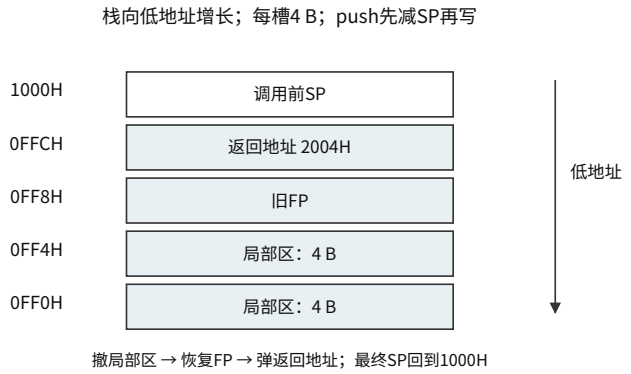


Figure 4.1: 调用、建立栈帧和返回时，SP 逐步移动

嵌套调用靠每次独立保存的返回位置区分 “回到哪里”；递归的各层也各有参数、局部变量和返回地址。局部变量同名不表示它们共用同一槽。调用者保存寄存器需在调用前保留自己还要使用的值；被调用者保存寄存器则在被调过程修改前保存、返回前恢复。

题目给出具体调用约定时，要按该约定追踪，不能凭熟悉的 x86、MIPS 或 RISC-V 经验补条件。特别要检查栈生长方向、SP 指向、入栈前后调整顺序和返回地址基准。这些细节都会影响地址计算与最终返回位置。

最小反例·转移不一定保存返回地址

最小反例：普通条件分支跳到标签，并不会自动为未来返回保留调用点。调用与分支都能改 PC，但保存返回位置的语义不同；把任意跳转都当成函数调用，会错误增加栈写和返回动作。

4.6.1 顺序、分支、调用和返回分别改变什么

执行方式	后继地址怎样得到	额外需要保留的信息	读代码时先核对什么
顺序执行	当前地址加指令字节数	不因普通顺序执行单独保存返回地址	变长指令必须实际数编码字节
条件分支	条件成立取目标，否则取顺序后继	通常无需调用返回地址	signed/unsigned 条件、位移符号、基准与缩放
调用	无条件转到被调入口	调用后的后继地址	返回位置存栈还是链接寄存器
返回	取先前保存的返回位置	还须按约定恢复栈和现场	返回地址来自哪层调用，不能重算成函数入口

相对寻址只规定 “基准地址加位移”，并不规定所有指令的位移都以字节为单位。例如下面 x86 的 call 用后继指令地址加有符号字节位移；某些定长 RISC 分支会把编码位移先左移两位。读机器码时应分别求出当前地址、指令长度、基准地址、扩展后的位移及缩放比例，最后核对目标。把汇编目标标签直接当成机器码里保存的绝对地址，会漏掉这条计算链。

4.6.2 例 4-5　递归调用的返回位置、机器编码与乘法溢出

例 4.6.1 (配套例题)

【配套例题】已知 $f(n)=n!=n\times(n-1)\times(n-2)\times\cdots\times2\times1$，计算 $f(n)$ 的 C 语言函数 f1 的源程序及其在 32 位计算机 M 上的部分机器级代码如下。

```
int f1(int n) {
    if (n > 1)
        return n * f1(n - 1);
    else
        return 1;
}
```

以下按 “行号／虚拟地址／机器指令（字节按地址递增排列）／汇编指令” 四列给出，每行是一条代码记录；省略行仍保留四列。

```
 1  00401000  55              push ebp
 …  …         …               …
11  00401018  83 7D 08 01     cmp dword ptr [ebp+8], 1
12  0040101C  7E 17           jle f1+35h (00401035)
13  0040101E  8B 45 08        mov eax, dword ptr [ebp+8]
14  00401021  83 E8 01        sub eax, 1
15  00401024  50              push eax
16  00401025  E8 D6 FF FF FF  call f1 (00401000)
 …  …         …               …
19  00401030  0F AF C1        imul eax, ecx
20  00401033  EB 05           jmp f1+3Ah (0040103A)
21  00401035  B8 01 00 00 00  mov eax, 1
 …  …         …               …
26  00401040  3B EC           cmp ebp, esp
 …  …         …               …
30  0040104A  C3              ret
```

其中，机器级代码行包括行号、虚拟地址、机器指令和汇编指令，计算机 M 按字节编址，int 型数据占 32 位。请回答下列问题。

（1）计算 f(10) 需要调用函数 f1 多少次？执行哪条指令会递归调用 f1？

（2）上述代码中，哪条指令是条件转移指令？哪几条指令一定会使程序跳转执行？

（3）根据第 16 行的 call 指令，第 17 行指令的虚拟地址应是多少？已知第 16 行的 call 指令采用相对寻址方式，该指令中的偏移量应是多少（给出计算过程）？已知第 16 行的 call 指令的后 4 字节为偏移量，M 采用大端方式还是小端方式？

（4）$f(13) = 6\,227\,020\,800$，但 f1(13) 的返回值为 1 932 053 504，为什么两者不相等？要使 f1(13) 能返回正确的结果，应如何修改 f1 的源程序？

（5）第 19 行的 imul 指令（带符号整数乘）的功能是 R[eax] ← R[eax] × R[ecx]，当乘法器输出的高、低 32 位乘积之间满足什么条件时，溢出标志 OF=1？要使 CPU 在发生溢出时转异常处理，编译器应在 imul 指令后加一条什么指令？

不变量·call 位移用后继地址校验

五字节 call 从 00401025H 开始，后继为 0040102AH；跳到 00401000H 所需位移为 −42，编码 FFFF FFD6H。把目标减起点会少算 5B。将求出的基准加位移回代，必须恰好恢复目标地址。

解析。先追踪源程序的调用层次，再读指令对 PC 和数据的作用。表中的省略号表示原题未展示的指令，行号差并不等于字节数。

（1）调用参数依次为 10、9、…、1，共 10 次，包含最外层调用；函数内部第 16 行 call 实际递归执行 9 次。参数为 1 时，第 12 行转到返回 1 的分支，不再递归。各层的返回位置虽都是调用点的后继地址，但栈帧和参数相互独立，返回时逐层乘回 2、3、…、10。

（2）第 12 行 jle 是条件转移，根据比较结果决定是否转移。第 16 行 call、第 20 行 jmp、第 30 行 ret 在执行时都会改变顺序后继：分别是调用、无条件跳转和返回。第 15 行 push 保存参数，本身并不转到 f1 入口。

（3）call 从 0040 1025H 开始，占 1 字节操作码及 4 字节位移，共 5 B，所以后继地址为

$$\texttt{00401025H} + 5 = \texttt{0040102AH}.$$

按后继地址为相对基准，位移为 $\texttt{00401000H} - \texttt{0040102AH} = -\texttt{2AH} = -42$，其 32 位补码为 FFFF FFD6H。表中后 4 字节按低地址到高地址为 D6、FF、FF、FF，低有效字节先存，因此为小端。复核：

`0040102AH` + (−42) = `00401000H`。不能用 call 自身起点减目标，也不能把 D6 当成正 214。

（4）13! 超过 32 位 int 上界 $2^{31}-1$。题给返回值是保留低 32 位后的机器结果：$6\,227\,020\,800 - 2^{32} = 1\,932\,053\,504$，它虽为正数，仍是溢出后的错误数学结果。标准 C 并没有规定有符号溢出必须这样回绕；这里解释的是原题给定的机器执行结果。

一种修改是把函数返回类型改为至少 64 位的 `long long`，使递归返回值及与 n 的乘法都按该较宽类型进行：

```
long long f1(int n) {
    if (n > 1)
        return n * f1(n - 1);
    return 1LL;
}
```

13! 可以用该类型精确表示。若只是先在 int 中乘完，再把溢出结果强制转换为宽类型，已经丢失的高位无法恢复。这里仅保证题目所问的 f1(13)，不意味着任意大的阶乘都能用固定 64 位整数表示。

（5）令精确 64 位乘积为高 32 位 H 与低 32 位 L。若 H 恰是 L 最高位的 32 位符号扩展，即 L 的符号位为 0 时 H 全 0、为 1 时 H 全 1，则截为 32 位仍保留真值，OF=0；否则 OF=1。等价地，乘积从第 63 位至第 31 位的高 33 位必须全 0 或全 1 才不溢出。只检查 H 是否为 0 并不够：可表示的负积需要 H 全 1。

题目要求的是发生溢出时转异常处理，因此应在 imul 后安排检查 OF 的**溢出自陷指令**：OF=1 时发出溢出异常并转入处理程序。普通条件转移 `jo` 能根据 OF 跳往一段代码，但跳转本身并不等于完成了异常入口动作。乘法器设置标志、指令检查标志、异常机制保存返回状态并转入口，是相接的三个步骤。

4.7 用语义、地址与状态三张清单读指令

机器级代码五步法

1. 列 ISA 给出的每条指令语义，区分它读寄存器、读主存、写主存还是只改 PC。
2. 标指令起址和字节长度，按题设确定顺序后继与相对基准。
3. 写有效地址，保留位移符号、索引比例和寻址级数。
4. 对分支列出真假两条路径；对调用单独记录返回地址、SP、FP 及需保存的值。
5. 用零次循环、一次循环、嵌套调用和负位移回代，最后数动态执行次数与访存。

4.7.1 扩展操作码为什么要保留前缀

译码互斥与编码空间

若某短操作码已被认作一条完整指令，同一前缀后的位就按其操作数字段解释；再把该前缀延长当作另一操作码，会让同一位串出现两种译码。因此扩展编码必须先留下未使用短前缀，再把它们的后续位作为更长操作码。每留下一个前缀并扩展 k 位，得到 2^k 个子前缀。逐层扣除已用子前缀，正是原例的容量算法。

不变量·编码数先核层级

扩展 k 位使一个保留前缀分裂成 2 的 k 次方个子前缀，已占用短格式对应的是整片子树。每一步统一数同样长度的前缀，才不会把指令条数与完整操作数字段位串混在一起。

□

课程连接·栈模型接到数据结构

栈的后进先出性质保持不变，但机器 SP 到底指向当前元素还是下个空位属于实现约定。数据结构中的 push 抽象不直接决定地址先减还是先写。题目给出约定后，再逐步画 SP 轨迹。

4.7.2 地址字段数不等于实际操作数数

隐含寻址不在指令中显式写出某个操作数的位置，例如约定累加器或栈顶。零地址算术指令仍可从栈顶取得两个操作数并把结果送回栈；只是这些位置由机器约定确定。一地址指令也可能隐含累加器，故题目问地址码个数时，不能直接回答参与运算的值有几个。

堆栈寻址以 SP 和约定确定元素位置。满栈/空栈、向低/高地址增长、先改 SP 还是先访存，都会改变读写地址。基址、变址和相对寻址虽然常都出现 “寄存器 + 位移” 的代数式，寄存器角色分别服务于重定位、数组下标与控制转移；代数相同不代表使用约定可互换。

4.7.3 指令类别与读写副作用

数据传送类把值在寄存器与存储器等位置间移动；算术逻辑类形成新值及可能的标志；移位类改变位位置；顺序控制类决定后继 PC；I/O 及系统控制类涉及端口、状态与受保护操作。具体指令是否改标志、访存或具有特权，须查给定语义，不能仅凭助记符字母猜测。

机器码解读和 C 语义需要两层核对。机器指令可以按规定生成截断结果，C 程序是否允许这种执行仍受溢出、对象范围与类型规则约束。把一次具体机器执行轨迹说清楚，并不等于证明对应 C 代码在所有实现上都有同样结果。

4.8 地址个数、指令长度与访存次数

地址字段数描述指令中显式给出的地址个数。三地址指令可分别给出两个源和一个结果地址；二地址指令常把一个源地址同时作为目的地址；一地址指令可隐含累加器；零地址指令可以不使用操作数，也可以隐含使用栈顶操作数。分析 “几个操作数” 时应包括隐含操作数，分析 “几个地址字段” 时只数编码字段。

若操作数和结果均在主存、指令一次取出，三地址运算可能需要取指 1 次、读源 2 次、写结果 1 次，共 4 次访存；寄存器操作数、间接寻址或多字指令都会改变次数。因此先列执行动作，再累计访问次数，不能把地址数直接当成访存次数。

指令字长、机器字长和存储字长分别描述指令编码、CPU 数据处理宽度和存储单元宽度。若存储字长 16 位，一条 32 位指令跨两个存储字，取指可能需要两次存储器访问；是否还能合并为一次总线事务，取决于具体存储组织。

4.8.1 扩展操作码的完整分配例

设 16 位指令，每个地址字段 4 位，要求 15 条三地址、12 条二地址、62 条一地址和 32 条零地址指令。基本 4 位操作码共有 16 种，先分配 15 种给三地址，剩余 1 种前缀扩展为 16 种 8 位操作码；分配 12 种给二地址，剩余 4 种再扩展为 64 种 12 位操作码；分配 62 种给一地址，剩余 2 种扩展为 32 种 16 位零地址操作码。

每次可继续扩展的数量为 “本层保留前缀数 $\times$ 下一段编码组合数”。已经分配的短操作码不能再作为其他操作码的前缀，否则译码器无法确定后续位是地址还是操作码。这里指令字长固定，变化的是操作码和

地址字段之间的分配。

课程连接·形式地址与有效地址

形式地址是指令字段，有效地址是计算后访问操作数的位置；立即寻址的字段本身表示操作数。

4.9 数据寻址方式的统一比较

下面把指令中的形式地址记为 A，寄存器内容记为 (R_i)，存储器单元内容记为 $M[A]$。执行阶段访存次数仅统计读取一个源操作数，取指和写回另计。

寻址方式	操作数或有效地址	执行阶段读主存次数
立即	操作数就是 A	0
直接	EA $=A$，操作数 $M[A]$	1
一次间接	EA $=M[A]$	2
寄存器	操作数为 (R_i)	0
寄存器间接	EA $=(R_i)$	1
相对	EA $=\mathrm{PC}_{\text{base}}+\mathrm{sext}(A)$	依操作数位置确定
基址	EA $=(\mathrm{BR})+A$	内存源通常 1 次

寻址方式	操作数或有效地址	执行阶段读主存次数
变址	EA $=(\mathrm{IX})+A$	内存源通常 1 次

相对寻址以题设规定的 PC 值为基准。常见机器用下一条指令地址作基准，位移为补码；若位移按字、半字对齐，还须先按约定缩放。基址寻址把一段程序或数据的起始位置交给基址寄存器，适合重定位；变址寻址把数组起点放在地址字段、元素变化量放在变址寄存器，适合循环访问。二者都表现为加法，寄存器含义和管理方式却不同。

堆栈寻址隐含使用 SP。采用向低地址增长、SP 指向栈顶元素的 32 位字栈时，压栈先使 SP 减 4 再写入，弹栈先读出再使 SP 加 4。若题目采用空栈顶指针或向高地址增长的约定，应重新写出操作次序。寄存器组组成的硬堆栈与主存组成的软堆栈在访存开销上不同。

4.10 x86 汇编：操作数顺序与宽度先行

课件采用 32 位 x86 示例，EAX、EBX、ECX、EDX 是 32 位通用寄存器；AX、AL、AH 分别对应 EAX 的低 16 位、最低 8 位和次低 8 位；ESP 和 EBP 常用于栈顶及栈帧基址。这里的具体寄存器约定属于示例 ISA，不能外推成所有机器的统一规则。

含义	Intel 格式	AT&T 格式
将立即数 5 送 EAX	`mov eax, 5`	`movl $5, %eax`
将 EBX 送 EAX	`mov eax, ebx`	`movl %ebx, %eax`
读地址 EBX+ECX×4+8 处 32 位数	`mov eax, dword ptr [ebx+ecx*4+8]`	`movl 8(%ebx,%ecx, 4), %eax`

Intel 格式目的在前、源在后；AT&T 格式源在前、目的在后，寄存器前有%，立即数前有 `$`，常用 b、w、l 后缀指定 8、16、32 位宽度。x86 的地址比例因子为 1、2、4 或 8，超出该集合的乘法不能直接当成这一寻址编码的比例因子。

`mov` 复制数据；`lea` 计算有效地址的算术表达式并把地址值送寄存器，不读取该地址中的数据；`add/sub` 修改目的数；`and/or/xor` 作逐位逻辑运算；`cmp` 计算减法标志而不保存差；`test` 计算与运算标志而

不保存按位与结果。移位时应区分 `shl/sal`、`shr` 和 `sar`，最后一个按算术右移保留符号扩展。

课程连接·调用边界

返回地址、保存的寄存器和局部变量各有职责；栈增长方向和参数位置必须采用题中调用约定。

4.10.1 条件转移如何解释相同标志位

以 Intel 格式 `cmp eax, ebx` 为例，标志来自 EAX−EBX。相等看 ZF；无符号小于看 CF；无符号小于等于看 CF 或 ZF；有符号小于看 SF $\neq$ OF；有符号小于等于看 ZF 或 SF $\neq$ OF。有符号大于要求 ZF=0 且 SF=OF，无符号大于要求 CF=0 且 ZF=0。

同样的寄存器位串可以被程序当作有符号或无符号数，硬件减法结果相同，所选分支条件不同。`loop` 在 32 位计数约定下先把 ECX 减 1，再判断是否非零，它本身不等价于“修改 ZF 后的普通 dec 指令”；在分析后续标志位时要遵守具体指令的副作用定义。

4.10.2 用机器级控制流还原循环

```
mov ecx, 0
mov eax, 0
L_test:
    cmp ecx, 10
    jge L_done
    add eax, dword ptr [ebx+ecx*4]
    add ecx, 1
    jmp L_test
L_done:
```

若 EBX 指向 10 个 32 位整数且运算结果在目标类型范围内，这段代码将数组元素累加到 EAX。读程序时依次确定初始化、循环条件、循环体和更新。条件转移写在循环开头时可能一次也不执行；写在循环末尾并由循环体起步时通常至少执行一次，二者对应的高级语言结构不同。

4.11 函数调用：参数、返回地址和栈帧

采用课件中的 32 位、向低地址增长的栈，以及常见的 EBP 栈帧形式：调用者安排参数并执行 `call`；`call` 把下一条指令地址压栈，再跳到被调用函数。被调用者执行 `push ebp; mov ebp, esp`，保存旧帧基址并建立新帧；需要局部空间时再减少 ESP。

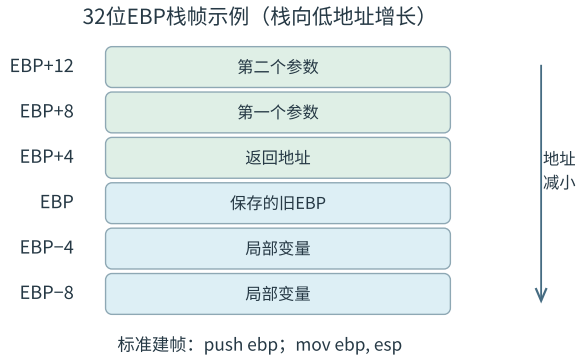


Figure 4.2: 32 位示例栈帧中，返回地址与参数位于 EBP 上方，局部变量位于下方

典型布局中，`[ebp]`为旧 EBP，`[ebp+4]`为返回地址，`[ebp+8]`、`[ebp+12]`为前两个栈参数，局部变量位于 `[ebp-4]`等位置。参数次序、哪些寄存器由谁保存、由谁回收参数空间，属于调用约定；这里按上述教学模型分析，不把某一布局当作全部编译器和优化级别的固定形式。

函数返回前常用 `mov esp, ebp; pop ebp` 恢复调用者栈帧，随后 `ret` 弹出返回地址到指令指针；常见整数返回值放在 EAX。递归调用每层拥有独立的返回地址和必要局部状态；全局变量则由各层共享。计算递归空间时应乘以最大同时存活的调用层数，而不是总调用次数。

4.11.1 计算与辨析自查

1. 16 位指令的三地址操作码为何最多分配 15 种后还能继续扩展？2. `lea eax,[ebx+4]`会读取地址 EBX+4 的内容吗？3. 有符号小于为什么不能只看 SF？4. 上述栈帧中返回地址位于哪里？

答案。1. 留出的一个 4 位前缀可再结合下一段 4 位形成 16 种长操作码。2. 不会，只计算地址值。3. 减法发生溢出时，结果符号与数学差的符号可能不同，须结合 OF。4. `[ebp+4]`，前提是已经执行标准建帧序列。

4.12 主动回忆自测

4.12.1 配套自测

1. 原创补充：沿用本章栈模型，初始 SP=1000H，连续 push 两个 4 B 值后，再 pop 一个值，SP 是多少？返回地址保存在链接寄存器时，是否每次调用都必然产生一次主存写？

答案。1. SP=0FFCH。否，是否写主存还取决于调用模型及是否需要溢出保存。

4.12.2 遮住答案复述知识关系

主动回忆·自拟

2. 自拟：说明零地址加法如何仍有两个源操作数；哪部分信息被隐含了？
3. 自拟：某 2B 转移指令从 2000H 开始，取完 PC 为 2002H，8 位

课程连接·按语义复查

地址计算、字节序和标志位分别检查，避免由指令名字直接猜数值结果。

补码位移 FAH。目标地址是多少?

4. 自拟：指令和操作数都占一个存储字，无 Cache 无预取，直接读操作数、一次主存间接读操作数、寄存器间接读操作数，各至少几次主存访问?
5. 自拟：原条件为 x<y，为何实现 if/else 时常先用 x>=y 跳到 else? 如果改成 unsigned，分支条件的解释要怎样改?
6. 自拟：返回地址存在链接寄存器时，递归调用为什么仍可能需要额外保存？调用后可以直接返回函数入口吗?

课程连接·按语义复查

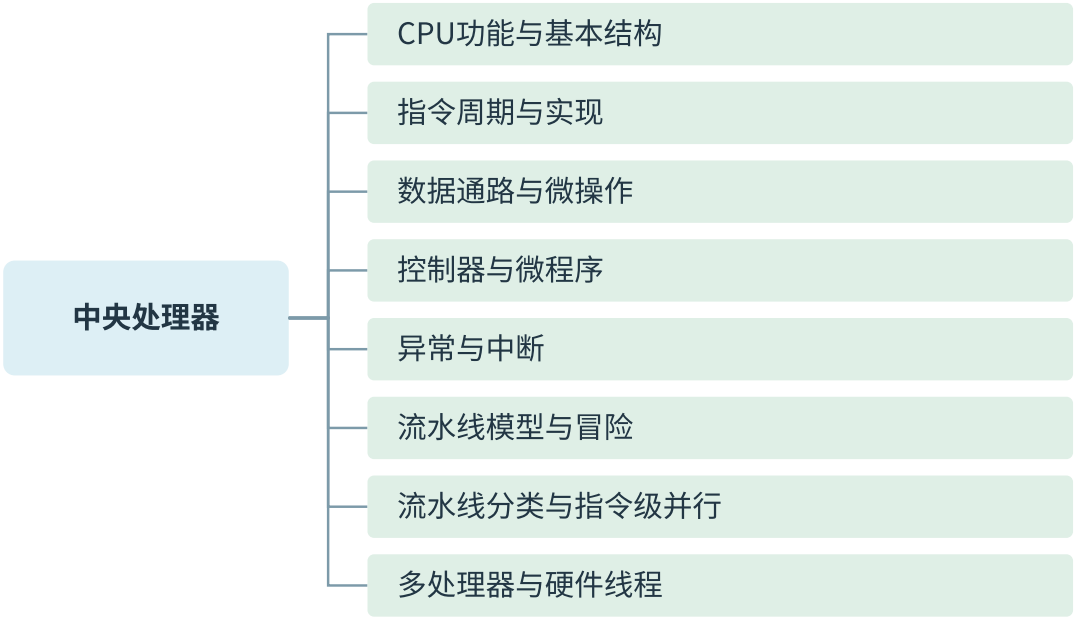
地址计算、字节序和标志位分别检查，避免由指令名字直接猜数值结果。

4.12.3 参考答案

2. 源来自约定的栈顶两项，目的为约定栈顶；指令没有显式地址字段，不表示没有操作数。
3. FAH 为 -6，目标 $2002H - 6 = 1FFCH$。
4. 分别至少 2、3、2 次：都需一次取指；主存间接额外读一次地址，寄存器间接的地址已在寄存器。
5. 取原条件的否定，使满足原条件者顺序进入 then，其他跳到 else。unsigned 须采用无符号比较条件，不能直接复用有符号标志组合。
6. 再次调用会覆盖链接寄存器，须在需要返回的旧值被覆盖前保存。返回到调用点后继地址，而非函数入口；各层栈帧与返回位置必须匹配。

中央处理器 5

章节脉络



课程连接·先看一次指令

跟踪指令从取指、译码、执行到可能的访存与写回；每一步由控制信号选择相应数据通路。

5.1 从指令语义到执行组织

课件先用 CPU 功能与结构图识别寄存器、运算器和控制器，再用指令执行导图连接取指、间址、执行和中断阶段。本章沿这条路径先解释一条指令怎样得到正确结果，随后把流水线、多发射、多处理器与硬件线程接入同一章，始终保留“值在何时产生、资源在何时被占用”的约束。

定义 5.1.1 (CPU 的功能与实现层次)

CPU 承担指令控制、操作控制、时间控制、数据加工和规定的异常/中断处理。运算器处理数据，控制器解释并组织执行，寄存器保存跨时刻状态；数据通路承载数据传送与运算，控制部件决定何时、按何种方式使这些动作生效。

ALU、累加器 ACC、通用寄存器组、移位器和计数器等可出现在运算部件的教学模型中；PC、IR、指令译码器、时序系统和微操作信号发生器等常放在控制器模型中。实际 CPU 划分依实现；例如 PSW 既包含运算标志，也包含处理器状态，不能仅凭图上位置认定其全部位都允许用户程序任意写。

课件把寄存器分为程序员可见和实现内部两层：通用寄存器、PC 及规定的程序状态可进入体系结构描述；MAR、MDR、IR 和临时寄存器通常属于实现内部。这里“可见”指软件模型的一部分，是否可由某条用户指令直接读取或写入仍由 ISA 与权限规定。

5.2 寄存器的名字说明它暂存什么

最小反例·地址宽与数据宽可不同

若按字节编址、可寻址 64MiB，地址需要 26 位；若一次访存交换 32 位，MDR 可为 32 位。MAR 为 26 位而 MDR 为 32 位完全自洽，不能因二者都接主存就要求等宽。

CPU 的基本任务是取指、解释指令并控制操作执行，同时处理规定的异常和中断。运算部件完成算术逻辑处理，寄存器保存当前操作需要的状态，控制器安排数据通路中的动作。每个部件的作用要放到一次实际传送中理解。

寄存器	保存内容及用途
PC	与后继取指位置有关的地址；更新规则由执行时序决定
IR	当前正在译码或执行的指令编码
MAR	本次主存访问的地址
MDR	与主存交换的指令或数据
通用寄存器组	指令能按编号访问的操作数、地址或临时结果
PSW/标志寄存器	运算标志及体系结构规定的处理器状态

“寄存器在 CPU 内” 不等于 “它属于通用寄存器组”。PC、IR 与通用寄存器的用途不同；MAR、MDR 在题目的抽象模型中标明访存接口，但这并不意味着所有现代芯片都用同样的名字设置独立物理寄存器。

MAR 位数由可寻址单元数决定，MDR 位数由该数据通路一次访存交换的位宽决定。若数据总线 32 位、按字节编址且有 26 位地址，MDR 可以为 32 位而 MAR 为 26 位，两者无须相等。MDR 刚完成取指时存指令，刚完成数据读时存操作数，它的内容随本次访存的类型改变。

5.2.1 例 5-1　哪些元件负责运算，哪些元件保存状态

例 5.2.1 (配套辨析)

【配套例题】数据通路由组合逻辑元件（操作元件）和时序逻辑元件（状态元件）组成。以下给出的元件中，属于操作元件的是（　）。

I. 算术逻辑部件（ALU）　II. 程序计数器（PC）
III. 通用寄存器组（GPRs）　IV. 多路选择器（MUX）

A. 仅 I、II　B. 仅 I、IV　C. 仅 II、III　D. 仅 I、II、IV

解析。ALU 按当前输入产生运算输出，MUX 按选择信号把某路输入送往输出，二者是组合逻辑。PC 和通用寄存器组保存跨时钟的状态，属于状态元件，选 **B**。寄存器组内部也有组合读出或选择电路，但这不改变它作为状态元件的分类。

5.3 指令周期、机器周期与时钟周期

时钟周期是同步 CPU 中状态推进的基本时间单位。指令周期是从一条指令开始执行到完成所需的整个过程。机器周期是某些实现中为取指、访存等基本操作划分的较大时间单位，不同机器的划分不同，不应把它当成跨体系结构固定的常数。

指令周期可能含取指、间址、执行、中断等阶段；不是每条指令都经过所有阶段。寄存器指令无需为操作数访问主存，间接寻址可能增加取有效地址的阶段，外部中断也只有在满足响应条件时才进入响应过程。

单周期 CPU 把一条指令的全部操作安排在一个时钟周期内，所以时钟必须容纳最慢指令的数据通路；简单指令也使用同样长的周期。多周期 CPU 把指令拆成若干步，周期由各步关键路径决定，不同指令使用不同步数。流水线让多条指令的不同阶段重叠，减少连续指令的平均完成间隔，但未必缩短一条指令自身的延迟。

比较维度	单周期	多周期	简单标量流水线
同一时刻处理的指令	一条	一条的当前步骤	多条各在不同阶段
时钟受什么路径约束	最慢完整指令	最慢基本步骤	最慢流水段及段寄存器开销
一条指令占几拍	1 拍，但该拍较长	随所需步骤变化	通常跨多拍；稳态可每拍完成一条
同一运算资源能否复用	同拍需要的不同用途要避免争用	可跨步骤复用	重叠阶段同拍所需资源要避免争用

例如 PC 增量和数据加法在同一长周期内都需要求和时，不能仅因“单周期一拍”就假定一个无复用路径的 ALU 能同时完成两项操作；多周期可以在不同拍复用，流水线则要核查两者是否会同时发生。CPI=1 的单周期与稳态 CPI 约 1 的流水线可能有完全不同的时钟长度，比较性能仍须回到执行时间。

若题目给“每次访存 2 周期”，应把它安排到取指和操作数读写等实际位置；不能拿“某类指令固定 3 个机器周期”代替具体数据通路的资源分析，实际步数仍以资源限制为准。

不变量·总线同拍只有一个驱动源

单内部总线上，R1 送 Y 与 R2 送 MAR 要求两个不同值同时驱动，发生冲突；R1 同时送 Y 和 MAR 则可能允许，前提是两目标都具备写入路径和独立写使能。

5.4 一条指令怎样变为微操作

微操作是数据通路能完成的基本动作，例如 PC 送 MAR、两寄存器值相加、结果写回。控制信号选择输出源和 ALU 功能，允许某个寄存器写入，或发出存储器读写请求。微操作是动作，微命令是发出该动作的控制命令，两者不是同一个对象。

下例采用单条 CPU 内部总线。一次总线传送只允许一个驱动源，但同一值可以同时送入多个目的寄存器。ALU 的一输入来自暂存器 Y，另一输入来自总线，结果先存 Z，再由 Z 上总线写回。这里先按图确认资源，再写时序，不另补未画出的专用路径。

> **按图排微操作**
>
> 先把机器指令写成寄存器传送语义；再列出每一步的源、目的和可用路径；同拍检查总线驱动冲突、端口数量及写使能；最后加入访存等待与结果可用时刻。若题目没有画某条旁路，就不能为了缩短周期自行增设。

5.4.1 例 5-2　从单总线图确定控制信号与取指时序

【配套例题】某 CPU 中部分数据通路如下图所示，其中，GPRs 为通用寄存器组；FR 为标志寄存器，用于存放 ALU 产生的标志信息；带箭头虚线表示控制信号，如控制信号 Read、Write 分别表示主存读、主存写，MDRin 表示内部总线上数据写入 MDR，MDRout 表示 MDR 的内容送内部总线。

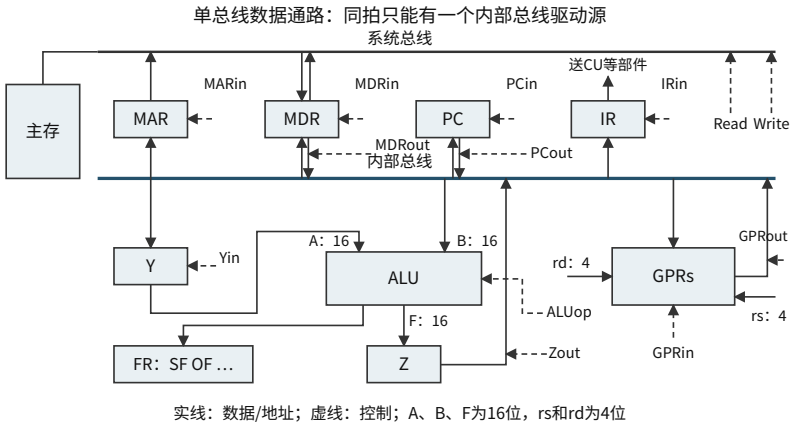


Figure 5.1: 单条内部总线连接 Y、ALU 与 Z；rs、rd 为 4 位寄存器编号

（1）设 ALU 的输入端 A、B 及输出端 F 的最高位分别为 A15、B15 及 F15，FR 中的符号标志和溢出标志分别为 SF 和 OF，则 SF 的逻辑表达式是什么？A 加 B、A 减 B 时 OF 的逻辑表达式分别是什么？要求逻辑表达式的输入变量为 A15、B15 及 F15。

（2）为什么要设置暂存器 Y 和 Z?

（3）若 GPRs 的输入端 rs、rd 分别为所读、写的通用寄存器的编号，则 GPRs 中最多有多少个通用寄存器？rs 和 rd 来自图中的哪个寄存器?已知 GPRs 内部有一个地址译码器和一个多路选择器，rd 应连接地址译码器还是多路选择器?

（4）取指令阶段（不考虑 PC 增量操作）的控制信号序列是什么?若从发出主存读命令到主存读出数据并传送到 MDR 共需 5 个时钟周期，则取指令阶段至少需要几个时钟周期?

（5）图中控制信号由什么部件产生？图中哪些寄存器的输出信号会连到该部件的输入端?

解析。(1) SF 直接记录结果最高位，所以 SF $= F_{15}$。OF 判断有符号运算是否越界。为简写，以下令 $a = A_{15}, b = B_{15}, f = F_{15}$，乘号表示逻辑与、加号表示逻辑或、上横线表示逻辑非：

$$\mathrm{OF}_{\mathrm{add}} = ab\overline{f} + \overline{a}\,\overline{b}f, \qquad \mathrm{OF}_{\mathrm{sub}} = a\overline{b}\,\overline{f} + \overline{a}bf.$$

加法只在两输入同号、结果异号时溢出；减法只在两输入异号、结果符号不同于 A 时溢出。例如 16 位 32767 加 1 得到 8000H，a=b=0、f=1，代入加法表达式得 OF=1；-32768 减 1 得到 7FFFH，a=1、b=0、f=0，代入减法表达式也得 1。SF 只保存结果的最高位，并不保证有符号运算的结果在数学上正确。

（2）同一时刻内部总线只能提供一个值，而 ALU 要同时接收两个输入。先把第一个输入存 Y，总线随后送第二个输入到 B 端，Y 提供 A 端。ALU 产生结果时总线还在送第二输入，若直接把结果也送回同一总线会争用；先用 Z 保存，下拍再由 Z 输出。Y 解决两个输入先后取得的问题，Z 解决结果输出的时机问题；二者并不是两个名字不同的通用寄存器。

（3）rs 和 rd 均为 4 位编号，最多 $2^4 = 16$ 个通用寄存器。它们来自指令寄存器 IR 中的对应字段。rd 决定本拍把结果写入哪一个寄存器，接写地址译码器；rs 决定从哪个寄存器读出，控制读出多路选择器。4 位编号能选 16 个寄存器，不表示每个寄存器只有 4 位。

（4）先用 PC 提供指令地址，访问主存，最后将读出的指令送 IR：

最小反例·同一动作不可重复计延迟

若题给从 Read 到主存数据送 MDR 共 5 拍，这 5 拍已经含送入 MDR。再加一次 “写 MDR” 会重复计数；但随后 MDR 经内部总线写 IR 仍是另一项真实传送。

控制步骤	有效控制信号	数据变化与所需时间
地址准备	PCout、MARin	PC 经内部总线写 MAR，1 周期
主存读	Read	主存按 MAR 地址读出，送 MDR，共 5 周期
装入指令	MDRout、IRin	MDR 经内部总线写 IR，1 周期

错项排除·编码对象先分清

32 条机器指令只需 5 位区分操作码，不说明控制存储器地址也只需 5 位。公共 2 步加每指令 4 步共有 130 位置，微地址需要 8 位；操作码、微地址和控制位数分别统计。

因此至少 $1+5+1=7$ 个时钟周期。5 周期已包含主存读出并送 MDR，无须再加一次 “写 MDR” 的额外周期。图中 MDRin 专指从内部总线写 MDR，并不出现在这条外部主存读入路径上。题目明确不考虑 PC 增量，也不必为更新 PC 多加一步。

（5）控制信号由控制器中的控制部件 CU 产生。IR 输出指令的操作信息，FR 输出 SF、OF 等状态，二者共同为控制决策提供输入。通用寄存器保存的是程序数据，PC 提供执行位置，不能因为它们在 CPU 内，就把所有寄存器都列为本图 CU 的判断输入。

5.4.2 例 5-3　间接操作数经过哪些部件

【配套例题】某指令功能为 $\mathrm{R}[r2] \leftarrow \mathrm{R}[r1] + \mathrm{M}[\mathrm{R}[r0]]$，其两个源操作数分别采用寄存器、寄存器间接寻址方式。对于下列给定部件，该指令在取数及执行过程中需要用到的是（　）。

I. 通用寄存器组（GPRs）　II. 算术逻辑单元（ALU）
III. 存储器（Memory）　IV. 指令译码器（ID）

A. 仅 I、II　B. 仅 I、II、III　C. 仅 II、III、IV　D. 仅 I、III、IV

解析。r1 的值和 r0 中的地址来自寄存器组；按 r0 值访问存储器得到第二个操作数；ALU 做加法并写回 r2，所以 I、II、III 必需，选 **B**。题目已限定 “取数及执行”，指令译码是此前确定操作的工作，不能把整个指令周期使用过的所有部件都加进来。

解数据通路题时，先写出指令的寄存器传送语义，再用图上的实际路径连接源和目的，最后安排周期。总线冲突、写入条件和内存延迟通常比 “背一张步骤表” 更能决定答案。不要为了省周期自行补出未画出的旁路。

5.5 控制信号由组合逻辑还是微程序产生

硬布线控制器根据操作码、当前节拍和状态条件，经译码与逻辑电路产生控制信号。它不必逐条读取控制存储器中的微指令，常有较短控制延迟；新增复杂指令可能需要修改更多逻辑。

微程序控制器把一条机器指令的控制过程组织为微指令序列，保存在控制存储器中。微指令的控制字段产生本拍的微命令，后继地址及条件字段决定下一条微指令。控制存储器通常是 CPU 内部的专用存储器，不是用户主存；微程序计数器或微地址寄存器也不是机器程序的 PC。

两个控制器实现同一 ISA 时，对同一机器指令应产生等价的软件可见结果；差别在于控制序列由逻辑电路直接形成，还是从控制存储器取出微指令形成。机器指令的操作码用于选择一种指令功能，微地址用于选择控制序列中的一步，微命令控制具体部件动作。因而 “机器指令有 32 条”“控制存储器有 130 个位置”“本拍需发 5 条微命令” 数的是三个不同对象，各自的位数不能互相套用。

微指令的直接控制方式为每条控制线分配一位，便于同拍并行，但字宽较大。字段直接编码把互斥微命令分组，每字段经译码选一个命

令，需要保留 “不发该组命令” 的编码。字段间接编码还借助其他字段共同解释。水平型倾向于显式并行多个微命令，垂直型以较短编码表示操作，往往需要更多微指令才能完成同一控制任务；不能笼统地说后者并行性更高。

最小反例·4 条命令不只 4 种状态

一个互斥字段含 4 个微命令，仍需 “不发该组命令” 的第 5 种状态，所以通常至少 3 位。互斥只限制同字段同时发一条，不能省掉本拍暂不用该字段的状态。

5.5.1 微程序控制器的状态怎样推进

控制存储器 CM 保存微指令；微地址寄存器 CMAR（也常称微程序计数器）保存下一次要访问的微地址；读出的微指令进入 CMDR/微指令寄存器，其中控制字段经直接控制或译码发出微命令。后继逻辑依据下地址字段、条件和节拍形成下一微地址。CMAR 与程序 PC 分别指向控制存储器和机器程序空间，不能共用 “指令地址” 的名称而忽略所属层次。

后继微地址可由断定法读取下地址字段，也可顺序递增；机器指令操作码用于确定执行微程序入口，取指或中断等公共微程序入口可由硬件规则给出。课件用固定 0 号作为公共取指入口，是该教学模型的约定，不是所有微程序 CPU 的统一物理地址。

水平、垂直与混合微指令是控制字组织的分类。直接控制用较多位明确允许并行微命令；字段编码用较短编码选择互斥动作；混合形式在紧凑编码与并行控制之间折中。“字短” 不等于 “一条机器指令执行更快”，应比较微周期数与各微周期时长。

5.5.2 例 5-4　互斥类为什么还要一个空操作编码

【配套例题】某计算机的控制器采用微程序控制方式，微指令中的操作控制字段采用字段直接编码法，共有 33 个微命令，构成 5 个互斥类，分别包含 7、3、12、5 和 6 个微命令，则操作控制字段至少有（　）。

A. 5 位　B. 6 位　C. 15 位　D. 33 位

解析。每组除了要能发出其中一条命令，还要能一条都不发。各组位数为 $\lceil \log_2(7+1) \rceil = 3$、2、4、3、3，总计 15 位，选 **C**。例如若某组有 4 条命令，2 位只有 4 种编码，没有 “本拍不用该组” 的位置，通常要 3 位。

5.5.3 例 5-5　控制存储器地址字段怎样估位宽

【配套例题】某计算机采用微程序控制器，共有 32 条指令，公共的取指令微程序包含 2 条微指令，各指令对应的微程序平均由 4 条微指令组成，采用断定法（下地址字段法）确定下条微指令地址，则微指令中下地址字段的位数至少是（　）。

A. 5　B. 6　C. 8　D. 9

解析。公共取指微程序只存一份，控制存储器至少需 $32\times4+2=130$ 个微指令位置，地址至少 $\lceil \log_2 130 \rceil = 8$ 位，选 **C**。32 种机器指令只需要 5 位编码，不表示微指令的存储地址也只需 5 位；两者编码的对象不同。

取指虽然不靠 “上一条指令的操作码” 决定读哪条指令，但仍需要控制信号来选择 PC、发出存储器读并写入 IR。流水线图有时省略这些固定发生的控制动作，但这不代表取指、译码电路不需要任何控制。题目说 “数据通路不包含控制部件” 时，是按功能划分数据处理通路与信号生成部分；这也不意味着数据通路能脱离控制运行。

5.6 异常、中断与返回位置

课程连接·缺页使当前指令重试

OS 调入缺页后，原访问尚未完成，处理器通常从该指令重试；设备完成中断对应一条已完成指令的边界。返回点取决于事件与 ISA 的精确状态规则，不由当前取指 PC 猜测。

本讲义按 408 的常用语境区分：异常与当前执行的指令有关，在执行过程中检测，例如缺页、非法指令、算术异常；外部中断来自指令执行之外的事件，例如设备完成、定时器请求，通常在指令边界接受响应。广义的 “中断机制” 有时统称两者，遇到题目要先辨清其用词范围。

故障 fault 通常可在修复后重新执行，例如缺页后调入页面；陷阱 trap 是有意发出的通知或调试事件，处理完后通常从后继位置继续；终止 abort 表示严重情况，通常不能可靠恢复。不同 ISA 的异常分类和保存 PC 的细则不同，此处不把每一种算术异常固定归入同一类别。

响应异常或中断时，硬件需保存足够的返回信息、切换到规定状态并找到处理程序入口。通用寄存器现场往往由入口汇编代码和软件继续保存。中断服务例程是软件；“中断隐指令” 则是对硬件自动响应动作的教材称呼，不是程序员在代码中插入的一条普通指令。

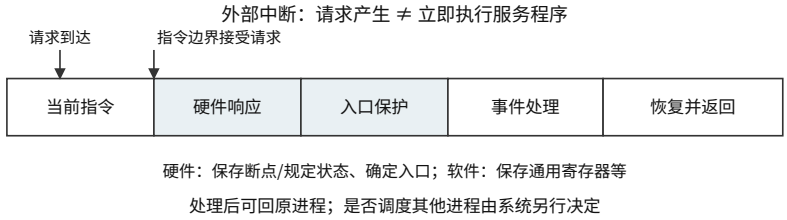


Figure 5.2: 外部请求先等待响应边界，再由硬件入口转入软件处理

5.6.1 例 5-6　缺页后为什么重试当前指令

【配套例题】异常事件在当前指令执行过程中进行检测，中断请求则在当前指令执行后进行检测。下列事件中，相应处理程序执行后，必须回到当前指令重新执行的是（　）。

A. 系统调用　B. 页缺失　C. DMA 传送结束　D. 打印机缺纸

解析。页缺失意味着当前指令需要访问的页面尚未就绪，指令的原定访问还未完成。处理程序调入页面后，应重新执行这条指令，选 **B**。系统调用已经发出了服务请求，不能不加区分地重做调用指令；DMA 完成和打印机事件来自外设，不要求重做某条已完成的普通指令。

外部中断的返回地址通常是下一条尚未执行指令的位置，不能写成 “刚执行完的那一条”。发生故障时可能保存当前指令起点，发生陷阱时可能保存后继位置。给出流水线或变长指令的题目，还要看体系结构怎样保证可恢复状态，不能单凭此时取指 PC 的数值推断精确断点。

处理器进入内核态不必切换到另一个进程。一次设备中断可能处理后返回原进程；调度器也可能决定切换执行主体。模式切换与进程上下文切换相关但不等价。第 7 章进一步讨论设备请求、屏蔽和中断嵌套。

5.6.2 基本控制巩固

1. 【配套例题】相对于微程序控制器，硬布线控制器的特点是（　）。A. 指令执行速度慢，指令功能的修改和扩展容易；B. 指令执行速度慢，指令功能的修改和扩展难；C. 指令执行速度快，指令功能的修改和扩展容易；D. 指令执行速度快，指令功能的修改和扩展难。
2. 原创补充:沿用本章单总线图,能否在一拍同时执行 `R1→Y`、`R2→MAR`? 能否同时执行 `R1→Y`、`R1→MAR`?

答案。1. D，比较的是教材所述常见控制实现的控制开销与修改难度。2. 前者不能，两源争用总线；后者可以，同一来源同时送两个写入端，只要各目的具备独立写使能。

5.7 延迟和完成间隔是两个量

不变量·第一条付满延迟，后续按间隔

理想 k 段、n 条指令总时长为 (k+n−1) 拍：第一条需 k 拍，剩余 n−1 条每拍完成一条。不能给每条都加一次填充延迟，也不能把阶段数直接当任何基线下的加速比。

把一条指令分为 k 段，每段使用自己的部件或占用自己的时段，让多条指令的不同阶段重叠执行，称为指令流水线。同一条指令内部的先后依赖依然存在：必须先取得地址，才能读取相应数据。它主要提高连续指令的吞吐率。

定理 5.7.1 (理想标量流水线的总时间)

理想 k 段流水线，每段耗时一个时钟周期 τ，无停顿且每周期启动一条指令，完成 n 条指令需

$$T_{\text{pipe}} = (k + n - 1)\tau.$$

由第一条延迟与后续完成间隔计数

前 k 个周期使第一条指令通过全线，此后每周期再完成一条。填充时间只在开始付一次，并非每条指令各付一次。吞吐率为 n/T_{pipe}，n 足够大时趋近 $1/\tau$；单条延迟约为 $k\tau$。加入停顿 s 周期后，总时间变为 $(k+n-1+s)\tau$；这里假定每个停顿周期都使最终完成时间推后一个周期。

□

若非流水方案中每条指令依次完整走完这 k 段、每段也占用同样的 τ，则非流水时间为 $nk\tau$，加速比为 $nk/(k+n-1)$。若原方案时钟不同、各段耗时不等或另有资源复用，基线要重新计算；流水级数本身并不等于加速比。

5.7.1 例 5-7　最慢的一段决定时钟下限

【配套例题】某计算机的指令流水线由四个功能段组成，指令流经各功能段的时间（忽略各功能段之间的缓存时间）分别为 90 ns、80 ns、70 ns 和 60 ns，则该计算机的 CPU 时钟周期至少是（　）。

A. 90 ns　　B. 80 ns　　C. 70 ns　　D. 60 ns

解析。相邻阶段寄存器在同一节拍推进，必须留足最慢段 90 ns，选 **A**。若实际阶段寄存器开销为 t_r，则时钟周期通常至少为 $\max_i(t_i) + t_r$；题目已说明忽略这项开销。

在本题时间下，理想完成 10 条指令需 $(4 + 10 - 1) \times 90 = 1170$ ns。若非流水方案每条直接耗时 $90 + 80 + 70 + 60 = 300$ ns，10 条为 3000 ns，加速比约 2.56；若基线规定每段都占 90 ns，基线时间又会不同。比较时必须明确分母采用哪种机器。

若每条指令都有效使用 k 个等时段，n 条共占用 nk 个“段 × 周期”，时空图总面积为 $k(k+n-1)$ 个“段 × 周期”，所以理想流水线效率为 $E = n/(k+n-1)$。有空闲段、停顿或不同功能指令时，应按实际有用面积重新计算，不能把这个理想式用于所有时空图。加速比比较两种方案时间，效率比较段的有效利用，二者不是同一个比例。

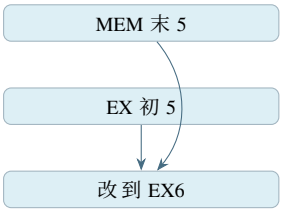
5.7.2 课件中的流水线分类

按使用层次，部件级流水线把算术运算拆段，处理机级流水线把指令过程拆段，处理机间流水线让多台处理机分担任务阶段。按功能可分单功能与多功能；按功能配置能否在同一时刻并行用于不同功能，分静态与动态流水线；按各段是否存在反馈回路，分线性与非线性。非线性流水线可能让一项操作重复经过某些段。

“动态流水线” 在上述分类中讨论功能配置，“动态调度” 则讨论就绪指令能否提前执行，不能仅因都含 “动态” 就当作同一技术。超标量、超流水与超长指令字又是多发射或细分阶段等技术维度，随后分别说明。

最小反例·尚未产生的值不能转发

LOAD 的值到第 5 拍 MEM 末才形成，下一条本应第 5 拍 EX 开头使用它，两者时刻不合。已有 MEM→EX 旁路仍需把消费者推至第 6 拍；“有转发” 不能抹去因果顺序。



同拍初早于同拍末，故需推迟消费者

5.8 阶段寄存器保存的不只是一份数据

五段教学流水线常写为 IF 取指、ID 译码/寄存器读、EX 执行或地址计算、MEM 访存、WB 写回。算术指令也可能经过 MEM 这一拍但不访问数据存储器；存储指令通常不在 WB 写通用寄存器。阶段名只描述一种通路划分，并不意味着每条指令都在各段做同一件事。

阶段之间的寄存器保存下一段所需的操作数、立即数、目的寄存器编号、指令地址及控制信息。若只传数据不传目的编号，结果就可能写到错误寄存器；若指令已经阻塞，写使能却继续向下游传递，就可能重复写入或错误访存。

时钟负责推进状态，控制信号决定状态能否写入、组合部件执行哪种功能。阶段寄存器常需写使能、保持和清空控制，因此 “控制仅作用于组合部件、时钟仅作用于寄存器” 的割裂描述不能当作普遍规则。气泡是在某阶段插入一个不改变有效状态的操作，冻结则让原状态保持，两者效果不同。

5.9 冒险先按冲突对象分类

结构冒险来自同一拍内争用数量不足的硬件资源。例如取指和数据读取同时需要单端口统一存储器；可通过停顿、增加端口或分离指令与数据存储来处理。寄存器组同拍读写是否冲突，取决于端口数及前后半拍的时序约定。

数据冒险来自指令间数据依赖与执行时序的矛盾。写后读 RAW：后指令读前指令产生的新值；读后写 WAR：后指令过早写破坏前指令尚未读取的旧值；写后写 WAW：两次写的完成顺序颠倒，使最终值错误。在固定五段、顺序发射、ID 读寄存器且 WB 顺序写回的简单流水线中，通常需要处理 RAW，而既定次序使 WAR、WAW 不会发生。变为乱序执行或不同延迟写回后，这个结论不再普遍成立。

控制冒险来自后继指令地址未确定，常见于条件分支、跳转、异常和中断处理。无条件转移仍需在某个时刻获得目标地址，因而也可能造成前端控制冒险。“不判断条件” 并不等于 “不会停顿”。

5.9.1 转发能省去等待写回，但不能送来尚未产生的数据

转发把已经形成的结果从阶段寄存器等位置直接送到后续指令使用点，不必等写回寄存器组。例如 ALU 结果在 EX 结束时产生，后继指令下一拍进入 EX，可以从 EX/MEM 寄存器取得该结果。

load 的数据通常到 MEM 结束才产生。紧随其后的指令若下一拍 EX 就需要该值，即使有转发也来不及，必须插入一个等待周期。转发路径是否通往存储数据端、分支比较端，还要看具体电路；不能一见“支持转发”就推断所有相关都已解决。

错项排除·段寄存器还要携带控制

只传操作数不传目的编号，结果可能写错寄存器；保持数据却让写使能继续推进，可能重复写入。流水段状态要同时说明值、目的、有效位与控制信息，不只数“几个数据寄存器”。

5.9.2 同一组 RAW，在不同实现中等到哪一拍

题设实现	消费者取得值的条件	排时序时使用的约束
无转发，寄存器同拍不能读写	必须等生产者 WB 结束，再在 ID 读	消费者 ID 至少比生产者 WB 晚 1 拍
无转发，寄存器同拍先写后读	生产者 WB 前半拍写，消费者 ID 后半拍读	可以让消费者 ID 与生产者 WB 同拍
ALU 结果向 EX 转发	生产者 EX 末已有结果，下一拍送消费者 EX	不必为该值等到寄存器 WB
load 结果向 EX 转发	生产者 MEM 末才有读出数据	消费者 EX 须在该 MEM 之后，紧邻 load 仍可能停 1 拍

“相关”由程序中谁产生、谁使用同一个值决定；“冒险是否导致停顿”还由结果产生点、使用点、转发路径和寄存器时序共同决定。因此，无转发题不能套用 load-use 只停一拍的结论，有转发题也不必机械等到所有源写回。STORE 的数据可能在 ID 读取、在 MEM 使用，分支可能在 ID 或 EX 比较，须先看清具体通路的约定。

5.9.3 例 5-8　装入后立即使用需要一个气泡

【配套例题】下列指令序列中，指令 I1 和 I3、I2 和 I3 之间发生数据相关。假定采用“取指、译码/取数、执行、访存、写回”五段流水线方式，那么在采用转发技术时，需要在指令 I3 之前加入（　）条空操作指令才能使这段程序不发生数据冒险。

```
I1: add  r1, r0, 1    // R[r1] <- R[r0] + 1
I2: load r3, 12(r2)   // R[r3] <- M[R[r2] + 12]
I3: add  r5, r3, r1   // R[r5] <- R[r3] + R[r1]
```

A. 3　B. 2　C. 0　D. 1

解析。I1 在第 3 拍 EX 末形成 r1 结果，到 I3 需要时已经可用；I2 到第 5 拍 MEM 末才有 r3 数据，而无停顿的 I3 本应在第 5 拍 EX 使用它，时间相撞。把 I3 的 EX 推迟到第 6 拍即可，因此选 **D**。

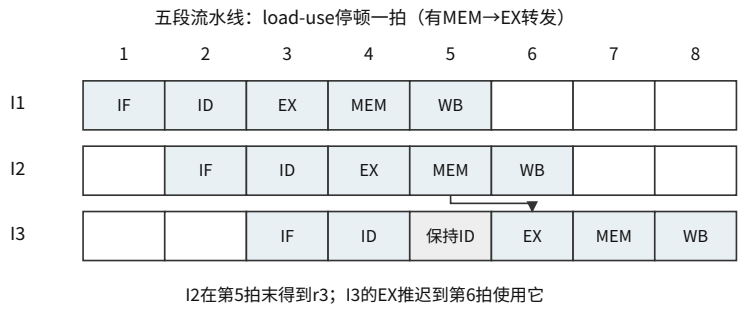


Figure 5.3: load 到 MEM 末才有数据；空泡把使用时刻推迟到下一拍

指令	成功推进的阶段与周期	等待状态
I1	IF1，ID2，EX3，MEM4，WB5	无
I2	IF2，ID3，EX4，MEM5，WB6	无
I3	IF3，ID4，EX6，MEM7，WB8	第 5 拍保持 ID，EX 为气泡

不变量·产生点必须早于使用点

先把每个值的产生段末与消费者所需段初标在时间线上，再检查是否有实际转发路径。STORE 数据端、分支比较端和 ALU 输入端可能不同，不能把一种 load-use 模板用于全部指令。

采用寄存器组同拍先写后读的时序，I1 第 5 拍 WB 写回的 r1 可在该拍被保持在 ID 的 I3 读到；若实现不同，须另有 WB 到 ID 的旁路。第 5 拍 EX 放入气泡，I3 在 ID 保持，下一拍取得装入结果。这里按题目所指的常见五段转发模型，把 r3 的使用点设在 EX；若某图要求在 ID 比较，所需等待和转发路径还可能不同。

5.9.4 例 5-9　同一段序列里同时有数据与控制等待

【配套例题】在采用 “取指、译码/取数、执行、访存、写回”5 段流水线的 RISC 处理器中，执行如下指令序列，其中 s0、s1、s2、s3 和 t2 表示寄存器编号。

```
I1: add  s2, s1, s0  // R[s2] <- R[s1] + R[s0]
I2: load s3, 0(s2)   // R[s3] <- M[R[s2] + 0]
I3: beq  t2, s3, L1  // if R[t2] = R[s3] jump to L1
I4: addi t2, t2, 20  // R[t2] <- R[t2] + 20
I5: L1: ...
```

若采用转发（旁路）技术处理数据冒险，采用硬件阻塞方式处理控制冒险，则在指令 I1～I4 的执行过程中，发生流水线阻塞的指令有（　）。

A. 仅 I3　　B. 仅 I2、I4　　C. 仅 I3、I4　　D. I2、I3、I4

解析。I1 的 ALU 结果可以及时转发给 I2 计算地址，所以 I2 不因这项 RAW 等待。I2 是 load，I3 马上使用 s3，仍有 load-use 等待。I3 的分支结果未确定时，硬件阻塞后继 I4，形成控制等待。因此发生阻塞的是 I3、I4，选 **C**。分析一条序列不能只找到第一组相关就停下；每个使用值的位置和后继 PC 都要检查。

5.9.5 三种处理办法各改变什么

硬件阻塞保持相关指令及其前方的必要状态，在前级暂停、后级放气泡，让生产者先前进。编译调度把不依赖该结果、且移动后不改变程序语义的指令填进等待间隔；无法填满才用 NOP。不能把有副作用或可能越界访存的指令随意移过分支。

分支预测先选择一条路径继续取指，正确时减少等待，错误时清除错误路径上尚不能提交的结果并从正确位置重新取指。若每条指令发生预测错误的概率为 p、每次平均损失 d 周期，且其他开销已包含在基本 CPI 中，可用 $\mathrm{CPI} = \mathrm{CPI}_0 + pd$；“分支比例” 并不等于 “所有指令的预测失败比例”。

旁路主要让结果更早可用，不直接决定分支目标地址；某些设计把数据转发给分支比较器，缩短因数据相关而产生的分支等待，但不等于所有控制冒险都由旁路消除。题目需按具体控制路径判断。

5.10 多发射、动态调度与按序提交

最小反例·改名字不能提前算出值

前指令产生 r1，后指令确实需要这个结果，即使把目的映射为物理寄存器 p7，后指令仍须等 p7 得到值。重命名消除 WAR/WAW 名字冲突，不能消除真实 RAW 因果依赖。

超标量 CPU 每周期可发射多条指令，需要相应的译码、调度、执行和写回资源。是否真能每周期发满，取决于数据依赖、分支、功能部件和访存带宽。最大发射宽度 w 只给出理想吞吐率上限，不能把 w 直接当作任意程序的加速比。

超流水把阶段划得更细，试图缩短时钟周期。阶段寄存器开销和冒险损失会限制收益。超长指令字 VLIW 把若干可并行的操作交给编译器打包，硬件和指令格式仍需支持相应并行执行，并非 “只改编译器，硬件没有要求”。

动态调度可让后面的就绪指令先执行，减少因前面一条长延迟指令而造成的闲置。寄存器重命名为每次逻辑写入分配不同的物理寄存器，可去除 WAR、WAW 这类假相关，但不能消除真实 RAW：消费者必须等对应生产者的值。按序提交使软件看到的状态仍符合程序顺序，并帮助实现精确异常。

例如前指令还要读取旧 r1，后指令将写新 r1 时，为新值分配另一物理寄存器，可避免覆盖旧值；若后指令确实需要前指令算出的 r1，改名并不能凭空产生结果。区分 “名字相同” 和 “值确有依赖” 比背三组字母更有用。

5.11 一条指令处理多个数据，还是多个指令流

Flynn 分类按指令流和数据流区分：SISD 为单指令流单数据流；SIMD 为单指令流多数据流；MISD 为多指令流单数据流；MIMD 为多指令流多数据流。分类描述执行组织，不是按机器装了几个屏幕、几块内存来判断。

向量处理器用一条向量指令对一组数据元素重复执行同类操作，可采用向量流水和多个通道，属于常见 SIMD 组织。它有启动代价，长向量能更好地摊薄这笔代价；向量长度不等于同时存在同样多套完整处理器。SIMD 也不能自然地处理这样的任务：每个元素都要求完全不同的控制流程；分支发散可能降低有效利用率。

MIMD 允许不同处理单元执行不同指令流。共享物理地址空间的多处理器属于 MIMD 的一种，分布式存储系统也可属于 MIMD。SMP 中的 “对称” 是各处理器在系统功能等方面地位对等，不表示每个时刻执行同一条指令。

5.12 硬件线程、多核与共享内存

硬件多线程让一个核保存多套线程体系结构状态，以便在一个线程等待时运行另一线程。细粒度多线程可按周期交替线程；粗粒度多线程常在较长延迟事件时切换；同时多线程 SMT 允许同周期从不同线程发射操作，利用已有执行资源。它们都不要求芯片必须有多个核。

多核是在一个处理器芯片内集成多个执行核心。各核可以独立推进指令流，通常还共享某些 Cache、互连或内存控制资源。增加核数并不保证单线程加速，也不意味着功耗必然高于任意单核设计；工艺、频率、负载和资源共享均影响结果。

共享内存多处理器让各处理器使用同一物理地址空间，但各核寄存器、部分 Cache 仍可私有。共享地址空间只保证各处理器能访问同一对象，不保证多个读改写操作自动具有原子性。并发程序还需要同步，

Cache 一致性与存储顺序也要分别满足相应规则。操作系统的进程同步讲义会沿用这一边界。

课程连接·共享地址并不自动互斥

多核共享物理地址空间，仍可能并发读到同一旧值并分别写回。OS 同步中的原子操作与锁解决复合更新的协调；Cache 一致性不等于所有读改写序列自动原子。

5.12.1 例 5-10　硬件多线程不等于多核

【配套例题】下列关于并行处理技术的叙述中，不正确的是（　）。

A. 多核处理器属于 MIMD 结构
B. 向量处理器属于 SIMD 结构
C. 硬件多线程技术只可用于多核处理器
D. SMP 中所有处理器共享单一物理地址空间

解析。单核也能保存多套线程状态并共享执行部件，所以 C 错，选 **C**。A、B 是按指令流和数据流分类，D 说的是共享物理地址空间；都没有断言所有资源完全相同或所有访问自动同步。

5.12.2 例 5-11　无转发流水线的阻塞传播与最短调度

【配套例题】某 16 位计算机中，带符号整数用补码表示，数据 Cache 和指令 Cache 分离。下面给出了指令系统中部分指令的汇编格式及对应功能，其中 Rs 和 Rd 表示寄存器，mem 表示存储单元地址，(x) 表示寄存器 x 或存储单元 x 的内容。

```
ADD   Rs, Rd
SHL   Rd
SHR   Rd
LOAD  Rd, mem
STORE Rs, mem
```

上面每行给出一种汇编格式；下表逐项给出各助记符对应的名称和功能，不改变源、目的操作数的次序。

名称	助记符	指令功能
加法	ADD	(Rs) + (Rd) → Rd
算术/逻辑左移	SHL	2(Rd) → Rd
算术右移	SHR	(Rd)/2 → Rd
取数	LOAD	(mem) → Rd
存数	STORE	(Rs) → mem

该计算机采用 5 段流水方式执行指令，各流水段分别是取指 IF、译码/读寄存器 ID、执行/计算有效地址 EX、访问存储器 M 和结果写回寄存器 WB。流水线采用 “按序发射、按序完成” 方式，没有采用转发技术处理数据相关，并且同一个寄存器的读和写操作不能在同一个时钟周期内进行。请回答下列问题。

（1）若 int 型变量 x 的值为 −513，存放在寄存器 R1 中，则执行指令 `SHR R1` 后，R1 的内容是多少（用十六进制表示）?

（2）若某个时间段中有连续 4 条指令进入流水线，在其执行过程中没有发生任何阻塞，则执行这 4 条指令需要多少个时钟周期?

（3）若赋值语句为 `x=a+b`，x、a、b 均为 int 型变量，其存储单元地址分别为 [x]、[a]、[b]。对应指令序列及流水过程如下，I3 的 ID 段和 I4 的 IF 段被阻塞的原因各是什么?

```
I1  LOAD  R1, [a]
I2  LOAD  R2, [b]
I3  ADD   R1, R2
I4  STORE R2, [x]
```

最小反例·一核也能保存两线程状态

一个核心可保存两套 PC 和寄存器状态，按周期或等待事件交替执行，也可 SMT 跨线程发射。线程状态数与物理核心数不同，硬件多线程不要求先有多个核心。

指令	成功推进的阶段与周期	因依赖或前级冻结而等待
I1	IF1，ID2，EX3，M4，WB5	无
I2	IF2，ID3，EX4，M5，WB6	无
I3	IF3，ID7，EX8，M9，WB10	第 4～6 拍在 ID 端等待
I4	IF7，ID11，EX12，M13，WB14	第 4～6 拍未获准取指；第 8～10 拍在 ID 端等待

上表将原图时空格紧凑写成“阶段名 + 周期数”，另列等待状态：I3 第 4～6 拍在 ID 端等待，I4 第 4～6 拍尚未获准取指，第 8～10 拍又在 ID 端等待；这些等待状态保留了原图中的阻塞信息；其他未列时刻表示该指令尚未进入或已经离开流水线。

（4）若赋值语句为 `x=x*2+a`，x 和 a 均为 unsigned int 型变量，其存储单元地址分别为 [x]、[a]，执行这条语句至少需要多少个时钟周期？要求模仿上图，画出对应指令序列及流水线执行过程示意图。

解析。本题与例 5-8 的转发模型不同。全题统一采用如下约束：源寄存器若在第 w 拍 WB 写回，则消费者最早第 w+1 拍在 ID 完成取数并向后推进；在此之前可以停留于 ID 端等待，IF/ID 寄存器保持，后继取指也被阻塞。按题中过程，所有指令都要走完五段才算完成，即使某一段不产生实际数据读写。

（1）−513 的 16 位补码为 `FDFFH`，即 `1111 1101 1111 1111`。算术右移复制符号位，得到 `1111 1110 1111 1111`，即 `FEFFH`，真值 −257。表中“除以 2”说明算术右移的数量关系；对负奇数按位右移相当于向负无穷取整，不能套用 C 整数除法向零截断，答成 −256。

（2）无阻塞时，第一条用 5 拍，后 3 条每隔 1 拍完成，共 $5+4-1=8$ 拍。指令 Cache 与数据 Cache 分离，题干未给取指与数据访存争用，不应额外添加。

（3）I3 要读 I1 写入的 R1 和 I2 写入的 R2，存在两条 RAW 依赖。I1 在第 5 拍、I2 在第 6 拍 WB 写回，且同一寄存器不许同拍读写，所以 I3 在第 4～6 拍于 ID 端等待，第 7 拍才可完成取数，而非第 6 拍。I3 占用的 IF/ID 状态尚未释放时，若 I4 继续取指，会覆盖这条待推进指令，因此 I4 的 IF 也推迟到第 7 拍。后一处是前方数据等待向取指端传播，不能误说成 STORE 在 IF 就要读 R2。

等 I3 在第 10 拍 WB 写出和，I4 才可在第 11 拍 ID 读取 R2，随后经过 EX、M、WB，在第 14 拍流出。原图的两段空白应分别追溯到它等待的生产者与受阻的资源，不能把空白一概叫作访存延迟。

（4）用两个寄存器保存 x 和 a，让 x 先装入，以便尽早开始连续依赖的移位与相加：

```
I1  LOAD  R1, [x]
I2  LOAD  R2, [a]
I3  SHL   R1          // 也可用 ADD R1, R1
I4  ADD   R1, R2      // R2 <- 2*x + a
I5  STORE R2, [x]
```

课程连接·状态与控制分工

寄存器保留当前状态，控制器依状态与指令信息发出控制信号；保存状态与产生信号是不同功能。

Figure 5.4: 无转发且寄存器不能同拍读写：分别等待 WB 后的下一拍，最短 17 拍

I1 第 5 拍 WB，故 I3 最早第 6 拍 ID、第 9 拍 WB。I4 依赖 I3 的 R1，最早第 10 拍 ID、第 13 拍 WB；I2 已在第 6 拍写回，不会再额外拖延 I4。I5 最早第 14 拍 ID，至第 17 拍流出，故至少 17 拍。

这个排程还给出了下界证明：本题给定指令中，需要两次 LOAD、一次翻倍、一次相加和一次 STORE；x 的“LOAD→ 翻倍 → 相加 →STORE”链不能颠倒。即使其余工作都尽早完成，这条链也要求 ID 分别不早于第 6、10、14 拍；所列排程恰好达到下界。若先装 a 再装 x，关键链反而晚一拍。STORE 在第 16 拍 M 已写入存储器，但按原题五段完整流出的统一口径，答案仍为 17 拍。

5.12.3 流水线巩固

1. 原创补充：理想 5 段流水线周期 2 ns，完成 20 条指令，中间有 3 个导致整体延长的停顿周期，求总时间与实际吞吐率。
2. 原创补充：若一条指令的结果要到 MEM 末才形成，而消费者在下一拍 EX 开始使用，有 MEM→EX 转发，是否仍需等到生产者 WB 结束？若消费者 EX 与生产者 MEM 是同一拍呢？

答案。1. $(5+20-1+3)\times 2=54$ ns，吞吐率 $20/54\approx 0.370$ 条/ns。2. 前者不需要，可在下一拍使用 MEM 末的结果；后者来不及，需要推迟消费者。关键是结果形成时刻与使用时刻，而不是 “二者有相关” 这句判断本身。

5.13 CPU 内的状态与控制信息

CPU 既执行数据运算，又负责指令控制、操作控制、时间控制和中断处理。通用寄存器可由指令显式指定；PC、IR、MAR、MDR、微地址寄存器等承担特定组织功能，其中哪些对程序员可见要按 ISA 区分。PSW 保存条件码及控制状态，既包含算术结果信息，也可能包含中断允许、特权状态等。

取指周期根据 PC 取得指令并更新顺序地址；间址周期按需要取得

有效地址；执行周期完成指令规定动作；中断周期在满足条件时保存返回所需状态并进入服务程序。不是每条指令都经过全部这些周期。机器周期用于描述某种基本操作组织，时钟周期是时序基准，三者不能按名称认定固定倍数。

课程连接·单总线冲突

同一拍通常只能由一个源驱动总线，多个目的同时接收还需各自路径和写使能。

5.14 单总线数据通路的节拍安排

内部单总线通常同一时刻只允许一个部件驱动总线，多个寄存器可以在满足时序时同时接收同一总线值。ALU 若需要两个输入，而总线一拍只能给出一个源，须由暂存寄存器保存另一操作数；ALU 结果再暂存在 Z 一类寄存器中，后续送总线。

例如计算 $R_1 \leftarrow R_1 + R_2$，可依次组织：$R_1 \rightarrow Y$；$R_2 \rightarrow$ BUS 并使 ALU 执行 $Y + \text{BUS} \rightarrow Z$；$Z \rightarrow R_1$。如果把两个不同源同时接到同一总线，或在结果尚未稳定时提前写入目的寄存器，都违反数据通路条件。专用通路、多总线结构可以增加并行传送能力，但还要检查每条路径和写使能的实际连接。

5.14.1 取指与间址微操作的依赖

取指中常见 PC $\rightarrow$ MAR、$M[\text{MAR}] \rightarrow$ MDR、MDR $\rightarrow$ IR 以及 PC 顺序更新。访存完成前，MDR 中的旧内容不能当作本条新指令。指令进入 IR 后，操作码才可用于译码；若地址字段本身用于间接寻址，还需把它送 MAR，再读出有效地址。

能否把 PC 加 1 与读存储器放在同一节拍，取决于 PC 增量器是否独立、总线是否被占用以及控制器的时序方案。课件节拍表是某一具体组织的实例，应先理解依赖关系，再照图列允许的并行动作。

5.15 硬布线控制器从何处获得控制信号

硬布线控制器把指令操作码、时序节拍、机器周期状态和有关条件标志输入组合逻辑，生成各寄存器写使能、总线输出使能、ALU 操作选择及存储器读写控制。设计时先分析指令微操作，再安排节拍，最后把“某指令、某周期、某节拍、某条件下发出某信号”合成为逻辑表达式。

例如某控制信号在指令 A 的 T_1 或指令 B 的 T_2 发出，可概括为 $C = A \cdot T_1 + B \cdot T_2$；若还取决于 ZF，应把条件乘入对应项。这里的加、乘是逻辑或、与。硬布线速度通常较快，但指令和控制条件增加时，设计、修改及验证复杂度也增加。

5.16 微程序控制：区分四种地址

主存地址定位机器指令或数据，PC 保存程序级指令地址；控制存储器 CM 保存微指令，CMAR 保存微指令地址；CMDR 保存当前微指令，操作控制字段输出微命令，顺序控制字段参与确定下一微地址。主存和控存是不同逻辑空间，机器指令地址与微指令地址也不是同一数值体系。

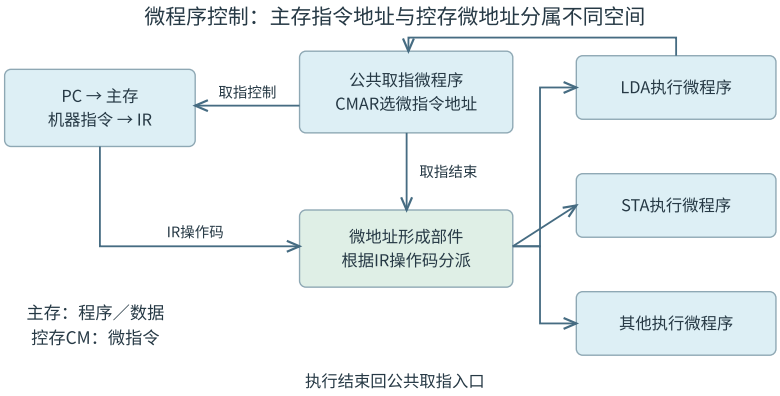


Figure 5.5: 公共取指微程序先运行，再由操作码映射到相应执行微程序入口

取指微程序可以由全部机器指令共享。取指结束后，根据 IR 操作码形成执行微程序入口；执行微程序结束后，通常回到公共取指入口。遇到条件转移、间址或中断时，还可能根据测试条件和周期状态选择相应微程序分支。

5.16.1 微指令编码与下地址形成

直接控制法为每个控制信号分配一位，译码简单、并行性强而字长较长。字段直接编码把互斥微命令分到同一字段，不同字段可并行；一个含 k 个可选微命令的字段若需额外表示“本字段不发命令”，至少 $\lceil \log_2(k+1) \rceil$ 位。相关编码还可利用字段之间的依赖进一步压缩，但增加译码层次。

水平型微指令一般允许同拍多个微操作并行，控制字段较宽；垂直型采用更强编码，字长较短而执行微操作的并行程度受到限制。这里的水平、垂直是控制信息组织方式，不是微程序在纸上的排版方向。

下一微地址可由当前微指令下地址字段直接给出，也可由微地址计数器顺序增量、指令操作码映射、条件测试或硬件强制入口形成。下地址位宽按控存实际需要区分的微地址数量估算，而不是按机器指令条数直接取对数。

设 32 条机器指令，各自执行段平均 4 条微指令，共享取指段 2 条，且这些段互不重叠，则至少需存储 $32 \times 4 + 2 = 130$ 条微指令，顺序地址字段至少 8 位。若微程序共享其他公共段、还含中断段或间址段，须据实际布局重新统计。

5.16.2 微程序控制单元的设计步骤

先列每条机器指令所需微操作并安排可并行的节拍，再补入“读下一微指令、形成微地址”所需的控制动作；选择控制字段编码和下地址形成方式，确定微指令字长，最后分配控存地址并编写各条微指令码。用当前微指令控制数据通路的同时，还应确保下一个微地址及时稳定。

静态微程序通常固化于 ROM，动态微程序允许通过可改写控存调整解释规则。毫微程序在微指令之下再设一层解释，以进一步压缩或组织控制信息。微程序修改依赖硬件支持及控制存储器类型，不能把修改普通程序文件等同于修改 CPU 微程序。

不变量·微指令字段各有职责

控制字段决定本拍发出的微命令，顺序字段参与形成下一微地址；操作码映射只用于需要选择微程序入口的阶段。

5.17 五段流水线中的装入、存储和转移

课程连接·值何时可用

装入结果在访存完成后可用；旁路路径和使用时刻共同决定需要几个气泡。

IF 取指，ID 译码并读取源寄存器，EX 进行算术运算或计算有效地址，MEM 访问数据存储器，WB 写回寄存器。不是每条指令都在每一段执行同样的实质工作，但简化固定五段模型可保留阶段位置以统一时序。

装入指令在 EX 算地址、MEM 取数据、WB 写寄存器；存储指令在 EX 算地址、MEM 写数据，通常没有寄存器写回；算术指令在 EX 产生结果，后续沿阶段寄存器传递；条件转移在哪段确定方向和目标，应按题设明确。阶段寄存器除数据外还须保存后续需要的目的寄存器号、控制位和异常信息。

在理想等长 k 段、每段时间 Δt、连续输入 n 条指令的模型中，总时间 $(k+n-1)\Delta t$，吞吐率 $n/[(k+n-1)\Delta t]$，相对完全串行的加速比 $nk/(k+n-1)$，效率 $n/(k+n-1)$。这些公式的前提是无冒险、无额外等待且阶段时间相同；不等长阶段通常按最慢段及寄存器开销确定时钟。

5.17.1 硬件多线程与共享内存组织

粗粒度多线程通常在较长停顿时切换线程，切换后还可能需要清空并重新填充流水线；细粒度多线程按较短固定粒度交替发射不同线程指令，用更多独立指令掩盖停顿；同时多线程 SMT 允许同一周期从多个线程选择指令占用不同发射槽。每个线程需要自己的体系结构状态，而功能部件等资源可以共享。

共享内存多处理器共享物理地址空间，可用 LOAD/STORE 访问共同主存；多计算机系统各有私有主存，通过消息传递通信。多核表示一块芯片集成多个处理核，是物理集成方式；SIMD 表示一个指令流控制多个数据通路，是并行控制方式，不能把二者当作同一分类轴。

5.17.2 三种硬件多线程的发射比较

时钟	细粒度：每拍选择一个线程	粗粒度：长停顿时换线程	SMT：一拍可选多个线程
i	A 的 j、j+1	A 的 j、j+1	A 的 j、j+1；B 的 k、k+1
i+1	B 的 k、k+1	A 的 j+2、j+3，发现 Cache 缺失	A 的 j+2；B 的 k+2；C 的 m
i+2	A 的 j+2、j+3	调度，从 A 切换到 B	A 的 j+3；C 的 m+1、m+2
i+3	B 的 k+2、k+3	B 的 k、k+1	按就绪指令及资源继续选择

该示例中的细粒度多线程允许同一拍从选中的线程发射多条指令；关键是这一拍仅选择一个线程。SMT 则可以在同一拍把不同线程的就绪指令安排到多个执行槽。粗粒度切换时可能需要重填流水线，因此切换的成本与粒度相关。

5.17.3 计算与辨析自查

1. PC、MAR、CMAR 保存的地址分别属于什么空间？2. 含 7 个互斥微命令及空操作的字段需要几位？3. 理想 5 段流水线执行 20 条指令需几拍？4. 为什么 load-use 冒险可能在有转发时仍需停顿？

答案。1. PC 为机器指令地址，MAR 为本次主存访问地址，CMAR 为控存微指令地址。2. 3 位，正好 8 种编码。3. $5+20-1=24$ 拍。4.

使用者需要操作数时，前一装入指令的数据可能尚未从 MEM 产生，转发只能传已有数据。

不变量·调度下界来自不可颠倒链

无转发且不许同拍读写时，LOAD→翻倍 → 相加 →STORE 链的各次 ID 必须晚于相应生产者 WB。先给关键链下界，再排出达到该下界的合法时序，才能证明 “最少几拍”。

5.18 主动回忆自测

以下均为自拟检查。沿章导图，先说明对象与前提，再写结论。

1. PC、IR、MAR、MDR 分别暂存什么？为何 MAR 与 MDR 位宽无须相同？
2. 一条指令是否总经过取指、间址、执行与中断四阶段？
3. 单内部总线同拍可否由两个不同源送两个不同目的？同一源送两个目的呢？
4. 机器指令、微指令与微命令各控制或表示哪个层次？PC 与 CMAR 有何区别？
5. 某互斥字段有 4 个微命令，为什么按通常约定至少需 3 位？
6. 故障处理后重执行当前指令，与外部中断后返回后继位置，依据是什么？
7. 理想 5 段、每段 2ns、20 条指令加 3 拍整体停顿，总时间和吞吐率分别是多少？
8. 在 EX 形成的 ALU 值与 MEM 末形成的 load 值，紧邻消费者所需等待为何可能不同？
9. 寄存器重命名能否消除真实 RAW？单核能否采用硬件多线程？
10. SIMD、MIMD、超标量与 SMT 分别按哪种并行维度分类？共享地址空间是否使读改写自动原子？

5.18.1 自测答案

1. PC 与后继取指位置有关，IR 存当前指令，MAR 存访存地址，MDR 存与主存交换的值；地址位数取决于可寻址单元数，MDR 位数取决于交换数据通路。2. 不总是；间接寻址、外部中断等有相应条件。3. 不同源通常冲突；同源多目的在独立写使能和实际路径允许时可以。4. 机器指令规定 ISA 操作，微指令组织一个控制步，微命令驱动具体部件；PC 访问机器程序，CMAR 访问控制存储器。5. 4 个命令加 “不发本组命令” 共 5 种状态，需 3 位。6. 故障尚未完成原定操作，可恢复时需要重试；外部中断通常发生在指令完成边界，但具体保存 PC 规则依 ISA。7. $T = (5 + 20 - 1 + 3) \times 2 = 54$ ns，吞吐率约 0.370 条/ns。8. 按结果形成时刻、消费者使用时刻及转发路径判断；数据尚未形成就无法转发。9. 不能消除真实值依赖；单核可保存多套线程状态并共享执行资源。10. SIMD/MIMD 按指令流与数据流，超标量按每周期多发射，SMT 按同周期跨线程利用执行部件；共享地址空间仍需原子操作与同步。

总线 6

章节脉络

6.1 从概念、性能到事务时序

本章沿课件导图的四组关系展开：先由 “共享通路” 辨认总线的组成、分类与结构，再把位宽、周期和传送率写成可计算的量，随后按申请、寻址、传输、结束四阶段追踪一次事务。定时方式回答 “何时允许推进”；仲裁回答 “由谁取得使用权”，两者分属不同问题。

定义 6.1.1 (总线的共享性)
总线是一组能被多个部件分时共享的信息传送通路。连接在同一条公共通路上，不表示多个主设备可以同时任意驱动线路；同一时刻允许的驱动者、接收者及信号解释由协议决定。

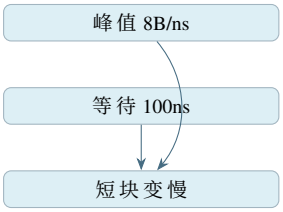
总线特性可以按四个维度检查：机械特性规定连接器、尺寸和形状；电气特性规定电平、负载等；功能特性规定每根线承载什么；时间特性规定信号建立、采样及撤销关系。它们分别回答 “如何接、什么电平、传什么、何时有效”。

按位置与用途可分片内总线、连接 CPU/主存/I/O 接口的系统总线，以及部件或系统间的通信通路；按数据形式分串行与并行；按定时分同步、异步等。一个具体接口可以同时具有 “串行”“包传输”“多通道” 的属性，分类维度不能混为一个互斥列表。

单总线结构让多个部件共享一条系统通路，连接简单但易形成争用；双总线、三总线以不同层次分担主存和 I/O 等流量。桥接器完成不同总线间的数据缓冲、协议转换和控制。增加通路能减少某类冲突，仍须核对桥接和主存是否成为瓶颈。

最小反例·峰值相同，事务速率可不同

两条总线数据拍都每 ns 传 8 B，一条每块额外等待 100 ns，另一条只等 1 ns；二者峰值相同，短块的有效速率却相差很大。分母是否包含等待必须先由题问确定。



相同数据拍峰值，等待改变事务有效速率

6.2 位宽、频率、传送率分别给出什么

总线是部件之间传送信息的公共通路。按信息用途可区分地址、数据和控制：地址说明访问哪个单元或端口，数据线承载被交换的值，控制线

说明操作类型、有效时刻及应答等。命令字虽然在语义上“控制设备”，CPU 写命令字时仍可能在**数据线**上传送；不能只凭名称判断该走哪组线。

数据线上可以传指令、操作数、状态字、命令字和中断类型号。它们如何解释取决于传送双方和操作类型。MAR 保存待访问地址，MDR 暂存交换的值；主存并不靠“某个数据看起来像地址”来判定这次访问的性质，而是由相应线路与控制时序确定。

总线位宽通常指并行数据线根数。串行传送逐位使用通道，并行传送同一时刻通过多条线传若干位；多个串行通道并用，并不等于一条传统并行总线。系统可用桥接部件连接不同速度和协议的通路，靠近 CPU 的部件通常要求更高带宽；有多条总线，并不表示它们共用同一时钟。

最小反例·串行不等于慢

一条高速串行链路与一条低频 8 位并行总线，谁带宽更大还需各自的传送率。位宽只决定每次能送多少，线路频率、编码和协议开销同样影响结果，不能用串/并行名称代替计算。

带宽先写每次字节数与每秒次数

若有效数据宽度为 W 位，每秒实际发生 R 次数据传送，理想单方向峰值带宽为 $R \times W/8$ B/s。R 与时钟频率不一定相同：每周期一次为 R=f，每周期两次为 R=2f；若已给 MT/s，则已经是每秒传送次数，不能再乘“每周期次数”。

总线周期是一次规定的总线操作所占的时间，可能有多个时钟周期。带宽为每次传送的有效数据量除以这次所用时间；控制、地址、等待、校验与空闲开销会使有效数据率低于只算数据节拍的峰值。

6.2.1 例 6-1　一个总线周期占两个时钟周期

【配套例题】假设某系统总线在一个总线周期中并行传输 4 B 信息，一个总线周期占用 2 个时钟周期，总线时钟频率为 10 MHz，则总线带宽是（　）。

A. 10 MB/s　　B. 20 MB/s　　C. 40 MB/s　　D. 80 MB/s

解析。时钟周期 100 ns，一次总线传送需 200 ns，传 4 B，因此带宽 $4/(200 \times 10^{-9}) = 20 \times 10^6$ B/s，选 **B**。若直接用 4 B×10 MHz，就忽略了每次传送占两拍。

6.2.2 例 6-2　已给传送率时不重复乘每周期次数

例 6.2.1 (自拟数值例)

某 64 位并行总线每时钟周期传 4 次数据，实际数据传送率已明确给为 1333 MT/s，求单方向理想峰值带宽。若改给的是 333.25 MHz 时钟频率，结果是否相同?

解析。每次 64 位 =8 B；1333 MT/s 就是每秒 1333×10^6 次，带宽为 $1333 \times 10^6 \times 8 = 10.664$ GB/s。若给 333.25 MHz 且每拍 4 次，则先算 $R = 4f = 1333$ MT/s，结果相同。若改成 1333 MHz 时钟、仍每拍 4 次，才会得到 42.656 GB/s。三个单位条件必须分别使用，不能把“频率”字样覆盖题干明示的物理量。

多通道带宽只有在各通道能够并行传送且工作分配允许时才相加。全双工总线要区分单方向带宽和两个方向合计带宽；校验位若不属于有效载荷，不应计入有效数据量的分子。带宽题第一步宜写出“每次有效几字节、每秒几次”，再统一单位。

6.3 一次总线事务从地址到应答

不变量·先数有效数据拍

32 B 块经 64 位数据线传送，需要 4 个有效数据单位。突发减少的是地址、准备等重复代价，不能把 32 B 整块误作一个 8 B 数据拍；每拍几次传送还要另核时钟口径。

总线主设备发起事务，从设备按地址被选中并完成操作。CPU 常是主设备，DMA 控制器也可在取得使用权后成为主设备。主从关系只就本次事务而言，不等于某个设备永久只能发送或只能接收。

一次读事务可以分为：主设备发地址和读命令；从设备译码并准备数据；从设备在数据线上给出有效数据；主设备采样，双方结束本轮操作。写事务则由主设备给出要写的数据。地址、数据若复用同一组线路，就需不同时间片分别传送，不能把这些线路同时算成两份独立的并行位宽。

6.3.1 同步定时：固定采样时刻，必要时插等待

同步总线用共同的时钟约定信号的有效和采样时刻。最短周期必须覆盖信号传播、译码、设备准备及建立时间等延迟。若从设备来不及，可按协议插入等待状态；“同步”不等于所有事务长度永远相同，也不表示每个设备可随意提供互不协调的时钟。

突发传送只给出一个首地址，再连续传送多个相邻数据，分摊地址和启动开销。非突发则可能为每个数据重复地址及准备过程。因此“读一块多长时间”应先数数据拍数，再判断地址、等待代价要付几遍。

6.3.2 例 6-3　不支持突发，就要反复准备每个数据

【配套例题】某存储器总线宽度为 64 位，总线时钟频率为 1 GHz，在总线上传输一个数据或地址需要一个时钟周期，不支持突发传送方式。若通过该总线连接 CPU 和主存，主存每次准备一个 64 位数据需要 6 ns，主存块大小为 32 B，则读取一个主存块所需的时间是（　）。

A. 8 ns　　B. 11 ns　　C. 26 ns　　D. 32 ns

解析。一拍 1 ns，每个 64 位数据为 8 B，一块需要 4 个数据。每个数据都单独付地址 1 ns、准备 6 ns、传送 1 ns，共 8 ns，总计 $4\times 8=32$ ns，选 **D**。11 ns 对应只付一次地址和准备、再连传 4 次，是突发模型，题目已排除。

6.3.3 例 6-4　峰值带宽与整次事务的有效速率

【配套例题】某存储器总线的时钟频率为 420 MHz，总线宽度为 64 位，每个时钟周期传送 2 次数据；其总线事务支持突发传送方式，最多传送 8 次数据，第 1 个时钟周期传送地址和读/写命令，从第 4 个至第 7 个时钟周期连续传送 8 次数据。该总线的总线带宽（最大数据传输率）为（　）。

A. 3.84 GB/s　　B. 6.72 GB/s　　C. 30.72 GB/s　　D. 53.76 GB/s

解析。问的是数据节拍的最大传送率，每周期 2 次、每次 8 B，峰值 $420\times 10^{6}\times 2\times 8=6.72$ GB/s，选 **B**。

若另问这次 8 次数据的事务有效速率，才把地址和等待计入分母：数据 64 B、事务 7 周期，$64/(7/(420\times 10^{6}))=3.84$ GB/s。A 正是按整次事务口径算出的结果，不能用它回答本题已经说明的峰值带宽。第 4 至第 7 拍共 4 拍，每拍 2 次，正好 8 次，不是 8 拍。

6.3.4 峰值、事务速率与 CPU 占用采用不同分母

不变量·撤销也要确认

全互锁中，从设备看到请求撤销后才撤应答。少看最后一步会把半互锁误作全互锁；画四个边沿并逐项标出依赖，比按波形长度猜类别可靠。

总线峰值带宽按数据节拍可持续搬多少有效字节来计，事务有效速率还把本次地址、等待等时间算入；设备速率是设备产生或消耗数据的速度；CPU 占用率则是 CPU 执行查询、服务或设置收尾的时间比例。这四个量可能出现在同一道题中，应先分别写成“什么量除以什么时间”。

例如在例 6-4 中数据节拍峰值 6.72 GB/s、整次事务有效速率 3.84 GB/s，都不能直接当成某个设备的实测速率；若设备只提供 40 MB/s，系统还受这个供数速度限制。反过来，DMA 可让设备数据直接经过总线进入主存，同时 CPU 仅做少量设置收尾，所以不能把总线使用率等同于 CPU 使用率。端到端吞吐要核最慢环节，资源占用要核实际争用的时段。

当一次突发在等待期间必须保持总线占有权时，等待也消耗总线资源；若题设明确采用分离事务、允许准备期间让出总线，则总历时与实际占用时间还会不同。画事务时间线时，应同时标出“在等什么”和“谁仍占有总线”，不能仅凭延迟长短直接推断总线忙碌比例。

6.3.5 异步定时：用握手确认进度

异步总线不用统一周期来强制双方同拍完成，而用请求和应答等信号交换进度。速度较慢的设备晚些应答，主设备相应等待。异步仍受顺序约束，也不能随时改变信号而不通知对方。

全互锁的四阶段握手可以写为：主设备置请求；从设备准备好后置应答；主设备看到应答后撤请求；从设备看到请求撤销后撤应答。各阶段都等待对方作出相应反应，因而不依赖预设的固定延迟来猜测对方已结束。

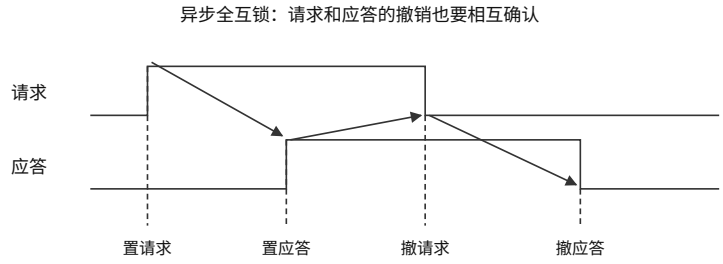


Figure 6.1: 全互锁握手的撤销顺序也由对方信号确认

非互锁方式中，主设备发出请求后按预定时间撤回，不等应答；从设备发出应答后也按时撤回，不等请求撤回。半互锁方式中，主设备等待应答出现才撤请求，而从设备仍可按时撤应答。全互锁方式中，从设备还必须等请求撤回才撤应答。给出时序图时，按图中信号的建立、撤销条件来识别，不单凭信号名称猜测。全互锁通常多了等待环节，不能因为最可靠就断定它最快。

半同步通信将时钟节拍与等待/应答结合：正常情况下按时钟节拍推进，慢设备可请求延长，握手状态在规定边沿采样。这里的“半同步”不等于一半数据串行、一半数据并行，两个分类维度不同。

6.3.6 如何读总线时序图

先标出地址何时稳定、读写命令何时有效、数据何时有效和何时被接收，最后数出从事务开始到完成的时间。若图中低电平有效，应以标出的有效电平判断，不可把所有上升沿都当成开始。不同信号能否同拍出

现由协议规定；“地址/数据复用” 和 “独立地址数据线” 会改变哪些操作可以重叠。

串行异步字符传送每帧另加起始位和停止位，校验位可选。若每字符 7 个数据位、1 个校验位、1 个起始位和 1 个停止位，则每字符实际发送 10 位，480 字符/s 需要 4800 bit/s，而有效字符数据只有 3360 bit/s。不要把物理线路传送率与有效数据率混为一谈。

错项排除·主设备角色随事务

DMA 可以发起一次主存写，CPU 也可能暂不占总线。主从描述谁发起并控制本次交换，并不表示某设备永远只能发送或永远只能接收。

6.3.7 事务速率巩固

1. 【配套例题】某存储器总线采用同步通信，时钟频率 50 MHz，突发传 8 个字，每字 4 B。1 周期接收地址、3 周期等待、8 周期传 8 个字。数据传输速率为（ ）。A. 114.3 MB/s　B. 126 MB/s　C. 133.3 MB/s　D. 144.3 MB/s。
2. 【配套例题】：将第 1 题改为每个字都独立付 1 周期地址、3 周期等待、1 周期数据，读同样 8 字需要多少时间，有效速率多少?

答案。1. C。共 12 周期，每周期 20 ns，32 B/240 ns ≈ 133.3 MB/s。2. 共 40 周期 =800 ns，32 B/800 ns=40 MB/s。两题块长相同，改变的是启动代价的重复次数。

6.4 总线仲裁与分离事务

总线传输在申请分配阶段处理多个主设备的使用权竞争，随后寻址、传输并结束。CPU 和 DMA 都可能成为主设备，主设备与从设备的角色针对一次事务；主设备申请总线，不表示它一定是本次数据的接收者。

课件将仲裁分为集中与分布两类。集中仲裁由一个仲裁部件授予使用权：链式查询以允许信号沿设备链传递，靠近仲裁端的请求通常先被满足，控制线少但固定优先级可能使低位请求长期等待；计数器定时查询依次给出设备号，请求者在号码匹配时取得资格，起始编号和轮转规则决定公平性；独立请求为各设备设置请求/允许通路，仲裁灵活但线路及逻辑开销较大。分布仲裁由参与者依共同规则竞争或判定，不能因此推断系统没有任何仲裁规则。

课件把总线仲裁与标准列为拓展。本章保留其机制联系，不凭文件名替读者确认某一考试年度的正式范围。总线标准还约定接口、电气、功能、时序等兼容条件；串行或并行只说明传输组织，不能单独决定峰值带宽或延迟。

分离事务允许主设备发出请求后释放总线，待从设备准备好再申请并返回结果；因此 “总历时” 与 “总线实际占用时间” 可以不同。它与普通同步/异步定时的分类角度不同，还需要标识请求与响应的对应关系。

定时/组织	推进依据	等待是否占通路	不能直接推出
同步	规定时钟沿；慢设备可插等待	由事务协议决定	每事务固定一拍
异步全互锁	请求/应答建立和撤销均互相确认	由握手占用规则决定	不需要时序约束
半同步	时钟节拍与等待/应答结合	需看等待状态的保持规则	一半串行一半并行
分离事务	请求与响应分开争用通路	准备期间可释放	总历时等于占用时长

6.5 总线按位置、信息和连接方式分类

课程连接·分类依据

数据、地址与控制描述信息类型；片内、系统与通信总线描述所处层次，分类角度各自独立。

总线是一组由多个部件共享的信息传输线路及配套协议。片内总线连接芯片内部部件，系统总线连接 CPU、主存和 I/O 接口，通信总线连接计算机、外设或系统之间的通信端点。数据总线传数据，地址总线选择存储单元或端口，控制总线规定读写、请求、应答、时序和状态。

地址总线宽度主要影响可给出的地址组合数，数据总线宽度影响一次可并行传送多少位。CPU 发起普通访存时地址由 CPU 输出；DMA 控制器成为主设备时也可发地址，所以“地址线总由 CPU 单向输出”只适用于限定的主设备关系。总线主设备启动事务，从设备响应事务，I/O 设备不必永远是从设备。

总线标准需要同时规定机械特性、电气特性、功能特性和时间特性：接插件尺寸与引脚布局属于机械规范；电压、负载等属于电气规范；各线表示什么信息属于功能规范；信号何时建立、何时采样属于时间规范。只做到插头相同并不能保证协议兼容。

6.5.1 单总线、多总线与桥接器

单总线组织连接简单、扩展方便，但不同部件争用同一通路，且速度受多方协调限制。多总线组织把 CPU—主存高速流量与较慢 I/O 流量分开，桥接器完成不同总线间的缓冲、转换和控制。增加总线可以减少争用，却也增加连接和控制成本；跨桥事务可能引入额外延迟。

串行传输每条数据通道按先后发送位，并行传输用多条线同拍传多位。串行链路可以使用多条 lane 并行传包；因此“串行协议”与“只有一条数据通道”不是同一判断。并行线间偏斜、串扰和接插件复杂度，会限制频率和距离；较少线数的高速差分串行链路可通过更高信号速率取得较高带宽。

6.6 四种定时方式的完成条件

同步定时用共同的时钟规定信号产生和采样时刻，简单而要求各模块满足统一时序；必要时用等待状态延长事务。异步定时不依靠统一固定等待时间，而用请求和应答信号确认进度，适合速度差异较大的部件；握手也有控制开销。

异步握手可按发送方撤销请求是否等待应答，以及接收方撤销应答是否等待请求撤销，区分不互锁、半互锁和全互锁。全互锁四阶段为：主设备置请求、从设备置应答、主设备撤请求、从设备撤应答。后一动作依赖前一动作，形成完整的建立和释放确认。

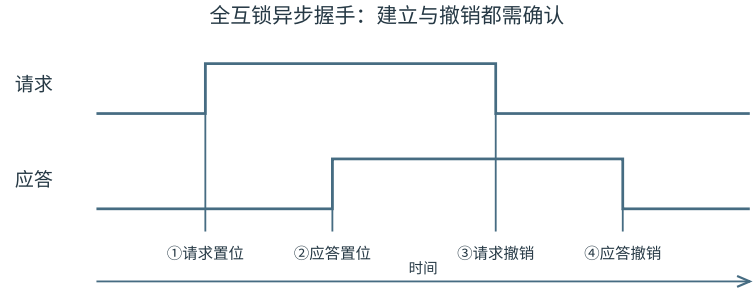


Figure 6.2: 全互锁握手中，请求和应答依次建立并依次撤销

半同步通信保留时钟基准，再由从设备用等待信号延长数据准备

阶段，兼顾采样规则和速度适应。分离事务把请求和返回数据分成两次总线占用；从设备内部准备数据时释放总线供其他事务使用，之后再申请总线返回结果。分离事务需要标识与排队机制对应请求和响应。

课程连接·集中式仲裁

链式查询、计数器定时查询和独立请求方式的连线与优先级规则不同，应分别判断。

6.7 拓展：三种集中式仲裁

多个主设备同时请求总线时，需要仲裁决定使用权。优先级决定谁先获得，公平性关系到低优先级设备会不会长期等待，二者不是同一个指标。

方式	选择机制	特点与限制
链式查询	允许信号沿设备链传播	结构简单，近仲裁器者通常优先；链路故障敏感
计数器定时查询	用计数编码轮询设备号	起始计数方式可改变优先顺序；需编码线和比较逻辑
独立请求	每个设备独立请求和允许线	响应快、优先级灵活；线路较多

按课件约定，n 个设备的链式查询可用请求、忙、允许共 3 条控制线；计数器方式需 $\lceil \log_2 n \rceil + 2$ 条；独立请求若每设备各有请求和允许并共用忙线，则为 $2n + 1$ 条。不同教材图的公共信号统计口径可能不同，计算必须先列线的种类。

6.8 拓展：常见总线和接口标准

标准	主要对象或组织	识别要点
ISA ／ EISA	早期 PC 扩展	传统并行总线，位宽及传送组织随版本变化
VESA 局部总线	早期高速外设	连接高速显示等局部流量
PCI	共享并行局部总线	支持总线主控、突发传输及配置机制
AGP	图形接口	为图形加速设计的历史接口
PCI Express	点到点串行链路	可多 lane，全双工；带宽取决于代际与 lane 数
USB	主机与外设	串行、支持集线器、即插即用和热插拔
SATA	存储设备	串行存储接口，标称速率按具体版本解释

标准	主要对象或组织	识别要点
SCSI	存储及其他设备接口体系	支持设备寻址、命令与控制机制
RS-232	串行通信	规定连接、电平及有关信号
PCMCIA ／ CardBus	便携设备扩展	历史上的卡式扩展标准

表中的名称用于识别结构和适用对象。某一课件列出的带宽是具体版本或理想条件下的值，不能当作该标准所有设备的统一实测速率。

按编码效率、包头、方向和通道数修正标称物理速率，才能得到有效载荷带宽。

例如 32 位数据通路、每秒 33×10^6 次有效数据传送时，理论数据率为 $33 \times 10^6 \times 4 = 132 \times 10^6$ B/s；若事务还需地址拍、仲裁和等待，有效事务带宽低于该峰值。DDR 表示每时钟周期可在两个边沿传数据，已给传送率时须避免再次乘 2。

课程连接·单位复核

先确定一次传输的字节数和每秒传输次数，带宽再据此计算；频率不必然等于事务完成率。

6.8.1 计算与辨析自查

1. 为什么数据线数不能直接决定最大寻址容量？2. 全互锁握手的四个动作按什么顺序发生？3. 分离事务释放的是什么资源？4. 8 设备独立请求按上述计线口径需要多少条控制线?

答案。 1. 寻址组合由地址编码及编址单位共同决定，数据线主要影响每次传送位数。2. 请求置位、应答置位、请求撤销、应答撤销。3. 设备准备数据期间的共享总线使用权。4. $2 \times 8 + 1 = 17$ 条。

6.9 主动回忆自测

沿导图回答，再核对单位与条件

以下为自拟检查，先遮住答案，按导图主枝回答。

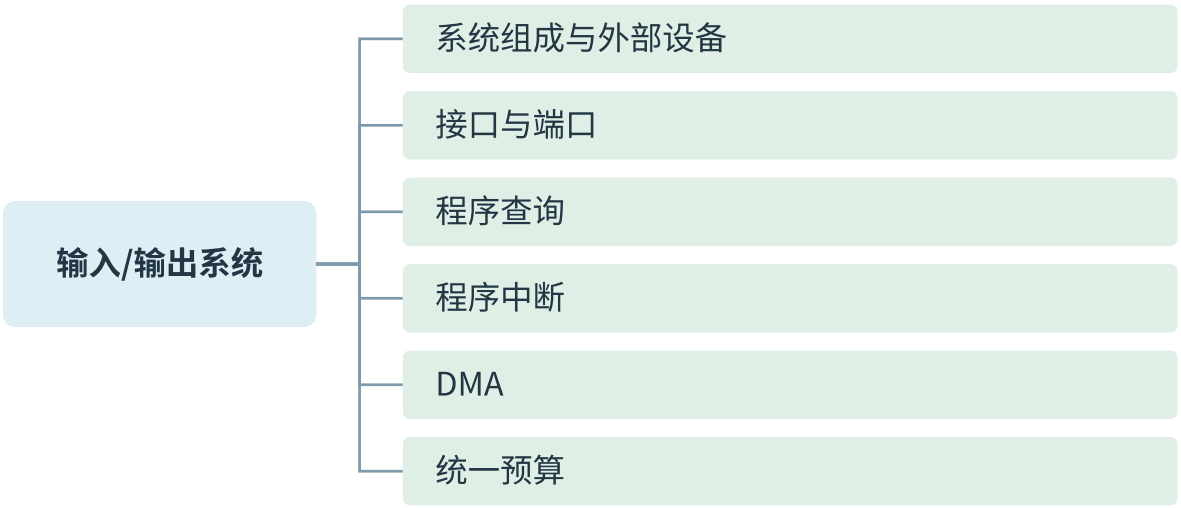
1. 地址、数据、控制三类线的分工，是否按所传信息名称里有没有 “控制” 二字决定?
2. 为什么 32 位、每拍两次的 100 MHz 总线，理想峰值是 800 MB/s，而不是 400 MB/s?
3. 给 200 MT/s 后，能否因 “每拍两次” 再乘 2?
4. 连读 8 个字时，突发方式节省的是哪类重复开销?
5. 全互锁握手四个事件的先后顺序是什么?
6. 一次读请求等待 100 ns，能否直接断言总线被独占 100 ns?
7. 仲裁与定时分别解决哪两个问题?
8. 双向合计带宽与单方向带宽相差多少，是否总能用同一个倍数转换?

6.9.1 自测答案

1. 不能；命令字、状态字等编码值可走数据线，控制线规定事务类型和时序。2. 每次 4 B，每秒 $2 \times 100 \times 10^6$ 次，乘得 800 MB/s，前提是全部数据位均有效。3. 不能，MT/s 已经计入实际传送次数。4. 首地址、启动与准备等是否只付一次，由题设确定；数据拍仍要逐项数。5. 请求置位、应答置位、请求撤销、应答撤销。6. 不能，分离事务可在设备准备时释放总线。7. 谁获得使用权；何时采样与推进。8. 需看全/半双工、两个方向是否能同时达到峰值以及题目统计口径，不可无条件翻倍。

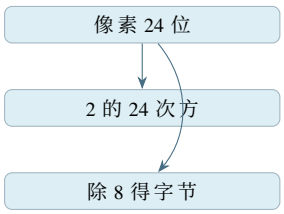
7 输入/输出系统

章节脉络



最小反例·24 位不等于 24 种颜色

每像素 24 位时最多有 2^{24} 种编码，而不是 24 种。容量计算乘 24 得到位数，再除 8 变字节；若题给的是灰度级 G，应先算编码位数 $\lceil \log_2 G \rceil$。



位数决定编码数，容量还须把 bit 换成 B

7.1 沿“谁等待、谁搬运、谁收尾”理解 I/O

课件把输入/输出系统分为硬件、软件与控制方式：设备和接口负责把物理事件变成可交换的数据；I/O 指令或驱动程序发出命令；查询、中断与 DMA 决定 CPU 何时介入、谁实际搬运。先固定设备速率、接口缓冲大小和每次服务粒度，再比较方式，才有统一的计数基础。

定义 7.1.1 (I/O 硬件、指令与通道)

I/O 硬件包括输入设备、输出设备、外存和 I/O 接口/控制器。I/O 指令是 CPU 执行、用于访问端口或控制 I/O 的机器指令；通道指令由专用通道执行，通道程序通常在主存中，CPU 启动后由通道组织相应设备工作。二者不是同一指令集的两个名称。

通道在硬件支持下承担比逐字 DMA 更完整的 I/O 组织任务，减少 CPU 的细粒度参与；它仍需软件准备、设备协议与完成通知。这里作为课件导图的背景分支保留，不把某一种历史实现推广为现代所有设备的结构。

7.2 外部设备及其缓冲需求

输入设备把用户或环境的信息送入系统，例如键盘、鼠标；输出设备把结果表现出来，例如显示器、打印机；外存提供较大容量的持久数据保存，磁盘与固态盘的具体访问机制回指存储系统一章。设备分类说明主要用途，不表示具有复合功能的设备只能归入一个绝对类别。

显示器参数包括屏幕尺寸、分辨率、每像素灰度或颜色位数、刷新频率。若每帧包含 N 个像素、每像素 b 位且无压缩，则一帧显存至少需

Nb 位，按每秒 F 帧连续刷新所需的有效像素数据率为 NbF 位/s；空白区、编码、总线等额外开销需按题设另计。字符点阵库保存每种字符的字模，帧缓冲保存当前屏幕的像素，两者统计对象不同。

例 7.2.1 (自拟：从分辨率算显存与数据率)

图像为 1920×1080 像素，每像素 24 位，每秒刷新 60 帧，不计压缩、空白区与传输编码。求单帧有效数据量、双缓冲容量及连续像素数据率。

解析。单帧为 $1920\times1080\times24/8=6220800$ B；双缓冲需要 12441600 B；数据率为 $6220800\times60=373248000$ B/s。24 位是每像素的编码位数，不是 “24 种颜色”；双缓冲使存储容量翻倍，并不要求把每秒实际输出帧数也乘 2。

打印机可按是否击打、按字符/行组织、按针式/喷墨/激光等机制分类。分类维度不同，不宜将 “非击打” 和 “行式” 作为互相排斥的并列项。接口中的缓冲可吸收设备与总线的瞬时速率差；持续输入超过长期消耗能力时，有限缓冲终会填满，不能靠 “有缓冲” 保证永不丢数据。

最小反例·同地址不等于同寄存器

某端口地址读出状态、写入命令，是由读写方向和接口译码区分的两项动作。只看数字地址相同就说状态寄存器被命令字覆盖，忽略了访问类型这一条件。

7.3 端口是接口中可访问的位置

CPU 通常通过 I/O 接口访问外部设备。接口把系统总线上的读写操作转换为设备所需的信号，并提供数据缓冲、状态记录、命令控制、地址识别及必要的中断或 DMA 连接。设备本体和接口并不是一回事：磁盘驱动器是设备，网络控制器、打印机适配器等承担接口功能。

CPU 能访问的接口寄存器或相应位置称为 I/O 端口。数据端口交换数据，状态端口报告就绪、忙或错误，控制端口接收启动、复位等命令。读状态与写控制可能共用同一个端口地址，由读写方向区分；不能因为地址相同就认为两个端口是同一个寄存器。

统一编址（存储器映射 I/O）把端口放进主存地址空间，用访存指令访问；独立编址为 I/O 设单独地址空间，通常使用专门 I/O 指令。同一个数值地址在独立编址下可分别对应内存位置和端口，在统一编址下则要避免地址译码冲突。端口地址不是设备内部磁盘扇区地址。

7.3.1 例 7-1　命令字走哪组线

【配套例题】下列选项中，在 I/O 总线的数据线上传输的信息包括（　）。

I. I/O 接口中的命令字　II. I/O 接口中的状态字　III. 中断类型号

A. 仅 I、II　B. 仅 I、III　C. 仅 II、III　D. I、II、III

解析。三者都是本次总线交换的编码值，可以由数据线承载，选 **D**。读写控制线表达 “本轮读或写” 等时序和操作性质，并不逐位承载所有名称里带 “控制” 二字的信号。

接口与设备侧交换数据、控制和设备状态，接口向 CPU 发送中断请求。打印机一般不会向其接口 “接收中断请求信号” 来获知下一个字符；这里要辨清信号的发送端与接收端。串并转换、缓冲和速度匹配也在接口或设备控制电路中完成，主机内部总线并行并不要求磁盘写入盘面的位也并行。

7.4 查询方式：等待和搬运都由 CPU 安排

不变量·最长间隔受缓冲期限限制

R B/s 填满 q B 缓冲需要 q/R 秒；定时查询必须在覆盖前读走数据，还要计入实际检查与搬运延迟。平均每秒查够次数并不自动保证某个最长间隔不会超过期限。

程序查询方式先启动设备，再读状态端口，未就绪就继续检查；就绪后由 CPU 执行指令，在数据端口与寄存器或主存之间搬运，并更新地址与计数。数据没准备好时，CPU 若一直轮询，等待时间也占用了执行机会。

```
设置设备命令、目标地址和剩余数量
while 剩余数量 > 0:
    while 状态端口表示未就绪:
        继续检查
    从数据端口读一个单位并存入目标地址
    目标地址 += 单位大小
    剩余数量 -= 1
```

定时查询则在规定时刻检查，其余时间可以执行其他工作。是否会丢数据取决于设备速率、接口缓冲容量和最长查询间隔，不能由忙等程序的表现推出“查询方式下 CPU 永远不能运行其他程序”。

7.4.1 例 7-2　定时查询占多少 CPU 时间

【配套例题】某计算机处理器主频为 50 MHz，采用定时查询方式控制设备 A 的 I/O，查询程序运行一次所用的时钟周期数至少为 500。在设备 A 工作期间，为保证数据不丢失，每秒需对其查询至少 200 次，则 CPU 用于设备 A 的 I/O 的时间占整个 CPU 时间的百分比至少是（　）。

A. 0.02%　　B. 0.05%　　C. 0.20%　　D. 0.50%

解析。每秒至少执行 $500 \times 200 = 100000$ 个查询周期，CPU 每秒有 50×10^6 个周期，比例为 0.002，即 0.20%，选 **C**。200 次/s 只决定查询次数，不是设备每秒搬运 200 B；题目也未要求把每次查询的 500 周期再当作 500 条指令。

若另给每个数据的搬运周期数，还要判断它已包含在查询程序开销中，还是必须另加。计算 CPU 占用率时，用 CPU 实际花掉的时间作分子，设备在外部独自准备数据的时间不直接算作 CPU 执行时间。

7.5 中断方式：先请求，再响应，再运行服务程序

中断 I/O 允许 CPU 启动设备后继续做其他工作，设备或接口就绪后提出中断请求。CPU 在允许响应的边界检查未屏蔽请求，完成硬件响应，再执行中断服务程序，由程序读取或写入数据端口。等待不再要求 CPU 持续轮询，但每次软件搬运数据仍要 CPU 执行指令。

请求需要保存在接口或中断控制器的触发器中，因为外设产生事件时 CPU 未必能立即响应。中断请求的产生、被允许响应、被调度执行以及处理完成是四个不同的时间点。CPU 可能正在执行指令，处于关中断状态，或已有更高优先级请求需要先处理。

7.5.1 硬件保存断点，与软件保护现场分开

典型硬件响应包括暂时禁止相应中断、保存返回位置及规定的状态、确定中断服务入口。中断向量是用于定位处理程序的信息，中断向量地址是保存向量的位置，处理程序入口地址是最终执行代码的位置。三者在某种实现中存在固定联系，但并不总是同一数值。

服务程序进入后保存其将修改且须恢复的通用寄存器，处理事件，再恢复并返回。单重中断处理期间不允许同类可屏蔽中断嵌套；多重中断则在保护必要现场、设置屏蔽后重新允许某些请求打断当前服务程序。入口处的现场保护未完成就开放嵌套，可能覆盖尚未保存的返回信息。

在教材采用的一种单重中断序列中，软件部分是“保护现场 → 事件处理 → 恢复现场 → 开中断 → 中断返回”，保存断点和初始关中断已经由硬件响应完成。实际 ISA 可能由中断返回指令原子恢复状态和允许位；不能把教材序列中的“开中断”和返回之间可被中断的窗口当作所有机器的共同实现。

不变量·每层恢复自己的屏蔽字

嵌套服务像调用栈逐层返回。C 退出应先恢复 B 的屏蔽状态，B 再恢复 A；直接恢复最外层全开放状态可能允许原本不该打断 B 的请求，破坏题设处理关系。

7.5.2 屏蔽改变允许谁打断谁

中断响应优先级通常由硬件仲裁关系决定，多个未屏蔽请求同时到来时据此选一个。处理优先级描述服务过程中允许哪些请求打断当前服务，可由各服务程序设置的屏蔽字确定。屏蔽会改变参与本次仲裁的集合，不会把仲裁电路本身的固定比较顺序改写。

7.5.3 例 7-3　响应顺序与完成顺序为什么可以不同

原创补充

有 A、B、C 三个请求源，硬件响应优先级 A>B>C。屏蔽字按 A、B、C 排列，1 表示屏蔽。要求处理优先级 C>B>A，服务程序均在保存现场并设置屏蔽字后开中断。给出三者的屏蔽字，并追踪三请求同时待处理时的一次嵌套过程。

解析。 A 处理中应屏蔽 A 自己，允许 B、C，故为 100；B 处理中屏蔽 A、B，允许 C，故为 110；C 处理中三者都屏蔽，故为 111。初始三者均允许响应时，硬件先选 A，不能因希望 C 处理优先就把首次响应写成 C。

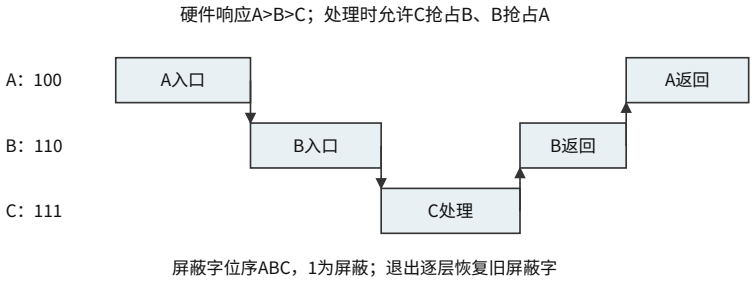


Figure 7.1: 屏蔽只改变可参与响应的集合；嵌套返回按栈次序进行

1. A 保护现场、设 100、开中断，B 和 C 仍待处理，硬件在二者中选 B。
2. B 保护现场、设 110、开中断，C 可以响应，转入 C。
3. C 设 111 完成处理，恢复 B 的状态和屏蔽字，返回 B。
4. B 完成后恢复 A，A 最后完成。服务开始顺序 A、B、C，完成顺序 C、B、A。

题目要的是给定规则下的模型。各层必须保存并恢复旧屏蔽字，不能在 C 退出时直接恢复成最外层的全开放状态。屏蔽字的位序和 0/1 含义也必须从题干确定。

7.5.4 例 7-4　允许延迟不是额外的服务开销

错项排除·块完成一次，单位传送多次

1024 B 块、4 B 接口缓冲可分 256 个数据单位搬运，而正常完成通知可仅一次。DMA 减少 CPU 事件数，不要求把整块压成一个总线数据拍；突发是否合并事务还需题设。

【配套例题】若某设备中断请求的响应和处理时间为 100 ns，每 400 ns 发出一次中断请求，中断响应所允许的最长延迟时间为 50 ns，则在该设备持续工作过程中，CPU 用于该设备的 I/O 时间占整个 CPU 时间的百分比至少是（　）。

A. 12.5%　　B. 25%　　C. 37.5%　　D. 50%

解析。每个请求实际占 CPU100 ns，请求间隔 400 ns，长期占用比例为 $100/400 = 25\%$，选 **B**。最长响应延迟 50 ns 约束请求产生后必须多快开始响应，不是额外要执行 50 ns 的服务程序，不能加到 100 ns 上。

若题给设备数据率 R B/s、每次中断交换 q B、每次 CPU 开销 t 秒，在每次均交换 q B 且持续工作的模型中，请求率为 R/q，CPU 比例为 $(R/q)t$。若结果超过 100%，说明该模型下 CPU 无法及时服务；不能把它继续当作可达到的利用率。

7.6 DMA：数据不经过 CPU 寄存器逐次搬运

DMA 控制器在设备接口与主存之间组织直接传送。CPU 仍需通过驱动程序设置方向、主存起始地址、传送数量和设备等参数，启动设备；传送过程中由控制器形成主存地址、请求总线、搬运数据并更新计数；结束或出错后再通知 CPU 处理。DMA 减少逐个数据的指令开销，不表示 CPU 从头到尾完全不参与。

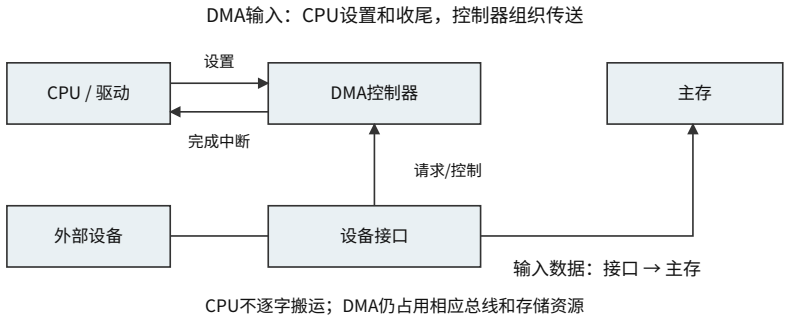


Figure 7.2: CPU 设置和收尾，DMA 传输阶段在接口与主存之间搬运

暂停 CPU 方式可在一块数据传送期间集中占用相关总线；周期挪用方式在设备准备好一个传送单位后申请一次或若干次总线使用；交替方式按约定把周期分给 CPU 和 DMA。CPU 能否同时访问某个存储体、Cache 或另一条总线，要由具体资源结构决定。只要 CPU 本拍不需要被占用的资源，它仍可能继续内部运算。

周期挪用中，设备准备数据后向 DMA 控制器请求服务，DMA 控制器再向系统申请总线。教材说 “DMA 请求” 时可能指设备侧请求，也可能指总线申请，解题要看请求的发送者和接收者，不能只认四个字母。

DMA 控制器通常包含主存地址计数器、传送长度计数器、数据缓冲寄存器、请求触发器、控制/状态逻辑以及完成中断机构。地址计数器决定下一次主存位置，长度计数器判断块是否完成；缓冲容量 q B 决定一次可交换的数据粒度，整块长度 Q B 决定 CPU 何时收到完成通知。两种 “计数” 不能交换使用。

先统一事件粒度，再列预算

设设备连续速率为 R B/s，接口每次可交换 q B，每次 CPU 服务耗时 t 秒，则理想事件率为 R/q 次/s，CPU 开销比例为 $(R/q)t$。若 DMA 每块 Q B 才完成一次且每块软件开销为 u 秒，则 CPU 开销比例为 $(R/Q)u$。另以总线每次占用 b 秒列总线占用 $(R/q)b$；这第三个式子不自动等于 CPU 开销。

课程连接·CPU 预算与总线预算分列

每块软件预处理/后处理影响 CPU 比例，DMA 实际访存影响总线和主存占用。CPU 可能在其私有 Cache 中继续运算，所以总线忙碌比例不能无条件等同 CPU 损失比例。

这些式子都先假定事件持续、粒度固定且无额外未计开销；若比值超过 1，说明所需服务能力超过预算。缓冲是否溢出还要核最长响应时间：平均 CPU 开销低于 100% 不是实时期限一定满足的充分条件。

7.6.1 例 7-5　周期挪用不等于整块期间封住主存

【配套例题】若设备采用周期挪用 DMA 方式进行输入和输出，每次 DMA 传送的数据块大小为 512 字节，相应的 I/O 接口中有一个 32 位数据缓冲寄存器。对于数据输入过程，下列叙述中，错误的是（　）。

A. 每准备好 32 位数据，DMA 控制器就发出一次总线请求
B. 相对于 CPU，DMA 控制器的总线使用权的优先级更高
C. 在整个数据块的传送过程中，CPU 不可以访问主存储器
D. 数据块传送结束时，会产生 “DMA 传送结束” 中断请求

解析。每次缓冲 32 位即 4 B，512 B 需要 128 次传送。周期挪用只在取得总线的时段占用资源，其余时段 CPU 仍可访问主存，不是整块传完前一直封锁，所以选 **C**。B 描述本题中 DMA 优先取得总线的模型，不表示任何系统的任意外部请求都永久压过 CPU。

一次总线占用时间为 b、每次搬 q B 时，DMA 数据率 R 对应的总线占用比例约为 $(R/q)b$，前提是这些占用不重叠且没有额外启动代价。CPU 实际损失还取决于它原本是否需要总线，因此不能把 “DMA 占总线 10%” 无条件改写成 “程序变慢 10%”。

7.6.2 例 7-6　DMA 的准备与完成仍需要软件

【配套例题】下列关于 DMA 方式的叙述中，正确的是（　）。

I. DMA 传送前由设备驱动程序设置传送参数
II. 数据传送前由 DMA 控制器请求总线使用权
III. 数据传送由 DMA 控制器直接控制总线完成
IV. DMA 传送结束后的处理由中断服务程序完成

A. 仅 I、II　B. 仅 I、III、IV　C. 仅 II、III、IV　D. I、II、III、IV

解析。I 是 CPU 上的驱动程序做传送前准备，II、III 是 DMA 硬件实际传输，IV 是完成通知后的软件收尾，四项均正确，选 **D**。这条链把 “CPU 不逐字搬运” 和 “CPU 完全不参与” 清楚区分开。

方式	CPU 介入时机与粒度	数据搬运主体	主要约束
忙等查询	持续检查，逐单位搬运	CPU 指令	等待和搬运均占 CPU 执行机会
定时查询	定期检查，每次查询及搬运	CPU 指令	查询间隔与服务时长须满足缓冲期限
中断 I/O	每请求被允许响应后，交换 q B	CPU 服务程序	请求率 × 每次响应处理时间
DMA	每块 Q B 前设置、结束后收尾	DMA 控制器	CPU 开销、总线与主存占用分别计算

适用性取决于设备速率、传送粒度、缓冲和系统实现。字符型低速设备通常适合查询或中断，网络和块设备的大批量传送常用 DMA；中断方式并非适用于所有设备，DMA 也没有固定的速度门槛。

比较三种方式时，可先固定同一设备数据率 R。若接口一次仅能容纳 q B，则数据积满时间为 q/R；定时查询的查询间隔要小于这个时间，中断方式每秒约响应 R/q 次。若 DMA 每块 Q B 才让 CPU 收尾一次，CPU 事件率变成 R/Q；但控制器可能仍按 q B 多次请求总线，不能把 “每块一次完成中断” 误作 “整块只有一次总线传送”。CPU 事件数、设备准备次数和总线事务数是三个不同计数。

最小反例·平均低占用也会超期限

每秒总共只需 1ms 服务，看似占用很低；若某次数据必须在 10 微秒内取走，而 CPU 最长关中断 100 微秒，仍可能丢数据。平均预算与最坏响应延迟必须分别核对。

7.6.3 例 7-7　同一接口缓冲下比较查询、中断和 DMA 预算

【配套例题】假定计算机的主频为 500 MHz，CPI 为 4。现有设备 A 和 B，其数据传输率分别为 2 MB/s 和 40 MB/s，对应 I/O 接口中各有一个 32 位数据缓冲寄存器。请回答下列问题，要求给出计算过程。

（1）若设备 A 采用定时查询 I/O 方式，每次输入/输出都至少执行 10 条指令。设备 A 最多间隔多长时间查询一次才能不丢失数据？CPU 用于设备 A 输入/输出的时间占 CPU 总时间的百分比至少是多少？

（2）在中断 I/O 方式下，若每次中断响应和中断处理的总时钟周期数至少为 400，则设备 B 能否采用中断 I/O 方式？为什么？

（3）若设备 B 采用 DMA 方式，每次 DMA 传送的数据块大小为 1000 B，CPU 用于 DMA 预处理和后处理的总时钟周期数为 500，则 CPU 用于设备 B 输入/输出的时间占 CPU 总时间的百分比最多是多少？

解析。数据率中的 M 按 10^6 计，缓冲 32 位 =4 B，CPU 每秒有 500×10^6 个时钟周期。先分别求设备填满缓冲的时间、CPU 每次服务时间，再求长期比例，不把设备工作时间直接当作 CPU 忙碌时间。

（1）A 每准备 4 B 需要

$$\Delta t_A=\frac{4}{2\times10^6}=2\ \mu\text{s}.$$

因此最多每 2 μs 就必须完成一次查询与取数；间隔更长，单个缓冲可能被下一组数据覆盖。每秒至少服务 $2\times10^6/4=5\times10^5$ 次；每次至少 10×4=40 周期，故 CPU 占用比例至少为

$$\frac{(5\times10^5)\times40}{500\times10^6}=4\%.$$

每次这项服务最少耗时 80 ns，2 μs 是设备产生一缓冲数据的间隔，不是 CPU 每次都要执行 2 μs。

（2）B 每 4 B 只允许 $4/(40\times10^6)=0.1\ \mu$s，相当于 50 个 CPU 周期；一次中断响应与处理却至少需 400 周期 =0.8 μs。每次服务还未完成，后续数据已连续到来，不能持续及时处理。按请求率核对，所需 CPU 比例为

$$\frac{(40\times10^6/4)\times400}{500\times10^6}=8=800\%.$$

这表示所需服务能力超过 CPU 预算，不能解释成实际 CPU 利用率能达到 800%。给定单缓冲与逐次中断模型下，设备 B 不适合这种中断 I/O 方式。

（3）DMA 把 CPU 事件粒度改成 1000 B 一块。以设备满速计，每秒最多 $40\times10^6/1000=40000$ 块；每块 CPU 预处理和后处理 500 周期，

所以 CPU 开销比例最多为

$$\frac{40000 \times 500}{500 \times 10^{6}} = 4\%.$$

课程连接·准备数据和传送数据

设备准备一批数据的时间与总线传送该批数据的时间各自计算；是否能重叠取决于设备与接口机制。

一块设备传送时间为 25 μs，CPU 设置收尾共 1 μs，二者相除也得 4%。这一项没有再乘 CPI，因为题干已直接给时钟周期数。第（1）问的 4% 是由 “至少 10 条指令” 得到下界，本问的 4% 按最大设备速率得到上界；数值相同，含义不同。

DMA 的 4% 统计 CPU 设置收尾，并未把 DMA 占用总线的时间当作 CPU 执行时间。若另问整机吞吐是否可达 40 MB/s，还须核对总线与主存带宽；若另问 CPU 程序因此慢多少，还要看 CPU 同时是否需要这些资源。第 6 章的总线预算与本题 CPU 预算应分别算，再按共享资源关系连接。

7.6.4 配套辨析巩固

1. 【配套例题】下列关于中断 I/O 方式的叙述中，错误的是（ ）。A. 中断屏蔽字用于确定中断响应的优先级；B. 保存断点和程序状态字在中断响应阶段完成；C. 保存通用寄存器和设置新中断屏蔽字由软件实现；D. 单重中断方式下中断处理时 CPU 处于关中断状态。
2. 【配套例题】DMA 控制 I/O 方式下，设备的输入/输出由 DMA 控制器控制完成，此时 DMA 控制器控制的数据传输通路位于（ ）。A. CPU 和主存之间；B. CPU 和 DMA 控制器之间；C. 设备接口和主存之间；D. 设备接口和 DMA 控制器之间。
3. 原创补充：设备数据率为 1 MB/s，每次中断交换 4 B，开销为 2 μs，CPU 占用多少？若改为每 1024 B 一块 DMA，假设 CPU 每块仅需 20 μs 设置及收尾，CPU 开销比例是多少？暂不计总线竞争。

答案。 1. A。按本章教材口径，屏蔽字规定允许的抢占关系和处理优先级，固定的响应仲裁次序不能由它直接改写。2. C。3. 中断为 $(10^{6}/4) \times 2 \times 10^{-6} = 50\%$；DMA 为 $(10^{6}/1024) \times 20 \times 10^{-6} \approx 1.953\%$。后者只是 CPU 设置收尾开销，尚不包含 DMA 占用主存总线对程序造成的影响。

7.7 外部设备怎样形成 I/O 请求

输入设备把外界信息转换为数字编码，例如键盘扫描按键并形成按键编码，鼠标把位移和按键状态转换为报告，扫描仪和摄像设备把光学信息采样编码。输出设备把编码转换为显示、声音或纸面记录；显示帧缓冲保存像素数据，显示控制器按时序读取并驱动显示。

若分辨率为 $W \times H$、每像素 b 位、刷新率 f 帧/s，仅有效像素的单帧容量为 $WHb/8$ B，最低像素载荷数据率为 $WHbf/8$ B/s。实际接口还可能包含消隐、包头、编码和压缩等因素，题目给出额外条件时需分别计入。

针式打印利用撞针、色带形成字符，适合多联复写；喷墨通过喷出墨滴成像，彩色及介质要求依实现而定；激光打印利用感光鼓成像、转印和定影。外设的机械或介质工作速度通常与 CPU 不同，接口中的数据缓冲和状态控制用于协调这种差异。

7.8 I/O 软件、接口与端口的分工

课程连接·端口与接口

端口是 CPU 访问接口寄存器的地址入口，接口提供设备状态、控制和缓冲等功能。

I/O 软件包括驱动程序及上层管理程序，I/O 硬件包括设备、接口及连接通路。接口承担设备选择、命令译码、数据缓冲、状态记录、格式和速度匹配，并可发出中断或 DMA 请求。端口是 CPU 可读写的接口寄存器或相应地址位置，接口通常含多个端口。

数据端口保存待传或已接收数据，状态端口报告就绪、忙、错误等状态，控制端口接收命令或工作方式。控制命令字和状态字都可能经数据总线传送；控制总线上的读写信号用于说明这次总线事务的性质，两种 “控制信息” 处于不同层次。

统一编址把 I/O 寄存器映射到与主存同一地址空间，用普通访存指令访问；独立编址设置单独 I/O 地址空间并用专门 I/O 指令访问。统一编址需要通过地址译码区分内存和设备区域；独立编址需要区分存储器事务和 I/O 事务的控制规则。

通道是一种可执行通道程序的专用 I/O 处理部件。CPU 发出启动命令，通道按通道指令组织一系列 I/O 操作；DMA 控制器主要负责设定数据块的传送，通道可承担更复杂的控制组织。它们都能减少 CPU 逐字搬运的工作量，实际能力取决于具体系统。

7.9 查询方式的可行性先于占用率

独占查询中，CPU 不断检查状态并等待，外设准备期间也不能转做普通程序；定时查询在查询间隔可执行其他程序，但必须保证缓冲区在下次查询前不会溢出。若设备每 t_d 产生一项、缓冲只能容纳一项，则查询间隔应满足及时取走的要求；存在多项缓冲时还要考虑连续到达及服务时间。

设 CPU 主频 50 MHz，一次查询需 100 周期。鼠标每秒查询 30 次，占用率为 $30 \times 100/(50 \times 10^6) = 0.006\%$。若磁盘每秒输入 2 MiB、每次查询传 4 B，则每秒需 2^{19} 次查询，占用周期 100×2^{19}，占比约 104.86%。这说明该查询组织即使耗尽 CPU 时间也来不及处理，不能把超过 100% 的值解释为实际可达到的 CPU 占用状态。

7.10 中断判优、屏蔽和现场恢复

中断请求标记用于记住待处理事件；判优电路在多个请求同时存在时选择响应对象。响应优先级是硬件在当前时刻选谁；处理优先级决定哪个服务程序允许被哪个中断打断，可通过屏蔽字组织。二者可以不同。

课件约定屏蔽位 1 表示屏蔽、0 表示允许；必须在例题中先写清位序和电平语义。某服务程序运行时通常屏蔽本级及不允许打断它的源，只允许规定的更高处理优先级进入；进入服务程序时保存旧屏蔽字，返回前恢复。屏蔽字改变的是请求可见性，不会抹掉已经发生的请求事件。

多重中断的一般顺序为：硬件关中断、保存断点并进入服务入口；软件保护现场和旧屏蔽字，装入新屏蔽字；在安全位置开中断，运行服务主体；结束时关中断，恢复屏蔽字和现场，再按返回规则恢复执行。若过早开放中断，尚未保存的状态可能被下一层覆盖。

非屏蔽中断与可屏蔽中断的区别在于是否受普通中断允许控制约束。非屏蔽不表示任何时刻、任何指令内部都立即任意打断；响应时点和保存规则由体系结构规定。

7.10.1 例：设备准备与服务程序怎样重叠

课程连接·DMA 仍需控制信息

DMA 控制器需知道内存地址、传送计数、方向和设备状态；数据传送期间 CPU 与 DMA 如何争用总线须依题设。

CPU 主频 50 MHz，CPI 为 4；设备每 0.5 ms 准备一个字符，每字符含 1 位起始、7 位数据、1 位奇校验及 1 位停止，共 10 位。设备持续工作时最多每秒形成 2000 字符。

中断响应需 10 周期，服务程序 20 条指令，第 15 条启动设备准备下一字符。按课件的稳定重复近似，每字符完成间隔为 $25000 + 10 + 15 \times 4 = 25070$ 周期，因为最后 5 条服务指令与下一字符准备重叠。1000 字符总历时约 25070000 周期；CPU 实际用于响应和服务的时间为 $1000(10 + 20 \times 4) = 90000$ 周期。

若从第一次启动设备准备第 1 个字符开始计时，到第 1000 次中断服务程序完全结束，且各次均采用上述理想时序，则最后 5 条服务指令没有下一字符的准备过程可供重叠，精确总历时为 $1000 \times 25070 + 5 \times 4 = 25070020$ 周期。这里 “约” 表示采用课件的稳定重复周期近似；题目改变起止事件时，应按新端点补计启动或排空时间。

7.11 DMA 控制器的寄存器和传送阶段

DMA 控制器通常包含主存地址寄存器、传送字数计数器、数据缓冲寄存器、控制与状态逻辑以及设备相关地址信息。CPU 预处理设置方向、主存起始地址、长度和设备参数并启动设备；数据准备好后，DMA 申请总线，在授权后直接在设备和主存之间搬运，自动更新地址和计数；完成或出错后通知 CPU 后处理。

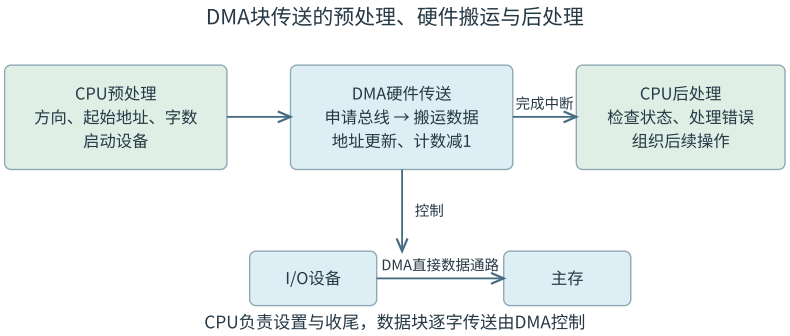


Figure 7.3: CPU 设置传送参数，DMA 逐次获得总线传数据，块结束后请求 CPU 处理

停止 CPU 访存的成组传送在一段时间内集中占用总线，控制简单但 CPU 访存受阻；周期挪用在需要时借用一个或若干存储周期，CPU 正在访存时通常先让该次访问完成；交替访存按固定时隙分配 CPU 和 DMA，减少重复仲裁但增加时序组织要求。

DMA 请求是获取总线或主存访问机会，中断请求是要求 CPU 改变执行流去运行服务程序。DMA 传数据不逐字执行 CPU 搬运指令，块完成通知仍可采用中断。DMA 占用总线时，CPU 若能在寄存器或 Cache 中继续执行某些操作，并不一定完全停止；是否发生停顿须看具体访问和总线关系。

7.11.1 计算与辨析自查

1. 写控制端口时，命令字为什么仍可走数据总线？ 2. 查询占用率算出 105% 意味着什么？ 3. 中断响应优先级和处理优先级分别决定什么？ 4. DMA 块传送前后 CPU 还要做什么？

答案。 1. 命令字是本次事务传送的数据，读写性质由控制线及地址共同说明。2. 该方案处理能力不足。3. 前者选择同时请求时先响应谁，后者限定服务程序间的嵌套关系。4. 预处理设置参数并启动；后处理检查结果、处理错误和唤醒后续工作。

课程连接·先列时间线

分别标设备准备、CPU 服务、总线传送的开始与完成时间，再判断是否可以重叠。

7.12 主动回忆自测

以下为自拟检查。请沿 “设备—接口—控制方式—预算” 回忆，不先看答案。

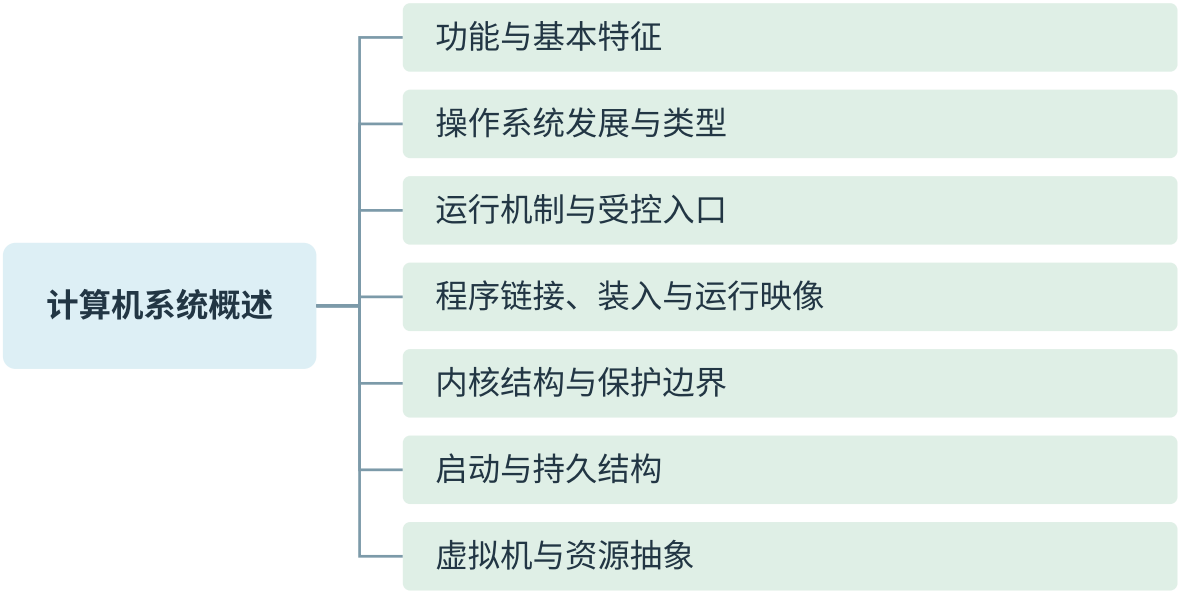
1. I/O 端口、接口、设备和磁盘扇区地址之间分别是什么关系?
2. 统一编址与独立编址，改变的是地址空间与指令中的哪两项约定?
3. 为什么定时查询允许 CPU 在两次查询间执行其他程序，而忙等查询会持续占用 CPU?
4. 硬件中断响应与软件服务程序分别保存哪些类型的信息？是否所有 ISA 完全相同?
5. 已知响应优先级 A>B>C，能否只据此确定三者最终完成顺序?
6. DMA 中，块长 1024 B、接口缓冲 4 B，块内可能有多少次数据单位传送、多少次块完成通知?
7. 显存容量和刷新数据率分别多乘了哪一个参数？双缓冲为何不能直接让输出帧频翻倍?
8. 设备每秒 1 MB、每次中断 4 B、每次开销 2 微秒，改为每 1024 B 一块、每块软件开销 20 微秒的 DMA，CPU 预算分别是多少?

7.12.1 自测答案

1. 端口是 CPU 可访问的接口位置；接口连接总线与设备；扇区地址标识设备内部数据位置，不能与 CPU 端口地址混同。2. 统一编址共享存储器地址空间并用访存指令；独立编址有独立端口空间，通常用专门 I/O 指令。3. 前者在期限允许下间歇检查，后者循环执行检查；均仍需安排实际搬运。4. 典型硬件保存断点与规定状态，软件保存其需要恢复的通用寄存器等；具体保存集合和开中断/返回时序依 ISA。5. 不能，还需屏蔽规则、允许嵌套关系、请求到达时间与服务时长。6. 可有 256 次 4 B 单位传送，正常块完成通常一次通知；总线突发合并会改变事务数，须看具体控制器。7. 有效容量为像素数 $\times$ 位数，刷新有效速率再乘帧频；双缓冲多存一帧，不自动多输出一帧。8. 中断 50%；DMA 约 1.953%，均仅按题定 CPU 执行开销，另核总线与实时期限。

计算机系统概述 1

章节脉络



课程连接·运行环境

硬件资源由内核管理；用户程序通过受控入口申请服务，权限级与地址空间提供不同保护。

1.1 资源、权限与控制流

概念辨析的四步路径

1. 先确定对象：是硬件资源、内核结构、用户程序，还是磁盘上的持久数据？
2. 再确定权限：操作能否影响其他进程或全局保护状态，是否需要受控入口？
3. 沿一次执行追踪：普通调用、陷入、异常响应、软件处理、返回，各由谁完成？
4. 最后检查条件：并发不自动推出并行，模式切换不自动推出进程切换，结构收益也要说明代价。

定义 1.1.1 (用资源与接口理解操作系统)

操作系统对处理器、内存、文件和设备实行受保护的管理，并向应用提供服务。学习时同时标出“管理对象”“允许谁操作”“通过哪个入口请求”；基本特征和内核结构都围绕这三件事展开。

1.1.1 受控入口为何不能由用户代码自行授予权限

从保护目标推出入口约束

假设用户程序能先用普通赋值把模式位改为内核态，再调用服务，那么它也能跳过服务的参数检查，直接关闭时钟中断或改写别的进程映射，资源隔离立即失效。因此权限变化必须由硬件按预定事件与入口

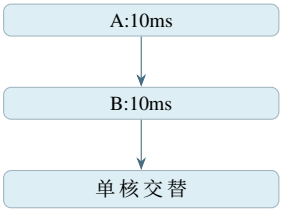
规则执行；内核取得控制权以后再检查请求。用户可以发起请求，并不等于用户能决定入口与授权结果。

□

程序执行一条普通加法指令时，通常直接使用处理器；读取文件时，却不能自己任意驱动磁盘。两种操作的差别来自资源保护：处理器、内存和设备由多个程序共享，操作系统必须管理它们，并给应用提供受控的使用接口。本章沿着一个程序的运行过程，梳理用户态、内核态、异常、系统调用和地址空间之间的关系。

最小反例·单核也有并发

一个 CPU 先运行 A 十毫秒，再运行 B 十毫秒，二者在同一时间段交替推进，已满足并发，却没有在同一时刻执行两条用户指令。把“系统有多个任务”直接推出“任务必并行”，就混淆了时间区间与时间点。



同一时间段交替推进是并发，不推出同一时刻指令并行。

1.2 操作系统管理什么，向程序提供什么

操作系统是控制和管理计算机硬件、软件资源，组织程序运行，并向用户和应用程序提供接口的系统软件。这里的“管理”要落到具体对象上：处理机管理决定谁运行；内存管理分配空间并隔离访问；文件管理组织持久数据；设备管理协调输入输出，屏蔽设备差异。

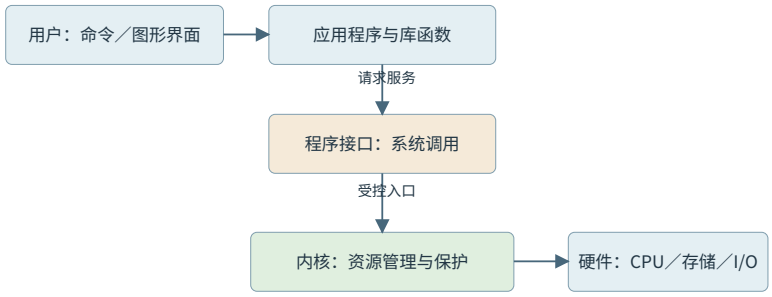
面向用户的命令行、图形界面便于发起操作；面向应用程序的系统调用接口允许程序请求内核服务。图形界面不是所有操作系统都必须具有的组成部分。编译器虽然也是系统软件，但把高级语言翻译成目标程序并不是操作系统的基本职责。

〖1.1-003〗下列哪项不是操作系统关心的基本问题：管理裸机、提供用户程序与硬件的接口、管理系统资源、提供高级语言编译器？答案是最后一项。判断依据是该功能是否承担资源管理与运行控制，不能只看它是否属于“系统软件”。

1.2.1 用户接口：联机命令、脱机命令与程序接口

操作系统向上提供的接口要区分使用者和使用方式。**联机命令接口**又称交互式命令接口，用户输入一条命令，系统解释并执行，再接收下一条；**脱机命令接口**又称批处理命令接口，用户预先组织一组作业控制命令，系统按约定顺序处理，执行期间通常不需要用户逐条干预。两者的“联机／脱机”描述的是命令提交与交互方式，不直接等同于有没有网络连接。

程序接口由一组系统调用构成，供程序请求服务。用户在文件管理器中点击“删除”属于使用图形用户界面，应用程序随后可能通过系统调用完成文件操作。图形界面、命令解释器、库函数和系统调用分别处在不同层次；把图形界面的一次点击直接视为 CPU 执行一条机器指令，会漏掉应用处理、库封装和内核服务。



命令接口、库函数、系统调用与机器指令属于不同层次。

Figure 1.1: 用户通过命令或图形界面使用应用，应用通过程序接口请求内核服务

概念辨析·脱机并不等于离线网络

联机与脱机命令接口按交互提交方式区分。批处理命令文件可以由联网计算机执行，分类依据仍是预先组织命令后批量运行。

操作系统对硬件进行组织与抽象，把原本不便直接使用的裸机扩充为便于使用的机器。因此“扩充机器”强调增加服务与抽象，并不表示凭空增加 CPU、内存或磁盘的物理容量。硬件资源仍有限，资源管理目标还包括安全、有效和公平地组织使用。

例如应用中的 `printf` 先进行格式转换，输出路径可能再调用系统服务；计算绝对值的库函数可以只执行用户态指令。判断接口类型时，应沿实际执行路径辨认，而不是依据函数名长短或是否属于标准库。

1.2.2 并发、共享、虚拟与异步怎样联系

并发指多个活动在同一时间间隔内推进；**并行**指多个活动在同一时刻执行。一个 CPU 交替执行两个进程，已有并发；多个核上分别执行，才有处理器层面的并行。单 CPU 的计算与设备的数据传输仍可能并行。

共享指资源供多个并发活动共同使用。打印机一类资源通常需要互斥使用；磁盘文件或内存中只读代码可按规定共享。所谓“同时共享”不等于所有硬件操作都在同一瞬间发生，而是允许多个活动在同一时期使用同一资源。

虚拟把有限的物理资源抽象为若干逻辑资源。例如时间片让每个用户似乎独占一个处理器，虚拟内存让进程使用独立的地址空间。**异步**指进程因竞争处理器、等待 I/O 等原因，以不可预先确定的速度向前推进。它不表示进程内部可以违背程序依赖，也不等同于接口中的“异步 I/O”。

并发活动会竞争资源，因而共享必须伴随保护；虚拟化为并发活动提供合适的使用视图；资源等待又造成执行速度的不确定。后续同步与调度问题都以这些关系为前提。

1.2.3 例 1-1　进程越多，处理器利用率一定越高吗

例 1.2.1 (配套例题)

【配套例题】下列关于多道程序系统的叙述中，不正确的是（　）。

A. 支持进程的并发执行　　B. 不必支持虚拟存储管理

C. 需要实现对共享资源的管理　　D. 进程数越多 CPU 利用率越高

解析。多道程序设计的直接收益是：一个进程等待 I/O 时，CPU 可

以运行其他就绪进程。但这一收益有前提：可运行进程过多会增加调度开销和内存压力，严重时频繁换页，CPU 反而难以有效执行用户计算。因此 D 把条件性收益写成了单调关系，答案为 **D**。多道程序可以使用不带请求分页的内存方案，B 正确；“多道” 不等于 “必须有虚拟内存”。

错项排除·实时性的判据是期限

控制任务必须在五毫秒内响应，即使某机器多数时候只用一毫秒，偶尔延迟十毫秒也可能违反硬实时要求。排除 “实时系统就是运行速度最快的系统” 时，应指出缺少最坏响应与截止期限这一保证。

1.3 批处理、分时与实时解决不同的问题

早期人工装入程序时，CPU 常等待操作者准备下一项作业。单道批处理把多个作业预先排好，自动逐个执行，减少人工干预；但一个作业等待 I/O 时，处理器仍可能空闲。多道批处理允许多项作业同时在内存中，利用 CPU 与 I/O 的重叠提高吞吐量。中断使设备完成时能够通知 CPU，存储保护、调度和资源管理使多道运行可控。

分时系统把处理器时间分成时间片，轮流服务交互用户，主要关注响应时间和交互性。一个时间片并不等于某个程序的全部运行时间；未运行完的程序会在以后继续执行。**实时系统**关注任务能否在规定期限内得到响应或完成。硬实时任务错过截止时间便不能满足系统要求；软实时允许一定程度的超期，但质量会下降。实时性不能只解释成 “运行得快”：提前 1 秒完成与晚于期限 1 毫秒完成，两者的结果差异可能与绝对执行速度无关。

类型	主要组织方式	评估时首先关心什么
多道批处理	作业成批提交，运行中较少交互	吞吐量、周转时间、资源利用率
分时	多用户交互，时间片轮转	响应时间、使用公平性
实时	按期限和任务紧迫性组织执行	截止时间、响应的可预测性

网络操作系统支持联网计算机上的资源与通信服务；分布式系统进一步组织多台计算机协作，可能提供统一资源视图。单用户、多用户、单任务、多任务则是另外的分类维度，不能把 “单用户” 自动理解为 “单任务”。

1.3.1 从人工操作到多道程序：CPU 为什么会空闲

人工操作阶段，装入纸带、检查结果和准备下一作业需要人参与，CPU 常等待外部准备。脱机输入／输出把部分慢速操作交给外围设备预先处理，减少主机等待人工操作的时间；单道批处理进一步由监督程序自动安排作业，但内存中仍只有一道作业可运行，该作业等待 I/O 时，CPU 缺少其他可切换的工作。

多道程序设计把多道作业同时放在内存中：某道作业等待设备时，另一道就绪作业可以使用 CPU。这里有两种不同的重叠：CPU 与 I/O 设备可以物理并行；同一 CPU 上的多道程序交替执行形成并发。收益来自填补空闲时间，而系统同时承担处理机分配、存储保护、设备协调等额外管理工作。

若只有进程 A，A 依次计算 2 ms、输入 4 ms、计算 2 ms，忽略系统开销，则完成时间 8 ms，CPU 有效计算 4 ms，利用率为 50%。若在 A 等待输入时运行另一个就绪进程 B 的 4 ms 计算段，时间线可成为 A 计算 0～2、A 输入与 B 计算 2～6、A 计算 6～8；总历时仍为 8 ms，却完成 8 ms 计算，CPU 利用率达到 100%。这是根据上述机制构造的示例，成立前提是设备与 CPU 可并行、B 始终就绪且忽略切换开销。

批处理主要追求吞吐量与周转；分时进一步强调交互、独立使用感和及时响应。实时系统还必须依据任务期限安排执行，硬实时与软实时

在能否容忍超期方面不同。几类系统可在现代实现中兼具若干特征，分类是在比较主要设计目标。

最小反例·内核也要做普通加法

内核计算页框号或更新计数器时，会执行普通算术指令。它们在用户态通常也合法，所以“内核态执行的都是特权指令”不成立。限制方向是特权操作要求高权限，不能把这个蕴含关系反过来。

1.4 用户态、内核态与特权指令

处理器的**工作模式**规定当前代码能够执行哪些受保护操作。用二态模型表示时，用户态权限较低，内核态权限较高。模式位保存在程序状态信息中，由硬件按规则改变；普通用户程序不能靠一条赋值指令就把自己切换成内核态。

特权指令只能在规定的高权限模式执行，例如修改关键内存映射、设置中断屏蔽状态、直接控制受保护 I/O。若允许任意用户程序关闭时钟中断，它就可能长期占用 CPU，使调度失去控制。普通算术、逻辑和多数寄存器操作没有这种系统级影响，可以在用户态执行，也可以在内核态执行。

由此要区分两件事：特权指令必须在内核态运行，但内核态运行的并不都是特权指令。前者约束的是特权指令；若把内核态理解成只能执行特权指令，内核中的普通计算就无从进行。陷入指令是用户程序主动进入受控入口的指令；它允许在用户态执行，执行后由硬件转入规定的内核入口，不能据“会进入内核”反推它是用户态禁止执行的特权指令。

1.5 中断和异常怎样把控制权交给内核

外部中断由正在执行的指令以外的事件引起，如时钟到期、设备完成。**异常**由当前指令执行过程中的事件引起，如除数为零、缺页、执行非法指令；主动执行陷入指令也通过受控异常入口请求服务。不同教材可能把“中断”作为包括内外事件的总称，本书需要区分时使用“外部中断”和“异常”。

一次典型控制转移包括以下连续过程：硬件检测事件，保存继续执行所需的断点和程序状态，按事件号找到处理入口并切换到所需模式。入口代码再保存会被处理程序改写的通用寄存器，然后执行处理；退出时恢复相应现场，使用专门返回指令恢复程序状态和继续位置。具体保存位置以及硬件、软件的分工取决于体系结构，不能把所有现场的保存都归给同一方。

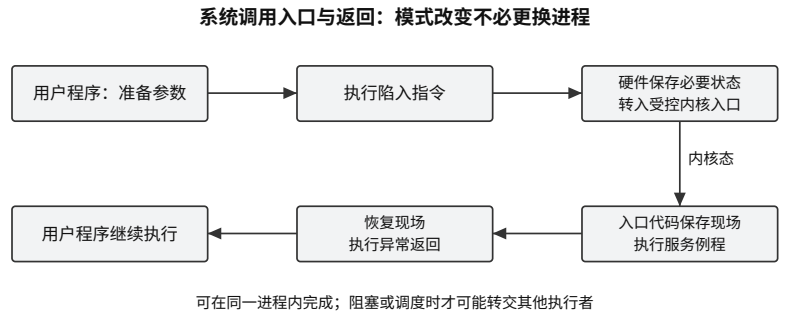


Figure 1.2: 从用户程序进入内核，返回时仍可继续原进程

外部中断通常在指令边界被响应；异常通常在指令执行期间被发现。可恢复故障处理完后，可能重新执行产生异常的指令，例如缺页后重试访存；陷入返回通常从陷入指令的后继指令继续执行；不可恢复错误可能终止进程。题目给出具体指令集时，以其断点与返回规则为准。

1.5.1 陷阱、故障与终止：异常返回地址为何不同

错项排除·系统调用不保证阻塞

读取的数据若已经在内核缓存且可立即返回，系统调用仍跨越了保护边界，却可能始终由原进程完成。判断阻塞要看所需事件是否尚未满足，判断进入内核要看是否通过受控入口，两项不能互相替代。

异常可按触发与恢复方式区分。**陷阱（trap）**通常是程序有意发起的受控事件，常用于请求内核服务，处理完后继续执行后继指令。**故障（fault）**表示当前指令执行条件暂不满足；若处理能够修复条件，如为合法虚拟页调入内容，返回后可重新执行引起故障的指令。**终止（abort）**表示难以恢复的严重事件，通常无法正常继续原来的指令序列。

类别	事件与当前指令的关系	常见后续行为
外部中断	来自设备、时钟等外部事件	处理后恢复被打断程序，或依据调度决定切换
陷阱	程序有意执行陷入指令	从规定的后继位置继续
故障	当前指令执行发现可处理问题	修复条件后重试原指令
终止	发生难以恢复的严重错误	结束相关执行或进入故障处理

分类用来说明机制，不宜只凭“除零”等事件名称断言所有体系结构的处理和返回方式完全相同。具体题目给出指令集、异常模型或操作系统行为时，应以题设为准。

中断向量提供中断处理入口等信息；**中断向量表**按事件编号组织这些入口信息。响应机制定位入口后，才进入相应的软件处理程序。中断响应、保存现场、执行处理、恢复现场和返回是连续环节，硬件与软件的职责应分别核对。

1.5.2 例 1-2　执行系统调用时，哪些操作属于软件

例 1.5.1 (配套例题)

【配套例题】执行系统调用的过程涉及下列操作，其中由操作系统完成的是（　）。

I. 保存断点和程序状态字　　II. 保存通用寄存器的内容

III. 执行系统调用服务例程　　IV. 将 CPU 模式改为内核态

A. 仅 I、III　　B. 仅 II、III

C. 仅 II、IV　　D. 仅 II、III、IV

解析。按题目采用的常规异常入口模型，I、IV 由响应机制中的硬件完成；II 由内核入口代码完成，III 本来就是内核软件执行的服务。因此答案为 **B**。将模式切换留给不受保护的用户代码会破坏入口的受控性；内核不能等到自己已经进入服务代码，才由普通指令任意决定是否取得权限。

中断处理不必导致进程切换。例如时钟中断只更新计数后返回原进程，模式虽然来回切换，但运行的仍是原进程。这个区别在第 2 章用于分析上下文切换。

1.6 系统调用与库函数：调用一段代码，还是请求内核服务

系统调用是应用通过规定接口请求内核服务的过程。程序准备调用号和参数，执行陷入指令；内核检查参数、权限及资源状态，分派到相应

服务；服务完成时，或进程阻塞后又被唤醒继续时，内核把结果返回用户程序。检查参数不能省略，例如用户提供的缓冲区地址必须落在允许访问的范围内。

库函数是程序可以调用的已有函数。纯字符串长度计算可以完全在用户态完成；文件读写库函数可能整理格式、维护用户缓冲区，再调用 `read`、`write` 等系统调用。因而“调用库函数”既不必然发生模式切换，也不排斥发生系统调用。反过来，系统调用也不是普通函数调用：它跨越保护边界，并通过内核规定的入口执行。

读取磁盘文件时，`read(fd, buf, n)` 中的 `fd` 标识已经打开的文件，`buf` 是接收数据的用户缓冲区，`n` 是请求字节数。若数据已在内核缓存，内核可能复制后直接返回；若必须等待设备，当前进程可能先阻塞，CPU 转去运行其他进程。不要把“系统调用进入内核”与“系统调用必阻塞”合为一条结论。

〖1.1-010〗库函数不直接处理所有受保护硬件请求。判断一个具体函数是否引起陷入，要看它在本次路径上是否请求内核服务，不能只凭“库函数”这个名称。

不变量·先判范围，再求和

界限寄存器按长度定义时，合法逻辑地址始终满足 $0 \le a < L$。即使 $B + a$ 仍落在已安装的物理内存内，也不能放行 $a = L$。这个不变量保护的是当前进程的合法区间，与整台机器是否还有空间无关。

1.7 从源程序到可运行映像：链接、装入与地址绑定

编译把源程序翻译为目标模块，目标模块可能仍含未确定的外部符号。**链接**把多个目标模块和所需库连接起来，解析符号并处理地址引用，形成可装入文件。静态链接在运行前完成所需模块的合并；装入时动态链接在装入时连接；运行时动态链接在首次调用等运行时机连接所需模块。

装入把可执行文件需要的内容映射或读入进程地址空间，并建立运行条件。地址绑定可能发生在不同阶段。

方式	地址何时确定	移动到另一个物理位置会怎样
绝对装入	编译或汇编时已给定绝对地址	通常须重新生成或修补地址
可重定位装入	装入时按实际起点修改相关地址	装入后不能仅换一个基址就随意搬移
动态运行时装入/重定位	执行访存时由地址变换机构转换	更新映射后可改变物理位置，需相应硬件支持

最小的动态重定位模型使用基址寄存器 B 和界限 L。程序产生逻辑地址 a，先检查 $0 \le a < L$，合法时物理地址为 $B + a$。例如 $B = 12000, L = 2000$，逻辑地址 1500 转为 13500；逻辑地址 2000 已越界，不能因为“两数相加仍在主存范围内”就判为合法。界限表示长度时，上界通常不包含；若题目把寄存器定义成最大合法地址，则应按其定义调整比较。

动态重定位在执行阶段进行，依赖重定位信息与硬件地址变换机构，不需要每次访存都调用装入程序。操作系统负责设置和维护映射，硬件完成运行中的快速转换。分页的非连续映射在第 3 章展开。

1.8 进程地址空间中，代码、数据、堆和栈各放什么

课程连接·指针自身与所指对象分开

数据结构中的链表头指针可能是函数局部变量，所指结点却由动态分配得到；函数退出与结点释放属于不同生存期问题。分析内存图时为“指针变量”和“动态对象”分别画位置，能避免把整个链表误判为栈对象。

地址空间是一组可由地址标识的位置。进程的虚拟地址空间提供其运行时看到的地址视图，不要求这些地址对应的内容全部同时存在物理内存。可执行文件的磁盘布局、进程的虚拟布局、物理页框布局是三个不同对象。

典型用户空间包含代码区、静态数据区、堆、栈和映射区。代码区存放机器指令，通常只读且可共享；已初始化的全局变量、静态变量通常在初始化数据区，未显式初始化或零初始化的数据可用零填充区表示。堆用于动态申请的对象，栈保存函数活动记录中的参数、返回信息与未被优化掉的自动局部变量。共享库和映射文件可位于映射区。具体地址增长方向和排列由平台决定，示意图不能代替题设。

考虑以下代码，假定采用常规 C 运行时布局且忽略编译优化：

```c
#include <stdlib.h>

int g = 3;
static int z;
int main(void) {
    int n = 16;
    char *p = malloc(n);
    /* 使用 p 指向的空间 */
    free(p);
}
```

`g` 是初始化静态存储期数据，`z` 是零初始化静态存储期数据；`n` 和指针变量 `p` 自身属于 `main` 的局部活动记录；`p` 指向的动态对象属于堆。指针存在哪里与它指向的对象存在哪里必须分别回答。“写在函数外”并不等于“属于代码段”；局部变量也未必都放在栈上，优化可能把它们放进寄存器或直接消除。

进程控制块 PCB 保存调度与管理信息，是内核数据结构，不是应用能够随意修改的用户栈对象。同一程序的两个运行实例可以共享只读代码，但具有不同的进程身份、逻辑状态和数据视图。

1.9 内核结构：把哪些功能放进同一保护域

分层结构按依赖关系划分功能，上层使用规定的下层服务，便于逐层验证；代价是层次难划分、调用路径可能增长。**模块化结构**把功能组织为接口清楚的模块，允许在约定接口下替换实现。模块划分不自动规定严格的层次调用关系，模块也不等于独立的保护域。模块设计追求模块内部功能紧密相关（高内聚），模块之间通过尽量少的依赖协作（低耦合）。接口需要在开发前充分设计；若需求尚不明确，预先规定所有接口较困难，跨模块协调也可能增加设计和维护成本。

宏内核把较多的管理功能放在内核地址空间中，功能间可以直接调用，通信开销较小；一个内核模块的错误也可能影响整个系统。**微内核**把最基本的调度、低级地址空间管理、进程间通信等保留在内核，把较多服务移到用户态服务器。隔离与替换更容易，但一次服务可能增加消息传递和保护域切换，不能无条件断言性能更好。

外核进一步把资源保护和复用机制与高层资源抽象分开，让应用

或库操作系统按需要实现抽象。它并非 “没有内核” 或 “不保护资源”；恰恰相反，保留必要的保护职责正是外核的基础。

比较宏内核与微内核时，先看服务放在哪个保护域，再分析通信和故障如何跨越这个边界。下表比较典型结构；实际系统可以混合采用这些设计。

共同维度	宏内核	微内核
服务的位置	文件系统、设备管理等较多服务在内核态	只保留基本机制，较多服务由用户态服务器实现
服务间通信	同一内核地址空间内可直接调用	常经进程间通信、消息传递和保护域切换
故障影响	内核模块错误可能破坏其他内核功能	用户态服务器的故障较易隔离，但仍需设计恢复机制
扩展和移植	模块接口有助维护，修改仍须考虑内核内部依赖	可分别调整服务器；小内核较便于验证、移植
性能代价	调用路径较短；功能多不等于任意实现都快	跨域通信可能增加开销；隔离收益不能直接推出性能更高

适用边界·关中断的作用范围

当前 CPU 暂不响应可屏蔽中断，不会自动阻止另一 CPU 访问共享结构。多处理机原子性还依赖锁、原子指令等机制。

1.9.1 例 1-3　微内核的收益与代价

例 1.9.1 (配套例题)

【配套例题】与宏内核操作系统相比，下列特征中，微内核操作系统具有的是（　）。

I. 较好的性能　II. 较高的可靠性　III. 较高的安全性　IV. 较强的可扩展性

A. 仅 II、IV　　B. 仅 I、II、III

C. 仅 I、III、IV　　D. 仅 II、III、IV

解析。服务间保护边界有利于限制故障影响，减少高权限代码有利于安全，服务模块也更容易扩充；但跨域通信会增加开销，不能把 I 列为一般优势。答案为 **D**。这些是结构层面的比较，不是对任意两个具体系统的性能或安全性作无条件排名。

1.9.2 原语、时钟与内核的基本机制

内核的底层机制通常包括时钟管理、中断处理和原语。时钟中断既可支持计时，也为时间片到期、定时等待等管理提供事件来源。原语负责完成需要保持一致性的关键操作，例如保存现场、改变进程状态和移动 PCB 队列。

原语的**原子性**要求其他执行者不能观察到破坏约定的中间状态。单处理器模型常用短暂关闭可屏蔽中断的方法保护关键操作；它的作用范围是当前处理器，且不能屏蔽所有可能的异常。多处理器上，别的处理器仍可能访问共享数据，因此还需要锁或原子指令等同步机制。不能把 “本 CPU 不响应某类中断” 当作 “整台机器没有并发访问”。

分层结构、模块化、宏内核和微内核比较的是功能组织和保护边界。模块之间接口清楚不意味着每个模块都必须独立运行在用户态；微内核将服务移出高权限域，也仍需通过内核机制保护通信和共享资源。

1.10 开机以后，内核怎样获得控制权

课程连接·开机结构与磁盘格式不同

中断入口、页表和调度队列属于本次运行所需结构；分区表、根目录和磁盘索引节点通常是持久存储的一部分。把题中对象按“随运行建立”与“跨重启保留”定位，才能判断初始化是否需要重新创建。

典型引导链为：处理器复位后执行固件中的启动代码，固件初始化必要硬件并选择启动设备；引导程序定位并装入内核；内核建立中断入口、内存管理和设备管理等基本环境，创建初始进程，最后进入正常调度与用户服务。引导程序可以分阶段，早期阶段只装入后续阶段，不能把所有启动代码都理解成同一段磁盘程序。

传统 BIOS/MBR 模型中，磁盘主引导记录可含引导代码和分区表；现代 UEFI/GPT 的组织不同。回答“引导程序在哪里”必须先确定所指的是固件初始代码、主引导代码，还是文件系统中的后续装载器。引导不等于格式化：已经安装的文件系统和分区信息可以跨多次开机保留，而内核运行所需的内存管理结构需要本次启动建立。

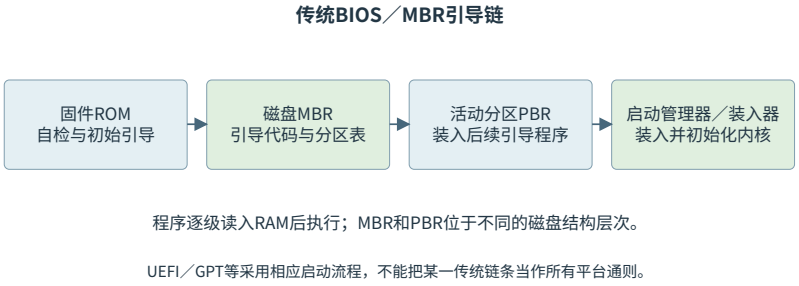


Figure 1.3: 传统 BIOS ／ MBR 模型中，固件、MBR、PBR 和后续装入程序逐级建立启动环境

1.10.1 例 1-4　每次开机需要重新建立什么

例 1.10.1 (配套例题)

【配套例题】下列选项中，需要在操作系统进行初始化过程中创建的是（　）。

A. 中断向量表　　B. 文件系统的根目录

C. 硬盘分区表　　D. 文件系统的索引节点表

解析。中断向量表是本次内核运行所需的控制转移结构，需要在初始化中建立，答案为 **A**。其余为磁盘格式与文件系统的持久结构，通常在分区或创建文件系统时形成，不应每次开机重建。

1.11 虚拟机：客户操作系统也可以成为被管理者

虚拟机管理程序 VMM 向客户操作系统提供虚拟处理器、内存和设备，安排多个虚拟机使用物理资源，并隔离其影响。客户操作系统中的“内核态”只是客户的权限视图，并不意味着客户操作系统可以越过 VMM 任意修改主机资源。敏感操作由硬件虚拟化支持、陷入模拟或其他受控机制处理，具体路径随实现变化。

第一类 VMM 直接运行在硬件上；第二类 VMM 依托宿主操作系统运行。二者都可能采用硬件加速，因此不能把它们简单等同于“硬件实现”和“软件实现”。多台虚拟机可以在单核上交替运行，也可以在多核上并行运行，所以“同时存在多台虚拟机”不能推出它们在每一时刻都真正并行。

进程虚拟地址空间隔离的是进程访存；系统虚拟机为客户操作系统提供完整机器抽象。容器通常共享宿主内核，通过名称空间、资源控制等隔离进程环境，但这不等同于每个容器都带一个独立客户内核。这里借容器辨析抽象层次，不要求记忆其实现细节。

抽象层次·客户与宿主分开判断

客户进程阻塞是客户操作系统内部事件；虚拟机暂停是整个客户机器失去物理运行机会。二者的管理者和影响范围不同。

1.11.1 虚拟机的两种组织方式

维度	第一类 VMM	第二类 VMM
直接依托的层次	直接运行在物理硬件上	作为宿主操作系统上的程序运行
设备服务路径	由 VMM 及其设备服务组件组织	还利用宿主系统的设备驱动与服务
客户系统看到的对象	虚拟 CPU、虚拟内存、虚拟设备	同样是完整的虚拟机器抽象
存储资源分配	由 VMM 管理物理存储并分配客户空间	虚拟磁盘常表现为宿主文件系统中的文件
性能与可支持数量	少一层宿主转接，通常具有较小的额外开销	还需宿主资源；数量与性能受具体负载和实现影响
虚拟机迁移	依赖管理程序的镜像、设备和迁移机制	虚拟磁盘文件便于复制；仍须匹配虚拟硬件与平台
运行权限	VMM 管理物理资源，特权机制由平台提供	可由用户态组件与宿主内核组件协同完成

客户内核使用虚拟 CPU 的高权限操作时，实际保护层仍须确认其影响只落在分配给该虚拟机的资源上。虚拟机暂停与客户进程阻塞不是同一个层次的状态：前者可使整个客户系统暂时没有物理 CPU 运行机会，后者是客户系统内部某个执行者在等待事件。

Figure 1.4: 第一类 VMM 直接依托硬件；第二类 VMM 使用宿主操作系统提供的服务

第一类 VMM 直接管理物理资源，可把物理存储空间等划分给客户系统；第二类 VMM 借助宿主文件、内存和设备服务，客户虚拟磁盘可以表现为宿主文件中的数据。通常第一类少一层宿主转接、资源控制更直接；第二类依赖宿主的资源与驱动，作为普通宿主文件保存的虚拟机器也较便于复制迁移。实际性能、客户种类和迁移能力还取决于硬件虚拟化支持及产品实现。

虚拟化还必须控制客户内核的特权操作：客户看到的高权限不等于可以越过 VMM 访问别的客户资源。支持虚拟化的 CPU 提供相应的执行与陷入机制；传统 Ring 0～3 权限示意中的 Ring 0 与用户 Ring 3，

说明的是受控权限等级，不能据此断定客户内核直接取得整台物理机的最高控制权。

课程连接·回忆顺序

先区分用户态与内核态，再分别说明异常入口、内核操作和返回的位置。

1.12 分支回顾与自测

以下为原创回顾设问，用于检查本章机制与边界。

1. 普通函数调用、系统调用、设备中断是否都必然更换当前进程?
2. 用户程序执行陷入指令后进入内核，是否说明陷入指令应禁止在用户态执行?
3. 基址为 4096、界限为 1024，逻辑地址 1023 与 1024 分别怎样处理?
4. 指针变量在栈上，是否说明它指向的对象也在栈上?
5. 单核运行四台虚拟机，是否要求四台同时执行机器指令?

1.12.1 自测答案

1. 都不必然。函数调用通常不跨保护域；系统调用和设备中断可能只切换模式后返回原进程。只有实际选中另一个进程时，才发生调度切换。
2. 否。陷入是用户可执行的受控入口；目标入口和权限变化受硬件及内核规则约束。
3. 1023 合法，物理地址为 5119；1024 等于长度上界，越界。
4. 否。须分别判断指针自身与所指对象的生存期和分配方式。
5. 不要求。它们可以分时复用一个处理器，并发而非同一时刻并行。

1.13 主动回忆自测

以下均为自拟题，先盖住答案，按 “对象—权限—执行路径—条件” 口述理由。

功能、权限与结构的综合辨析

1. 单 CPU 交替运行两个程序，哪些特征成立？能否据此断言两程序指令并行?
2. 为什么增加进程数不一定持续提高 CPU 利用率?
3. 陷入指令为何能在用户态执行，而修改中断屏蔽状态通常不能?
4. 一次设备中断后仍返回原进程，发生了什么切换，又没有发生什么切换?
5. 基址 12000、界限 2000，逻辑地址 1999 与 2000 怎样处理?
6. 函数局部指针指向 malloc 得到的空间，两个对象各属于哪种存储?
7. 微内核减少高权限代码，为什么仍不能直接断言性能更高?
8. 第一类、第二类 VMM 的区别是什么？单核运行多台虚拟机是否矛盾?

1.13.1 核对答案与关键理由

1. 同一时间段交替推进是并发；单核同一时刻通常只执行一条用户指令，不能据此判用户指令并行。
2. 可隐藏部分 I/O 等待，但过多进程会增加调度、同步和内存压力，严重时抖动，收益有条件。

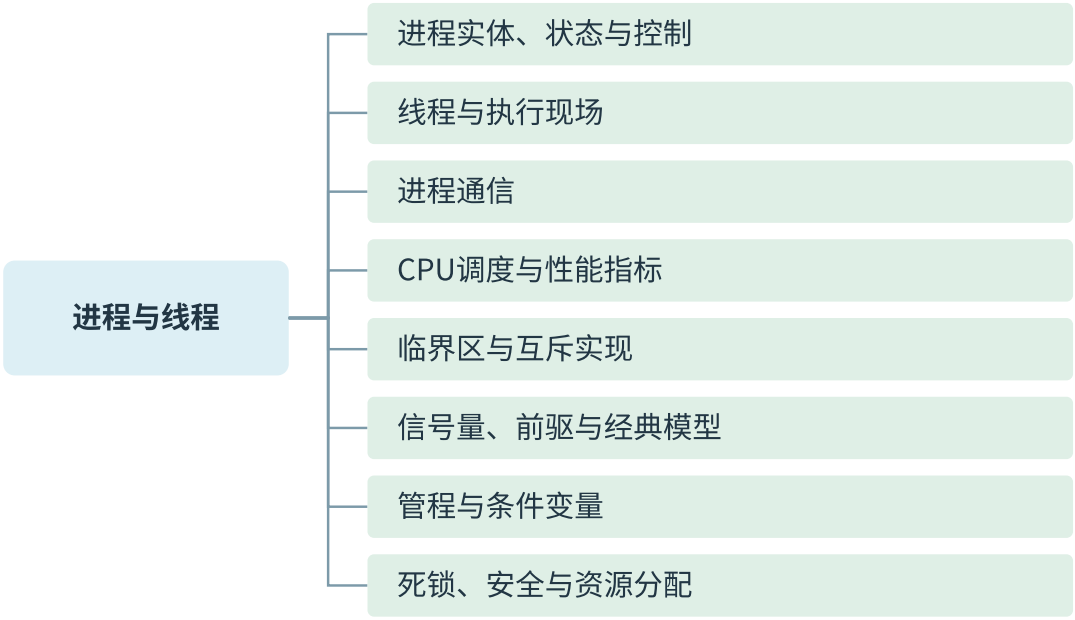
3. 陷入只进入预定的受控入口；任意用户修改全局屏蔽状态会破坏调度与保护，因而受权限约束。
4. 中断可导致用户态与内核态之间的模式变化；实际运行身份仍是原进程时，没有切换到另一个进程。
5. 1999 满足 $0 \le a < 2000$，物理地址 13999；2000 等于长度上界，越界，不能先相加再放行。
6. 指针变量本身通常位于函数活动记录中，动态对象在堆；优化可能改变局部变量的实际存放，题设优先。
7. 服务移出内核改善隔离，却可能增加 IPC、消息复制和保护域切换；不同实现的总性能须实际比较。
8. 第一类直接运行于硬件，第二类依托宿主系统；二者都可用硬件加速。多虚拟机可分时复用单核，无矛盾。

课程连接·自主检验

遇到调用流程题，依次写出发起者、特权转换、内核处理和返回条件。

进程与线程 2

章节脉络



课程连接·四条主线

进程与线程描述执行实体；调度分配处理器；同步规定先后；死锁涉及多资源等待。

2.1 进程执行、调度、同步与资源分配

进程把程序的一次执行和资源环境联系起来，线程进一步划分执行流。一个执行者能否继续前进，既取决于是否分到 CPU，也取决于所需数据、同步事件与资源是否到位。下面先建立状态与调度模型，再用互斥、信号量和死锁分析描述执行者之间的关系。

2.2 进程、线程、调度与上下文切换

程序是指令和数据的静态描述，进程是程序的一次执行过程及其资源环境。同一可执行文件运行两次，可以产生两个进程；同一进程又可以包含多个执行线程。判断状态变化时，要分别问：资源属于谁，当前谁在执行，谁还在等一个事件，谁只是等待处理器。

2.2.1 进程与线程：资源单位和执行单位

进程是系统进行资源分配和管理的基本单位，具有独立的运行状态和地址空间视图。进程实体通常由程序段、相关数据和进程控制块 PCB 等构成。PCB 记录标识、状态、调度信息、资源使用信息及保存的处理器现场，内核据此管理进程；“一个进程没有正在占用 CPU” 不等于它不存在。

线程是进程内部的一条执行流。在引入线程的系统中，处理器调度通常落实到可调度线程，而地址空间、打开文件等资源主要归所属

进程。每条线程需要自己的程序计数器、寄存器现场、栈和线程控制信息，否则无法分别记住执行位置和函数调用链。

同一进程的线程可以访问共同的代码、静态数据、堆及进程资源，但拥有独立的执行现场。“线程有自己的栈”指的是各线程的函数调用状态分别管理，但这不意味着同进程的其他线程在地址上无法访问该栈。只要得到有效地址并具备权限，线程仍可能错误地访问另一线程的对象；私有用途与硬件隔离不同。

进程通常具有动态性、并发性、独立性、异步性和结构性。动态性体现进程随执行而创建、变化和结束，是进程的最基本特征；结构性指进程实体由程序段、数据段和 PCB 组成。独立性指其运行和管理环境，不表示不能通信或共享；异步性也不意味着可以违反自身指令的必要依赖。线程有利于把服务器中等待网络、处理请求等活动组织成多个执行流，但线程数增加同样会带来同步、栈空间和切换开销。

错项排除·唤醒与分派是两步

I/O 完成把阻塞者所缺的事件补齐，通常只使其进入就绪队列。处理器仍可由原进程或另一个高优先级者使用，因此“设备一完成，该进程立即运行”遗漏了调度决定。画状态图时应保留阻塞到就绪、就绪到运行两条边。

比较维度	不同进程之间	同一进程的不同线程之间
资源与地址空间	各有地址空间和进程资源；共享须按机制建立	共享代码、数据、堆和打开文件等所属进程资源
执行现场	由各自的控制信息保存 PC、寄存器、栈等	每条线程仍有独立 PC、寄存器、栈和线程控制信息
切换要改变什么	还需切换相应地址空间环境	通常无需更换所属进程的地址空间，仍须换执行现场
数据交换	通过共享内存、消息、管道等机制	可直接访问共同对象，仍须处理同步与互斥
隔离边界	默认地址隔离有助限制误写的影响	一个线程可破坏共同空间；线程栈并非在地址上对其他线程不可访问

这里比较的是资源归属；能否由内核分别调度、阻塞是否牵连其他线程，还要看下一节的线程实现与映射模型。

2.2.2 就绪、运行和阻塞：先判断缺少什么

就绪态表示除处理器外，执行所需条件已满足；**运行态**表示正在处理器上执行；**阻塞态**表示等待 I/O、资源、同步事件等条件，暂不能因获得 CPU 就继续。单处理器同一时刻至多一个相应执行实体在运行，就绪与阻塞却都可以有多个。

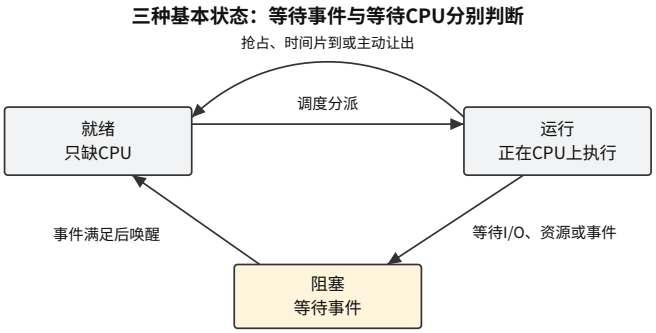


Figure 2.1: 唤醒先进入就绪，调度以后才运行；箭头上的条件不能省略

运行者用完时间片或被更高优先级者抢占，通常转为就绪；执行阻塞式 I/O 而数据未就绪，转为阻塞；等待事件发生后，由内核唤醒为就绪。阻塞者一般不能直接跳到运行，因为事件满足与分配处理器是两个决定。就绪者没有在执行，也不能主动发起一项新的阻塞操作。

创建态用于建立运行环境，终止态用于退出和清理；若进程被换出内存，还可引入挂起就绪、挂起阻塞等状态。挂起描述的是进程是否驻留内存，阻塞描述的是缺少事件，二者不是同义词。等待事件已满足的挂起阻塞进程，可以先成为挂起就绪，仍须调入并获得 CPU 才能运行。

辨析·挂起与阻塞

阻塞表示事件未满足；挂起表示映像暂时不驻留。事件发生后，阻塞挂起转为就绪挂起，调回后才进入就绪队列。

挂起与激活：七状态模型

课件在五状态模型上增加**就绪挂起**与**阻塞挂起**：前者执行条件已具备，但进程映像暂不在内存；后者既在等待事件，又处于挂起状态。在此经典模型中，挂起指暂时调出进程映像，普通阻塞态仍驻留内存；进程控制块仍须保留以便系统管理。不同实际系统可采用更细的驻留和换出策略。

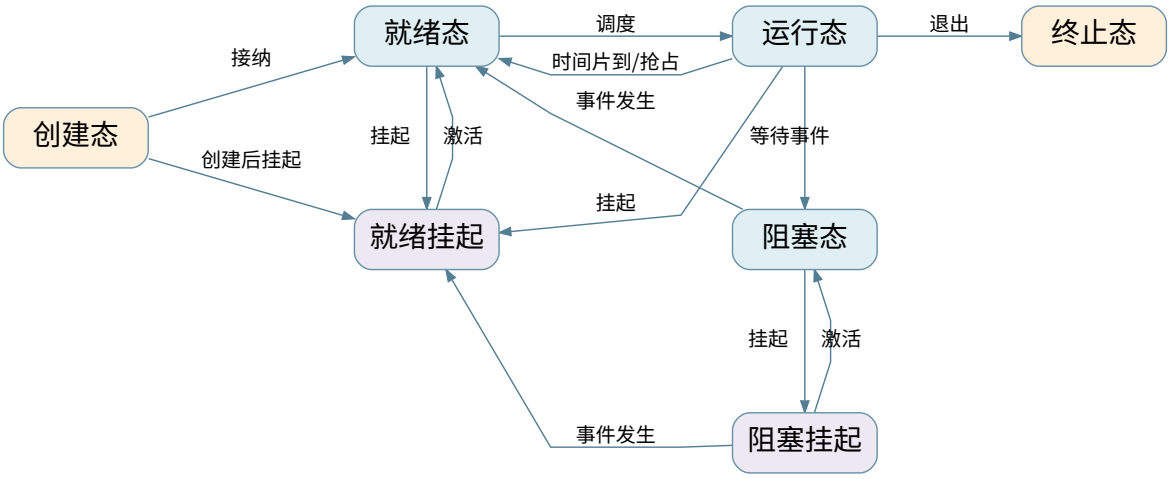


Figure 2.2: 七状态进程模型中的挂起、激活与事件转换

事件在阻塞挂起时发生，进程转入就绪挂起，而不是立即取得处理器。激活把就绪挂起者转为就绪、把仍在等待的阻塞挂起者转为阻塞；运行者也可能被挂起。创建后的进程可按系统策略进入就绪或就绪挂起。挂起队列可按就绪与阻塞区分，阻塞挂起者还可按等待事件进一步分类。

例 2A-1　主动让出处理器后去哪里

例 2.2.1 (配套例题)

【配套例题】下列由当前线程引起的事件或执行的操作中，可能导致该线程由执行态变为就绪态的是（　）。

A. 键盘输入　　B. 缺页异常　　C. 主动出让 CPU　　D. 执行信号量的 wait() 操作

解析。主动出让处理器时，线程可能仍具备执行条件，只是把本次运行机会交出去，因而变为就绪，答案为 **C**。缺页且需等待调页、等待未到来的输入，通常使线程阻塞。信号量 wait 若能取得许可，可继续运行；若取不到，则阻塞，而不是仅因调用了 wait 就进入就绪。

键盘输入或另一进程退出临界区，可能满足某个阻塞者等待的条件，使它被唤醒；当前进程的时间片用完则主要改变当前运行者的状

态。判断 “可能唤醒” 时要指明被满足的是哪个等待条件，不能把所有调度事件都当作唤醒。

课程连接·模型决定多核并行度

多个用户线程若全部复用一个内核执行实体，内核无法同时把它们安排到两个 CPU 上。若映射到多个内核线程，才可能获得多核并行。判断时把用户线程数、承载实体数、物理核数分开，不能仅凭线程名字推结论。

2.2.3 用户级、内核级线程与映射模型

用户级线程由用户空间线程库建立和管理，内核可能只看见承载它们的进程或少量内核调度实体。纯用户级线程切换可以只修改用户现场，开销较小；但在多对一模型中，一个承载实体因阻塞式系统调用被阻塞，其他用户线程也失去在该实体上执行的机会；内核也无法把这些用户线程分派到多个核真正并行。

内核级线程由内核识别、管理和调度。某线程阻塞时，同进程的其他内核线程仍可运行；多个线程也可分派到不同核。代价是创建、管理和切换通常需要内核参与；因此 “切换开销小” 是用户级线程的长处，不能反过来算作内核级线程的优势。

共同维度	用户级线程 ULT	内核级线程 KLT
谁维护线程控制信息	用户空间的线程库	操作系统内核
谁选择这一级的运行者	线程库选择某个承载实体内的用户线程	内核调度器选择可运行的内核线程
切换怎样发生	纯用户级切换可只保存、恢复用户现场，不必进入内核	由内核保存、恢复相应执行现场并分派
内核能区分到哪一级	不能分别调度其不可见的用户线程，只能调度承载实体	可分别阻塞、唤醒和分派各内核线程

是否牵连其他线程、能否多核并行，还要看这些管理对象在下表中如何映射，不能仅凭 “用户级” 三个字判断所有实现。

映射	内核看到什么	阻塞和并行的结果
多对一	多个用户线程对应一个内核执行实体	一个阻塞可能挡住全部；不能在多核上同时执行这些用户线程
一对一	每个用户线程对应一个内核线程	阻塞影响可局部化，可多核并行；内核管理成本较大
多对多	多个用户线程复用若干内核线程	线程库与内核分层调度，并行度受可用内核线程和核数约束

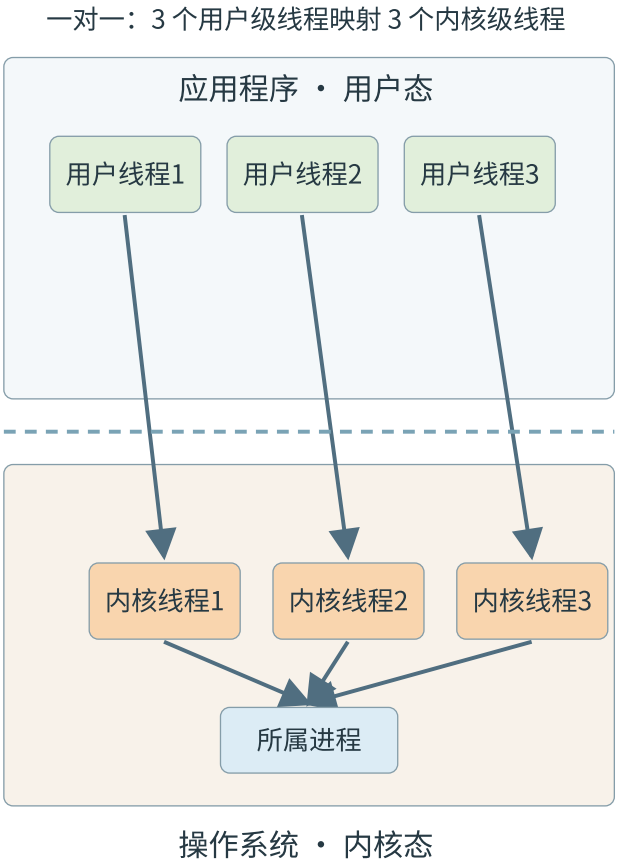
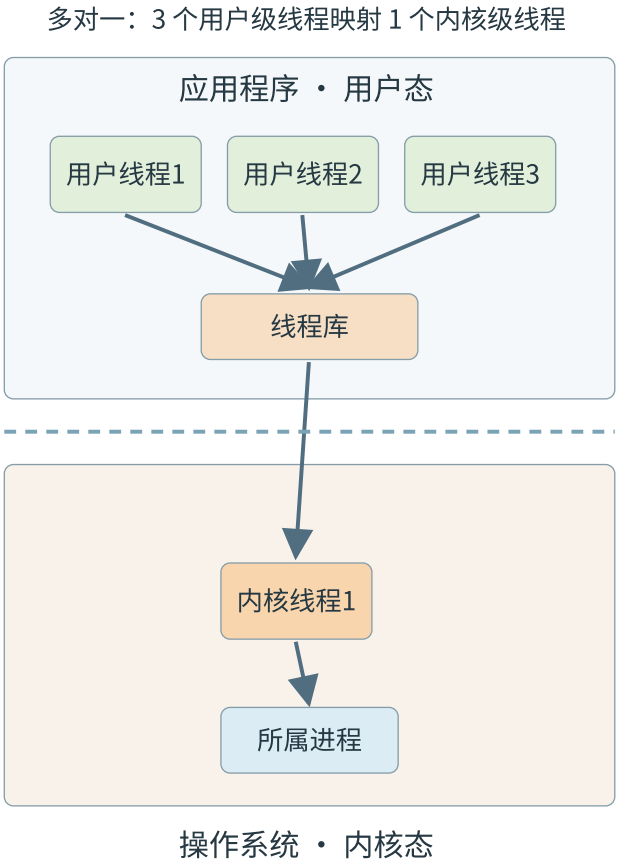


Figure 2.3: 一对一模型：三个用户线程各自对应一个内核线程

课程连接·模型决定多核并行度

多个用户线程若全部复用一个内核执行实体，内核无法同时把它们安排到两个 CPU 上。若映射到多个内核线程，才可能获得多核并行。判断时把用户线程数、承载实体数、物理核数分开，不能仅凭线程名字推结论。



课程连接·模型决定多核并行度

多个用户线程若全部复用一个内核执行实体，内核无法同时把它们安排到两个 CPU 上。若映射到多个内核线程，才可能获得多核并行。判断时把用户线程数、承载实体数、物理核数分开，不能仅凭线程名字推结论。

Figure 2.4: 多对一模型：三个用户线程通过线程库映射到一个内核线程

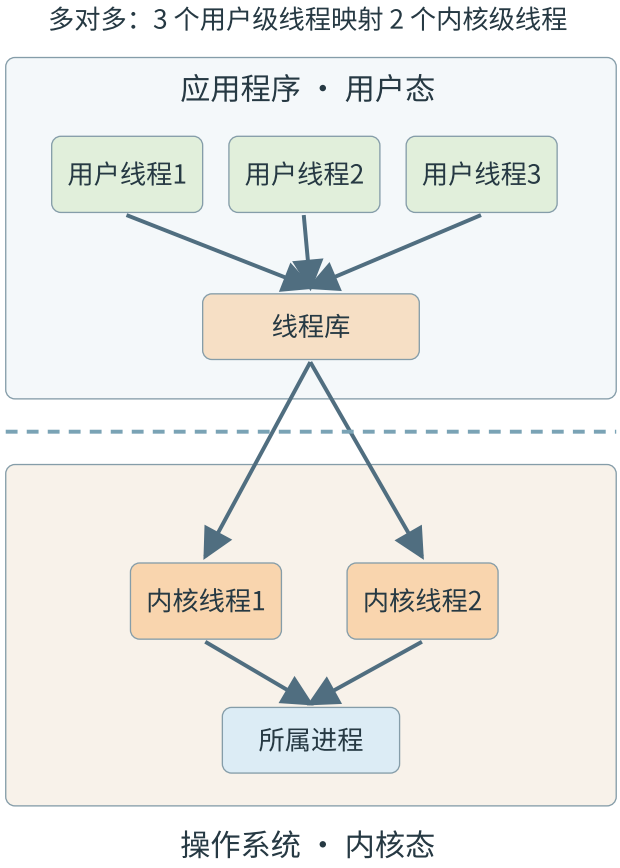


Figure 2.5: 多对多模型：三个用户线程通过线程库映射到两个内核线程

图中分别展示三个用户线程对应三个、一个和两个内核线程的例子。多对多的“多”不是要求两侧线程数量相同；用户线程库负责用户线程的管理，而内核调度其可见的内核线程。线程阻塞是否影响其他线程，还要看具体映射及阻塞发生的位置。

“内核级”描述管理方式，不能理解成该线程的应用代码始终在内核态执行。用户线程进入系统调用时也可能执行内核代码。线程类型、CPU 当前模式、进程所属地址空间是三个不同判断维度。

2.2.4 创建、撤销与组织：PCB 为什么是管理入口

创建进程一般需要分配标识和 PCB、建立地址空间和初始数据、关联资源、初始化执行现场，再加入适当的调度队列。用户登录、启动程序或系统创建服务进程可能触发创建；给已有进程分配设备只是改变其资源状态，不必产生新进程。

终止进程时，内核停止其执行，处理相关等待与通知，回收资源并释放管理结构。子进程是否随父进程终止，取决于系统的父子关系政策；不能把级联终止当成所有系统的通则。某些系统会保留最少退出信息，供父进程取得状态后再完全释放。

PCB 可以用链表、队列或索引表组织。就绪队列按调度规则组织可运行者，事件等待队列组织等待同类条件者；状态转换时要同步维护状态字段和所在队列。不能只改一个“状态字”，却把进程留在错误的队列中。

课程连接·PCB 的作用

PCB 记录身份、状态、现场与资源信息；进程存在期间，系统需维护其对应管理信息。

线程的创建同样要建立 TCB、栈和初始执行现场，但可以复用所属进程的地址空间等资源。在常规父子进程模型中，父子是两个独立进程，可并发执行、有不同 PCB；即使最初采用写时复制暂时共享物理页，也不是共用一个无区分的虚拟地址空间。真正共享的部分须由共享映射等机制建立。

线程控制块 **TCB** 保存线程的管理信息。课件所列字段包括线程标识符 TID、程序计数器 PC、其他寄存器内容、栈指针、运行状态和优先级；线程表汇集相应线程的控制信息，供线程管理和调度使用。保存和恢复这些现场，才能让线程从原来的执行位置继续。用户级线程的信息由用户空间线程库管理；内核级线程的控制信息由内核维护，具体表结构随实现而异。

错项排除·共享内存只提供共同位置

两个进程映射同一物理页后，写入可被对方访问，但这并未规定谁先写、何时读，也未把读改写自动变成原子操作。通信方式解决数据放在哪里，同步机制解决操作怎样协调，二者承担的任务不同。

2.2.5 共享内存、消息、管道和信号

共享内存让多个进程把某些虚拟地址映射到共同的物理存储区。建立映射后，数据交换可以通过普通访存完成，减少反复拷贝；但谁先写、读者何时可读、多个修改者是否冲突，仍需要同步机制。共享内存本身并不保证不发生竞争。

课件将共享内存分为两种组织方式。**基于数据结构的共享**对共享区中的数据形式作约束，例如规定只能放固定长度的数组；**基于存储区的共享**由操作系统提供共享区，通信进程自行约定数据布局和访问方式。课件把前者称为低级、后者称为高级通信；这里的区别是接口约束和进程控制范围，具体速度受实现、同步与访问模式影响，不能把“高级”直接当作固定倍数的性能结论。

消息传递把数据交给消息接口，由发送、接收及相关队列组织通信。直接通信指定对方进程，间接通信通过信箱或端口；同步发送、异步发送等语义由接口规定。一次接收若没有消息，可能阻塞，也可能立即报告暂不可用，不能只凭“消息传递”决定状态。

Figure 2.6: 消息先由进程 P 发往进程 Q 消息队列或信箱 A，再由进程 Q 接收

管道提供有次序的数据流通道。经典无名管道常用于有亲缘关系的进程，通常按半双工模型理解；双向通信可用两个管道。命名管道允许按名称建立联系。管道容量有限，空管道上的阻塞读、满管道上的阻塞写都会等待；写入数据后，不代表读取者会立即得到 CPU。

信号是向进程通知某类事件的机制，如终止请求、定时事件等。信号与信号量不同：前者通知事件，后者通过许可计数和等待队列协调资

源与次序。普通信号并不携带任意大量的数据，也不等同于可靠的消息队列；是否排队等细节须看具体信号类型与接口。

例如父进程经管道把一批数交给子进程：父进程先写数据，子进程读到数据后计算。管道本身组织数据和读写等待；若两者还要更新另一个共享统计量，就仍需为该统计量建立同步保护。不要把 “已经用了一种通信方法” 推成 “所有共享状态都已受保护”。

课程连接·状态转换

状态转换通常还伴随队列迁移与现场处理；原语的整体操作不可被中断地执行。

2.2.6 进程控制原语：状态、队列与现场一起更新

进程控制不仅改一个状态字段，还要使 PCB、队列与资源归属一致。以下步骤是逻辑职责，具体实现可以按内核设计合并或调整。

原语	主要工作	对应状态变化
创建	申请空白 PCB，分配必要资源，初始化标识、现场及管理信息，插入就绪队列	创建完成后进入就绪
撤销	确定被终止者，必要时停止其运行，回收资源，移出相关队列并回收 PCB	从原状态进入退出清理
阻塞	保存现场，修改状态，将当前执行者移入相应事件等待队列，重新调度	运行 → 阻塞
唤醒	因相应事件满足而移出等待队列，修改状态，插入就绪队列	阻塞 → 就绪
切换	保存旧执行者现场，更新队列与状态，恢复新执行者现场	运行者与就绪者更替

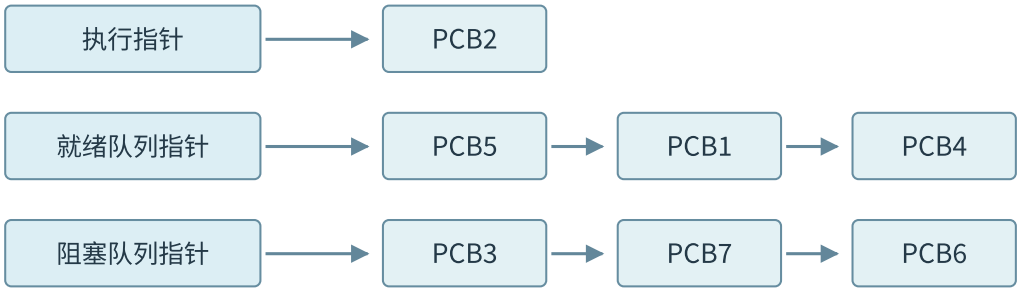
若只把 `state` 从阻塞改成就绪，却尚未把 PCB 从等待队列移走，另一管理操作便可能同时把它当作两种状态处理。因此相关更新要形成受保护的原子操作。原语保护的是关键更新步骤，进程已经阻塞后的整个等待过程可以持续很久，CPU 仍应运行别的就绪者。

PCB 组织方式可以用状态队列，也可以用索引表。队列中的链接字段把同类状态的 PCB 连起来；索引方式在表中保存相应 PCB 地址或索引。系统还可按等待原因分设多个阻塞队列，以便发生事件时定位应唤醒的执行者。

2.2.7 PCB 的链式和索引组织

单处理器的一个状态快照中，执行指针指向 PCB2；就绪队列为 PCB5→PCB1→PCB4，阻塞队列为 PCB3→PCB7→PCB6。链式组织把同一队列的 PCB 通过链接字段串接；由于等待事件不同，也可为各事件分别建立阻塞队列。下图的多核运行队列只是另一种状态快照，不能与单核瞬间的运行指针混为同一个系统时刻。

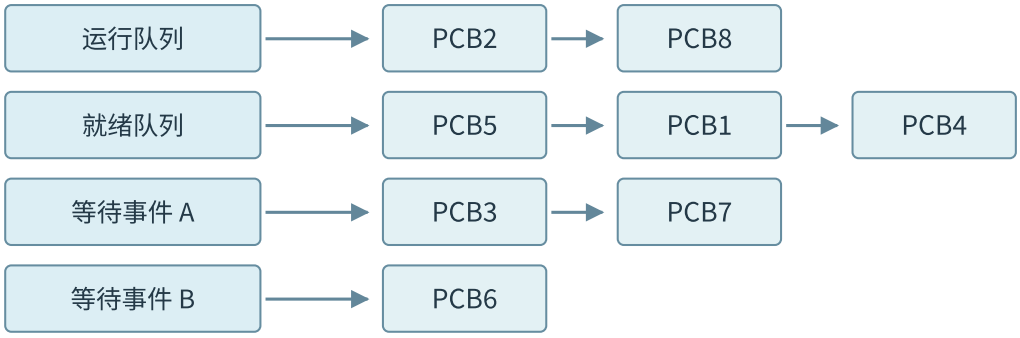
链式组织：同一状态的 PCB 链接成队列



单CPU在同一时刻最多一个进程运行；就绪队列顺序由调度规则决定。
阻塞进程可按等待事件分别组织为多个队列。

Figure 2.7: 执行指针与就绪、阻塞 PCB 链

多核及按事件区分的等待队列



多核CPU可以有多个进程同时处于运行态。

Figure 2.8: 多核运行队列及不同事件的两个阻塞队列

索引组织将各状态对应的 PCB 定位信息放入索引表。另一个示例的就绪索引依次指向 PCB5、PCB1、PCB4，阻塞索引依次指向 PCB6、PCB7、PCB3；两个示意图是各自的快照，其阻塞顺序不必相同。

课程连接·状态转换

状态转换通常还伴随队列迁移与现场处理；原语的整体操作不可被中断地执行。

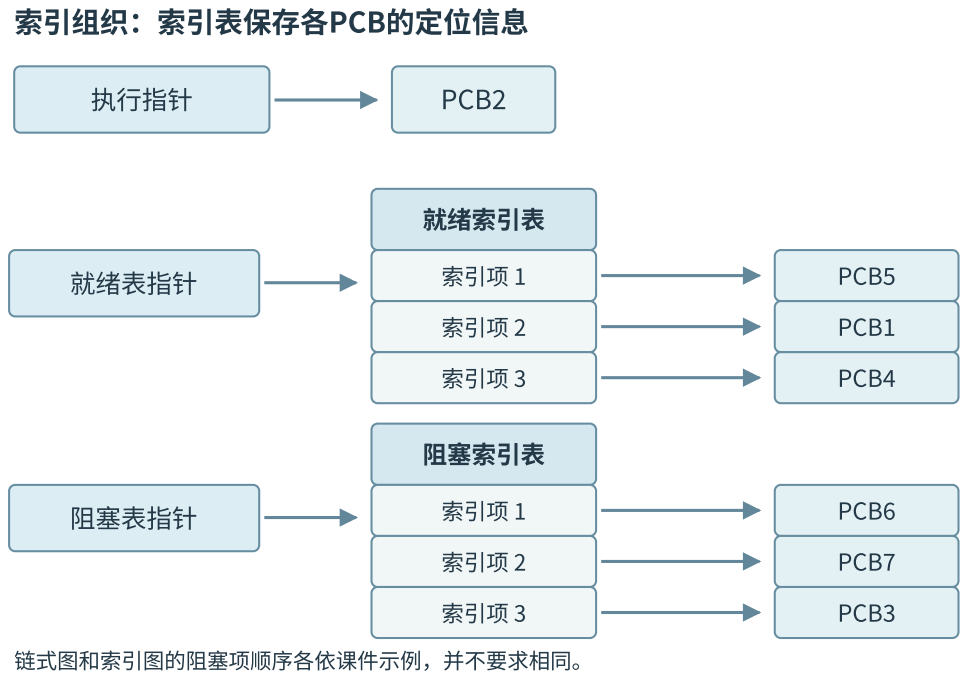


Figure 2.9: 就绪与阻塞索引表分别指向 PCB

辨析·屏蔽信号不是阻塞进程

blocked 位图控制递送，pending 保存待处理事件。暂缓某个信号时进程仍可能执行；进程阻塞态是等待 I/O 或同步条件的另一类状态。

2.2.8 信号：pending、blocked 与用户处理程序

信号用于通知进程某类事件已经发生。发送者可以是获授权的其他进程，也可以是内核。课件以普通信号位向量模型说明：`pending` 记录待处理信号，`blocked` 记录暂时屏蔽的类型；当前可递送集合为

$$D = \text{pending} \ \& \ \sim \text{blocked}.$$

这里的“阻塞一个信号”表示暂缓递送该类型，并不等于使整个进程进入阻塞态。信号到达后先保存在待处理状态，适当时机（例如从内核态返回用户态的路径）检查递送。可执行默认动作，也可在允许时执行进程设置的用户处理程序。处理程序一般运行在用户态；内核负责组织递送并保存、恢复相关执行环境。

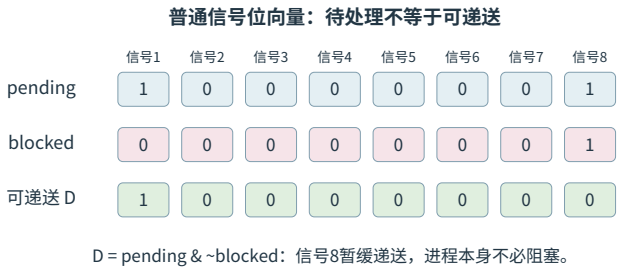


Figure 2.10: 信号 1 与 8 均待处理，信号 8 被屏蔽，因此只有信号 1 可递送

例如按信号编号 1 至 8 排列，待处理位为 `1 0 0 0 0 0 0 1`，屏蔽位为 `0 0 0 0 0 0 1 1`，可递送位为 `1 0 0 0 0 0 0 0`。处理信号 1 后清除其待处理记录；信号 8 仍应保持待处理，直到解除屏蔽且满

足递送条件。普通信号只用一个位表示同类事件时，多次到达可能合并，因此不能据此精确统计事件次数；具有排队语义的信号机制需另按接口说明。

用户处理程序可以改变异常后的处理策略，但并非所有信号都允许捕获、忽略或屏蔽。异常由处理器检测与内核处理；信号是操作系统向进程通知事件的机制。一次缺页可以由内核修复而不向应用递送信号；一次异常也可能使内核进一步向进程发送信号，二者应分层判断。

不变量·等待时间要扣对部分

无 I/O 的单 CPU 段模型满足周转＝就绪等待＋服务；若还有阻塞，必须再加阻塞时间。始终把一个进程的整个生存区间按状态分段相加，再套公式，能避免把等待设备的时间全部算进就绪队列等待。

2.2.9 管道的数据与等待条件

管道是容量有限的内核缓冲数据流。读出通常消耗已有数据，区别于读者—写者模型中“读后仍保留文件内容”的共享读。管道已有数据时，读者不必等待写满整个缓冲区；管道尚有可用容量时，写者也不必等待全部读空。空、满条件决定能否推进，具体一次读写能传多少数据及是否原子，要依据接口和请求长度。

经典半双工模型可用两个管道构成双向同时通信。允许多个读写者的具体系统仍需约定读取分配、消息边界及原子写长度；普通字节流管道不能天然保留应用自定的记录边界。课件为避免多个读者竞争造成应用层数据分配混乱，列出“多个写进程、一个读进程”的组织方案，以及允许多个读者共同读取的方案：后者中一段已读取的数据会从管道中消耗，不会自动复制给每个读者；应用应明确任务分配与记录边界。具体系统的多个读者按内核及调度行为取得数据，不能据此假定严格轮询、公平或广播给所有读者。

2.2.10 调度层次与评价指标

高级调度从后备作业中选作业进入内存并取得参与运行的资格；**中级调度**通过换出、换入调节内存中的进程数量；**低级调度**从就绪实体中选择处理器使用者。并非所有现代系统都以经典批处理作业队列组织这三级调度，但三者处理的对象和时间尺度应区分。

设作业或进程 i 的到达时刻为 A_i，首次运行时刻为 S_i，完成时刻为 C_i，实际服务时间为 B_i。在只含一个 CPU 段、无 I/O、忽略切换开销的题目中：

$$T_i = C_i - A_i, \qquad W_i = T_i - B_i, \qquad R_i = S_i - A_i, \qquad T_i^{\mathrm{w}} = \frac{T_i}{B_i}.$$

T_i 为周转时间，W_i 为就绪等待时间，R_i 为响应时间，T_i^{w} 为带权周转时间。平均值是逐个算完再取算术平均；平均带权周转时间一般不等于平均周转时间除以平均运行时间。

若进程有 I/O 等待，周转时间还包含阻塞时间，此时不能只用 $T_i - B_i$ 当作就绪等待；应再扣除相应阻塞时间。吞吐量是单位时间完成的任务数，CPU 利用率是观察区间内 CPU 忙时长占比。题目是否把切换时间计为“忙”，以及忙于内核开销是否算有效服务，需要按口径说明。

课件例：利用率和吞吐量。单道程序依次使用 CPU 5 秒、打印机 5 秒，再使用 CPU 5 秒后结束；观察区间共 15 秒，其中 CPU 工作 10 秒，打印机工作 5 秒。因此 CPU 利用率为 $10/15 \approx 66.67\%$，打印机利用率为 $5/15 \approx 33.33\%$。若另一观察区间内完成 10 道作业，共用 100 秒，则系统吞吐量为 $10/100 = 0.1$ 道／秒。多道程序执行时应先画出各设备忙闲时间，再按同一观察区间计算。

2.2.11 什么时刻可以调度，抢占改变了哪一步

最小反例·不能挑未来的短任务

在时刻 2 选择作业，时刻 3 才到达的短作业即使只需 1 毫秒也不在候选集合。算法先受 “已到达且就绪” 的约束，随后才比较运行长度或响应比；跳过集合筛选，会把最短优先错误变成预知未来的调度。

当前进程结束、主动阻塞或让出处理器，必须考虑选择其他可运行者；新进程到达、I/O 完成使更合适者就绪、时间片到期，也可能触发重新选择。**非抢占方式**允许当前运行者继续，直到结束或主动放弃；**抢占方式**允许在满足规则的事件点中断当前运行者，把它放回就绪队列。

调度器负责选择，分派器负责把处理器实际交给所选者，包括必要的现场恢复与切换。发生一次中断不必更换运行者，检查一次调度条件也不等于一定发生进程切换。题目询问 “调度次数” 时，应核清是否计首次分派、是否计选择原进程、是否计最后转入空闲；不能把事件点数和切换次数混算。

内核维护调度结构时，短小临界区等处不允许任意抢占，否则可能读到队列的半更新状态；处理中断的阶段也有相应约束。所谓内核可抢占，并非每条内核指令都可以无条件中断并调度。

普通临界区访问的是需互斥的普通资源，例如打印机；进程持有相应互斥权期间，仍可能被调度切换，其他进程不能因此使用该资源。课件所说的内核程序临界区用于访问内核管理数据结构，例如就绪队列，执行有关原子操作时不能调度。判断能否切换应结合临界区和系统的具体约束，不能仅凭 “处于临界区” 作一概判断。

2.2.12 调度器与闲逛进程

调度器在允许调度的事件点检查就绪集合并选择执行者。一次检查可能继续选中当前进程；只有实际改变执行者时才进行相应上下文切换。非抢占策略下，新进程到达或 I/O 完成可使其就绪，但通常不能仅因其更优就夺走当前执行者的 CPU；抢占策略则可在规定的时钟、唤醒等事件点重新比较。

没有普通就绪进程时，系统运行**闲逛进程（idle）**或相应空闲线程。它的优先级最低，通常执行低功耗等待等操作，仍需允许中断使系统重新处理事件。空闲线程处于运行状态不意味着 CPU 正在进行有效用户计算；计算利用率时，须依据题目是否把空闲、内核管理和切换分别计入。

2.2.13 沿时间线执行调度算法

处理调度题时，先明确到达时间、服务时间、抢占规则、同刻事件次序及开销。每到一个可能改变选择的事件点，只在**已经到达且就绪**的进程中选择，记录剩余 CPU 时间。最后用完成时刻计算指标，并检查 CPU 总服务量是否与题表一致。

FCFS、SJF、SRTF 与高响应比

先来先服务 FCFS 按就绪到达次序选择，通常不抢占。长 CPU 段在前时，后面的短进程和 I/O 型进程可能等很久，产生护航效应。**短作业/短进程优先 SJF** 在选择时挑预计运行时间最短者，通常指非抢占版本；它需要预测运行时间，持续到来的短任务可能使长任务饥饿。

最短剩余时间优先 SRTF 是抢占版本，新进程到达时比较的是 “新进程的服务时间” 与 “当前进程剩余时间”，不能拿当前进程最初总时间比较。**高响应比优先 HRRN** 通常非抢占，选择时计算

$$\text{响应比} = \frac{\text{等待时间} + \text{预计服务时间}}{\text{预计服务时间}} = 1 + \frac{W}{B}.$$

等待越久，响应比越大，有助于缓解长任务长期等候。这个 “响应比” 是调度优先量，与前节的 “响应时间” 不是同一个指标。

课程连接·调度时机

进程阻塞、终止或被抢占可能触发调度；内核临界区等场合需结合系统规则判断能否切换。

例 2A-2　先排除尚未到达者，再比较

例 2.2.2 (配套例题)

【配套例题】4 个作业到达系统的时刻和运行时间如下表所示。

作业	到达时刻	运行时间
J1	0	3
J2	1	3
J3	1	2
J4	3	1

系统在 $t=2$ 时开始作业调度。若分别采用先来先服务和短作业优先调度算法，则选中的作业分别是（　）。

A. J2、J3　　B. J1、J4　　C. J2、J4　　D. J1、J3

解析。此时只有 J1、J2、J3 到达。FCFS 选择最早到达的 J1；SJF 比较三者运行时间 3、3、2，选择 J3。答案为 **D**。J4 尚未到达，即使其时间最短也不能提前选择。

原创变式。沿用本题数据，仅把算法改为 HRRN，检查它与 SJF 是否等同。$t=2$ 时，J1、J2、J3 的响应比分别为 $1+2/3$、$1+1/3$、$1+1/2$，应选 J1。改变的是优先量，因此不能仍然只比较运行时长。

课件算例：同一组进程比较 FCFS、SJF、SRTF 和 HRRN

四进程的到达时刻为 $(0,2,4,5)$，CPU 服务时间为 $(7,4,1,4)$，依次对应 P_1 至 P_4。均只有一个计算段，忽略切换开销；同优先量按先到达者选择。

算法	按时间先后的运行片段	完成时刻 (C_1,C_2,C_3,C_4)
FCFS	$P_1:0\sim7$，$P_2:7\sim11$，$P_3:11\sim12$，$P_4:12\sim16$	$(7,11,12,16)$
非抢占 SJF	$P_1:0\sim7$，$P_3:7\sim8$，$P_2:8\sim12$，$P_4:12\sim16$	$(7,12,8,16)$
SRTF	$P_1:0\sim2$，$P_2:2\sim4$，$P_3:4\sim5$，$P_2:5\sim7$，$P_4:7\sim11$，$P_1:11\sim16$	$(16,7,5,11)$
HRRN	本例与非抢占 SJF 相同	$(7,12,8,16)$

FCFS 的周转时间为 $(7,9,8,11)$，等待时间为 $(0,5,7,7)$，带权周转为 $(1,9/4,8,11/4)$；平均值分别为 8.75、4.75、3.5。

非抢占 SJF 的周转时间为 $(7,10,4,11)$，平均周转为 8，平均等待为 4，平均带权周转为 $41/16=2.5625$。SRTF 的周转时间为 $(16,5,1,6)$，平均周转为 7，平均等待为 3，平均带权周转为 $169/112\approx1.5089$。计算时保留分数到最后，避免把 $16/7$ 先截成 2.28 影响平均值。

HRRN 在时刻 7 比较：P_2 的响应比为 $9/4$，P_3 为 4，P_4 为 $6/4$，故选 P_3；时刻 8 再比较 P_2 的 $10/4$ 与 P_4 的 $7/4$，选 P_2。本例顺序相同只

是数据造成，不能把 HRRN 当成 SJF 的同义词。

“平均等待时间最少” 等最优性结论必须附模型条件，例如所有作业同时可运行、已知运行时间、忽略切换成本等。最少平均周转也不自动等价于最少平均带权周转。

课程连接·闲逛进程

就绪队列没有普通可运行进程时，闲逛进程让处理器处于可管理的等待状态。

优先级：先看数值方向，再看是否抢占

优先级可以由任务性质静态规定，也可以随等待、CPU 使用等动态调整。数值大代表高还是低，必须按题目读；同为 “优先级调度” 不保证使用同一方向。提高长等候者优先级称为老化，有助于避免低优先级进程饥饿；实时任务则常需要按紧迫性或期限组织优先次序。

同组进程的抢占与非抢占优先级

四个进程的到达时间为 $(0, 2, 4, 5)$，运行时间为 $(7, 4, 1, 4)$，优先数分别为 $(1, 2, 3, 2)$；数值越大优先级越高，同优先级按进入就绪队列的先后选择，忽略切换开销。

模式	执行区间
非抢占	P1[0,7) → P3[7,8) → P2[8,12) → P4[12,16)
抢占	P1[0,2) → P2[2,4) → P3[4,5) → P2[5,7) → P4[7,11) → P1[11,16)

非抢占时 P1 在 7 时完成，此时已到达的 P3 优先级最高；8 时 P2 与 P4 同级，P2 先入队。抢占时 P2 在 2 时到达并抢占 P1，P3 在 4 时到达并抢占 P2；5 时 P3 结束且 P4 到达，P2 已经在队列中，故先运行 P2。其轨迹与这一组数据下的 SJF/SRTF 轨迹碰巧相同，但选择依据分别是优先级与服务时间。

例 2A-3　被抢占者的剩余时间必须保留

例 2.2.3 (配套例题)

【配套例题】进程进入就绪队列的时刻、优先级（值越大优先权越高）及 CPU 执行时间如下表所示。

进程	到达	优先级	CPU 时间
P1	0 ms	1	60 ms
P2	20 ms	10	42 ms
P3	30 ms	100	13 ms

采用基于优先权的抢占式 CPU 调度算法，从 0 ms 开始调度，三个进程的平均周转时间为（ ）。

A. 60 ms　　B. 61 ms　　C. 70 ms　　D. 71 ms

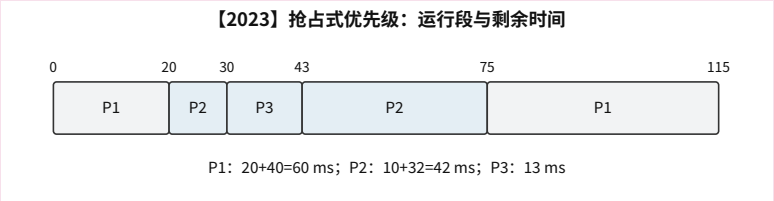


Figure 2.11: 每次抢占保留剩余时间，不重新开始整个 CPU 段

最小反例·不能挑未来的短任务

在时刻 2 选择作业，时刻 3 才到达的短作业即使只需 1 毫秒也不在候选集合。算法先受 “已到达且就绪” 的约束，随后才比较运行长度或响应比；跳过集合筛选，会把最短优先错误变成预知未来的调度。

解析。0—20 ms 只有 P1，执行 20 ms 后还余 40 ms。P2 到达并抢占，执行至 30 ms，还余 32 ms。P3 优先级最高，30—43 ms 执行完毕；随后 P2 执行 32 ms，于 75 ms 完成；最后 P1 执行 40 ms，于 115 ms 完成。

三者周转时间依次为 $115-0=115$ ms、$75-20=55$ ms、$43-30=13$ ms，平均为 $(115+55+13)/3=61$ ms，答案为 **B**。完成时刻之和不是周转时间之和；每项都要减各自到达时刻。

例 2A-4　一次到达不必增加一次进程调度

例 2.2.4 (配套例题)

【配套例题】P0、P1、P2、P3 的到达、优先级（值越小优先权越高）和执行时间如下表所示。采用基于优先权的抢占式进程调度，从 0 ms 开始，到四个进程都运行结束为止，发生进程调度的总次数为（ ）。

进程	到达	优先级	CPU 时间
P0	0 ms	15	100 ms
P1	10 ms	20	60 ms
P2	10 ms	10	20 ms
P3	15 ms	6	10 ms

A. 4　　B. 5　　C. 6　　D. 7

解析。实际运行段依次为 P0[0,10)、P2[10,15)、P3[15,25)、P2[25,40)、P0[40,130)、P1[130,190)。P1 在 10 ms 到达，但优先级低于当时可选者，不单独取得一个运行段。按题目计首次分派且不计最后转入空闲，六次分派，答案为 **C**。若只数相邻两个实际运行进程的更换，则是五次；两种口径不可混用。

时间片轮转与多级反馈队列

课程连接·调度时机

进程阻塞、终止或被抢占可能触发调度；内核临界区等场合需结合系统规则判断能否切换。

时间片轮转 RR 按就绪队列的先后，让每个进程取得至多一个时间片。进程在时间片内结束或阻塞，就提前让出；用完时间片仍未结束者回到队尾。时间片过长会接近 FCFS，过短会增加切换开销。若每次有效执行 q、切换耗时 s，持续满负荷时有效服务占比可近似写为 $q/(q+s)$，但最后一个短段等边界应逐段算。

同刻有新进程到达又有时间片到期时，谁先入队可能改变结果，应按题设；题设没有规定且确实影响答案时，必须声明采用的次序。不能把刚被抢占的进程总放在队首，新到者也不能越过当前时间片先运行。

王道课件：用同一组进程比较时间片。四个进程 P1、P2、P3、P4 分别在 0、2、4、5 到达，服务时间分别为 5、4、1、6；忽略切换开销。同一时刻有新进程到达和时间片到期时，先将新到进程入队，再将未完成的当前进程放队尾。

时间片	执行顺序	依次结束时刻
2	P1、P2、P1、P3、P2、P4、P1、P4、P4	2、4、6、7、9、11、12、14、16
5	P1、P2、P3、P4、P4	5、9、10、15、16

例如时间片为 2 时，时刻 2 的 P2 先入队，随后未完成的 P1 回到队尾，因此下一段执行 P2；时刻 4 的新到 P3 也先于 P2 回队，但 P1 已经在队首，故 4～6 运行 P1。时间片为 5 时，P1、P2、P3 各可在一个时间片内结束，P4 仍需两个时间片；连续两片均为 P4，并非它的服务时间只有 5。若同刻入队规则或切换开销改变，队列轨迹和计时应重新计算。

多级队列把不同类型进程固定放在不同队列；**多级反馈队列**允许进程在队列间移动，常用较高队列的短时间片及时响应交互，CPU 使用时间较长者逐级下移。队列间是否抢占、各级规则和提升规则必须逐项读取，不能靠固定模板代替题设。

固定多级队列的队内、队间规则

将系统进程、交互进程和批处理进程分别放在三个队列。例中队内分别采用优先级、RR 和 FCFS；队列之间可以采用固定优先级，也可以按 50%、40%、10% 的比例分配处理器时间。这是两种不同的队间规则：采用固定优先级时，较高优先级队列为空，较低优先级队列才能被选中；采用比例分配时按所给份额安排。

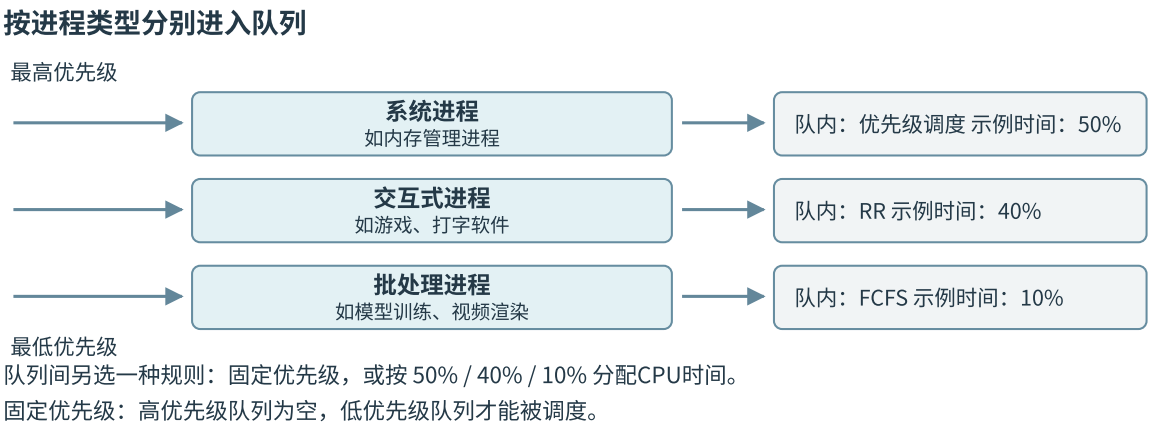


Figure 2.12: 系统、交互和批处理三个队列的队内策略及队间份额示例

例 2A-5　二级反馈中，第二级按剩余量选择

课程连接·闲逛进程

就绪队列没有普通可运行进程时，闲逛进程让处理器处于可管理的等待状态。

例 2.2.5 (配套例题)

【配套例题】系统采用二级反馈队列调度。Q1 采用时间片轮转，时间片为 10 ms；Q2 采用短进程优先。系统优先调度 Q1，Q1 为空才调度 Q2；新创建进程先进入 Q1，在 Q1 执行一个时间片后仍未结束则转入 Q2。当前两队列为空，系统依次创建 P1、P2 后开始调度，两者需要的 CPU 时间分别为 30 ms、20 ms。平均等待时间为（　）。

A. 25 ms　　B. 20 ms　　C. 15 ms　　D. 10 ms

解析。P1 在 Q1 执行 0—10 ms，余 20；P2 执行 10—20 ms，余 10。此时 Q1 为空，Q2 中 P2 剩余较短，先执行 20—30 ms 完成，再运行 P1 至 50 ms。两者初始时均已创建，等待分别为 $50-30=20$ ms、$30-20=10$ ms，平均 15 ms，答案为 **C**。转入 Q2 以后，已经执行的时间不能重复计算。

例 2A-6　同一组到达事件，SRTF 与 RR 如何分别推进

例 2.2.6 (配套例题)

〖2.2-006〗系统有 4 个进程，预计运行时间和到达就绪队列的时刻如下，单位均为时间配额。分别采用可抢占式短进程优先和时间片轮转调度（时间片为 2），求各进程的调度次序与平均周转时间。

进程	到达时刻	预计运行时间
P1	0	8
P2	1	4
P3	2	9
P4	3	5

解析。SRTF 在 0 时先选 P1；到 1 时 P1 余 7，而 P2 只需 4，故抢占。到 2 时 P2 余 3，小于新来的 P3 的 9；到 3 时 P2 余 2，小于 P4 的 5，均不抢占。P2 在 5 完成后，等待者的剩余量依次为 P1 的 7、P3 的 9、P4 的 5，因此先 P4，再 P1，最后 P3。

进程	SRTF 实际运行区间	周转时间
P1	[0,1)、[10,17)	17
P2	[1,5)	4
P3	[17,26)	24
P4	[5,10)	7

平均周转为 $(17+4+24+7)/4=13$。P3、P4 到达都使调度器重新比较，但比较后可以继续当前 P2，不能把每次到达都强制切换。

RR 按本题参考解采用 “同刻新到者先入队，再把时间片到期者放队尾” 的约定。P1 在 [0,2) 后回队尾，此时队列为 P2、P3、P1；P4 在 3 到达，排在当时的等待队尾。以后按同样规则，每次只扣实际使用的一个时间片；剩余不足 2 者提前结束。

进程	RR 实际运行区间	周转时间
P1	[0,2)、[6,8)、[14,16)、[20,22)	22
P2	[2,4)、[10,12)	11
P3	[4,6)、[12,14)、[18,20)、[23,25)、[25,26)	24
P4	[8,10)、[16,18)、[22,23)	20

平均周转为 $(22+11+24+20)/4=19.25$。最后只有 P3，它在 25 用完一个片后仍可再次获选，不能因为 “轮转” 而强行等待一个不存在的其他进程。若同刻入队约定改变，2 时的队列及以后轨迹也要随之重算。

最小反例·不能挑未来的短任务

在时刻 2 选择作业，时刻 3 才到达的短作业即使只需 1 毫秒也不在候选集合。算法先受 “已到达且就绪” 的约束，随后才比较运行长度或响应比；跳过集合筛选，会把最短优先错误变成预知未来的调度。

调度算法的选择依据与抢占条件

下表比较的是前述 CPU 调度规则。“需要时长” 指选择时需要已知或估计的 CPU 服务量，不是说运行时间必须由用户准确预报。各种算法都只在已到达、已就绪的实体中选择。

算法	下一个运行者怎样选	是否抢占	是否需要预计 CPU 时长
FCFS	按进入就绪队列的先后	通常不抢占	不需要
SJF	预计服务时间最短者	本章指非抢占版	需要预计服务时间
SRTF	预计剩余时间最短者	可以抢占	需要预计剩余时间
HRRN	选择 $1+W/B$ 最大者	通常不抢占	需要预计服务时间 B
优先级	按题设优先级高低	两种版本都有	通常不以时长为必需输入
RR	队首运行至片满、结束或阻塞	片满时抢占	不需要
多级反馈队列	先选队列，再执行该队列规则	常见模型允许高队列抢占	不需事先估计总服务时间

算法	等待方面的主要边界
FCFS	正常推进时不因后到者插队而饥饿；短任务可能被长任务拖住
SJF	持续到来的短任务可能使长任务饥饿
SRTF	比较剩余量；持续短任务同样可能阻挡长任务
HRRN	等待使响应比增加，兼顾长短任务；不等于固定等待上界
优先级	固定低优先级者可能饥饿；老化可逐步提高其优先级
RR	同队列轮流获得机会；片过小会增加切换开销
多级反馈队列	若高队列始终有任务，低队列可能长期无机会；是否定期提升须看规则

“短任务优先” 不自动包含抢占；“按优先级” 也不自动包含时间片。题干同时给出多个规则时，先判断哪个规则决定候选队列，再判断队内顺序，最后处理到达、片满和阻塞等事件。

例 2A-7　把 “等待越久越优先” 写成动态优先数

例 2.2.7 (配套例题)

【配套例题】某进程调度程序采用基于优先数（priority）的调度策略，即选择优先数最小的进程运行。进程创建时由用户指定一个 nice 作为静态优先数。为了动态调整优先数，引入运行时间 cpuTime 和等待时间 waitTime，初值均为 0。进程处于执行态时，

cpuTime 定时加 1，且 waitTime 置 0；进程处于就绪态时，cpuTime 置 0，waitTime 定时加 1。请回答下列问题。

（1）若调度程序只将 nice 的值作为进程的优先数，即 priority=nice，则可能会出现饥饿现象，为什么？

（2）使用 nice、cpuTime 和 waitTime 设计一种动态优先数计算方法，以避免产生饥饿现象，并说明 waitTime 的作用。

解析。（1）数值小才优先。若就绪队列中一直有 nice 较小的进程，nice 较大的进程便可能一直落选。系统仍有其他进程在运行，因此这是饥饿，不能解释成全部进程互等的死锁。

（2）一种方法是

$$priority = nice + k_1\, cpuTime - k_2\, waitTime, \qquad k_1 > 0,\ k_2 > 0.$$

运行时间增长使优先数增大，降低持续占用 CPU 者的优先级；就绪等待增长使优先数减小，提高长期等待者的优先级。两个符号都由“数值越小越优先”决定，不能沿用数值越大越优先的公式。

例如仅为检验公式方向，取 $k_1 = k_2 = 1$：nice=10 的等待者在等待 11 个计时单位后，优先数成为 -1，比 nice=0 的刚就绪进程更小，因而先被选中。这是**原创代入检查**，不是原题新增数据。等待者一旦运行，waitTime 按题设清零；再次就绪时 cpuTime 清零，不能把这两个量误当成从创建起永久累计的总量。

waitTime 提供的是老化机制。但若把它用作无饥饿的保证，还需满足一些前提：采用通常的有限进程或有界 nice 模型，持续计时和重算，适当抢占或有限运行段，以及同优先数下不无限越过等待者。不能把一个公式解释成对任意到达流和任意调度实现都存在固定等待上界。

多级队列与多级反馈：是否允许改变所属队列

多级队列调度按任务类型分设队列，进程通常固定归属某类队列。队列之间可用固定优先级，也可按时间比例分配 CPU；各队列内部可分别使用优先级、RR 或 FCFS 等策略。固定优先级下，高级队列持续非空可能使低级队列饥饿。

多级反馈队列还允许根据实际运行行为在队列间移动。新进程先进入较高队列，时间片逐级增大，用满时间片且未完成者下降；较高队列出现就绪者可抢占较低队列。是否保留剩余时间片、被抢占者回到队首还是队尾、I/O 返回后进入哪一级，都必须采用题目给定规则。

课件例子使用三级时间片 1、2、4，P_1(到达0, 运行8)、$P_2(1,4)$、$P_3(5,1)$。按课件被抢占者回原队列尾、再次轮到时分配该级时间片的规则，CPU 片段为：

$$P_1(1) \to P_2(1) \to P_1(2) \to P_2(1) \to P_3(1) \to P_2(2) \to P_1(4) \to P_1(1).$$

括号为此次实际运行时长，时间端点依次为 $0, 1, 2, 4, 5, 6, 8, 12, 13$。时刻 5 的抢占没有把 P_2 原总服务时间重新计入，只改变其尚未完成部分的后续安排。

2.2.14 多处理机调度：负载与缓存都有代价

多个处理器可以使用公共就绪队列，也可以各自维护队列。公共队列便于共享负载，但并发访问需同步；每核队列降低争用，却可能出现一核

课程连接·迁移与 Cache 局部性相互影响

把线程移到空闲核可减少排队，却可能丢掉此前核的 Cache 与 TLB 热状态。组成原理的命中率和操作系统的负载均衡在这里共同影响总时间；每核任务数量相等也不保证剩余工作量相等。

排队、一核空闲，需要负载均衡。课件中的**推迁移**由专门的系统程序周期性检查各处理器的负载，将任务从繁忙处理器迁往较空闲的处理器；**拉迁移**由空闲处理器的调度程序主动从繁忙处理器取得任务。不同系统可采用其他负载监测与迁移实现，因此应按题目描述识别触发者。

处理器亲和性倾向让线程继续在此前运行的处理器上执行，利用缓存和地址转换缓存中的内容。软亲和性是调度倾向，硬亲和性是允许处理器集合的明确限制。负载均衡可能要求迁移，缓存局部性可能要求保留，二者需要权衡；不能把“每核进程数相同”当作负载完全相同，因为各任务剩余计算量可能不同。

同时多线程 SMT 让一个物理核维护多个硬件线程现场，共享某些执行资源；它不等于每条逻辑处理器都具有独立完整的执行单元。调度题未给多处理机规则时，不要自行把一个进程拆到多个核同时加速。

课程连接·迁移与 Cache 局部性相互影响

把线程移到空闲核可减少排队，却可能丢掉此前核的 Cache 与 TLB 热状态。组成原理的命中率和操作系统的负载均衡在这里共同影响总时间；每核任务数量相等也不保证剩余工作量相等。

2.2.15 中断执行后，PCB 怎样保存运算中间状态

考虑 `int x=1; x++;` 的一个教学指令序列：指令 1 把 1 写入内存变量 x；指令 2 把 x 读入通用寄存器；指令 3 使寄存器加 1；指令 4 将寄存器结果写回 x。执行完指令 3 而尚未执行指令 4 时，内存中的 x 仍为 1，通用寄存器为 2，PC 指向指令 4。

如果此刻切换到另一个进程，内核须先将前一进程的 PC、程序状态字 PSW 和必要寄存器值保存到其 PCB。后一个进程可把自己的通用寄存器改为 250；该值属于它自己的执行现场。再切回前一进程时，从其 PCB 恢复 PC 与寄存器值 2，接着执行指令 4，内存中的 x 才变为 2。保存寄存器现场并非把其他进程运行时的 250 当作前一进程结果；不恢复正确现场就可能把错误值写回。

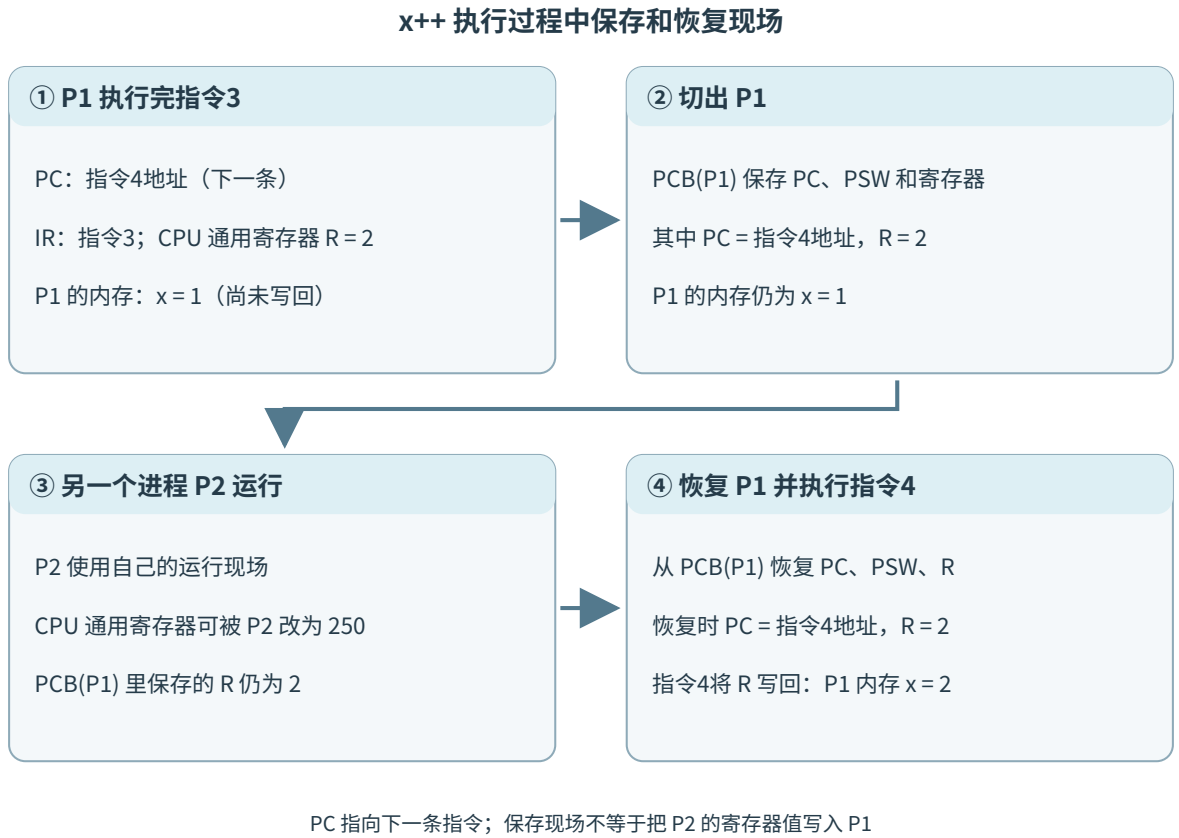


Figure 2.13: x++ 过程中保存 PCB、切换进程、恢复 PC 与寄存器并写回的四阶段

2.2.16 多处理器就绪队列与负载迁移

公共就绪队列供多个 CPU 选择任务，访问共享队列要互斥；它便于分配负载，但线程迁往其他处理器时可能损失原有缓存亲和性。软亲和性是调度器尽量让线程留在原处理器，硬亲和性由进程或系统明确限定允许执行的 CPU 集合。

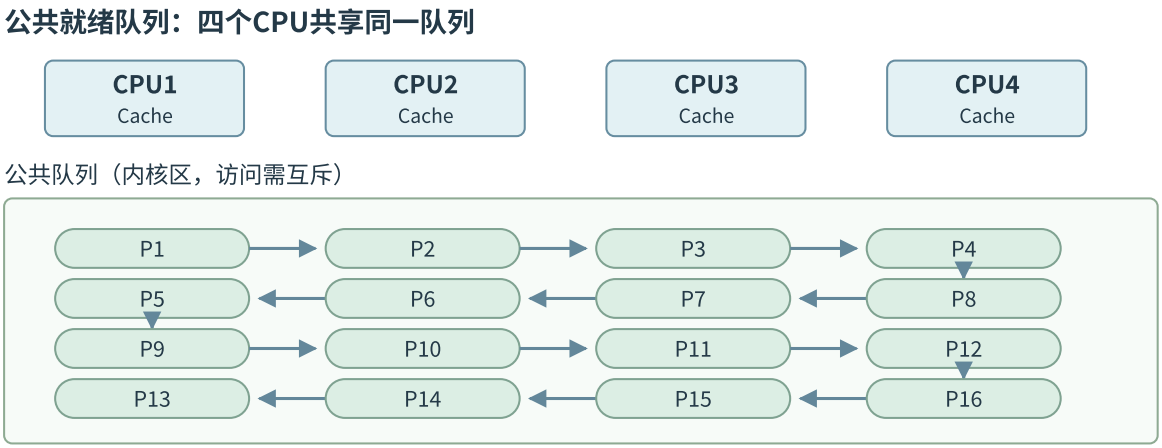


Figure 2.14: 四个 CPU 共享同一个就绪队列

使用私有队列时，假设四个 CPU 的负载依次为 50%、80%、80%、95%。将 P16 从 CPU4 队列迁至 CPU1 后，示例负载变为 75%、80%、80%、70%。Push 由专门系统程序定期检查负载并推动迁移；Pull 由空闲或负载低的处理器的调度程序从忙处理器拉取任务。数值只是该示例的一次状态变化，不能推断每次迁移都改变相同百分比。

课程连接·迁移与 Cache 局部性相互影响

把线程移到空闲核可减少排队，却可能丢掉此前核的 Cache 与 TLB 热状态。组成原理的命中率和操作系统的负载均衡在这里共同影响总时间；每核任务数量相等也不保证剩余工作量相等。

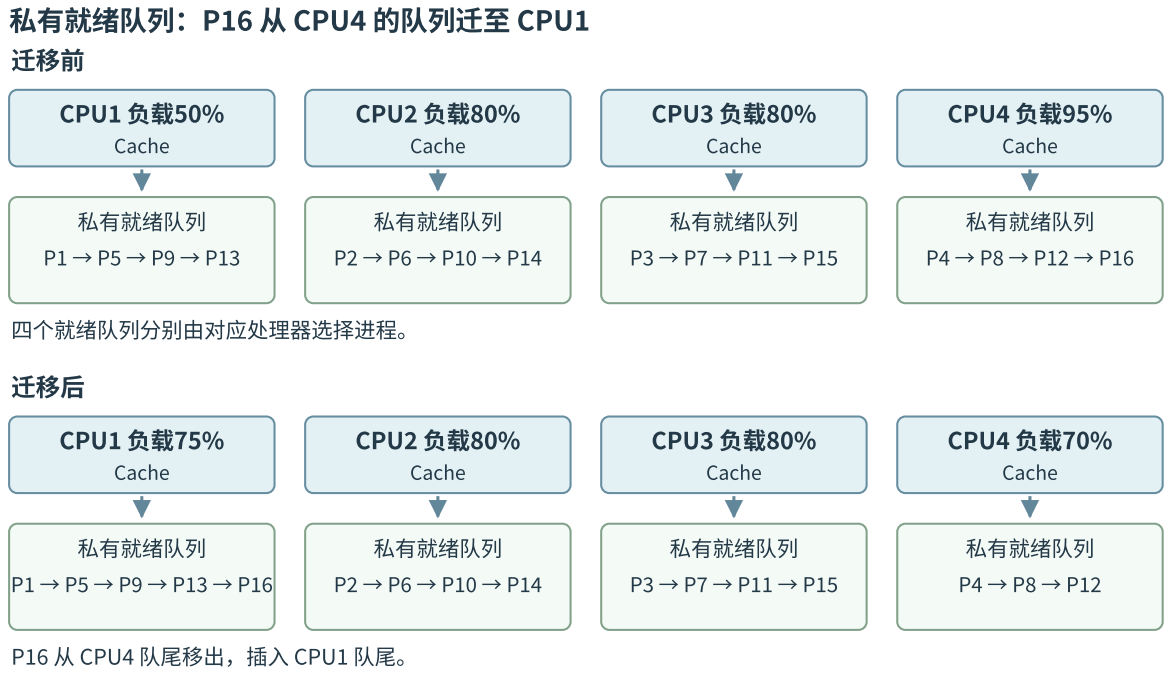


Figure 2.15: 私有队列负载均衡前后，P16 从 CPU4 迁至 CPU1

课程连接·两种切换

用户态进入内核态不必更换当前进程；发生上下文切换则需要保存和恢复执行现场。

2.2.17 上下文切换与模式切换

上下文是恢复执行所需的状态，包括程序计数器、寄存器、栈等；进程切换还可能更换地址空间相关状态。**模式切换**只是当前执行权限模式改变，例如同一进程进入系统调用再返回；它不必改变执行身份。**上下文切换**则把 CPU 从一个执行实体交给另一个。

同进程的线程切换通常要换 PC、寄存器和栈，而不必更换进程页表基址和共同的文件资源。跨进程切换还要建立新地址空间环境；TLB 是否必须全部清空，取决于是否支持地址空间标识等硬件条件。不能把“跨进程切换一定清空全部 Cache”列作一般步骤。

〖2.2-009〗同一进程中 T1 切到 T2，需要更新各自执行位置和栈环境；进程页表和进程打开文件资源并不因同进程线程切换而自然改变。判断题列出的每项寄存器，要先说明它保存线程状态还是进程共享状态。

【配套例题】“执行系统调用”完成并返回调用者时，可以使 CPU 从内核态回到用户态；阻塞、唤醒和执行 CPU 调度本身属于内核管理路径，不能认为这些操作完成时都必然返回用户态。具体返回到哪个线程，由当时调度与返回路径决定。

2.2.18 分支自测：用相同时间线检查不同问题

以下为原创回顾设问。

1. 已等到 I/O 数据的进程是否立即运行？线程发出 I/O 请求是否必然阻塞？

2. 在例 2A-3 中，P2 的响应时间、等待时间、周转时间分别是多少？
3. 同一进程的两个内核线程切换，是否必须改页表基址？
4. 预计服务时间相同的两个作业，HRRN 如何反映等待差别？
5. 多核上迁移线程，一定能提高性能吗？

课程连接·同步与互斥

同步约束事件先后，互斥限制临界资源同时访问；同一场景常同时需要二者。

自测答案

1. 数据到达使相应等待条件满足，通常先就绪，再等调度。若请求非阻塞或数据已可用，I/O 接口可不使线程阻塞。
2. P2 在到达 20 ms 时立即开始，响应时间 0；周转时间 55 ms，服务 42 ms，所以等待 13 ms。
3. 通常不必，二者使用同一进程地址空间，但 PC、寄存器和线程栈等要切换。
4. B 相同则 $1 + W/B$ 随 W 增大，等待较久者响应比更高。
5. 不一定。迁移可能缓解排队，也可能丢失缓存局部性、产生同步和迁移成本。

2.3 同步、互斥与信号量

进程合作时，需要回答两个不同的问题：一项操作必须等哪项操作完成，几个进程能否同时操作同一份共享数据。前者决定先后次序，后者决定互斥范围。信号量可以表达这两种约束，但其含义、初值和放置位置都要从题目条件推出。

本章代码是描述并发关系的伪代码。`P(S)` 与 `wait(S)` 同义，`V(S)` 与 `signal(S)` 同义；条件变量的 `wait`、`signal` 会另行定义。进程编号与数组下标从 0 开始。除特别说明外，信号量采用本章的记录型语义，不假定等待队列按先进先出调度。

2.3.1 临界区约束什么，为什么还需要同步

从一次丢失的更新看原子操作

两个进程共享整数 `x`，初值为 0，各执行一次 `x=x+1`。一条高级语言语句未必对应一条不可分割的机器操作。若每次更新分为 “读入本地寄存器—加 1—写回”，就可能出现下图的交错：两个进程都读到 0，最后都写入 1，结果只有 1。

次序	进程 A	进程 B	共享 x
1	读 x：rA=0		0
2		读 x：rB=0	0
3	rA=rA+1=1		0
4		rB=rB+1=1	0
5	写 x=rA		1
6		写 x=rB	1

Figure 2.16: 两次更新都完成，但后一次写回覆盖了前一次结果

即使只有一个 CPU，进程也可能在读与写之间被切换出去，所以单处理机上同样存在竞争。多处理机上还可能真正同时执行。这里把某

段操作视为**原子操作**，是说其他并发操作无法插入其内部，也无法观察到破坏该操作语义的中间状态；不能仅凭“这是一行代码”认定原子性。

进程在运行态时占用处理机；在就绪态时具备运行条件，等待调度；在阻塞态时等待事件，暂时不能被调度执行。因资源不足而主动阻塞，与反复执行检查指令的忙等待不同。唤醒把阻塞进程变为就绪进程，不保证它立即运行。

课程连接·临界区

同一临界资源应由有效的互斥协议保护；剩余区无需因此全部串行。

资源、代码与先后关系

临界资源是在同一时段只允许一个进程使用的共享资源。**临界区**是进程中访问相应临界资源的代码段。进入区负责申请进入，退出区负责释放；二者不属于被保护的临界区本身。资源是一份共享对象，相关临界区可以有多段，分布在不同进程中。

〖2.3-027〗5 个并发进程各有一段访问同一变量 A 的代码，对应 5 个相关临界区。临界区要按代码段计数，不能按共享变量的份数计数。一般程序若在多处访问共享数据，也不能仅凭进程数推出代码段总数。

互斥限制重叠：某进程正在访问临界资源时，其他需要访问它的进程须等待。**同步**规定合作中的先后关系：后续操作需要某项结果，就必须等产生该结果的操作完成。同一对进程可能同时受这两种约束。

例 2B-1　共享缓冲区中的两种约束

例 2.3.1 (配套例题)

〖2.3-011〗进程 A 和进程 B 通过共享缓冲区协作完成数据处理，进程 A 负责产生数据并放入缓冲区，进程 B 从缓冲区读数据并输出。进程 A 和进程 B 之间的制约关系是（　）。

A. 互斥关系　　B. 同步关系　　C. 互斥和同步关系　　D. 无制约关系

解析。进程 B 必须在进程 A 放入数据后读取，这是同步；对共享缓冲区的访问还须协调，避免操作冲突，这是互斥。答案为 **C**。选项 B 只概括了先后关系，没有涵盖互斥。

本题指出两类约束，并不意味着任何实现都必须恰好使用“一个同步量加一个互斥量”。操作是否反复执行、容量是否为 1，以及同步本身是否已排除重叠，都会影响实现；例 2B-4会分别处理。

怎样判断互斥方案是否合格

检查一种方案时，先找违反条件的具体交错，不要只看顺序执行的结果。

检查项	要问的问题
空闲让进	临界区无人使用，是否仍被一个不参与竞争的进程挡住?
忙则等待	一个进程尚未退出，另一个能否进入同一临界区?
有限等待	请求提出后，其他进程获准进入的次数是否存在有限上界？还须满足进程持续获得执行机会、临界区操作有限等前提。
让权等待	不能进入时，是阻塞并让出处理机，还是继续占用 CPU 检查?

前三项分别检查进入机会、互斥安全和等待上界；让权等待检查等待方式。一个自旋方案可能保证互斥，却不满足让权等待。**无死锁不等于无饥饿**：系统整体仍在前进，但某个进程可能反复被越过。

错项排除·原子锁不自动保证公平

TestAndSet 保证检查旧值和置位不可被插入，因而可以建立互斥；它没有规定等待者先后次序。某进程持续抢不到锁仍可能饥饿，所以 “硬件原子指令保证有限等待” 缺少排队、公平和持续运行等条件。

2.3.2 软件互斥与锁：检查和占用必须连起来

三种失败方案留下了什么问题

以下双进程模型假定单次共享变量读写原子、内存顺序与伪代码一致。`flag[i]` 表示进程 P_i 想进入。现代机器上的编译器重排与存储重排不在这些伪代码的默认模型内，实际程序还需相应原子变量与内存序保证。

方案	进入区的核心操作	一条足以否定它的执行情况
单标志法	等待 `turn==i`，退出时将机会交给对方	`turn=0`，但 P_0 一直在剩余区工作（不访问该临界资源的其他代码）；P_1 不能进入空闲临界区。
双标志先检查	先等对方 `flag` 为假，再置自己为真	双方都在对方置真前完成检查，随后同时进入，不能保证互斥。
双标志后检查	先置自己为真，再等对方为假	双方先置真，都等待对方退出；此时没有任何一方进入，无法执行清标志。

这些模型与反例见王道书 §2.3。先检查法的问题是 “检查与声明占用分离”；简单调换两句又可能造成双方互等。

Peterson 算法怎样处理双方同时申请

Peterson 算法用 `flag` 表达意愿，用 `turn` 在双方都想进入时决定谁先进入。两个 `flag` 初值均为 `false`；对 $i \in \{0,1\}$，令 $j = 1 - i$：

```
flag[i] = true;
turn = j;
while (flag[j] && turn == j) { }  // 忙等待
访问临界资源;
flag[i] = false;
```

若只有自己请求，对方的 `flag` 为假，可直接进入。若双方都请求，两次对 `turn` 的写入总有先后：后写的一方把进入机会让给对方，自己等待。退出时清除自己的意愿，等待者随后通过检查。

还须排除 “一个已进入，另一个又进来” 的情况。假设 P_i 在临界区，此时 `flag[i]` 为真；后来的 P_j 声明意愿并将 `turn` 置为 `i`，就必须等待。如果双方已在进入区竞争，后一次 `turn` 写入同样使其中一方等候。由此排除同时进入。在上述内存模型、进程继续执行且临界区有限等前提下，它也能保证有限等待；它仍在循环检查，不能让权等待。

TestAndSet 为什么能够上锁

硬件提供的 TestAndSet（TS，也称 TSL）以一次原子操作返回 `lock` 的旧值，并把 `lock` 置为真。以下只是该指令的逻辑含义，不能把普通语言实现的三句代码直接当作原子指令：

```
TestAndSet(lock):
    old = lock
    lock = true
    return old

// lock 初值 false
while (TestAndSet(lock)) { }
访问临界资源;
lock = false;
```

错项排除·原子锁不自动保证公平

TestAndSet 保证检查旧值和置位不可被插入，因而可以建立互斥；它没有规定等待者先后次序。某进程持续抢不到锁仍可能饥饿，所以 “硬件原子指令保证有限等待” 缺少排队、公平和持续运行等条件。

第一个读到假值的调用者同时把锁置真；后续调用者得到真值，继续等待。关键在于 “取旧值并置真” 不能被其他竞争者插入，而不在于 `while` 语句本身有什么特殊之处。Swap/XCHG 则原子交换共享的 `lock` 与进程私有的 `key`。共享 `lock` 初值为假，每次申请先令本进程的 `key=true`，再反复原子交换二者，直到 `key=false`，表示取得了交换前的空闲锁；退出时令 `lock=false`。

这类锁若一直循环，就是**自旋锁**。它避免睡眠和唤醒的切换开销，适用于持锁时间很短、持锁者能够继续运行的情形。它不保证公平；有限等待也不能只从指令的原子性推出。阻塞锁在不可用时使线程睡眠，更适合可能较长的等待；不能把 “互斥锁” 这个名称一概等同于自旋实现。

单处理机上，自旋者正在运行时，持锁者不能同时运行；若调度还能让持锁者重新运行，锁仍可能被释放。若相关内核区又禁止了能让持锁者运行的切换，便不能指望忙等自行结束。多处理机上，其他核可继续执行持锁者。

屏蔽本 CPU 的可屏蔽中断，只能在单处理机、相关执行路径不会主动阻塞等限定下保护短小的内核临界操作；它无法阻止另一 CPU 访问共享数据，因此不适用于一般多处理机互斥。开关中断是特权操作，普通用户程序无权执行。分析原语的原子性时，应说明保护范围，不能笼统写 “所有原语都靠关中断”。

四种软件互斥写法的进入区

以下双进程伪代码中 $i \in \{0,1\}$、$j = 1 - i$，单次共享变量读写原子且按程序次序可见。它们的适用边界与前述反例共同构成分析依据。

```
// 单标志：turn初值0
while (turn != i) { }
临界区;
turn = j;

// 双标志先检查：flag[0]=flag[1]=false
while (flag[j]) { }
flag[i] = true;
临界区;
flag[i] = false;

// 双标志后检查：flag[0]=flag[1]=false
flag[i] = true;
while (flag[j]) { }
临界区;
flag[i] = false;
```

三段分别暴露严格交替、检查与占用分离、双方先声明后互等的问

题。Peterson 算法在声明后再用 `turn` 解决双方同时竞争，不能只删掉其中任一变量。

Swap 原子交换可写为：

```
// lock共享且初值false；key为本进程私有变量
key = true;
while (key) Swap(lock, key);
临界区；
lock = false;
```

原子交换一次完成两变量值的互换，`key` 取得交换前的旧锁值。把 Swap 展开成三条普通赋值后，仍必须有额外机制保证三条作为整体原子执行，才能保留上述结论。

不变量·记录型计数的符号含义

采用本节记录型定义时，正数表示未分配许可，负值绝对值表示等待者数量。P 先减再判断，结果等于 0 仍已拿到最后许可；V 先加再判断，结果等于 0 仍需唤醒。不能把整型忙等信号量的非负规则混进来。

整型信号量与记录型信号量的分界

整型信号量的典型定义是：`wait(S)`先在 `S<=0` 时循环，发现 `S>0` 后减 1；`signal(S)`加 1。检查与递减仍需原子化。等待者占 CPU 忙等，不能让权等待，计数也不利用负数记录队列长度。

记录型信号量把“资源不足”转为进入等待队列并阻塞。比较题中，先确认采用哪套定义，再解释 0、负值和等待人数；不能把整型的 `S<=0` 忙等条件直接套到记录型 P 操作减 1 后的判断。

2.3.3 信号量：先说明它代表什么，再计算初值

记录型 P、V 的完整语义

记录型信号量 S 包含整数 `value` 与等待队列。初始化只做一次；以后通过 P、V 改变状态。以下“修改计数—判断—入队或唤醒”须由系统实现为一个原子操作；但进程阻塞之后，整个等待期间并不因此变成不可调度的原子区。

```
P(S):
    S.value = S.value - 1
    if S.value < 0:
        将当前进程加入 S 的等待队列并阻塞
        // 被唤醒后，本次 P 返回；不再次减 1

V(S):
    S.value = S.value + 1
    if S.value <= 0:
        从 S 的等待队列移出一个进程，将其置为就绪
```

P 表示申请一个许可。减 1 后为 0，说明最后一个可用许可刚被取得，**此时不阻塞**；减 1 后小于 0，才说明本次申请需要等待。V 增加一个许可；加 1 后仍小于或等于 0，说明原先有人等待，需要把这个机会交给一个等待者。V 本身不会因资源不足而阻塞。

`value>0` 表示尚未分配的许可数；`value=0` 表示没有未分配许可，也没有在该量上等待的进程；`value<0` 时，$-$value 是等待队列长度。这些解释采用上述记录型定义。整型信号量的另一种实现是在 `S<=0` 时循环检查，再取得许可；其计数通常不为负，且可能忙等，不能混用两套数值规则。

设两台同类打印机对应初值 2，A、B、C、D 先后申请，再由 A、B 归还；下图为便于追踪，假定按入队顺序唤醒。

操作（初值 2）	操作后 S.value	等待队列	本次唤醒
A 执行 P	1	空	无
B 执行 P	0	空	无
C 执行 P	-1	C	无
D 执行 P	-2	C → D	无
A 执行 V	-1	D	C
B 执行 V	0	空	D

Figure 2.17: 资源归还会唤醒一个等待者，计数不必先变为正数

A 归还后值为 -1，C 已得到可继续执行本次 P 的机会，D 仍等待。此时值仍为负数，但 C 不必再做一次 P。唤醒 C 也不意味着 C 立即运行：取得许可与分到 CPU 是两件事。

不变量·记录型计数的符号含义

采用本节记录型定义时，正数表示未分配许可，负值绝对值表示等待者数量。P 先减再判断，结果等于 0 仍已拿到最后许可；V 先加再判断，结果等于 0 仍需唤醒。不能把整型忙等信号量的非负规则混进来。

初值来自初态，不来自“同步量”这个名称

互斥量 `mutex=1` 表示只有一个进入许可。若某项一次性工作尚未完成，完成通知 `done=0`，后续进程先等通知。若缓冲区已有 k 个可消费的产品，产品许可应从 k 开始；容量为 n、已有 k 件时，空位许可应为 $n-k$。

例 2B-2　同步信号量一定从 0 开始吗

例 2.3.2 (配套例题)
〖2.3-018〗用 P、V 操作实现进程同步，信号量的初值为（　）。
A. -1　　B. 0　　C. 1　　D. 由用户确定

答案为 **D**。这里的“由用户确定”指根据资源或事件的初始状态确定，不能任意设数。B 适用于尚未发生、尚无可用结果的一次通知，不能覆盖所有同步情形。判断初值时，应检查题目有没有给出“尚未发生”这个限定。

将完成条件放在正确位置

若操作 B 必须在 A 完成后开始，设置 `done=0`：

```
前进程: A; V(done);
后进程: P(done); B;
```

若后进程先到，P 使它等待；若前进程先到，V 留下一个许可，后进程后来做 P 时也能通过。通知必须放在 A **完成之后**，不能在 A 开始前发布。互斥则是各参与者在受保护代码前 P、后 V；同步的一次 P 与产生它的一次 V 通常分布在不同进程中。因此，“P、V 要配合”不等于“必须在同一进程紧挨着出现”。

信号量通常没有互斥锁的持有者限制：一个进程可 P，另一个可 V。这允许它表达合作事件，也意味着不能把信号量与具有所有权要求的 mutex API 完全混同。

2.3.4 从前驱图到线程代码

错项排除·满足次序还要保留并发

若要求 C 完成后 E 可开始，却在 D 完成后才通知 E，虽然没有违反 C 先于 E，却额外加上 D 先于 E。前驱题需同时检查 “不提前” 和 “不增加无关等待”，通知应紧随真正的前驱条件成立时发出。

前驱图的边 $A \to B$ 表示 B 不能在 A 完成前开始。先标出哪些顺序已由同一线程的程序顺序保证，再为剩下的跨线程依赖传递完成通知。给每条边各设一个初值 0 的信号量是常见的直接构造，但已有顺序可能使部分量不再需要；反过来，两个操作都等 C，也推不出它们之间要互相等待。

课件例：六个操作的三路汇合

进程 P_i 执行操作 S_i。课件前驱关系为 $S_1 \to S_2, S_3$，$S_2 \to S_4, S_5$，以及 $S_3, S_4, S_5 \to S_6$。每条边各用一个初值为 0 的信号量：a, b, c, d, e, f, g 分别对应图中的七条边。前一操作完成后发出许可，后一操作开始前取得许可。

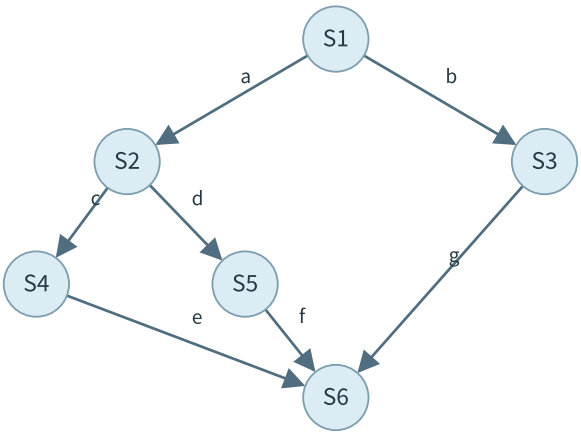


Figure 2.18: 六操作前驱图：S1 分出两路，S2 再分出两路，S3、S4、S5 三路汇合到 S6

```
P1: S1; V(a); V(b);
P2: P(a); S2; V(c); V(d);
P3: P(b); S3; V(g);
P4: P(c); S4; V(e);
P5: P(d); S5; V(f);
P6: P(e); P(f); P(g); S6;
```

例如，S_4 的许可来自 S_2 完成后的 `V(c)`；S_6 必须先依次取得三路许可，才可执行。三个 P 的顺序可以调整，但不能省去任意一路，也不能将 `V(e)` 放到 S_4 之前。该例每个信号量只有一个发送者与一个接收者，均用于一次同步，不是互斥锁。

例 2B-3　哪些边需要跨线程通知

例 2.3.3 (配套例题)

【配套例题】某进程的两个线程 T1 和 T2 并发执行 A、B、C、D、E 和 F 共 6 个操作，其中 T1 执行 A、E 和 F，T2 执行 B、C 和 D。题

46 图表示上述 6 个操作的执行顺序所必须满足的约束：C 在 A 和 B 完成后执行，D 和 E 在 C 完成后执行，F 在 E 完成后执行。请使用信号量的 wait()、signal() 操作描述 T1 和 T2 之间的同步关系，并说明所用信号量的作用及其初值。

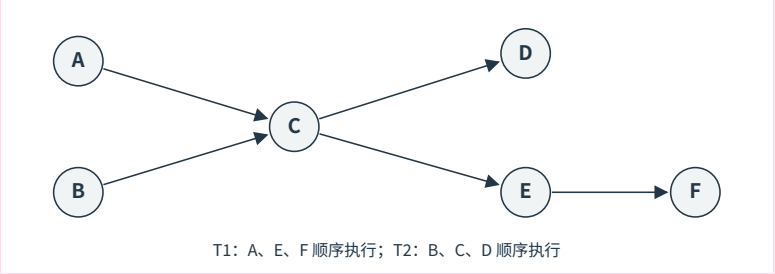


Figure 2.19: 原题前驱图；图下文字标明题干给出的线程归属

解析。在所写程序中，让 T1 按 A、E、F，T2 按 B、C、D 顺序执行，B→C、C→D、E→F 就已满足。跨线程仍需保证 A→C 和 C→E，分别设 `S_AC=0`、`S_CE=0`。

```
semaphore S_AC = 0, S_CE = 0;

T1() {                  T2() {
    A;                      B;
    signal(S_AC);           wait(S_AC);
    wait(S_CE);             C;
    E;                      signal(S_CE);
    F;                      D;
}                       }
```

T2 到达 C 前已完成本线程的 B，又收到了 A 完成的通知，因此 C 的两个前提齐备。C 完成后先发布 `S_CE`，T1 才能开始 E，随后执行 F；T2 自行继续 D。

例如，A、B、C、E、F、D 是允许的完成次序，A、B、C、D、E、F 也允许。D 与 E、F 没有额外次序要求。若把通知放到 D 后，虽然仍不违反图中的先后约束，却会强加 E 等待 D 的限制，减少题目允许的并发。结果安全不等于完整保留了题目允许的并发。

错项排除·满足次序还要保留并发

若要求 C 完成后 E 可开始，却在 D 完成后才通知 E，虽然没有违反 C 先于 E，却额外加上 D 先于 E。前驱题需同时检查 “不提前” 和 “不增加无关等待”，通知应紧随真正的前驱条件成立时发出。

2.3.5 缓冲区：容量许可与互斥范围分别建立

例 2B-4　同一个缓冲区为什么有不同的信号量方案

例 2.3.4 (配套例题)

【配套例题】计算机系统中的进程之间往往需要相互协作以完成一个任务。在某网络系统中，缓冲区 B 用于存放一个数据分组，对 B 的操作有 C1、C2 和 C3。C1 将一个数据分组写入 B 中，C2 从 B 中读出一个数据分组，C3 对 B 中的数据分组进行修改。要求 B 为空时才能执行 C1，B 非空时才能执行 C2 和 C3。

（1）假设进程 P1 和 P2 均需要执行 C1，实现 C1 的代码是否为临界区？为什么？（2 分）

（2）假设 B 初始为空，进程 P1 执行 C1 一次，进程 P2 执行 C2 一次。请定义尽可能少的信号量，并用 wait()、signal() 操作描述进程 P1 和 P2 之间的同步或互斥关系，说明所用信号量的作用及其初值。（3 分）

（3）假设 B 初始不为空，进程 P1 和 P2 各执行 C3 一次。请定义尽可能少的信号量，并用 wait()、signal() 操作描述进程 P1 和 P2 之间的同步或互斥关系，说明所用信号量的作用及其初值。（3 分）

解析。（1）是。两个 C1 都操作同一缓冲区，若同时发现 B 为空后分别写入，可能覆盖数据；检查并改变缓冲状态的代码需要保护。

（2）先把“初始为空、各一次”转成 C1 完成 →C2 开始。设置 `done=0`，表示唯一一次写入完成的通知：

```
semaphore done = 0;
P1() { C1; signal(done); }
P2() { wait(done); C2; }
```

不设同步量，P2 就可能先读；设一个同步量就足以使 C2 必在 C1 之后，两次操作不会重叠，故无需再加互斥量。这里同步关系已经保证了本情境下的互斥，不能据例 2B-1的分类机械增加变量。

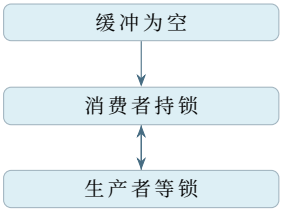
（3）B 已非空，且这里只有两个 C3，修改不会将 B 清空。无需等待新的产品，只须保证两次修改不重叠：

```
semaphore mutex = 1;
P1() { wait(mutex); C3; signal(mutex); }
P2() { wait(mutex); C3; signal(mutex); }
```

三问的共享对象相同，操作类型与执行次数却不同。“容量为 1”不能单独决定信号量个数；必须结合参与者、初态和活动范围一起判断。

最小反例·锁住改变条件的人

初始空缓冲区中，消费者若先 P(mutex) 再 P(full)，就会持锁等产品；生产者随后等同一把锁，无法放入产品。两方等待的事件互相依赖。修正时应让消费者先取得产品许可，再申请只保护单次取出的缓冲锁。



消费者持 mutex 等待 full，生产者也需要 mutex，二者无法推进。

有限缓冲区为什么通常需要 empty、full 和 mutex

经典生产者—消费者模型中，一组生产者反复放入，一组消费者反复取走，共享容量为 $n>0$、初始为空的缓冲区。每次放入或取出一件，对缓冲区及相关指针的操作都须互斥。基本模型见王道书 §2.3。

三项条件分别决定三种许可：`empty=n` 表示尚未被申请的空槽，`full=0` 表示尚未被申请的产品，`mutex=1` 保护单次缓冲操作。生产者在本地生产、消费者在本地使用产品，这些操作都不占用缓冲区锁。

Figure 2.20: 空位许可与产品许可沿相反方向流动；五个槽与课件示例一致，算法容量仍为 n

不变量·许可与物理槽位有中间态

容量 2 且初始为空，生产者刚做 P(empty) 尚未放入时，empty ＝ 1、full ＝ 0，物理空槽却仍为 2。许可已预留而物理状态未改变，因此不能把 empty ＋ full ＝容量写成每条指令后都成立的不变量。

```
semaphore empty = n, full = 0, mutex = 1;

producer() {
    while (true) {
        x = produce();
        P(empty);           // 先取得一个空槽许可
        P(mutex);
        put(x);
        V(mutex);
        V(full);            // 放入完成，才能发布产品
    }
}
```

```
consumer() {
    while (true) {
        P(full);            // 先取得一个产品许可
        P(mutex);
        x = get();
        V(mutex);
        V(empty);
        consume(x);
    }
}
```

“尚未被申请” 这一限定不能省略。容量为 2、初始为空时，一个生产者刚执行 P(empty) 而尚未放入，`empty=1, full=0`，物理空槽却仍有 2 个。空槽已预留，信号量先减少，实际空位数还没变；消费者取得 `full` 后、实际取出前也类似。因此，执行中每一步的 `empty+full` 都未必等于 n，`full` 也未必等于实存产品数。只有两个计数均非负，且既无已取得但尚未使用的许可、也无操作完成却尚未发布的许可时，才能把它们分别与物理空槽数、产品数对照。计数为负时，其绝对值表示相应等待者数。

为什么不能持有缓冲区锁等待产品

若消费者先 `P(mutex)` 再 `P(full)`，初始为空时，它会持锁阻塞。生产者随后申请 `mutex` 也阻塞；消费者等生产者放入，生产者等消费者解锁，形成等待环。

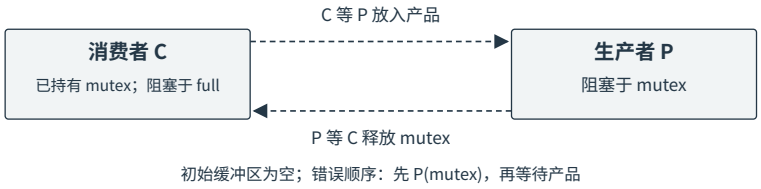
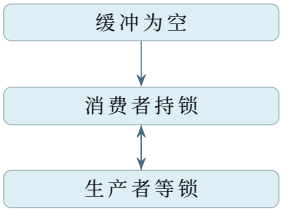


Figure 2.21: 错误的 P 顺序使能够改变条件的进程也被挡住

最小反例·锁住改变条件的人

初始空缓冲区中，消费者若先 P(mutex) 再 P(full)，就会持锁等产品；生产者随后等同一把锁，无法放入产品。两方等待的事件互相依赖。修正时应让消费者先取得产品许可，再申请只保护单次取出的缓冲锁。



消费者持 mutex 等待 full，生产者也需要 mutex，二者无法推进。

所以在这个算法中，容量或产品的 P 在外，缓冲区锁的 P 在内。判断别的算法时仍要追问：当前持有什么，等待什么，谁能使条件成立，对方是否也要用所持资源。不能把 “同步 P 总在所有互斥 P 前” 当成普遍定理；整批取数和读者组会给出更具体的锁范围。

放入之后，本算法的 `V(mutex)` 与 `V(full)` 可交换：消费者即使先收到产品许可，仍须取得 `mutex` 才能取出。原顺序可避免提前唤醒后再等锁。这个判断依赖放入已经完成、且后续访问也持锁；不是所有 V 都能任意挪动。

〖2.3-039〗生产者—消费者模型处理的是进程之间的同步与互斥。它本身不保证公平；错误的 P 顺序还可能制造死锁。避免这一错误顺序只是正确实现的必要条件，不能据此声称该模型能解决任意死锁。

例 2B-5　消费者只能取特定类别时怎样计数

例 2.3.5 (配套例题)

【配套例题】三个进程 P1、P2、P3 互斥使用一个包含 N（N>0）个单元的缓冲区。P1 每次用 produce() 生成一个正整数并用 put() 送入缓冲区某一空单元中；P2 每次用 getodd() 从该缓冲区中取出一个奇数并用 countodd() 统计奇数个数；P3 每次用 geteven() 从该缓冲区中取出一个偶数并用 counteven() 统计偶数个数。请用信号量机制实现这三个进程的同步与互斥活动，并说明所定义信号量的含义（要求用伪代码描述）。

解析。“缓冲区非空” 不足以允许 P2 取数：其中可能只有偶数。保留共同的空位许可，把产品许可按可取条件拆为 `odd` 与 `even`；三种缓冲操作共用 `mutex`。

原题未明说缓冲区初态。以下按参考答案采用**初始为空**：`empty=N, odd=even=0, mutex=1`。令 `x` 是 P1 本次生成的本地整数，`put()` 放入它；保留题设函数名。

```
semaphore empty = N, odd = 0, even = 0, mutex = 1;

P1() {
    while (true) {
        x = produce();
        P(empty);
        P(mutex);
        put();
        V(mutex);
        if (x % 2 != 0) V(odd); else V(even);
    }
}
```

```
P2() {
    while (true) {
        P(odd);
        P(mutex);
        getodd();
        V(mutex);
        V(empty);
        countodd();
    }
}
```

不变量·许可与物理槽位有中间态

容量 2 且初始为空，生产者刚做 P(empty) 尚未放入时，empty ＝ 1、full ＝ 0，物理空槽却仍为 2。许可已预留而物理状态未改变，因此不能把 empty ＋ full ＝容量写成每条指令后都成立的不变量。

```
P3() {
    while (true) {
        P(even);
        P(mutex);
        geteven();
        V(mutex);
        V(empty);
        counteven();
    }
}
```

每次 put 只发布所放入的产品所属类别的一份许可，消费者不能拿另一类别的许可代用。getodd/geteven 完成后，空位可立刻交还；各自的统计不访问共享缓冲区，留在锁外。

若初始已有 a 个奇数、b 个偶数，则应改为 `odd=a, even=b, empty=N-a-b`，其中 $a,b \geq 0$ 且 $a+b \leq N$。若长期没有某类别的产品产出，该类别的消费者就可能持续等待，直到该类产品出现；只要生产者和另一消费者仍前进，就不能仅凭这一等待认定系统死锁。题目没有保证所有类别都会出现。

例 2B-6　连续取 10 件改变的是哪一层互斥

例 2.3.6 (配套例题)

【配套例题】系统中有多个生产者进程和多个消费者进程，共享一个能存放 1000 件产品的环形缓冲区（初始为空）。当缓冲区未满时，生产者进程可以放入其生产的一件产品，否则等待；当缓冲区未空时，消费者进程可以从缓冲区取走一件产品，否则等待。要求一个消费者进程从缓冲区连续取出 10 件产品后，其他消费者进程才可以取产品。请使用信号量 P、V（或 wait()、signal()）操作实现进程间的互斥与同步，要求写出完整的过程，并说明所用信号量的含义和初值。

解析。“连续” 限制其他消费者插入取数，不禁止生产者在这期间补货。由此需要两个范围：`mutex=1` 保护一次缓冲操作，`batch=1` 由一个消费者持有直到它取满 10 件。生产者不申请 `batch`。

```
semaphore empty = 1000, full = 0, mutex = 1, batch = 1;
item buf[1000];
int in = 0, out = 0;

producer() {
    while (true) {
        x = produce();
        P(empty);
        P(mutex);
        buf[in] = x;
        in = (in + 1) % 1000;
        V(mutex);
        V(full);
    }
}
```

辨析·容量 1 是互斥结论的前提

一只盘子通过空盘与特定水果许可交接访问权。容量大于 1 后，生产者可并发操作共享容器，通常还要独立 mutex 保护结构。

```
consumer() {
    while (true) {
        P(batch);
        for (int k = 0; k < 10; ++k) {
            P(full);
            P(mutex);
            local[k] = buf[out];
            out = (out + 1) % 1000;
            V(mutex);
            V(empty);
        }
        V(batch);
        consume(local);
    }
}
```

`empty`、`full` 的含义与初值来自容量和初态。`in/out` 分别指向下一次放入、取出的位置；`local[0..9]` 是消费者私有的十件产品存储。每次取走一件就归还一个空槽，不必等整批结束。

假设消费者已取出 3 件，缓冲区暂时为空。它持有 `batch`，等 `full`；但它没有持有 `mutex`，生产者能继续放入并 V(full)，因此可接着取第 4 件。其他消费者不能插入，所要求的连续性得到保持。若把 `mutex` 从整批开头一直持有到末尾，消费者会在缺货时挡住补货者，形成死锁。

本解不要求开始时凑齐 10 件。不要让多个消费者在批锁之外各申请若干产品许可，再等凑够十个；这会把产品预留给多个消费者，破坏整批协调。`batch` 在前并非违反上一节的锁顺序：真正要检查的是生产者是否也依赖它，以及消费者等待产品时是否仍给生产者留下执行通路。

2.3.6 课件模型：一只盘子与两类水果

一只盘子一次只能放一个水果；父亲放苹果，母亲放橘子，女儿取苹果，儿子取橘子。盘子初始为空。需要区分 “空盘”“有苹果”“有橘子” 三个条件，设 `plate=1, apple=0, orange=0`。

```
father:                          mother:
while (true) {                   while (true) {
    准备苹果;                        准备橘子;
    P(plate);                        P(plate);
    放入苹果;                        放入橘子;
    V(apple);                        V(orange);
}                                }

daughter:                        son:
while (true) {                   while (true) {
    P(apple);                        P(orange);
    取出苹果;                        取出橘子;
    V(plate);                        V(plate);
    吃苹果;                          吃橘子;
}                                }
```

不变量·登记与首末判断一起保护

第一个读者申请文件权，最后一个读者释放。若只保护计数加减而不保护是否为首末读者的判断，新读者可能在旧末读者释放后直接进入，与写者重叠。保护对象是整个判定与登记过程，不能只盯住一条加法。

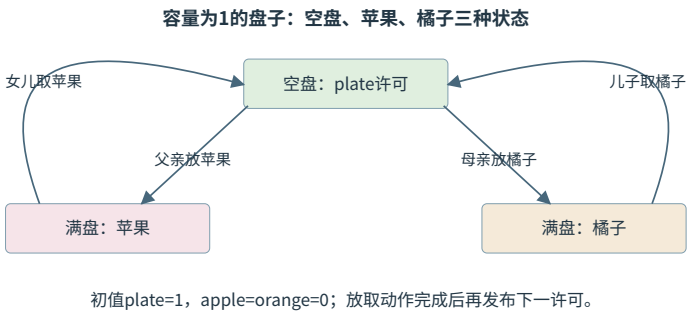


Figure 2.22: 空盘在两个生产者间竞争；两种满盘状态分别只允许对应消费者取走

盘子容量为 1 且每次按上述顺序完整交接，访问权在空盘许可、某生产者、某类水果许可、对应消费者之间转移，不能同时有两个进程访问盘子；因此这份特定模型可由同步关系兼顾互斥。记录型信号量的数值可能为负，解释访问权时应跟踪已取得但尚未归还的许可，不能简单把三个原始计数相加。

若容量变为 2，父母可以分别取得一个空位并发修改共享存放结构，单靠容量计数已不足以保证互斥。此时增加 `mutex=1`，在生产者 `P(plate)`后、消费者 `P(apple)`或 `P(orange)`后取得互斥量，只保护实际放取动作，再释放互斥量及发布相应条件。等待资源前不持有盘子锁，才能让能够改变条件的其他进程继续工作。

2.3.7 多个读者怎样作为一组申请资源

读者—写者模型允许多个读者同时读共享文件；写者之间互斥，写者与任何读者互斥。读者读完不清空文件，因此 “已有数据” 不应被当作一个被读走就消耗的产品许可。以下采用王道书 §2.3 的经典模型；每次读或写的时间有限，所有访问者遵守同一协议。

首读者申请，末读者释放

令 `rw=1` 表示文件访问权：一个写者独占，或者一组读者共同持有。共享计数 `readCount=0` 记录已登记但尚未退出的读者数，`countMutex=1` 保护该计数和首末读者判断。

```
semaphore rw = 1, countMutex = 1;
int readCount = 0;

reader() {
    P(countMutex);
    if (readCount == 0) P(rw);
    readCount++;
    V(countMutex);

    读文件;

    P(countMutex);
    readCount--;
    if (readCount == 0) V(rw);
    V(countMutex);
}
```

不变量·登记与首末判断一起保护

第一个读者申请文件权，最后一个读者释放。若只保护计数加减而不保护是否为首末读者的判断，新读者可能在旧末读者释放后直接进入，与写者重叠。保护对象是整个判定与登记过程，不能只盯住一条加法。

```
writer() {
    P(rw);
    写文件;
    V(rw);
}
```

第一个读者为整组取得 `rw`，后续读者只登记，不重复申请 `rw`；最后一个读者离开才归还。若每个读者都在读文件前 P(rw)、读后 V(rw)，读者也被串行化，题目允许的并发便丢失了。

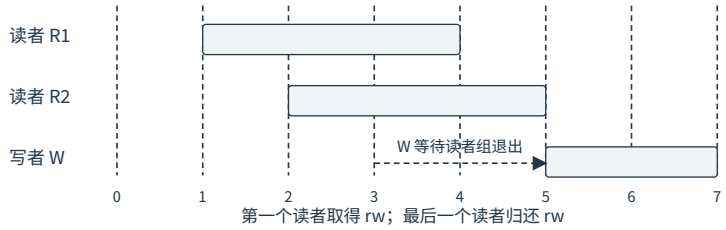


Figure 2.23: 重叠的两个读者共同占有一个读者组的文件访问权；横轴为示意时间

计数的检查和修改须在同一个受保护范围。假设只保护加减、不保护前面的判断：`readCount=1` 时，新读者检查到非零，准备跳过 P(rw)，此时暂停；原有的最后一个读者退出，把计数减到 0 并释放 rw；写者取得 rw 后，新读者才恢复，将计数加 1 并开始读。新读者没有申请 rw，却与写者同时访问。把判断和登记一起保护，才能排除这条交错。

首读者可能持有 `countMutex` 等待 `rw`，此处为何不必然死锁？它等待时 `readCount=0`，所以没有已登记的读者需要取得 `countMutex` 来做末读者退出；正在写的进程归还 `rw` 也不需要 `countMutex`。

读者优先与公平需要分别说明

上述算法中，只要仍有读者在读，新来的读者就能加入同一组，即使已有写者等 `rw`。不断到来的读者可能使 `readCount` 始终不归零，写者因而饥饿。

一种改法是加入口信号量 `gate=1`。每个读者在登记前 P(gate)，登记后 V(gate)；写者则 P(gate)、P(rw)，写完后依次 V(rw)、V(gate)。其余读者退出代码不变：

错项排除·四人申请不等于四人吃饭

五根筷子下，每个吃饭者要占两根，最多只能有两人同时吃。room = 4 限制的是开始申请到全部归还这一段内的人数，用来避免五人各拿一根；它既不是同时吃饭人数，也不直接保证所有等待者公平。

```
// 读者的入口，替换上一算法的入口部分
P(gate);
P(countMutex);
if (readCount == 0) P(rw);
readCount++;
V(countMutex);
V(gate);

// 写者完整过程
P(gate);
P(rw);
写文件;
V(rw);
V(gate);
```

一旦写者取得 `gate`，新读者不能再登记；原有读者仍能按原退出代码离开，最后一个释放 `rw`，写者便可进入。若 `gate` 队列按先进先出服务，且进程持续获得执行机会、每次读写有限，可进一步说明等待者不会被后来者无限越过。**仅有信号量而没有公平排队约定，不能无条件宣称“不会饥饿”或“严格先来先服务”**。它也不是绝对的写优先策略：写者并不在任何到达次序下都先于读者进入。

2.3.8 哲学家进餐：申请多个资源时怎样断开等待环

五个哲学家 P_0 至 P_4 围桌而坐，每人须同时持有左右两根筷子才能吃饭。一根筷子只供一人使用，吃完归还，再思考。统一编号：P_i 需要 C_i 与 $C_{(i+1)\bmod 5}$，每个筷子信号量初值为 1。经典模型见王道书 §2.3。

如果所有人都先 P(C[i]) 再 P(C[(i+1)%5])，一种允许的交错是每人先拿到一根。此时每人都在等右邻已经拿着的那一根，无人能到达归还步骤。

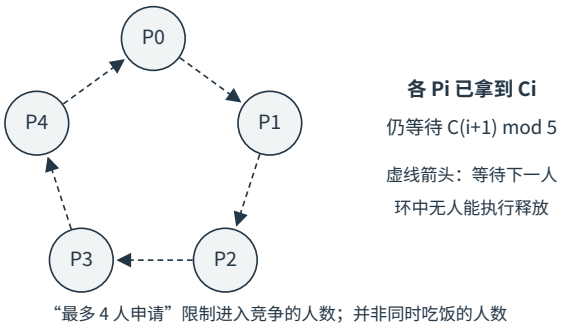


Figure 2.24: 每个进程只持有一根筷子，也足以形成整体死锁

限制竞争人数，与限制同时吃饭人数不同

增加 `room=4`，使最多四人处在“开始拿筷子到全部归还”的区间内：

```
semaphore room = 4;
semaphore C[5] = {1, 1, 1, 1, 1};

philosopher(i) {
    while (true) {
        思考;
        P(room);
        P(C[i]);
        P(C[(i+1) % 5]);
        吃饭;
        V(C[(i+1) % 5]);
        V(C[i]);
        V(room);
    }
}
```

最小反例·通知不会留下许可

条件变量没有等待者时发 signal，随后才调用 wait 的线程不会获得一张已保存的通知券。相比之下，信号量 V 会增加可供以后 P 取得的许可。用“通知在前、等待在后”这一最小时间线就能排除二者作用相同的说法。

若所有竞争者都各持一根并等待下一根，要形成完整环就必须五人都参与；`room=4` 阻止了这种状态。沿等待方向追下去，要么遇到一根还没被占用的筷子，要么遇到正在吃饭、能够归还筷子的人。只要持有者继续执行、进餐有限，就不会陷入上述整体等待环。

这里的 4 是同时申请区间内的人数上限。五根筷子最多支持 $\lfloor 5/2 \rfloor = 2$ 人同时吃饭，不能把“最多四人申请”误读为“四人同时进餐”。该办法排除了这种死锁情形，但并不保证某个饥饿者最终抢到许可；仍需公平条件。

另一种办法是全局排序：所有人都先取编号较小、再取较大的筷子。若存在资源等待环，沿环等待的资源编号就必须严格递增，最后又回到起点，不可能成立。每人独立“先左后右”不是全局排序；编号相邻位置在取模后更须检查。

也可用额外互斥量把“拿两根”的过程串行化，拿齐后立即释放这个额外量，吃完直接归还筷子。这能断开多人各拿一根的环，但可能让一个正在等筷子的人挡住另一个本可拿齐的人。它也不等于“先原子检查两根都空闲，然后同时取得”；若题目要求整组资源同时分配，就需要更强的原子检查与分配机制。

2.3.9 管程与条件变量：等待时必须让出改变条件的机会

管程把共享数据、对数据的操作过程和初始化封装在一个同步模块中，参与者通过规定的入口访问这些数据；同一时刻至多一个进程在其中执行。这个互斥规则由实现管程的语言或运行系统保证，不要求调用者在每个入口外重复编写 P、V。管程不是一个自行运行的进程。

课件拓展：Java 对象锁。实例方法上的 `synchronized` 使用调用该方法的对象作为锁。同一对象的两个同步实例方法共享这把锁，一个线程执行期间，另一个线程需等待；不同对象实例通常持有不同的锁。静态同步方法使用对应类对象的锁，不能把它与任意实例锁当作同一把锁。

```
class Buffer {
    private int count = 0;
    public synchronized int getCount() { return count; }
    public synchronized void clear() { count = 0; }
}
```

这个例子只展示互斥入口；生产者、消费者需要等待条件时，还须

正确使用条件等待和通知，单有 `synchronized` 不会自动实现完整的生产者—消费者逻辑。

但 “每次只准一人进入” 还不够：消费者进入后可能发现无产品。若它在管程里循环等待并一直保有管程，生产者就无法进入改变条件。**条件变量**为这种等待提供队列；实际条件由共享数据上的布尔判断表达，例如 `count>0`。条件变量本身不是保存产品数的整数。

最小反例·通知不会留下许可

条件变量没有等待者时发 signal，随后才调用 wait 的线程不会获得一张已保存的通知券。相比之下，信号量 V 会增加可供以后 P 取得的许可。用 “通知在前、等待在后” 这一最小时间线就能排除二者作用相同的说法。

wait 负责等待，不替调用者判断条件

例 2B-7　条件变量的 wait 做什么

例 2.3.7 (配套例题)

【配套例题】若 x 是管程内的条件变量，则当进程执行 x.wait() 时所做的工作是（　）。

A. 实现对变量 x 的互斥访问

B. 唤醒一个在 x 上阻塞的进程

C. 根据 x 的值判断该进程是否进入阻塞态

D. 阻塞该进程，并将之插入 x 的阻塞队列中

答案为 **D**。调用者先检查共享状态, 只有需要等待时才调用 x.wait()；调用后进入 x 的等待队列，并释放管程使用权，让其他进程能进入。C 把 “调用前的条件判断” 放进了 wait 的职责，还把 x 当作可直接取值的资源计数。

管程中执行 x.signal()，是通知一个已经在 x 上等待的进程。若没有等待者，通知不积累；以后进入的进程须重新检查共享状态。信号量 V 即使没有等待者，也会使计数增加，留下一个可供后续 P 使用的许可。

〖2.3-029〗“管程中 signal 操作的作用和信号量机制中的 V 操作相同” 不成立。检查无等待者的边界即可区分：前者不留下通知，后者增加许可。管程作为同步模块，与一个可独立运行的进程也不是同一类对象；具体语言的管程对象能否实例化，应由语言规定。

通知以后，何时重新检查条件

不同管程定义对 signal 后的运行次序有不同规定。Hoare 式语义可立即转交管程给被唤醒者；Mesa 式语义让通知者继续，等待者以后重新竞争进入权。题目若给出特定语义，应据此推演，不能混用。

下面用 Mesa / POSIX 式条件等待说明可独立执行的做法。`wait(cv,m)` 把释放互斥锁 m 与进入等待合并为一个原子操作；返回前重新取得 m。通知不保证等待者马上运行，也不保证等它重新取得 m 时条件仍成立，所以用 `while` 重检。实际接口还可能出现虚假唤醒，同样需要重检。

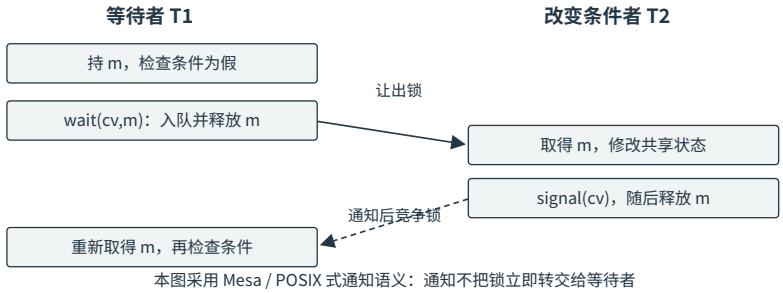


Figure 2.25: wait 释放锁，使另一个线程能够改变共享状态；等待返回前须重新取得锁

设这个模块的容量为 $n > 0$，初始为空，m 初始化为未加锁；插入和取出维护同一个缓冲区。

```
// 单个缓冲模块的共享状态
mutex m;
condition notEmpty, notFull;
int count = 0;

put(x) {
    lock(m);
    while (count == n) wait(notFull, m);
    插入 x; count++;
    signal(notEmpty);
    unlock(m);
}
```

```
get() {
    lock(m);
    while (count == 0) wait(notEmpty, m);
    x = 取出元素; count--;
    signal(notFull);
    unlock(m);
    return x;
}
```

若消费者发现空就调用 wait，它释放 m；生产者才能取得 m 放入，并通知消费者。被通知者回来时再次检查 `count`，是因为这期间可能已有另一个消费者先取走产品。不能把 `wait` 拆成普通的 “先 unlock，再单独入队”：另一线程可能恰在两步之间发出无人接收的通知，等待者随后入队，就错过了这次机会。

这里的接口语义用于把等待机制讲清，并非要求记忆 POSIX 参数及错误码。参见 The Open Group《Base Specifications Issue 8》条件等待与通知条目；管程概念与条件变量的比较见王道书 §2.3。

2.3.10 写出并发代码后，怎样检验

题干中的名词不会直接决定信号量个数。先把每个进程的一次活动写清，标出它开始前必须成立的条件，再说明谁能使该条件成立。把 “可用槽位”“某类产品”“阶段已经完成” 等条件转成许可，把会冲突的共享状态操作放进相应互斥范围，才能确定变量及初值。

课程连接·检验方法

检查初值、每个 P/V 位置、资源上限及所有阻塞路径，尤其注意持锁等待是否构成环。

题干给出的条件	转成什么关系	应检查的代码位置
必须等某操作完成	完成通知，初值由该事件是否已发生决定	前操作完成后 V，后操作开始前 P。
最多容纳 n 件	容量许可，有放入与归还两个方向	先申请空位，放入后发布产品，取出后归还空位。
只能取某一类	每一类独立的可取条件	发布对应类别，不以总的 full 代替各类别许可。
多个读者可以并行	读者组与写者互斥	保护首末读者判断，而非将整个读操作串行化。
连续取 k 件后才轮到别人	消费者整批排他	批锁覆盖 k 次，缓冲锁只覆盖单次；等待时允许补货。

课程连接·并发复核

对每个信号量写明其代表的资源或事件、初值，以及改变计数的具体时机。

最后手推几条会暴露错误的轨迹：后继先运行、缓冲区初始为空或已满、两个请求几乎同时到达、一个进程拿到许可但尚未实际操作、通知发出但被通知者尚未重新运行。每一步记下“持有—等待—可执行者”。检查安全性后，再分别检查死锁、饥饿与不必要的串行化；三者不能互相代替。

2.3.11 分支自测：先判断，再看答案

原创回顾设问：下列问题依据本章例题与经典模型编写，用于检查中间状态和适用边界。

1. 按记录型语义，P 前 `S.value=1`，P 后为 0，要阻塞吗？一次 V 使值从 -1 变为 0，要唤醒吗？被唤醒者是否还要为这次申请再减 1？
2. 在前驱题中，能否在 D 完成后才 signal(S_CE)？这样是否保留了图允许的所有并发？
3. 在分类缓冲题中，初态有 a 个奇数、b 个偶数，三个计数信号量怎样初始化？如果缓冲区只有偶数，P2 等待能否说明整个系统死锁？
4. 在成批取数题中，某消费者取了 3 件后暂时缺货，应继续持有哪个量、等待哪个量？生产者还能否运行？
5. 哲学家最多允许四人进入申请区，是否能有四人同时吃饭？
6. 条件变量 signal 在没有等待者时，是否留下一份以后可取得的许可？

自测答案

1. P 后为 0 不阻塞，表示取走最后一个许可。V 后为 0 仍须唤醒一个等待者；它已取得本次申请的机会，不再为同一次 P 减 1。
2. 能保证 E 不早于 C，却额外让 E 等 D，缩小并发范围。通知应在 C 完成后、D 前发出。
3. `empty=N-a-b, odd=a, even=b`；需 $a,b \geq 0$、$a+b \leq N$。若只有 P2 无奇数可取，不能推出整个系统无法前进。
4. 持有 `batch`、等待 `full`，此时没有持有 `mutex`。生产者不需要 `batch`，仍能取得空位与缓冲区锁，补入产品。
5. 最多两人同时吃饭，四是申请区间内的人数上限。
6. 条件变量通知不积累为以后可消费的许可；有资源初态要用共享状态表达。

条件变量等待语义另参 The Open Group 条件等待与通知接口、Oracle《Using Condition Variables》。

2.4 死锁：从资源分配状态判断能否继续

错项排除·有环还要看资源实例数

P 持 A 等 B、Q 持 B 等 A 构成环；若 A 另有空闲实例，Q 仍可拿到它并完成，归还 B 后 P 也能前进。单实例图中的环足以判断死锁，多实例时还须化简；不能只看箭头形状就跳过可用实例。

进程因为得不到资源而等待，并不一定发生死锁。要判断是否会死锁，还需追问：谁持有该进程所需的资源，持有者能否完成并归还，归还又依赖谁。本章把这种追踪分别用于死锁条件、资源分配图、安全检查和死锁检测。

除特别说明，讨论的是可重复使用的资源：每类可以有多个同类实例，资源只能在释放后再次分配；进程请求后若不能满足便等待，正常完成会归还已持资源。分析安全性还假定进程声明的最大需求可信，不超过系统允许的资源总量，并能在获得所需资源后有限完成。

2.4.1 死锁、饥饿和不安全状态

死锁指一组进程中，每个进程都在等待一个只能由组内进程引起的事件（其中也包括自身尚未执行的释放动作），因而这一组无法自行推进。最小的双资源例子是：P 持有 A 等 B，Q 持有 B 等 A；没有进程能到达释放步骤。

饥饿指某个进程长期得不到所需服务，而其他进程仍可能持续前进。例如读者不断加入，使写者一直拿不到写权限。**死循环**是执行路径自身不能结束，进程可能仍持续占用 CPU，不必等待其他进程。三者的机制不同，处理方法也不同。

四个必要条件是同时约束

条件	本章资源模型中的含义	怎样在状态中识别
互斥	某类实例同一时刻只能给一个进程使用	资源不可被两个申请者同时使用
占有并等待	保持已有资源，同时等待新增资源	一个进程既有分配边，又有请求边
不可剥夺	未用完的资源不能被系统任意收回	只能由持有者按规则释放
循环等待	存在首尾相接的资源等待链	P 等 Q，Q 又直接或间接等 P

发生这类资源死锁时，四项同时成立；只满足其中一项或几项不能推出死锁。“四项必要条件都允许存在” 也不等于任何时刻都已经死锁，还要检查实际持有与等待状态。单进程再次请求一个自己已经持有且不可重入的锁，也可能自我阻塞，不能把 “死锁至少要两个进程” 当作适用于所有锁语义的结论。

资源分配图中的环什么时候足够

进程用圆表示，资源类型用方框表示，框内点表示实例。$P_i \to R_j$ 是请求边，$R_j \to P_i$ 是一个实例的分配边。没有有向环，可排除这种资源等待死锁；有环时，若环中相关资源类型均只有一个实例，则可判定死锁。

若资源类型有多个实例，环只说明可能死锁。例如 P 持有 A 的一个实例、等待 B，Q 持有 B、等待 A，但 A 还有一个空闲实例，Q 取得它并完成后就能释放 B。图中已有等待环，却仍有解开它的执行通路。此时必须结合实例数化简或执行检测算法。

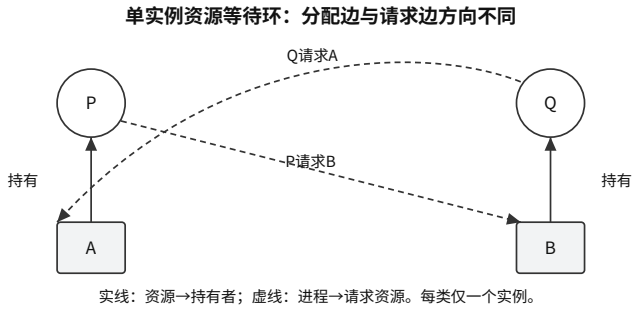


Figure 2.26: 单实例环中每个人都等别人；增加可用实例后必须重新检查能否推进

课程连接·破坏条件

预防死锁只需破坏四个必要条件之一，但具体规则可能牺牲利用率或增加申请约束。

安全状态问的是“能否保证全部完成”

安全序列是进程的一种次序，使每个进程在轮到它时，现有可用资源加上此前已完成进程释放的资源，足以满足它尚需的最大资源量。若存在至少一个安全序列，则称当前状态安全。

安全状态并不意味着系统只能按这一序列运行；该序列是可完成性的证据。**不安全**表示无法在最大需求假设下给出这样的保证，不代表此刻已经死锁。进程可能还没提出全部需求，也可能实际少用资源或提前释放。死锁状态必不安全；不安全状态可能在以后的某种申请序列中发展成死锁。

2.4.2 死锁预防：在规则中破坏一个条件

预防是在分配规则上使至少一个必要条件不能成立，不必等到死锁已经出现。

让可共享的资源按共享方式使用，可以去掉相应的互斥条件；但打印机输出、独占锁等具有不可共享的语义，不能为了避免死锁而取消必要保护。一次性申请运行所需的全部资源，全部满足才开始，可以破坏占有并等待，但会降低利用率，并要求提前知道需求。申请失败时先释放已有资源再重试，也是破坏“占有并等待”的办法。

课件以 **SPOOLing** 为例：各进程先将打印数据送入输出井并提交请求，再由输出进程按队列驱动打印机。这样各进程可以共享逻辑上的打印服务，不必直接占有物理打印机等待输出；实际打印仍由输出进程串行管理，输出井和请求队列本身也需要正确同步。它改变的是应用进程申请打印资源的方式，并未使物理打印机同时打印多份内容。

对可保存、恢复状态的资源，可以在适当时机剥夺并回滚，例如某些处理器或存储资源；正在打印到纸张上的半份输出却不能简单撤回。对所有资源规定统一编号，只允许按严格递增次序申请，可以破坏循环等待：若有环，沿环的资源编号必须不断增大又回到起点，无法成立。各进程都只按“先拿左边、再拿右边”申请，并不等于有了统一的全局编号，仍可能形成环。

预防可能限制请求方式并造成资源闲置。**避免**则保留更多请求自由，在每次拟分配时检查分配后的状态是否仍然安全。二者都在实际进入死锁前采取措施，但依据不同。

例 2C-1　避免与检测需要的信息不同

例 2.4.1 (配套例题)

【配套例题】若系统 S1 采用死锁避免方法，S2 采用死锁检测方法。下列叙述中，正确的是（　）。

I. S1 会限制用户申请资源的顺序，而 S2 不会

II. S1 需要进程运行所需的资源总量信息，而 S2 不需要

III. S1 不会给可能导致死锁的进程分配资源，而 S2 会

A. 仅 I、II　　B. 仅 II、III　　C. 仅 I、III　　D. I、II、III

解析。银行家算法这类避免方法根据最大需求和当前状态决定批准还是推迟，并不规定所有进程必须按统一资源顺序申请；I 混入了预防中的资源有序分配法。避免要判断未来最大需求是否仍可满足；检测根据当前已分配与当前等待请求判断，不要求事先掌握全过程最大需求。检测允许某些分配在以后导致死锁，事后再检测并解除。因此 II、III 正确，答案为 **B**。

不变量·Work 增加的是原持有量

安全检查假想先借出 Need、再收回 Max，净效果是 Work 增加 Allocation。若直接增加 Max，就把尚未借出的资源也算成新增资源，破坏总量守恒。算法中的每次假定完成，都应能还原成这次借出与归还。

2.4.3 银行家算法：可用不等于可以批准

设有 n 个进程、m 类资源。向量按同一资源次序排列，各分量逐项比较，不能用向量元素之和代替比较。

名称	含义
`Available[j]`	第 j 类当前未分配的实例数
`Max[i][j]`	进程 i 声明的第 j 类最大需求
`Allocation[i][j]`	进程 i 已持有的实例数
`Need[i][j]`	进程 i 尚可能需要的最大数量，等于 `Max-Allocation`

安全性检查中的 Work 为什么要加 Allocation

先复制 `Work=Available`，所有 `Finish=false`。反复寻找未完成且 `Need[i] <= Work` 的进程：假定给足它的剩余需求，它完成后会归还所有资源，净效果是 `Work += Allocation[i]`，并标记完成。若最终所有进程都能标记，则安全；若再无可选者但仍有未完成者，则不安全。

之所以只加原 `Allocation`，是因为假定借给它的 `Need` 已从 Work 中暂时扣掉，完成时又随资源归还，二者相抵。若把 `Max` 直接加到 Work，就会凭空增加资源；若把 Work 逐步减到负数，再说“以后会归还”，则跳过了本算法的可完成前提。

```
Work = Available
Finish[0..n-1] = false
while 存在 i: !Finish[i] 且 Need[i] <= Work:
    Work = Work + Allocation[i]
    Finish[i] = true
若全部 Finish 为真，则安全；否则不安全
```

例 2C-2　能先完成两人，仍然可能不安全

例 2.4.2 (配套例题)

【配套例题】某时刻进程的资源使用情况如下表所示，可用资源向量为 $(0,2,1)$。

进程	已分配 (R_1,R_2,R_3)	尚需分配 (R_1,R_2,R_3)
P1	(2,0,0)	(0,0,1)
P2	(1,2,0)	(1,3,2)
P3	(0,1,1)	(1,3,1)
P4	(0,0,1)	(2,0,0)

此时的安全序列是（　）。

A. P1、P2、P3、P4　　B. P1、P3、P2、P4

C. P1、P4、P3、P2　　D. 不存在

最小反例·拿得出仍可能不能批

总量 4，P 最多 4、Q 最多 3，已各持 1，可用 2；当前可先完成 Q。若再给 P 一个，可用只剩 1，两者都还需 2，安全序列消失。这个状态通过了现存量检查，却未通过试分配后的安全检查。

解析。初始只有 P1 的 Need 不超过 Work。假定 P1 完成，Work 从 $(0,2,1)$ 变为 $(2,2,1)$。此时 P2、P3 均需要 3 个 R_2，只有 2 个，不能选；P4 可以完成，释放 $(0,0,1)$，Work 变为 $(2,2,2)$。

已假定完成	Work	下一步
无	(0,2,1)	只能选 P1
P1	(2,2,1)	只能再选 P4
P1、P4	(2,2,2)	P2、P3 均差一个 R_2，停止

没有覆盖所有进程的安全序列，答案为 **D**。C 的前两步合法，不代表整条序列合法；不能因为 “已释放了不少资源” 就略过逐类比较。本题没有给出每个进程此刻实际阻塞的请求，所以不据此宣称已经死锁。

例 2C-3　把每个候选可用向量独立试一遍

例 2.4.3 (配套例题)

〖2.4-026〗系统有三类资源 A、B、C 和五个进程，某时刻分配如下。若系统安全，则可用资源向量 (x,y,z) 可能是哪些?

进程	Allocation	Max
P0	(0,0,3)	(0,0,4)
P1	(1,0,0)	(1,7,5)
P2	(1,3,5)	(2,3,5)
P3	(0,0,2)	(0,6,4)
P4	(0,0,1)	(0,6,5)

I. (1,4,0)　II. (0,6,2)　III. (1,1,1)　IV. (0,4,7)

A. I、II、IV　　B. I、II　　C. 仅 I　　D. I、III

解析。五个 Need 依次为 $(0,0,1)$、$(0,7,5)$、$(1,0,0)$、$(0,6,2)$、$(0,6,4)$。每个候选都从初态重新检查，不能沿用上一候选已经 “释放” 的资源。

I 中，P2 可先完成，Work 从 $(1,4,0)$ 增至 $(2,7,5)$，其余四个的 Need 都能满足，因此安全。II 中可先完成 P0、P3、P4，Work 最多达到 $(0,6,8)$；P2 差 A，P1 差 B，不能继续。III 中可先完成 P0、P2，Work 到 $(2,4,9)$，

其他三个都至少需要 6 个 B，仍只有 4 个。IV 中只能先完成 P0，Work 到 $(0,4,10)$ 后停止。故仅 I 安全，答案为 **C**。

II 和 III 不安全的缘由并不相同：II 即使 C 很多，A、B 的短缺也无法互相抵补；III 也无法靠增加 A、C 弥补 B 的不足。

不变量·Work 增加的是原持有量

安全检查假想先借出 Need、再收回 Max，净效果是 Work 增加 Allocation。若直接增加 Max，就把尚未借出的资源也算成新增资源，破坏总量守恒。算法中的每次假定完成，都应能还原成这次借出与归还。

新请求的三个关口

进程 i 请求向量 `Request`，依次检查：

1. `Request <= Need[i]`。否则超出原先声明的最大需求，不是正常等待问题。
2. `Request <= Available`。否则当前没有足够实例，应等待，不能先把 `Available` 减成负数。
3. 试分配：Available 减 Request，Allocation 加 Request，Need 减 Request；对新状态做完整安全检查。安全才提交，否则恢复原表并推迟请求。

第二步只说明 “拿得出来”，第三步才说明 “拿出以后仍有完成保证”。下面的最小反例用来区分这两步。**原创机制示例**：一类资源共 4 个，P、Q 分别最多要 4、3 个，已各得 1 个，可用 2 个。当前可先让 Q 取得剩余 2 个、完成后释放，再供 P，故安全。若再给 P 1 个，可用只剩 1，P 尚需 2，Q 也尚需 2，变为不安全。请求既不超最大需求，也不超现存量，仍应拒绝立即分配。

总资源界限不能代替银行家逐态检查

若 n 个进程每个最多需要 k 个同类不可剥夺资源，为了在**任意分配顺序**下都保证至少有一个进程能够拿齐，可用最坏状态论证：每个进程都持有 $k-1$ 个而仍未完成，共持有 $n(k-1)$ 个，再多 1 个就可使至少一个进程拿齐。所以充分的总量界限是 $m \geq n(k-1)+1$，即 $nk < m+n$。

这个结论回答的是不加安全检查时的最坏分配保证，不能当作银行家算法能够工作的必要条件。银行家可以主动推迟不安全请求，使资源较少的系统仍保持安全。例如两个进程各最多需要 3 个、系统共 3 个，完全可以先满足一个，再满足另一个；最大需求之和 6 不满足 $6 < 3+2$，仍能安全运行。遇到题干把这两种问题混写时，应先明确要求的是任意分配保证，还是某个状态能否批准。

王道课件：三类资源的安全性与试分配

系统资源总量为 $(10,5,7)$，当前可用资源 $Available = (3,3,2)$。各进程的最大需求、已分配量与尚需量如下，按资源分量逐项相减得到 $Need = Max - Allocation$。

进程	Max	Allocation	Need
P0	(7,5,3)	(0,1,0)	(7,4,3)
P1	(3,2,2)	(2,0,0)	(1,2,2)
P2	(9,0,2)	(3,0,2)	(6,0,0)
P3	(2,2,2)	(2,1,1)	(0,1,1)
P4	(4,3,3)	(0,0,2)	(4,3,1)

一种安全序列为 $P1 \to P3 \to P0 \to P2 \to P4$。假定各进程顺次完成并归还自己的已分配资源，$Work$ 依次为 $(3,3,2) \to (5,3,2) \to (7,4,3) \to (7,5,3) \to (10,5,5) \to (10,5,7)$。存在一条安全序列即可证明当前状态安全，序列不一定唯一。

P0 请求 $(2,1,1)$。它逐项不超过 P0 的 Need，也不超过 Available。试分配后，$Available' = (1,2,1)$，$Allocation'_0 = (2,2,1)$，$Need'_0 = (5,3,2)$。再次执行安全性检查，可选 $P3 \to P1 \to P4 \to P0 \to P2$，其 $Work$ 依次为 $(1,2,1) \to (3,3,2) \to (5,3,2) \to (5,3,4) \to (7,5,5) \to (10,5,7)$。存在完整安全序列，因此这一请求可以批准；试分配时不应把旧的 Allocation 当作完成后的归还量。

另一组不安全状态。保持上表 Allocation 与初始 Available，将 P0、P2、P4 的 Max 分别改为 $(8,5,3)$、$(9,5,2)$、$(4,3,6)$。P1、P3 依次完成后 $Work = (7,4,3)$；剩余 P0、P2、P4 的 Need 分别为 $(8,4,3)$、$(6,5,0)$、$(4,3,4)$，各有至少一个分量超过 Work，无法继续完成，因此该状态不安全。不安全表示无法保证全部进程完成，并不等同于此刻已经死锁。

错项排除·Need 不等于当前请求

银行家用未来可能达到的 Need 验证完成保证；死锁检测用已经提出、正在等待的 Request 检查当前能否推进。不安全说明最坏未来缺少保证，不等于当前已经死锁，更不能直接宣布所有未标记者都已死锁。

课件银行家例子：可借出不代表可批准

一种资源总量 100，三个进程 B、A、T 的最大需求分别为 70、40、50，已分配 20、10、30，因而可用量为 40，尚需量为 50、30、20。

试探请求	试分配后 Available	试分配后 Need(B,A,T)	安全性
B 再申请 30	10	$(20,30,20)$	没有任何进程可先满足最大剩余需求，不安全
A 再申请 20	20	$(50,10,20)$	可取安全序列 T→B→A，或 A→T→B

第二种请求以 T→B→A 检查：`Work=20` 先满足 T，假定其完成后 `Work=20+30=50`；B 再完成后 `Work=50+20=70`；A 最后完成后 `Work=70+30=100`。A 的 Allocation 已包含试分配的 20，因此最后归还量为 30。

第一种状态尚未必已经死锁，因为进程可能实际需要少于申报最大值或提前释放资源；银行家算法按最大需求作保证，拒绝的是无法保证安全的分配。这里 B 试分配后的 Allocation 为 $20+30=50$，资源守恒必须成立，不能把它误写成 60。

2.4.4 检测、解除和恢复

检测允许系统运行到某种资源等待状态后，再判断是否已经死锁。单实例资源可以构造进程等待图；多实例资源要结合当前持有量、可用量和**当前未满足请求**进行化简。这里使用的 Request 是已经提出、正在等待的数量，不是未来最大 Need。

令 Work 为当前可用量。对当前请求能由 Work 满足的进程，假定其继续执行并释放已持资源，删除其相关分配与请求边，再寻找下一可化简进程。若有持有资源的进程最后仍无法化简，则该资源等待状态中存在死锁。没有占有任何资源的等待进程可能受死锁影响，却不一定属于维持资源环的核心；“死锁进程集合”的具体范围，还取决于所用定义与检测初始化规则。

例如 A、B 各 1 个，P 持 A 等 B，Q 持 B 等 A，Available 为 $(0,0)$，P、Q 的当前请求分别为 $(0,1)$、$(1,0)$。没有任何进程能化简。若第三个进程持有另一个 B 且能不等待地完成，则它归还后 P 可推进；这就是多实例有环却未必死锁的同一机制。

课件例：资源有多个实例时怎样化简

资源 R_1 有 3 个实例，R_2 有 2 个实例。P_1 已持有 2 个 R_1，P_2 已持有 1 个 R_1 和 1 个 R_2，并正在请求 1 个 R_1。比较以下两种资源分配状态；绿

色是资源实例指向进程的分配边，蓝色是进程指向资源类型的请求边。

辨析·当前请求与可用量

先化简可完成的进程并回收已分配实例，再检查剩余进程的当前请求；未化简的等待核心才是此例中的死锁进程。

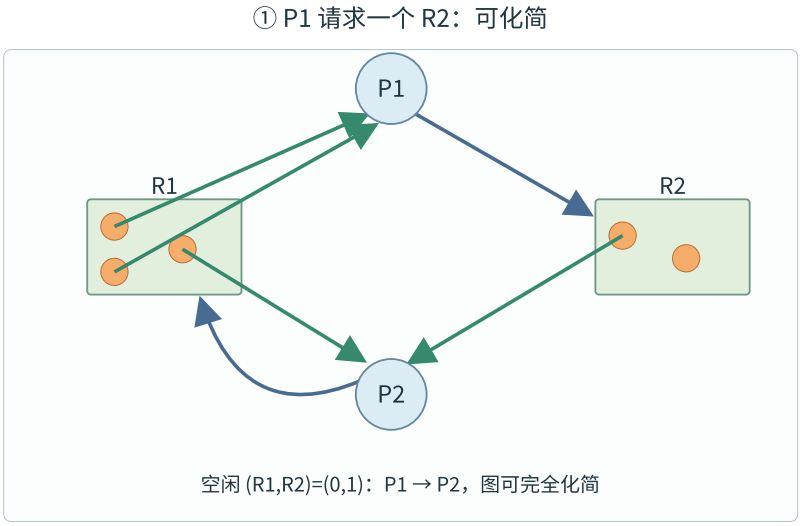


Figure 2.27: 课件资源分配图：P1 请求一个 R2，P1 和 P2 依次完成，图可完全化简

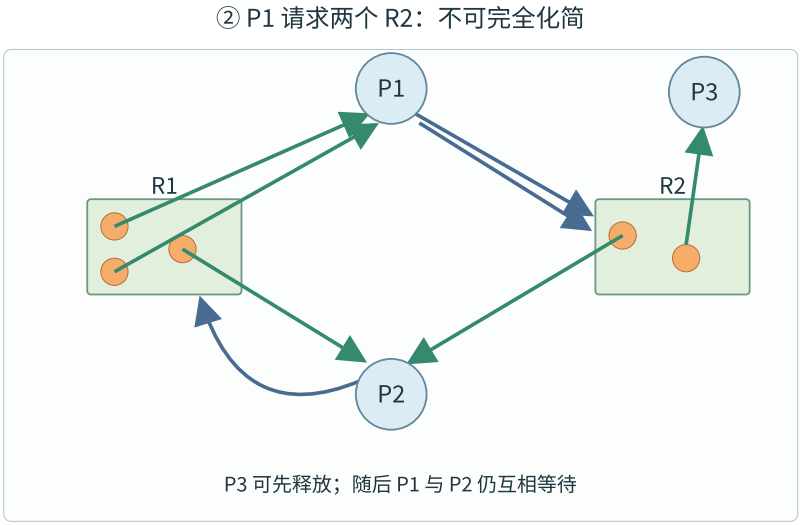


Figure 2.28: 课件资源分配图：P3 先完成后，P1 请求两个 R2 而 P2 请求一个 R1，二者仍互相等待

1. P_1 **请求 1 个** R_2。另一个 R_2 尚空闲，初始 `Work=(0,1)`。可先满足 P_1 的当前请求；假定它执行完后归还所持资源，净增加原有分配量 $(2,0)$，于是 `Work=(2,1)`。再满足 P_2 并回收 $(1,1)$，得到 `(3,2)`。所有边均可消去，图可完全化简。
2. P_1 **请求 2 个** R_2，P_3 **持有另一个** R_2。初始 `Work=(0,0)`。P_3 没有待满足请求，可以先完成并归还 1 个 R_2，得到 `Work=(0,1)`。此时 P_1 仍需 2 个 R_2，P_2 仍需 1 个 R_1，二者均无法继续，因此 P_1、P_2 留在不可化简部分。P_3 已能完成，不应把它也算作未化简的死锁进程。

检测确认死锁后，可以终止全部死锁进程，或逐个选择牺牲者直到环被打破；也可在资源允许恢复时剥夺资源，并把受影响进程回滚到安全检查点。选择牺牲者需要考虑已完成工作、资源占用、恢复代价和优先级；每次都牺牲同一个低优先级进程，可能造成饥饿。仅降低某个死锁进程的优先级，并不会使它释放正持有的资源，因此不能单独解除等待环。

课程连接·资源复核

安全性检查从现有 Available 开始，假设能结束的进程释放 Allocation，逐步更新 Work。

三类处理策略检查的不是同一件事

共同维度	预防	避免	检测与解除
何时作决定	事先规定请求、持有或分配规则	每次请求拟获批准时检查新状态	允许正常申请，定期或按事件检测当前状态
主要依据	资源是否可共享、能否剥夺、申请次序等	最大需求 Max、已分配量和当前可用量	当前分配、当前未满足请求 Request 和可用量
怎样处理有风险的请求	不符合规则便不能按该方式申请或持有	试分配后不安全则回滚并推迟	不以安全性为每次分配的必要条件，可能事后发现死锁
保证来自哪里	至少一个死锁必要条件被规则破坏	始终保留一个能覆盖全部进程的安全序列	检出以后终止、剥夺或回滚，释放维持等待环的资源
主要代价	申请受限、资源可能闲置	需要可信的最大需求，且有反复安全检查开销	需要检测及恢复，可能丢失已经完成的工作

两种 “假定某进程完成并释放资源” 的算法外形相似，输入却不同：银行家用未来可能达到的 Need 检查完成保证，检测用已经提出的 Request 判断当前等待能否解除。不能因为化简步骤相似，就把 “不安全” 直接改称 “已死锁”。

用当前 Request 化简资源分配图

检测使用的是进程**当前仍在等待的请求 Request**，避免死锁中的安全性检查使用的是**最大剩余需求 Need**。两者的判定目标不同。

```
Work = Available;
对每个进程i:
    Finish[i] = (Allocation[i]全部为0);
反复寻找 Finish[i]==false 且 Request[i]<=Work 的进程:
    Work = Work + Allocation[i];
    Finish[i] = true;
找不到时停止；仍未标记完成的持有资源者形成无法化简的部分。
```

向量不等式均逐资源种类比较。一个进程被假定满足当前请求并完成后，先给出的 Request 又随完成归还，所以可用量净增加其原 Allocation。全零分配者本身不持有资源，不是资源等待环的占有者；它仍可能因等待死锁集合释放资源而无法继续。

解除时可选剥夺资源、撤销部分进程或回退到检查点。选择牺牲者时比较优先级、已执行工作量、距完成的时间、占用资源量和任务性质；反复牺牲同一进程还需防止饥饿。回退要求已有能够恢复的一致状态，不能凭空撤销已经产生的外部效果。

2.4.5 分支自测：分别判断现在和最坏未来

以下为原创回顾设问。

1. 资源分配图有环，是否立即判死锁？还缺什么信息？
2. 银行家安全检查中，一个进程可完成时，Work 应增加 Max、Need 还是 Allocation？
3. 新请求不超过 Need 且不超过 Available，能否直接批准？
4. 安全检查失败，能否立即宣布所有剩余进程已经死锁？
5. 3 个进程每个最多要 2 个同类资源，多少个资源可以在任意分配下保证不发生这种资源死锁？若只有 2 个，银行家是否一定无法运行？

课程连接·综合检查

区分何时能运行、先后如何约束、当前资源状态是否可能形成环形等待。

自测答案

1. 要看资源实例数及可化简状态。相关类型均为单实例时环足以判定，多实例时还要检查能否由其他实例打破等待。
2. 增加 Allocation；假定借给它的 Need 在完成后归还，净效应相抵。
3. 不能，还要试分配并保持安全。
4. 不能，Need 是最大未来需求；实际当前 Request 可能更小或尚未提出。
5. $3(2-1)+1=4$ 个提供任意分配保证。只有 2 个仍可采用安全分配，先让一个进程拿齐、完成后轮到其他进程；资源总量界限不是银行家的必要条件。

2.5 主动回忆自测

以下 10 题为自拟综合回忆，按导图主枝独立作答；各分支保留的练习用于进一步检查中间状态。

从执行身份回忆到资源安全

1. 为什么 I/O 完成通常使进程就绪，却不保证立即运行？
2. 同进程线程共享哪些资源，切换时仍要保存哪些独立现场？多对一模型为何不能利用多核同时运行这些用户线程？
3. 共享内存建立后，还要不要为共享计数器加同步保护？信号与信号量有何区别？
4. 无 I/O 时周转、等待、响应怎样计算？有 I/O 时等待公式应怎样修正？
5. Peterson 互斥证明依赖哪些模型假设？TestAndSet 为何不直接保证公平？
6. 记录型 P 使值从 1 到 0 要阻塞吗？V 使值从 -1 到 0 要唤醒吗？
7. 容量 2 初始空，生产者 P(empty) 后未放入，能否要求 empty $+$ full $=2$？为什么？
8. 为什么 Mesa 式条件等待使用 while 复查，并原子地释放锁和进入等待？
9. 资源图有环、状态不安全、系统已经死锁，这三种说法能否互换？
10. 银行家新请求要过哪三个关口？Work 为什么增加 Allocation 而非 Max？

2.5.1 核对答案与边界

1. 设备事件满足后，执行条件恢复；CPU 归属仍由调度决定。就绪到运行是另一条状态转换。
2. 共享地址空间、代码、数据、堆、打开文件等；独立保存 PC、寄存器、栈与线程控制信息。多对一只给内核一个承载实体，无法同时

派到两个核。

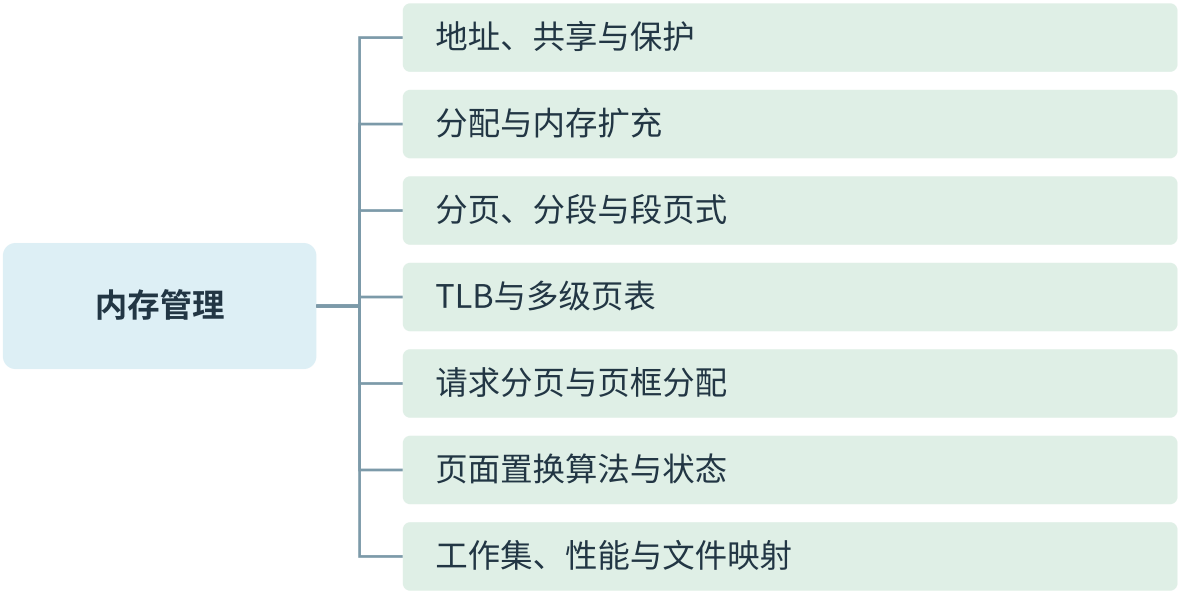
3. 要。映射只提供共同存储，不给出互斥与可见时序。信号通知事件；信号量通过许可计数与等待协调资源和次序。
4. $T = C - A$，无 I/O 时 $W = T - B$，首次响应 $R = S - A$；有阻塞时还须从周转中扣除相应阻塞时间，不能全部当作就绪等待。
5. 双进程、单次共享读写原子、给定内存顺序，以及持续执行和有限临界区等假设；TS 的原子读改写没有规定等待者的公平次序。
6. P 结果 0 表示取得最后许可，不阻塞；V 结果 0 表示原先有人等待，须唤醒一个。该等待者不为同一次 P 再次减 1。
7. 不能。许可已预留，物理空位尚未改变，数值可为 empty ＝ 1、full ＝ 0；还须区分负数计数表示的等待者。
8. 通知不立即转交锁，重新取得锁前条件可能被别的线程改变，也可能虚假唤醒。原子释放并入队防止检查后丢失通知。
9. 不能。单实例资源图有环足以判死锁，多实例还要化简；不安全是无法保证最大未来需求，不等于当前已死锁。
10. 请求不超 Need、不超 Available、试分配后仍安全，失败则恢复。借出 Need 与完成归还的 Need 抵消，净增加原 Allocation。

课程连接·综合检查

区分何时能运行、先后如何约束、当前资源状态是否可能形成环形等待。

内存管理 3

章节脉络



课程连接·地址转换

先判断地址合法和权限，再确定驻留与映射；最后按题目模型计数访问和缺页代价。

3.1 地址、驻留、权限与访问代价

地址与缺页综合题的顺序

1. 写明编址单位、地址位数、页大小和表项长度，先按二进制位宽拆分字段。
2. 检查范围与权限，再根据 TLB、各级页表的状态寻找有效映射。
3. 遇合法但不驻留的页面，跟踪空闲页框、置换、脏页回写、映射更新和原指令重试。
4. 沿完整引用串保存状态；最后分别汇总缺页、置换、访存和设备 I/O 次数，并统一时间单位。

定义 3.1.1 (一次访存要回答三个独立问题)

地址是否属于合法范围、页面此刻是否驻留、当前操作是否有权限，是三个独立判断。页号用于查映射，页内偏移定位页中的字节；页表项的位置也不是最终数据的位置。

3.1.1 页内偏移为何保持不变

从整数除法得到地址分解

页大小为 P 字节时，任意非负字节地址 a 唯一写成 $a = pP + w$，其中 $0 \le w < P$。页表把第 p 页整体映射到第 f 页框，页内第 w 个字节便落在 $fP + w$。因此转换只替换页的整体位置，不改变页内坐标；只有 $P = 2^d$ 时，才可直接把偏移解释为低 d 个二进制位。

□

课程连接·与组成原理按地址串联

组成原理研究 Cache 怎样定位已得到的存储地址，操作系统还要维护虚拟地址到物理页框的转换与权限。做综合访存题时先确认 Cache 使用虚拟还是物理字段，再排列 TLB、页表、Cache 和主存路径，避免混成一次查询。

3.2 内存扩充：覆盖、交换与请求调页

覆盖、交换和按需分页都在缓解内存容量限制，但管理对象与实施条件不同。**覆盖**把不会同时使用的程序段安排到同一覆盖区，在需要时替换，通常要求事先分析模块调用关系；常驻部分与覆盖区应共同满足执行需求。它并非把任意两个正要同时使用的模块强行放在一起。

交换按经典模型把暂不执行的进程整体或较大运行映像换到后备存储，需要时再换入；换出与进程阻塞不是同一概念，挂起状态用于表达不驻留。可否换出还受固定缓冲、正在进行的 I/O 与地址重定位等约束，不能随意移动设备仍在直接访问的内存。

请求调页把管理粒度缩小为页，合法页面可暂不驻留，真正引用时由缺页机制调入。它依赖页表状态、异常处理与后备关系，能够利用局部性；并不要求程序员预先列出覆盖模块。覆盖、交换和分页可以在不同层次讨论，不能仅凭“把内容移到磁盘”便认作同一种机制。

内存管理同时处理三个问题：程序用什么地址描述对象，这个地址此刻对应哪里，访问是否被允许。连续分配、分页和分段使用不同的映射单位；请求分页又允许某些合法内容暂不在物理内存。做题时先区分地址、映射和驻留状态，再进行位数或时间计算。

本章容量中的 KB、MB、GB 按题目所采用的二进制单位理解，即 2^{10}、2^{20}、2^{30} B；需要严格区分标准时可写 KiB、MiB、GiB。地址与页号、页框号从 0 编号；十六进制数以 H 结尾。页、页框大小相同；没有特别说明时，地址按字节编址。

3.3 逻辑地址、物理地址和可寻址范围

逻辑地址或虚拟地址是程序在自身地址空间中使用的地址；**物理地址**标识主存中的实际位置。CPU 执行访存指令形成虚拟地址，地址变换机构按当前进程的映射得到物理地址，再访问主存。操作系统创建和维护映射，MMU 等硬件完成通常的运行时查询；缺页或保护异常再进入内核处理。

若虚拟地址宽 v 位、按字节编址，可表示 2^v B 的虚拟地址范围；物理地址宽 p 位，对应的可寻址物理范围为 2^p B。实际可用空间还受保留区域、硬件实现的有效位数、进程限制及后备存储等约束。不能用“内存容量加硬盘容量”直接求虚拟地址的位数，也不能从逻辑地址位数自动推出物理地址位数。

内存以页为单位调入调出，不等于每条 CPU 访存都以整页为单位。指令仍可读取一个字节或一个字，页是映射、分配或传输的管理单位。题目中的地址单位、页大小单位和数据类型长度应分别记录。

3.3.1 编译、链接、装入与重定位

课程连接·覆盖与交换

覆盖要求程序按预定结构安排共用内存区；交换将进程数据在内外存之间调度。

源程序经编译或汇编形成目标模块，模块之间的外部引用由链接处理，形成可装入程序。静态链接在运行前把所需模块连接成整体；装入时动态链接在装入时处理外部连接；运行时动态链接把部分连接工作推迟到程序运行需要相关模块时。

绝对装入在生成程序时就确定绝对地址，适合已知装入位置的环境。**可重定位装入**在装入时根据实际基址修改需重定位的地址，完成后程序通常按改好的地址执行。**动态运行时装入／重定位**则把逻辑地址到物理地址的转换推迟到执行访存时，由硬件地址变换机构完成；它支持程序在运行期间按映射变化移动位置。

3.3.2 同一程序的三种装入方式

若装入起始物理地址为 100，程序中变量的逻辑地址为 79，则该变量对应物理地址 179。绝对装入在编译时已确定装入位置，指令直接使用 179；可重定位装入在装入前使用 79，装入时统一加 100 并把地址改为 179；动态运行时装入后指令仍保留 79，真正访问时由重定位寄存器提供 100，得到物理地址 179。三种方式的区别在于地址何时确定或转换，计算结果相同。

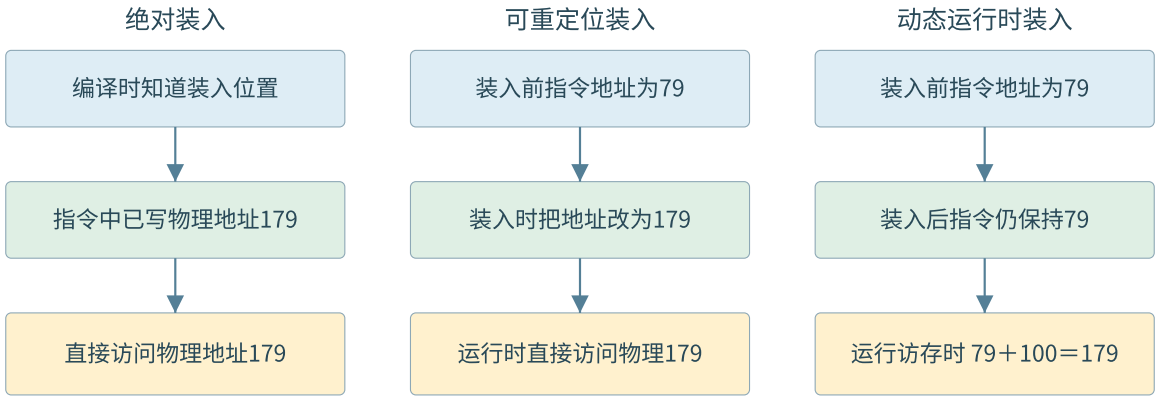


Figure 3.1: 同一变量的三种装入：地址 79 与物理基址 100

地址保护的寄存器含义要与比较式一致。若基址为 100，合法逻辑地址为 0～179，界限寄存器保存最大合法逻辑地址 179，则应检查 $0 \le a \le 179$，物理地址为 $100 + a$；若界限保存长度 180，则应检查 $0 \le a < 180$。最大地址与长度相差 1，属于两种字段定义，不能只凭“界限寄存器”这个名称选择比较符。

以上下限寄存器保护物理区域时，分别比较要访问的物理地址与 100、279；采用重定位寄存器和界地址寄存器时，可先将逻辑地址与最大合法值 179 比较，再把合法的逻辑地址 80 加基址 100，得到物理地址 180。两种结构须分别看清所比较的地址及寄存器含义。

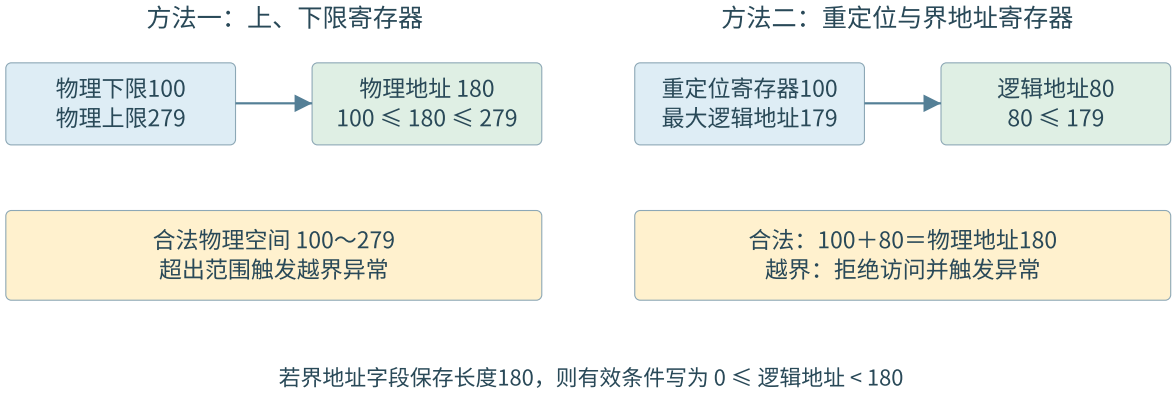


Figure 3.2: 上下限保护与基址加界地址保护的比较流程

进程的典型虚拟映像可含只读代码／数据、可读写静态数据、堆、共享库映射、栈及内核映射区域。代码中的宏常量可能在预处理后被替换，并不因此单独分配一个运行时对象；局部指针自身与 `malloc` 得到的堆对象仍要分别定位。具体地址分布、增长方向与是否保留内核映射，以题目平台为准。

课程连接·独立条件

共享对象、访问权限、地址合法性和是否驻留是不同问题，不能用其中一项代替其他检查。

Figure 3.3: 32 位进程虚拟映像的课件示例，标明用户栈、共享库、堆及代码数据区域

3.4 共享、保护、分配和回收

分配决定哪些物理空间供谁使用，保护限制哪些操作合法，共享则允许多个映射指向同一内容。基址与界限可以保护连续区域；分页与分段通常在映射项中记录存在、读写、用户/内核等权限。一个地址有数值，不表示它属于合法映射；一个页面驻留内存，也不表示用户可以任意写入。

共享只读代码时，各进程的页表可以指向同一个物理页框，并禁止写入。共享可写数据则还需要同步。写时复制先把多个映射指向同一只读物理页，在某个进程需要写时触发受控异常，为写者分配并复制私有页，再更新映射；它与让所有写者始终修改同一共享页不同。

回收时必须维护使用计数、映射与空闲结构；若另一个进程仍在合法使用共享页，不能因为本进程退出就立即把页框分给无关对象。页表与 TLB 的映射变化也须保持一致，否则 TLB 中的过期转换可能继续生效，使访问落到已经回收或改变权限的内容上。

最小反例·空闲总量够仍不能分配

两个不相邻的空洞各为 6MB，总空闲 12MB；申请连续 8MB 仍失败，因为没有单个足够大的洞。只有物理相邻的空闲区才能直接合并，隔着占用区时需要搬移或改用非连续分配，不能把表中相邻行直接相加。

3.4.1 例 3-1　页号可以不同，共享的页框必须相同

例 3.4.1 (配套例题)

【配套例题】进程 R 和 S 共享数据 data。若 data 在两进程中的页号分别为 p1、p2，对应页框号分别为 f1、f2，下列叙述中，正确的是（　）。

A. p1、p2 一定相等，f1、f2 一定相等

B. p1、p2 一定相等，f1、f2 不一定相等

C. p1、p2 不一定相等，f1、f2 一定相等

D. p1、p2 不一定相等，f1、f2 不一定相等

解析。两进程的虚拟布局可以不同，所以共享对象可以分别位于不同页号；题目所指的同一份共享数据在物理内存中是一份，对应页框号应相同。答案为 **C**。各自复制了一份内容相同的数据，不等于本题的共享映射。

3.5 连续分配与动态分区：空闲总量不等于可分配块

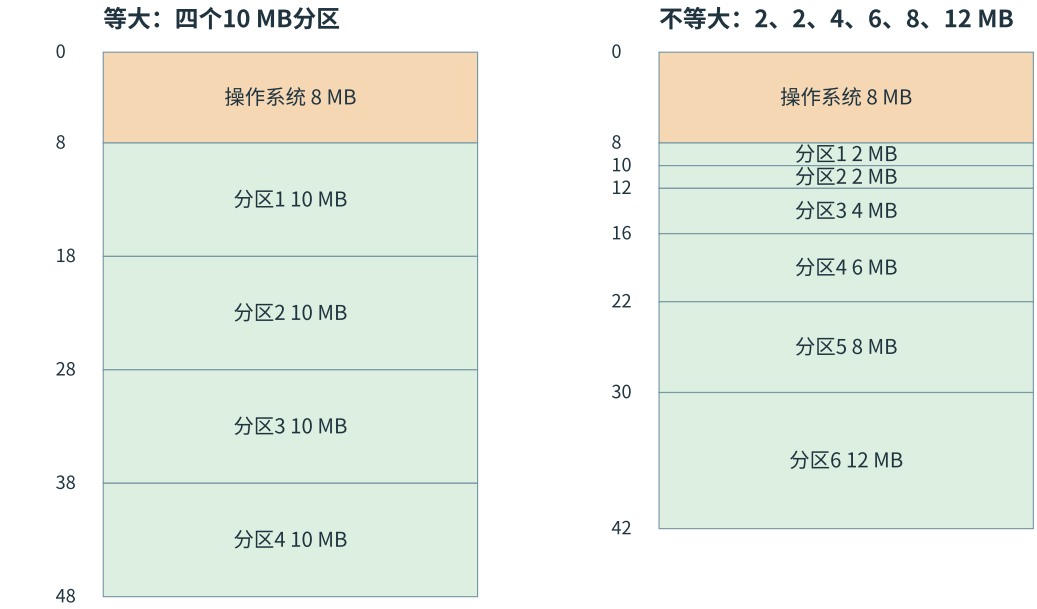
单一连续分配把内存划分为系统区和一个用户区，不能有效支持多个驻留用户作业。固定分区预先划定若干区，每区容纳一个作业；作业小于分区时，分区内未用部分形成**内部碎片**。动态分区按请求划出连续区域，空闲洞可能被已分配区隔开，形成**外部碎片**：空闲总量够，仍可能没有一个足够大的连续洞。

3.5.1 固定分区的划分与分区说明表

固定分区可以等大，也可以不等大。等大分区易管理，适合多个大小相近的作业；大小不等可按常见作业规模划分多个小分区、适量中等分区和少量大分区，使不同作业更容易找到合适空间。两种情况每个分区同一时刻都只装入一道作业，程序太大而没有任何分区容得下时，不能把几个不连续的固定分区直接当成一块连续区。

王道课件：固定分区可等大，也可不等大

各分区一次只装入一道作业；两列分别是独立内存示意，单位MB。



每个表项记录分区大小、起始地址和分配状态；分区内的未用空间为内部碎片。

Figure 3.4: 固定分区课件示例：系统区 8 MB；等大分区每个 10 MB，不等大分区依次 2、2、4、6、8、12 MB

操作系统用分区说明表记录分区号、大小、起始地址及已分配状态；分配时检索可容纳作业的未分配分区，再把该表项改为已分配。课件示例表的前三项如下：

分区号	大小（MB）	起始地址	状态
1	2	8 MB	未分配
2	2	10 MB	未分配
3	4	12 MB	已分配

例如把 1 MB 作业装入一个 2 MB 分区，剩余 1 MB 仍属于该已分配分区，是**内部碎片**；它不是可独立分配的空闲分区。若题目要求计算碎片，需根据实际作业大小、分区选择和所规定的管理粒度逐一确定。

首次适应按地址顺序找第一个足够大的洞；循环首次适应从上次位置继续；最佳适应选择最小但够用的洞，常需按大小查找；最坏适应选择最大的洞。按课件的常见实现，首次适应与邻近适应将空闲链按地址递增排列，邻近适应可采用循环链并从上次检索结束处继续；最佳适应按容量递增，最坏适应按容量递减。首次适应常反复经过低地址的小洞，但也较可能保留高地址大洞；邻近适应分散检索，可能更早切分高地址大洞。最佳适应可能产生难利用的小碎片，最坏适应可能较快耗尽大洞；实际效果取决于请求序列与回收情况。算法只是在现有空闲区中选择一块，并不会把它们自动合并。最佳适应会留下很小的余量，“最接近请求”不保证长期整体利用率最好。

辨析·空闲链顺序是状态

最佳和最坏适应按容量排序，余区大小改变后应维护顺序；邻近适应还须保存查找位置。

3.5.2 课件实例：空闲链如何随分配变化

图中的前三组以同一组空闲区开始：大小 20 MB、起始地址 8；大小 10 MB、起始地址 32；大小 4 MB、起始地址 60。方框中的“大小 @ 起始地址”表示一段连续空闲区，按该算法维护的空闲链次序排列；分配从

所选空闲区的低地址端切出。三个算法的进程请求大小以各自课件演示为准，不能把不同演示的请求混成同一组。

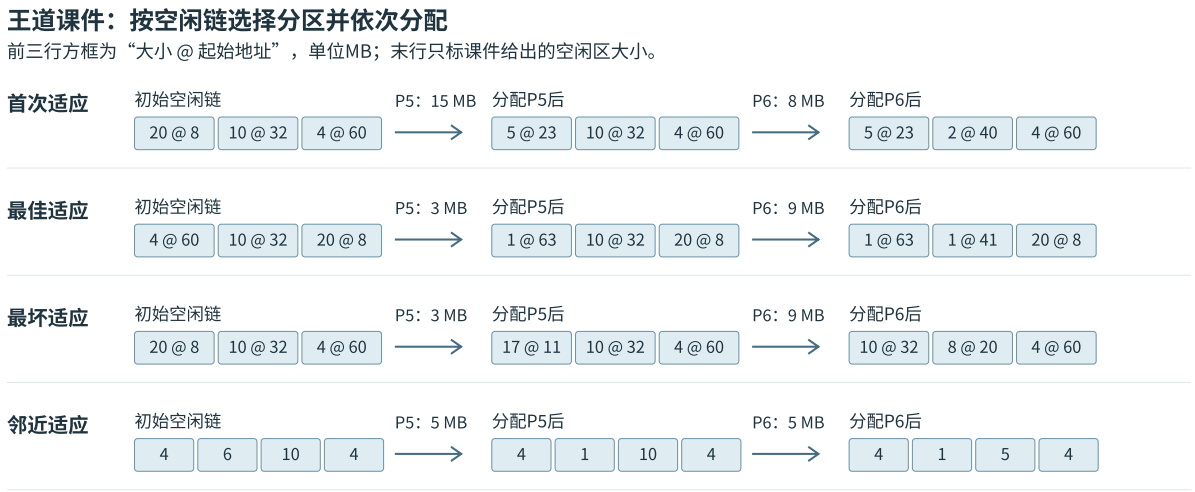


Figure 3.5: 四种动态分区分配算法的课件空闲链演示

辨析·空闲链顺序是状态

最佳和最坏适应按容量排序，余区大小改变后应维护顺序；邻近适应还须保存查找位置。

首次适应依地址递增：P5 请求 15 MB，从 20 MB 区切出后留下大小 5、起始 23 的空闲区；P6 请求 8 MB，5 MB 不足，再从起始 32 的 10 MB 区切出，留下大小 2、起始 40。最佳适应按容量递增：P5 的 3 MB 先选起始 60 的 4 MB 区，P6 的 9 MB 再选起始 32 的 10 MB 区，分别留下两个 1 MB 碎片。最坏适应按容量递减：两个请求先后从原 20 MB 区切出，余区依次为 17 MB 和 8 MB；第二次分配后，空闲链须重新按大小排列，10 MB 排在 8 MB 之前。

邻近适应的课件演示使用地址递增的循环空闲链，大小依次为 4、6、10、4 MB。P5 的 5 MB 由 6 MB 区分出，P6 的 5 MB 由 10 MB 区分出，余区大小依次为 4、1、5、4 MB。它从上次查找结束处继续，查至链尾仍未找到时回到链头；具体起始指针位置是确定下一次结果所必需的状态。四组图展示的是课件演示过程，算法长期表现还取决于请求与回收序列。

释放分区时检查物理地址是否相邻：只与前洞相邻则合并前洞，只与后洞相邻则合并后洞，前后都邻接则合并三块。在按大小排序的空闲区表中，相邻表项未必在物理地址上相邻。紧凑通过搬移已分配内容把分散空洞合成大洞，需要可重定位支持和搬移成本，这与释放时合并相邻空闲区不是一回事。

3.5.3 例 3-2　最佳适应每次都重新比较当前空闲区

例 3.5.1 (配套例题)

【配套例题】某基于动态分区存储管理的计算机，主存容量为 55 MB（初始为空闲），采用最佳适配算法。分配和释放顺序为：分配 15 MB，分配 30 MB，释放 15 MB，分配 8 MB，分配 6 MB。此时最大空闲分区为（　）。

A. 7 MB　　B. 9 MB
C. 10 MB　　D. 15 MB

解析。前两次分配后剩余 10 MB；释放最初的 15 MB 后，有两个不相邻的洞：15 MB 和 10 MB。分配 8 MB 时，最佳适应从 10 MB 中切

出，留下 2 MB；分配 6 MB 时，2 MB 不够，只能从 15 MB 中切出，留下 9 MB。因此两个洞为 9 MB、2 MB，最大 9 MB，答案为 **B**。

操作后	当前空闲分区大小
分配 15 MB	40 MB
再分配 30 MB	10 MB
释放最初的 15 MB	15 MB、10 MB
分配 8 MB	15 MB、2 MB
分配 6 MB	9 MB、2 MB

这里不能先把 15 与 10 相加，因为中间仍有占用区；也不能在分配 8 MB 时沿用首次适应的地址顺序。

辨析·邻接按物理地址判断

空闲表可能按地址或容量排序。两个表项相邻并不表示物理连续；合并须检查区间端点，回收量保持守恒。

3.5.4 空闲区表与回收时的四种合并

动态分区用空闲区表或空闲区链记录起始地址和长度。分配时选中一块，若只使用其一部分，就更新剩余区间；释放时根据物理地址邻接关系合并，避免把已经连成一片的空闲区误保留为多个洞。

课件中的分配更新。空闲表含起始 8、大小 20 MB，起始 32、大小 10 MB，以及起始 60、大小 4 MB 的三个表项。给 P5 分配 4 MB 时，若选前 20 MB 区并从低地址切出，分配区为 $[8, 12)$，把该表项改为起始 12、大小 16 MB；若选末尾 4 MB 区，则 $[60, 64)$ 恰好用完，应删除对应空闲表项。其他空闲表项保持原值。两种情况是从同一初始状态出发的独立演示，具体选哪一块由题设采用的分配算法决定。

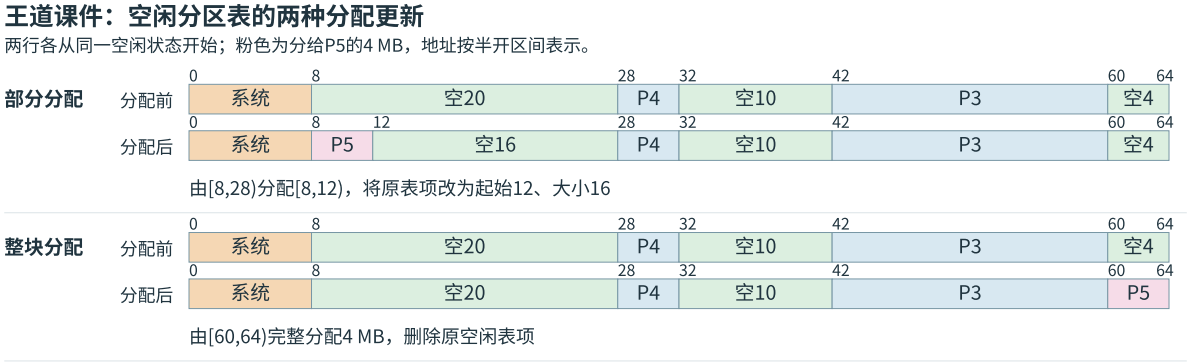


Figure 3.6: 空闲分区表中切出一部分与恰好用完整块的两种更新

课件 64 MB 模型中系统区占前 8 MB，空闲区可为 $[8, 28)$、$[32, 42)$、$[60, 64)$，长度分别 20、10、4 MB；区间右端不包含。若释放 $[28, 32)$ 的 4 MB 区域，它同时连接前后两个洞，应合成 $[8, 42)$，长度 34 MB，并保留 $[60, 64)$。

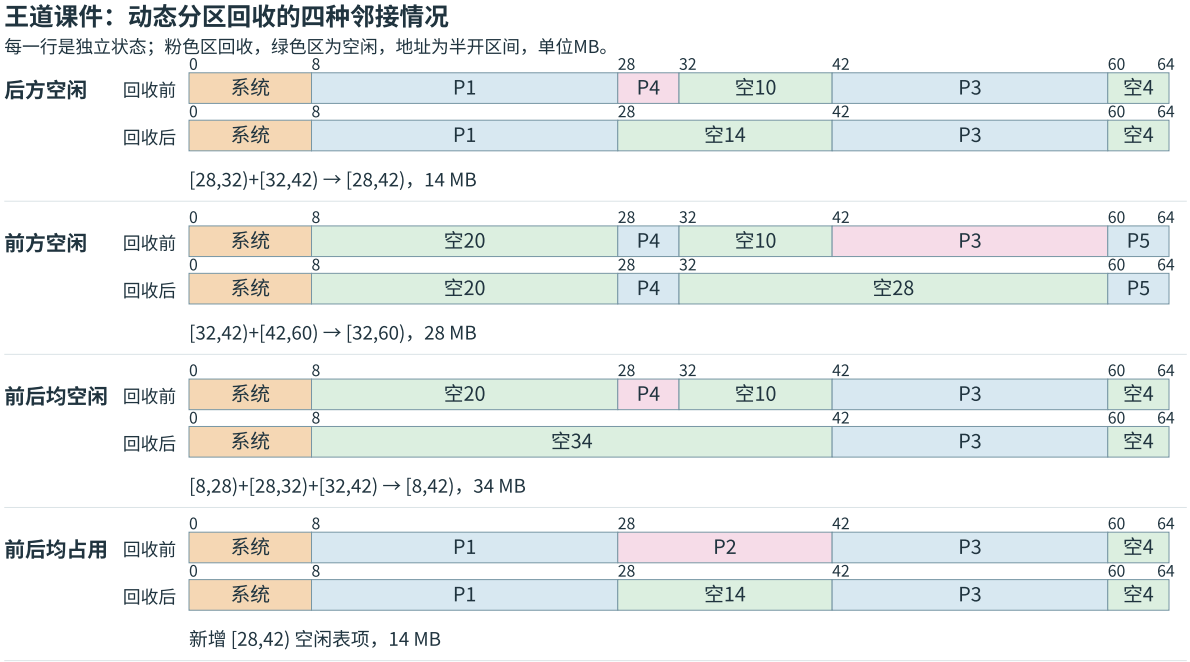


Figure 3.7: 课件四种回收情况：前后空闲区分别相邻、两侧都相邻或都不相邻

图中四行分别是独立的课件状态，并非连续的四次回收：只与后方相邻时，释放 $[28, 32)$ 并与 $[32, 42)$ 合成 $[28, 42)$，长度 14 MB；只与前方相邻时，释放 $[42, 60)$ 并与 $[32, 42)$ 合成 $[32, 60)$，长度 28 MB。前后都相邻时合成 $[8, 42)$、长度 34 MB；前后均不空闲时，释放 $[28, 42)$ 后新增长度 14 MB 的空闲表项。

辨析·邻接按物理地址判断

空闲表可能按地址或容量排序。两个表项相邻并不表示物理连续；合并须检查区间端点，回收量保持守恒。

回收区与已有空闲区关系	更新方法
前后均不相邻	新建空闲表项
只与前方空闲区相邻	前方表项长度增加
只与后方空闲区相邻	后方表项起址前移、长度增加
前后都有相邻空闲区	三段合并，保留一个表项并删除多余表项

首次适应按地址递增检索；最佳适应按足够容纳请求的最小长度选择；最坏适应选最大长度；邻近适应从上次检索位置继续。表若按容量排列，回收后可能重新排序，因此“表项挨在一起”不是判断物理相邻的证据。

3.5.5 覆盖结构的容量计算

课件覆盖模型含主模块 A 为 8 KB，B、C 分别 8、10 KB，D、E、F 分别 12、4、10 KB。把 A 放入固定区，让 B 与 C 共享 10 KB 覆盖区、D/E/F 共享 12 KB 覆盖区，则这一安排需要 $8 + 10 + 12 = 30$ KB，而六模块长度之和为 52 KB。

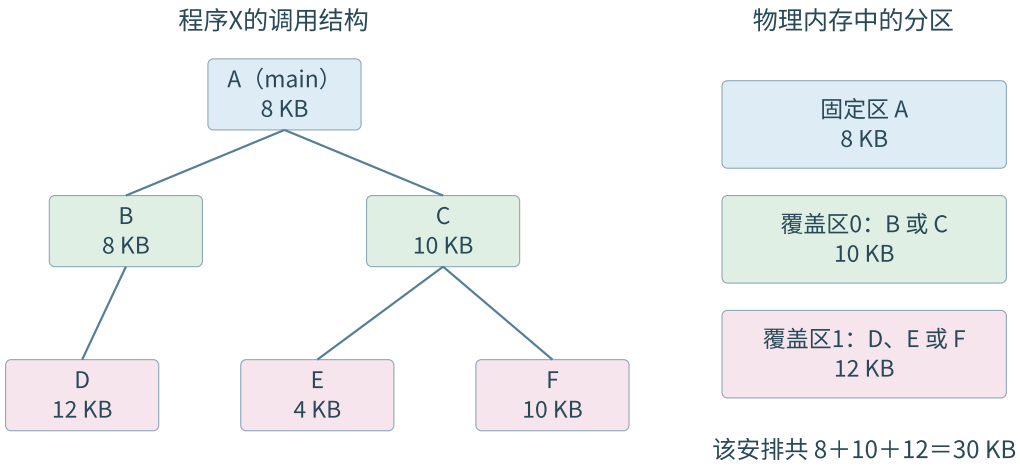


Figure 3.8: A 调用 B、C，B 调用 D，C 调用 E、F；按调用结构划分固定区与两个覆盖区

该安排依赖模块调用关系，必须保证共享覆盖区内的模块不会同时需要保留有效执行内容。30 KB 是这份覆盖安排的容量，不是在任意调用、递归或返回约束下自动成立的全局最小值。覆盖通常要求程序员提供覆盖结构；交换和请求分页由操作系统按其管理策略处理，透明程度不同。

不变量·映射替换页号不改偏移

页大小为 2^d 字节且页框同样大，转换前后低 d 位表示同一页内位置，保持不变；高位才由虚拟页号改成物理页框号。若计算结果的低 d 位发生变化，应先检查是否混入了页表项地址、字段位宽或容量单位。

3.6 分页：页号查表，偏移不变

分页把虚拟地址空间划成固定大小的页，把物理内存划成同样大小的页框，页可以装入不相邻的页框。页表记录虚拟页到物理页框的映射。分页不再要求整个进程连续分配，从而消除了连续分配造成的外部碎片，但进程最后一页可能未用满，仍有内部碎片；页表也占空间。

页大小 $P = 2^d$ B，虚拟地址 a 分解为

$$p = \left\lfloor \frac{a}{P} \right\rfloor, \qquad w = a \bmod P.$$

按除法与取余的抽象示例，即使页长设为 50 B，逻辑地址 110 仍可分解为页号 $\lfloor 110/50 \rfloor = 2$、偏移 $110 \bmod 50 = 10$。该算例说明分解方法；常见硬件采用 2 的幂大小，是为了直接用位字段完成分解。

查得页框号 f 后，物理地址为 $fP + w$。低 d 位偏移不变，高位的虚拟页号被页框号替换。页表项本身也存放在物理内存中，它的地址是另一个物理地址：单级线性页表基址为 $PTBR$、每项 e B 时，页 p 的项地址为 $PTBR + pe$。不要把页表项地址当成最终数据地址。

课件位拼接例。页大小 4 KiB，逻辑地址 4097 分解为页号 1、页内偏移 1；若页 1 映射到页框 9，则物理地址为 $9 \times 4096 + 1 = 36865$。页框号 9 的二进制为 1001_2，将其与 12 位偏移 000000000001_2 拼接，得到 $1001\,000000000001_2 = 9001_H = 36865$。拼接时偏移必须保留完整的 12 位。

配套分页例中，页大小 1 KB，逻辑地址 0A6FH，页 0、1、2、3 分别映射页框 3、7、11、10。$0A6F_H = 2 \times 400_H + 26F_H$，故页号为 2，偏移 26FH；查到页框 11，即 BH，物理地址为 $B \times 400_H + 26F_H = 2E6F_H$。偏移字段跨越十六进制数位，不能简单截取最后两位或三位。

页表项至少要能表示页框号，通常还包含存在、保护、访问和修改

等信息。虚拟地址宽度决定潜在页数，物理地址宽度决定页框号所需位数；两者分别进入“页表多少项”和“每项至少多宽”的计算。

错项排除·多级表的收益来自稀疏

多级页表可不建立未使用虚拟区间对应的下级表，但每个有效映射仍要能表示页框和必要状态。若整个空间都使用，目录反而带来额外开销。因此不能说多级结构减少了每项位数，或对任何进程都一定节省总空间。

3.6.1 基本地址变换的完整检查链

页表寄存器保存页表起始地址和表长等信息；进程未运行时，相关环境可保存在 PCB 中，切换时由内核恢复。设页大小 L、页表基址 F、表长 M 项、每项 e 字节，地址 A 对应 $p = \lfloor A/L \rfloor$、$w = A \bmod L$：

1. 先检查 $0 \le p < M$，等于表长时已经越界。
2. 在紧密连续存放的单级页表中读取 $F + pe$ 处的表项，得到页框号 b 及权限等。
3. 确认本次访问合法后，形成物理地址 $E = bL + w$，再访问目标单元。

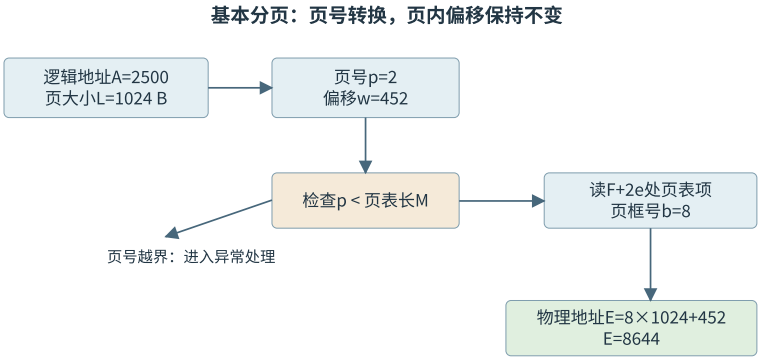


Figure 3.9: 逻辑地址分解、范围检查、页表访问和目标访问是不同步骤

课件十进制例中 $L = 1024$ B、$A = 2500$，故 $p = 2, w = 452$；页 2 映射到页框 8，物理地址为 $8 \times 1024 + 452 = 8644$。页内偏移始终为 452，不能在最后一步误换为 425。页表项地址和最终数据地址则各自按对应公式计算。

3.6.2 页表项位宽、对齐与装表空间

若可寻址物理内存为 4 GiB、页大小 4 KiB，共有 2^{20} 个页框，单是页框号就需 20 位。向上按字节取整至少 3 B；还需容纳权限、存在、访问、修改等字段，实际表项可能更大。

若每项 4 B，一页可放 1024 项，表项边界与页边界整齐对应。若每项 3 B 且表项不得跨页，一页最多 1365 项，并有 1 B 尾部空隙；跨到下一页时，项地址还须计入该布局的空隙。公式 $F + pe$ 采用的是紧密线性布局，不能同时假定每页留空隙又照用同一地址式。

页表总项数、每项长度、一个页面大小分别描述“映射多少页”“一项占多大”和“映射单位多大”，三者不能互换。多级页表的扇出计算也应先确认每张下级页表是否限定占一页。

3.7 TLB 与多级页表：空间与时间分别计算

3.7.1 多级页表为什么可以节省空间

若虚拟页很多，单级页表需要大量表项，即使大部分虚拟区间根本没有使用。多级页表先用高层目录找到所需下级页表，仅为实际使用区域建

立相应下级表；各下级表可以分别放置，不需要一块能容纳整张大页表的连续内存。

多级页表不能减少“一个最终映射项本身需要的位数”，也不保证为一个占满全部虚拟空间的进程节省总空间；全展开时还增加目录开销。它的收益主要体现在稀疏地址空间和分级分配上，代价是未命中转换缓存时要多次查表。

课程连接·缓存对象不同导致分支不同

TLB 缓存地址转换，数据 Cache 缓存内容。TLB 没找到转换时，可从主存页表补入；数据仍可能已在 Cache。缺页则要检查合法页面是否不驻留。把三种事件分别列条件，才能准确计算一次访问是否需要磁盘 I/O。

3.7.2 王道课件：逐级转换与级数计算

逐级转换。某二级分页系统的地址依次由 10 位一级页号、10 位二级页号和 12 位页内偏移构成。逻辑地址为 `(0000000000,0000000001,111111111111)`。一级页号为 0，二级页号为 1，页内偏移为 $2^{12}-1=4095$。按题目页表从目录项 0 取得下级页表，再查其第 1 项，得到页框号 4；最终物理地址为 $4\times4096+4095=20479$。逐级查询得到的页框号才用于末步计算，不能把一级目录项中的下级页表页框号当作目标数据页框号。

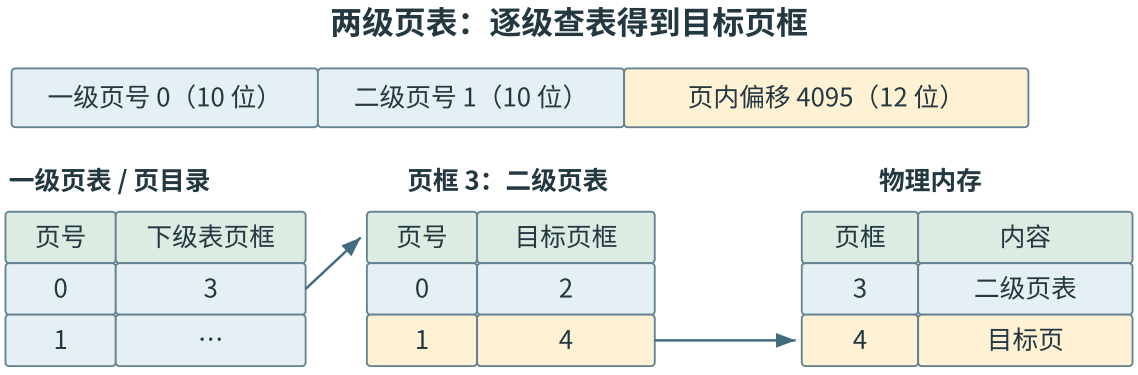


Figure 3.10: 10 ／ 10 ／ 12 位地址依次查页目录、二级页表并形成物理地址

需要多少级页表。逻辑地址 40 位，按字节编址，页面 4 KB，页表项 4 B，且每张页表最多占一个页面。页内偏移为 12 位，虚拟页号为 $40-12=28$ 位；每页最多存放 $4096/4=1024=2^{10}$ 个页表项，每级最多覆盖 10 位页号。因此至少需要 $\lceil 28/10\rceil=3$ 级，顶层不必使用满 10 位。这里的“至少 3 级”依赖每张表最多占一页和题目给出的页表项大小；页表可跨页时须重新确定分级方案。

3.7.3 例 3-3 一张下级表能覆盖多少页

例 3.7.1 (配套例题)

【配套例题】某计算机采用二级页表，按字节编址，页大小为 2^{10} B，页表项大小为 2 B，逻辑地址依次由页目录号、页号、页内偏移量组成。逻辑地址空间大小为 2^{16} 页，表示整个逻辑地址空间的页目录表至少包含多少表项?

A. 64　　B. 128

C. 256　　D. 512

解析。一张按页存放的下级页表能容纳 $2^{10}/2 = 2^9$ 项，故能映射 2^9 个虚拟页。要覆盖 2^{16} 页，需要 $2^{16}/2^9 = 2^7 = 128$ 张下级表，也就是至少 128 个目录项，答案为 **B**。相应字段为目录 7 位、页号 9 位、偏移 10 位；“逻辑空间 2^{16} 页” 不是 16 位字节地址，而是 16 位虚拟页号。

错项排除·多级表的收益来自稀疏

多级页表可不建立未使用虚拟区间对应的下级表，但每个有效映射仍要能表示页框和必要状态。若整个空间都使用，目录反而带来额外开销。因此不能说多级结构减少了每项位数，或对任何进程都一定节省总空间。

3.7.4 例 3-4　10 位字段不能按十六进制整位截断

例 3.7.2 (配套例题)

【配套例题】某计算机主存按字节编址，二级分页地址结构依次为页目录号 10 位、页号 10 位、页内偏移 12 位。虚拟地址 2050 1225H 对应的页目录号、页号分别是（　）。

A. 081H、101H　　B. 081H、401H

C. 201H、101H　　D. 201H、401H

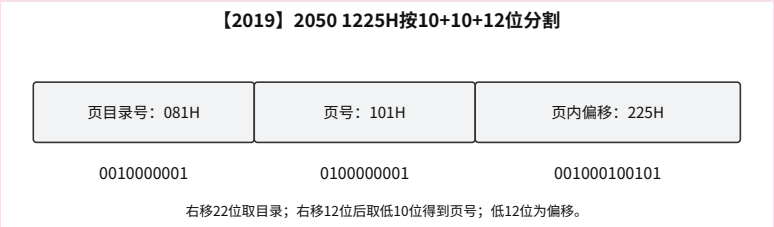


Figure 3.11: 先按二进制位宽分割，再将每一段单独转为十六进制

解析。偏移为低 12 位 225H。目录号为地址右移 22 位，得 081H；下级页号为地址右移 12 位后取低 10 位，得 101H。答案为 **A**。若先把 “205” 当成某个字段，就把十六进制的 4 位分组误当成题设的 10 位字段。

3.7.5 TLB 缓存的是转换，不是文件内容

快表 TLB 缓存近期地址转换及相关权限信息。命中时直接得到页框和权限；未命中时遍历页表，找到有效映射后可填入 TLB。TLB 未命中只说明转换不在该缓存，页面仍可能在内存；页表项表明合法页面不驻留才可能引发缺页。数据 Cache 缓存的则是实际数据，不能把 TLB 命中当成数据已经在 Cache。

课件中的一种状态更新策略。访问命中时可先更新快表中的访问状态，写操作还需更新修改状态；移除该快表项前再把更新的状态写回内存页表，以减少频繁更新慢表带来的访存。页面置换和映射失效处理必须取得一致的状态。不同硬件和系统的状态更新策略可能不同，题目有明确规定时按题设处理。

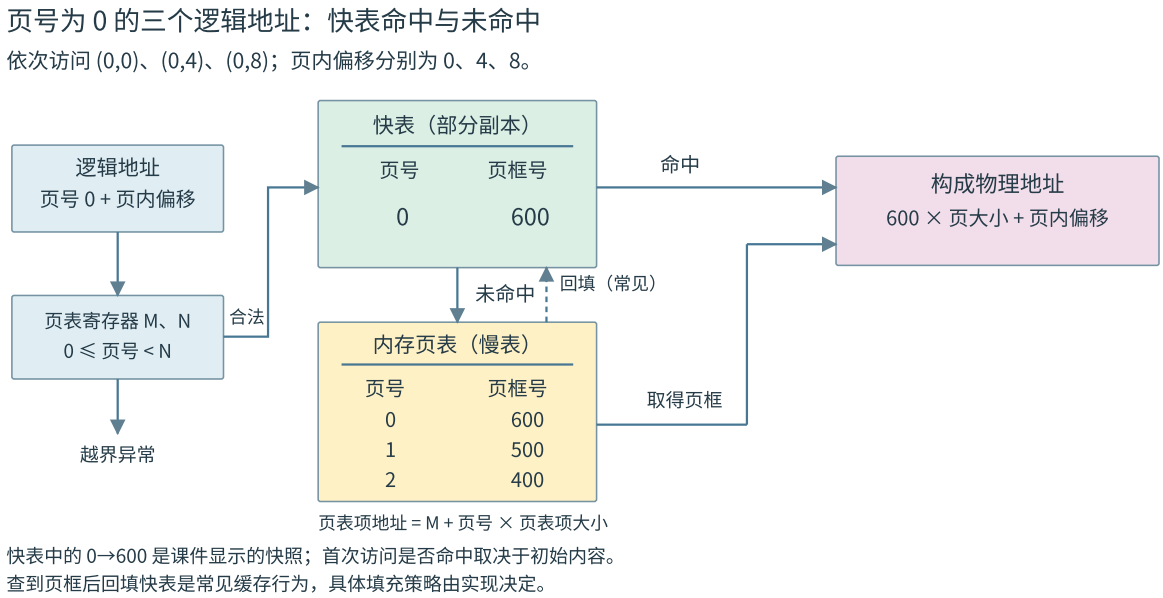


Figure 3.12: 页表与快表分别记录页号到页框的转换；命中及未命中的处理

〖3.2-014〗二级页表、TLB 命中率 75%，忽略 TLB 查询时间，页表均在内存且不考虑缺页。命中访问一次数据内存；未命中需要两次读页表和一次读数据，故平均访存次数为 $0.75 \times 1 + 0.25 \times 3 = 1.5$。这里每个分支都含最后的数据访问，不能只计算两次查表。

若 TLB 查询耗时 t、一次主存访问耗时 m，并且二者串行，则上述平均时间为 $t + 1.5m$。若题目设定 TLB 与其他硬件并行查询、页表遍历缓存或不同层次命中率，必须按相应路径重列分支，不能硬套同一公式。

辨析·并行查询只重叠查表阶段

TLB 与主存页表同时起查时，未命中路径已经消耗部分查表时间。命中后访问数据仍有独立费用，不能把整个访存都当成并行。

3.7.6 课件 TLB 例：串行与并行查询的平均时间

基本分页、单级页表、页面均驻留，查询 TLB 耗时 1 μs，访问一次主存耗时 100 μs，TLB 命中率 90%。若先查 TLB，未命中后才查主存页表，命中路径为 101 μs，未命中路径为 201 μs：

$$E[T] = 0.9 \times 101 + 0.1 \times 201 = 111\ \mu s.$$

若 TLB 与主存页表同时开始查询，命中时可按该模型在 1 μs 后得到页框、再访问目标，仍为 101 μs；未命中时主存页表查询已从时刻 0 开始，随后再访问目标，总计 200 μs：

$$E[T] = 0.9 \times 101 + 0.1 \times 200 = 110.9\ \mu s.$$

无 TLB 时，每次查表和读数据共 200 μs。以上比较限定相同存储访问模型、单级页表和无缺页；增加页表层数或引入其他缓存后，要重列每条实际访问路径。

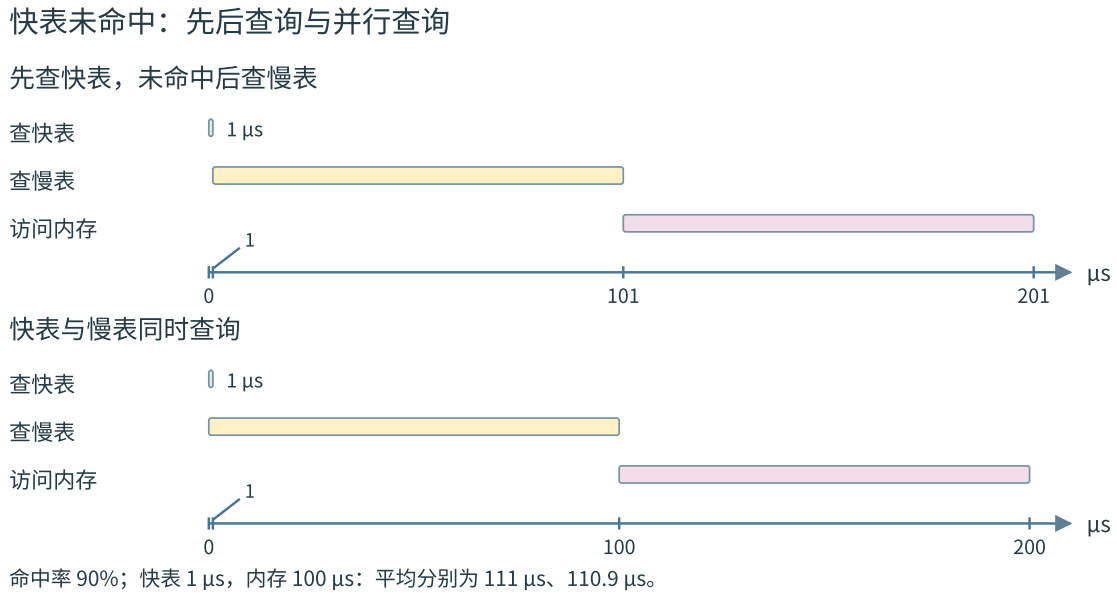


Figure 3.13: 快表未命中时串行与并行查询的时序比较

辨析·并行查询只重叠查表阶段

TLB 与主存页表同时起查时，未命中路径已经消耗部分查表时间。命中后访问数据仍有独立费用，不能把整个访存都当成并行。

3.8 分段和段页式：先检查段内地址

分段按程序的逻辑单位划分地址空间，例如代码段、数据段、栈段，每段长度可不同。逻辑地址用“段号、段内偏移”表示；段表记录段基址、段长和保护等。段号要落在段表范围内，偏移要小于段长，权限要允许本次操作，然后才计算基址与偏移之和。

课件分段例。段 2 的基址为 40 KiB，段内偏移为 1024 B。先检查段 2 存在、权限满足且 1024 小于段长；这些条件成立时，物理地址为 $40 \times 1024 + 1024 = 41984$。若偏移越界，不能直接用基址相加所得值作为有效访问地址。

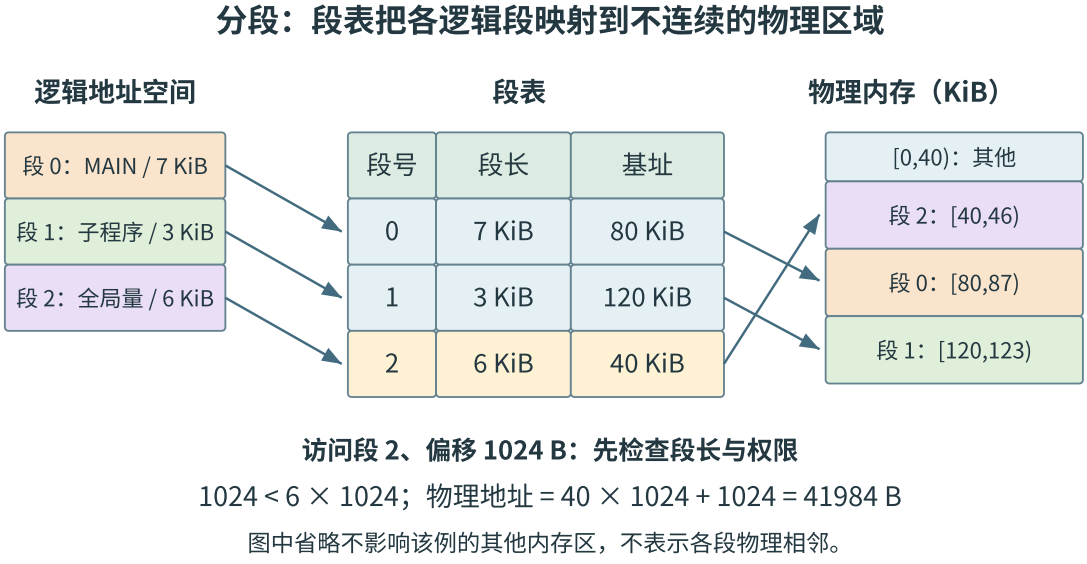


Figure 3.14: 三个逻辑段经段表映射至非相邻物理内存区间

段页式先按逻辑意义分段，再把各段分页；段表指向本段页表，页表再映射页框。它用段组织逻辑单位，用页框安排物理存储，不是“先把物理内存分段，再统一分页”。

转换时先检查段号和段级权限，再按段表给出的页表长度检查段内页号，合法后才查该段页表。题设若另外给出以字节为单位的段长，还须检查完整段内偏移是否越界；末页存在于内存，不会自动取消这个段长限制。

辨析·字段数与逻辑维度分开

段页式常编码段号、页号和偏移，但页号与偏移共同来自段内地址。教材二维地址的结论是在比较独立逻辑坐标。

比较维度	基本分页	基本分段	段页式
划分依据和大小	系统规定的定长页，与程序逻辑模块无必然对应	代码、数据等逻辑单位，段长可变	先按逻辑分段，各段再分成定长页
地址怎样解释	线性地址拆为页号、页内偏移	段号、段内偏移	段号、段内页号、页内偏移；后两项来自段内地址
对程序的可见性	分页通常对程序透明	显式区分各逻辑段	段可见，段内分页通常透明
物理连续性与碎片	页内连续，页间可离散；末页可能有内部碎片	每段连续，段间可离散；可能有外部碎片	每页内连续，同一段可跨离散页框；各段末页可能有内部碎片
共享与保护	通过页映射和逐页权限实现	以完整逻辑段为单位较直观	段级逻辑组织与页级物理映射共同起作用
基本地址转换路径	页表 → 目标单元	段表 → 目标单元	段表 → 本段页表 → 目标单元

最后一行若按**单级页表、各表均在主存、没有 TLB 或其他缓存命中，也不发生缺页、越界或权限异常**计算，分别需要 2、2、3 次主存访问。改为多级页表或引入 TLB 后，应按实际路径重新计数。

【配套例题】某进程段表如下，访问段号 2、段内地址 400 的逻辑地址，结果是（ ）。

段号	段长	内存起始地址	权限
0	100	6000	只读
1	200	—	读写
2	300	4000	读写

段号	状态
0	在内存
1	不在内存
2	在内存

A. 段缺失异常　　B. 内存地址 4400
C. 越权异常　　D. 越界异常

段 2 虽驻留且可读写，但偏移 $400 \geq 300$，已经越界，答案为 **D**。必须先检查长度，不能直接得到 4400。这里的数字是题设地址单位，不必自行加上 KB。

3.8.1 段内分页的空间与地址维度

课件程序的三个逻辑段分别为 MAIN 7 KiB、X 3 KiB、D 6 KiB，页大小 4 KiB，则分别需要 2、1、2 页，共 5 页、20 KiB，最后一页未用空间分别为 1、1、2 KiB，共 4 KiB。每段在逻辑上完整，物理上则可分散到不同页框。

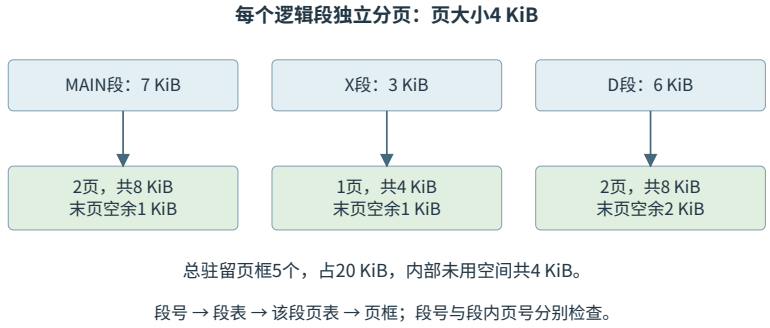


Figure 3.15: 每个逻辑段分别分页，段表定位该段页表，页表定位物理页框

若地址字段依次为段号 16 位、段内页号 4 位、页内偏移 12 位，最大段数为 2^{16}，每段最多 16 页，每页 4 KiB。程序通常给出 “段号、段内地址” 两个独立逻辑坐标，硬件再把段内地址拆成页号和偏移，所以该教材模型仍把段页式地址视为二维；地址编码出现三个字段不自动等于三个独立逻辑维度。

基本转换中先读段表取得该段页表位置和长度，再读页表取得页框，最后读目标。段号与页号都要分别检查，不能因高层查到一个表项就跳过下层越界判断。

错项排除·修改位不是写权限

可写页面刚装入且只被读取时，可以写权限为 1、修改位为 0；只读页被写时，应产生保护异常，不能靠换入页面解决。修改位帮助判断淘汰是否回写，权限位限制行为，两项表示的状态不是同一件事。

3.9 请求分页：合法但不驻留时会发生什么

局部性是程序在一段时期内倾向反复访问某些对象或其相邻位置。时间局部性来自循环与重用，空间局部性来自顺序指令、数组邻近元素等。虚拟内存利用这种特征，只保留当前所需部分，并在需要时调入其他部分，不要求整个进程同时装入。

请求分页的页表项需要区分页面是否驻留，并记录调页所需位置与状态。存在位表示是否在内存；访问位帮助估计近期使用；修改位表示装入后是否被写过，关系到淘汰时是否必须回写。**修改位不是写权限位**：允许写但尚未写的页可以是可写且 M=0；只读保护检查则发生在实际写入前。

比较维度	基本分页	请求分页
执行前的驻留要求	按基本模型，待运行程序所需页面先装入内存	可先装入一部分，合法页面允许暂不驻留
映射记录承担什么	给出页到页框的映射与相应保护信息	还需驻留状态、调页位置及置换所需的访问、修改等信息
TLB 未命中以后	查主存页表取得已有映射	先查页表；驻留则继续，不驻留才进入缺页分支
内存不足时怎样继续	不能只靠基本分页的地址变换解决	可能置换、必要时回写，再调入并更新映射
本次指令怎样结束	正常转换后访问目标单元	缺页处理完成后重试原访存；异常处理开销不能漏计

TLB 是地址转换缓存，基本分页和请求分页都可以使用它；是否有 TLB 与是否允许缺页是两个维度。

一次访存的处理应按执行顺序展开：检查地址合法性，查询 TLB/页表并检查存在状态与访问权限；合法但不驻留则引发缺页异常，内核定位后备内容，取得空闲页框或选择牺牲页；牺牲页若为脏页且需要保留

则回写，调入所需页，更新页表和 TLB，重新执行被中断的访存。访问非法地址应按错误处理，不能一概“给它调一页”。

不变量·调页完成还要重新执行

合法页面不驻留时，原访存尚未成功。缺页处理把内容装入并更新映射后，通常回到原指令重试。因此费用表要区分失败查询、服务开销、重试访问；题目已包含的更新成本不能重复加，重试成本也不能漏掉。

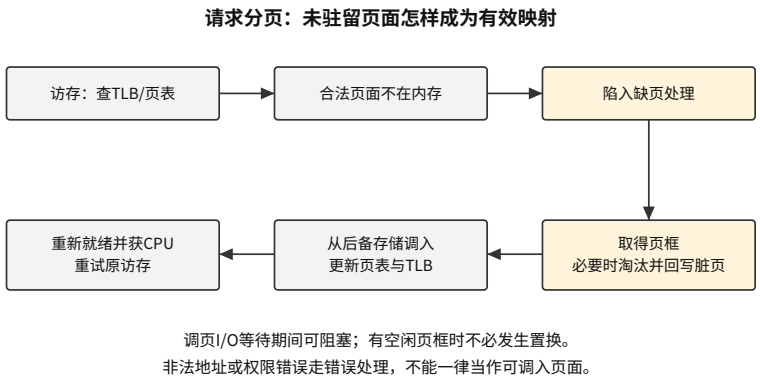


Figure 3.16: 缺页处理会阻塞等待 I/O；页面就绪以后还要重试原访存

3.9.1 课件示例：空闲页框与脏页置换

假设请求页 0、页内偏移 1024。页表中页 0 当前不驻留，其后备地址为 x；页 1 驻留页框 b，页 2 驻留页框 c 且已修改。访问字段 10 和 6 是该课件用于置换参考的记录值，不是单个访问位。

处理前：

页号	内存页框	状态位	访问字段	修改位	外存地址
0	—	0	0	0	x
1	b	1	10	0	y
2	c	1	6	1	z

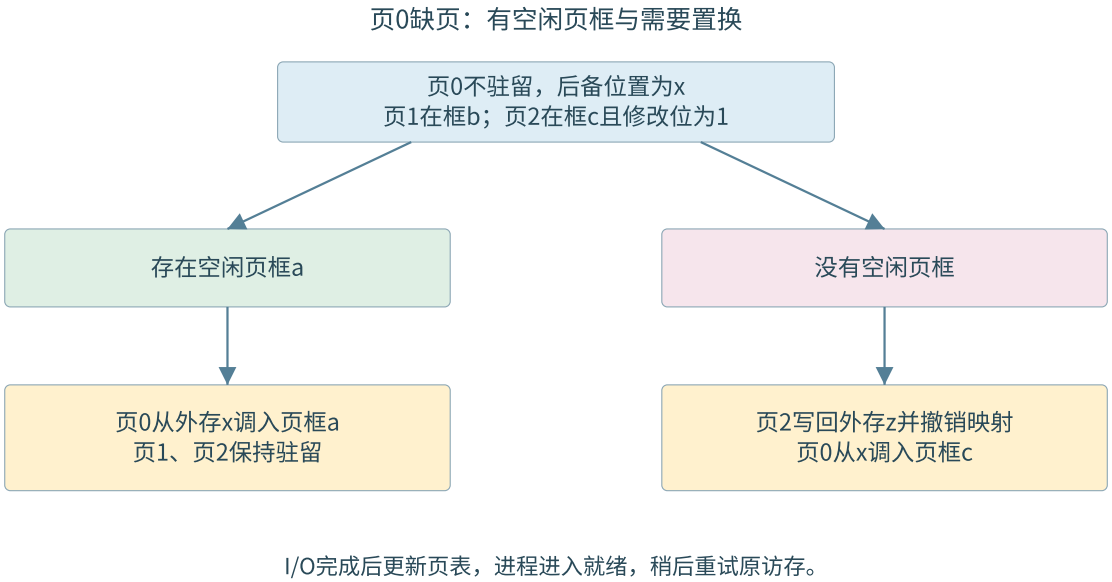


Figure 3.17: 页 0 缺页后，有空闲页框和需要回写置换的两条处理路径

有空闲页框 a：将页 0 从外存 x 调入 a，页 1、页 2 维持原映射，调入后页表为：

页号	内存页框	状态位	访问字段	修改位	外存地址
0	a	1	0	0	x
1	b	1	10	0	y
2	c	1	6	1	z

没有空闲页框：假设置换算法选中页 2，因修改位为 1，先将页 2 写回外存 z，再将页 0 从 x 调入框 c 并更新映射。页 2 已不驻留，状态位变为 0。更新后、重试原访问前的课件页表为：

页号	内存页框	状态位	访问字段	修改位	外存地址
0	c	1	0	0	x
1	b	1	10	0	y
2	—	0	0	0	z

表中的访问字段按课件所展示的处理时刻记录，进程之后重试访问时，系统可按采用的访问统计规则更新它；页 2 的修改内容必须在复用 c 前完成相应回写。

缺页不一定发生置换：有空闲页框就可直接装入。此外，一次复杂指令可能访问多个页面，执行期间可能产生不止一次缺页，系统必须保存足够的重启状态；最少页框数还受指令寻址方式等约束。页面置换算法不能改变 “一条指令所需页面无论如何无法同时提供” 这一基本执行条件。

错项排除·固定分配约束页框数量

本节固定分配把各进程拥有的页框数保持为既定值；若允许直接淘汰别人的页并占为己用，就改变了双方数量。题目采用可变分配时才可讨论相应全局调整，不能把分配策略与置换范围的名字任意拼接。

3.10 页框分配与置换范围

驻留集是进程当前在内存的页面集合。固定分配是指给每个进程固定数量的页框；可变分配允许按运行状况调整。**局部置换**只在本进程驻留集中选牺牲页；**全局置换**可从其他进程等更大集合中选择，会改变各进程实际占有的页框量。

因此经典组合中，固定分配通常配局部置换；可变分配可以局部或全局置换。把固定分配与全局置换直接组合，会使被换出进程的页框量改变，违背这里 “固定” 的约定。常用的分配方法有平均分配、按进程大小分配和按优先级分配等，但还须满足体系结构要求的最少页框数。

请求调页在真正缺页时装入，能减少无用装入；预调页根据局部性提前带入可能需要的内容，可以减少后续等待，也可能调入没用的页。回写策略、清洁页缓冲和空闲页队列影响缺页处理代价，不能只看替换算法的名称。

3.10.1 调入来源、锁定页与虚拟内存特征

课件容量例。 32 位字节地址的寻址上限为 2^{32} B，即 4 GiB。若抽象模型假设 512 MiB 物理内存和 2 GiB 后备外存均可用于该进程，则课件模型的可支持量为 $\min(4\ \text{GiB}, 512\ \text{MiB} + 2\ \text{GiB}) = 2.5\ \text{GiB}$。这是在给定后备容量和使用条件下的示例：实际系统还受到地址保留、共享映射、后备策略和资源限制，不能把物理内存与整块磁盘容量相加当成无条件结论。

虚拟内存的**多次性**指运行信息可以分多次装入，**对换性**指运行期间允许将暂不用内容换出、需要时再换入，**虚拟性**指向程序提供的逻辑使用空间可以超过当前可分配物理内存。虚拟容量首先受地址表示范围约束，也受系统映射与后备能力限制；物理内存和磁盘的简单相加不能替代这些约束。

课件容量例。按字节编址，逻辑地址为 32 位，内存为 512 MiB、外存为 2 GiB，地址表示范围为 2^{32} B，即 4 GiB。课件采用理想化估算：以给定内存、外存容量之和作为可提供空间，再取其与地址范围的较小值，得到 $\min(4,\ 0.5+2)=2.5$ GiB。实际系统还受可用后备空间、保留区域和映射策略约束，不能默认整块外存都可用于该进程。

调页必须知道页面的后备位置。未修改且有有效文件副本的页面，淘汰时可以丢弃内存副本，下次从文件重读；需要保留的脏页则须写到规定的文件或对换区域。匿名页、私有映射与共享文件映射的后备关系不同，写回对象要按模型确定。

部分关键内核页或设备正在直接使用的页会被锁定，不能作为一般置换候选。

页框分配可采用平均、按大小比例或按优先级等方法，最少页框数还要满足一条指令及其寻址过程的需求。可变分配的缺页率控制依据一段运行观察调整资源，某一次缺页本身并不证明进程长期驻留集不足。

最小反例·命中也要更新 LRU 次序

两页框先访问 A、B，再命中 A，随后访问 C。LRU 应淘汰较久未用的 B；FIFO 仍可能淘汰最早装入的 A。只在缺页时移动队列会遗漏命中对最近使用关系的影响，得到的就不是正确的 LRU 轨迹。

3.10.2 三种调入与调出组织

下面三种组织是讨论页面后备位置的典型模型。**对换区充足**时，在进程运行之前，可将有关文件内容复制到对换区，运行中的页面调入和调出均在内存与对换区之间进行。这样可在课件给出的对换区连续分配模型下减少相应磁盘访问开销；具体性能仍取决于实际存储设备和实现。

对换区不足时，可将不会修改的数据保留在文件区，每次需要时从文件区调入；这类页面未被修改，换出时可以丢弃内存副本。需要保留修改的数据在换出时写入对换区，再次需要时从对换区调入。

UNIX 式模型中，尚未使用的页面第一次可从文件区调入；页面被换出并保存到对换区后，再次使用时从对换区调入。是否确实需要回写仍由页面状态与具体实现决定，图中表示该模型的调入来源关系。

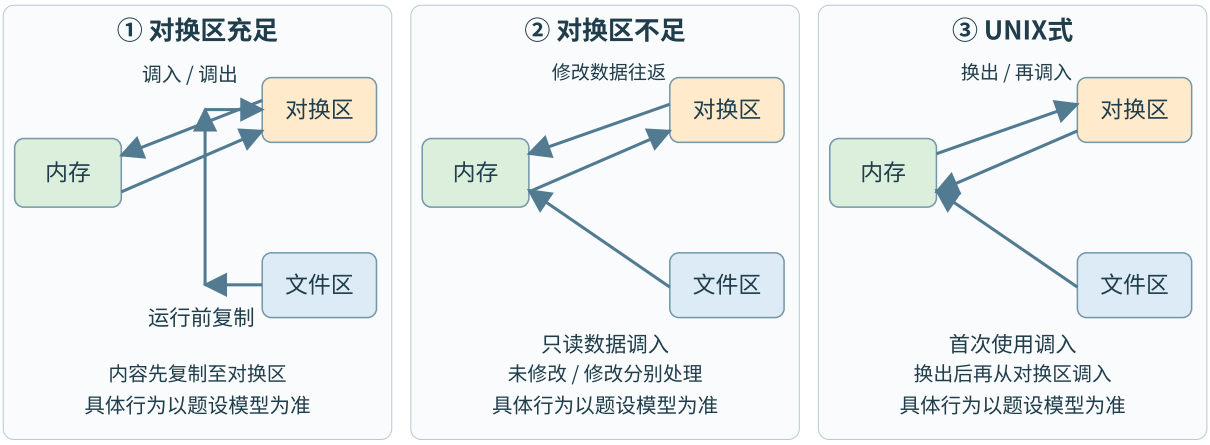


Figure 3.18: 对换区充足、对换区不足及 UNIX 式模型中，内存、文件区与对换区之间的调入和调出方向

3.11 页面置换：必须跟踪实际状态

OPT 淘汰未来最长时间不再访问的页，未来不再访问者优先；它需要未来信息，主要用作比较基准。**FIFO** 淘汰最早装入者，命中通常不改变装入顺序。**LRU** 淘汰最近最久未使用者，每次命中也要更新最近使用关系。若只在缺页时更新 LRU 次序，实现出来就接近 FIFO。

3.11.1 王道课件中的四组置换轨迹

辨析·缺页与置换

空页框装入也缺页；置换须淘汰旧页。

下表按固定物理页框编号展示每次访问后的状态；“—” 表示尚为空，“命” 表示命中，“装” 表示缺页后使用空框，“换 x” 表示缺页并淘汰页 x。每组从空页框开始；缺页次数包含初次装入，置换次数只统计淘汰原页。

OPT，3 个页框。引用串：7、0、1、2、0、3、0、4、2、3、0、3、2、1、2、0、1、7、0、1。共缺页 9 次，其中置换 6 次。

步骤	1	2	3	4	5	6
访问页	7	0	1	2	0	3
页框 1	7	7	7	2	2	2
页框 2	—	0	0	0	0	0
页框 3	—	—	1	1	1	3
结果	装	装	装	换 7	命	换 1

步骤	7	8	9	10	11	12
访问页	0	4	2	3	0	3
页框 1	2	2	2	2	2	2
页框 2	0	4	4	4	0	0
页框 3	3	3	3	3	3	3
结果	命	换 0	命	命	换 4	命

步骤	13	14	15	16	17	18
访问页	2	1	2	0	1	7
页框 1	2	2	2	2	2	7
页框 2	0	0	0	0	0	0
页框 3	3	1	1	1	1	1
结果	命	换 3	命	命	命	换 2

步骤	19	20
访问页	0	1
页框 1	7	7
页框 2	0	0
页框 3	1	1
结果	命	命

例如第 4 步访问 2 时，在现有 7、0、1 中，7 下一次出现在第 18 步，晚于 0 和 1，故淘汰 7；未来不再访问的页可优先淘汰。

FIFO，3 个页框。引用串：3、2、1、0、3、2、4、3、2、1、0、4。共缺页 9 次，其中置换 6 次。

步骤	1	2	3	4	5	6
访问页	3	2	1	0	3	2
页框 1	3	3	3	0	0	0
页框 2	—	2	2	2	3	3
页框 3	—	—	1	1	1	2
结果	装	装	装	换 3	换 2	换 1

步骤	7	8	9	10	11	12
访问页	4	3	2	1	0	4
页框 1	4	4	4	4	4	4
页框 2	3	3	3	1	1	1
页框 3	2	2	2	2	0	0
结果	换 0	命	命	换 3	换 2	命

FIFO，4 个页框。引用串：3、2、1、0、3、2、4、3、2、1、0、4。共缺页 10 次，其中置换 6 次。

步骤	1	2	3	4	5	6
访问页	3	2	1	0	3	2
页框 1	3	3	3	3	3	3
页框 2	—	2	2	2	2	2
页框 3	—	—	1	1	1	1
页框 4	—	—	—	0	0	0
结果	装	装	装	装	命	命

步骤	7	8	9	10	11	12
访问页	4	3	2	1	0	4
页框 1	4	4	4	4	0	0
页框 2	2	3	3	3	3	4
页框 3	1	1	2	2	2	2
页框 4	0	0	0	1	1	1
结果	换 3	换 2	换 1	换 0	换 4	换 3

同一引用串下，FIFO 从三个页框增加到四个页框，缺页从 9 次增加到 10 次，出现 Belady 异常；命中不改变 FIFO 的装入顺序。

辨析·缺页与置换

空页框装入也缺页；置换须淘汰旧页。

LRU，4 个页框。引用串：1、8、1、7、8、2、7、2、1、8、3、8、2、1、3、1、7、1、3、7。共缺页 6 次，其中置换 2 次。

步骤	1	2	3	4	5	6
访问页	1	8	1	7	8	2
页框 1	1	1	1	1	1	1
页框 2	—	8	8	8	8	8
页框 3	—	—	—	7	7	7
页框 4	—	—	—	—	—	2
结果	装	装	命	装	命	装

步骤	7	8	9	10	11	12
访问页	7	2	1	8	3	8
页框 1	1	1	1	1	1	1
页框 2	8	8	8	8	8	8
页框 3	7	7	7	7	3	3
页框 4	2	2	2	2	2	2
结果	命	命	命	命	换 7	命

步骤	13	14	15	16	17	18
访问页	2	1	3	1	7	1
页框 1	1	1	1	1	1	1
页框 2	8	8	8	8	7	7
页框 3	3	3	3	3	3	3
页框 4	2	2	2	2	2	2
结果	命	命	命	命	换 8	命

步骤	19	20
访问页	3	7
页框 1	1	1
页框 2	7	7
页框 3	3	3
页框 4	2	2
结果	命	命

辨析·缺页与置换

空页框装入也缺页；置换须淘汰旧页。

第 11 步装入 3 时，7 最近一次使用在第 7 步，早于其余驻留页，故淘汰 7；第 17 步装入 7 时，8 最久未用，故淘汰 8。命中也必须更新最近使用时间。

3.11.2 例 3-5　缺页次数与置换次数相差初装入次数

例 3.11.1 (配套例题)

【配套例题】某系统采用 LRU 页置换和局部置换。为进程预分配 4 个页框，访问页号依次为 0、1、2、7、0、5、3、5、0、2、7、6。产生页面置换的总次数为（　）。

A. 3　　B. 4

C. 5　　D. 6

解析。下表按“最久未用 → 最近使用”记驻留页，因而每次命中也会移动相应页的位置；开始页框为空。

访问	访问后最近使用次序	结果
0	0	缺页，空框装入
1	0、1	缺页，空框装入
2	0、1、2	缺页，空框装入
7	0、1、2、7	缺页，空框装入
0	1、2、7、0	命中
5	2、7、0、5	缺页，淘汰 1
3	7、0、5、3	缺页，淘汰 2

访问	访问后最近使用次序	结果
5	7、0、3、5	命中
0	7、3、5、0	命中
2	3、5、0、2	缺页，淘汰 7
7	5、0、2、7	缺页，淘汰 3
6	0、2、7、6	缺页，淘汰 5

共 9 次缺页，前 4 次利用空框，实际置换 5 次，答案为 **C**。题目问置换时，不要把所有缺页都记为替换。

3.11.3 Clock 的访问位给页面第二次机会

条件·扫描起点决定首个候选

同类候选超过一个时，选择取决于题设给出的指针起点和扫描方向。

基本 Clock 把页框排成环，指针指向下一个候选。页面被访问时置 A=1；缺页需替换时，检查指针所指页，A=0 就选它，A=1 则清为 0 并前进，直到选中牺牲页。新装入并被本次访问的页设 A=1，指针前移。指针初始位置、装入后位置若题设另有规定，要按题设执行。

课件 Clock 轨迹。五个空页框按 1 至 5 循环，初始指针指向页框 1；引用串依次为 1、3、4、2、5、6、3、4、7。装入页后访问位置 1，指针前移；命中置访问位 1 但不移动置换指针。前五次装入后，页框为 `[1,3,4,2,5]`，访问位均为 1，指针回到页框 1。访问 6 缺页时，从页框 1 起依次将五个 1 清零，随后淘汰 1 并装入 6，状态为 `[6,3,4,2,5]`、访问位 `[1,0,0,0,0]`，指针指向页框 2。之后访问 3、4 命中，访问位变为 `[1,1,1,0,0]`；访问 7 缺页时，从页框 2 起依次清除 3、4 的访问位，在页框 4 遇到访问位 0 的 2 并淘汰，最终页框为 `[6,3,4,7,5]`、访问位 `[1,0,0,1,0]`，指针前移至页框 5。五次初装加两次置换共 7 次缺页；如果题目规定了不同的指针推进规则，应按其规则重算。

访问页	页框 1 至 5	对应访问位	下一扫描页框	结果
1	1、—、—、—、—	1、—、—、—、—	2	装入空框
3	1、3、—、—、—	1、1、—、—、—	3	装入空框
4	1、3、4、—、—	1、1、1、—、—	4	装入空框
2	1、3、4、2、—	1、1、1、1、—	5	装入空框
5	1、3、4、2、5	1、1、1、1、1	1	装入空框
6	6、3、4、2、5	1、0、0、0、0	2	置换 1
3	6、3、4、2、5	1、1、0、0、0	2	命中

访问页	页框 1 至 5	对应访问位	下一扫描页框	结果
4	6、3、4、2、5	1、1、1、0、0	2	命中
7	6、3、4、7、5	1、0、0、1、0	5	置换 2

机制小例（原创）。四页框按环依次为 p, q, r, s，访问位为 1、0、1、0，指针从 p 开始。需淘汰时，先将 p 的 1 清 0，再选 q；r、s 本次未被扫描，原位不变。不能每次缺页都把所有访问位统一清零。这一小例只检查扫描中间状态，未替代真题的选页任务。

改进型 Clock 再考虑修改位。在本书采用的四类优先关系中，依次考虑 (A,M) 为 (0,0)、(0,1)、(1,0)、(1,1)：先选近期未访问且不需回写者，再考虑近期未访问的脏页，近期访问过者留待后续。具体多轮扫描会修改 A，必须按当时状态判断，不能把开始扫描前的分类当作固定次序。

按课件的四轮扫描规则，各轮均从本次置换开始时的指针位置出发，沿环的同一方向最多扫描一圈：

1. 寻找第一个 $(A, M) = (0, 0)$ 的页，找到即选中；本轮不修改访问位。
2. 若首轮失败，寻找第一个 $(0, 1)$ 的页，找到即选中；对本轮经过但未选中的页，将其访问位清为 0。
3. 若第二轮也失败，重新寻找第一个 $(0, 0)$ 的页；本轮不修改访问位。
4. 若第三轮失败，重新寻找第一个 $(0, 1)$ 的页。经过完整的第二轮后，所有候选页的访问位都已清零，所以第三、第四轮中必能选出页面。

扫描过程中页的类别会变化：第二轮被清零的 $(1, 0)$ 可在第三轮变为 $(0, 0)$。因此，应逐轮执行并更新状态，不能只按最初的四类标签一次性排序。扫描只改变 A，M 不能因扫描而任意清零；脏内容完成相应回写后才能标记为干净。

图中的每个状态对按环次序排列，分别表示不同的候选集合；只给状态、未给扫描起点时，不能确定同类候选中最终选中的具体页框。状态 1 和状态 3 含 $(0,0)$，按第一轮规则选择扫描中第一个 $(0,0)$；状态 2 没有 $(0,0)$ 但有 $(0,1)$，首轮失败后第二轮可以选到扫描中第一个 $(0,1)$；状态 4 全部为 $(0,1)$，首轮失败，第二轮选中遇到的第一个页框。实际扫描时，应同步记录第二轮已经过页的访问位变化。

条件·扫描起点决定首个候选

同类候选超过一个时，选择取决于题设给出的指针起点和扫描方向。

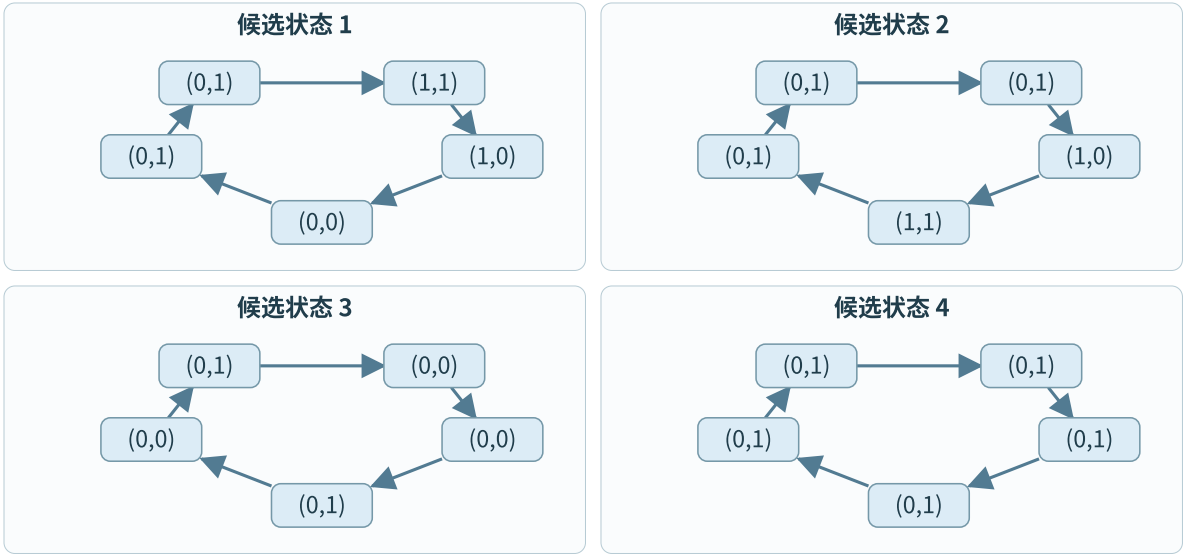


Figure 3.19: 改进型 Clock 四组环状候选状态；括号内依次是访问位与修改位

3.11.4 例 3-6　缺页后，旧页框字段不能直接沿用

例 3.11.2 (配套例题)

【配套例题】某请求分页系统页大小为 4 KB，按字节编址。为进程 P 分配 2 个固定页框，采用改进型 Clock，页表部分内容如下。

页号	页框号	存在位	访问位
2	20H	0	0
3	60H	1	1
4	80H	1	1

页号	修改位
2	0
3	0
4	1

P 访问虚拟地址 02A01H，经地址变换得到的物理地址是（　）。

A. 00A01H　　B. 20A01H

C. 60A01H　　D. 80A01H

解析。低 12 位偏移为 A01H，页号为 2。其存在位为 0，旧的 20H 不是这次可以直接使用的有效驻留映射。两个页框已被页 3、4 占据，需要置换；它们都被访问过，但页 3 干净、页 4 脏，按本题改进型 Clock 先淘汰页 3，把页 2 装到 60H 页框。于是物理地址为 $60_H \times 1000_H + A01_H = 60A01_H$，

答案为 **C**。

若有空闲页框，则不需要淘汰页 3；本题“固定 2 个且均已占用”决定了这一分支。若存在位本来为 1，则无须先执行同样的换页过程。

FIFO 可能出现 **Belady 异常**：增加页框反而增加缺页。LRU 与 OPT 具有栈性质，在相同引用串和一致初态下，增加页框不会使其缺页次数增加。不要把 FIFO 的异常推广成所有算法必有的现象，也不要把“LRU 通常利用局部性”写成对任意引用串必优于 FIFO。

课程连接·映射写入与磁盘持久化分开

程序通过映射地址修改内容，首先改变驻留页并可能置脏；何时写回文件还取决于映射属性和写回机制。文件系统维护后备关系，虚拟内存维护映射和驻留，二者协作不代表每一条赋值都会立即产生一次磁盘写入。

3.12 内存映射文件：把文件内容放入虚拟地址视图

内存映射文件把文件的某个范围与进程一段虚拟地址范围关联，程序用普通访存指令访问映射内容，系统用分页等机制按需建立驻留页。它省去了应用反复显式调用读写、维护用户副本的部分工作，但不表示文件整体一次性进入物理内存，也不表示每次访存立即写到磁盘。

共享映射可让多个进程看到同一文件内容，因而支持进程间通信；对共享可写页的竞争仍须同步。私有映射的修改可通过写时复制保留为本进程私有，不必改原文件。页到文件块的后备关系由映射与文件系统共同维护，不能把这种机制理解成绕过文件系统而直接把磁盘地址当内存地址。

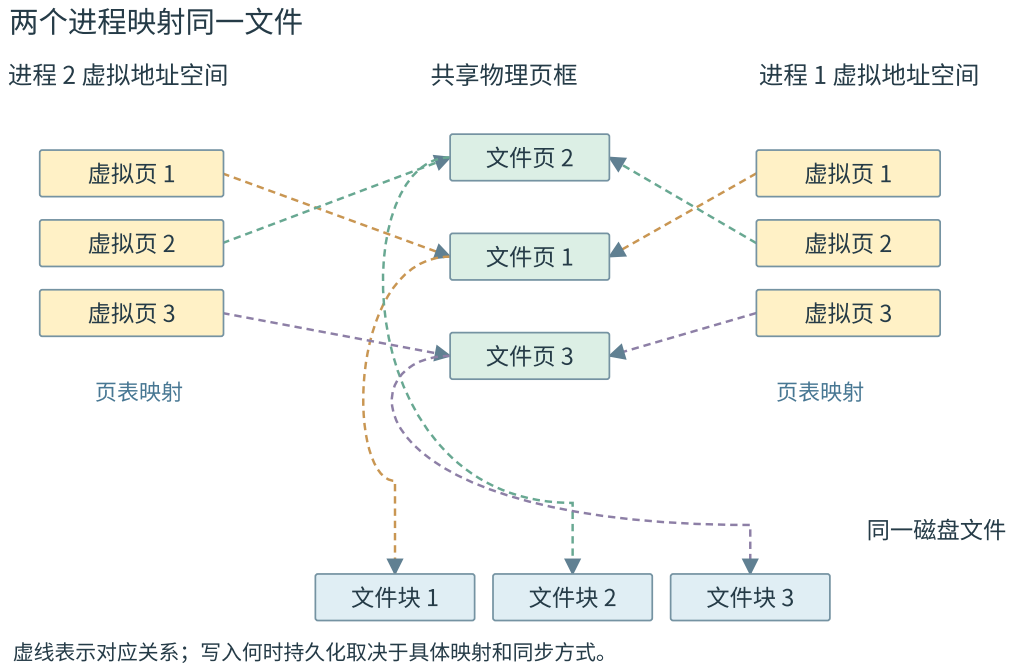


Figure 3.20: 两个进程的文件页映射到同一组物理页框并对应同一文件块

映射以后仍受地址范围和权限约束，映射建立、页面驻留、写回持久化是不同事件。题目若问磁盘 I/O 次数，要检查页面是否已缓存、是否发生缺页、脏页是否回写，不能仅按源代码中的读写次数计算。

3.13 工作集、抖动与平均访问时间

不变量·窗口先取引用再去重

工作集窗口若规定最近 6 次引用，应先截出准确的 6 次，再对页号去重。同一页出现三次仍只占集合中一项。不能先找 6 个不同页，也不能从当前时刻向未来补引用，否则集合不再代表题设的近期需求。

工作集是在给定窗口内实际访问过的不同页面的集合。若窗口按最近 Δ 次引用定义，就从当前位置向前数 Δ 次引用，再去重；不能先数 Δ 个不同页。驻留集描述当前物理状态，工作集描述近期访问需求，工作集不必总是驻留集的子集：刚被引用、尚未调入的页就可能在工作集中而不在驻留集中。

窗口为 4 的原例。页面引用串为 24、15、18、23、24、17、18、24、18、17、17、15。访问完前 4 项时，窗口含 $24, 15, 18, 23$，工作集为 $\{24, 15, 18, 23\}$；访问完第 10 项时，最近 4 次引用为第 7 至 10 项 $18, 24, 18, 17$，去重得到 $\{18, 24, 17\}$，工作集大小为 3。窗口长度是 4，工作集大小可以小于 4，因为同一页面可在窗口内重复出现。

Figure 3.21: 窗口大小为 4 时，两个时刻的工作集

3.13.1 例 3-7　窗口内有重复，集合只记一次

例 3.13.1 (配套例题)

【配套例题】某进程访问页面的序列如下，t 位于所示竖线处，尚未访问竖线右侧的 0。

$$\ldots, 1, 3, 4, 5, 6, 0, 3, 2, 3, 2 \,\big|_t\, 0, 4, 0, 3, 2, 9, 2, 1, \ldots$$

工作集窗口大小为 6，则 t 时刻工作集为（　）。

A. {6,0,3,2}　　B. {2,3,0,4}

C. {0,4,3,2,9}　　D. {4,5,6,0,3,2}

解析。向前取最近六次引用 6、0、3、2、3、2，去重为 {6,0,3,2}，答案为 **A**。集合无需保留重复次数，也不能从 t 向未来取引用；把窗口理解成“六个不同页面”会错误扩大范围。

抖动指页面频繁换出又很快需要换入，大量时间花在换页，实际计算进展很少。若多进程工作集所需总页框超过可用内存，过度增加并发度可能引发抖动。减少活动进程、增加相关进程的可用页框、依据工作集或缺页率调节分配，才能缓解驻留页框不足；仅扩充磁盘交换区容量或提高某进程优先级，不会自动提供它需要的页框。

3.13.2 数组访问怎样变成页引用串

〖3.2-012〗页框为 512 B，数组 `int a[128][128]` 按行优先存放，代码已经驻留，数据只有 1 个页框。下列代码执行会产生多少次缺页?

```
for (int i = 0; i < 128; i++)
    for (int j = 0; j < 128; j++)
        a[j][i] = 0;
```

A. 1　　B. 2
C. 128　　D. 16384

解析。本题采用 4 B 整数，数组按页对齐，数据初始未装入；一行正好 $128 \times 4 = 512$ B，占一页。内层循环 j 增加，访问的是不同的行，即依次跨页；一份数据页框只能留下刚访问的那一行。每次引用都换到另一行，下一列回到第 0 行时，该行那一页也早已被淘汰，因此 $128 \times 128 = 16384$ 次，答案为 **D**。

这里缺页与置换算法无关，因为只有一个数据页框。若把内层访问改为 `a[i][j]`，按行连续访问且上述对齐条件不变，每行初次访问缺页一次，总共 128 次。这是**原创变式**，只改变下标次序，用来检查空间局部性，而非另一道原题。若整数大小、初始驻留或数组页对齐改变，先重建地址和页引用串，再讨论算法，不能只记住 16384 这个数。

不变量·窗口先取引用再去重

工作集窗口若规定最近 6 次引用，应先截出准确的 6 次，再对页号去重。同一页出现三次仍只占集合中一项。不能先找 6 个不同页，也不能从当前时刻向未来补引用，否则集合不再代表题设的近期需求。

3.13.3 有效访问时间要列出所有分支

设不缺页的一次访问平均耗时为 T_0，缺页率为 p，一次缺页的完整分支平均耗时为 T_f，且 T_f 已包括调页、必要写回、处理程序及重试访问，则

$$E[T] = (1-p)T_0 + pT_f.$$

若题目把缺页服务时间 D 定义成正常访存以外的附加开销，则可写 $E[T] = T_0 + pD$，两种写法只是费用定义不同。也不能把 TLB 命中率与缺页率相加，当作一个统一的命中率；应按条件概率和完整路径分支。

原创计算检查。若 $T_0 = 100$ ns，缺页完整分支为 10 ms，希望平均访问不超过 200 ns，则

$$p \le \frac{200 - 100}{10^7 - 100} \approx 10^{-5}.$$

毫秒先换成纳秒，才可与 100 相减。这个数量级说明为什么低缺页率仍可能显著影响时间，不是对任何实机性能的统计结论。

3.13.4 例 3-8　一次缺页要把“失败、调页、重试”全部计时

例 3.13.2 (配套例题)

【配套例题】请求分页管理系统中，假设某进程的页表内容如下表所示。

页号	页框号	有效位（存在位）
0	101H	1
1	—	0
2	254H	1

页面大小为 4 KB，一次内存的访问时间为 100 ns，一次快表（TLB）的访问时间为 10 ns，处理一次缺页的平均时间为 10^8 ns（已

含更新 TLB 和页表的时间）。进程的驻留集大小固定为 2，采用最近最少使用置换算法（LRU）和局部淘汰策略。假设：①TLB 初始为空；② 地址转换时先访问 TLB，若 TLB 未命中，再访问页表（忽略访问页表之后的 TLB 更新时间）；③ 有效位为 0 表示页面不在内存中，产生缺页中断，缺页中断处理后，返回到产生缺页中断的指令处重新执行。设有虚地址访问序列 2362H、1565H、25A5H，请问：

（1）依次访问上述三个虚地址，各需多少时间？给出计算过程。

（2）基于上述访问序列，虚地址 1565H 的物理地址是多少？请说明理由。

不变量·窗口先取引用再去重

工作集窗口若规定最近 6 次引用，应先截出准确的 6 次，再对页号去重。同一页出现三次仍只占集合中一项。不能先找 6 个不同页，也不能从当前时刻向未来补引用，否则集合不再代表题设的近期需求。

解析。4 KB=1000_H B，低 12 位是页内偏移。三个地址分别访问页 2、页 1、页 2，偏移依次为 362H、565H、5A5H。先连续跟踪这三个访问，不能把每个地址都当作从 “TLB 为空” 开始。

本次访问	转换与驻留状态	耗时
2362H	TLB 未命中；页 2 驻留，读页表后读目标内存；将页 2 的转换留在 TLB	$10+100+100=210$ ns
1565H	TLB 未命中；页 1 不驻留；处理缺页，更新页表和 TLB；重试时命中 TLB，再读目标内存	$10+100+10^8+10+100=100000220$ ns
25A5H	页 2 仍驻留，其转换仍在 TLB，直接命中后读目标内存	$10+100=110$ ns

第二次访问时两个页框已占满，需在本进程的页 0、页 2 中局部淘汰。页 2 刚被第一次访问使用，因此 LRU 选择页 0。将页 1 装入页框 101H，页 0 的存在位清零，并使其旧 TLB 转换失效；页 2 的映射不变。于是

$$PA(1565_H)=101_H\times 1000_H+565_H=101565_H.$$

题目把页表、TLB 更新包含在 10^8 ns 中，却明确要求返回原指令重试。因此更新成本不重复加，重试的 10+100 ns 仍须加。第三次访问能命中，也依赖连续执行所得的状态；TLB 未命中、缺页、页面置换是三个不同事件。

3.13.5 例 3-9　数组偏移怎样逐级找到页表项

例 3.13.3 (配套例题)

【配套例题】某 32 位系统采用基于二级页表的请求分页存储管理方式，按字节编址，页目录项和页表项长度均为 4 字节，虚拟地址结构如下所示。

页目录号（10 位）	页号（10 位）	页内偏移量（12 位）

某 C 程序中数组 `a[1024][1024]` 的起始虚拟地址为 1080 0000H，数组元素占 4 字节，该程序运行时，其进程的页目录起始物理地址为 0020 1000H。请回答下列问题。

（1）数组元素 `a[1][2]` 的虚拟地址是什么？对应的页目录号和页号分别是什么？对应的页目录项的物理地址是什么？若该目录项中存放的页框号为 00301H，则 `a[1][2]` 所在页对应的页表项的物理地址是什么？

（2）数组 a 在虚拟地址空间中所占的区域是否必须连续？在物理地址空间中所占区域是否必须连续？

（3）已知数组 a 按行优先方式存放，若对数组 a 分别按行遍历和按列遍历，则哪种遍历方式的局部性更好？

课程连接·计算顺序

先列地址字段、页表状态和访问分支，再计算偏移、访问次数或换页代价。

解析。（1）C 的下标从 0 开始，按行优先存放。到 `a[1][2]` 的字节偏移为 $(1 \times 1024 + 2) \times 4 = 4104 = 1008_H$，故其虚拟地址为

$$1080\,0000_H + 1008_H = 1080\,1008_H.$$

然后按位宽拆分，并把 “表项地址” 和 “表项中保存的页框号” 分开计算。

所求对象	计算	结果
页目录号	$VA \gg 22$	042H
页号（下级表索引）	$(VA \gg 12)\ \&\ 3FF_H$	001H
页内偏移	$VA\ \&\ FFF_H$	008H
页目录项物理地址	$0020\,1000_H + 042_H \times 4$	0020 1108H
下级页表起始物理地址	$00301_H \times 1000_H$	0030 1000H
页表项物理地址	$0030\,1000_H + 001_H \times 4$	0030 1004H

题目没有给出这个页表项中的最终页框号，不能把 0030 1004H 当作 `a[1][2]` 的物理地址；它是下一步需要读取的映射项的位置。

（2）该 C 数组的元素按规定连续排列，所以所占虚拟区域必须连续；相邻虚拟页可映射到离散的页框，物理区域不必连续。页内连续与跨页离散可以同时成立。

（3）每行 $1024 \times 4 = 4096$ B，恰为一页，起始地址也按页对齐。按行遍历在同一页中连续取元素，空间局部性更好；按列遍历的相邻元素相差 4096 B，会逐行跨页。但原题没有给驻留集大小、替换规则和初始状态，不能进一步声称按列必然每次缺页。地址跨度决定访问模式，缺页次数还取决于驻留状态。

3.14 分支自测：先写地址字段和状态，再计算

以下为原创回顾设问。

1. 32 位虚拟地址、4 KB 页、4 B 页表项，单级页表若覆盖全部空间，需要多少表项和多少字节？
2. TLB 未命中而页表存在位为 1，是否一定访问磁盘？存在位为 1、写权限为 0，写访问应怎样处理？
3. 某页修改位为 0，是否表示程序没有读过该页？
4. 例 3-5 中为何 9 次缺页只有 5 次置换？
5. 工作集窗口增加，集合大小是否每次恰好增加 1？

3.14.1 自测答案

1. 页号 20 位，共 2^{20} 项，占 2^{22} B，即 4 MB；这与物理页框总数不是同一个计算。
2. 未命中可通过主存页表取得映射，不必读盘。写权限不足应触发保护异常，不能当作普通缺页调入后允许写。
3. 否。读访问可以使访问位置 1，却不使修改位置 1。
4. 最初四次使用空闲页框，尚未淘汰任何页；之后满框缺页才置换。

5. 不会。新纳入的一次引用可能已在集合中，窗口内重复引用只算同一个页。

课程连接·检查清单

对于每次引用，说明页号与偏移、命中与缺页，以及地址转换是否需要重新执行。

3.15 主动回忆自测

以下为自拟回忆题，先画地址字段或状态，再写结论。

从合法地址追踪到完整费用

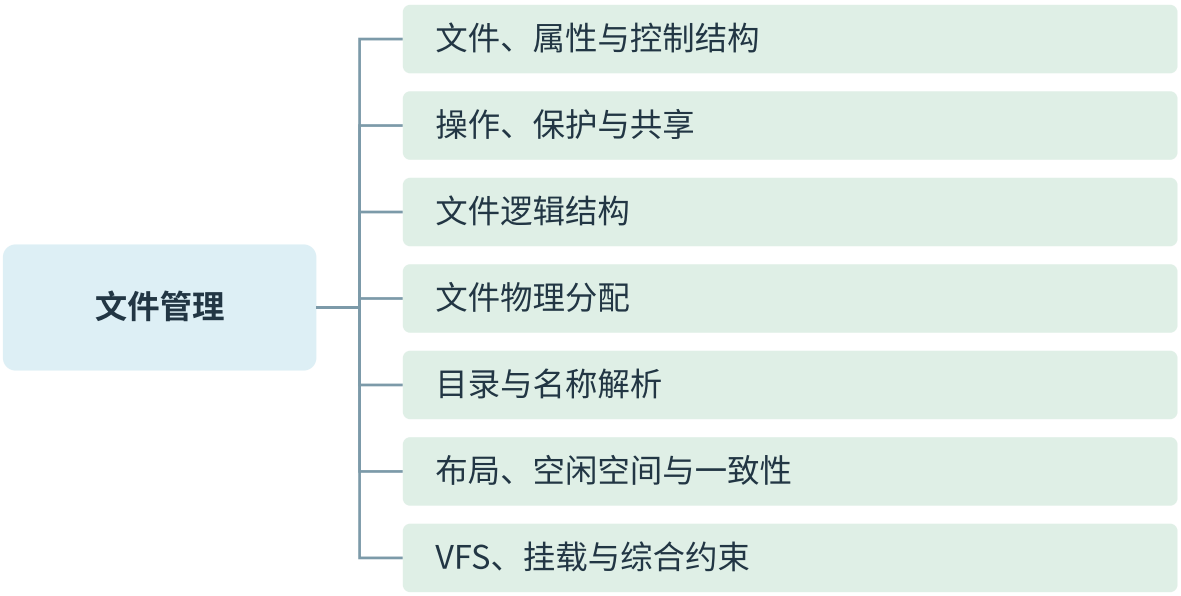
1. 虚拟地址宽 32 位，页 4KiB，页表项 4B，覆盖全部虚拟空间的单级页表有多少项、占多少空间?
2. 两个各 6MiB 的不相邻空洞，能否满足连续 8MiB 请求？分页消除了哪类碎片，又留下哪类?
3. 页大小 1KiB，虚拟地址 0A6FH，页 2 映射到页框 BH，物理地址是多少?
4. 二级页表 TLB 命中率 75%，忽略 TLB 耗时且无缺页，每次查询均串行，平均需几次主存访问?
5. 段已驻留且可写，段长 300，偏移 400，能否访问基址加 400?
6. 两页框访问 A、B、A、C，LRU 与 FIFO 分别淘汰谁?
7. 缺页 9 次，其中 4 次使用初始空页框，置换多少次？何时还要计算回写?
8. 解释工作集、驻留集、内存映射建立、脏页写回四个概念为什么不能混用。

3.15.1 核对答案与计算

1. 页内偏移 12 位，虚拟页号 20 位，共 2^{20} 项，占 2^{22}B，即 4MiB；物理地址宽度影响表项中页框号，而非虚拟页数。
2. 不能，连续洞不够大。分页消除对整个进程连续分配产生的外部碎片，最后一页仍可能有内部碎片，页表也有开销。
3. $A6F_H = 2\times400_H + 26F_H$, 偏移保持 $26F_H$, 结果 $B\times400_H + 26F_H = 2E6F_H$。
4. 命中 1 次读数据, 未命中 2 次查表加 1 次读数据; $0.75\times1 + 0.25\times3 = 1.5$ 次。若查询并行或有缺页须重新列路径。
5. 不能。$400 \geq 300$ 越界，驻留与可写均不撤销段长检查。
6. 命中 A 使它在 LRU 中最近，故淘汰 B；FIFO 仍按装入先后淘汰 A。
7. 置换 5 次；被淘汰页若脏且需保留内容，应另计写回，服务时间已包含时不重复加。
8. 工作集是近期需求集合，驻留集是当前物理状态；建立文件映射规定地址与后备关系，不代表所有页驻留；脏页写回实现相应持久化，发生时刻另由机制决定。

文件管理 4

章节脉络



课程连接·两条路径

目录路径解决名称定位，文件映射解决逻辑内容到磁盘块的定位，两者的 I/O 可能分别发生。

4.1 名称路径、数据路径与分配单位

文件题先画四类节点

1. 名称路径：根或当前目录，逐层目录项，以及解析出的 inode。
2. 使用关系：进程 fd、打开实例、共享 inode，标明位置指针属于哪一层。
3. 数据路径：文件内偏移到逻辑块号，再经过直接或间接索引取得物理块号。
4. 成本与容量：逐个扣除已缓存节点；分别计算数据块、索引块、inode 和空闲管理的约束。

定义 4.1.1 (名称、文件本体和一次打开是三个对象)

目录项建立名字与文件标识的联系；inode 保存文件本体的属性和块定位；打开实例保存这次使用的访问方式及当前位置，进程描述符再引用打开实例。三者的共享关系、计数与生命周期分别判断。

4.1.1 混合索引容量的逐层推导

每增加一级索引，覆盖数乘一次扇出

设块长为 B 字节，每个块号占 e 字节，一个纯索引块可容纳 $q = \lfloor B/e \rfloor$ 个地址；inode 有 d 个直接项和各一个一级、二级、三级间接项。直接区覆盖 d 个数据块；一级覆盖 q 个，二级覆盖 q^2 个，三级覆

盖 q^3 个。因此理论用户数据上限为 $(d+q+q^2+q^3)B$。这要求相应地址字段可表示所需块号、所有分支可用，且未遇文件大小字段或卷容量等更紧上限；索引块开销另外计算。

□

错项排除·同号不一定同文件

inode 号要在所属文件系统内解释。两个不同分区上都出现 inode10，不足以证明它们指向同一文件；同一文件系统中，两个名字解析到同一个 inode 才表示这种对象共享。判题时先确定命名空间，再比较数字。

文件系统把 “按名称找到数据” 分成几步：由路径查目录项，由目录项找到文件控制信息，再把文件内偏移转换成存储块位置。目录、索引节点、打开文件表和数据块分别承担不同工作；把它们笼统当成 “文件的信息”，就会把访问次数算混。

本章文件内字节偏移和逻辑块号从 0 开始；“第几个记录” 若采用自然序号则从 1 开始。除题目另有定义，1 KB 为 1024 B。计算访盘次数时，要逐次说明所访问的目录块、索引块或数据块中哪些尚不在内存；已缓存的内容不重复计入。

4.2 文件、文件控制块与索引节点

文件是具有名称和一定属性的持久数据集合。从使用者看，它可以是字节流，也可以由记录组成；从存储看，它最终占据若干物理块。文件的名称与内容不是同一个对象，一个文件可以通过多个名称被访问。

文件控制块 FCB 是系统描述和管理文件的数据结构，通常记录类型、大小、所有者、权限、时间和存储位置等元数据。将文件名与其他描述信息分离时，目录项可只保留 “文件名—索引节点号”，其余信息由**索引节点 inode** 保存。inode 是这种设计中的文件控制信息载体，不是文件数据块，也不只是 “指向文件的一个指针”。

索引节点号的意义局限于相应文件系统，不能只凭两个分区上的编号相同就判断是同一个文件。内存中的 inode 还可包含运行时锁、引用计数等；它与磁盘中持久保存的 inode 相关，但字段不必完全相同。

目录本身也是文件，其中存放的是用于查找对象的目录项；普通文件的名字出现在目录中，数据内容并不因此存入目录。采用 inode 的模型还应区分三个位置：名字在目录项中，属性和块地址在 inode 中，用户内容在数据块中。

对象	记录什么	由什么找到它	何时改变或回收
目录项	当前目录内的名字及 inode 号	逐层解析路径、查找所在目录	建立、重命名、增删链接时改变；关闭文件不删除它
磁盘 inode	文件类型、大小、权限、链接数、数据块地址等持久信息	在本文件系统解释 inode 号	文件内容或属性改变时更新；须满足无链接、无有效使用引用等回收条件
内存 inode	已载入的文件信息及运行时锁、引用状态等	inode 缓存或打开对象引用	可由多个打开者共享；无打开者后是否立即释放取决于缓存模型
打开实例与进程描述符表	打开实例记录文件位置、访问方式；进程表把 fd 关联到实例	已返回的 fd 在所属进程内解释	open 建立，dup 或继承可共享实例，close 释放相应引用

因此 “知道文件名”“知道 inode 号”“inode 已在内存” 和 “数据已缓存” 依次消除了不同查找步骤。访盘题要把题设明确命中（已在内存）的对象从访问路径中扣除，不能由其中一个条件推得其他对象也都在内存。

4.3 建立、打开、读写、关闭与删除

最小反例·关闭只撤销本次使用引用

一个文件有名字 a，进程打开后再关闭，目录里的 a 仍存在，文件数据也应保留。close 减少的是打开使用关系，unlink 取消的是名字关系。只有满足没有硬链接、没有有效使用引用等条件，才讨论对象最终回收。

建立文件要分配文件控制信息，在父目录增加名字与文件标识的关联，并按规则初始化属性；空文件可以暂不分配实际数据块。**打开文件**按路径查找、检查权限，在内存建立或引用打开状态，返回文件描述符。打开不等于把整个文件内容读入内存。

一个常用抽象分为三层：进程文件描述符表把小整数 `fd` 指向打开实例；打开实例保存本次打开的状态，例如当前文件位置和访问方式；内存 inode 保存文件本身的共享属性及定位信息。不同教材有时把后两层合称系统打开文件表，本书计算前会说明各字段属于哪一层，避免因表名不同把状态混在一起。

4.3.1 课件示例：打开目录 M 中的文件 A

课件模型中，外存目录 M 含 `FCB_A`、`FCB_B`、`FCB_C`、`FCB_D`，最近访问的目录项也可缓存在内存。调用 `open(.../M/A, 只读)` 时，系统沿路径查找并读入必要目录，找到 A 的目录项，建立或引用系统打开文件表项，再为调用进程建立表项并返回描述符。

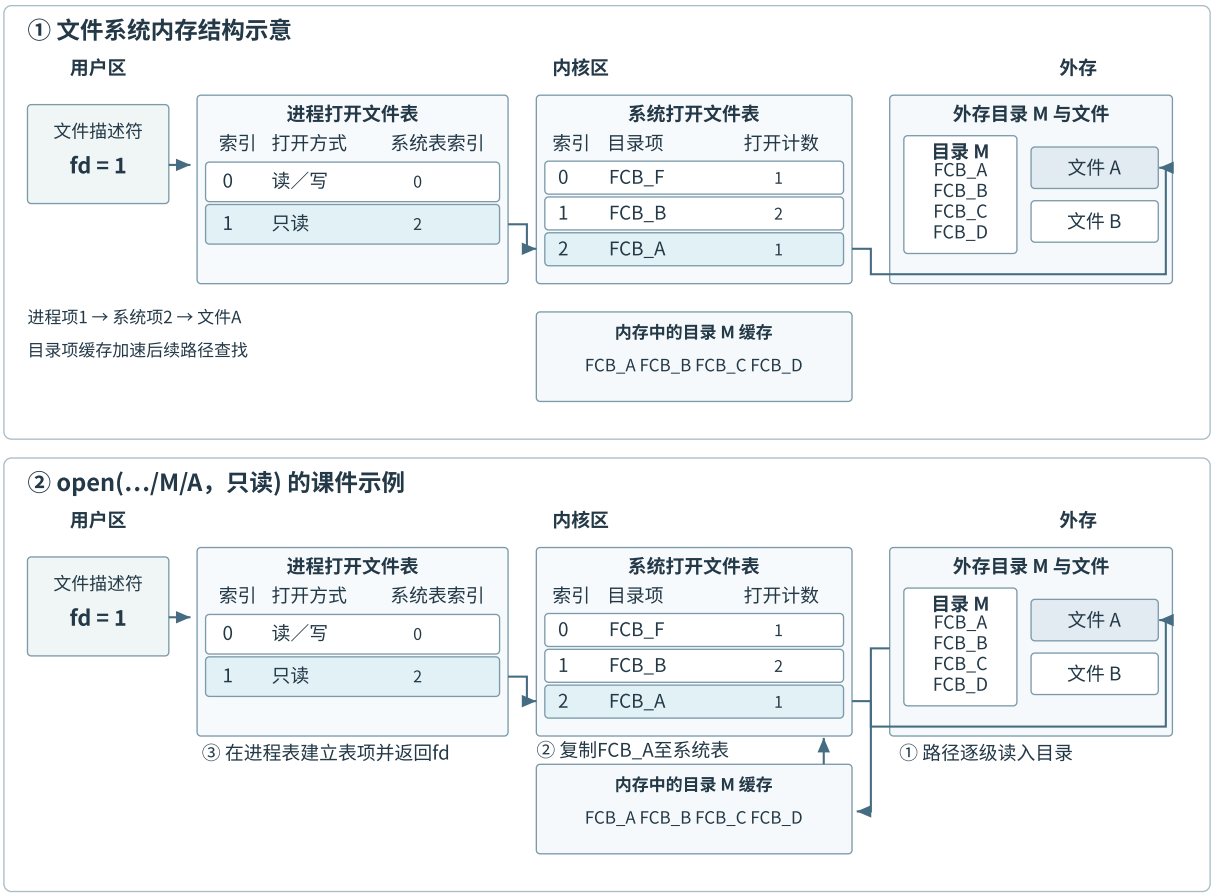


Figure 4.1: 课件中的目录缓存、进程打开表、系统打开表及文件 A 的打开路径

图中进程打开表的 1 号项以只读方式指向系统表的 2 号项；系统项 2 是 `FCB_A`，打开计数为 1，返回 `fd=1`。同一系统表还有 `FCB_F`、`FCB_B` 两项，其打开计数分别为 1 和 2；这两个数字用于示例中各表项的引用关系，不能看成 A 的硬链接数。目录缓存是名称查找的加速结构，打开文件表管理已经打开的对象，两者职责不同。

`read(fd,buf,n)` 根据打开状态和当前偏移找到目标数据，必要时发起 I/O，向用户区返回实际读到的字节，并更新相应文件位置。`write` 可能分配新块、改变大小、把缓存标记为脏；返回成功不必表示数据已立即持久化到盘片。文件结束、短读、权限错误等应按题给语义处理。

关闭释放本次打开引用，必要时更新元数据；它不等于删除文件。**删除名字**从目录取消关联，硬链接计数随之变化；文件内容何时真正回收，还要检查是否仍有其他硬链接或有效打开引用。两类计数含义不同：磁盘硬链接数统计名字关联，打开引用数统计运行中的使用关系。

辨析·同一文件，多次打开

图按课件模型把指针记在进程表、文件打开计数记在系统表。具体系统的表项划分与偏移共享规则需按题目说明判断。

4.3.2 例 4-1　最后关闭，释放的是哪一层

例 4.3.1 (配套例题)

【配套例题】若文件 F 仅被进程 P 打开并访问，则当进程 P 关闭 F 时，文件系统需要完成的是（　）。

A. 删除目录文件中 F 的目录项

B. 释放 F 的索引节点所占的内存空间

C. 释放 F 的索引节点所占的外存空间

D. 将文件磁盘索引节点中的链接计数减 1

解析。本题按 “最后关闭时释放相应内存控制信息” 的简化模型处理，答案为 **B**。A、C 属于删除持久对象的工作，D 对应硬链接关系改变；关闭并不取消文件名。实际系统可以把暂时没有打开者的 inode 留在缓存里，缓存回收时机与本题抽象不同，不能据此把 close 与 unlink 混为一谈。

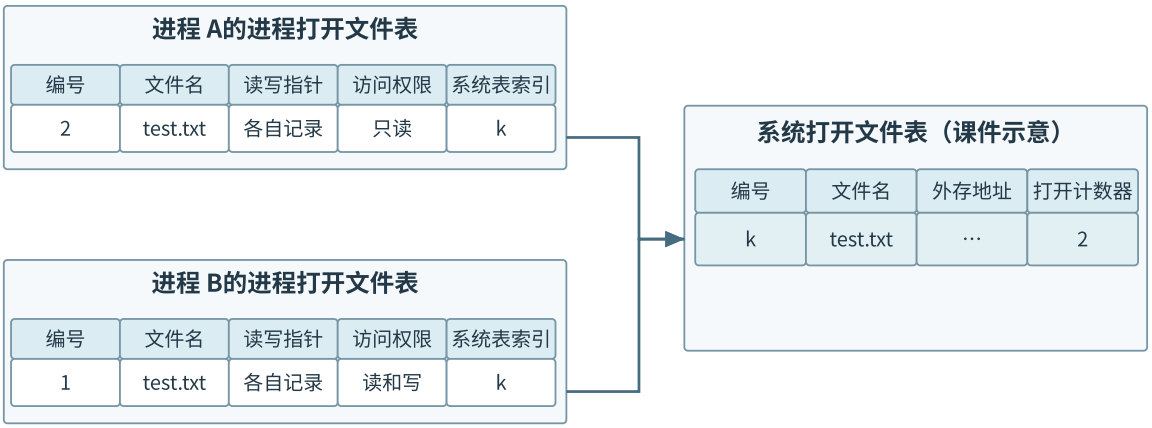
4.3.3 文件位置可以共享，也可以独立

两个进程分别 `open` 同一文件，通常得到两个打开实例，各有自己的文件位置，但都指向同一文件的 inode。通过 `dup` 或进程继承共享同一打开实例时，文件位置也可能共享。因此 “同一个文件” 不决定读写位置是否相同；先看是否同一次打开的引用。

多个进程以写方式打开同一文件，不表示文件系统自动满足应用所需的完整事务互斥。要防止记录被交错修改，应使用规定的锁或其他同步方式。系统对单次写的原子性规则也不能任意推广为整个 “读—修改—写” 序列的原子性。

4.3.4 课件示例：两个进程打开同一文件

课件的抽象模型把每个进程的读写指针、访问权限放在各自的进程打开表中，把文件及其打开计数放在系统打开表中。进程 A 的 2 号表项以只读方式打开 `test.txt`，进程 B 的 1 号表项以读写方式打开同一文件；两个表项均指向系统表的 k 号表项，系统表所示打开计数为 2。



课件模型：两进程记录各自权限和读写指针，系统表记录文件及打开计数。

Figure 4.2: 两个进程的打开文件表指向系统表中同一文件项，打开计数为 2

这一课件示例中，两进程的读写指针和权限各自记录。它与前面把偏移存于“打开实例”的模型采用了不同的表项划分：对具体系统要先确定表项代表共享文件信息还是一次打开，再判断偏移是否共享；同一个文件身份本身并不决定两次使用是否共享偏移。系统表中的打开计数也不能当作磁盘 inode 的硬链接数。

错项排除·权限集合不能压成一个编号

每类用户可能同时有读、写等多种权限；若有 5 个独立许可就要表达其全部子集，需要 5 位。给用户类别和某一操作编码，只能描述一次选择，不能保存每一类用户各自的完整许可组合，二者信息量不同。

4.4 文件保护：用户类别与权限组合

保护回答谁能对对象执行什么操作。口令、加密和访问控制各有作用，不能相互替代：加密保护内容不被未授权读取，访问控制由系统按身份与权限限制操作；读权限、写权限、执行权限也不同于调度优先级。

访问矩阵以主体为行、对象为列，格中保存允许操作集合。按对象列存放形成访问控制表 ACL，便于查“谁能使用这个文件”；按主体行组织授权可形成能力表一类机制，便于查“这个主体能用哪些对象”。保护还涉及授权、撤销和检查时机；权限存进去之后，访问时仍须逐次检查。

课件以一个文件的访问控制表说明：1 表示该用户允许对应操作，0 表示不允许。此表记录的是各主体针对同一文件的权限，并非文件内容本身。

用户	读	写	执行	添加	删除	列表清单
father	1	1	1	1	1	1
mother	1	0	1	0	0	1
son	0	0	0	0	0	0

例如 mother 可以读取或执行，但表中未授予写入、追加和删除。若用户很多，可以按组记录权限；课件的另一示例中，系统管理员拥有五项权限，文件主拥有执行、修改、读取和写入，文件主的伙伴拥有执行和读取，其他用户均未被授予这些权限。访问时仍须先确定主体所属组，再按表和系统冲突处理规则检查。

4.4.1 例 4-2　权限可以同时成立，不能只编码一种

例 4.4.1 (配套例题)

【配套例题】某文件系统针对每个文件，把用户分成安全管理员、文件主、文件主的伙伴、其他用户 4 类；访问权限有完全控制、执行、修改、读取、写入 5 种。若 FCB 中用二进制位串表示不同类别用户对一个文件的访问权限，至少需要多少位?

A. 5　　B. 9　　C. 12　　D. 20

解析。对一类用户，五种权限可分别有或没有，需要 5 个独立二进制位。四类用户分别保存其权限组合，需要 $4 \times 5 = 20$ 位，答案为 **D**。用 2 位编码用户类别、3 位编码一种操作，只能表达 “是哪一类、做哪一种”，不能同时记录四类用户各自的权限组合。题目未给权限之间必然蕴含关系，因此不能因 “完全控制” 这个名称擅自压缩其他位。

辨析·目录权限不等于内容权限

能遍历目录与能读文件内容属于不同操作。修改文件和修改目录项也不同，必须按被访问对象及具体动作验证。

4.4.2 口令、加密与权限表的具体差别

课件的口令模型为每个文件保存验证信息，用户提交正确口令后才可访问。其空间和比较开销较小，但口令的保管、共享和泄露会影响保护效果。实际系统可以保存校验结果而不是明文口令，具体格式由系统规定。

加密将可读明文转换成密文，解密需要正确的密钥。课件用异或说明可逆性：明文 `0010101100011101000 1`，重复使用密钥 `01001`，得到密文 `0110000101001111100 0`；再次用同一重复密钥异或恢复明文。若采用另一密钥 `01111`，则不能还原同一内容。这里展示的是异或的代数过程，不能据此把这一简单重复密钥方案等同于现代保密系统。

权限表控制某主体被允许执行的操作，口令验证身份或访问凭证，加密保护数据的可读性，三者可以配合。精简 ACL 将用户分组，记录各组的操作位，再在访问时确定用户所属组。目录的查找、列出与修改权限还影响能否到达下级名字；目录权限与文件内容权限应按各自操作检查，不能仅凭 “目录允许访问” 推断所有文件均可读写。

课件还用精简 ACL 演示分组授权：表中 1 表示允许该项操作，0 表示不允许；系统据用户所属组检查相应权限。

用户组	完全控制	执行	修改	读取	写入
系统管理员	1	1	1	1	1
文件主	0	1	1	1	1
文件主的伙伴	0	1	0	1	0
其他用户	0	0	0	0	0

例如，将用户加入 “文件主的伙伴” 组后，该用户在此课件模型中具有执行和读取权限。此表是演示用权限配置；实际系统中的组成员关系、权限继承与拒绝规则须按具体机制判断。

4.4.3 课件示例：Windows 文件夹访问控制

课件以文件夹 `E:\照片`和名为 “临时访客” 的本地账户演示访问控制。主要关系如下:

步骤	课件中的操作或现象
准备主体	创建 “临时访客” 本地账户，作为需要限制访问的用户
选择对象	打开 `E:\照片`的属性，在 “安全” 设置中查看该文件夹的用户、组与权限
建立规则	把 “临时访客” 加入权限设置，对选定权限在 “拒绝” 列勾选
系统提示	Windows 提示：若用户所属的两个组对同一权限分别允许和拒绝，拒绝项会优先于允许项
验证结果	切换到 “临时访客” 尝试打开该文件夹，课件示例显示当前用户无权访问

课程连接·两个索引属于不同层次

逻辑记录索引把关键字对应到记录或文件内偏移；物理索引把逻辑块号对应到磁盘块。数据结构中的查找复杂度可解释前一层，操作系统的间接索引层数解释后一层。遇到 “索引” 先写它的输入与输出对象。

这个例子把主体、对象和权限明确对应起来。显示的界面与具体用户组是课件运行环境的实例；实际有效权限还要按系统规则检查适用的显式条目、继承条目和用户所属组，不能仅凭一项允许就断定一定可访问。

4.5 逻辑结构：字节流和记录怎样被使用

无结构文件向应用提供字节序列，记录边界由应用自行解释；所谓无结构不是没有字节顺序。**有结构文件**由记录组成，可以定长或变长。定长记录且不跨块时，记录位置容易由长度计算；变长记录需要额外的边界、长度或索引信息。

顺序文件按记录次序存放，适合顺序处理；按关键字组织的顺序文件可利用有序性查找，但插入维护可能较贵。索引文件为记录或记录组维护索引，便于定位；索引顺序文件把顺序组织与索引结合。这些是应用看到的、或逻辑层规定的记录组织；它们与下面讲的物理块分配（连续、链接、索引）不是同一维度。

例如变长记录文件可以用一个逻辑索引定位记录所在字节范围，文件本身再通过物理索引块定位磁盘块。两个 “索引” 服务不同层次，题目问哪个对象的空间、哪个访问链，就沿对应关系计算。

4.5.1 顺序文件的串结构与顺序结构

顺序文件中的记录按逻辑次序排列。若记录间按存入的先后关系排列，称为**串结构**；若按关键字的顺序排列，称为**顺序结构**。这里的 “串” 不是把记录强制变为字符，也不规定其底层磁盘分配方式。

给定第 0 条记录的逻辑地址 F，定长记录长度为 L，若记录在逻辑文件中连续排列，则第 i 条记录起址为 $F + iL$，因此支持按记录号直接定位。若记录变长，必须先知道前面记录的长度或另设索引；即使是定长记录，若逻辑记录通过链接组织，也通常要沿链接查找。

“按记录号找第 i 条” 与 “按关键字找某个人” 是两个问题。定长、连续布局解决前者；再按关键字有序，才可利用有序性做折半等检索。串结构中的关键字一般只能顺序查找。增删时也要分别维护记录布局和关键字次序。

4.5.2 索引文件与索引顺序文件怎样折中

索引文件为记录建立定长索引项，如 “关键字、记录长度、记录逻辑地址”。索引项连续且定长，定位第 i 个索引项容易，再沿其地址找到可

变长记录。不同数据项可各建索引，例如学号索引与姓名索引；新增、删除或修改记录时，相关索引也要保持一致。

每记录一项会有空间代价：若记录平均 8 B、索引项 32 B，索引本身是原记录数据的 4 倍。**索引顺序文件**改为一组记录对应一个索引项，先定位组再在组内顺序查找；组与组之间要有能用于定位的关系，组内可按课件模型无需关键字有序。

辨析·两次映射先后不同

先由记录组织确定文件内逻辑位置，再由文件系统映射逻辑块到物理块；物理连续不会自动让链式记录支持按位序直接定位。

Figure 4.3: 逐记录索引与分组索引的目标不同，索引保存记录的逻辑地址

课件例有 10000 条记录，按 100 组、每组 100 条组织。顺序扫描组索引与组内记录，平均比较数近似为 $100/2 + 100/2 = 100$，相对直接扫描 10000 条的约 5000 次明显减少。若严格假定成功位置等概率、各层都用包含成功比较的线性扫描，精确期望是 $50.5 + 50.5 = 101$；“一半” 是大规模近似口径。

一般地，N 条记录分成 g 组，平均比较数近似为 $g/2 + N/(2g)$，在 $g \approx \sqrt{N}$ 时两项平衡。若为 10^6 条记录建立两级索引，按每级 100 项分组，则顶级 100 项、低级共 10000 项、数据 1000000 条，查找需经过顶级、低级、组内三个 100 项的顺序扫描，近似 $50 + 50 + 50 = 150$ 次，等概率精确值为 151.5 次。更一般的 k 级索引，等规模分组为 $N^{1/(k+1)}$ 项，近似比较数为 $(k+1)N^{1/(k+1)}/2$。

这些比较数属于给定的线性扫描模型。若索引层改用折半或树结构，应按新的检索方法计算，不能继续沿用各层一半的公式。

4.5.3 用户记录组织与系统块分配各解决哪一步

以每条记录 64 B、每磁盘块 1024 B 为例：用户按记录号 i 得到逻辑字节偏移 $64i$，所以逻辑块号为 $\lfloor i/16 \rfloor$，块内偏移为 $(i \bmod 16)64$。接下来才由文件系统将逻辑块号映射为物理块号。

用户自建的记录索引映射 “关键字 $\rightarrow$ 记录逻辑地址”；文件系统的块索引映射 “逻辑块号 $\rightarrow$ 物理块号”。因此索引文件可以采用连续、链接或索引分配，顺序文件也可以采用这些物理分配方式。某一层支持直接定位，不自动保证另一层也只有一次访问。

4.5.4 课件示例：记录组织与磁盘块组织

同一份顺序文件的记录可以连续排列，也可以由用户设计的链接组织。课件示意中，学生 0、1、2、3 分别位于记录存储槽 0、3、2、5，链接次序为 `0 → 3 → 2 → 5`；记录间的链接先解决 “下一个记录在哪”。在文件系统层面，文件被划分成逻辑块；这些逻辑块可另行采用连续分配或链接分配。因此，即使某种物理分配支持直接找到给定逻辑块，链式记录仍不一定能直接算出第 i 条记录的位置。

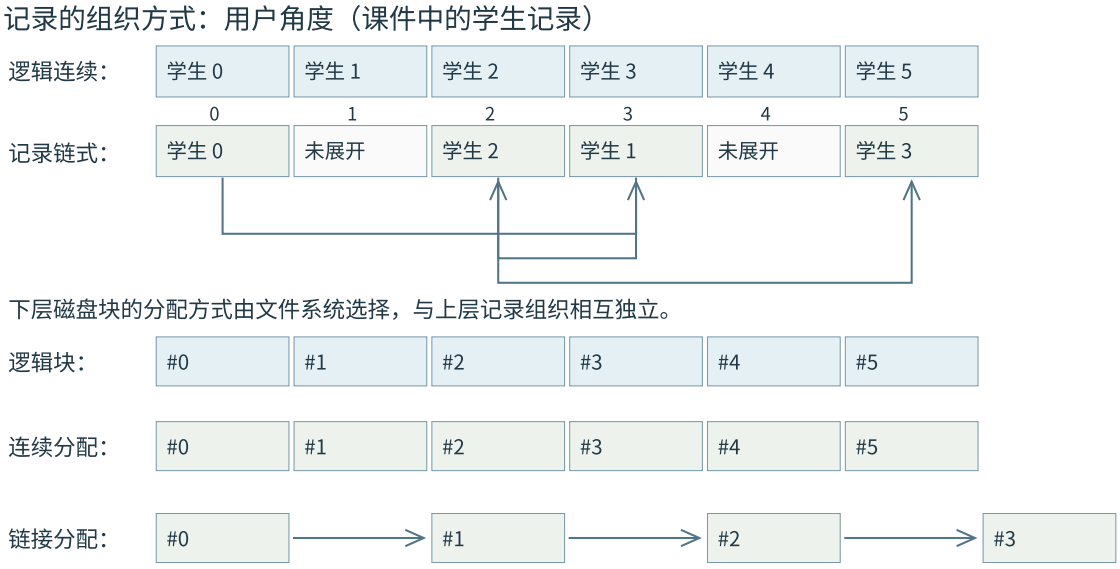


Figure 4.4: 学生记录的连续和链式组织，与文件逻辑块的连续和链接分配是两层独立关系

辨析·两次映射先后不同

先由记录组织确定文件内逻辑位置，再由文件系统映射逻辑块到物理块；物理连续不会自动让链式记录支持按位序直接定位。

课件以学生记录给出一个按记录号读取的实例，字段定义为 `number`、`name[30]`、`major[30]`；在课件平台的 `int` 为 4 B、结构体大小为 64 B 时，编号 5 的记录从逻辑偏移 $5 \times 64 = 320$ B 开始。以下保留课件的记录类型与 `fseek` ／ `fread` 接口，并检查返回值：

```c
#include <stdio.h>

typedef struct {
    int number;
    char name[30];
    char major[30];
} Student_info;

int main(void) {
    FILE *fp = fopen("students.info", "rb");
    if (fp == NULL) return 1;
    Student_info stu;
    if (fseek(fp, 5L * sizeof(Student_info), SEEK_SET) == 0 &&
        fread(&stu, sizeof(Student_info), 1, fp) == 1)
        printf("学生编号：%d\n", stu.number);
    fclose(fp);
    return 0;
}
```

这里假定文件按同一实现写入、至少有 6 条完整记录；`sizeof(Student_info)`依赖平台的整型宽度与结构体对齐，64 B 是课件平台的具体结果。课件演示使用`"r"` ／`"w"` 模式；读写原始结构体时使用`"rb"` ／`"wb"` 可以避免某些平台的文本模式转换。`fseek` 指定文件中的逻辑位置，随后系统按文件分配方式完成逻辑块到物理块的转换；读取是否实际触发磁盘 I/O，还受缓冲与缓存状态影响。

课件的无结构文件示例将`"Hello world!"` 重复写入 10000 次。若每字符一字节、内容没有额外换行，文件长 $12 \times 10000 = 120000$ B；从 0 起编号的第 16 个字符在每 12 字节循环中的位置为 $16 \bmod 12 = 4$，是

o。`fgetc` 读取一个字符时可经标准 I/O 缓冲，因而一次库函数调用不必然对应一次底层读系统调用。

错项排除·FAT 少读数据块仍需追链

FAT 驻留内存时，定位后面簇不必逐个读取前面数据块，但仍要沿内存中的链获取后继号。其优势是减少磁盘 I/O，不能直接推成按逻辑下标常数时间定位；表规模还由整个受管卷的簇数决定。

4.6 物理分配：连续、链接与索引

连续分配记录起始块与长度，逻辑块 k 直接映射为起始块加 k，顺序和随机访问都容易；但文件增长可能受相邻空间限制，空闲空间也可能有外部碎片。按区段扩展可缓解增长问题，但不是每个文件永远只有一个连续区间。

隐式链接分配在各数据块内保存后继块号，目录记录首块等信息。逻辑上连续的数据可散在任意物理块；增长容易、无需一个大连续洞，但要找后面的块通常必须沿链读取，链指针也占用空间。损坏一个指针会影响后续定位。

显式链接分配把后继关系集中到文件分配表 FAT。表若驻留内存，沿链查找不必每步读取数据块，但定位第 k 个簇仍要执行链追踪，不等于具有直接随机下标。FAT 一般为每个受管簇保存一项，规模取决于受管卷的簇数；不能仅凭某个文件的最大长度推断整个卷的 FAT 大小，除非题目另外限定了两者的关系。

索引分配为文件记录各逻辑块的物理块号，可单级、多级或混合索引。根据逻辑块号查相应索引项即可定位，支持随机访问；索引本身占空间，小文件使用大索引块时可能浪费。UNIX 式混合索引让小文件直接索引，大文件逐渐使用间接层级。

比较访盘次数时，先统一起点：逻辑块号 k 从 0 开始，目录项或 inode 已在内存，查找路径上的数据块和索引块均未缓存，也没有预存的中间块号；FAT 行另假定分配表已驻留内存。

分配方式	怎样定位第 k 个逻辑块	读取目标块需访盘	额外组织空间
连续	起始块号加 k，可直接定位	1 次	起始块号、长度等描述信息
隐式链接	从首块读取后继指针，逐块追踪	$k+1$ 次	数据块中的后继指针
显式链接 FAT	在内存表中追踪 k 条链接，再读目标块	1 次，但仍有内存追链开销	全卷每簇的表项；驻留表占内存
单级、多级或混合索引	按块号确定索引项，逐级取得下层块号	经 h 级间接索引时 $h+1$ 次；直接指针为 1 次	文件的索引项和索引块

分配方式	增长与碎片
连续	扩展受相邻空间制约；可能有外部碎片
隐式链接	可离散追加；无外部碎片
显式链接 FAT	可离散追加；受空闲簇及表规模约束
单级、多级或混合索引	可离散追加；范围受各级索引容量限制

FAT 支持不读取前面数据块而访问指定块，这正是它相对隐式链接的优势；这不表示由块号定位物理块只需常数次内存访问。“无外部碎片” 也不表示最后一个数据块一定装满，各方式仍可能留下块内未用空间。题目改变 inode、FAT 或索引块的缓存条件时，先删去已命中的那一级访问，再计总数。

4.6.1 例 4-3　混合索引中，一次访问要经过几层

不变量·容量与索引开销分开累计

一个索引块可装 B 除以指针字节数个地址。计算最大文件长度时只累加可指向的数据块，索引块本身另占外存。若题目转问总空间消耗，才把实际使用的各级索引块加回，两个问题不能共用一个总数。

例 4.6.1 (配套例题)

【配套例题】文件索引节点中有 10 个直接索引指针，一级和二级索引指针各 1 个。磁盘块为 1 KB，每个索引指针 4 B。某文件索引节点已在内存，把该文件偏移量为 1234 和 307400（按字节编址）处所在的磁盘块读入内存，需要访问的磁盘块个数分别是（　）。

A. 1、2　　B. 1、3　　C. 2、3　　D. 2、4

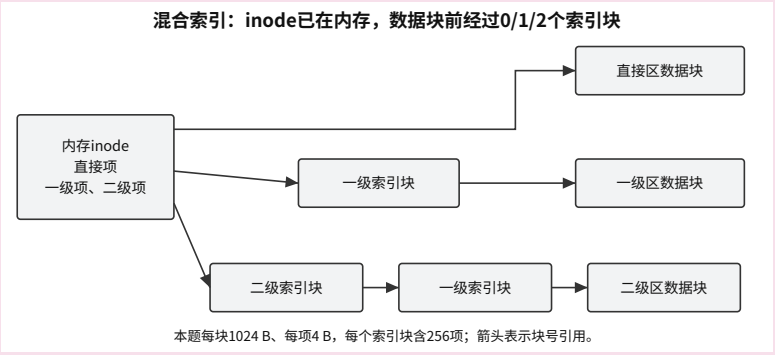


Figure 4.5: 直接、一级、二级路径经过的索引块数不同；图中只画少量代表项

解析。一个索引块能存 $1024/4 = 256$ 项。直接索引覆盖前 10 KB；一级间接再覆盖 256 KB；二级间接再覆盖 256^2 KB。

1234 小于 10×1024，属于直接区。inode 已在内存，可直接取目标块号，只读 1 个数据块。307400 大于 $(10 + 256) \times 1024 = 272384$，进入二级区；需读二级索引块、对应的一级索引块和数据块，共 3 次，答案为 **B**。

还可定位到项：$307400 = 300 \times 1024 + 200$，逻辑块号 300；减去前面的 $10 + 256 = 266$ 块，二级区内偏移 34 块，即先取二级索引的第 0 项，再取其所指索引块的第 34 项。块内偏移 200 不影响要经过的索引层数。

本题结构能表示的数据长度为 $(10 + 256 + 256^2) \times 1024$ B，即 64 MB + 266 KB；其中不把索引块计为文件用户数据长度。若索引块已经缓存，访盘次数会减少；若 inode 尚不在内存，还应计入取得它的相应访问。题目说 “索引节点已在内存”，不能再加一次 inode 读盘。

4.6.2 例 4-4　链接文件修改一条记录，要把读和写都计入

例 4.6.2 (配套例题)

〖4.2-025〗一个记录文件采用隐式链接分配，定长逻辑记录为 100 B，采用记录成组分解，盘块为 512 B。目录项已读入内存，对第 22 个逻辑记录完成修改，共启动磁盘多少次?

A. 3　　B. 4　　C. 5　　D. 6

解析。按原题记录不跨块、每块容纳 5 条记录的模型，第 1—5 条在第 1 块，依次类推，第 22 条位于第 5 块。隐式链要从第 1 块沿后继指针读到第 5 块，共读 5 次，再把修改后的第 5 块写回 1 次，总计 6 次，答案为 **D**。目录已在内存只免去目录读取，不能免去沿链的数据块读取。

盘块中的链接指针若占用部分字节，本题只要每块剩余空间仍足够容纳 5 条 100 B 记录，结论不变；若允许跨块记录、索引已缓存或采用 FAT，则必须重画访问路径。不要把“修改第 22 条”理解成写 22 次，也不要漏掉最后的回写。

不变量·缓存只删去对应访问

目录内容已在内存，只免去读取该目录内容；若目标 inode 和数据尚未缓存，仍要分别取到。写访盘路径时为每个节点标“已缓存/未缓存”，再计边上的实际读取，避免一个命中条件被扩张成整条路径全命中。

4.6.3 FAT 与索引表的具体块号轨迹

课件的 FAT 模型中，文件 aaa 从块 2 开始，后继关系为 $2 \to 5 \to 0 \to 1 \to -1$；文件 bbb 从块 4 开始，为 $4 \to 23 \to 3 \to -1$。-1 表示链尾。aaa 的第 0、1、2、3 号逻辑块分别映射到 2、5、0、1 号物理块。若 FAT 已驻留内存，定位第 3 号逻辑块需要查询内存中的三个链接，但只需为最终数据块发起一次读盘。

同一课件的索引分配例为：aaa 的索引块号 7，索引项依次保存 2、5、13、9，因而逻辑块 2 直接查第 2 项得到物理块 13。索引块 7 是元数据块，不是文件的第 0 号用户数据块。FAT 按受管卷组织表项，而块索引通常按文件分别组织。

4.6.4 索引块装不下时：链接、多层和混合

一个索引块不够时，可以将多个索引块**链接**：目录记录首个索引块，查后面的索引块还要读取前面的索引块来获得链接。它节省单块容量限制，但不再具备直接到任意索引块的能力。

多层索引让顶层索引指向下层索引块。每块 1024 B、地址 4 B，则扇出为 256；两层索引覆盖 256^2 个数据块，即 64 MiB。顶层尚未缓存时，读取用户数据要访问顶层、下一层和数据块，共 3 次。小文件仍会承担固定层级开销。

课件的**混合索引**含 8 个直接地址、一个一级间接地址和一个二级间接地址，覆盖

$$(8 + 256 + 256^2) \times 1\ \text{KiB} = 65800\ \text{KiB}.$$

在顶级控制信息尚未读入内存的题设下，逻辑块 0～7、8～263、264～65799 分别经过直接、一级间接和二级间接路径，总读盘数为 2、3、4；顶级信息已缓存则各减 1。该例的 8 个直接地址与前例 10 个直接地址是两套数据，计算时各自使用其前提。

4.7 目录和路径解析：逐层找到名字

单级目录把所有文件名放在一张表，简单但命名冲突多；两级目录按用户分开，缓解重名但共享和多层分类受限；树形目录让目录内再有子目录，便于按层次组织。绝对路径从根开始，相对路径从当前工作目录开始；`.` 表示当前目录，`..` 表示父目录，二者的解析都受权限与挂载规则约束。

访问 `/home/u/a.txt` 并不是在一张全局名字表里直接跳到数据。典型 inode 模型先从根目录查 `home`，取得其 inode，再查 `u`，最后在该目录中查 `a.txt` 并取得文件 inode。各层目录、inode、数据是否缓存，决定实际 I/O 数；路径分量的数量本身不是总访盘次数。

采用“名字与 inode 分离”可以使目录项更小，一块容纳更多名字，从而减少线性查找的目录块数。以王道书 §4.2 的目录模型为例，3200 项、每项 16 B、每块 1024 B，需要 50 个目录块。若目标等概率落在任意一项，且逐块线性查找，各块恰有相同数量目录项，则平均读取

$(1+50)/2 = 25.5$ 个目录块；随后再读 1 个 inode 块，总期望 26.5 次。用 $50/2+1 = 26$ 是“大量目录块”的近似，不能当作相同概率模型下的精确值。

若目录使用哈希或树索引，查找次数应按相应结构计算；若预先只读入目录的一部分，也不能把剩余查找都视为零成本。当前工作目录使常用路径可以省去重复解析共同前缀，但不取消权限检查。

辨析·共享不是复制

多个目录项指向共同文件，更新可通过不同名字看到。删除一个目录项只取消一条关联，回收还需检查链接与打开使用状态。

4.7.1 课件示例：绝对路径与当前目录

课件的根目录含 `qianlong`、`照片`和`对账单 4`；`qianlong` 与`照片`目录下分别有名为 `demo` 的文件，它们属于不同目录，可以同名。`照片`下的 `2015-08` 目录中有`自拍.jpg` 和`自拍 2.png`。

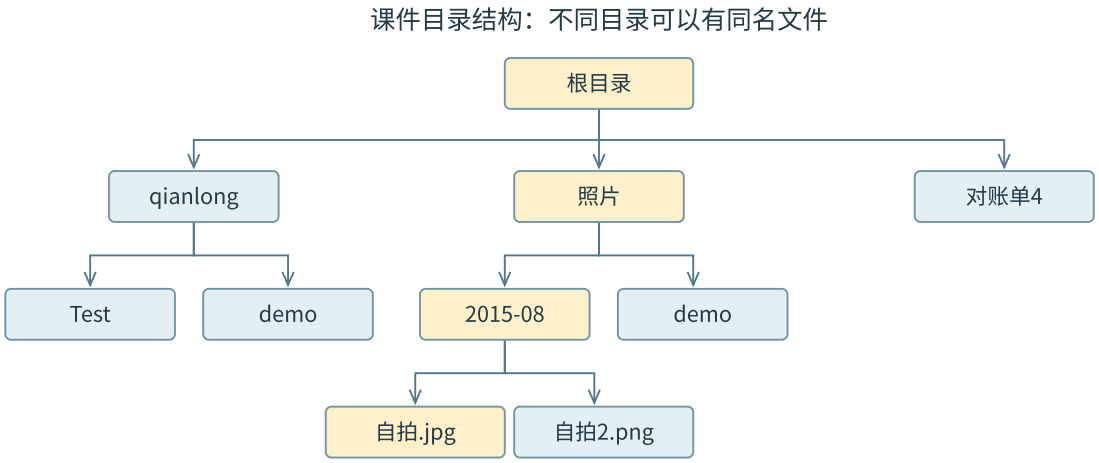


Figure 4.6: 根目录、qianlong 与照片目录、2015-08 目录以及文件的父子关系

`自拍.jpg` 的绝对路径为`/照片/2015-08/自拍.jpg`。在课件给定的简化模型里，若根目录、照片目录和 2015-08 目录的内容初始均不在内存，且每读取一张目录表恰需一次磁盘 I/O，则从根开始依次读入三张目录表，需要 3 次目录读盘；这里统计到找到目录项为止，文件内容或其他未缓存元数据的读取须依题设另计。

若已打开“照片”目录且其目录表驻留内存，把它设为当前目录后，可用相对路径`./2015-08/自拍.jpg`。查找 `2015-08` 时无需重新读取根目录和照片目录，在同一简化模型下只需再读入 2015-08 目录，故为 1 次目录读盘。当前目录和缓存是否存在是这个数值成立的前提。

4.7.2 无环图目录与目录项瘦身的收益

树形目录中的每个非根节点只有一个父目录。无环图目录允许多个父目录引用同一文件，或在受控模型中共享子目录，让用户使用不同名字访问共同对象。为避免形成难以回收的目录环，实际系统通常限制任意硬链接目录；允许什么链接由文件系统政策决定。

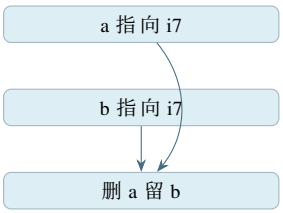
共享计数统计指向该对象的目录关联。删除一个名字只取消一条边；仍有其他名字时保留共同内容。共享与复制也不同：共享者修改共同对象，其他名字随后仍访问该对象；复制得到的文件则拥有各自数据和元数据。

将大 FCB 中的文件名与其余属性分离，可提高每块目录的容纳量。课件数据为 640 个目录项、原 FCB64 B、块长 1024 B：原目录每块 16 项，共 40 块；改成文件名 14 B 加索引节点指针 2 B 后，每块 64 项，共 10 块。按成功位置均匀的逐块查找，精确平均目录读盘数分别为 $(40+1)/2 = 20.5$ 与 $(10+1)/2 = 5.5$；课件用 “一半” 近似为 20 与 5。找到名字以后，再按题给缓存状态计入读取 inode 的成本。

这种改进让目录查询更紧凑，不是消除了文件属性，也不是所有文件属性都应直接放在一个小目录项里。

最小反例·删一个名字不等于删文件

同一文件 inode 为 i7，目录项 a 和 b 都指向它。删除 a 后仍有 b 这一硬链接，文件内容继续可达。把 a 叫作原文件不赋予它特殊地位；硬链接之间对称，不能说删除 “原名字” 就必然使另一个硬链接失效。



两个目录项对应同一 inode；删除名字 a 后，名字 b 仍可引用文件。

4.8 硬链接、软链接与什么时候回收文件

硬链接是另一个目录项直接指向同一文件 inode；多个名字地位相同，不存在 “原名字一删，另一个就失效” 的必然关系。增加一个硬链接通常增加 inode 的硬链接计数，删除一个名字减少计数；只有计数归零，并且不再有有效使用引用等其他条件，才能回收数据和 inode。硬链接通常不能跨文件系统，因为 inode 编号必须在本文件系统解释；通常也限制用户为目录建立硬链接，以避免目录环等管理问题。

软链接或符号链接是一个特殊文件，内容保存目标路径。访问软链接时，系统再解析这个路径；目标被删或移走后，软链接本身仍可存在，只是成为悬空链接。它可以跨文件系统，也可指向目录；相对目标路径的解释与链接所在位置有关，不能一概按调用者当前工作目录处理。

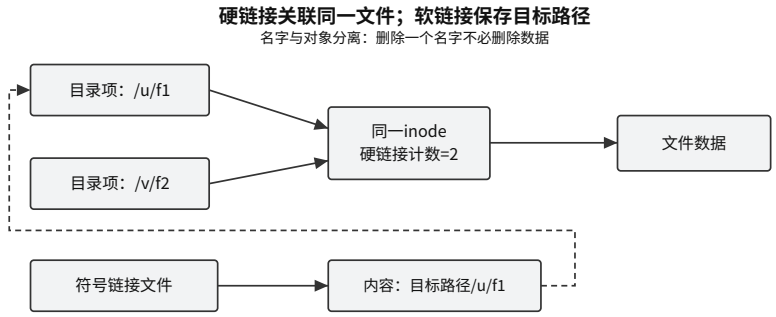


Figure 4.7: 两个硬链接指向同一 inode；软链接保存路径，经过名称解析再找到目标

4.8.1 课件示例：硬链接计数与符号链接的路径

课件中，User1 的目录项 `aaa` 和 User2 的目录项 `bbb` 都指向 inode 1，硬链接计数为 2；User3 的目录项 `ccc` 指向另一个 inode 2，其文件内容为目标路径 `C:/User1/aaa`。访问 `ccc` 时，系统先读取这个符号链接保存的路径，再按该路径查找目标。

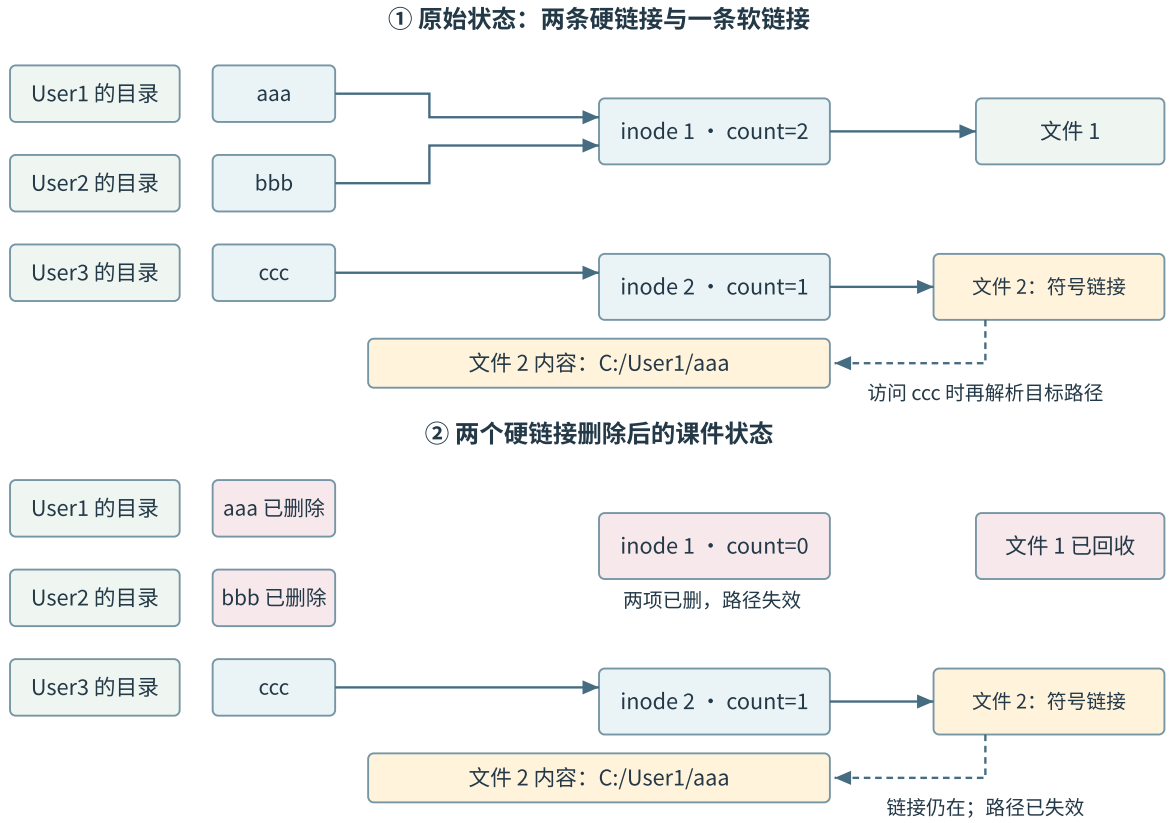


Figure 4.8: 课件中的两条硬链接、符号链接及删除后路径失效状态

若两个硬链接对应目录项均被删除，inode 1 的硬链接计数为 0。在课件假定不存在其他有效打开引用的情形下，文件 1 可以回收。符号链接文件 2 与 ccc 目录项仍存在，但其保存的路径已找不到 aaa，因此是悬空链接；若文件仍有打开引用，实际回收还须等待相应引用释放。只删除 aaa 时，即使 bbb 仍能访问文件，ccc 保存的特定路径也已失效。

辨析·符号链接认路径

即使另一硬链接 bbb 仍可访问文件，删除 aaa 也会使保存 C:/User1/aaa 的 ccc 成为悬空链接。

4.8.2 例 4-5　同一 inode 不要求同一读写位置

例 4.8.1 (配套例题)

【配套例题】若文件 f1 的硬链接为 f2，两个进程分别打开 f1、f2，获得文件描述符 fd1、fd2，则正确的是（　）。

I. f1 和 f2 的读写指针位置保持相同

II. f1 和 f2 共享同一个内存索引节点

III. fd1 和 fd2 分别指向各自的用户打开文件表中的一项

A. 仅 III　　B. 仅 II、III　　C. 仅 I、II　　D. I、II、III

解析。f1、f2 是同一文件的两个名字，两次打开最终引用同一个 inode，II 正确。两进程分别打开，读写位置可以独立，I 错误。每个文件描述符在相应进程的描述符表中解释，分别关联各自的打开表项，III 正确。答案为 **B**。这与共享同一个打开实例的继承或复制描述符情形不同。

4.9 文件系统分层：从用户请求到块设备

辨析·按职责识别不同分层

名称查找、权限检查、记录地址、物理块地址和设备命令分别处理。教材与课件可合并或拆分层名，职责关系才是判断依据。

文件系统把一个名称层的请求分解为查目录、验证权限、定位记录、转换块地址和操作设备等职责。课件采用下表的分层方法；教材还可把相近职责合并为逻辑文件系统、文件组织模块、基本文件系统和 I/O 控制，层名有所不同，但应能将每项职责放到正确位置。

层次／模块	主要职责
用户接口	接收 open、read、write、close 等系统调用请求
文件目录系统	按路径查找 FCB 或 inode，管理目录和打开信息
存取控制模块	检查主体是否拥有相应访问权限
逻辑文件系统与文件信息缓冲区	将记录号等逻辑使用位置转换为文件逻辑地址
物理文件系统	将文件逻辑地址转换为外存物理位置
辅助分配模块	分配和回收文件存储空间
设备管理模块	I/O 缓冲、设备调度、启动及完成处理等设备服务

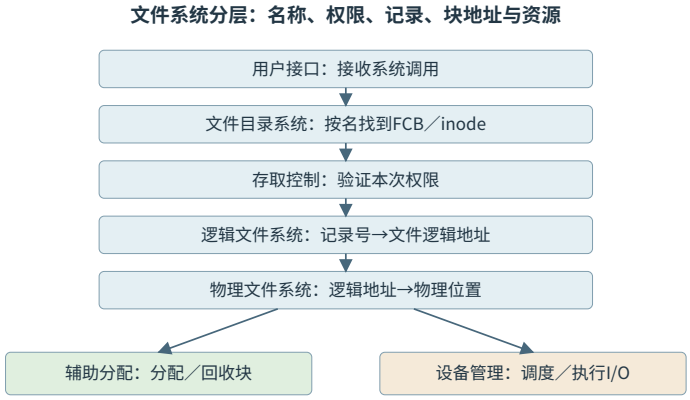


Figure 4.9: 删除文件末尾记录时，各层完成名称、权限、地址和资源更新

例如删除`/工作目录/学生信息.xlsx`末尾 100 条记录：接口接收请求，目录系统找到文件，存取控制验证权限，逻辑层确定被删除记录的逻辑范围，物理层定位相应块，设备管理执行必要更新，辅助分配回收不再需要的空间。若末块仍含保留记录，则只调整文件长度等信息，不能把整块中有效数据一并释放。

4.10 文件系统的磁盘结构与内存结构

磁盘上的引导控制信息用于启动；卷控制块或超级块记录块大小、卷规模、空闲管理等总体信息；目录和 inode 区描述对象与名字；数据区保存内容。具体文件系统可采用其他布局，题目若给不同结构，应以其字段为准。超级块并不保存每个文件的索引信息。

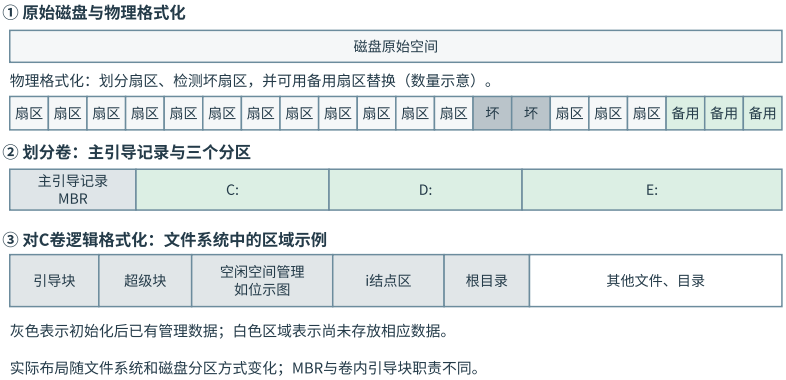


Figure 4.10: 物理格式化划分扇区并处理坏扇区，分区后对 C 卷逻辑格式化得到卷内管理区

在课件的 MBR 示例中，磁盘包含主引导记录和 C、D、E 三个分区。对 C 卷逻辑格式化后，卷内依次示出引导块、超级块、空闲空间管理信息、i 结点区、根目录和其他文件／目录区域；图示灰色部分已有初始化数据。MBR 属于整个磁盘的引导与分区管理信息，卷内引导块属于该卷的引导信息；二者所在位置和职责不同。上述排列是该教学模型，不代表所有文件系统都按相同顺序放置。

挂载后，内存中维护已挂载文件系统信息、目录缓存、inode 缓存、系统打开对象以及进程描述符表等。缓存加快访问，但修改与写回必须遵守规定的次序和时机。一次文件扩展可能同时修改数据块、空闲位图、inode 大小和块地址；若只写了一部分就断电，会出现泄漏或结构不一致。

一致性检查可以扫描目录、inode 与块使用关系，找出不一致；日志机制先记录需要完成的更新，再按规则提交和重放，减少崩溃后恢复工作。日志不表示任意用户数据都自动获得完整事务语义，其保证取决于记录元数据还是数据、提交和写回的具体规则。这里只解释多结构更新之间的因果，不展开某种文件系统的实现细节。

4.11 空闲空间：位图、空闲链和成组链接

空闲表记录连续空闲区的起点与长度，适合按区段管理；**空闲链**把空闲块或空闲区串起来；**位图**为每个受管块保留一位，值表示空闲或占用，具体 0、1 意义由题目规定；**成组链接**把若干空闲块号存成一组，用组内的连接信息指向后续组，并在内存保留当前组的块号栈。

位图大小主要由受管总块数决定，不随当前空闲块数改变；空闲表、空闲链所需内容则与当前空闲分布相关。成组链接靠少量的组间连接把大量空闲块号组织起来，并不是把所有文件的数据链混在一起。

4.11.1 空闲表、盘块链和盘区链怎样分配与回收

空闲表的每项为 “起始空闲块号、连续块数”。申请连续的 K 块时，可以用首次适应、最佳适应或最坏适应选区；分配后缩小或删除表项。回收时依地址检查前邻、后邻、两侧相邻或两侧均不相邻四种情况，分别扩展、三段合并或新建表项，与内存动态分区的物理邻接判断相同。

课件示例中，空闲区依地址排列，首次适应为新文件分配 3 块：起始于 0 的 2 块和起始于 5 的 1 块都不足，选中 10～14 并分配 10～12，

错项排除·位图长度取决于管理范围

管理一百万个块，即使当前只剩一个空闲块，位图仍须为每个块保留一位。空闲块数量影响位值分布，不缩短受管对象集合。位图内相对位置算完后再加其起始磁盘块号，不能先从受管块号减去起点。

留下 13～14。此时另有空闲区 18～20；回收 15～17 后，按物理地址合并 13～14、15～17、18～20，得到起始块 13、长度 8 的空闲区。

空闲**盘块链**把每个空闲块链接起来，内核保存链头等信息，申请 K 块就依次摘取 K 个节点，回收块再挂回链。它适合离散分配，但申请多个块可能要多次追链。空闲**盘区链**每节点代表一段连续空闲区，首块保存长度和后继盘区信息，一次可分出多个块；回收时仍要检查相邻盘区合并。

位图适用于连续或离散分配。按课件的 0 基模型，字长 n 位、字号 i、位号 j 对应盘块号

$$b = ni + j, \qquad i = \lfloor b/n \rfloor, \qquad j = b \bmod n.$$

$n = 16$ 时，$(i, j) = (1, 10)$ 对应块 26；块 31 对应 $(1, 15)$。若题目按教材另一模型让块号、字号、位号都从 1 开始，则应为

$$b = n(i-1) + j, \quad i = \lfloor (b-1)/n \rfloor + 1, \quad j = (b-1) \bmod n + 1.$$

两套定义必须整套使用。分配时找空闲位并改为占用，回收时做相反更新；0 代表空闲还是占用先读题设。

辨析·承载块交付前先读目录

当前组最后的空闲块还保存下一组信息。必须先把组目录装入内存，再把该块用于文件数据，才能保住后续空闲块入口。

4.11.2 成组链接：组满、组空时怎样转移

在课件每组最多 100 个块号的模型中，内存超级块缓存当前组；该组最后取出的某个空闲块同时存放下一组目录。组内块号不要求连续。

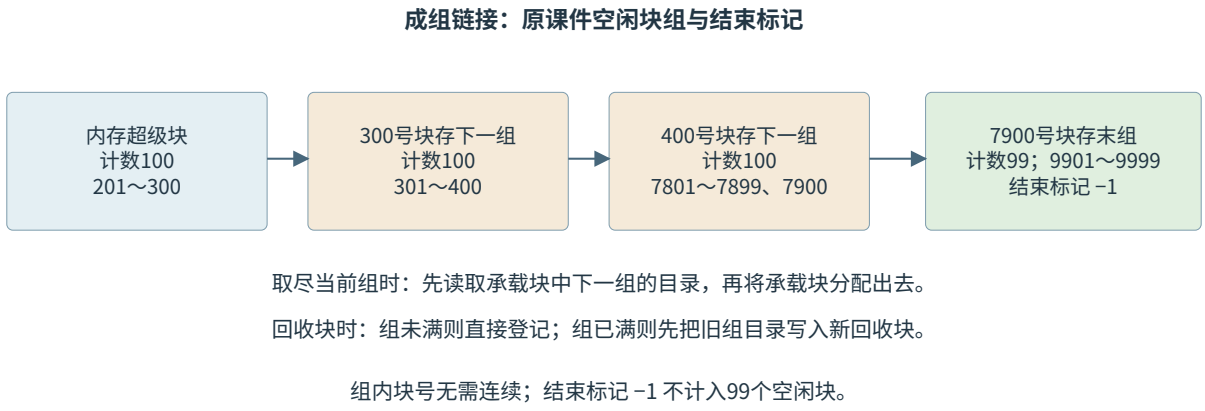


Figure 4.11: 当前组用尽时先从承载块装入下一组，再把承载块交付使用

假设当前组登记 201～300 共 100 个空闲块，300 号块内登记下一组 301～400。申请 1 块时可取出 201 并将计数由 100 改为 99；申请至当前组用尽时，先把 300 号块中保存的下一组信息读入超级块，再把 300 也交给申请者。若先覆盖 300 的内容，下一组入口就会丢失。课件继续给出：400 号块记录的下一组含 7801～7899 和 7900 共 100 个块；7900 号块记录末组 9901～9999 共 99 个块，连接位置的特殊值 -1 表示没有后续组，不计入 99 个空闲块。

回收时，当前组未满则将块号压入并增加计数；当前组已满又回收一个新块 x，先把当前 100 项组信息写到 x 中，再让超级块仅保存 x 作为新组入口。以后分配到 x 时，就能恢复被保存的那一组。末组以专门的结束标记表示无后续组，不能把标记当成可分配块号。

超级块的内存状态及磁盘状态要按规定同步；位图、空闲表和组目录都属于分配元数据，与文件自己的数据块索引分别管理。

课程连接·统一接口可以隐藏多种布局

VFS 把查找、打开、读写等操作抽象成共同接口，再由具体文件系统实现。它和面向接口编程相似：调用约定统一，并不要求内部数据结构一致。不能因为应用都调用 read，就推断各磁盘格式具有相同的 inode 字段。

4.11.3 例 4-6　先定位位图块，再定位块内字节

例 4.11.1 (配套例题)

【配套例题】文件系统用位图表示磁盘空间分配，位图存于磁盘 32—127 号块，每块 1024 B，盘块和块内字节都从 0 编号。释放盘块号 409612 时，要修改的位所在盘块号和块内字节序号分别为（　）。

A. 81、1　　B. 81、2　　C. 82、1　　D. 82、2

解析。每个盘块的位图可表示 $1024 \times 8 = 8192$ 个数据块。$409612 = 50 \times 8192 + 12$，故目标位在位图第 50 块，即磁盘块 $32 + 50 = 82$；余数即块内第 12 位，它位于字节 $\lfloor 12/8 \rfloor = 1$，在该字节中的位序号为 4。答案为 **C**。

位图自身的起始磁盘块号 32，只在“位图内相对块号”算出以后加；不能先把受管盘块号减去 32。修改成 0 还是 1，要看位图的空闲值约定，本题只问位置，不必自行确定。

4.12 虚拟文件系统与挂载

虚拟文件系统 VFS 为不同具体文件系统提供共同的操作抽象，把打开、查找、读写等请求通过相应操作接口分派给具体实现。统一接口不表示磁盘格式也统一，更不要求所有设备保存相同的 inode 布局。

课件先用未统一接口的教学模型说明差异：P1 要分别使用示意的 `open(a,b)`、`fopen(x)` 和 `openf(p,q)` 访问 UFS、NTFS 和 FAT，不同接口参数与具体实现相互绑定。这里的函数名用于表达接口差异，不是对这些文件系统现实 API 的逐一规定。

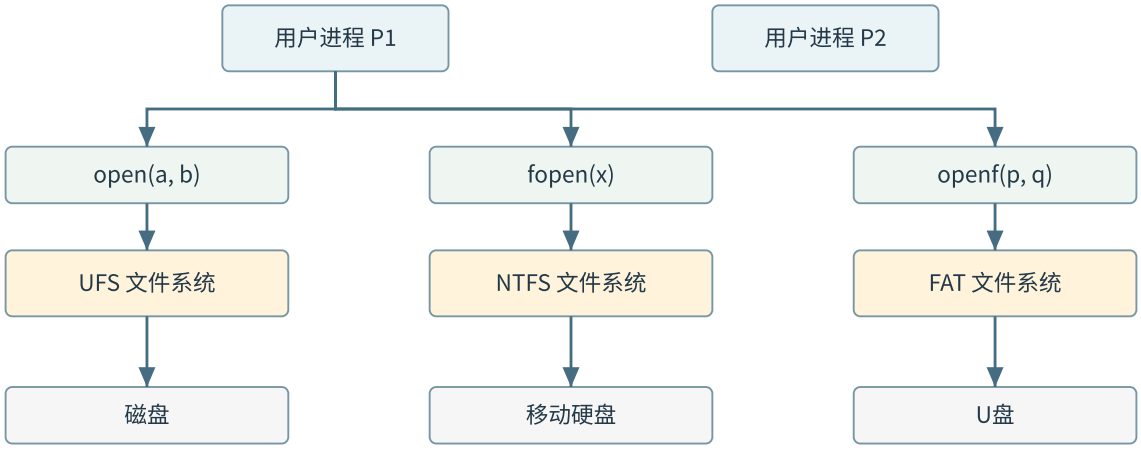


Figure 4.12: 课件示意：没有统一接口时，P1 分别调用三种文件系统的不同接口

Linux 式抽象常区分超级块对象、inode 对象、目录项对象 dentry 和文件对象：其中超级块对象描述一个已挂载文件系统，inode 描述文件

本体，dentry 缓存名字与父目录等关系，file 对象描述一次打开。dentry 与 inode 不是同一个英文术语；目录项对象的名称不能误写成 inode 对象。

挂载把某文件系统的根关联到现有名称空间的一个挂载点，使路径解析到该处时转入新文件系统。挂载点之下原有内容不因此被删除，只是在相应名称解析视图下被新挂载内容遮住。卸载要处理尚在使用的对象和待写回信息，不能只删一个路径字符串就认为设备已经可以安全移除。

课程连接·统一接口可以隐藏多种布局

VFS 把查找、打开、读写等操作抽象成共同接口，再由具体文件系统实现。它和面向接口编程相似：调用约定统一，并不要求内部数据结构一致。不能因为应用都调用 read，就推断各磁盘格式具有相同的 inode 字段。

4.12.1 VFS 的统一对象与具体操作入口

课件用 **vnode** 表示打开文件时在内存中使用的统一虚拟节点，它保存通用文件信息，并通过操作函数指针转入具体文件系统的 open、read、write 等实现。vnode 是内存抽象，磁盘上的 inode 格式则由具体文件系统决定；不能要求所有磁盘都直接存放同一种 vnode。

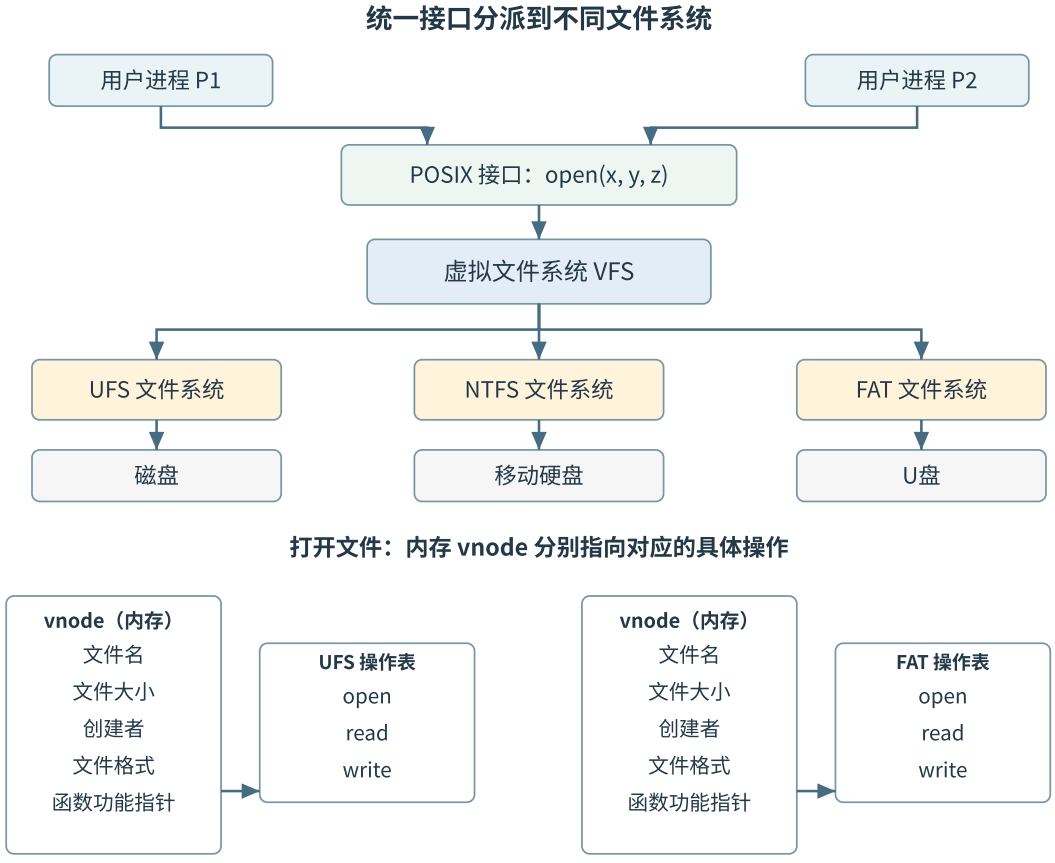


Figure 4.13: 课件中的 POSIX 接口、VFS 与 UFS、NTFS、FAT 分派，以及不同 vnode 操作表

例如图中用户进程通过统一接口请求操作，VFS 再根据目标文件所属的具体文件系统转交给 UFS、NTFS 或 FAT。课件示意的 vnode 记录文件名、大小、创建者、文件格式等通用信息，函数功能指针指向相应具体实现的 `open`、`read`、`write` 等操作。图中的磁盘、移动硬盘和 U 盘只表示三个具体文件系统实例，介质与文件系统的对应关系由实际格式化和挂载情况决定。

挂载时要在 VFS 登记文件系统信息，提供所需操作入口，并把其

根关联到一个挂载点。随后同样的路径和调用接口可以转入不同底层实现。课件的 vnode 模型与前述 Linux 式 inode/dentry/file 等细分对象，是说明统一抽象的不同组织方法，不能用一张对象表替代另一模型的全部关系。

不变量·文件数量由最紧资源约束

每个图像既需要一个 inode，也需要足够数据簇。应分别计算 inode 区能容纳的个数和数据区能容纳的个数，再取较小者；若还要间接索引簇或目录空间，应按题设另计，不能只用剩余字节除文件长度。

4.13 综合例题：目录、索引与容量共同约束

4.13.1 例 4-7　同一索引节点怎样连接目录查找与文件扩展

例 4.13.1 (配套例题)

【配套例题】某文件系统的磁盘块大小为 4 KB，目录项由文件名和索引节点号构成，每个索引节点占 256 字节，其中包含直接地址项 10 个，一级、二级和三级间接地址项各 1 个，每个地址项占 4 字节。该文件系统中子目录 stu 的结构如下图所示，stu 包含子目录 course 和文件 doc，course 子目录包含文件 course1 和 course2。各文件的文件名、索引节点号、占用磁盘块的块号如图中表所示。请回答下列问题。

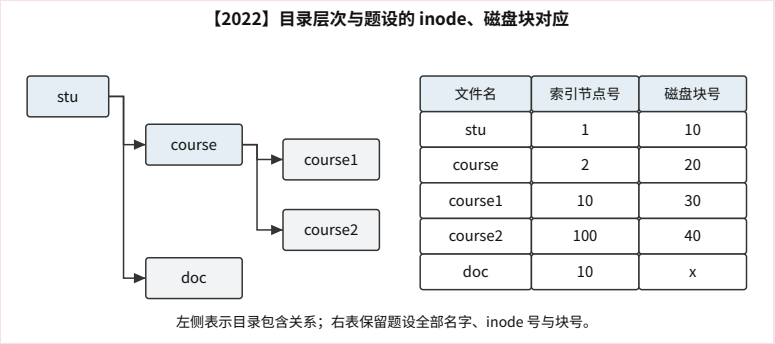


Figure 4.14: stu、course 的目录层次，以及各文件的索引节点号和数据块号

（1）目录文件 stu 中每个目录项的内容是什么?

（2）文件 doc 占用的磁盘块的块号 x 的值是多少?

（3）若目录文件 course 的内容已在内存，则打开文件 course1 并将其读入内存，需要读几个磁盘块？说明理由。

（4）若文件 course2 的大小增长到 6 MB，则为了存取 course2 需要使用该文件索引节点的哪几级间接地址项？说明理由。

解析。(1) stu 的直接子项只有 course 和 doc。目录项存名字和 inode 号，不存其数据块号，故内容分别为 `(course,2)`、`(doc,10)`；course1、course2 的目录项在 course 中，不在 stu 中。图未列出 `.`、`..`，本问按图列出两个给定子项。

（2）doc 和 course1 的索引节点号同为 10，且属于同一文件系统，说明它们是同一个文件对象的两个名字。course1 的数据在 30 号磁盘块，因此 $x = 30$。不是因为名字相似而共享，也不是把 inode 号 10 直接当作数据块号。

（3）按本问给出的缓存起点，course 的目录项可在内存查到，先取得 course1 的 inode 号 10。随后读入该 inode 所在的磁盘块，从 inode 得到数据块号 30，再读 30 号数据块，共 **2 个磁盘块**。打开所需的内存表项由系统建立，不另算作磁盘块；256 B 的 inode 放在磁盘块中，读取

它仍按块计。题目只说目录内容已在内存，不能顺带认定文件 inode 和数据也已经缓存；若另给这些缓存命中条件，才相应减少次数。

（4）本题按 1 MB = 1024 KB 计算，6 MB 文件需要

$$\frac{6 \times 1024\ \text{KB}}{4\ \text{KB}} = 1536\ \text{个数据块}.$$

每个间接索引块容纳 $4096/4 = 1024$ 个地址项。10 个直接项与一级间接项合计覆盖 $10 + 1024 = 1034$ 个数据块，仍少 $1536 - 1034 = 502$ 块；二级间接项可再覆盖 1024^2 个数据块。因此需用**一级和二级间接地址项**，不必用三级。这里的 1536 是用户数据块数，索引块另占空间；按先用直接区、再用各间接区的组织，一级区需要 1 个索引块，二级区需要 1 个二级索引块及其指向的 1 个一级索引块。

不变量·文件数量由最紧资源约束

每个图像既需要一个 inode，也需要足够数据簇。应分别计算 inode 区能容纳的个数和数据区能容纳的个数，再取较小者；若还要间接索引簇或目录空间，应按题设另计，不能只用剩余字节除文件长度。

4.13.2 例 4-8　先算地址覆盖，再比较 inode 与数据簇的上限

例 4.13.2 (配套例题)

【配套例题】某文件系统采用索引节点存放文件的属性和地址信息，簇大小为 4 KB。每个文件索引节点占 64 B，有 11 个地址项，其中直接地址项 8 个，一级、二级和三级间接地址项各 1 个，每个地址项长度为 4 B。请回答下列问题。

（1）该文件系统能支持的最大文件长度是多少？（给出计算表达式即可。）

（2）文件系统用 1M（$1\text{M} = 2^{20}$）个簇存放文件索引节点，用 512M 个簇存放文件数据。若一个图像文件的大小为 5600 B，则该文件系统最多能存放多少个这样的图像文件？

（3）若文件 F1 的大小为 6 KB，文件 F2 的大小为 40 KB，则该文件系统获取 F1 和 F2 最后一个簇的簇号需要的时间是否相同？为什么？

解析。（1）一个索引簇可存 $4096/4 = 1024$ 个簇号。直接、一级、二级、三级区分别覆盖 8、1024、1024^2、1024^3 个数据簇，所以最大文件长度为

$$L_{\max} = (8 + 1024 + 1024^2 + 1024^3) \times 4096\ \text{B}.$$

按二进制容量单位，这等于 4 TiB + 4 GiB + 4 MiB + 32 KiB。它只统计文件数据，不把索引簇自身算入用户文件长度。

（2）先独立计算两个容量限制。每个 inode 簇放 $4096/64 = 64$ 个 inode，故 inode 区能容纳 $2^{20} \times 64 = 2^{26} = 64\text{M}$ 个文件。5600 B 大于一个簇而小于两个簇，每个图像占 $\lceil 5600/4096 \rceil = 2$ 个数据簇，数据区可容纳 $512\text{M}/2 = 256\text{M}$ 个。两个数据簇都在 8 个直接地址项覆盖范围内，不额外占间接索引簇。因此最多存放

$$\min(64\text{M}, 256\text{M}) = 64\text{M} = 67\,108\,864\ \text{个图像文件}.$$

这里 M 表示个数的 2^{20} 倍；数据簇尚有剩余，也不能突破 inode 的数量上限。计算采用题给的两个可用区域，不另假设目录占用或其他保留量。

（3）按本题比较的索引访问层级，时间不同。F1 占 $\lceil 6/4 \rceil = 2$ 个簇，末簇是逻辑簇 1，其簇号就在 inode 的直接项中。F2 占 $40/4 = 10$ 个簇，

末簇是逻辑簇 9；前 8 个逻辑簇 0—7 用直接项，逻辑簇 8—9 进入一级间接区，所以还须沿一级间接地址读索引簇，再取其第 1 项。

在 inode 同样已在内存、相关间接索引未缓存的共同起点下，F1 的末簇号可直接从内存取得，F2 多一次索引簇读盘。若连末簇数据也要读入，则路径分别是“数据簇”与“一级索引簇 → 数据簇”，为 1 次、2 次访盘。原问只求簇号，不能把目标数据簇的读取强行加进原问；若另有索引缓存命中，实际耗时还会改变。

课程连接·检查顺序

区分目录查找、inode 读取、索引块查询和数据块访问，再按题目缓存条件计数。

4.14 分支自测：沿名称与数据两条线检查

以下为原创回顾设问。

1. 关闭最后一个文件描述符，是否要求硬链接数减 1?
2. 例 4-3 中，偏移恰好为 10240 B，属于直接区还是一级间接区?
3. 某文件的硬链接数从 2 减为 1，文件内容是否回收？软链接是否增加目标的硬链接计数?
4. 管理 2^{20} 个块需要多少位图空间？若目前只有 1 个空闲块，位图是否只需 1 位?
5. 同一 inode 能否由两个打开实例保存不同的当前偏移?

4.14.1 自测答案

1. 不要求。打开引用数统计运行中的打开，硬链接数统计目录名字的关联。
2. 前 10 块覆盖偏移 0—10239，10240 是一级间接区第一个字节。
3. 仍有一个硬链接，不回收内容。软链接只保存路径，不因此增加目标硬链接数。
4. $2^{20}/8 = 2^{17}$ B，即 128 KB。位图仍为每个受管块保留一位。
5. 可以，独立打开与 inode 共享是两个不同关系。

4.15 主动回忆自测

以下为自拟题，先画对象与路径，再回答。

从名字一直追到数据与回收

1. 两个名字指向同一 inode，是否要求两个进程的当前读写位置相同?
2. close、unlink、回收 inode 与数据块分别改变哪种关系?
3. 同一文件的硬链接删去一个，另一名字是否失效？软链接为何可能悬空?
4. 4 类用户各有 5 个独立权限，用位串至少需要多少位?
5. 块长 1KiB、指针 4B、10 个直接项及一二级间接项，各区分别覆盖多少数据块?
6. inode 已在内存，相关索引与数据未缓存，读二级间接区数据块需几次访盘?
7. 位图管理 2^{20} 个盘块，需多少字节？当前只有一个空闲块会缩短位图吗?
8. 挂载新文件系统会删除挂载点下原有内容吗？VFS 是否要求各文件系统磁盘格式相同?

4.15.1 核对答案与关键条件

文件层次与容量核对

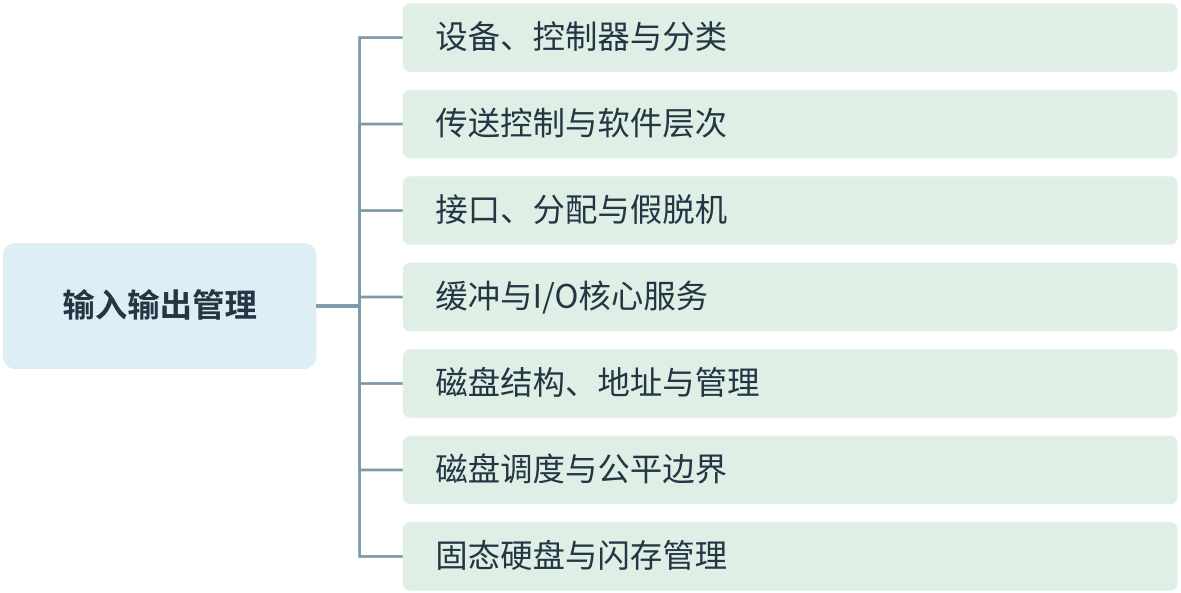
1. 不要求。独立 open 通常各有打开实例与位置；dup 或继承共享打开实例时才可能共享位置。inode 表示文件身份。
2. close 释放打开引用；unlink 取消目录名字并减少硬链接计数；最终回收还须无硬链接和有效使用引用等条件，不能混为一次操作。
3. 硬链接仍直接指向同一对象，保留名字可继续访问；软链接保存路径，目标路径失效后链接文件仍可存在。
4. 每类保存 5 个独立是非许可，合计 $4 \times 5 = 20$ 位；不是“类别编号加单个操作编号”。
5. 每个索引块 256 项，直接区 10 块，一级 256 块，二级 256^2 块；用户容量乘块长，索引块另计。
6. 二级索引块、一级索引块、数据块，共 3 次；若某一级已缓存，按实际路径减去对应读取。
7. $2^{20}/8 = 2^{17}$B，即 128KiB；管理范围未变，位图长度不变。
8. 原内容通常只是被当前名称解析视图遮住，不因此删除；VFS 统一操作接口，具体磁盘布局可以不同。

课程连接·回忆层次

从路径到目录项，再到索引节点和数据块；链接数与打开位置分别维护。

输入输出管理 5

章节脉络



课程连接·设备到进程

I/O 路径贯穿用户接口、设备无关层、驱动、控制器和中断处理；不同角色承担不同任务。

5.1 一次 I/O 的职责分工与时间线

先定位责任，再计算过程

1. 画请求向下和完成向上的路径，标明谁设置控制器、谁搬数据、谁唤醒等待者。
2. 对缓冲题逐块写输入、复制、分析，只有不争用同一资源且不覆盖有效数据的阶段才能重叠。
3. 对磁盘题先换单位、算真实柱面，再按当前磁头与方向逐步决定下一请求；端点和停止规则单独列出。
4. 对 SSD 题分别画页编程、块擦除、有效页迁移与逻辑映射更新，不套机械寻道模型。

定义 5.1.1 (控制方式、接口语义和软件层次分别判断)

轮询、中断、DMA 回答如何控制传送；阻塞、非阻塞、异步回答调用者何时等待和怎样取得完成；用户库、设备无关层、驱动与中断处理回答软件职责。三组分类可以组合，不能相互替代。

5.1.1 双缓冲总时间为何包含启动与排空

两阶段流水的递推

在正文规定的资源模型中，把设备输入一块看成耗时 T 的阶段 A，把 CPU 复制与分析看成耗时 $M+C$ 的阶段 B；A 与 B 可对不同块重叠。第一块先输入，末块仍须完成 B。稳定推进的最小间隔受较慢阶段限

制，为 $\max(T, M+C)$，故 $n \geq 1$ 块的总时长为 $T + (n-1)\max(T, M+C) + M + C$。取 $n = 1$ 得到 $T + M + C$，恰好检验没有流水可重叠时的边界；如果输入、复制、分析不能遵守这些资源假设，须重新建立递推。□

错项排除·字符单位不等于字节随机寻址

终端按字符流收发，并不意味着可按任意字节地址跳到某个字符；磁盘作为块设备可按块号定位。数据传送单位、是否可寻址、是否独占属于不同分类维度，不能把 “字符” 与 “内存字节地址” 机械对应。

应用说 “读一个文件块”，设备却接收具体的命令、地址和长度。操作系统在中间完成命名、保护、排队、缓冲和状态通知。本章沿一个请求追踪软件与硬件的分工，再把缓冲时间和磁盘运动转成可计算的过程。

5.2 设备、控制器、接口与端口

设备完成实际输入、输出或存储，**设备控制器**管理设备动作并向主机提供寄存器等接口。控制寄存器接收命令与参数，状态寄存器反映就绪、忙、错误等状态，数据寄存器或缓冲区承载数据。**端口**在此通常指可访问的接口寄存器位置，不是网络中传输层端口号。

独立编址的 I/O 端口由专门 I/O 指令访问；内存映射 I/O 把接口寄存器放入地址空间，通过相应访存指令访问，但这类访问的副作用和缓存属性必须按设备规则处理。接口地址是控制器寄存器的地址，不是设备所保存文件的字节偏移。

字符设备通常按字符流组织，不适合按任意字节地址定位，例如终端；**块设备**按可寻址数据块组织，可进行块号访问，例如磁盘。网络设备按分组收发，还涉及通信地址和协议，不宜机械套成磁盘的寻道块模型。

独占设备在一次使用期间交给一个使用者；共享设备允许多个作业在一段时期内交替提交请求。共享不要求多个物理操作在同一瞬间执行。设备是否可寻址、是否共享、是否可随机访问是不同分类维度，也不能因此把字符设备说成 “可按字节地址随机访问”。

5.2.1 分类维度与控制器的三个接口部分

按使用特性，设备可分为人机交互、外存储和网络通信；按传输速率可分低、中、高速；按信息交换单位可分字符与块设备；按共享性质又可分独占、共享和虚拟设备。这些是交叉分类，不能用一个维度替代另一个。

控制器由 CPU—控制器接口、I/O 控制逻辑、控制器—设备接口等部分组成。CPU 侧以地址线选寄存器、以控制线表达操作、以数据线交换命令参数或数据；I/O 逻辑译码命令、管理状态，设备侧传送设备所需的数据、状态和控制信号。一个控制器可以连接多个设备，因而可以有多组寄存器和设备接口。

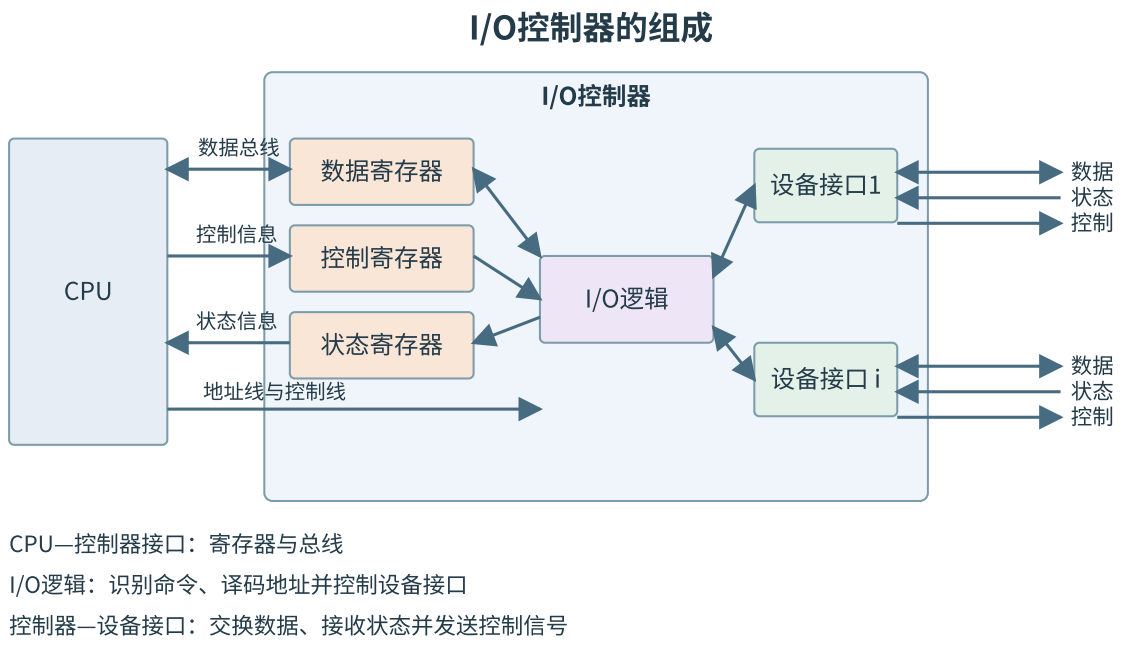


Figure 5.1: I/O 控制器的 CPU 侧寄存器、I/O 逻辑与多设备接口

状态位的 0、1 意义由设备定义：一种打印机可以 0 表示空闲，另一种则 1 表示空闲。驱动正是要把这些具体差异屏蔽在统一接口之下。寄存器独立编址需要相应 I/O 指令或 I/O 访问机制；内存映像方式复用地址空间和相应访存操作，但不把设备寄存器变成普通可随意缓存的数据。

课程连接·DMA 与中断可以协作

组成原理说明 DMA 如何取得存储访问机会并搬运数据，操作系统说明驱动如何设置地址长度以及怎样处理完成。DMA 结束常用中断通知，二者分别承担传送与通知，不能因为 DMA 无需逐字搬运就排除中断或内核参与。

5.3 轮询、中断和 DMA：谁搬数据，谁等待

程序查询或轮询由 CPU 反复读状态，设备就绪时再执行数据传送。实现简单，但忙等期间 CPU 不能有效做别的工作。**中断方式**启动设备后允许 CPU 执行其他任务，设备在需要服务或完成时发中断，CPU 进入中断服务程序处理；若每次数据通过 CPU 的读写指令搬运，数据通路仍经过 CPU。

DMA 由专门硬件按驱动设置的地址、长度和方向，在设备与内存之间传送数据。CPU 负责启动、异常和结束处理，不必逐字执行搬运指令；DMA 需要获得总线或内存访问机会，仍可能与 CPU 竞争存储带宽。DMA 结束通常通过中断通知，所以 “DMA 与中断不能共存” 不成立。

控制方式	数据搬运的直接执行者	CPU 可以从什么工作中解脱
轮询	CPU 执行数据传送指令	通常不能免除状态忙等
中断	CPU 执行中断服务中的传送指令	不必连续轮询设备
DMA	DMA 控制硬件	不必逐字/逐块执行软件搬运，但仍需初始化与完成处理

读取数据时，程序直接控制先发命令，再反复读取状态寄存器；设备就绪后，CPU 把数据读入寄存器并写入内存。中断驱动在设备准备期间让 CPU 执行其他任务，收到中断后检查状态并由 CPU 完成数据搬运。以下示意采用逐字传送的教材模型，具体设备的中断粒度可不同。

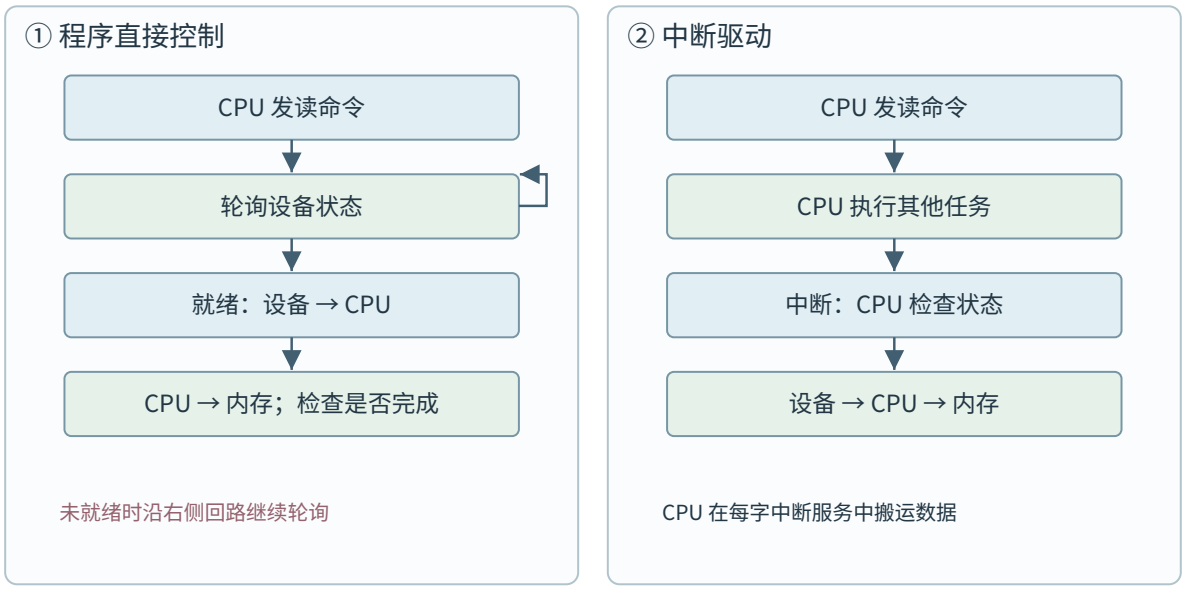


Figure 5.2: 程序直接控制与中断驱动的读入过程

DMA 由 CPU 设置方向、主存地址和计数后启动，传送期间由 DMA 控制器组织设备与主存的数据交换；通道则依照主存中的通道程序组织一组 I/O 操作。两者在完成或出现需处理的异常时通知 CPU，具体中断次数由设备和控制协议决定。

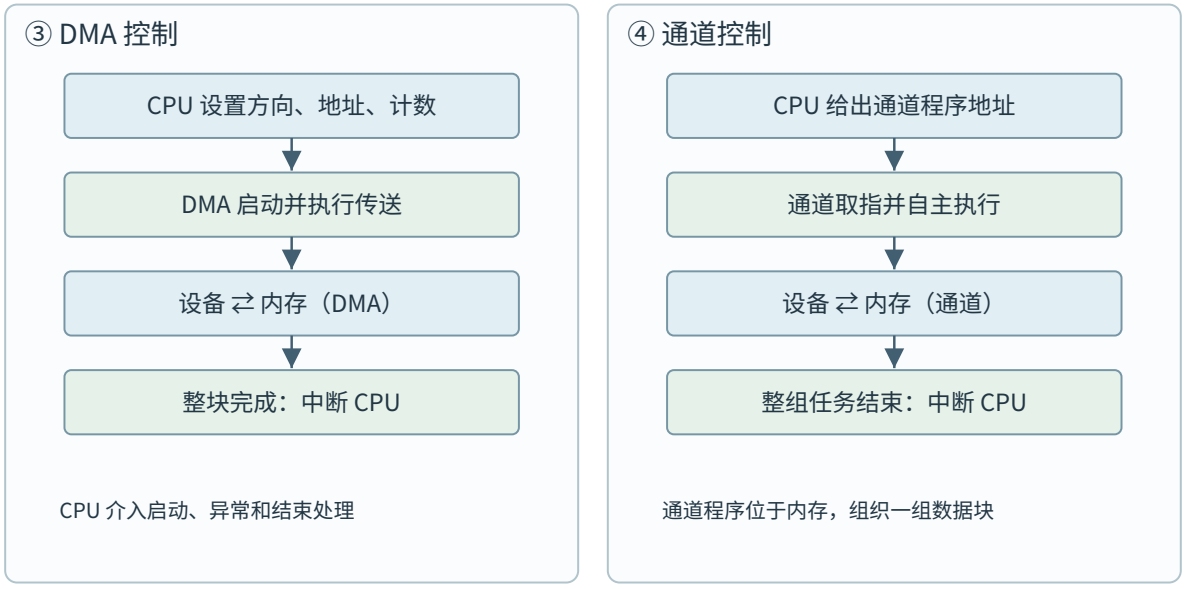


Figure 5.3: DMA 与通道方式的启动、传送和完成通知

请求 I/O 的线程是否阻塞，是接口语义问题；设备采用哪种传送控制，是硬件和驱动问题。一个阻塞读在中断或 DMA 工作期间可以睡眠；一个非阻塞请求也可立刻返回。不能仅凭 “DMA” 这一点就推断用户进程必然处于阻塞态。

DMA 地址还要按系统规则提供：不能让设备直接按用户给出的地址任意写入。实际系统可能使用固定页、内核缓冲和 IOMMU 等机制；

课程连接·设备到进程

I/O 路径贯穿用户接口、设备无关层、驱动、控制器和中断处理；不同角色承担不同任务。

本书题目只要求按其给定的物理地址与控制模型推演，不把 “DMA 无需 CPU 每字搬运” 误写成 “完全不需要操作系统参与”。

辨析·谁搬数据与谁等待分开

DMA 让硬件执行内存与设备间搬运，调用线程是否等待由接口语义决定。结束仍可中断 CPU，二者能够组合。

5.3.1 DMA 的寄存器与通道程序

课件 DMA 控制器包括数据寄存器 DR、内存地址寄存器 MAR、数据计数器 DC、命令／状态寄存器 CR。输入时 MAR 指向目标内存位置，输出时指向源位置；每完成规定的传送，地址和剩余计数按硬件规则更新，到结束条件后报告完成。

下图将 DMA 控制器的四类寄存器放在同一传送路径中。DR 暂存设备与主存间的数据；MAR 和 DC 共同限定当前传送位置与剩余数量；CR 保存命令或状态。具体寄存器宽度及单位以设备为准。

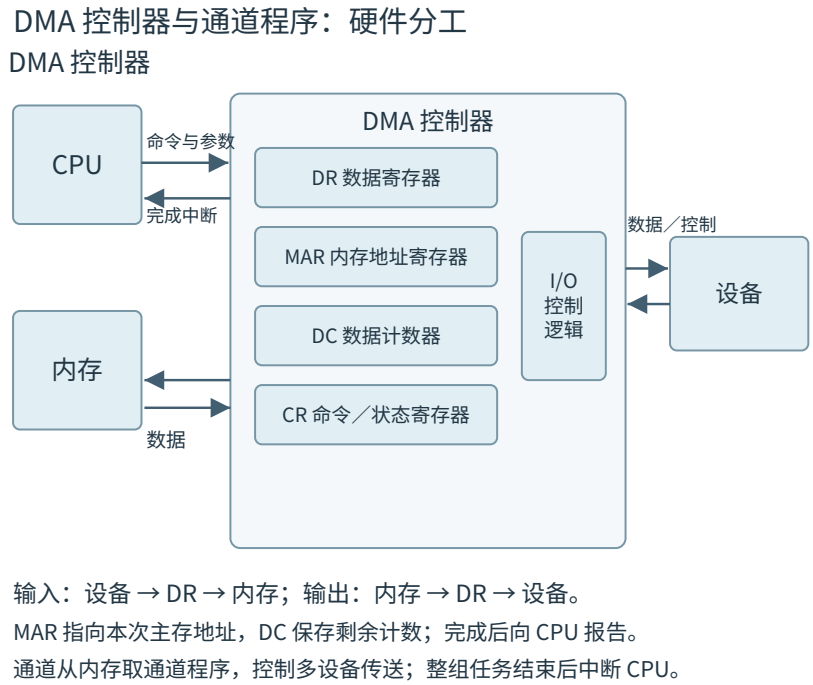


Figure 5.4: DMA 控制器的 DR、MAR、DC 和 CR 及数据路径

教材的基本 DMA 模型每次描述一个或多个连续设备块与连续内存区；若数据离散，CPU 需发起多次操作。支持散布／聚集描述符的扩展硬件可以放宽这一限制，题目采用基本模型时应按其给定地址、长度和连续性计算。

通道是能执行通道指令的 I/O 控制硬件。CPU 提供通道程序在主存中的地址及操作对象，通道再取出任务清单中的各条指令，组织一组数据块的传送，完成后中断 CPU。通道程序使用主机内存，通道指令服务于 I/O 控制，其能力和通用 CPU 不同。

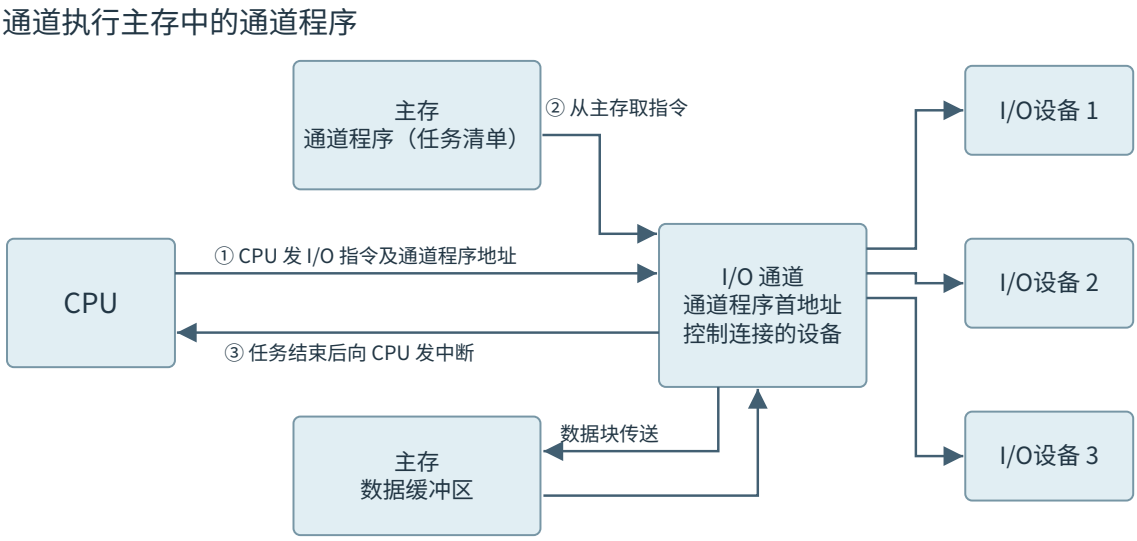


Figure 5.5: CPU 启动通道，通道从主存取得程序并管理设备传送

控制方式	教材模型的传送单位	CPU 主要介入时机	输入数据路径
程序直接控制	字	发命令、持续轮询、每字搬运	设备 →CPU 寄存器 → 内存
中断驱动	字	启动与每次中断服务	设备 →CPU 寄存器 → 内存
DMA	一块或若干连续块	设置参数、完成或异常处理	设备 → 内存
通道	一组块、由程序组织	给出通道程序、整组完成或异常处理	设备 → 内存

减少 CPU 介入并不免除内核的权限检查、缓冲管理和资源协调。控制器向 CPU 发出完成中断后，是否恢复某个用户线程，仍要通过唤醒及调度路径决定。

错项排除·驱动知道设备细节

文件逻辑位置属于上层抽象，控制器寄存器、传统 CHS 参数和启动命令属于设备细节。驱动在提交时形成具体请求，中断处理在完成或异常时推进状态。若题设改为设备内部转换 LBA，应按新接口重新分配职责。

5.4 I/O 软件层次与一次请求的完整路径

用户层 I/O 库组织格式和用户缓冲；系统调用进入内核。设备无关软件处理共同的命名、权限、缓冲、分配和错误接口；设备驱动程序把抽象请求变成控制器能执行的命令，设置寄存器并启动设备；中断处理在完成或异常通知后记录状态、推进请求进度并唤醒等待者。

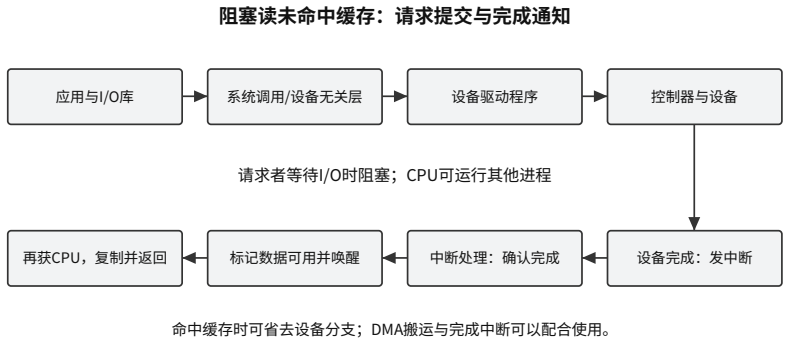


Figure 5.6: 提交路径向下，完成通知向上；唤醒以后仍由调度器决定何时运行

对一个缓存未命中的阻塞式磁盘读，完整过程可以是：应用发出 read，内核查找文件块并申请缓冲，驱动把请求排队、设置 DMA 并启动；调用者等待数据而阻塞，CPU 运行其他进程；设备完成后发中断，处理程序确认状态并标记缓冲可用，唤醒等待者；调用者再次运行时完成复制和返回。若数据早已缓存，设备分支可能整个省去。

5.4.1 例 5-1　哪里需要知道具体设备参数

例 5.4.1 (配套例题)

【配套例题】用户程序发出磁盘 I/O 请求后，系统处理流程为：用户程序 → 系统调用处理程序 → 设备驱动程序 → 中断处理程序。其中，计算数据所在磁盘柱面号、磁头号和扇区号的程序是（　）。

A. 用户程序　　B. 系统调用处理程序　　C. 设备驱动程序　　D. 中断处理程序

解析。柱面、磁头和扇区是具体磁盘寻址参数，应由屏蔽设备差异的驱动处理，答案为 **C**。中断处理主要响应已发生的完成或异常事件，并不负责在收到初始请求时重新规划文件逻辑位置。现代磁盘常接受逻辑块地址 LBA，内部几何转换可由设备完成；这不改变本题按传统 CHS 模型划分驱动职责的答案。

5.4.2 I/O 核心子系统的职责怎样分层

I/O 核心子系统主要包含设备独立性软件、设备驱动和中断处理。共同的命名映射、保护、请求调度、分配回收、缓冲与公共差错处理通常归设备独立性软件；具体寄存器、命令格式和状态编码由驱动处理；中断入口处理完成或异常通知。

I/O 调度为待处理请求选择顺序，例如磁盘选择寻道顺序，打印服务可以按到达、优先级或任务长度排队。设备保护按用户身份、设备对象及访问权限检查；设备视为特殊文件时，可复用文件访问控制机制。差错处理则需区分可重试错误与永久故障，达到规定重试次数后向上报告。

SPOOLing 服务在课件模型中由用户层的假脱机进程组织，并请求内核磁盘 I/O 服务；它使用核心子系统的缓冲、文件和驱动功能。因此“某功能与 I/O 有关”不表示其所有代码都必须放在同一内核层。

不同系统的驱动可在内核或受控服务进程中执行。能否独立成为进程是实现政策，判断题应以题给架构为准，而不要把“驱动”直接等

辨析·共性管理与硬件细节

命名、保护、缓冲与分配是通用职责；寄存器地址和状态编码依设备而变。中断处理确认已发生事件，驱动设置具体操作。

同于 “永远独立调度的进程”。

最小反例·不阻塞仍可能忙等

程序不断调用非阻塞 read，得到暂不可用后立即再次调用，虽然每次调用都不睡眠，CPU 却可能一直空转。接口不阻塞只描述本次调用的等待行为，不能直接推出自动提高利用率；异步完成通知另有接口语义。

5.5 阻塞、非阻塞与异步接口

阻塞 I/O 在操作暂不能完成时，使调用线程等待；**非阻塞 I/O** 在操作不能立即完成时返回相应状态，让程序决定重试、处理其他事务或等待通知。反复调用非阻塞接口也可能形成忙等，接口不阻塞不表示自动提高利用率。

异步 I/O 把提交与完成分开，提交后程序继续执行，以后通过通知、完成队列等取得结果。非阻塞主要回答 “本次调用等待不等待”，异步主要回答 “操作完成如何与调用者联系”，不能把二者作为完全相同的术语。具体接口是否可能立即完成、如何报告部分完成，要按题设。

字符设备常有字符流、行缓冲等语义，块设备有块号和对齐约束，网络接口涉及连接与分组的收发。设备无关性是给上层适当统一的抽象，不是强求所有设备具有完全一样的操作集合。

5.5.1 字符、块与网络设备接口

字符设备接口可提供 get ／ put 等字符读写；块设备接口可提供 read ／ write 及 seek 定位，参数需表达设备或文件、当前偏移、用户缓冲地址和长度。实际一次 read 可以返回多个字符，操作名本身不决定底层传输单位。

操作系统也为设备驱动规定统一的调用接口。课件以 `read(p1, p2,p3)`示意上层调用约定：不同厂商各自实现对应驱动，上层依接口传参，驱动再处理具体设备差异。设备独立软件和中断处理程序仍各有职责；接口名称、参数和设备能力须以实际系统为准。

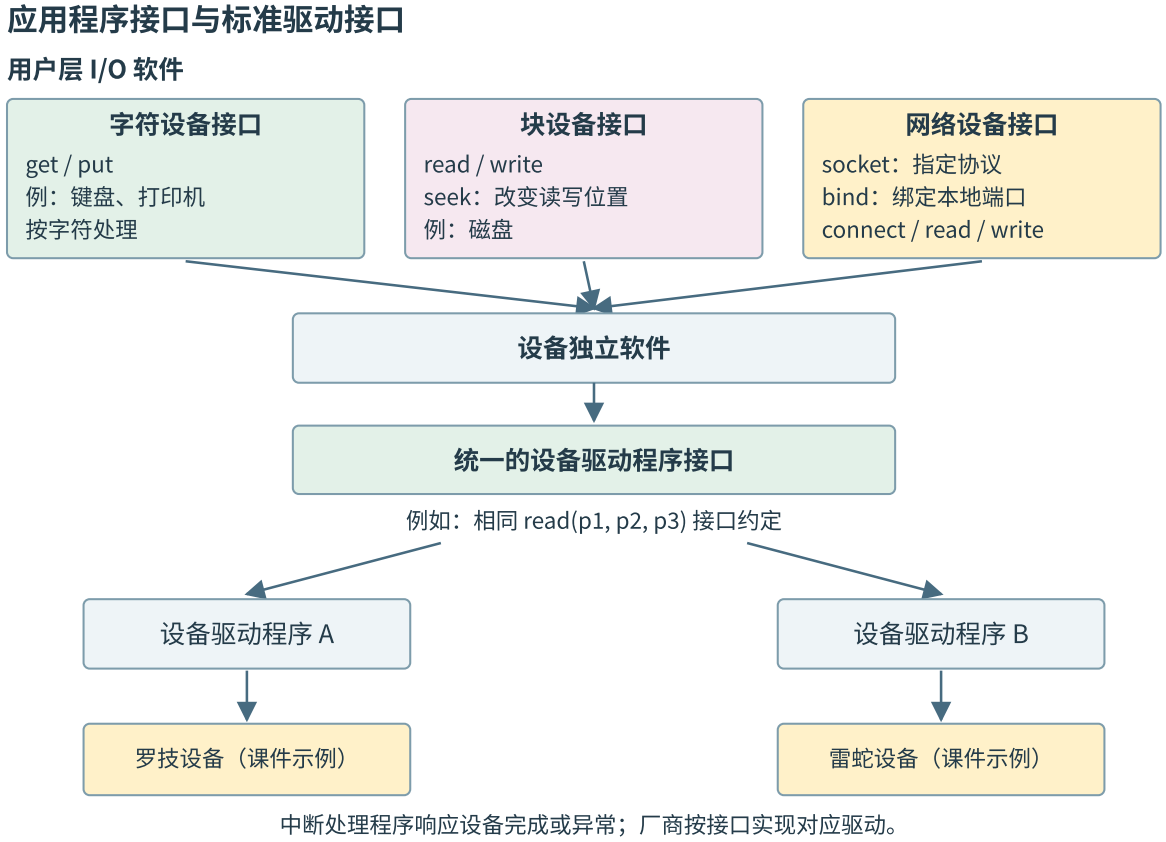


Figure 5.7: 字符、块和网络应用接口及统一驱动接口的分层关系

网络设备还须说明通信对端和协议。课件用网络套接字接口说明：`socket` 建立套接字，`bind` 关联本地地址和端口，`connect` 建立与远端的关联，再通过规定的读写操作交换数据。数据经协议栈、设备无关软件及网卡驱动下传；接收时网卡报告事件，驱动和协议处理将数据交给相应套接字，满足条件后唤醒等待者。

辨析·同名 read 不表示同一设备语义

字符、块、网络接口可能共用操作名，但寻址、连接、部分完成和等待规则不同，应按各自参数与返回条件解释。

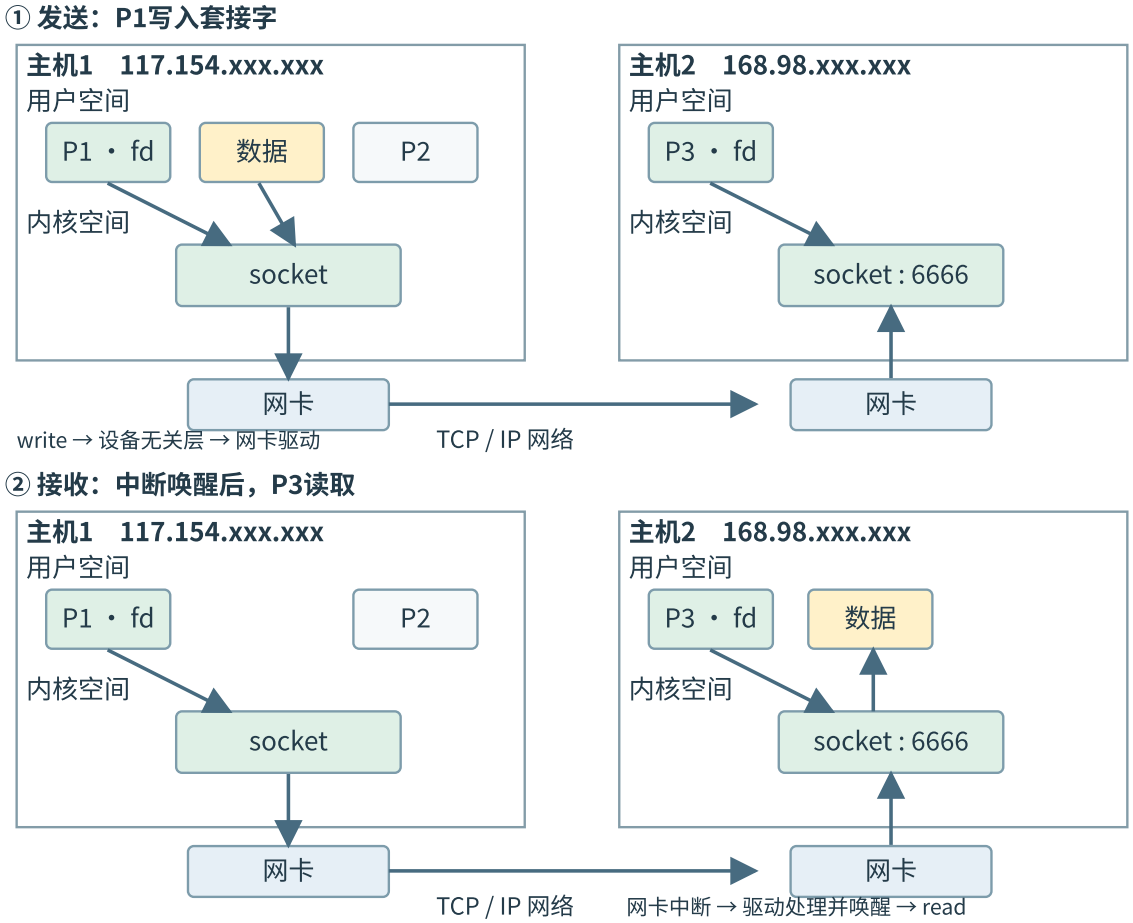


Figure 5.8: 两台主机的套接字写入与读取过程：P1 发出数据，P3 在中断处理后通过 fd 读取

图中主机 1 的 P1、P2 和主机 2 的 P3 分别拥有自己的用户空间；P1 通过文件描述符写入套接字，数据经网卡和 TCP/IP 网络送往主机 2 绑定端口 6666 的套接字。网卡中断后，驱动和协议层处理数据，再唤醒等待该数据的 P3；P3 实际运行后经自己的描述符读取。进程 P2 没有参与这次传送。

字符、块和网络设备可以共享某些操作名，但操作集合及参数语义并不完全相同。阻塞与非阻塞还可作为接口选项：磁盘写返回快可能只表示已经交给缓存，并不意味着任何块设备写都必定是非阻塞；有数据可直接取得的阻塞式读也可能立即完成。

不变量·缓冲复用不能覆盖未用数据

设备只有在对应系统缓冲已可复用时才能填下一块，复制也不能覆盖仍被分析的用户工作区。按这两个约束检查每段重叠，再求 max 形式的周期；若复制不用 CPU 或存在多个工作区，原公式的资源模型已改变。

5.6 缓冲：把输入、复制与计算放到同一时间线

缓冲在生产者和使用者之间暂存数据，可以缓和速度差、适配传送单位、允许阶段重叠。若设备及控制方式允许积累一批数据后再通知 CPU，缓冲还可减少中断频率，并为响应留出时间；缓冲是否改变中断次数取决于具体机制。高速磁盘缓存还可利用重复访问、预读和延迟写，减少某些实际 I/O。缓冲区不改变设备本身的物理速度，也不会无条件消除所有等待。

设每块从设备输入系统缓冲耗时 T，从系统缓冲复制到用户工作区耗时 M，CPU 分析该块耗时 C。假定输入可与 CPU 工作重叠，复制与分析共用相应 CPU 资源，不能互相重叠，只有一个用户工作区，并忽略其他开销。

单缓冲中，设备不能在缓冲尚未复制完时覆盖它；复制下一块也不能覆盖仍在分析的用户工作区。因此第一块先经 $T+M$，此后相邻两块复制完成点之间的间隔是 $\max(T,C)+M$，最后还要完成最后一块的分析。对 $n \geq 1$ 块：

$$T_{\text{single}} = T + M + (n-1)[\max(T,C) + M] + C.$$

双缓冲允许设备填另一个系统缓冲，而 CPU 使用前一个。流水可看成设备输入 T 与 CPU 的复制加分析 $M+C$ 两个阶段，在上述资源模型下：

$$T_{\text{double}} = T + (n-1)\max(T, M+C) + M + C.$$

这些是从阶段占用关系推出的公式。题目改成两个用户工作区、复制不用 CPU、半双工链路或其他限制时，先重画时间线，不能只凭“单缓冲/双缓冲”的名称直接套用公式。

最小反例·一块数据没有流水收益

只有一个块时，输入 T、复制 M、分析 C 仍须完成，单缓冲与双缓冲同为 T + M + C。双缓冲的收益来自后续块输入与前一块 CPU 工作重叠，不能把“增加一个缓冲区”直接解释成任何工作量都耗时减半。

5.6.1 例 5-2　首块要启动，末块要排空

例 5.6.1 (配套例题)

【配套例题】某文件占 10 个磁盘块，逐块读入主存缓冲，再送用户区分析。一个缓冲区与一个磁盘块大小相同，读一块到缓冲区为 100 μs，复制到用户区为 50 μs，CPU 分析一块为 50 μs。在单缓冲、双缓冲结构下，读入并分析完该文件的时间分别是（　）。

A. 1500 μs、1000 μs　　B. 1550 μs、1100 μs

C. 1550 μs、1550 μs　　D. 2000 μs、2000 μs

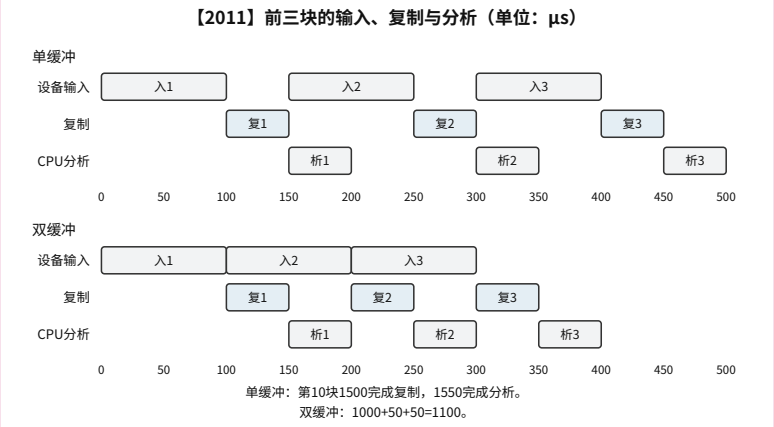


Figure 5.9: 前三块的时间线显示缓冲复用条件；第 10 块仍须完成复制和分析

解析。单缓冲第 1 块输入 0—100，复制 100—150，分析 150—200；第 2 块可在第 1 块分析时开始输入，即 150—250，然后复制 250—300、分析 300—350。以后每 150 μs 推进一块，第 10 块复制结束为 1500 μs，再分析 50 μs，合计 1550 μs。

双缓冲第 1 块输入后，设备立即在另一缓冲输入第 2 块，即 100—200；CPU 同期完成第 1 块的复制和分析，也需 100 μs。10 块输入可连续到 1000 μs，最后一块再复制、分析各 50 μs，总计 1100 μs，答案为 **B**。只算稳定流水的块数而漏掉最后排空，会误得 A。

5.6.2 循环缓冲与缓冲池

辨析·稳态间隔不含首尾全部费用

每块平均处理间隔取重复状态之间的时间；整批从空开始还需启动和排空。先统一计时起点与终点，再套对应公式。

循环缓冲让若干缓冲区按环使用，分别维护下一可填和下一可取位置；必须区分空、已填满、正在使用等状态，不能在消费者取走数据之前覆盖该缓冲区。缓冲池把多个缓冲按空闲、输入、输出等用途组织，按请求分配与回收，可让多个活动复用。

缓冲管理既要选择大小、数量，也要协调生产与消费。双缓冲若输入远快于计算，设备仍会在两个缓冲都被占用后等待；缓冲区数量有限，输入持续过快时总会发生等待。相关计数与互斥规则可回指第 2 章的有限缓冲区，数据传送与并发状态必须同时正确。

5.6.3 稳态平均间隔与整批完成时间

课件常先指定一个重复初态来求“平均每处理一块”的稳态间隔。单缓冲取工作区满、系统缓冲空为初态，周期为 $\max(T, C) + M$；双缓冲取一满一空且工作区空为初态，周期为 $\max(T, M + C)$。前面的整批公式还包含首块装入和末块排空，两种口径不能混算。

课件的单缓冲初态是工作区已有数据、系统缓冲区为空：CPU 处理工作区的同时，设备向缓冲区输入；两者都完成后才能把下一块复制到工作区。下图两组时间数值仅为自选示意，单位一致即可：$T > C$ 时等待输入，周期为 $T + M$；$T < C$ 时等待 CPU 处理，周期为 $C + M$。两者相等时周期仍为 $T + M = C + M$。

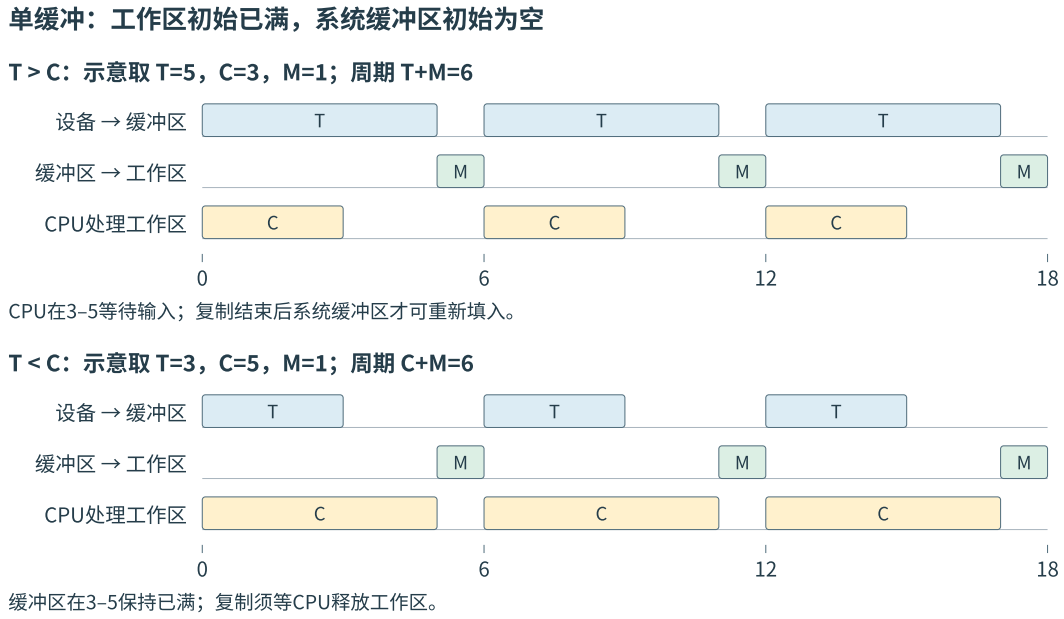


Figure 5.10: 单缓冲两种稳态情况的输入 T、复制 M 和处理 C 时间线

课件的双缓冲初态是工作区为空、缓冲区 1 满、缓冲区 2 空。下图中的 M1、M2 分别从缓冲区 1、2 向工作区复制，处理与复制共享 CPU、不能重叠。$T > M + C$ 时设备输入决定节拍；$T < M + C$ 时复制和处理决定节拍，设备可能等待空缓冲区。$T = M + C$ 时两阶段节拍相同，周期即 $T = M + C$。图中数字是为展示这些条件自选的时间值，并非课件给出的数值。

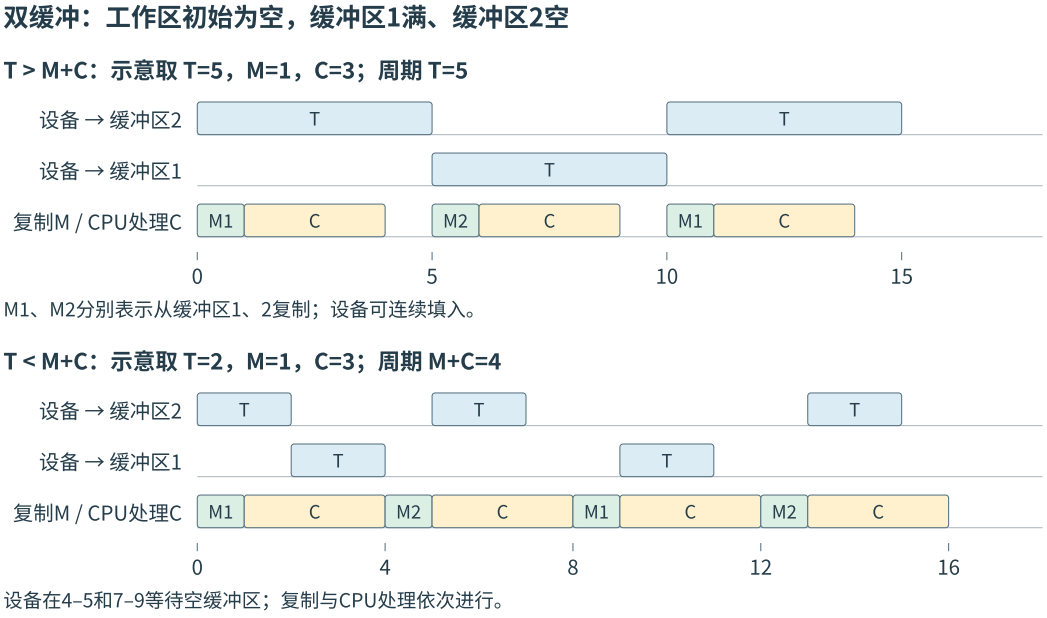


Figure 5.11: 双缓冲两种稳态情况的缓冲区轮转和 CPU 时间线

通信缓冲也取决于用途：若一台机器只有一个共用收发缓冲，而且发送与接收都独占它，同一时刻只能使用一种方向；若分别设置发送、接收缓冲，具备相应链路和控制能力时可并行组织双向数据。双缓冲提供的是条件，并不改变半双工物理链路的限制。

辨析·工作角色不是固定四块

hin、sin、hout、sout 描述缓冲被谁用于何种阶段。缓冲在队列间流转，取走与归还时都要维护状态及同步。

5.6.4 缓冲池中的三条队列与四种工作角色

缓冲池按状态维护空缓冲队列、装满输入数据的输入队列、装满输出数据的输出队列。`hin`、`sin`、`hout`、`sout` 分别表示收容输入、提取输入、收容输出、提取输出时占用的工作缓冲角色，不是永远固定的四块存储。

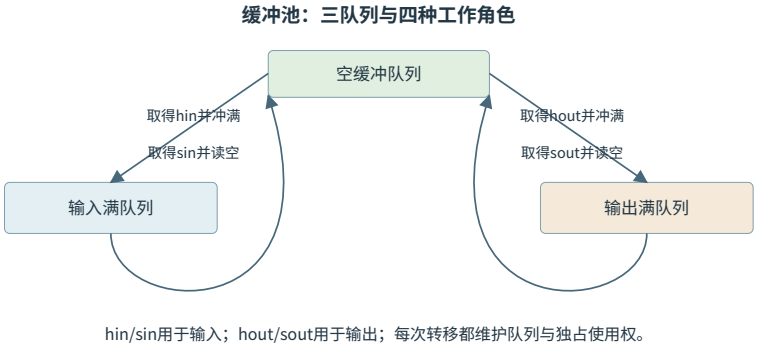


Figure 5.12: 同一缓冲区随输入或输出过程，在空闲、工作角色和满队列之间转移

请求	取得缓冲的位置与角色	完成后的去向
设备输入进程收容数据	从空队列取一块作 hin	冲满后挂入输入队列
计算进程提取输入	从输入队列取一块作 sin	读空后归还空队列
计算进程产生输出	从空队列取一块作 hout	冲满后挂入输出队列
设备输出进程提取输出	从输出队列取一块作 sout	取空后归还空队列

课程连接·设备链也可能形成资源环

进程拿到设备后再等控制器，另一进程持控制器又等所需设备，就可能形成占有并等待与循环等待。设备分配和进程资源分配共享死锁分析方法；仅把某张设备表标为已分配，并不代表完整传输路径已经可用。

队列为空时，相应请求按策略等待；队列和角色转换要受同步保护，避免同一缓冲同时归两个请求所有。

5.7 设备分配、逻辑名称与驱动接口

设备分配要检查设备类型、访问权限、当前占用、逻辑名称到实际设备的映射，并选择可用设备与相关控制器、通道等资源。若需要的资源暂不可得，要按规则排队，不能只分配设备却忽略它必须使用的传输路径。资源逐项申请还可能凑成第 2 章所述的死锁条件。

静态分配在作业开始前取得所需设备，减少运行中占有并等待，但利用率可能低；动态分配在使用时申请，更灵活，也需处理等待和安全。独占与共享是设备使用性质，静态与动态是分配时机，不能完全一一对应。

设备独立性让应用使用逻辑名称或统一接口，设备无关软件完成适当映射；更换设备通常调整映射和驱动，上层不必重写业务逻辑。驱动还须遵守操作系统的入口、同步、内存和完成通知接口；同一硬件的驱动通常不能不加修改地用于所有操作系统。

设备释放后，要更新控制结构、处理队列并通知合适等待者。内核记录 “空闲” 以前必须确认上一请求确实结束，避免两个使用者同时认为自己拥有设备。

5.7.1 DCT、COCT、CHCT、SDT 与 LUT

设备分配不仅要找到空闲设备，还要取得它所依赖的控制器和通道。课件用以下控制结构组织资源：

结构	管理对象与关键字段
设备控制表 DCT	每设备一张，含类型、标识、忙闲／故障状态、控制器指针、重试限额、等待队列
控制器控制表 COCT	每控制器一张，含标识、状态、通道指针、控制器等待队列
通道控制表 CHCT	每通道一张，含标识、状态、所属控制器列表及通道等待队列
系统设备表 SDT	全系统设备目录，每项关联设备类型、标识、DCT 与驱动入口
逻辑设备表 LUT	建立逻辑设备名、所分配物理设备名与驱动入口的对应关系

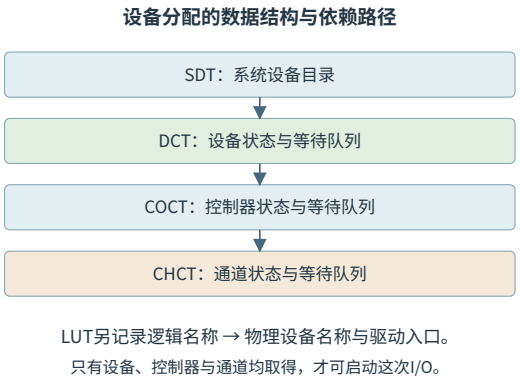


Figure 5.13: 先定位设备，再沿依赖找到控制器与通道；各级都可能有等待队列

在课件的物理设备名方案中，按名称查 SDT，定位 DCT；设备忙则排其等待队列，空闲则分配；再由 DCT 查 COCT，由 COCT 查 CHCT，分别检查和分配，三者全部成功后才能启动 I/O。只取得设备而未取得路径资源，不能认为此次请求已能传输。

改进方案由用户提出逻辑名称或所需类型，系统选择同类空闲设备并登记 LUT，后续使用同一逻辑名即可找到原分配。课件以两台打印机说明查表结果：

逻辑设备名	物理设备标识	驱动程序入口地址
`/dev/打印机 1`	3	1024
`/dev/打印机 2`	5	2046

例如，通过`/dev/打印机 1`找到物理设备 3 及其驱动入口 1024；这些数字只是课件的示意值。系统使用一张 LUT 时，逻辑名称须在该命名空间内唯一；每用户一张 LUT 时，不同用户可使用相同名称。名称隔离与设备实际共享性质仍是两个问题。

5.7.2 安全与不安全分配的特定模型

课件的**安全分配方式**规定，进程取得一个设备后立即因本次 I/O 阻塞，完成后才唤醒，因此在该模型下同一进程同一时段只持有一个正在使用的设备，不会继续持有它去申请另一个。它破坏设备间的请求和保持关系，但同一进程的计算与该次 I/O 串行。

不安全分配方式允许进程取得设备后继续执行，并可继续申请其他设备，直到某请求无法满足才阻塞；这提高重叠机会，却可能构成占有并等待，需要结合死锁避免、检测或解除措施。这里“不安全”是分配方式名称，不表示每次分配后必然已经死锁。静态／动态分配描述资源取得时机，应与这组策略分别判断。

5.8 SPOOLing：先进入磁盘井，再使用独占设备

SPOOLing 利用磁盘上的输入井、输出井和相应进程，把低速独占设备的使用与应用运行分开，形成假脱机工作方式。输入进程把设备数据预先送入输入井，作业随后从井取数据；输出请求先进入输出井，由输出进程按队列统一送到真实设备，井管理负责记录请求与存储位置等。

错项排除·逻辑共享没有增加打印头

多个作业可同时提交到输出井，由后台服务逐份交给物理打印机。应用看到可共享的服务，但真实打印装置仍一次处理一个任务。SPOOLing 把提交与慢设备使用解耦，不能把这种虚拟化说成硬件同时打印多份纸张。

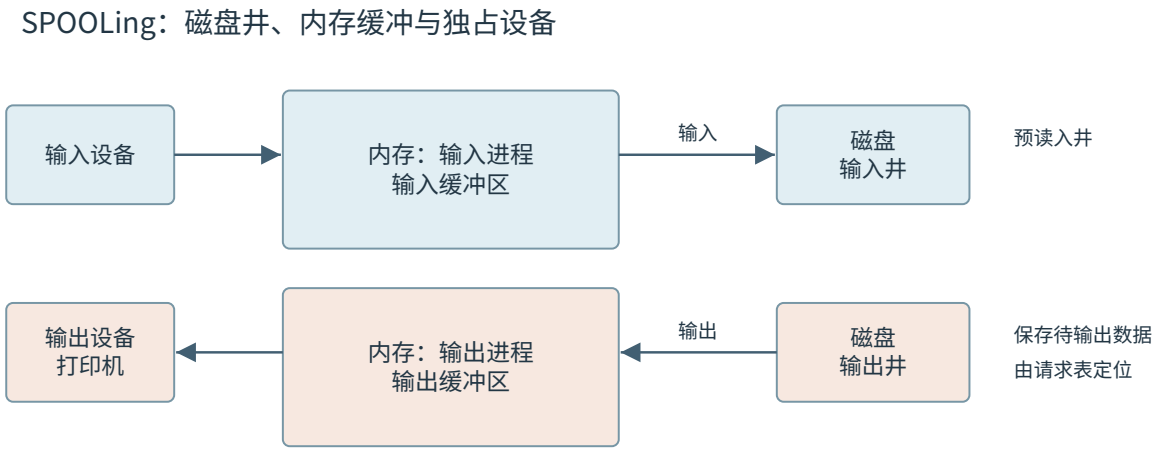


Figure 5.14: 输入设备经内存输入缓冲写入磁盘输入井，输出井经输出缓冲送往输出设备

打印时，假脱机管理进程先在输出井中为每个请求分配存储区，保存待打印数据，再建立记录数据位置等信息的打印请求表并排入队列。输出进程取得队首请求后，将井中的数据经输出缓冲区送至打印机。多个进程可以分别提交打印文件和请求，提交后继续做别的工作；打印守护进程再逐份占用物理打印机。用户看到逻辑上可共享的打印服务，打印机仍一次打印一个任务。不能把“将独占设备改造成共享设备”理解成硬件从此能同时输出多份互不干扰的纸张。

井通常在磁盘，不是简单把所有输出永久留在内存；SPOOLing 也不是用一台专用外围机完全脱离主机运行。多道程序设计让井管理、设备服务与用户作业可以并发，缓冲负责暂存数据，二者分别承担不同作用。

不变量·旋转延迟先看转速单位

7200 转每分先除 60 得到每秒 120 转，一圈约 8.333 毫秒，随机角位置平均等待半圈。若题目直接给秒每转，就改用其一半。先把单位写在每个量后面，能避免把 r 除 2 与 1 除 2r 这两种形式混用。

5.9 磁盘结构、地址与格式化

磁盘盘面划分成磁道，同一半径上的磁道形成柱面，磁道再分扇区。移动磁头到目标磁道需要**寻道时间**，等待目标扇区转到磁头下需要**旋转延迟**，实际数据经过磁头需要**传输时间**。常规平均模型为

$$T_{\text{access}} = T_{\text{seek}} + T_{\text{rotation}} + T_{\text{transfer}}.$$

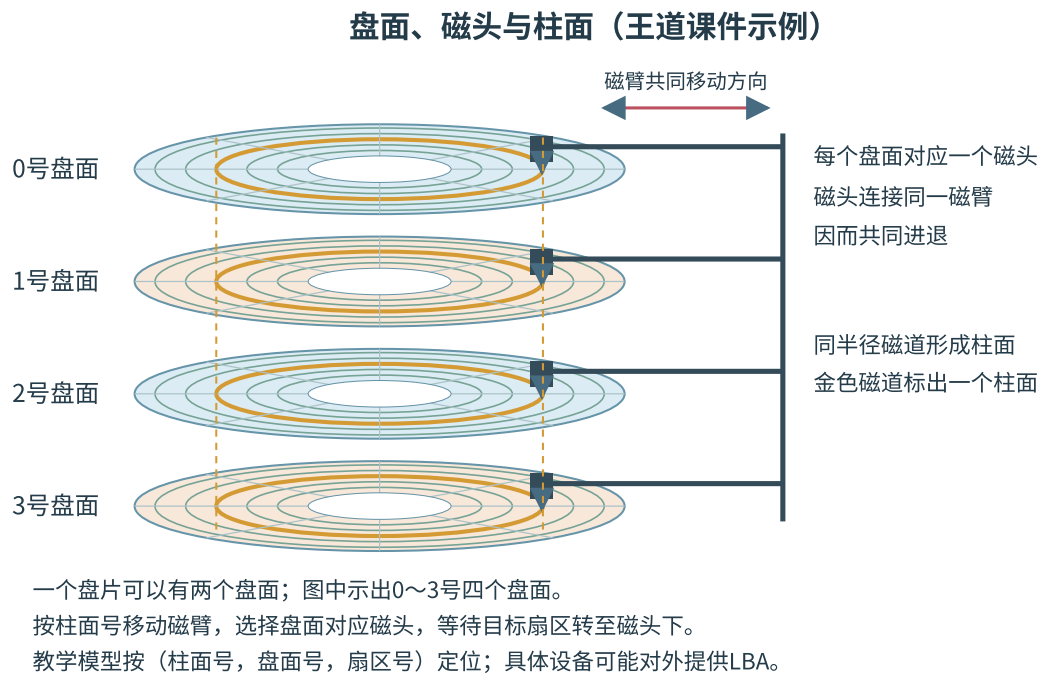


Figure 5.15: 四个盘面的同半径磁道组成柱面，各盘面磁头随同一磁臂移动

不变量·旋转延迟先看转速单位

7200 转每分先除 60 得到每秒 120 转，一圈约 8.333 毫秒，随机角位置平均等待半圈。若题目直接给秒每转，就改用其一半。先把单位写在每个量后面，能避免把 r 除 2 与 1 除 2r 这两种形式混用。

图中以四个盘面为例，每个盘面对应一个磁头；同半径磁道形成柱面，磁头随同一磁臂一起移动。按传统物理地址读数据时先定位柱面，再选择对应盘面的磁头，等待扇区转到磁头下；实际设备可能通过逻辑块地址屏蔽内部几何结构。

按磁头是否随磁臂移动，磁盘可分为活动头磁盘和固定头磁盘；固定头模型为每条磁道配置磁头。按盘片能否更换，还可分为可换盘和固定盘。两组分类描述不同性质。

若一次寻道包含磁臂启动时间 s，且匀速跨越每条磁道耗时 m，跨越 n 条磁道的简化模型为 $T_{\text{seek}} = s + mn$；启动开销及线性假设均以题目给定为准。

若转速为 r 转/秒，一圈为 $1/r$ 秒，随机角位置下平均旋转延迟为 $1/(2r)$。若题目直接给 r 秒/转，则平均延迟为 $r/2$，两种 r 单位不能混用。一个磁道能传 N B，连续读取 b B、忽略跨道与间隙时，传输时间近似为 $(b/N)/r$ 秒。

例如 7200 转/分为 120 转/秒，一圈约 8.333 ms，平均旋转延迟约 4.167 ms。若平均寻道 5 ms、目标连续数据传输 0.5 ms，则平均访问约 9.667 ms。这是**原创单位示例**，用于把 rpm 换成时间；跨道访问若需额外寻道，不能继续当作同一连续磁道传输。

5.9.1 例 5-3　柱面、盘面、扇区是混合进位

例 5.9.1 (配套例题)

〖5.3-016〗磁盘有 100 个柱面，每柱面 8 个磁道，每磁道 8 个扇区。文件 A 有 6400 个逻辑记录，每记录与扇区大小相同，顺序存放；第 0 个记录在 (0 号柱面、0 号盘面、0 号扇区)。磁盘地址 (78、6、6) 存放文件的第几个逻辑记录?

A. 5045　　B. 5046　　C. 5047　　D. 5048

解析。先跨 78 个柱面，每柱面 $8 \times 8 = 64$ 个记录，再跨本柱面的 6 个盘面，最后跨 6 个扇区，得到 $78 \times 64 + 6 \times 8 + 6 = 5046$，答案为 **B**。题目明确从 0 编号，不能再加 1。反算时先除 64 取柱面，余数再除 8 取盘面，最后余数为扇区。

5.9.2 分区、物理格式化和逻辑格式化

物理格式化或低级格式化建立控制器识别的扇区结构及校验等信息；分区划分可独立使用的逻辑区域；逻辑格式化在某区域上建立文件系统结构，如超级块、根目录和空闲管理结构。创建文件系统不是每次开机都要做的初始化。

5.9.3 例 5-4　根目录属于文件系统，不属于扇区编码

例 5.9.2 (配套例题)

【配套例题】下列选项中，磁盘逻辑格式化程序所做的工作是(　)。

I. 对磁盘进行分区　　II. 建立文件系统根目录

III. 确定磁盘扇区校验码所占位数　　IV. 初始化保存空闲磁盘块信息的数据结构

A. 仅 II　　B. 仅 II、IV　　C. 仅 III、IV　　D. 仅 I、II、IV

解析。II、IV 创建的是文件系统结构；I 是分区，III 属于物理扇区组织。答案为 **B**。传统主引导扇区、分区引导区域和文件系统数据区的建立时机也不同，不能见到“引导”就一律归到 BIOS 或逻辑格式化。

5.9.4 引导块与坏块管理

物理格式化建立扇区头、数据区和尾部，记录标识及校验等信息；分区形成逻辑卷；逻辑格式化创建文件系统根目录和空闲管理结构。文件系统块可由一个或多个物理扇区组成，题目中的扇区、盘块、簇若不同长，要先换算。

启动需要一段预先可执行的固件代码，传统模型中先从 ROM 的自举装入程序定位磁盘引导块，再把后续启动程序及内核装入内存。引导块、主引导记录、分区引导区域不是同一个字段，具体流程应与第 1 章给定启动模型对应。

坏块可能由文件系统显式标记，例如在分配表中表示不可用；也可能由控制器用备用扇区重映射，对上层隐藏实际替换。物理坏块管理的目标是避用或替换位置，不是由操作系统软件修复已经损坏的介质。

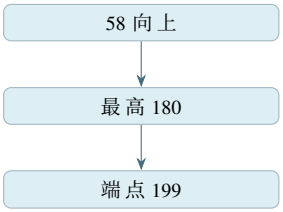
5.10 磁盘调度：把运动顺序和运动距离分开

FCFS 按请求到达次序服务，公平直观，但相邻请求可能使磁头来回移动。**SSTF** 每次选离当前磁头最近的请求，可减少当前一步的寻道，但不是求整个序列的全局最短路径；远处请求可能长期被新来的近处请求越过。

SCAN 沿一个方向服务，达到规定端点后反向；**LOOK** 到该方向最后一个请求便反向，不必到物理端点。**C-SCAN** 只在一个方向服务，到端点后回到另一端，返回过程中不服务；**C-LOOK** 在本方向最后请求处回跳到另一侧第一个请求。回跳是否计入移动距离、是否规定到

最小反例·最高请求不一定就是端点

磁头 58 向上，剩余请求含 180 和 15，物理端点 199。LOOK 可在 180 折返，SCAN 按端点规则继续到 199 再折返，二者距离不同。原配套题恰好请求 199 只能说明那组数据相同，不能推出两算法普遍等价。



仍有 15 号请求时，SCAN 到 199 折返，LOOK 可在 180 折返。

物理端点、何时停止统计，要按题干，不能只看算法名。王道课件提示：题目没有特别说明时，SCAN 按 LOOK 的折返点处理，C-SCAN 按 C-LOOK 处理；若题干明确要求到物理端点，则按该要求计算。

下表按物理端点与最后请求位置区分扫描算法。若题干另给折返、回跳或停止规则，应按具体规则重建轨迹。

算法	怎样选择、沿什么方向服务	折返或回跳的位置	返回途中是否服务
FCFS	按请求到达顺序，方向可来回改变	无固定端点要求	不设专门回跳阶段
SSTF	每次选距当前磁头最近的请求	无固定端点要求	不设专门回跳阶段
SCAN	沿当前方向逐个服务，随后反向服务	到规定的物理端点再反向	反向扫描仍服务
LOOK	同 SCAN，沿两个方向服务	本方向已无待处理请求便反向	反向扫描仍服务
C-SCAN	只沿规定的一个方向服务	到物理端点，再回到另一物理端点	回跳不服务
C-LOOK	只沿规定的一个方向服务	本方向最后请求处，回到另一侧首个待处理请求	回跳不服务，到目标后恢复服务

算法	等待方面的边界
FCFS	后到者不插队；前面的请求仍可能造成很长等待
SSTF	持续近处新请求可能越过远处请求，存在饥饿风险
SCAN	扫描持续推进时按方向轮到；不同位置的服务间隔可不同
LOOK	省去无请求的端段；等待仍受负载和扫描推进规则约束
C-SCAN	循环服务次序使位置机会较均匀；不直接给出固定时间上界
C-LOOK	缩短无请求的端段；循环次序仍需配合有限处理时间等条件

对固定有限的一批请求，若每个请求的处理时间也有限，SSTF 同样会处理完，不能把 “可能饥饿” 说成 “必然饥饿”。持续到达时，扫描能否保持前进还与新请求的接纳和服务规则有关；没有请求耗时、负载及推进条件，就不能宣称任何扫描变体具有无条件的固定等待上界。回跳不服务也不等于物理移动耗时为零。

错项排除·SSTF 每走一步重算距离

初始磁头 85，服务 100 以后，剩余 101 只差 1，而 60 差 40。算法比较的是距当前磁头的距离，不能一次按距初始 85 的差值排完。有限固定批次最终会完成，“持续到达可能饥饿” 也不等于任何批次必然饥饿。

5.10.1 例 5-5　方向决定先服务哪一侧

辨析·平均量用当前轨迹重算

六种调度算法分别重建服务顺序，算每段距离后累加，再除以请求数。不能复用另一个算法的平均值或先排序一次就不再更新。

例 5.10.1 (配套例题)

【配套例题】某硬盘有 200 个磁道，最外侧磁道号为 0，请求序列为 130、42、180、15、199。当前磁头位于 58 号磁道，从外侧向内侧移动。按 SCAN 处理完这些请求，磁头移动的磁道数为（　）。

A. 208　　B. 287　　C. 325　　D. 382

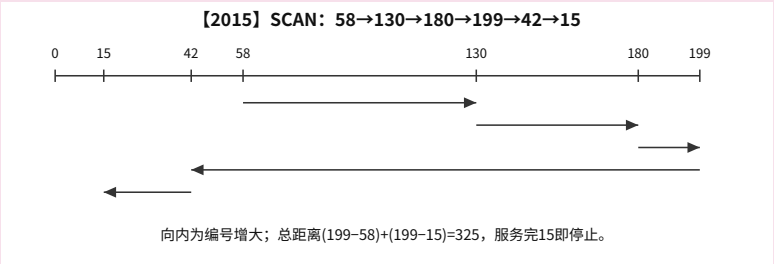


Figure 5.16: 先向编号增加方向，再反向；本题最高请求恰为物理端点 199

解析。向内是编号增大，先服务 130、180、199，再反向服务 42、15。移动距离为 $(199-58)+(199-15)=141+184=325$，答案为 **C**。不能按原输入序列依次访问，那是 FCFS；也不能先服务距 58 最近的 42，那是 SSTF 的首步。

本题最高请求就是端点 199，因此 SCAN 与 LOOK 恰好有相同折返点，不能凭这道题认为两算法永远相同。处理完最后请求 15 就停止统计，也不应继续加到 0 的距离。

若仅用相同数据比较算法：FCFS 按给定序列移动共 $72+88+138+165+184=647$；SSTF 依次 42、15、130、180、199，移动 $16+27+115+50+19=227$。这是题后方法比较，原题仍只问 SCAN；不同方法应各自重建顺序，不能复用另一个算法的轨迹。

调度主要影响寻道，但布局、顺序读预取等也能改变旋转和传输开销。把 “磁头移动更少” 直接等同于 “任何情况下总 I/O 时间最短”，忽略了请求大小、缓存、旋转位置及设备内部重排等条件。

5.10.2 课件同一批请求的六种轨迹

初始磁头在 100，磁道 0～200，请求到达顺序为 55、58、39、18、90、160、150、38、184。扫描类先向编号增加方向，所有请求已在队列中；本例明确规定 SCAN 与 C-SCAN 均需到物理端点，按服务完最后请求停止统计，回跳计实际移动。

算法	服务顺序（括号为非服务折返点）	总移动量
FCFS	55、58、39、18、90、160、150、38、184	498
SSTF	90、58、55、39、38、18、150、160、184	248
SCAN	150、160、184、(200)、90、58、55、39、38、18	282
LOOK	150、160、184、90、58、55、39、38、18	250
C-SCAN	150、160、184、(200)、(0)、18、38、39、55、58、90	390
C-LOOK	150、160、184、18、38、39、55、58、90	322

辨析·平均量用当前轨迹重算

六种调度算法分别重建服务顺序，算每段距离后累加，再除以请求数。不能复用另一个算法的平均值或先排序一次就不再更新。

相应平均寻找长度为总移动量除以 9，依次约 55.333、27.556、31.333、27.778、43.333、35.778。平均量的小数要独立相除，不能将另一轨迹的近似结果直接沿用。

以下按同一位置坐标绘制各算法的完整移动路径，纵向表示处理次序；空心点是初始位置或经过但不服务的折返点，实心点对应被服务的请求。循环算法的回跳路径包含在总移动量中。

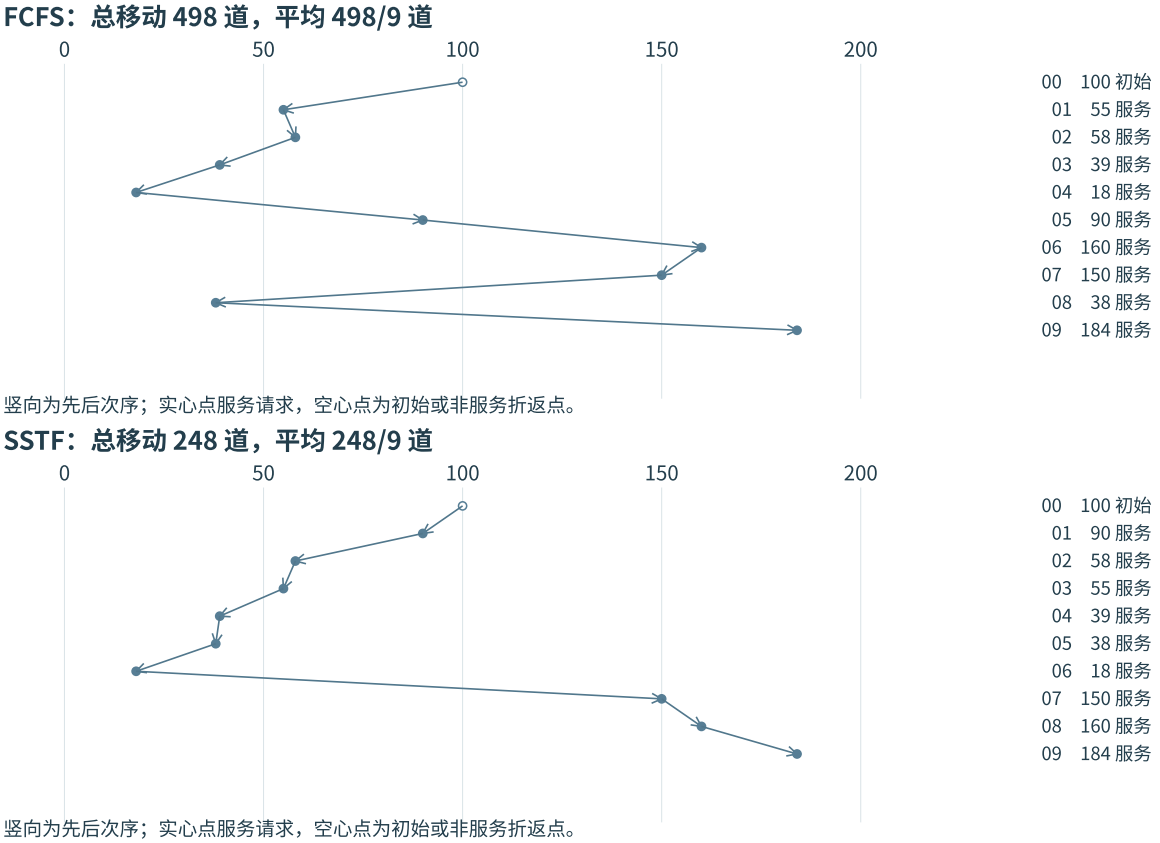


Figure 5.17: 同一批请求下 FCFS 和 SSTF 的完整移动过程

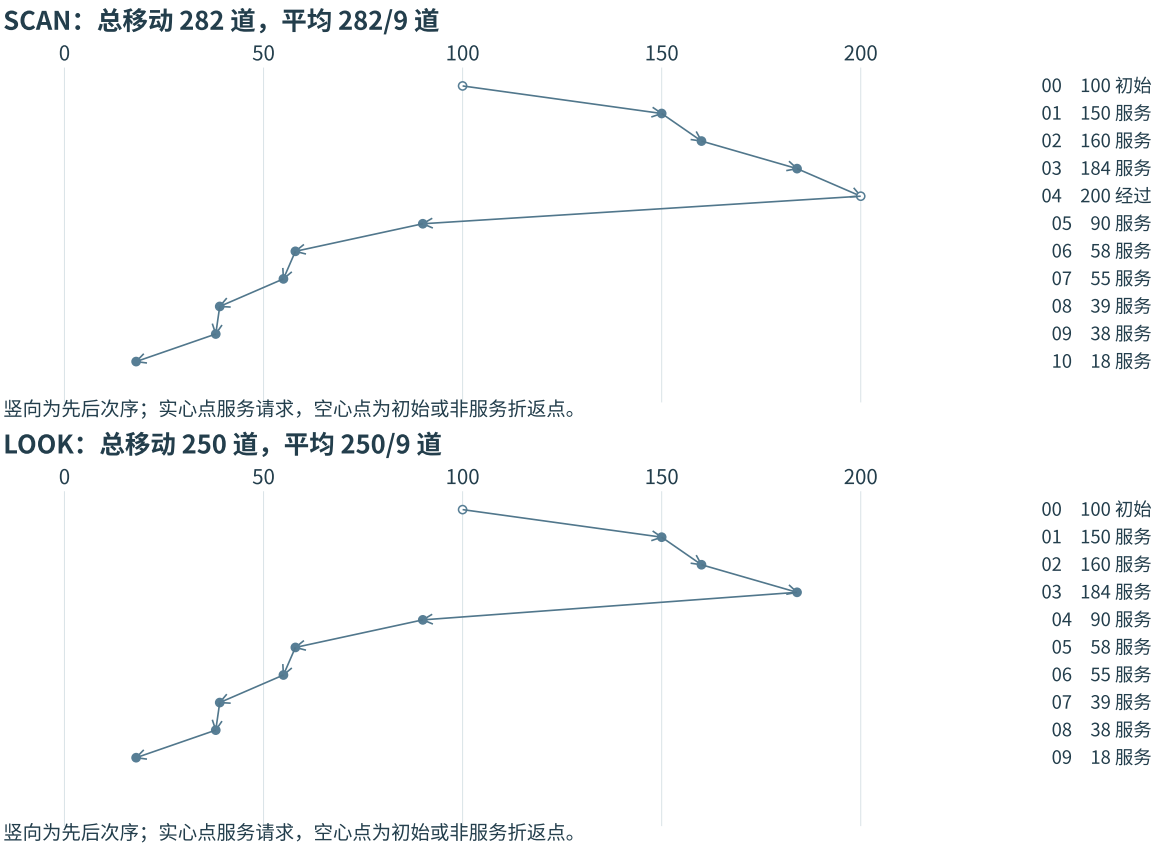


Figure 5.18: 同一批请求下 SCAN 和 LOOK 的完整移动过程

辨析·平均量用当前轨迹重算

六种调度算法分别重建服务顺序，算每段距离后累加，再除以请求数。不能复用另一个算法的平均值或先排序一次就不再更新。

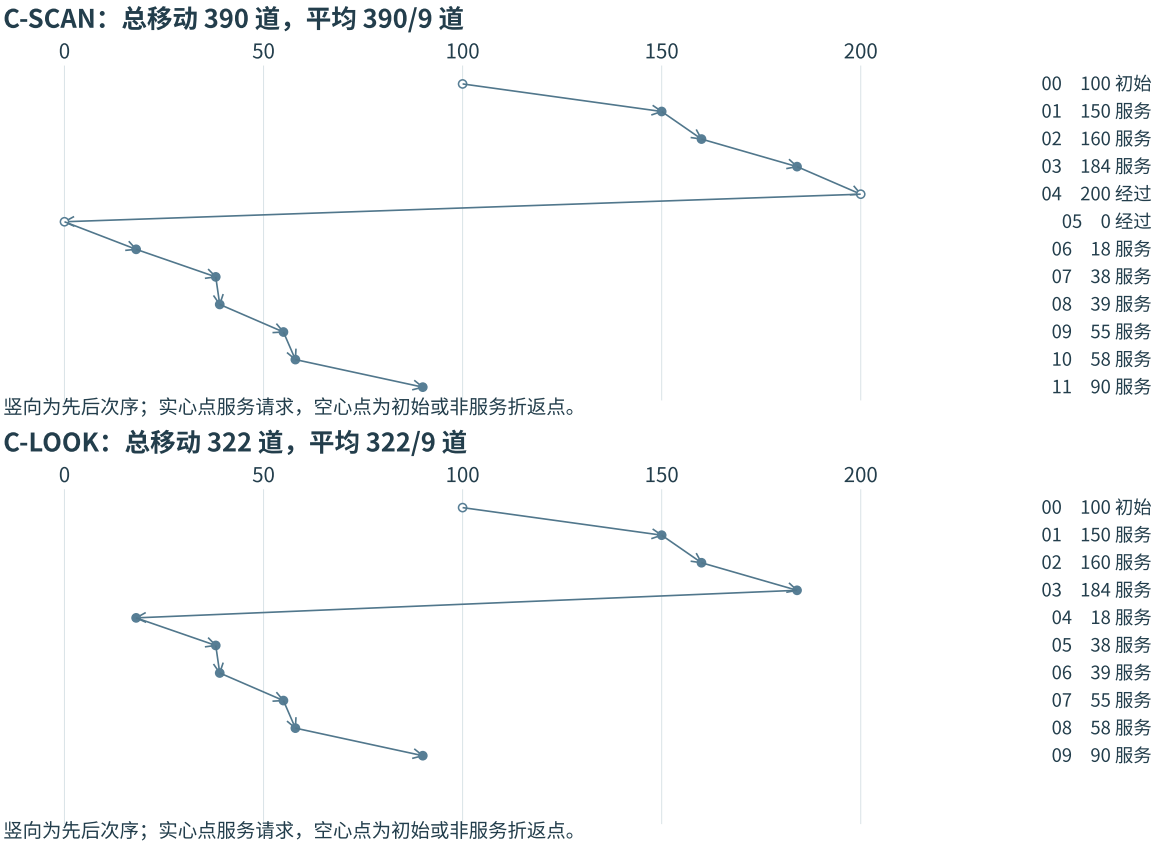


Figure 5.19: 同一批请求下 C-SCAN 和 C-LOOK 的完整移动过程

5.10.3 交替编号、错位命名与柱面优先地址

读完一个扇区以后，控制器可能还需处理时间。若紧邻的下一个逻辑扇区在处理期间已经划过磁头，就要再等待一圈。**交替编号**让相邻逻辑扇区在物理角位置上留出适当间隔，使处理结束时后继扇区恰好到达。

错位命名考虑切换盘面或柱面也需要时间，让下一盘面的逻辑起始扇区相对当前盘面错开，避免切换后立即错过目标。二者调整数据布局和角位置，不是提高电机转速。

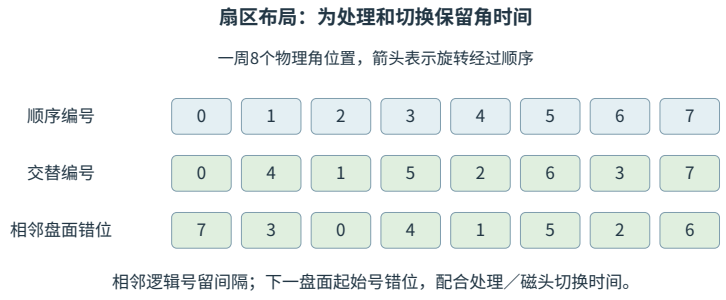


Figure 5.20: 交替编号为处理留出时间；相邻盘面错位可配合磁头切换延迟

以 8 个柱面、4 个盘面、每磁道 8 扇区的模型为例，字段可用 3、2、3 位表示。把线性地址组织为“柱面、盘面、扇区”，连续读完一条磁道后可先切换同一柱面的盘面，无需先移动磁臂；若按“盘面、柱面、扇

辨析·布局优化与转速不同

交替编号和错位命名让后继目标在处理结束时到达，减少额外等转。柱面优先的连续地址则减少不必要的磁臂移动。

区” 递增，则更容易在切换到下一段地址时要求寻道。实际收益仍取决于布局、控制器缓冲和切换耗时。

课程连接·文件删除不等于立即擦除

文件系统取消名字与块使用关系，TRIM 通知设备哪些逻辑数据不再需要，FTL 再按内部有效页和擦除块组织回收。三个层次的状态变化可以有时间差；闪存通常不能像内存字节那样任意原地改写，也不按单页直接擦除。

5.11 SSD：没有寻道，不等于没有管理开销

SSD 采用闪存，访问不需要机械寻道与旋转。闪存通常按页读取或编程，按更大的擦除块擦除；已写内容一般不能像内存字节那样任意原地覆盖，修改时通常异地写入新页，再更新映射。**闪存转换层 FTL** 维护逻辑地址到实际闪存位置的映射，与文件系统的文件偏移映射不是同一层。

异地更新使旧页失效，垃圾回收挑选合适的块，把仍有效的内容迁走，再擦除整块以供后续使用。这会增加内部写入量，形成写放大；备用空间和回收策略会影响性能，不能只用接口带宽计算每次小写的真实代价。

擦写次数有限，**磨损均衡**避免少量块被反复使用。动态磨损均衡主要在新写入时平衡可选空闲块；静态磨损均衡还迁移长期不变的冷数据，使这些数据占用的低磨损块也参与均衡。目的是均衡磨损、延长寿命，不保证每次请求更快。

TRIM 等通知让设备知道哪些逻辑数据已不再需要，有助于回收；文件删除、文件系统取消映射和设备实际擦除也不是同一瞬间。由于没有机械位置，传统寻道调度的优势不再原样成立，但请求合并、公平性、队列深度与写入管理仍有意义。

5.11.1 闪存芯片、块、页与磨损均衡

SSD 由控制与地址转换部分及多个闪存芯片组成；每芯片包含多个擦除块，每块包含多个页。主机给出逻辑块地址，FTL 将其映射到具体芯片、块和页；设备内部并行组织与主机文件索引属于不同层次。

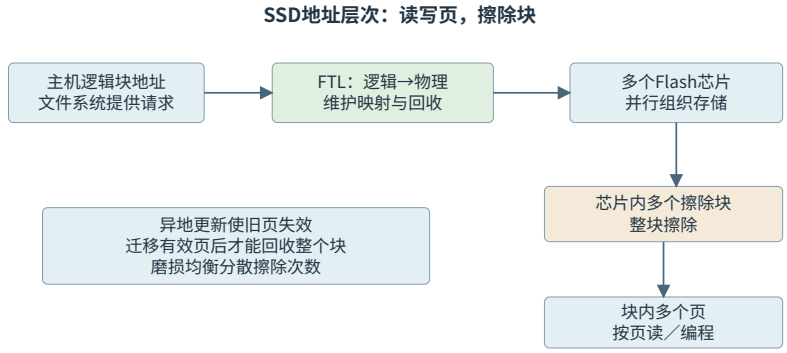


Figure 5.21: 主机逻辑地址经 FTL 定位到芯片、擦除块和页，读取／编程与擦除的单位不同

王道课件用四个闪存芯片的示例展示层次：每个芯片由多个块组成，每个块包含多个页。示意中块大小约为 16～512 KB、页大小约为 512 B～4 KB，实际规格由设备决定。

Figure 5.22: 课件示意：逻辑块号经 FTL 映射到四芯片中的块和页

擦除后的空闲页可以被编程，已写页修改时通常需异地写。回收一个仍含有效页的擦除块，要先保存或迁移有效内容再擦除。读操作可反复进行；擦除次数有限，决定了均衡写入和预留空间的重要性。

动态磨损均衡优先从可用块中选择累计擦除次数较少者；静态磨损均衡还可以搬迁长期不变的冷数据，让磨损较少的块腾出承受更多写入，使较老块更多承担读取为主的存储。SSD 具有无机械噪声、抗震、低功耗等特点，价格、容量及寿命则受具体介质和实现影响。

课程连接·从请求到完成

先确定调用后进程何时阻塞，再说明设备完成、中断处理、唤醒以及之后的调度。

5.12 综合例题：从等待事件到设备地址

5.12.1 例 5-6　一次键盘输入怎样使阻塞者重新获得运行资格

例 5.12.1 (配套例题)

【配套例题】进程 P 通过执行系统调用从键盘接收一个字符的输入，已知此过程中与进程 P 相关的操作包括：① 将进程 P 插入就绪队列；② 将进程 P 插入阻塞队列；③ 将字符从键盘控制器读入系统缓冲区；④ 启动键盘中断处理程序；⑤ 进程 P 从系统调用返回；⑥ 用户在键盘上输入一个字符。以上编号 ①—⑥ 仅用于标记操作，与操作的先后顺序无关。请回答下列问题。

（1）按照正确的操作顺序，操作 ① 的前一个和后一个操作分别是上述操作中的哪一个？操作 ⑥ 的后一个操作是上述操作中的哪一个？

（2）在上述哪个操作之后 CPU 一定从进程 P 切换到其他进程？在上述哪个操作之后 CPU 调度程序才能选中进程 P 执行？

（3）完成上述哪个操作的代码属于键盘驱动程序？

（4）键盘中断处理程序执行时，进程 P 处于什么状态？CPU 处于内核态还是用户态？

解析。先把六个操作放入一次完整请求中。P 执行系统调用，经受控入口进入内核；此时字符尚未输入，P 等待该事件，执行 ② 进入阻塞队列，让出 CPU。用户随后执行 ⑥，键盘控制器产生中断请求；CPU 响应后转入内核入口，执行 ④ 启动键盘中断处理。处理代码执行 ③，把控制器中的字符读到系统缓冲区；数据可用后执行 ①，唤醒 P。P 以后被调度运行，继续尚未完成的系统调用，按调用语义把结果交给用

户，最后执行 ⑤ 返回。

因此，在题目列出的六个操作之间，先后顺序是

② → ⑥ → ④ → ③ → ① → ⑤。

（1）① 的前一个操作是 ③，后一个操作是 ⑤；⑥ 的后一个操作是 ④。这里 “前一个、后一个” 只比较题列操作，① 与 ⑤ 之间仍可有中断返回、等待调度、恢复执行等步骤，不能理解为相邻的两条机器指令。

（2）② **之后**，P 已阻塞，不能继续作为当前运行者，CPU 转去运行其他进程；若没有其他普通就绪进程，则进入系统的空闲执行路径。① **之后**，P 才恢复可调度资格。就绪不等于已经运行，更不保证中断处理一结束就必然选中 P。

（3）完成 ③ 的代码属于键盘驱动程序：它知道控制器接口，执行具体的 I/O 读操作并把字符放入系统缓冲。硬件产生中断请求、CPU 响应中断和驱动代码读字符是不同步骤。这里的字符搬运由执行驱动代码的 CPU 完成，不能因本章传送控制小节讲过 DMA 就改写成 DMA 自动搬运。

（4）在键盘中断处理入口及读取字符的阶段，P 仍在阻塞队列中，处于**阻塞态**；处理数据后执行 ① 才转为就绪。CPU 执行内核的中断处理代码，处于**内核态**。被中断的当前执行者可能是其他进程，内核正在为 P 完成 I/O 并不意味着 P 此刻就在 CPU 上运行。硬件响应负责受控转入并保存体系结构规定的必要状态，入口和驱动代码再保存其余现场、取数据并更新进程状态。

课程连接·从请求到完成

先确定调用后进程何时阻塞，再说明设备完成、中断处理、唤醒以及之后的调度。

5.12.2 例 5-7　把簇号转换成柱面，再逐步执行 SSTF

例 5.12.2 (配套例题)

【配套例题】某计算机系统中的磁盘有 300 个柱面，每个柱面有 10 个磁道，每个磁道有 200 个扇区，扇区大小为 512 B。文件系统的每个簇包含 2 个扇区。请回答下列问题。

（1）磁盘的容量是多少?

（2）假设磁头在 85 号柱面上，此时有 4 个磁盘访问请求，簇号分别为 100260、60005、101660 和 110560。若采用最短寻道时间优先（SSTF）调度算法，则系统访问簇的先后次序是什么?

（3）第 100530 簇在磁盘上的物理地址是什么? 将簇号转换成磁盘物理地址的过程由 I/O 系统的什么程序完成?

解析。（1）先按物理扇区计总容量，簇只是文件系统的分配单位，不会把物理容量再乘 2。

$$C = 300 \times 10 \times 200 \times 512 = 307\,200\,000\ \text{B} = 300\,000\ \text{KB}.$$

本式按 1 KB = 1024 B。每簇 $2 \times 512 = 1024$ B，每柱面有 $10 \times 200/2 = 1000$ 个簇，所以也可用 $300 \times 1000 \times 1024$ B 复核。

（2）本题按从 0 开始的线性簇号，把簇号除以 1000 取整得到柱面号。四个请求的柱面依次为 100、60、101、110。SSTF 每完成一次访问，就以新的柱面重新比较剩余请求，不能只按距初始柱面 85 的距离排序。

当前柱面	剩余请求的柱面与距离	选中的簇	到达柱面
85	100：15；60：25；101：16；110：25	100260	100
100	60：40；101：1；110：10	101660	101
101	60：41；110：9	110560	110
110	60：50	60005	60

因此访问顺序为 **100260、101660、110560、60005**。总柱面移动量可复核为 $15+1+9+50=75$；原问只要求顺序，不需假设转速或旋转位置。

（3）每磁道 $200/2=100$ 个簇。先分出柱面，再分出柱面内磁道及磁道内簇位置：

$$100530=100\times1000+530,\qquad 530=5\times100+30.$$

故柱面号为 100，磁头（盘面）号为 5。磁道内第 30 簇由第 $30\times2=60$ 和 61 号扇区组成，所以该簇的起始物理地址为 **（100 号柱面，5 号磁头，60 号扇区）**，随后还有同一磁道的 61 号扇区。这里各字段都按本题的 0 基编号模型，不能额外加 1。

簇号到该传统物理地址的转换由**磁盘驱动程序**完成：上层给出抽象访问位置，驱动按设备参数形成具体命令；中断处理在设备通知完成时确认状态，不能代替发起请求前的地址转换。若实际设备以 LBA 接口隐藏几何布局，转换分工可不同，本题按给定 CHS 模型回答。

课程连接·计算顺序

缓冲题先画阶段时间线，磁盘题先列访问柱面与方向，再计算总移动距离。

5.13 分支自测：先找可重叠阶段，再核单位

以下为原创回顾设问。

1. DMA 期间 CPU 能运行其他进程，是否意味着传输完全不与 CPU 竞争资源？
2. 例 5-2 若只有 1 块数据，单、双缓冲各需多久？
3. 改用 LOOK 后，例 5-5 总移动为何不变？若最高请求不是 199，会影响哪里？
4. 6000 转/分的磁盘，随机角位置下平均旋转延迟是多少？
5. SSD 删除一页数据后，能否立即按这一页为单位完成擦除？

5.13.1 自测答案

1. 不能。DMA 与 CPU 可能竞争总线、内存带宽等，且还要 CPU 完成启动和结束处理。
2. 都需 $100+50+50=200$ μs。只有一个块时没有后续输入可与它重叠，双缓冲不会自动减半时间。
3. 本题最后的向内请求正好为物理端点。若最高请求更小，LOOK 提前折返，SCAN 仍按端点规则移动。
4. 100 转/秒，一圈 10 ms，平均半圈为 5 ms。
5. 通常不能。编程与读取以页为单位，擦除以较大的擦除块为单位，要结合有效页迁移和垃圾回收。

5.14 主动回忆自测

以下为自拟题，先给机制理由，再计算。

从接口职责检查到机械与闪存过程

1. 块设备、字符设备、共享设备是同一个分类维度吗？字符设备是否能按任意字节地址随机读？
2. DMA 完成后可否用中断通知？用户线程采用阻塞接口能否与 DMA 组合？
3. 为什么驱动负责设备细节，而唤醒的进程仍需等待 CPU 调度？
4. T = 100 微秒、M = 50 微秒、C = 50 微秒、n = 1 与 10 时，单双缓冲分别多久？
5. SPOOLing 为何使打印服务看似共享，却没有让物理打印机同时输出多个任务？
6. 6000 转每分的磁盘，平均旋转延迟多少？若寻道 4 毫秒、传输 1 毫秒，总平均访问多少？
7. 磁头 58 向上、请求 180 和 15，规定 SCAN 到物理端点 199 再折返；SCAN 与 LOOK 处理完两个请求时分别移动多远？
8. SSD 为什么需要垃圾回收和磨损均衡？页读写与块擦除、FTL 与文件索引分别处在哪一层？

课程连接·单位与过程

读写等待、中断、DMA 和磁盘寻道属于不同代价；计算前先确认重叠条件及单位。

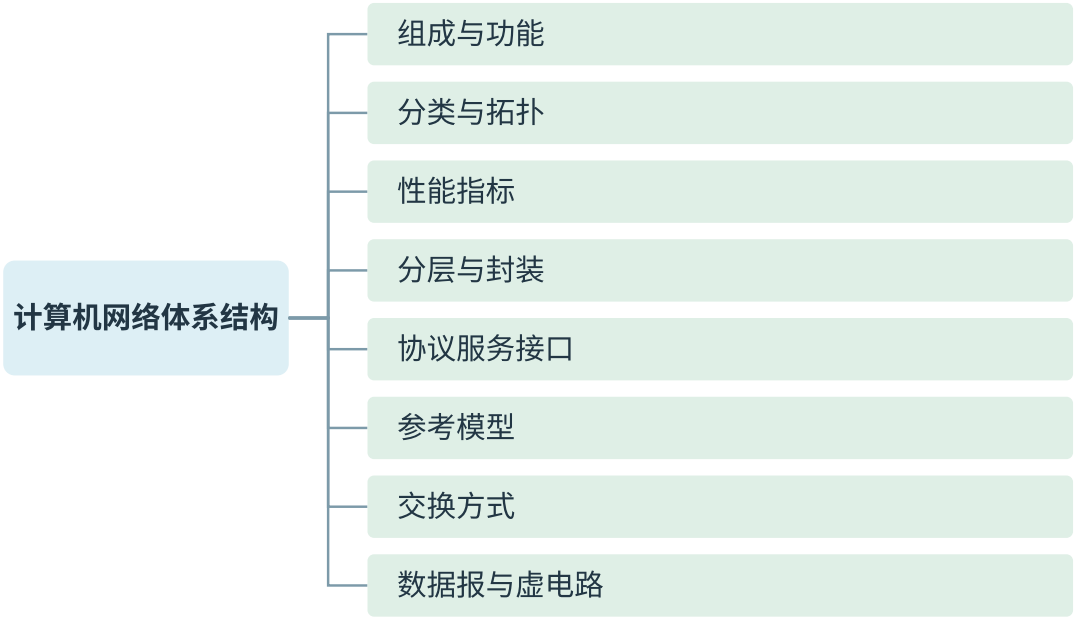
5.14.1 核对答案与关键计算

沿机制核对八个答案

1. 不是。前两者涉及数据组织与寻址，是否共享是另一维度。字符流不因此获得任意字节位置随机寻址能力。
2. 可以，DMA 负责搬运，结束常以中断通知；阻塞接口使请求线程等待数据，与硬件使用 DMA 并不冲突。
3. 驱动按控制器或设备接口生成具体命令；完成中断使等待条件满足、进程就绪，CPU 归属仍由调度决定。
4. 一块时都为 200 微秒；十块时单缓冲 $100+50+9(100+50)+50 = 1550$ 微秒，双缓冲 $100 + 9 \times 100 + 50 + 50 = 1100$ 微秒。
5. 作业先把请求与内容放入磁盘输出井，后台按队列独占实际打印装置；共享的是可提交的服务视图，物理设备仍串行使用。
6. 每秒 100 转，一圈 10 毫秒，平均旋转 5 毫秒；总计 $4+5+1 = 10$ 毫秒，按随机角位置及给定模型。
7. SCAN 先到 199 再回 15，$(199-58)+(199-15) = 325$；LOOK 在 180 折返，$(180-58)+(180-15) = 287$。两者处理完最后请求后停止计数。
8. 异地更新留下失效页，回收需迁移有效页再整块擦除；磨损均衡分散有限擦写次数。FTL 把逻辑块映射到闪存位置，文件系统索引先把文件内偏移映射到逻辑存储块，二者层次不同。

计算机网络体系结构 1

章节脉络



研究一次网络通信，先固定三个问题：数据从哪个端点到哪个端点，正在观察哪一层，时间从哪一个事件算起。同一个“地址”“确认”或“带宽”，在不同端点、不同层次上所指并不相同；端点与层次确定之后，含义才明确。

本书速率中的 k, M, G 分别表示 $10^3, 10^6, 10^9$；1 B=8 bit。容量题若采用二进制数量级，会明确写 KiB、MiB 或题目原约定。除特别说明，传播时延为单向时延，RTT 表示往返时间。

课程连接·系统边界先于分层

操作系统研究一台主机内部如何管理资源；计网研究不同自治主机如何协作。后续遇到“缓存满”时，先判断缓存属于网卡、路由器还是接收进程，才能选对控制范围。

1.1 网络由什么组成，提供什么能力

定义 1.1.1 (计算机网络)

计算机网络是把具有独立功能的计算设备通过通信链路和网络设备连接起来，按共同协议实现通信与资源共享的系统。这里的独立功能说明设备不是只能依赖某一中央处理机工作的被动终端；共同协议保证发送方表达的格式、动作和次序能够被接收方理解。

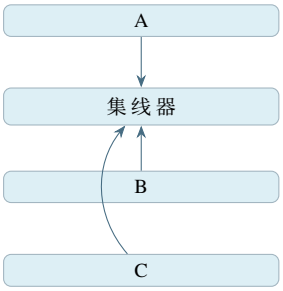
从使用者看，网络提供数据通信、软硬件资源共享、分布式处理等能力。文件放在远端仍可访问，是资源共享；多台机器分工完成计算，是分布式处理。增加通信路径或服务副本可以提高可用性，但“接入网络”本身不保证任何设备都不会失效。

从工作方式看，**边缘部分**包括用户直接使用的端系统和应用，**核心部分**由互联的网络与路由设备组成，负责在端系统之间传送分组。从功能分工看，还可区分资源子网与通信子网：前者侧重计算、存储和共享资源，后者侧重数据传送。两种划分的观察角度不同，因此不要把“边缘”机械地等同于某一物理机房。

链路连接相邻节点，路径由一串链路组成。主机 H1 经 R1、R2 到 H2 的路径有三段链路；R1 和 R2 是中间节点。下文的逐跳确认、路由转发和流水传输都以这一区别为基础。

最小反例·星形也可能共享

三台主机各用一根线连接集线器，物理图是星形；内部仍向其他端口重复信号，三站共享冲突域。把中心换成交换机后并行能力改变，外形本身没有证明介质独占。



星形接线，四站仍共享介质

1.2 网络分类与拓扑描述的是哪些维度

按覆盖范围可区分个人区域网、局域网、城域网和广域网。范围是常用的划分依据，但这些名称并没有统一的公里数界限。局域网通常由一个组织管理，并采用以太网或无线局域网等技术；广域网把较远的网络或节点相连，常包含运营商提供的通信服务。

按传输方式可分为广播式与点到点式网络。广播式网络的多个节点共享传输介质，发送后可能被同一介质上的多个节点接收；点到点链路连接两个端点。IP 广播是一种目的地址语义，能转发 IP 广播并不说明设备工作在共享介质上。

拓扑描述节点及链路的连接关系。总线共享一条主干，星形把节点接到中央设备，环形构成闭环，网状提供多条连接。物理星形与逻辑共享介质可以同时成立：集线器外观上是星形连接，内部仍把比特转送到其他端口，各站共享一个冲突域。交换机同样可组成物理星形，但各端口可以并行工作。判断性能时，还要看中央设备工作在哪一层，不能只看画出的形状。

按使用对象还可分公用网与专用网，按介质可分有线网与无线网。这些分类相互独立，一张专用网完全可以同时包含无线局域网和有线广域链路。

1.3 发送、传播、排队：怎样确定时延

长度为 L bit 的数据经速率为 R bit/s 的链路发送，**发送时延**为

$$t_{\text{tx}} = \frac{L}{R}.$$

这段时间从第一位比特进入链路算起，到最后一位比特进入链路为止。提高速率会缩短这段时间，改变距离不会。信号在长度为 d、传播速度为 v 的链路上走完所需的**传播时延**为

$$t_{\text{prop}} = \frac{d}{v}.$$

传播时延描述同一个比特从链路一端移动到另一端所需的时间。数据变短不会使单个比特传播得更快。若没有排队、处理等开销，接收方收到整个数据的时刻是 $t_{\text{tx}} + t_{\text{prop}}$，因为最后一位也要走完整段链路。

中间设备检查首部、选择出口需要时间；分组等待处理、等待输出链路空闲，都会产生排队时间。逐跳总时延应按题意累加各段的发送、传播、处理和排队时间。有流水重叠时，不能把“每个分组独立走全程”的时延乘以分组数，本章后续沿时间线计算。

带宽在链路性能题中通常表示额定数据速率；**吞吐量**表示某一观察区间内实际成功传送的数据率；**有效吞吐量**只计应用需要的有效载荷。100 Mb/s 链路不等于应用每秒一定能得到 100 Mb，首部、确认、竞争和重传都会消耗时间。若一条路径各链路分别为 100、10、50 Mb/s，长期单流吞吐量不可能超过 10 Mb/s；多个流竞争时还可能更低。

时延带宽积 Rt_{prop} 的单位是 bit，表示在链路持续被填满时，沿一个方向正在传播的比特数量。可靠传输若要让发送方不因等待确认而停顿，更常用 $R \cdot RTT$ 估算往返期间需要容纳的在途数据量。前者采用单向传播时间，后者采用往返反馈时间，不能在同一公式里不加说明地互换。

课程连接·像函数调用的接口

同机上层请求下层服务可类比函数调用：约定接口，不需要知道实现细节。两端对等层的协议则约束交换消息的格式与动作，不能把本机调用关系画成远端数据直接穿越介质。

> **性能题先固定边界**
>
> 先写数据量与速率单位，明确问第一位还是最后一位、单向传播还是往返反馈；再在路径上标记存储转发时刻。单链路发送与传播可前后相接，多组多跳可能重叠，不能把一组总时间直接乘组数。

1.3.1 一个可直接检查单位的小例

1500 B 分组经 100 Mb/s、长 200 km 的链路发送，传播速度取 2×10^8 m/s。发送时延为 $1500\times8/10^8=120\ \mu s$，传播时延为 $200000/(2\times10^8)=1$ ms，最后一位的到达时刻为 1.12 ms。链路中的单向时延带宽积为 100000 bit，即 12.5 kB，已经大于这个分组的长度。链路足够长时，“发送已结束但分组仍在路上”完全可能。

若改用 1 Gb/s，传播时延仍是 1 ms，发送时延降为 12 µs。此例区分三个会独立变化的量：文件有多大、比特送入链路有多快、比特沿介质走多远。

1.4 分层与封装：对等层怎样交换数据

一台主机要把应用数据送出，既要说明应用的含义，又要标识收发进程、选择网络路径、组织一跳传送和产生物理信号。分层把这些任务按相对独立的功能划开。上层调用下层提供的服务，而不需要知道该服务内部采用哪一种队列或硬件实现。

第 n 层的协议数据单元称为 n 层 PDU。交给该层的上层数据是其服务数据单元 SDU；该层附加的首部、必要时还有尾部，包含协议控制信息 PCI。常见封装关系为

$$n\text{-PDU}=n\text{-PCI}+n\text{-SDU}.$$

这里“+”表示封装组合：PCI 不一定只出现在首部，也不一定是一段连续字段。上层的 PDU 成为下层的 SDU。接收端按相反方向解封装：先取出本层需要的信息并检查，再把数据向上交付。

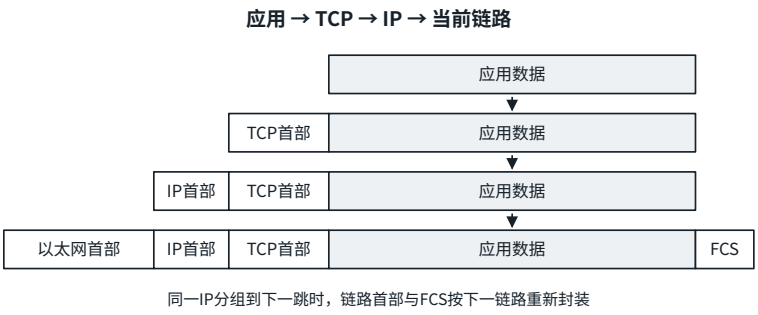


Figure 1.1: 应用数据逐层封装；链路首尾部只用于当前一跳

两个主机的传输层按 TCP 规则进行逻辑上的对等通信，但 TCP 段实际仍须交给本机 IP、链路层和物理层传送。图中的水平联系描述协

议关系，竖直联系描述同一节点内的服务调用。普通路由器检查转发所需的网络层信息，不把经过它的每个 TCP 段交给本机应用。

分层能够统一术语、规定接口、减少功能间依赖，**不要求把各层内部实现算法一并固定**。判断“体系结构是否规定某内容”时，先区分必须遵守的外部行为与具体实现方法。〖1.2-001〗所考的“定义功能执行的方法”属于后者，不能把标准化理解成所有机器必须使用同一种程序。

错项排除·确认不自动等于可靠

无线链路 ACK 只确认当前一跳，后续路由仍可能丢包；CRC 只检出部分差错。题目出现“确认”“检错”时，先核对其作用端点，再排除把一项局部机制扩大为端到端保证的说法。

1.5 协议、服务和接口分别约束谁

协议是对等实体通信时共同遵守的规则，包括语法、语义与时序。语法规定字段格式；语义规定字段与控制信息表示什么、接收后做什么；时序规定什么时候能执行、事件应按何种次序发生。只列出报文的字段，还没有给出完整协议。

服务是本层与下层协作后向上层提供的能力。**接口**描述同一节点内相邻层如何使用该能力；服务访问点 SAP 是访问服务的位置。协议是水平关系，服务与接口是相邻层的垂直关系。端口是传输层中区分应用端点的标识，MAC 地址、IP 地址和端口各属不同层次，不能把三者互相替代。

把同一次普通通信放在三个层次比较，先问“这个标识交给谁解释、它指向哪一种端点”，再看它会不会随转发改变。

比较项	以太网 MAC 地址	IPv4 地址	TCP/UDP 端口
所在首部与长度	MAC 帧头，源、目的各 48 bit	IPv4 首部，源、目的各 32 bit	传输层首部，源、目的各 16 bit
基本指向	当前以太网一跳的收发接口	IP 分组的源与目的主机接口	对应主机上的应用端点
谁据此作基本处理	网卡接收、二层交换机查目的 MAC 转发	路由器按目的 IP 查路由	目的主机传输层分用
普通跨路由转发	下一跳重新封装源、目的 MAC	不经 NAT 时源、目的通常保持	不经端口转换时源、目的通常保持

表中关于 MAC 地址的结论只适用于普通以太网；802.11 还要区分当前无线跳与原始源/目的。路由器收到的帧，目的 MAC 可以是自己，其中 IP 分组的目的地址却仍是远端主机。因此“这一层收到数据”不等于“最终应用已收到数据”。第 3～5 章的格式图会把这些标识放回各自首部。

还要把“建立连接”“可靠交付”“有确认”分开判断。面向连接服务在传送前建立逻辑联系，通常需要保留状态；无连接服务让各数据单元独立提交。可靠服务承诺按其定义处理丢失、重复、次序等问题；有确认只说明存在应答机制。连接不等于预留固定带宽，检错不等于可靠交付，一次确认也不能独自解决重复接收。

这三个问题应分别作答：虚电路先建立逻辑路径，是否可靠还要看具体协议；以太网有 CRC 检错，基本 MAC 却没有逐帧确认重传；802.11 的单播确认只反馈当前无线跳，不能证明远端应用交付。比较服务时固定端点和协议，才能避免把一项机制扩成整条通信路径的保证。

1.6 OSI 与 TCP/IP：先看模型，再看端点

OSI 自下而上为物理层、数据链路层、网络层、传输层、会话层、表示层、应用层。它把功能关系抽象出来，不是要求现实网络的每个设备都

有七层独立软件。

层次	判断功能时先找的对象	典型任务
物理层	一段物理链路上的原始比特	电平、编码、接口、比特传输
数据链路层	相邻节点之间的帧	成帧、检错、按协议实现流控与可靠传输、介质访问
网络层	跨网络传送的分组	寻址、路由选择与转发
传输层	不同主机上的应用进程端点	复用分用、按协议实现可靠传输与流控
会话层	应用之间的会话	对话管理、同步点与恢复
表示层	数据的表示方式	编码格式转换、压缩与加密
应用层	应用所需的网络功能	Web、邮件、名称服务等

不变量·模型与实例分开

OSI 列出的某层可用功能，不等于该层每一种实际协议都实现了全部功能。IP 使用无连接数据报、TCP 提供可靠字节流、UDP 不提供可靠交付，必须先固定题目给的是哪种模型。

数据链路层与传输层都可能包含流量控制，区分依据是控制范围。相邻节点之间，接收方来不及处理帧，是链路层问题；两个端系统之间，接收进程处理不及，是传输层问题。噪声使比特翻转属于物理传输现象，是否发现差错、是否重传，则要看上层协议，不能由“差错发生在物理链路”推出“物理层完成全部差错控制”。

1.6.1 例 1-1　端到端服务从哪层开始

例 1.6.1 (配套例题)

配套例题。在 OSI 参考模型中，自下而上第一个提供端到端服务的层次是（　）。

A. 数据链路层　B. 传输层　C. 会话层　D. 应用层

解析。“自下而上第一个”要求找到跨越中间网络、服务于两端应用进程的最低层次。数据链路层只处理相邻节点，网络层把分组送到目的主机；传输层开始为进程端点提供服务。因此选 **B**。会话层、应用层也工作在端系统，但都不是题目所问的第一个层次。

1.6.2 例 1-2　相邻节点间的流量控制

例 1.6.2 (配套例题)

配套例题。在 ISO/OSI 参考模型中，实现两个相邻结点间流量控制功能的是（　）。

A. 物理层　B. 数据链路层　C. 网络层　D. 传输层

解析。题干已经把范围限定为“相邻结点间”，应在帧的一跳交付层次判断，选 **B**。若条件改成两端主机中应用进程间的流量控制，应转到传输层；不能只背“流控属于某一层”。

TCP/IP 常用四层模型为网络接口层、网际层、传输层、应用层。网络接口层对应具体链路技术，网际层以 IP 实现尽最大努力的数据报交付，传输层包括 TCP 与 UDP，应用层合并 OSI 上三层的任务。本书为展开链路机制，把网络接口层再分为数据链路层、物理层，采用五层教学表示；五层与四层模型的分界不同，但协议关系不变。

还应区分抽象模型中允许的功能与某种实际协议提供的服务。OSI 网络层可以讨论面向连接和无连接服务；现实 IP 提供无连接服务。以太网帧含 FCS，但通常检测到错误后丢弃，不负责链路层确认重传；不能把“链路层可提供可靠传输”写成“所有链路层协议保证无差错、有序交付”。TCP/IP 的传输层同时有 TCP、UDP，也不能拿 UDP 直接回答

只针对教材 OSI 模型约定的题。

课程连接·流水与 CPU 同一个计数

分组流水和指令流水都应分成 “填满流水” 与 “稳定间隔”。等速链路中首组付出多跳发送代价，之后按瓶颈服务间隔完成；直接把首组总延时乘组数，会把已经重叠的时间重复计算。

1.7 电路、报文和分组交换如何沿时间线计算

电路交换先建立端到端电路，通信期间按分配的资源传送，结束后释放。其完成时间通常包括建立时间、源端发送时间和沿路径传播时间；电路一旦建立，中间节点就不必收全整份文件再重新发送。电路在空闲时仍可能保留资源。

报文交换把一整份报文作为存储转发单位；中间节点收全报文后才能向下一段链路发送。**分组交换**把报文切成较小分组，每组独立存储转发，因此不同链路可以同时传送不同分组。分组需要附加首部，且可能排队、丢失或乱序；“分成越小越快” 忽略了首部和处理代价。

比较三种方式，要放在同一条时间线上：什么时候可以开始，中间节点要等多少数据，链路空闲时资源能否供别人使用。

比较维度	电路交换	报文交换	分组交换
传送前准备	先建立电路并分配资源	可直接提交报文	数据报可直接提交；虚电路另需建立逻辑路径
中间节点的基本转送单位	已建立电路中的比特流	完整报文	一个分组
开始向下一段发送的条件	电路已接通，可连续传送	收齐整个报文且出口可用	存储转发模型下收齐该分组且出口可用
资源使用	分配给连接的资源在空闲时也可能保留	链路按报文动态共享	链路按分组动态共享，可形成流水
时间与存储的主要代价	建立时间、专用资源占用	长报文逐跳收齐的等待与缓存	分组首部、排队与处理；小组过多会增加开销

完成时间的先后没有固定顺序，必须结合具体条件判断。短数据可能来不及抵消建立电路的开销；长数据可从分组流水获益，但首部与排队又会影响结果。题目给出直通等转发规则时，还需修改 “收齐才转发” 的事件条件，不能同时套用存储转发公式。

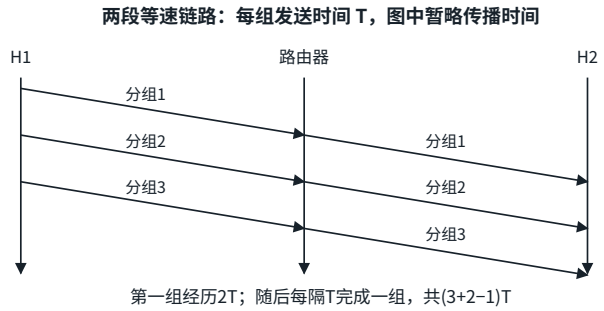


Figure 1.2: 两段等速链路上的三个分组；箭头连接本组开始发送与下一节点收齐的时刻，并非单个比特的传播线

若有 h 段等速链路、每组总长 L bit、速率 R，共 n 个等长分组，无排队和处理开销，第一组需 hL/R 完成发送，后续每组间隔 L/R 到达，故

定理 1.7.1 (等速存储转发流水的完成时间)

在上述等长分组、固定等速路径、无竞争及处理开销的模型下，

$$T = (n + h - 1)\frac{L}{R} + \sum_{i=1}^{h} t_{\text{prop},i}.$$

最小反例·有连接仍可共享

两条虚电路沿同一物理链路依次发送分组，各节点维护不同标识映射即可区分。它们都已建立逻辑连接，却无需各占一根线，这就排除了“虚电路必预留固定带宽”的绝对说法。

第一组要在每段链路完整发送一次，所以发送项为 hL/R；流水填满后，每个出口每隔 L/R 送完下一组，后续 $n-1$ 组只再增加 $(n-1)L/R$。沿途固定传播项对首组累加，后续相邻组间隔不再重复加入整条路径传播时间，得到上式。

若一个报文长 F bit，按整报文存储转发，则相应发送项为 hF/R。链路速率不同时，先按各链路的开始与结束事件递推，不能随意把 R 换成平均速率。对等长分组、固定路径和无其他流量的理想流水，第一组时延之后，后续组受最慢链路的服务间隔限制。

1.7.1 例 1-3　时延带宽积先转成传播时间

例 1.7.1 (配套例题)

配套例题。在下图所示的分组交换网络中，主机 H1 和 H2 通过路由器互连，2 段链路的带宽均为 100 Mb/s，时延带宽积（即单向传播时延 × 带宽）均为 1000 bit。若 H1 向 H2 发送 1 个大小为 1 MB 的文件，分组长度为 1000 B，则从 H1 开始发送时刻起到 H2 收到文件全部数据时刻止，所需的时间至少是（注：$M = 10^6$）（　）。

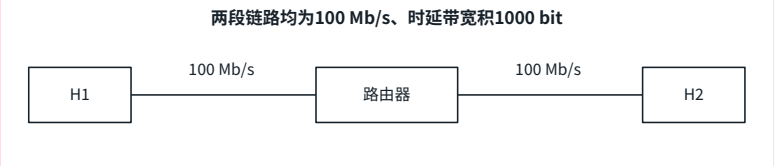


Figure 1.3: 原题的两段链路，均为 100 Mb/s 且时延带宽积 1000 bit

A. 80.02 ms　　B. 80.08 ms　　C. 80.09 ms　　D. 80.10 ms

解析。按题设分组长度直接切分文件，分组数为 $10^6/1000 = 1000$，每组发送时间为 $8000/10^8 = 80\ \mu s$。每段传播时间由时延带宽积求得 $1000/10^8 = 10\ \mu s$。

两段链路上第一组用 $2 \times 80 + 2 \times 10 = 180\ \mu s$ 到齐；后续 999 组每隔 80 μs 到齐。总计

$$180 + 999 \times 80 = 80100\ \mu s = 80.10\ \text{ms},$$

选 **D**。1000 个分组并不是各自独占两段链路、依次走完全程，因此不能算成 $1000 \times 180\ \mu s$。本题没有额外给出分组首部，不要自行把每组 1000 B 再增加 20 B。

1.8 数据报与虚电路：连接不等于电路

数据报方式让每个分组独立携带寻址信息、独立选择下一跳。同一报文的分组可能走不同路径，可能乱序；IP 属于这种方式。网络不保证交付，并不妨碍端系统在其上建立 TCP 可靠连接。

虚电路方式先建立一条逻辑路径，在各中间节点保存映射状态。分组到达某节点后，按输入链路及虚电路标识查出输出链路和新的局部标识。VCID 通常具有链路局部意义，不要求一路数值不变。按既定路径转发有利于保持分组顺序，但是否提供检错重传、带宽保障，仍取决于具体服务。

按相同的维度比较两者：数据报在每组中带目的寻址信息，网络不为这次通信预建逐跳虚电路状态，每组可独立选路；虚电路在建立时选路并保存映射，后续组按局部 VCID 转发。同一路径、较短标识和按序到达是虚电路模型的特点，资源是否独占却不是由“有连接”决定的。电路交换与虚电路都可能先建立联系，前者分配电路资源，后者仍把链路按分组共享。

术语·网络的连接对象

网络连接设备，互连网连接多个网络；Internet 采用 TCP/IP 协议族。

1.8.1 例 1-4　虚电路必须预留带宽吗

例 1.8.1 (配套例题)

配套例题。下列关于虚电路网络的叙述中，错误的是（　）。

A. 可以确保数据分组传输顺序

B. 需要为每条虚电路预分配带宽

C. 建立虚电路时需要进行路由选择

D. 依据虚电路号（VCID）进行数据分组转发

解析。虚电路建立了路径与转发状态，但依然属于分组交换；多条虚电路可以共享链路，不必各自占用一份固定带宽。错误项为 **B**。建立时选路、传送时按已建状态转发，与“每个分组重新独立选路”的数据报方式不同。永久虚电路可以长期保留，不能说所有虚电路都在一次会话后立即撤销。

1.9 网络、互连网与互联网：连接的对象不同

一组设备通过交换机等构成的网络，与多个网络经路由器互连形成的互连网，观察的对象不同。小写的 internet 泛指互连的网络；大写 Internet 指全球性的互联网，依靠共同的 TCP/IP 协议族实现互通。接入互联网通常需要互联网服务提供商 ISP 提供接入服务。家庭内部设备可以先构成局域网，再经接入链路进入 ISP 的网络。

网络的物理组成包括硬件、软件和协议。硬件承担处理、存储和传输，软件实现应用与管理，协议规定参与通信的实体必须遵守的规则。按功能分为资源子网与通信子网，按工作方式分为边缘与核心，是两种不同的分类。服务器集群把计算分派给多个节点是分布式处理；把请求分摊给多个服务器是负载均衡；复制服务和数据有助于提高可靠性，但冗余也需要故障检测和一致性机制配合。

1.10 性能指标还要说明统计口径

速率通常指连接在单位时间内能传送的比特数，**吞吐量**是给定网络或接口在观察时间内实际通过的数据量。对应用有用的数据与首部、填充、重传数据应分开统计。例如一秒传了 100 个 1000 B 帧，每帧只有 900 B 有效数据，则总帧吞吐量为 800 kb/s，应用有效吞吐量为 720 kb/s；物理线路的额定速率还可能更高。

不变量·出口何时开始

下一跳既要等本组收齐并处理，也要等出口链路空闲，取两个时刻的较大值。

信道利用率可表示观察时间内信道被使用的比例；某些可靠传输题特指发送有效数据所占比例，需按题意确定分子。网络利用率描述整个网络资源的使用程度，并不等于某一条链路的利用率。利用率接近饱和时，等待同一资源的分组增加，排队时延可能迅速上升。教材常用理想化关系

$$D = \frac{D_0}{1 - U}$$

说明这种趋势：D_0 为网络空闲时延，U 为归一化利用率。它依赖队列模型，只用于解释负载与时延的关系，不能把每个真实网络的排队时延都直接代入此式。例如按此模型，$U = 0.5$ 时 $D = 2D_0$，$U = 0.9$ 时 $D = 10D_0$。

往返时延 RTT 必须有起止事件：课件采用从发送方发送完数据，到收到相应确认的口径。性能题若把确认发送时间、处理时间忽略，RTT 常近似两倍单向传播时延；题目明确给出实际 RTT 时，应按其边界直接使用。发送请求本身的发送时延是否已包含在所给 RTT 中，应由题意确定，不能重复加入。

误码率是在一定统计条件下，错误比特数与传输总比特数的比值。若独立比特误码概率为 p，一个 L bit 帧至少有一位错误的概率为 $1-(1-p)^L$；只有 pL 较小时才近似 pL。这是额外的独立误码模型，突发错误条件下不能直接套用。误码率低也不保证分组不会因拥塞被丢弃。

1.11 存储转发有处理时间时怎样递推

设 A 经 B、C 到 D，三段链路各 0.5 kb/ms，单向传播各 1 ms；报文 4 kb，切成 4 个 1 kb 分组。B、C 收齐一个分组后各处理 0.5 ms，再请求下一条链路发送；没有其他流量，处理可与其他分组在链路上的传输重叠。每组发送时间为 2 ms。

第一组在 A 的发送区间为 $[0, 2]$，B 于 3 ms 收齐、3.5 ms 开始发送；C 于 6.5 ms 收齐、7 ms 开始发送；D 于 10 ms 收齐。源端其余分组每隔 2 ms 发出，在本例的独立处理模型下，D 的收齐时刻为 10、12、14、16 ms。最后一组完成时间为 16 ms。

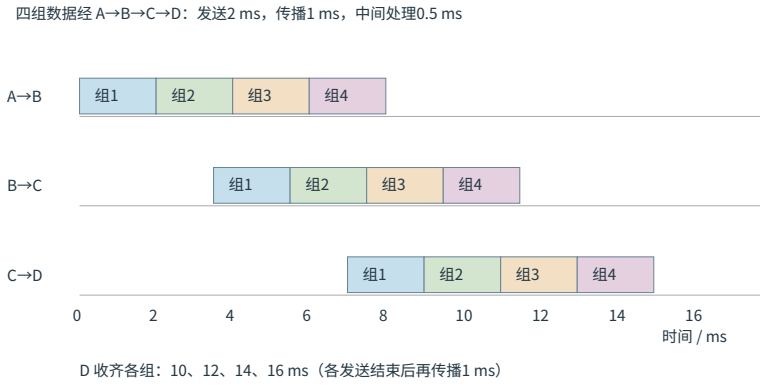


Figure 1.4: A 到 D 四个分组的存储转发过程，各中间节点处理 0.5 ms

若整份 4 kb 报文逐跳存储转发，每段发送 8 ms，最后到达时间为 $3(8+1)+2\times 0.5 = 28$ ms。课件另一个报文交换算例把每个中间节点的处理时间设为 2 ms，故对应总时延为 $3(8+1)+2\times 2 = 31$ ms。若

采用已建立的理想电路，中间节点不再按报文存储处理，则文件发送 8 ms、传播 3 ms，共 11 ms；建立和释放开销按题意另计。这样比较时，三种模型使用相同数据量、路径和传播条件，差别来自中间节点何时允许转发。

更一般地，令 $F_{i,j}$ 表示第 j 组在第 i 段链路发送结束的时刻，链路 i 的发送时间为 L_j/R_i。开始时刻应同时满足 “本链路已空闲” 和 “上一段已收齐且处理完” 两个条件，因此用两者的最大值递推。异速链路、变长分组或串行处理器应回到这个事件关系，而非勉强套等速公式。

不变量·回忆时说出计时起点

同一网络从请求建立连接、首次发送数据或收到第一位开始计时，会得到不同表达式。主动回忆时先口述起点与终点，再写公式；把条件也回忆出来，才可能迁移到变式题。

1.11.1 异速两跳与信道使用比例

1000 B 分组经过 100 Mb/s 和 80 Mb/s 两段链路，传播时延分别 0.01 ms、0.05 ms。存储转发要求第二段等路由器收齐，因此总时延为

$$\frac{8000}{10^8}+\frac{8000}{80\times10^6}+0.01\ \text{ms}+0.05\ \text{ms}=0.24\ \text{ms}.$$

若路由器处理、排队合计 0.04 ms，总计 0.28 ms。这里两段发送时间不同，不能取两段速率的算术平均来计算。

H1 与 H2 连接一分钟，H1 以 80 Mb/s 传输 20 张各 10 MB 的照片。总量为 $20\times10\times10^6\times8$ bit，实际发送占 20 s，按该方向信道被使用时间统计，利用率为 $20/60=1/3$。应先把 Byte 换为 bit，再求发送占时，不能把照片总量直接除以一分钟当作利用率。

1.12 服务原语与对等协议如何配合

上层请求本层执行动作，常用请求原语；本层把事件告知上层，使用指示原语。对需要应答的服务，还可能有响应与证实。原语描述同机层间发生的服务事件，对等协议报文则在通信路径上传送。一次连接请求可能引起若干协议报文、超时或重试，不能把 “一次原语调用” 当成 “一次物理发送”。

面向连接服务通常经历建立、传送、释放三个阶段。无连接服务允许独立提交每个数据单元。无应答无连接、有应答无连接、面向连接是不同服务形式；可靠程度、流量控制和是否保留连接状态应分别检查。体系结构规定各层功能和相互关系，具体算法和内部数据结构则由实现选择。

1.13 主动回忆自测

以下为自拟回顾题。先遮住答案，按导图主枝口述条件，再进行数值计算。

1. 按工作方式划分的核心/边缘，与按功能划分的通信/资源子网是否完全同义?
2. 1 MB 文件经 10 Mb/s、单向传播 20 ms 的一条链路传送，不计首部和其他开销，最后一位何时到达?
3. 某帧从交换机端口 1 进入、端口 3 出去，能证明远端应用进程已收到数据吗?
4. 为什么协议描述的是水平关系，服务和接口描述的是垂直关系?
5. OSI 和 TCP/IP 的层数不同，能否据此断定其中一个模型没有应用层?

6. 报文切为 10 个等长分组，经 3 段等速链路存储转发；每组发送 1 ms、每段传播 2 ms，无其他开销，最后一组何时到达？
7. 虚电路先建立逻辑路径，能否直接推出必须为其预留一份固定带宽？
8. 收到帧后 CRC 通过，是否已经证明端到端数据无差错且不会丢失？
9. 按 $D = D_0/(1-U)$ 模型，利用率从 0.5 到 0.9，时延变为原来的几倍？独立误码模型下，帧错概率如何由比特误码率得到？
10. A 经 B、C 到 D，各段发送 2 ms、传播 1 ms，B 和 C 各处理 0.5 ms，共 4 个等长组、独立处理且无竞争，最后收齐时刻为何为 16 ms？

课程连接·原语与报文

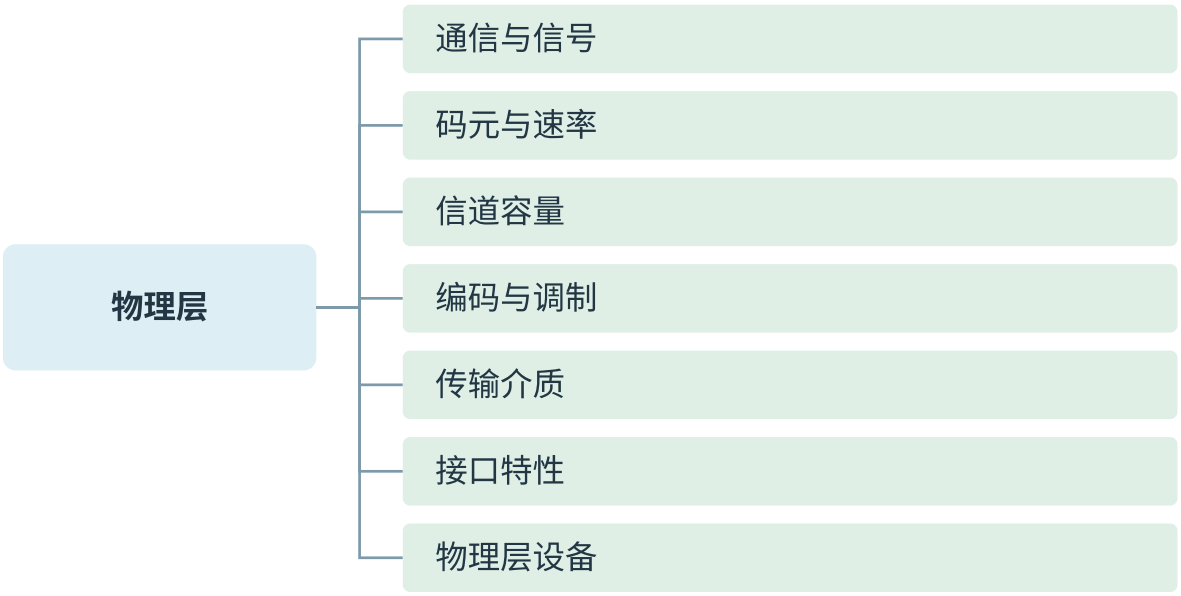
服务原语是同机层间事件；一次原语可引起多个对等协议报文。

1.13.1 核对答案

1. 不完全同义，两者分别强调位置/工作方式与承担功能。
2. 发送 0.8 s，传播 0.02 s，共 0.82 s；不是只算发送时间。
3. 不能，只能说明当前交换机完成了一次链路层转发，还需后续路径与目的端解封装。
4. 对等实体共同遵守协议；同机上层调用下层服务，接口约束调用方式。
5. 不能，TCP/IP 应用层合并了 OSI 上三层的相关任务。
6. $(10+3-1)\times 1 + 3\times 2 = 18$ ms；第一组 9 ms 到齐，之后每组间隔 1 ms。
7. 不能，虚电路仍可以按分组共享链路，资源保证取决于具体协议与服务。
8. 不能，检验存在漏检，链路检验也不能承诺后续路径可靠交付。
9. 从 $2D_0$ 变为 $10D_0$，是 5 倍；帧错概率为 $1-(1-p)^{L}$，近似 pL 须满足小误码条件。
10. 第一组 $3(2+1)+2(0.5) = 10$ ms，后续三组每隔 2 ms，共 16 ms。

物理层 2

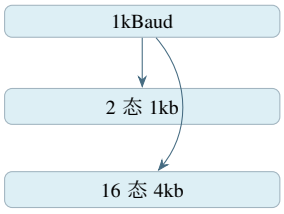
章节脉络



把“每秒传多少码元”换成“每秒携带多少比特”，必须知道一个码元有多少种状态；把“带宽多大”换成“容量上限”，还要区分带宽单位、噪声与调制约束。本章将这些量放回信号、介质与接口中理解。

最小反例·Baud 不等于 bit/s

同为 1000 Baud，二进制每码元 1 bit 得到 1 kb/s，16 态每码元 4 bit 得到 4 kb/s。两个系统码元率相同、数据率不同，已经足以排除“波特率总等于比特率”的说法。



同一码元率可以有不同比特率

2.1 数据、信号、码元与两种带宽

定义 2.1.1 (数据与信号)

数据承载信息，**信号**是数据在介质中的物理表示。数字数据可以表示为离散电平，也可以调制在连续载波上；模拟语音可以直接调制传输，也可以采样量化后编码为数字数据。因此不能把“模拟数据”与“模拟信号”当作同一概念。

码元是在一个固定时间间隔内选取的一种信号波形或状态。若有 M 种可区分状态，一次可表示 $\log_2 M$ bit；码元速率 R_s 的单位为 Baud，数据速率 R_b 的单位为 bit/s。在题目把每个状态按等长二进制编码、忽略其他冗余时，

$$R_b = R_s \log_2 M.$$

例如 1000 Baud 达到 4 kb/s，需要每码元 4 bit，即 $M = 2^4 = 16$ 种状态，不能把 4 误当成状态数。若题给“每 $1/8$ s 取一个可独立传送的 16 态符号”，则速率是 $8 \times 4 = 32$ bit/s；这里 8 已是每秒符号数，不必再乘 2。〖2.1-007〗〖2.1-016〗

频率带宽指可通过频率范围的上、下限之差，单位 Hz；**数据率意义的带宽**指单位时间能传送的数据量，单位 bit/s。看到“4 kHz 信道”应

想到容量定理，看到 “4 kb/s 链路” 通常已给出数据传输速率。两者单位不同，数值不能直接互换。

信道是某一方向上传送信号的通道。一条物理线路可以承载多个按频率、时间或编码区分的逻辑信道。单工只能单向传送，半双工可双向但不能同时，全双工可同时双向。两端之间有一根网线，不足以判断它采用哪一种工作方式。

错项排除·分贝不能直接入对数

30 dB 是功率比 1000 的对数表示，香农式需要无单位的 1000。如果直接把 30 填入 1+S/N，错误发生在输入量的含义，后续对数运算再准确也无法补救。

2.2 奈奎斯特与香农：哪个条件限制了速率

理想无噪声低通信道频率带宽为 B Hz 时，无码间串扰的极限码元速率是 $2B$ Baud；若每码元有 M 种状态，奈奎斯特式给出

$$R_b \le 2B \log_2 M.$$

带宽限制了可独立辨认的变化速度。把接收端采样次数增加到 $2B$ 以上，不会自动创造更多独立码元。〖2.1-010〗中的 8 kHz、8 态信道，即使写 “每秒采样 24k 次”，仍受 $2 \times 8000 \times 3 = 48$ kb/s 约束。

有加性高斯白噪声的带限信道，香农容量为

$$C = B \log_2(1 + S/N).$$

S/N 是无单位的功率比。若给出分贝数 γ，先计算 $S/N = 10^{\gamma/10}$。30 dB 对应 1000，不能把 30 直接代入对数。容量是理论上的可靠传输极限：编码合适、码长足够长时，实际速率可以趋近它，但任何具体调制器都不一定达到。

题目同时给出带宽、状态数和信噪比时，通常要检查固定调制状态数与噪声各自施加的限制，再按题目的教材信道模型取较小的上界；两式不是相加关系。若二进制信号在 4 kHz、信噪比 127 的信道上传送，固定二进制给出 8 kb/s，而香农给出 28 kb/s，能采用的上界为 8 kb/s。〖2.1-012〗

> **容量题的四步换算**
>
> 先把频率带宽换为 Hz、分贝换为功率比，再由状态数求每码元比特数；分别计算题设允许的奈氏与香农上界，最后选有效约束。只给码元率时直接乘 $\log_2 M$，不要无依据再乘 2。

已知信息	计算关系	条件与单位
每秒码元数与状态数	$R_b = R_s \log_2 M$	理想等长编码，不计额外冗余
无噪声低通带宽	$R_s \le 2B$	B 用 Hz，码元率用 Baud
带宽与功率比	$C = B \log_2(1 + S/N)$	高斯噪声模型，结果 bit/s
分贝信噪比	$S/N = 10^{\gamma/10}$	功率比，无单位

每个码元有 M 种可区分状态，连续 q 个码元共有 M^q 种序列；若想表示 k 位二进制信息，需要至少 2^k 种序列，故 $k \le q \log_2 M$。将每秒码元数乘进去，得到理想速率关系。状态数是指数空间，不能把 M 直接当每码元比特数。

2.2.1 例 2-1　由调制状态数求最大速率

例 2.2.1 (配套例题)

配套例题。在一条带宽为 200 kHz 的无噪声信道上，若采用 4 个幅值的 ASK 调制，则该信道的最大数据传输速率是（　）。

A. 200 kb/s　B. 400 kb/s　C. 800 kb/s　D. 1600 kb/s

解析。“无噪声、带宽、4 个幅值” 分别给出了奈氏模型、$B = 200000$ Hz 和 $M = 4$。每个码元携带 2 bit，极限码元率为 400 kBaud，因此最大数据率为 800 kb/s，选 **C**。把 200 kHz 直接当成 200 kb/s，会同时丢掉码元速率与每码元比特数两个转换。

2.2.2 例 2-2　增加信噪比是否一定提高固定调制的极限

例 2.2.2 (配套例题)

〖2.1-017〗某信道带宽为 10 MHz, 信噪比为 30 dB, 采用 QAM-32 调制。若带宽提高到 20 MHz，信噪比提高到 40 dB，则信道的极限数据传输速率大约提高到原来的（　）倍。

A. 2　B. 2.2　C. 2.4　D. 2.6

解析。QAM-32 每码元 5 bit。按题设模型，原奈氏上界为 100 Mb/s，原香农上界约为 $10\log_2(1001) = 99.67$ Mb/s；新奈氏上界为 200 Mb/s，新香农上界约为 $20\log_2(10001) = 265.76$ Mb/s。因此新旧上界的比值约为 $200/99.67 = 2.01$，最接近 **A**。

原条件下两个上界非常接近，按近似 $\log_2 1000 \approx 10$ 也得到约 2 倍。新增信噪比使噪声约束放宽，但固定 32 态调制仍限制每个码元只能携带 5 bit，所以不能仅用两个香农容量之比作答。

2.3 编码、调制与 PCM 怎样区分

基带编码用电平及跳变表示数字数据。非归零 NRZ 在一个比特间隔保持某一电平，连续相同比特可能没有跳变，时钟恢复困难。反向非归零 NRZI 用比特起点是否跳变表示数据，具体 0、1 约定要由题目给出；长串相同比特仍可能缺少跳变，常需填充等机制。

曼彻斯特编码在每个比特中点都跳变，以跳变方向表示数据。本书示意约定低到高为 0、高到低为 1；题目若采用相反约定，应按题意读图。差分曼彻斯特同样在每个比特中点跳变，但由**比特起点是否跳变**表示数据，本书约定起点跳变为 0、不跳变为 1。判断差分编码的第一位需要知道前一电平；不能只看该位中点的升降方向。

最小反例·中点不能解所有编码

差分曼彻斯特中每位中点都会跳变，只看中点方向无法按一般规则恢复比特；起点是否跳变才携带差分编码的信息。第一位还需要前一电平，这正是读波形时应补齐的初始条件。

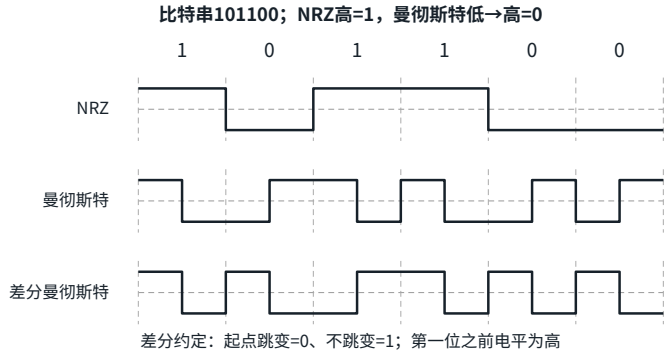


Figure 2.1: 同一比特串的三种基带表示；虚线划定比特间隔

曼彻斯特每位需要两个半位信号区间，因此在教材 “每秒最多可承载多少电平区间” 的模型中，数据率只有该区间变化能力的一半。不要因此认为比特中点的每次跳变都额外携带一位数据；中点跳变还承担时钟作用。

调制把数据映射到载波参数。ASK 改变振幅，FSK 改变频率，PSK 改变相位；QAM 联合使用振幅与相位区分状态。4 个相位、每个相位 4 种振幅共有 16 态，每码元 4 bit。QAM-64 表示 64 种可分辨组合，不是 64 个比特，也不是 64 个独立频率。

模拟数据转为数字数据常采用 PCM：先采样取得离散时刻的幅值，再量化为有限级别，最后将级别编码。对最高频率为 $f_{\max}$ 的理想带限信号，采样频率应不低于 $2f_{\max}$；若每样本编码 q bit，未计信道编码冗余的数据率为 $f_s q$。这里的 “采样至少两倍” 与前节 “码元至多两倍带宽” 分别限制采样与无码间串扰传输，不能一看到 “2” 就无条件相乘。

例如电话语音按 8 kHz 采样、每样本 8 bit，产生 64 kb/s 数据；若再用带冗余的线路编码，物理线路需要承担的编码后速率还会增加。4B/5B 用 5 个线路比特表示 4 个数据比特，编码效率为 4/5；这一效率与每码元的调制阶数是两个不同因素。

2.4 传输介质：比较受什么条件约束

双绞线由两条相互绝缘的导线绞合而成，以减少电磁干扰；屏蔽双绞线进一步增加屏蔽结构。同轴电缆用内导体、绝缘层和外导体屏蔽层构成，传统条件下具有较好的屏蔽性能。介质的实际速率同时取决于编码、设备、频段、长度等条件，不宜一概断言 “同轴一定比所有双绞线快”。

光纤利用光信号传播，绝缘且不受外界电磁干扰，衰减较低。多模光纤允许多种传播模式，不同模式到达时间不同，会产生模间色散；单模光纤减轻这类展宽，适用于更长距离和较高容量的通信。多模与单模的区别不只是 “同时发送几种颜色”；多种波长复用是另一种复用技术。

无线介质包括无线电、微波、红外等。低频无线电较易绕射，地面微波常要求视距传播，卫星通信能覆盖较大区域，但高轨卫星的长传播距离带来明显时延。不能把光纤 “速率高” 理解为其中信号的传播速度一定高于无线电波在真空中的速度。

常见以太网名称也需要读出限定：10BASE-T 采用双绞线；10BASE5 是传统粗同轴、单段上限 500 m 的制式；100BASE-T 主要使用双绞线。500 m 不是一切同轴网的共同极限。吉比特以太网不同物理制式使用不同编码，不能沿用传统 10 Mb/s 以太网的曼彻斯特编码概括全部制式。

错项排除·颜色和模式不是同义

单模/多模描述传播模式，不等于只用一种/多种波长。波分复用可以让不同波长共享光纤。比较光纤时先分清传播模式、复用方式和容量，避免拿一个维度直接替代另一个。

2.5 物理层接口规定什么，不规定什么

不变量·同是引脚也要分维度

引脚位于何处属于机械特性；引脚承担请求发送还是接收准备属于功能特性；电压范围属于电气特性。按“这个描述回答什么问题”归类，比按“引脚”这个共同名词归类更稳。

机械特性规定连接器形状、尺寸及接点排列等；电气特性规定电压、电平、阻抗与传输速率等；功能特性说明各条信号线承担什么功能；规程特性规定信号出现的顺序和双方动作次序。接点“位于哪里”属于机械特性，接点“用来请求发送”属于功能特性，不能都按“引脚”一个词归类。

2.5.1 例 2-5　MAC 地址属于物理层接口规范吗

例 2.5.1 (配套例题)

配套例题。下列选项中，不属于物理层接口规范定义范畴的是（　）。

A. 接口形状　B. 引脚功能　C. 物理地址　D. 信号电平

解析。形状、引脚功能和电平分别对应机械、功能与电气特性。这里的“物理地址”指 MAC 地址，是链路层寻址标识，并不因为名称中有“物理”就属于物理层接口规范，选 **C**。

2.6 中继器与集线器：再生比特，不解释地址

数字中继器在信号衰减、失真后再生符合规范的比特信号，重点是判决、整形和重新发送，不是把噪声和有用信号一起按幅度放大。它不检查 MAC 地址，也不学习转发表或进行网络层路由。可在满足速率、编码和协议等兼容条件时连接不同介质；“能换介质”不等于“能翻译不同网络协议”。

集线器是多端口中继器。一个端口收到比特后向其他端口转送，所有站仍共享一个冲突域、同一份总带宽。5 台主机接 10 Mb/s 集线器，在理想均分且不计竞争开销的模型下每台平均为 2 Mb/s；不是每台始终保证 2 Mb/s，也不是五台主机可以同时各以 10 Mb/s 发送。

一个集线器两侧各 100 m、传播速率 2×10^{8} m/s 时，最远站点间单向路径为 200 m，传播 1 μs，往返 2 μs。〖3.6-020〗的计算先合计两侧距离，再考虑往返；只把 100 m 乘 2 得到单向路径就停下，会漏掉回程那一段。中继设备若有额外时延也应计入，不能只看电缆长度。

2.7 从信源到信宿：编码器与调制器各做什么

信源产生待传的数据，**信宿**接收数据。数据经过变换器成为适合介质的信号，接收端反变换后恢复数据。数字数据可经线路编码成为数字基带信号，也可经调制映射到载波的幅度、频率或相位上。编码器与解码器配合，调制器与解调器配合；设备的商业名称不能代替所做的信号变换。

基带传输直接传送适合介质的基带信号；**频带传输**将信号频谱搬移到所需频带。所谓数字数据和模拟信号并不矛盾：数据的取值类型与承载它的波形类型属于两个维度。把数字数据映射为四种载波状态后，每个状态仍可表示 2 bit。

2.8 五种线路编码逐位怎样画

边界·第一位先看初态

差分编码要根据前一电平判断位起点有无跳变。

读图先画比特边界，再按约定画半位电平。下表采用 NRZ 低 0 高 1、NRZI 起点跳变表示 0、曼彻斯特中点上跳表示 0、差分曼彻斯特起点跳变表示 0；另设差分编码的初始电平为高。不同题图可以给出另一套合法约定，必须先确认再译码。

编码	一个比特内的规则	同步与代价
NRZ 不归零	整个位保持相应电平	长串相同比特缺少边沿，自同步困难
RZ 归零	前半位按 0/1 给低/高电平，后半位回到零电平	每位有回零过程，占用更多信号变化能力
NRZI 反向非归零	位起点按数据决定是否跳变，位中保持	按本表约定，长串 1 可能无跳变，可结合限制连续码型等方法
曼彻斯特	位中必跳变，以跳变方向表意	中点提供同步；每位使用两个半位区间
差分曼彻斯特	位中必跳变，以位起点是否跳变表意	依赖相邻电平关系，读首位需给初始状态

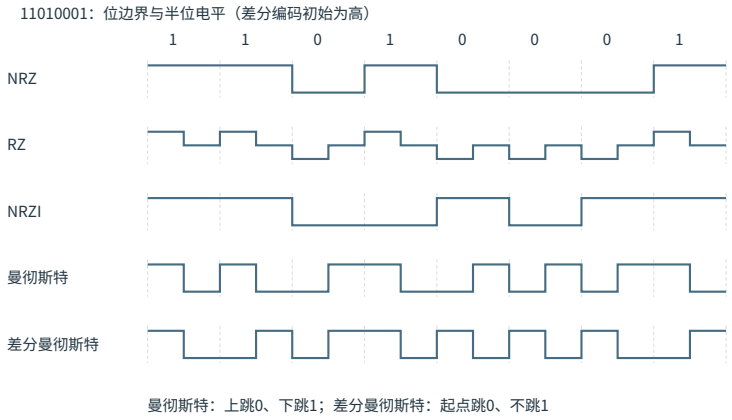


Figure 2.2: 比特串 11010001 的五种线路编码；RZ 的中线表示零电平

以 `11010001` 为例，差分曼彻斯特在各位起点的判断依次是 “不跳、不跳、跳、不跳、跳、跳、跳、不跳”。曼彻斯特则直接由各位中点的跳变方向恢复数据。两者都具有中点跳变，单凭这一特点不能判断编码类型。

若题目给定 “每秒至多提供 $2R$ 个半位信号区间”，曼彻斯特数据率至多 R bit/s；若给的已是最终比特率，则直接使用该数，不再减半。自同步能力、编码冗余、抗干扰能力分别描述不同性质，不能由某一项自动推得其他项。

2.9 容量与调制的两个完整计算

带噪声条件。频率带宽为 8 kHz、信噪比 30 dB，先换算 $S/N = 10^3 = 1000$，香农容量为

$$C = 8000 \log_2(1001) \approx 79737.8 \text{ bit/s}.$$

若实际达到该上限的 50%，则实际数据率约 39868.9 bit/s，按题目

近似为 40 kb/s。不能把 30 代入 $\log_2(1+S/N)$，也不能把 “实际达到 50%” 再解释为编码状态数。

课程连接·容量不等于速度

提高容量不等于提高传播速度；距离和介质速度决定传播时延。

无噪声条件。频率带宽为 3 kHz，调制使用 4 种幅度、4 种相位，并且这些组合互不重合，故共有 16 态，每码元 4 bit。奈氏极限码元率为 6000 Baud，最大数据率为 $6000 \times 4 = 24000$ bit/s。若某些幅度和相位的组合实际上给出同一波形，状态数必须按可区分的实际点数计算，不能只做形式上的乘法。

2.10 介质和设备的边界条件

传输介质本身位于物理层之下，物理层规定如何在介质上产生和识别信号。导向介质把能量约束在电缆或光纤中；非导向介质通过自由空间传送电磁波。频率和波长满足 $v = f\lambda$，真空时 $v = c$；某种介质能用多宽的频带，还取决于衰减、色散和设备能力。

单模光纤的 “单模” 指只允许一个空间传播模式，并不是只准传一个比特、一个频道或一种波长。多模的模间色散会展宽光脉冲，限制距离和速率；单模、波分复用及多芯结构可以分别用于不同目的。外皮颜色是工程习惯，不能代替实际型号参数。

传统 10BASE5、10BASE2、10BASE-T 中的数字和后缀应按具体标准解释：10 表示 10 Mb/s，BASE 表示基带，5 和 2 关联传统同轴段的长度等级，T 表示双绞线。10BASE2 常取实际最大 185 m。具体编码、距离和双工能力依标准确定；高代际以太网采用的编码不能由 “BASE” 一个词推出。

中继器按规定电平判决并再生信号，无法修复已经判错的比特。集线器扩展同一共享介质，站点仍共用一个冲突域。对于 8 台主机共享 10 Mb/s 集线器，理想均分为 1.25 Mb/s；它只是带宽分配模型，实际竞争与协议开销会降低有效吞吐量。

历史 10BASE5 网络中的 “5-4-3” 规则指最多 5 个网段、4 个中继设备，其中 3 个网段可挂接终端，用于约束传播和中继延迟。应用到其他物理标准前，应重新检查其距离、时延和连接规范。

2.11 主动回忆自测

以下为自拟题，先标出单位和模型条件再作答。

1. 每秒 2000 个码元、每码元 8 态，忽略冗余，数据率是多少？改为 16 态而码元率不变呢？
2. 频率带宽 1 MHz、信噪比 30 dB，香农式应代入 30 还是 1000？
3. 2 kHz 无噪声低通信道采用 16 态，理想最大码元率和比特率分别为多少？
4. 曼彻斯特的中点跳变与差分曼彻斯特的起点跳变各表示什么？
5. 最高频率 4 kHz 的带限模拟信号每次采样编码 8 bit，按最低理想采样率得到多少 bit/s？
6. 连接器形状、某引脚用于请求发送、信号电平和信号出现次序分别属于哪类接口特性？
7. 五台主机连接 10 Mb/s 集线器，是否能同时各发 10 Mb/s？
8. 光纤提高传输容量，是否必然使同距离上的传播时延比真空无线传播更短？
9. 归零编码、NRZI 与差分曼彻斯特各在哪些位置允许或要求跳变？
10. 8 kHz、30 dB 信道的香农容量约多少？若只达到 50% 呢？

2.11.1 核对答案

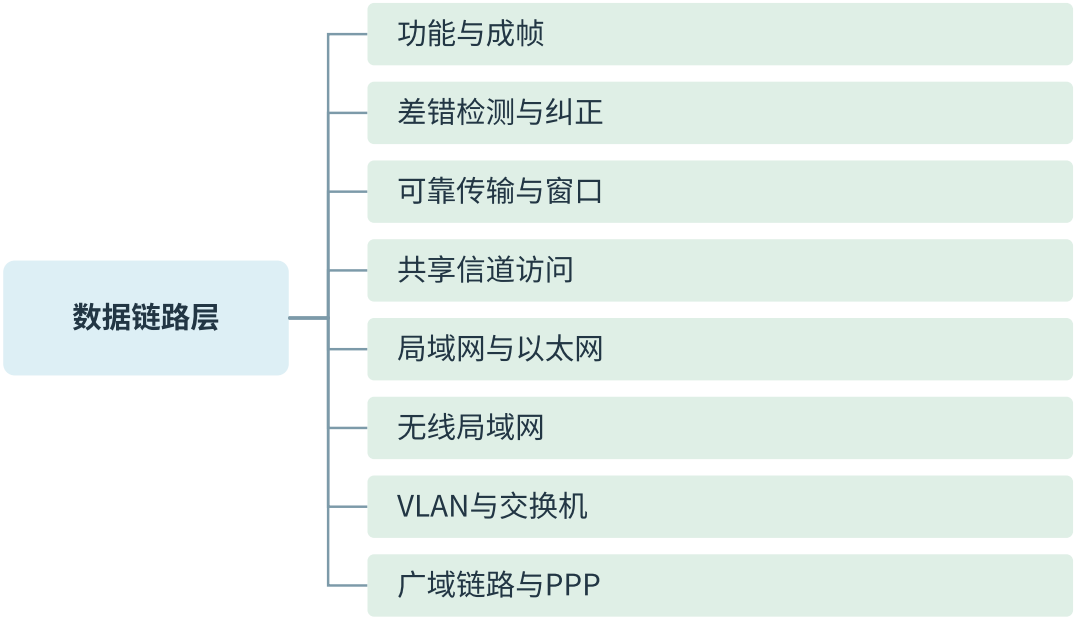
课程连接·容量不等于速度

提高容量不等于提高传播速度；距离和介质速度决定传播时延。

1. 分别 6 kb/s 和 8 kb/s，按码元率乘每码元信息量。
2. 1000，容量为 $10^6 \log_2(1001)$ bit/s。
3. 4000 Baud 与 16 kb/s；必须保留题设低通无噪声模型。
4. 曼彻斯特中点跳变承担同步并由方向表意；差分曼彻斯特中点仍用于同步，起点是否跳变表意，具体 0/1 约定按题目。
5. 最低理想采样 8000 次/s，所得 64 kb/s，未计其他冗余。
6. 依次为机械、功能、电气、规程特性。
7. 不能，它们共享总带宽和冲突域；理想均分 2 Mb/s 只是特定平均模型。
8. 不必然，容量与传播速度是不同量；传播时延仍按介质中的速度和距离计算。
9. RZ 每位后半回零；NRZI 按约定位起点跳或不跳、位中保持；差分曼彻斯特位中必跳，位起点按数据决定是否跳变。
10. 约 79.74 kb/s 和 39.87 kb/s，题设近似可写 80 与 40 kb/s。

数据链路层 3

章节脉络



数据链路层先解决“这一串比特属于哪一帧、有没有检测出错误”，再按协议决定是否确认和重传。若多个节点共用介质，还必须决定谁现在可以发送。这三类问题不能互相替代：CRC 不负责仲裁信道，介质访问成功也不保证帧已经可靠交付。

本章帧序号按模 2^k 循环。除题干另有约定，`ACK i` 表示“已正确收到 i 号帧”；TCP 的确认号采用“下一期待字节”，将在第 5 章单独说明。

课程连接·链路反馈只看一跳

操作系统中的缓冲区保护解决本机生产者与消费者速度不匹配；链路流控也限制发送量以保护接收缓存。但链路 ACK 只能覆盖当前相邻节点，不能像端到端协议一样替后续整条路径承诺可靠。

3.1 一跳链路提供什么功能

物理链路提供比特传输，数据链路在其上增加协议控制。基本任务包括成帧与透明传输、差错检测；具体协议还可能提供流量控制、确认重传、连接管理或共享介质访问控制。点到点链路不需要在多个竞争站之间分配介质，不能因此说它没有数据链路层。

流量控制协调发送速度与接收能力，防止接收缓存被压满；**差错控制**处理损坏、丢失、重复和次序问题。反馈窗口可以同时限制在途数据，既影响接收缓存需求，也影响重传管理。但只把发送器调慢一些，并不构成随接收状态变化的反馈控制。〖3.4-018〗

判断某项功能属于哪层，还要看范围。链路层确认只说明当前相邻节点的交付情况；路由器收到一帧后，下一跳仍可能丢失。端到端可靠性不能只由某一跳的 ACK 推出。

3.2 怎样从连续比特中划出帧

定义 3.2.1 (帧边界与透明传输)

帧定界使接收方找到首尾，**透明传输**使任何合法上层数据都不会被误认为定界信息。主要方法的区别如下。

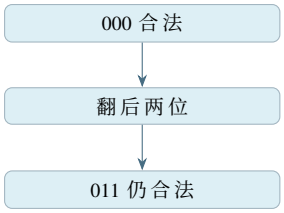
方法	接收方怎样定位边界	必须处理的问题
字符计数	读首部长度，再取相应字节	长度字段损坏可能使后续边界也错位
字节填充	识别特殊定界字节	数据中的定界字节和转义字节都要转义
零比特填充	识别 `01111110` 等标志	数据中每连续 5 个 1 后插 0，避免形成标志
违规编码	利用数据编码中不用的信号组合	依赖具体物理编码中确有可保留的组合

字节填充必须按协议的转义规则执行。例如把数据中的 `FLAG` 写成 `ESC FLAG`、`ESC` 写成 `ESC ESC`，接收方遇到 `ESC` 时把后一个字节当作数据。若只转义 `FLAG`，原有 `ESC` 就会改变后续字节的解释。PPP 的具体异步字节转义还会改变被转义字节的数值，不能把所有字节填充都当成这一种简化写法。

零比特填充只扫描应填充的数据与控制区，不把首尾标志本身再填充。数据 `01111110` 中第五个连续 1 之后插 0，成为 `011111010`；去填充时删掉这个插入 0，恢复原数据。遇到恰好 5 个 1，随后原数据还有一个 0，线上会出现两个 0，接收方只能删去填入的那一个。

最小反例·合法码字之间仍能跳转

取生成式 11，000 与 011 均能被它整除。合法串 000 若恰好翻转后两位成为 011，检验余数仍为 0。这个两位翻转例子直接排除 “CRC 通过就证明没有任何错误” 的绝对断言。



生成式 11 下，错误仍可能通过检验

3.2.1 例 3-1　填充按连续 1 计数，不按字节重置

例 3.2.1 (配套例题)

配套例题。HDLC 协议对 `01111100 01111110` 组帧后对应的比特串为（　）。

A. `01111100 00111110 10`

B. `01111100 01111101 01111110`

C. `01111100 01111101 0`

D. `01111100 01111110 01111101`

解析。本题考查数据区的零比特填充；首尾标志不计入选项。原数据第一段中的 5 个连续 1 后插 0，第二段的前 5 个 1 后也插 0。连接原数据并逐位扫描，结果为 `011111000011111010`；按选项中的空格分组，所得结果与 A 相同，选 **A**。

空格只帮助阅读，并不表示连续 1 的计数在这里清零。本题借用 HDLC 背景说明填充机制，不要求另记该协议的完整控制字段。

3.3 奇偶校验与 CRC：检验通过能推出什么

奇偶校验在数据后增加一位，使整个码字中 1 的个数满足奇数或偶数约定。任意奇数位翻转都会改变奇偶性，所以能够检测；偶数位翻转可能仍满足原奇偶性，无法保证发现。一个检验位不能指出是哪一位翻转，因此不具备单比特纠错能力。

循环冗余检验 CRC 把比特串视为二元多项式。生成多项式 $G(x)$ 的次数为 r，对应 $r+1$ 位比特。发送数据 D 先左移 r 位，再按模 2 除法除以 G，余数 R 固定写成 r 位，发送 $D\,R$。模 2 减法就是逐位异或，不产生借位；最高位为 0 时也是如此。

以王道书 §3.3 算例的数据 `101001`、生成式 `1101` 为例，$r=3$，应计算 `101001000` 除以 `1101`。每次把除数首位对准当前最高 1，异或后得到以下中间数：

```
101001000
011101000  // 对齐110100000并异或
000111000  // 对齐011010000并异或
000001100  // 对齐000110100并异或
000000001  // 对齐000001101并异或
```

余数为 `001`，发送串为 `101001001`。接收方直接对整个接收串做同一除法，不再额外补 3 个 0。余数非零，说明一定检测到了错误；余数为零，只说明收到的码字满足该生成式，不能从数学上排除所有差错。

CRC 的发送端与接收端分开算

先按生成多项式次数确定冗余长度 r；发送端给数据补 r 个 0 后求余，将 r 位余数附在原数据后。接收端对收到的完整码字直接求余，不再次补 0。两端都做模 2 除法，任何一位的减法都用异或。

3.3.1 例 3-2　找出满足生成式的接收串

例 3.3.1 (配套例题)

配套例题。若甲向乙发送数据时采用 CRC 校验，生成多项式为 $G(X)=X^4+X+1$（即 $G=10011$），则乙接收到下列比特串时，可以断定其在传输过程中未发生错误的是（　）。

A. `1 0111 0000`　B. `1 0111 0100`

C. `1 0111 1000`　D. `1 0111 1100`

解析。依题目的检验语义，应选择模 2 除以 `10011` 后余数为零的接收串。四个余数依次为 `1100`、`1000`、`0100`、`0000`，故选 **D**。也可看出四项前 5 位同为数据 `10111`，将其补 4 个 0 后求余得 `1100`，应附加该 FCS。

题干的 “可以断定未发生错误” 按考试语境解释为 “CRC 检验通过”。严格说，若错误多项式恰被 G 整除，检验也会通过。比如从合法码字中异或一个非零的 G 倍式，得到的仍是合法码字。作答时选择检验通过的项，理解时保留漏检边界，不能把有限冗余当成绝对无差错的证明。

CRC 的保证取决于生成式。最高、最低系数为 1 且次数为 r 时，能检测所有长度不超过 r 的突发错误；若还含因子 $x+1$，能检测所有奇数位错误。更长突发错误的漏检概率，需要额外的随机误差模型才能讨论；不能把 2^{-r} 当作每一种具体错误的确定发生率。

两个不同位置出错的错误多项式可写为 $x^j(x^k+1)$。当生成式常数项为 1 时，是否漏检取决于 $G(x)$ 是否整除 x^k+1；要保证给定码长内所有双比特错可检出，须对该码长允许的位间距逐一满足不整除条件，不能把一种生成式的保证推广到任意码长。

CRC 在特定码长、合适生成式及译码规则下也可以用于单错纠

不变量·填充只删自己插入的位

数据中五个连续 1 之后插入的 0 在接收端删除，原始数据的 0 必须留下。遇到数据 111110 时线上会出现 1111100；如果把两个 0 都删掉，成帧机制反而破坏了上层数据。

正：为每个可能的单错位置建立余数表，只有各位置的非零余数互不相同，才能由接收余数唯一定位。K 个信息位、r 个检验位至少需要 $2^r \geq K + r + 1$ 个综合征状态，这只是计数上的必要条件，还需检查生成式和码长确实使单错余数互异。实际链路通常用 CRC 检错，再由重传恢复。

不变量·综合征须固定编号

位置 1、2、4 的检验位与书写方向是两套约定。先给每位编号，再按二进制位置选校验集合；单错时综合征才指向位置。若有双错，直接把综合征当单错位置翻转可能产生新的错误。

3.4 码距与海明码：怎样从检验位定位单错

两个等长码字之间不同的比特数称为**海明距离**，合法码字间的最小距离为 $d_{\min}$。要保证检测不超过 e 位错误，需要 $d_{\min} \geq e+1$；要保证纠正不超过 t 位错误，需要 $d_{\min} \geq 2t+1$。检测只要求错误后的串不是另一个合法码字；纠正还要求它不会同时落在两个合法码字的可纠正范围内。

定理 3.4.1 (最小码距的检测与纠正界)

若合法码字最小距离为 $d_{\min}$，保证检出至多 e 位错要求 $d_{\min} \geq e+1$；保证纠正至多 q 位错要求 $d_{\min} \geq 2q+1$。

检错只要求从一个合法码字翻转至多 e 位后，不能恰好走到另一个合法码字。若要纠正 q 位错，则每个合法码字周围半径 q 的错误集合必须互不相交；否则同一接收串会同时像两个原码字。两球不相交需要码字间距离大于 $2q$，由此得到纠正界。

3.4.1 例 3-3　先比较所有码字，再谈纠错

例 3.4.1 (配套例题)

配套例题。若某差错编码的编码集为 `{1001 1010, 0101 1100, 1111 0000, 0000 1111}`，则该差错编码的检错和纠错能力是（　）。

A. 不超过 2 位错的 100% 检错、不超过 1 位错的纠错

B. 不超过 2 位错的 100% 检错、不超过 2 位错的纠错

C. 不超过 3 位错的 100% 检错、不超过 1 位错的纠错

D. 不超过 3 位错的 100% 检错、不超过 2 位错的纠错

解析。四个码字共有 6 对，海明距离依次为 4、4、4、4、4、8，最小值为 4。故保证检测 3 位错、纠正 1 位错，选 **C**。不能只比较第一对就假定已找到最小值；“有 4 个码字” 也不等于 “码距为 4”。

普通海明码把检验位置于编号 1、2、4、8……，数据放其余位置。每一检验位负责校验一组位置，这些位置的编号在该检验位编号对应的二进制位上为 1。例如 7 位码中，位置 1 检查 1、3、5、7，位置 2 检查 2、3、6、7，位置 4 检查 4、5、6、7。若采用偶校验，三组检查所得综合征写为 $s_4 s_2 s_1$；单错假设下，该数正是出错位置。

第 1 至 7 位中，数据位 3、5、6、7 依次填 `1`、`0`、`1`、`0`，得到检验位 $p_1 = 1, p_2 = 0, p_4 = 1$，码字按位置递增写为 `1011010`。位置 6 翻转后收到 `1011000`，检查结果为 $110_2 = 6$，翻转第 6 位即可恢复。若书中按位置 7 至 1 展示，字符串顺序会反过来；**位置编号和书写方向必须先固定**。

m 个数据位需要 r 个单错检验位时，至少满足 $2^r \geq m + r + 1$，右侧包括每个可能出错位置及 “无错”。普通海明码通常码距 3，可纠正单错；要兼具单错纠正、双错检测的明确判别，需增加总体奇偶位等扩展，不能用单错综合征无条件纠正双错。

3.5 反馈、计时器与窗口分别解决什么

最小反例·没收到 ACK 也可能交付过

数据 0 已被接收并交付，返回 ACK0 却丢失。发送方超时重发 0，接收方必须重复确认而不重复交付。若没有序号，接收端就无法区分这次重传与内容恰好相同的新数据。

可靠传输至少要回答三个问题：损坏或丢失怎样发现，重复到达怎样识别，哪些发送数据还须保留。检错负责发现部分位错；确认表达接收进度；超时使发送方在没有反馈时重试；序号帮助接收方识别新数据与重传。确认也可能丢失，故发送方超时不等于接收方一定没收到原帧。

发送窗口覆盖当前允许处于“已发未确认”或“可发送”状态的序号范围；**接收窗口**覆盖当前允许接收并保留的序号范围。窗口前移必须满足相应确认或按序交付条件。更高序号的 ACK 到来时，窗口能否越过中间帧，取决于该协议采用累计确认还是独立确认。

在没有差错、确认处理无延迟的单流模型中，发送窗口越大，越有可能覆盖往返反馈期间的等待空隙。若确认帧长度不能忽略，周期应包含它的发送时间；传播时延要往返各计一次，共一个 RTT。不能把“窗口大”直接等同于“必达 100% 利用率”，链路竞争、错误和接收能力仍可能限制吞吐量。

3.6 停止—等待：为什么序号至少有 0 和 1

停止—等待每次只允许一个未确认数据帧。发送方发出帧后启动计时器，收到期望确认则发送下一帧，超时则重发原帧。接收方若第一次收到正确的新帧就交付并确认；若因 ACK 丢失再次收到同号帧，应重复确认但不重复交付。

0、1 交替的序号用来区分“刚收到的重传”与“真正的下一帧”。若完全不编号，接收方收到同样内容时无法判断是重传还是应用确实又发了一份相同数据。这里默认旧帧不会存活到序号重复使用后仍造成歧义，分析实际协议还要考虑寿命及序号空间。

设数据发送时间为 $T_d = L_d/R$，ACK 发送时间为 $T_a = L_a/R$，单向传播时延为 τ，忽略处理时间，往返一次周期为 $T_d + 2\tau + T_a$。发送方用于发送数据帧的信道利用率为

$$U = \frac{T_d}{T_d + 2\tau + T_a}.$$

3.6.1 例 3-4　长确认不能从周期中消失

例 3.6.1 (配套例题)

配套例题。假设主机甲采用停—等协议向主机乙发送数据帧，数据帧长与确认帧长均为 1000 B，数据传输速率是 10 kb/s，单向传播延时是 200 ms。则甲的最大信道利用率为（　）。

A. 80%　　B. 66.7%　　C. 44.4%　　D. 40%

解析。数据帧和确认帧都需要 $8000/10000 = 0.8$ s 发送，往返传播为 0.4 s。一个周期共 $0.8 + 0.4 + 0.8 = 2$ s，甲发送数据占 0.8 s，所以 $U = 40\%$，选 **D**。若忽略确认帧，会误得 66.7%。本题的利用率只计甲发送数据占用的时间；不能把返回的确认也放进分子当作有效数据。

3.7 GBN：累计确认越过了哪些帧

错项排除·累计确认覆盖缺失的 ACK

GBN 收到 ACK3 通常已证明本轮 0 至 3 全部按序接收，即使 ACK1 没有单独到达。重传集合由累计左沿决定，不能按 “哪些 ACK 没看见” 逐个列出；换成 SR 独立确认时结论又会改变。

后退 N 帧协议 GBN 允许发送方连续发送多帧，接收方只按序接收，接收窗口为 1。若期望 2 却收到 3，通常丢弃 3，并重复确认已按序收到的最后一帧。对 i 号帧的累计确认说明 0 至 i 在本轮序列中都已按序收到。

部分教材题把 “连续 ARQ” 用来专指 GBN，此时 “接收方可以不按序接收” 违背本题模型；不能再把这个限制推广到允许失序缓存的 SR。〖3.4-008〗

发送方通常为最早未确认帧维护计时器；该计时器超时后，重发从该帧到最后已发帧的整个未确认区间。某些后续帧可能曾经到达过接收方，但若当时失序被丢弃，仍须重发。帧编号 k bit 时应满足 $W_s \le 2^k - 1$，否则窗口覆盖全部序号后，旧帧与新一轮同号帧会产生歧义。

3.7.1 例 3-5　收到哪些确认，不等于确认了哪些帧

例 3.7.1 (配套例题)

配套例题。数据链路层采用后退帧（GBN）协议，发送方已经发送了编号为 0～7 的帧。当计时器超时时，若发送方只收到 0、2、3 号帧的确认，则发送方需要重发的帧数是（　）。

A. 2　B. 3　C. 4　D. 5

解析。ACK3 是累计确认，已覆盖 0、1、2、3；ACK1 虽未单独到达，1 号帧也不需要重发。未确认区间为 4、5、6、7，共 4 帧，选 **C**。若换成独立确认的 SR，同样的 ACK 集合不能证明 1 已确认，重发集合就要重新按各帧状态确定。〖3.4-010〗与此同一模型，只是把末帧和最大确认号换为 6、4，应使用同一判断方法。

3.7.2 例 3-6　先求填满链路的窗口，再求序号位数

例 3.7.2 (配套例题)

〖3.4-019〗假设两台主机之间采用后退 N 帧协议传输数据，数据传输速率为 16 kb/s，单向传播时延为 250 ms，数据帧长度为 128 B，确认帧长度也是 128 B。为使信道利用率达到最高，帧序号的比特数至少为（　）。

A. 2　B. 3　C. 4　D. 5

解析。$T_d = T_a = 128 \times 8/16000 = 64$ ms。第一帧开始发送至其 ACK 到齐需 $64 + 500 + 64 = 628$ ms。为了在第一个 ACK 到来前不断发送，需

$$W_s T_d \ge 628 \text{ ms}, \qquad W_s \ge \lceil 628/64 \rceil = 10.$$

GBN 还要求 $W_s \le 2^k - 1$。3 bit 最多窗口 7，不够；4 bit 最多 15，足够，选 **C**。窗口单位是帧数，求得 10 后不能直接答成 10 bit。以上采用全双工、每帧立即确认、无额外开销的题设模型。

3.8 SR：缓存失序帧为何仍不能立即滑窗

最小反例·缓存不等于越过缺口

接收窗口允许 2 和 3，3 先到只能缓存，左沿仍停在 2；2 到达后才一起交付 2、3。画出这两个时刻就能看出 “收到高序号帧后立刻把窗口移到它后面” 为什么会丢掉尚未收到的数据。

选择重传 SR 对各帧分别确认、分别计时，超时只重发相应帧。接收方可以缓存接收窗口内正确到达的失序帧，但仍须等缺口补齐后，才连续按序交付并移动窗口左沿。收到一个高序号帧与已经完成连续交付是两件事。

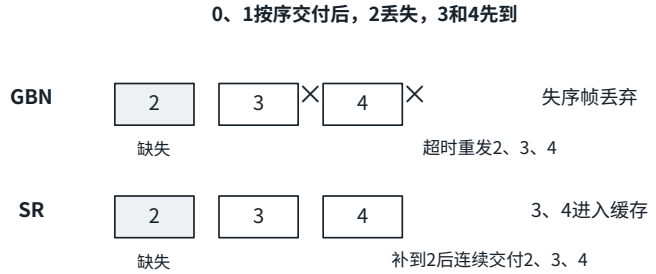


Figure 3.1: 2 号帧丢失后，GBN 丢弃后续失序帧，SR 缓存后等待补齐

例如发送窗口与接收窗口均为 4，编号空间为 0～7。0～3 发出后，0、1、3 已确认而 2 未确认，发送窗口左沿只能到 2，可覆盖 2～5；其中 3 已确认，4、5 可新发。接收端若已缓存 3，2 到达后可连续交付 2、3，才把接收窗口推进到 4。SR 不会因为 ACK3 出现就把 ACK2 也当作已收到。

为避免接收窗口把旧一轮重传误当成新帧，需 $W_s + W_r \le 2^k$；常见对称窗口为 $W_s = W_r \le 2^{k-1}$。例如 5 位序号、对称最大窗口为 16，不是 31；4 位序号且 $W_r = 7$ 时，仅由一般不重叠条件得到 $W_s \le 9$，不能无条件用 “半个序号空间” 代替不对称条件。王道书 §3.4 中这两类设问应分别判断。

主书与课件还对常用配置采用 $W_r \le W_s$，以免接收窗口大于发送窗口时，多出的部分没有对应新帧、白白占用窗口额度。题目若采用这套教材配置，应同时满足不重叠条件；不能只验证两窗之和就断言任意配置都符合该题模型。上例取 $W_s = 9, W_r = 7$ 同时满足这两项，最大对称配置则两窗都取 8。

序号空间相同，不意味着 SR 的最大利用率一定超过 GBN。3 位序号时 GBN 最大发送窗口 7，对称 SR 为 4；无差错时若只有窗口约束，GBN 可能更容易填满链路。SR 的优势在于有差错时能保留已正确收到的失序帧，减少重传；并非任何条件下发送窗口都更大。

把三种 ARQ 放在同一组条件下比较，关键是确认覆盖范围、失序缓存和超时后的动作。以下沿用本章 “ACK i 确认已收到 i 号帧” 的约定，序号共 k bit。

比较维度	停止—等待	GBN	SR
发送/接收窗口	$W_s = W_r = 1$	$W_r = 1$，$W_s \le 2^k - 1$；取 1 时退化为停等	$W_s + W_r \le 2^k$；本书常用 $W_r \le W_s$；对称时各不超过 2^{k-1}
确认的含义	确认当前帧	累计确认按序收到的帧	分别确认每个正确接收帧
缺口后的正确帧	尚未发送下一新帧	失序到达通常丢弃	在接收窗口内缓存，补齐缺口后连续交付
超时重传范围	当前未确认帧	从最早未确认帧起的全部已发未确认帧	相应超时未确认帧
典型计时与接收存储	一个未确认帧的计时	通常计时最早未确认帧；不缓存后续失序帧	各未确认帧分别计时；须存储失序帧

窗口都以**帧数**计，不能直接代入 TCP 的字节窗口。停等牺牲流水换取简单状态；GBN 用累计确认和简单接收端降低管理开销；SR 增加缓存与计时状态以减少差错后的重复传送。无差错、小窗口、长传播或高误码等条件改变时，比较结论也会改变。

课程连接·正交像向量基的分解

理想 CDMA 中，与目标码正交的其他用户贡献在内积中为 0；目标码与自身内积为码长。恢复的是目标发送者的数据，接收站名称不会改变要选哪条码；同步与功率假设也是消去干扰的条件。

3.9 频分、时分、波分与码分怎样共享介质

频分多路复用 FDM 把频带划成互不重叠的子带，多个用户可同时发送；时分 TDM 把时间划成重复时隙，各用户按时隙使用；波分 WDM 以不同光波长承载不同信号，可视为光频域复用。同步 TDM 分到的时隙在用户无数据时也可能空着；统计时分按实际有数据的用户动态分配，需要说明数据归属和排队规则。

码分 CDMA 用不同码片序列区分用户。按本节理想同步、等功率、正交模型，把比特 1 映射为本用户码片 c，比特 0 映射为 $-c$，未发送为 0。多用户信号按码片位置相加；接收者对目标码 c 做内积并除以码长 m。正交的其他用户内积为 0，目标值为 +1、-1 或 0，分别表示 1、0 或未发送。该结论依赖码序列、同步及功率假设；把任意非正交码直接代入，不能指望干扰仍完全消失。

3.9.1 例 3-7　CDMA 先按码片分组，再做内积

例 3.9.1 (配套例题)

配套例题。站点 A、B、C 通过 CDMA 共享链路，A、B、C 的码片序列分别是 `(1,1,1,1)`、`(1,-1,1,-1)` 和 `(1,1,-1,-1)`。若 C 从链路上收到的序列是 `(2,0,2,0,0,-2,0,-2,0,2,0,2)`，则 C 收到 A 发送的数据是（　）。

A. 000　B. 101　C. 110　D. 111

解析。问的是 A 的数据，使用 A 的码，不因接收站是 C 就改用 C 的码。按每 4 个码片分成三组，与 `(1,1,1,1)` 内积后除 4，依次为 $4/4=1$、$-4/4=-1$、$4/4=1$，恢复为 101，选 **B**。若不先按码长分组，会把三个比特混成一次判决。

3.10 ALOHA 与 CSMA：监听为何不能消除冲突

纯 ALOHA 允许有数据就发送，冲突后随机重发。若一帧发送时间为 T，另一帧只要在它开始前后各 T 的区间内开始发送，就可能重叠，易受冲突的时间跨度为 $2T$。时隙 ALOHA 只在统一时隙边界开始，冲突区间缩为 T。经典泊松负载、等长帧模型下，吞吐量分别为 Ge^{-2G} 与 Ge^{-G}，最大值约为 $1/(2e)$ 与 $1/e$；脱离该随机模型，不把这些数值当成任何网络固定可得的效率。

CSMA 在发送前监听载波。1-坚持遇忙持续监听、空闲立即发送；非坚持遇忙随机等一段时间再监听；p-坚持适用于时隙信道，空闲时以概率 p 发送，以 $1-p$ 延后一个时隙再判断。坚持策略描述的是“如何等到机会”，不代表先到者必获公平顺序。

监听只能看到已经传播到本地的信号。A 刚开始发出信号，远端 B 可能仍检测为空闲并开始发送，所以任何这类 CSMA 都不能仅靠“先听”消除传播期间的冲突。遇忙时的动作以及发生冲突后的动作，要分开读题。

3.11 CSMA/CD：最短帧与网络跨度的关系

不变量·最短帧要覆盖往返

最远站点开始发送的时刻可以恰在本端信号到达前，冲突反馈还要传播回来。最短发送时间约束的是两倍单向传播时间；把最短帧缩短 800 bit 时，距离变化必须再除以往返系数 2。

共享半双工以太网采用载波监听、多路访问与冲突检测。站点一边发送一边检测；发现冲突后停止正常帧的发送，改发强化冲突信号，再随机退避。全双工点到点交换以太网无需争用共享介质，不使用 CSMA/CD；不能把 “所有以太网必采用 CSMA/CD” 作为前提。

设最远两站单向传播时间为 τ。最不利时，远端在 A 信号将到前开始发送，冲突的影响再传回 A，接近 2τ 后 A 才察觉。为保证 A 还在发送时能检测，必须有

$$\frac{L_{\min}}{R} \geq 2\tau, \qquad L_{\min} \geq 2R\tau.$$

若还包含中继器延时，应计入最坏往返路径。提高速率而保留帧长，会减少可允许的传播时间；不是速率越高、可允许的共享网跨度就越大。

3.11.1 例 3-8　最短帧缩短后要缩短多少电缆

例 3.11.1 (配套例题)

配套例题。在一个采用 CSMA/CD 协议的网络中，传输介质是一根完整的电缆，传输速率为 1 Gb/s，电缆中的信号传播速度为 200000 km/s。若最小数据帧长度减少 800 bit，则最远的两个站点之间的距离至少需要（　）。

A. 增加 160 m　　B. 增加 80 m　　C. 减少 160 m　　D. 减少 80 m

解析。最短发送时间减少 $800/10^9 = 0.8\ \mu s$，这一时间约束的是往返传播时延；允许的单向传播时间减少 0.4 μs。距离应至少减少 $2 \times 10^8 \times 0.4 \times 10^{-6} = 80$ m，选 **D**。漏掉往返关系会误得 160 m。

截断二进制指数退避在第 n 次冲突后，令 $k = \min(n, 10)$，从整数 0 至 $2^k - 1$ 中随机取 r，等待 r 个争用时间片后重新竞争。10 Mb/s 以太网争用时间片为 51.2 μs；连续 4 次冲突，最大退避为 $15 \times 51.2 = 768\ \mu s$；连续 11 次时指数截断为 10，最大为 $1023 \times 51.2 = 52.3776$ ms。这两个计算分别展示未截断与已截断的退避边界。达到规定的重试上限会报告失败，而不是无限指数增大。

3.12 CSMA/CA、确认与 NAV：隐藏站等到何时

无线发送端自身信号远强于接收信号，且不同站点的可听范围不同，不能沿用有线的边发边检测。CSMA/CA 采用物理载波监听、虚拟载波监听、帧间间隔和随机退避，尽量减少冲突；接收方对成功收到的单播数据帧发送 ACK。没有收到 ACK，发送方才按协议重试；缺 ACK 也可能是 ACK 本身丢失，不一定证明数据没到。

DIFS 用于一般数据竞争，SIFS 较短，使 ACK、CTS 等属于当前交换的响应获得更高优先权。倒计时遇信道忙时冻结，信道恢复空闲并满足相应间隔后继续，不是每听到忙都把已退避的时间无条件作废。

RTS/CTS 是可选的预约机制。A 向 AP 发 RTS，AP 回复 CTS 后，A 发 DATA，AP 发 ACK，相邻响应间隔为 SIFS。隐藏站 B 可能听不到 A，却能听到 AP 的 CTS；它根据持续时间字段设置**网络分配向量 NAV**，在剩余预约期间推迟竞争。RTS/CTS 减少某些长数据冲突损失，但有自

身开销，不是所有无线单播必用。

错项排除·NAV 只预约未来

收到 CTS 的一刻，RTS、CTS 和前面的等待都已过去。此后还剩的 SIFS、DATA、SIFS、ACK 才构成基本预约余量。不同题目从不同事件起计时，应先画起点，不能只凭协议名背加法项。

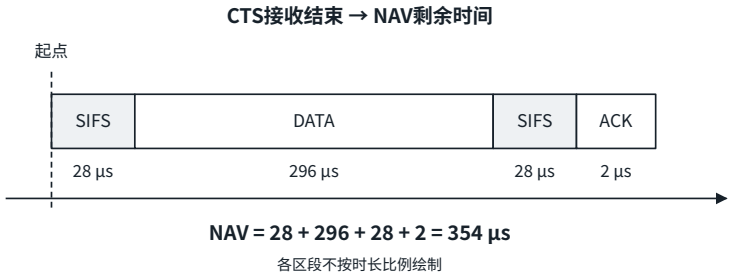


Figure 3.2: CTS 结束后，剩余预约由 DATA、ACK 及两个 SIFS 组成

3.12.1 例 3-9　收到 CTS 后只计算剩余时间

例 3.12.1 (配套例题)

配套例题。在采用 CSMA/CA 协议的 802.11 无线局域网中，DIFS=128 µs，SIFS=28 µs，RTS 帧、CTS 帧和 ACK 帧的传输时延分别是 3 µs、2 µs 和 2 µs，忽略信号传播时延。若主机 A 要向 AP 发送一个总长度为 1998 B 的数据帧，无线链路带宽为 54 Mb/s，则隐藏站 B 收到 AP 发送的 CTS 帧时，设置的 NAV 值是（　）。

A. 326 µs　B. 354 µs　C. 385 µs　D. 513 µs

解析。起点为 CTS 已经收到，RTS、前面的 DIFS 和 CTS 自身都已经过去。数据发送为 $1998 \times 8/54 = 296\ \mu s$，余下顺序为 SIFS—DATA—SIFS—ACK，所以

$$NAV = 28 + 296 + 28 + 2 = 354\ \mu s,$$

选 **B**。图中 CTS 之前还有一个 SIFS，它计入从 RTS 结束开始预约的区间，不计入收到 CTS 后的剩余区间。把起点画在时间线上，比直接背 “加几个间隔” 更可靠。

3.13 令牌传递与轮询：如何给出有界的使用机会

轮询由控制站依次询问各站是否需要发送，避免数据站任意竞争，但控制站、轮询开销与等待次序会影响性能。令牌传递把发送许可放在沿逻辑环传递的控制帧中，获得令牌的站才能发送，发送后按规则释放。

正常运行、只有一个有效令牌的模型下不会发生数据争用冲突；需有机制处理令牌丢失、重复与站点失效。为防止其他站长期等待，站点持有令牌有时间或发送量上限。无数据的站应及时转交令牌，不必把上限时间耗满；有很多数据的站也不能一直占用到全部发完。〖3.5-014〗中的两个绝对说法都忽略了这一约束。重负载下轮流使用可减少反复碰撞，轻负载下则可能多付令牌等待开销。

3.14 局域网与 IEEE 802 的层次

局域网技术通常覆盖物理层与数据链路层。IEEE 802 体系把数据链路层进一步划为逻辑链路控制 LLC 与介质访问控制 MAC，MAC 负责与

具体介质有关的寻址、成帧和访问控制等工作。网卡通常同时实现物理信号收发与链路层帧处理，不能因为 “用 MAC 地址” 便认为它没有物理层功能。〖3.6-004〗〖3.6-023〗

以太网、无线局域网可采用不同 MAC 机制，仍可在网络层共同承载 IP。MAC 地址只在适用的链路技术和广播范围内有意义，交换机学习的目的不是建立全互联网的 MAC 路由表。

不变量·帧与线上周期不同

无标签最短 MAC 帧 64 B，若再计 8 B 前导部分与 96 bit 时间的帧间隔，一个发送周期相当于 84 B。间隔是时间而非额外字段；IP 总长度、MAC 帧长与线上周期必须各自保持边界。

3.15 以太网帧、MAC 地址与有效载荷

本节采用 **Ethernet II, 即 DIX Ethernet V2、无 VLAN 标签**的格式。MAC 帧从目的地址起依次为目的 MAC 6 B、源 MAC 6 B、类型 2 B、数据及必要填充 46～1500 B、FCS 4 B。前 3 个字段组成 **14 B MAC 帧头**；帧头、数据/填充与 FCS 一起才是完整 MAC 帧，长度为 64～1518 B。FCS 是帧尾，不能把首尾共 18 B 都称为首部。

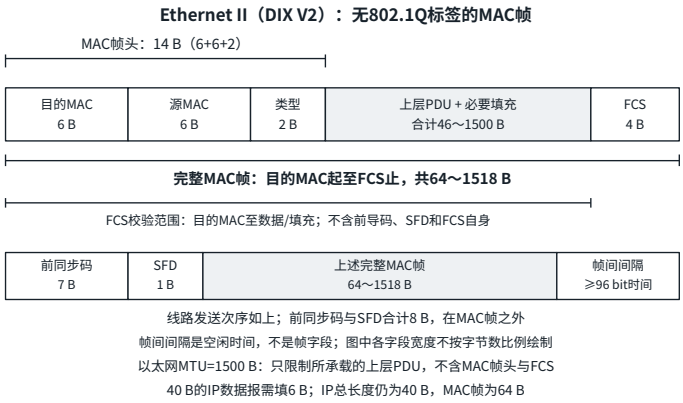


Figure 3.3: Ethernet II 无标签帧：帧头 14 B，MAC 帧 64～1518 B；前同步码、SFD 与帧间间隔另计

线路在 MAC 帧前另发送 7 B 前同步码和 1 B 帧开始定界符 SFD，二者常合称 8 B 前导部分，用于同步与开始定界，均不计入上述 MAC 帧长。帧间间隔至少为 96 bit 的发送时间，它是空闲时间而不是帧字段。10 Mb/s 时相当于 9.6 μs；若只将这项时间折算成字节的发送时间，相当于 12 B，但线上并没有因此再增加一个 12 B 帧字段。

类型字段说明承载的上层协议，例如 `0x0800` 表示 IPv4、`0x0806` 表示 ARP。IEEE 802.3 的基本格式在同一位置使用长度字段，不能把它和 Ethernet II 的类型字段同时算成两个独立字段。两种格式的区别不改变本节无标签 MAC 帧的源、目的各 6 B 与 FCS 4 B 口径。

MAC 地址通常 48 bit，以 6 组十六进制表示。目的全 1 是广播地址；首字节最低位指示单播与组播。源地址标识发送该帧的接口，一般不能填成广播地址。以太网 FCS 使用 CRC，只检查当前帧；错误帧通常丢弃，以太网 MAC 不负责确认重传。

FCS 占 32 bit，发送端对从目的 MAC 到数据末尾（含必要填充）的比特计算 CRC，接收端按对应规则检验；前同步码、SFD、帧间间隔不参加该 MAC 帧校验。以太网 MTU=1500 B 限制的是所承载的上层 PDU，如完整 IP 数据报，不包括 14 B 帧头和 4 B FCS。因而 “IP 总长度 1500 B” 和 “以太网 MAC 帧 1518 B” 描述的是同一次封装的不同边界。

一个 40 B 的 IP 数据报装入普通无标签帧，需要填充 6 B。IP 总长度仍写 40 B，填充属于以太网帧数据区，接收 IP 层依据自己的长度字段取出数据，不把填充当成 IP 载荷。以太网以帧作为传送/交换单位，属

于分组交换技术，不能因为有“交换机”就推断为电路交换。〖3.6-003〗

计算效率要先确定分母范围。若只问 MAC 帧内应用数据效率，用应用字节数除 MAC 帧长；若问线上连续发送的有效吞吐量，还可能需要加入前导码与帧间间隔。吉比特与更高速以太网的物理编码各有制式，不能把 10 Mb/s 的曼彻斯特编码直接套用到更高速率。

例如连续发送最短帧，若题目明确计入 8 B 前导部分及 96 bit 时间的间隔、忽略物理编码额外开销，一个周期为 $(8+64)\times 8+96=672$ bit 时间，即 84 B 的发送时间。64 B 对应的是 MAC 帧长，84 B 对应的是这一模型下每帧占用的周期；两者分母不同，不能互换。最短 MAC 帧长的规则也继续适用于普通全双工帧，不能因为不使用 CSMA/CD 就删去必要填充。

错项排除·当前跳发送者未必是源

AP 向接收站转发来自远端主机的数据时，无线帧地址 2 标记当前发送 AP，地址 3 保留原始源。把三个地址都按以太网“源和目的”两字段硬套，会把无线跳的接口和端到端主机混在一起。

3.16 802.11：接收者、发送者和原始端点可能不同

基础设施无线局域网由接入点 AP 与相关联站点形成基本服务集 BSS，AP 的无线接口地址用作 BSSID。多个 BSS 经分布式系统互联可组成扩展服务集；站点移动时可重新关联。AP 负责无线接入及桥接，不因名为“接入点”就自动具有跨子网路由功能。

无线帧常需要同时记录当前无线跳的收发接口和原始源/目的。常见三地址数据帧可按下表判断；这里 DS 表示分布式系统。

方向与控制位	地址 1、地址 2	地址 3
站点 →AP，ToDS=1、FromDS=0	当前接收者 AP、当前发送站点	最终目的 MAC
AP→ 站点，ToDS=0、FromDS=1	当前接收站点、当前发送者 AP	原始源 MAC
同一自组织网，二位均 0	目的、源	BSSID

两位都为 1 的无线分布式转发可能使用第四地址，不能把三地址表硬套过去。A 经过 AP1、有线分布式系统、AP2 转发给 B 时，在 AP2→B 这一无线跳上，地址 1=B、地址 2=AP2 的 BSSID、地址 3= 原始 A；AP1 不是该跳的发送接口。〖3.6-031〗考查的正是这个端点区分。

以太网与无线局域网都可承载 IP，但应在相同介质条件下比较访问和交付机制。

比较维度	共享半双工以太网	交换全双工以太网	本章 802.11 DCF 无线模型
发送机会	多站共享，使用 CSMA/CD 竞争	每条点到点链路可同时双向发送，无 CSMA/CD 竞争	共享无线信道，使用 CSMA/CA 与退避
冲突怎样处理	发送中检测，发现后中止并退避	正常全双工链路没有传统共享介质冲突	难以可靠边发边检测；用监听、预约及反馈减少损失
基本链路确认	无逐帧 MAC 确认重传	无逐帧 MAC 确认重传	普通单播数据帧成功后 ACK，必要时重试；广播/组播不套用此 ACK 流程
地址的判断	当前以太网帧的源与目的	当前以太网帧的源与目的	还要区分当前收发接口、原始端点及 DS 方向位

无线 ACK 与 TCP ACK 所在层次、确认单位和作用范围不同。即使当前无线帧已确认，后续网络仍可能丢失 IP 分组；反过来，没有无线 ACK 也可能是 ACK 丢失。比较时应先指出反馈来自哪一层，再决定重传谁。

3.17 VLAN：广播域如何跨交换机延伸

最小反例·端口并行不隔离广播

A→B 与 C→D 能在四端口交换机上并行，只说明出口互不冲突。一帧广播仍可传播至其余允许端口，所以普通二层交换的独立端口不能自动构成多个广播域；VLAN 边界需要另行配置。

VLAN 把一个交换网络划成多个逻辑广播域。一个 VLAN 中的广播只在该 VLAN 允许的端口传播；不同 VLAN 之间的 IP 通信需要路由等三层功能。VLAN 不等于链路聚合，前者划分逻辑广播范围，后者把多条链路组成一条逻辑连接。安全性改善也取决于正确配置，不能把 VLAN 当成所有攻击的隔离保证。

接入口常给主机提供无标签帧，交换机根据端口配置确定所属 VLAN；交换机之间的干道可携带多个 VLAN，使用 802.1Q 标签标识归属。因此同一 VLAN 可以跨多台交换机，不要求主机发送的每一帧都自带标签。

802.1Q 在源地址之后插入 4 B 标签，其中含 TPID 和标签控制信息，VID 占 12 bit；可编码 4096 个数值，但 0、4095 有保留用途，通常可配置的普通 VLAN ID 为 1～4094。含单个标签的常见最大 MAC 帧长度由 1518 B 变为 1522 B，原 FCS 应相应更新，不能要求增加标签却保持最大长度不变。〖3.6-032〗

3.17.1 例 3-10　填充前先识别有无标签

例 3.17.1 (配套例题)

〖3.6-008〗一个长度为 40 B 的 IP 数据报需要封装成 802.1Q 帧进行传输，则此 802.1Q 帧的数据载荷部分需要填充的字节数是（　）。

A. 2　　B. 4　　C. 6　　D. 8

解析。本题按王道书的计量模型：含标签 MAC 帧最短仍按 64 B 计，首尾部共 $6+6+4+2+4=22$ B，数据区下限为 42 B。因而需填充 $42-40=2$ B，选 **A**。若采用前节普通无标签帧，首尾部 18 B、数据区下限 46 B，同一个 IP 数据报会填 6 B。不要混用两套字段长度；实际设备插入标签时原有填充是否保留，是另一项处理规则。

3.18 广域链路与 PPP：不需要争用不等于没有成帧

广域网侧重把较远节点互连，核心常由路由器和点到点链路组成；具体设备和传输协议不必与某一种局域网相同。PPP 在点到点链路上封装网络层分组，不必解决多站争用，因此不使用 CSMA/CD，也没有按争用时间片规定的 64 B 最小帧要求。

PPP 常见基本帧依次为标志 `7E`、地址 `FF`、控制 `03`、协议字段、信息字段、FCS、标志 `7E`。协议字段区分 IP、LCP 或某一 NCP 等。LCP 负责建立、配置与测试数据链路，NCP 是一组针对不同网络层协议的配置协议；LCP 建立不等于已经取得 IP 通信所需的全部配置。

异步 PPP 使用字节转义，例如将数据 `7E` 发送为 `7D 5E`，`7D` 发送为 `7D 5D`；接收端逆向恢复。教材同步 PPP 模型采用零比特填充。PPP 以 FCS 检错，基本协议不靠数据帧编号和确认重传来提供可靠交付；不能因名称中“协议”或连接曾建立便认定它必可靠。

3.19 交换机如何学习、过滤和转发

不变量·学源查目的

帧从端口 1 带着源 A 进入，才有证据把 A 记在端口 1；目的 B 未知时只能泛洪寻找。用目的地址学习会把尚未找到的 B 误记到发送者位置，下一次转发将沿错误出口走。

交换机收到帧后，先根据**源 MAC 地址与入端口**学习或更新表项，再查**目的 MAC 地址**决定出口。若目的表项在另一端口，定向转发；若目的表项就在入端口，过滤；若目的未知或是广播，在当前 VLAN 内除入端口外的允许端口泛洪。学习表项时不能从目的地址反推其位置，因为帧可能正是由于目的未知才发来。

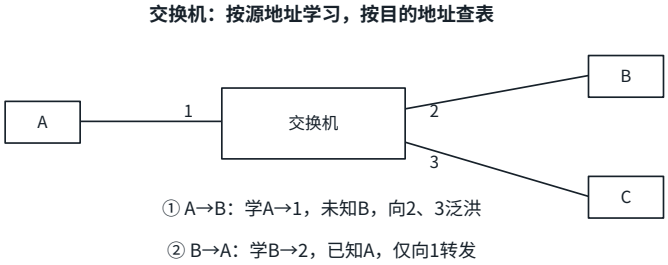


Figure 3.4: 首次 A→B 使交换机学习 A；B 回复时定向转发并学习 B

图中转发表初始为空。A 由端口 1 发给 B，交换机记 `A→1`，向 2、3 泛洪；B 从端口 2 回复 A，记 `B→2`，查 A 只向 1 转发。若以后端口 1 上的另一站发来目的为 A 的帧，交换机查出目的也在 1，就不再向其他端口转发。表项会老化，拓扑有环还需要生成树等机制避免广播持续循环。

交换机各端口可形成独立冲突域，普通二层交换机不因端口不同自动隔离广播域；VLAN 配置另行划分广播域。全双工独立链路上本来就不发生传统共享介质冲突，“每端口一冲突域” 是题目常用的拓扑比较口径，不等于每条全双工链路都有实际碰撞。

3.19.1 例 3-11　直通交换最早何时能决定出口

例 3.19.1 (配套例题)

配套例题。对于 100 Mb/s 的以太网交换机，当输出端口无排队，以直通交换方式转发一个以太网帧（不包括前导码）时，引入的转发时延至少是（　）。

A. 0 μs　　B. 0.48 μs　　C. 5.12 μs　　D. 121.44 μs

解析。直通交换不必收全帧，但至少需要读完位于帧开头的 6 B 目的地址，才能作转发决策。时间为 $6 \times 8/(100 \times 10^6) = 0.48\ \mu s$，选 **B**。5.12 μs 是接收 64 B 所需时间，属于另一种等待情形；题目已明确不计前导码，也不要再增加 8 B。

存储转发要收齐帧并能完成 FCS 检查后才转发；直通在帧尾未到前便可能开始，降低转发等待，却不能预先排除帧尾才发现的错误。总吞吐量题还要明确统计发送量还是收发之和。独立端口可以并行，但经过同一低速出口的流仍共享那个瓶颈；把 11 个端点都写成 100 Mb/s 之前，应检查是否真能安排不超端口能力的通信。

3.19.2 例 3-12　双向 GBN：确认号、回绕与重传要分开追踪

辨析·确认与发送分开追踪

捎带确认号说明期望收到的下一帧；发送序号记录本方发送。先画两个方向的序号轴，再处理丢失、超时和模 8 回绕。

例 3.19.2 (配套例题)

配套例题。甲乙双方均采用后退 N 帧协议（GBN）持续进行双向数据传输，始终采用捎带确认，帧长均为 1000 B。$S_{x,y}$、$R_{x,y}$ 分别表示甲方、乙方发送的数据帧，x 为发送序号，y 为确认序号，表示希望接收对方的下一帧序号。两个序号字段均为 3 bit，信道传输速率为 100 Mb/s，RTT=0.96 ms。下图为甲方发送和接收数据帧的两种场景；t_0 时甲方的发送序号和确认序号均为 0，t_1 时甲方有足够多的数据待发送。

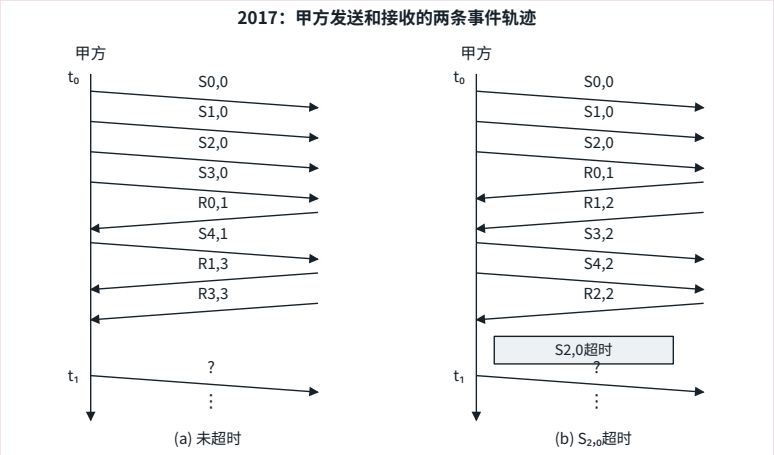


Figure 3.5: 箭头从甲方发出为 S、指向甲方为 R；两列分别对应未超时和 S2,0 超时的原题场景

（1）对于图 (a)，t_0 到 t_1 期间，甲方可以断定乙方已正确接收多少个数据帧？是哪几个帧？请用 $S_{x,y}$ 表示。

（2）对于图 (a)，从 t_1 起，在不出现超时且未收到乙方新的数据帧之前，甲方最多还可以发送多少个数据帧？第一个和最后一个分别是什么？请用 $S_{x,y}$ 表示。

（3）对于图 (b)，从 t_1 起，在不出现新的超时且未收到乙方新的数据帧之前，需要重发多少个数据帧？重发的第一个帧是什么？请用 $S_{x,y}$ 表示。

（4）甲方可以达到的最大信道利用率是多少？

解析。读题时先分清两条状态线：收到帧的确认序号决定 “我发出的哪些帧已被确认”；收到帧的发送序号决定 “我下次应捎带哪个确认号”。GBN 的接收窗口为 1，收到失序帧不能跳过缺口向前确认。序号空间为 $2^3 = 8$，发送窗口最多为 7，所有序号按模 8 回绕。

图 (a) 最后收到 $R_{3,3}$。其确认号 3 累计确认甲方序号 0、1、2 已正确到达，因此第（1）问为 3 帧：$S_{0,0}$、$S_{1,0}$、$S_{2,0}$。不能因它的发送序号为 3，便推断甲方也收齐了乙方 0～3 号帧：甲方依次正确收到 $R_{0,1}$、$R_{1,3}$，随后缺少乙方 2 号帧，$R_{3,3}$ 作为失序帧被丢弃，所以甲方仍希望收到乙方 2 号帧。

此时甲方已发送到 $S_{4,1}$，未确认的是发送序号 3、4 两帧。7 个窗口位置中还剩 $7 - 2 = 5$ 个；下一发送序号从 5 开始，捎带确认号始终为 2，故第（2）问可发送 $S_{5,2}$、$S_{6,2}$、$S_{7,2}$、$S_{0,2}$、$S_{1,2}$，首帧为 $S_{5,2}$，末帧为 $S_{1,2}$。回绕后的 0、1 是新一轮数据，不能与已经确认的旧 0、1 混为一谈。

图 (b) 乙方最后通告确认号 2，意味着甲方 0、1 号已确认，2、3、4 号仍未确认。$S_{2,0}$ 超时触发从最早未确认帧开始的后退重传，因此第（3）问重发 3 帧。甲方此时已按序收到乙方 $R_{2,2}$，期待乙方下一帧为 3，故重传首帧应更新捎带确认号，写成 $S_{2,3}$；随后为 $S_{3,3}$、$S_{4,3}$，不能照抄旧帧的确认号 0。

第（4）问须保留捎带确认帧的发送时间。发送一个数据帧需 $T_d = 1000 \times 8/(100 \times 10^6) = 0.08$ ms；确认也装在 1000 B 的数据帧中，$T_a = 0.08$ ms。沿用教材模型：题目中的 RTT 表示往返传播时延；从发送首帧到完整收到其确认，周期为 $T_d + \mathrm{RTT} + T_a = 1.12$ ms。最多连续发 7 帧，故 $U_{\max} = 7 \times 0.08/1.12 = 50\%$。全双工允许两个方向同时工作，但本题求甲方单方向利用率；也不能把长捎带确认当成长度可忽略的纯 ACK。

条件·先固定书写方向

位号 1、2、4 是校验位置；从 H7 向 H1 书写与从 H1 向 H7 书写不能混用。

3.20 海明码从位号到纠错判决

设信息按 $D_4D_3D_2D_1 = 1010$ 给出，7 位码按 $H_7H_6 \cdots H_1$ 从高位写到低位。校验位分别放在 H_1, H_2, H_4，数据放在 $H_3 = D_1, H_5 = D_2, H_6 = D_3, H_7 = D_4$。偶校验给出

$$
\begin{aligned}
P_1 &= D_1 \oplus D_2 \oplus D_4 = 0,\\
P_2 &= D_1 \oplus D_3 \oplus D_4 = 1,\\
P_3 &= D_2 \oplus D_3 \oplus D_4 = 0.
\end{aligned}
$$

位号	7	6	5	4	3	2	1
含义	D_4	D_3	D_2	P_3	D_1	P_2	P_1
正确码字	1	0	1	0	0	1	0
第 2 位出错	1	0	1	0	0	0	0

接收 `1010000` 时，三个分组的异或结果为 $S_1 = 0, S_2 = 1, S_3 = 0$，综合征 $S_3S_2S_1 = 010_2$，在单错假设下定位第 2 位。若改成 $D_1D_2D_3D_4 = 1010$ 且从 H_1 向 H_7 书写，信息在各位中的位置会变化；两道题给出的字符 “1010” 相同，却不是同一个码字布局。

增加一位总体偶校验后，在最多两位错的模型下，判决可写为：

原海明综合征	全体偶校验	判决
0	通过	在给定错误数模型下无错
非 0	失败	原海明位中有一位错，按综合征纠正
0	失败	总体校验位本身出错
非 0	通过	两位错，报告检出，不能当作单错纠正

这张表说明 “能检测双错” 与 “对所有双错都能确定位置” 不同。超过规定错误数时，判决仍可能被误导，不能把扩展海明码称作任意多位错的可靠纠错码。

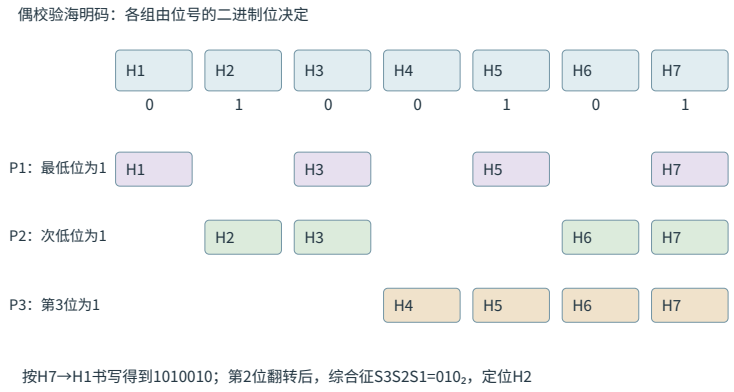


Figure 3.6: 海明码的三个校验分组与位号关系

方法·发送时间也占周期

首个确认返回周期含本帧发送、往返传播和 ACK 发送。

3.21 窗口利用率与回绕反例放在一起算

课件算例取数据率 1 kb/s、数据帧 4 kb、确认帧 1 kb、单向传播 7 s。每帧发送 4 s，确认发送 1 s，第一帧从开始发送到确认返回共 $4+14+1=19$ s。无错、立即确认、全双工且反向确认链路不成为额外瓶颈时：

$$U_{\text{停等}}=\frac{4}{19}\approx 21.05\%,\quad U_{W=4}=\frac{16}{19}\approx 84.21\%,\quad U_{W=5}=1.$$

当 ACK 长度可忽略时，停等利用率改为 $4/18\approx 22.22\%$。连续发送窗口为 W 时，数据占用比例取 $\min(1, WT_d/(T_d+2\tau+T_a))$，其中“取 1 封顶”表示流水已经覆盖等待空隙，不能产生超过 100% 的时间占用。

窗口限制还必须检查序号空间。以 2 bit 序号的 GBN 为例，若错误取 $W_s=4$，接收方收到 0、1、2、3 后又期待 0，恰好所有 ACK 都丢失；发送方重发旧 0，接收方却会把它当成下一轮新 0。把 W_s 限制为 3 时，发送方旧窗口与接收方将来的合法区间保留了区分余地。SR 允许接收多个序号，因此要同时检查 $W_s+W_r\le 2^k$，对称窗口才进一步化为半个序号空间。

对比三种协议时，把“数据帧丢失”和“确认帧丢失”分别画时间线：停等通过同序号重复确认避免重复交付；GBN 较后的累计 ACK 可以覆盖前面丢失的 ACK；SR 独立确认，某帧的 ACK 丢失只使该帧保持未确认，重复帧到来时接收方仍须重新发 ACK。NAK 可用于提前请求重传，但 NAK 本身也会丢失，超时机制仍有必要。

3.22 复用、令牌和无线控制帧的补充边界

同步 TDM 给各用户固定时隙，用户无数据时相应时隙闲置；统计 TDM 按需求分配，需要用地址、标识或确定次序说明时隙属于谁。FDM 为不同用户分频带，保护频带减少相互干扰；数字数据也能先调制到不同载波后采用 FDM，因此“FDM 只能承载模拟数据”并不成立。WDM 在不同光波长上复用；CDM 用码片序列分离用户，三者的区分维度不同。

令牌环的典型流程是：源站捕获发送许可、发出数据帧，目的站复制并设置接收状态，数据帧回到源站后由源站移除，再按协议释放令牌。课件的一帧后释放模型是具体发送额度；一般令牌协议也可以通过持有时间或额度约束，不能把某一示例的“一帧”推广成所有实现。

802.11 的基本服务集 BSS 由一个 AP 及关联站点构成，BSA 表示覆盖的地理范围，SSID 为服务集名称，BSSID 通常由 AP 的无线接口地址标识。多个 BSS 经分布式系统组成 ESS；门户把无线分布式系统接到其他局域网。站点在基础设施 BSS 中的通信通常由 AP 转发，自组织模式应另看其规则。

RTS、CTS、ACK 均为控制帧。常见基本 RTS 含帧控制 2 B、持续期 2 B、接收地址 6 B、发送地址 6 B、FCS4 B，共 20 B；CTS 与 ACK 只有接收地址而没有单独的发送地址字段，共 14 B。持续期从本帧结束后计剩余预约时间，接收地址的含义按这一次控制交换确定。数据帧、控制帧、管理帧三类用途也不能互换，探测和关联属于管理，RTS/CTS/ACK 属于控制。

课程连接·先 LCP 后 NCP

先把链路配置好，再配置承载的网络层协议，之后传 IP 分组。

3.23 PPP 链路怎样进入可传送 IP 的状态

PPP 由封装方法、链路控制协议 LCP 和一族网络控制协议 NCP 组成。典型建立过程为物理连接、LCP 协商、可选鉴别、NCP 协商，再传送网络层分组；结束时依次撤销网络层配置、LCP 链路和物理连接。LCP 协商链路参数和鉴别方式，NCP 配置对应网络层协议，不能把 NCP 当成另一种 IP 应用。

PPP 基本帧依次为标志 `7E`、地址 `FF`、控制 `03`、协议字段、信息、FCS、结束标志 `7E`。协议字段说明信息区承载的协议，例如 IPv4、LCP 或 NCP；固定地址不承担以太网式的多站寻址。异步链路用字节转义，同步链路可用零比特填充。PPP 基本链路并不依靠数据帧序号和逐帧 ACK 提供可靠交付，检出错后通常丢弃，由相应上层按需恢复。

PPPoE 用以太网承载 PPP 会话，是封装关系而不是把 PPP 当作 IP 之上的应用。读取封装层次时，从外到内依次识别本跳以太网、PPPoE/PPP 及其协议字段，最后才解释其信息区中的 IP 分组。

3.24 主动回忆自测

以下为自拟题。先说明所用 ACK 语义、序号空间与链路模型，再回答。

1. 数据 `111110` 按零比特填充后是什么？去填充时删除几个 0？
2. 3 位序号下，GBN 最大发送窗口与对称 SR 最大窗口分别是多少？
3. SR 接收窗口为 2～5，已缓存 3、4，随后收到 2。哪些帧可按序交付？
4. A、B、C、D 接入普通无 VLAN 交换机的四个全双工端口，能同时 A→B 和 C→D 吗？是否自动成为两个广播域？
5. CRC 余数为 0，能否证明所有可能的传输错误都未发生？码距 4 能保证检测、纠正多少位错？
6. 为什么共享半双工以太网最短帧必须持续发送至少一个最坏往返传播时间？
7. 收到 CTS 以后设置 NAV，是否应把已经过去的 RTS 和 CTS 发送时间再算一次？
8. 一个 40 B IP 数据报装进普通无 VLAN 的 Ethernet II 帧，至少填多少字节？IP 总长度字段会跟着增大吗？
9. 交换机收到从 A 发向未知 B 的帧，应从源地址还是目的地址学习端口？PPP 是否需要 CSMA/CD？

3.24.1 核对答案

1. `1111100`，删除第五个 1 后插入的 0，保留原数据末尾的 0。

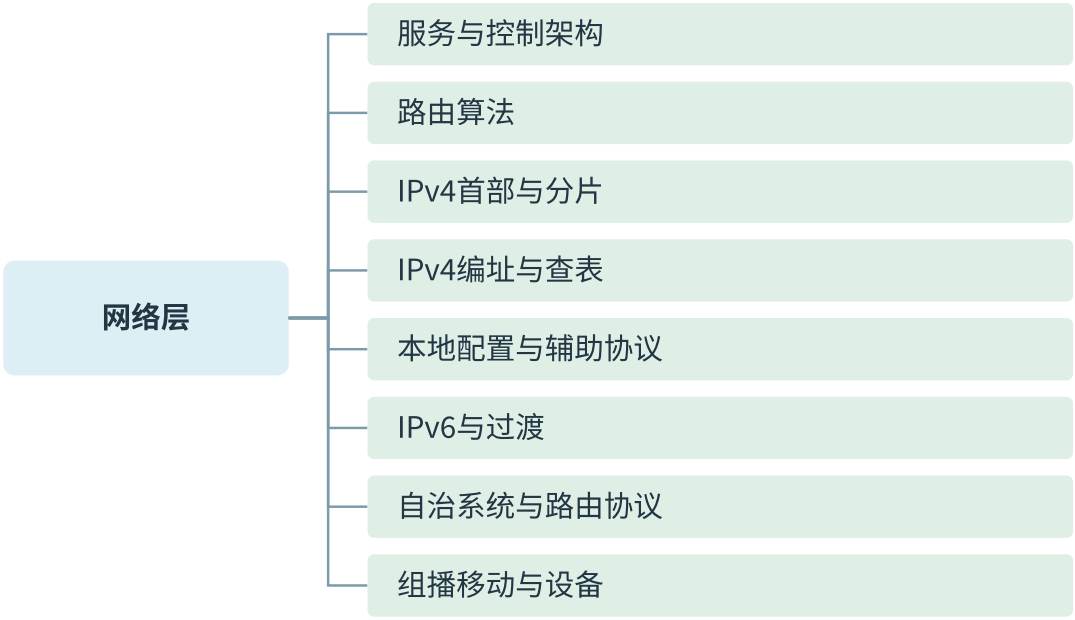
2. 7 和 4；一般 SR 约束是两窗和不超过序号空间，对称时才直接取一半。
3. 连续交付 2、3、4，左沿移到 5，不能越过还缺的 5。
4. 可以，两个流的输出端口不同；仍是一个广播域，隔离广播需相应 VLAN 或三层边界。
5. 不能，存在非零错误模式被生成式整除；码距 4 可保证检出 3 位错、纠正 1 位错。
6. 让最远端最晚产生的冲突反馈返回时，发送站仍在发送，才能检测到冲突。
7. 不应，NAV 描述当前时刻之后的剩余预约区间。
8. 填 6 B；IP 总长度仍为 40 B，填充属于链路封装。
9. 学源地址 A 与入端口，目的 B 未知时在当前 VLAN 内泛洪；点到点 PPP 无需多站争用，因而不使用 CSMA/CD。

课程连接·先 LCP 后 NCP

先把链路配置好，再配置承载的网络层协议，之后传 IP 分组。

网络层 4

章节脉络



网络层要把分组从源主机送向目的主机；链路层每次只完成当前一跳。做地址题时，先分清“分组最终到谁”和“这一帧先交给谁”，再根据子网、路由表和地址解析决定下一步。

本章 IPv4 地址按 32 位无符号值处理，掩码按位与记为 &。普通多址子网计算可分配地址数时，除去网络地址和广播地址；点到点 `/31`、主机路由 `/32` 等特殊用途另论，不强套 $2^h - 2$。

不变量·路径形成与查表分开

动态路由更新可以较慢，当前分组的查表转发仍须高速执行。题目给出“路由器收到一份分组”并不意味着它立刻运行一次全网最短路；应先使用已经形成的转发表判断下一跳与出口。

4.1 路由与转发：先选路径，再逐包查表

定义 4.1.1 (路由选择与逐包转发)

路由选择根据网络可达性、度量和策略建立到各目的的路径信息；**转发**在分组实际到达时，查询目的地址等信息，选出输出接口和下一跳。前者决定表怎样形成，后者决定当前分组怎样走。路由协议通常运行较慢，转发需要按分组高速执行。

IP 把底层不同的物理链路和链路层协议联在一起，对外提供统一的网络层服务。一段可以是以太网，下一段可以是点到点链路；路由器取出 IP 分组后，用下一跳所需的链路格式重新封装。源和目的主机可以使用不同操作系统，不影响它们共同遵守 IP 协议。

普通 IP 路由转发不自动翻译任意两种网络层协议。IPv4/IPv6 转换需要明确的转换机制，应用协议转换也需要相应网关功能。王道书 §4.1 个别题把特定转换路由器纳入“路由器”讨论，解释时应限定这项额外能力，不能由此推出所有路由器都能在任何协议之间互译。转发经过路由器的正常流量，也不意味着每份数据都由路由器本机应用处理。

4.2 SDN：控制平面与数据平面怎样分工

课程连接·控制像策略数据像执行

操作系统调度器决定策略，CPU 按选定上下文执行；SDN 也把规则计算与逐包执行分开。这个类比只帮助理解分工，不能据此认定控制器必须单机、必须在每份分组的实际转发路径上。

传统设备既参与路由计算，也执行分组转发。软件定义网络 SDN 在逻辑上分离这两项职责：控制平面由控制器掌握网络状态并计算规则，数据平面的交换设备按下发的流表处理分组。逻辑集中不要求整个网络只能有一台物理控制服务器。

流表通常包含匹配字段、优先级、计数器和动作，匹配不必只看目的 IP，也可看入端口、MAC、协议、传输层端口等。动作可转发、丢弃、改写字段或上送控制器。已匹配既有规则的分组不必逐包等待控制器批准；未匹配时如何处理由表项与系统配置决定。

控制器向数据平面设备下发规则的接口称为**南向接口**；上层应用向控制器表达需求的接口称为北向接口。OpenFlow 是南向通信的一种协议，不等于 SDN 的一切实现。SDN 改变的是控制架构，不是新发明一种星形或环形物理拓扑；控制器也不必位于每台 OpenFlow 交换机内部。〖4.1-018〗

4.3 拥塞与流量控制：瓶颈究竟在哪里

多个输入流竞争同一输出链路，短期到达量超过服务量就会排队；持续过载且缓存有限则会丢包。路由器处理器很快，不代表输出链路的发送能力没有上限。增加缓存可以吸收部分突发，但长期到达速率大于发送速率时仍会积压，并增加排队时延。

流量控制保护一个接收端的处理或缓存能力，拥塞控制保护网络中共享链路与节点的承载能力。接收方窗口很大，路径中间仍可能拥塞；某个接收端窗口很小，也不能证明全网已拥塞。控制思路包括准入与资源安排、整形、合理调度、主动丢弃/标记、反馈并降低注入量等。第 5 章将用 TCP 窗口具体推演端系统的反馈控制。

分析树形网络，要对每个方向分别合计可能进入该输出链路的流量，再与该方向带宽比较。全双工的两个方向各有发送能力，不能把正反向流量先混加；同样，上层链路比单条下层链路快，并不意味着可以忽略多条下层链路的汇聚。〖4.7-018〗相关模型应在输出接口处画出所有竞争输入。

4.4 静态、动态与层次路由

静态路由由管理员配置，开销低、行为明确，但无法自行适应所有故障。动态路由根据协议交换信息、更新路径，能适应拓扑变化，也会付出报文、计算和收敛成本。动态不等于每个分组都重新运行完整路由算法。

把全网所有细节交给所有路由器会增加表项和更新范围。层次路由把网络划为区域或自治系统，对外按汇总信息处理，减少全局细节。代价是路径选择可能受到层次和策略约束，不保证按一种简单全局权值永远得到最短路。讨论 “最优” 必须先说明优化的是跳数、时延、费用还是政策要求。

4.5 距离—向量：邻居报出的距离还要加哪一段

距离—向量算法让节点向邻居通告自己到各目的的距离。节点 x 收到邻居 v 的通告后，候选距离为 “x 到 v 的直接链路代价” 加 “v 通告的到

目的距离”，即

$$D_x(d) = \min_{v \in N(x)} \{c(x,v) + D_v(d)\}.$$

不能只在邻居报告中取最小数，也不能把本机到邻居的链路代价当成最终目的距离。下一跳是使总和最小的邻居；同值时按题目规则保留或选定。

不变量·暂定距离尚可变小

Dijkstra 的未确定点距离只是当前最好候选，选定最小点后再松弛其他点。A 到 C 初值 4，在经 B 的路径代价 1+2 出现后应降为 3；算法正确性还依赖非负边权，不能省略条件。

4.5.1 例 4-1　逐列做加法，再比较候选

例 4.5.1 (配套例题)

配套例题。某网络中的所有路由器均采用距离向量路由算法。路由器 E 与邻居 A、B、C、D 的直接链路距离分别是 8、10、12、6，收到的距离向量如下。E 更新后到 Net1～Net4 的距离分别是（　）。

目的网络	A、B 的通告	C、D 的通告
Net1	1，23	20，22
Net2	12，35	30，28
Net3	24，18	16，36
Net4	36，30	8，24

A. 9，10，12，6　　B. 9，10，28，20

C. 9，20，12，20　　D. 9，20，28，20

解析。对 Net1，经 A、B、C、D 的总距离为 9、33、32、28，取 9；Net2 为 20、45、42、34，取 20；Net3 为 32、28、28、42，取 28；Net4 为 44、40、20、30，取 20。因此选 **D**。Net3 经 B、C 并列最优，不影响距离答案；若另问下一跳，就须保留并列或按指定规则选择。

这种算法只掌握邻居的距离摘要，坏消息传播时可能形成互相误导的路由环并 “计数到无穷”。水平分割等规则可以抑制部分环路，但并不意味着所有异步故障情形都已解决。

4.6 链路状态：全网拓扑怎样变成最短路径树

链路状态算法先发现邻居并测量链路代价，再把本机相邻链路状态洪泛到规定范围。各节点获得同一区域的拓扑数据库后，各自以自身为根计算最短路径树；不是收到一条通告就把通告者选成所有目的的下一跳。

Dijkstra 算法从源点开始，维护已确定最短距离的集合和各未确定点的暂定距离。每轮取暂定距离最小的点，加入集合，再用该点的出边更新其他点。对非负链路代价可使用该过程；若题设出现负权，不能无条件沿用这一结论。

用四点网络作最小推演：边 AB=1、AC=4、BC=2、BD=5、CD=1，边为双向。以 A 为源，初始 B=1、C=4、D=∞；选 B 后 C 降为 3、D 降为 6；选 C 后 D 降为 4；最后选 D。到 D 的路径为 A—B—C—D，下一跳 B。暂定 C=4 不是最终结论，不能把初始值当成最终值。

链路状态报文需要序号、老化等机制区别新旧信息，洪泛也不等于把每个收到的副本都无限次重发。与距离向量相比，链路状态通告的是本地连接信息，接收者据拓扑独立计算；二者的 “已知信息” 不同。

4.7 IPv4 首部、MTU 与分片重组

IPv4 分组由首部和数据组成。首部固定部分 20 B，共 5 个 32 bit 行；其后可有选项与填充，首部总长 20～60 B。下图固定部分按位宽比例绘制，可变区单独压缩示意。读字段时先确定它在首部中的位数，再将字段数值按指定单位换成字节量。

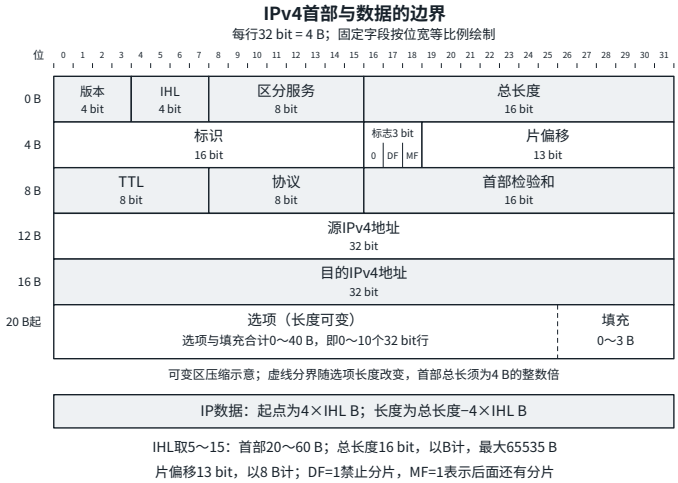


Figure 4.1: IPv4 固定首部五行，每行 32 bit；选项与填充合计 0～40 B，IP 数据在首部之后

版本字段 4 bit，IPv4 取 4；IHL（首部长度）4 bit，合法值为 5～15；区分服务 8 bit，用于携带服务类别与拥塞标记信息。总长度 16 bit，最大可表示 65535 B，包含整个 IP 首部与数据。源、目的 IPv4 地址各 32 bit，分别标识这次通信的两端，也是地址与路由题读取的字段；它们不直接替代当前链路的 MAC 地址。

三种与长度相关的字段尤其容易混用，应固定同一换算顺序：

字段	位数与单位	换成实际字节量	对应的边界
IHL	4 bit，以 4 B 为单位	首部长 $H = 4 \times IHL$	从 IP 首部起点到 IP 数据起点
总长度	16 bit，以 1 B 为单位	IP 数据长 $D =$ 总长度 $- H$	当前这个 IP 分组的首部与数据总和
片偏移	13 bit，以 8 B 为单位	本片数据起点 $O = 8 \times$ 片偏移	相对原始 IP 数据区，排除原始首部与各片首部

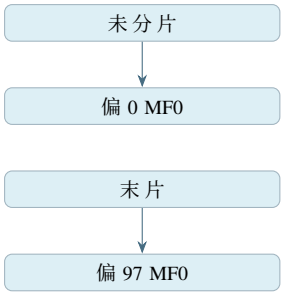
例如 IHL=6、总长度 =100，表示首部 24 B、IP 数据 76 B；不能把 6 直接当字节数。选项结束后只在必要时补 0，使首部达到 4 B 的整数倍，填充也包含在 IHL 所计的首部中。读取原始十六进制首部时，首字节 `0x45` 要拆成版本 4 与 IHL=5，总长度则读取后面的独立 16 bit 字段。

TTL 占 8 bit，实际按路由跳数递减：转发路由器减 1，耗尽时丢弃，并在允许条件下报告超时。协议字段 8 bit，标识承载的数据交给哪个协议，例如 ICMP=1、TCP=6、UDP=17；它是网络层分用依据，目的端口则在传输层继续分用。首部检验和 16 bit，只覆盖 IP 首部（含选项、填充），不覆盖 IP 数据，也不加入伪首部；TTL 等首部内容变化后须更新检验和。IPv4 没有以全 0 字段关闭首部校验的约定。〖4.2-004〗

链路 MTU 限制可承载的完整 IP 分组长度。IPv4 分组大于输出 MTU 且允许分片时，路由器可以拆成多个分片；每片均有自己的 IP 首

最小反例·MF 零不等于未分片

未分片数据报可有偏移 0、MF=0；一个最后分片也可有偏移 97、MF=0。两个状态共享 MF 数值，已足以排除“看到 MF 为 0 就断定没有分片”，必须同时读取片偏移。



MF 同为 0，偏移揭示不同状态

部。DF=1 禁止分片，无法通过时应丢弃并在允许条件下通知源端。重组在目的主机完成，避免中间路由器长期为每份报文保留重组状态。

标识字段 16 bit，用于关联同一原始数据报的各个分片，分片后各片保留其标识；它不是 TCP 式的有序交付字节号。重组还要结合源、目的地址和协议等信息，不能只凭一个 16 bit 标识合并任意分片。3 bit 标志由保留位（0）、DF、MF 组成，DF=1 禁止分片，MF=1 表示原始数据后面还有分片。片偏移以**原始 IP 数据区起点**为基准，单位为 8 B；除最后一片外，数据区长度必须为 8 B 整数倍。MF=0 既可能是未分片分组，也可能是最后一片，要结合偏移判断。

因此分片题应依次读总长度、IHL、DF，再计算每片的数据容量与 8 B 对齐量，最后写回每片的总长度、MF 和片偏移。IPv4 分片只是将 IP 数据切成多个 IP 分组，不会把片偏移变成 TCP 序号，也不会给每片自动增加一个新的 TCP 首部。

不变量·偏移坐标从原数据起

分片的偏移以原始 IP 数据起点为零，单位 8 B，不把任何分片首部累计进去。再次分片也沿用同一原坐标，因此原片的最后子片若后面仍有原数据，MF 仍应保持 1。

分片先守住三个长度边界

原数据长度是原 IP 总长减首部；每片的总长不能超过出口 MTU；非末片的数据长度取不超过 MTU 减首部的最大 8 B 倍数。列出数据区间后，以原数据起点计算偏移，再写 MF。遇到再次分片，仍沿用原数据坐标与原片“后面还有没有数据”的状态。

4.7.1 例 4-2　为何 800 B 链路上的分片总长只有 796 B

例 4.7.1 (配套例题)

配套例题。若路由器向 MTU=800 B 的链路转发一个总长度为 1580 B 的 IP 数据报（首部长度为 20 B）时进行了分片，且每个分片尽可能大，则第 2 个分片的总长度字段和 MF 标志位的值分别是（　）。

A. 796，0　　B. 796，1　　C. 800，0　　D. 800，1

解析。原始数据区为 1560 B，每片至多容纳 780 B 数据，但非末片必须 8 B 对齐，因此取 $\lfloor 780/8 \rfloor \times 8 = 776$ B。三片数据依次为 776、776、8 B，总长度为 796、796、28 B，片偏移为 0、97、194，MF 依次为 1、1、0。第 2 片之后还有 8 B，故选 **B**。

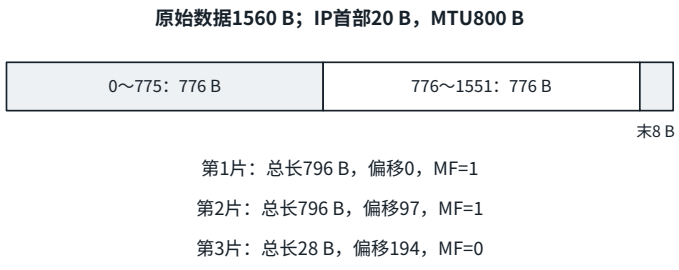


Figure 4.2: 偏移只累计原始数据，第二片从第 776 字节开始

把 800 直接填成总长度会使非末片数据长 780 B，不能用整数 8 B 偏移表达下片起点。重组时也不应把三个首部计入原始数据内容。再次分片时，偏移仍相对原始报文数据区，原分片若 MF=1，其最后子分片也仍为 MF=1。

4.8 子网掩码、CIDR 与可分配地址

错项排除·普通主机式有用途前提

普通多址子网常从地址总数减去网络和广播两个地址；主机路由/32、点到点/31 等用途有其他规则。先判断题设用途，再套主机数公式，不能把二的幂减二写成所有前缀通用定律。

CIDR 把 IPv4 地址写为 `a.b.c.d/p`，前 p 位是网络前缀，其余是该地址块内的偏移。掩码由连续 p 个 1 后接 $32-p$ 个 0 组成。给出主机地址 A 与掩码 M，网络地址为 $A\&M$，广播地址为网络地址加 $2^{32-p}-1$。

判断两个地址是否同网，应把两个地址分别与发送方所用的掩码按位与，再比较结果，不能只比较前三个十进制字节。子网划分可以缩小广播范围、组织路由，却不会凭空增加 IPv4 地址总数；普通子网越多，网络地址与广播地址等开销也可能越多。CIDR 改善分配与聚合，并不能彻底消除 32 位地址空间的限制。〖4.2-021〗〖4.2-024〗

4.8.1 例 4-3　非整字节前缀怎样求地址边界

例 4.8.1 (配套例题)

配套例题。主机 `168.16.84.24/20` 所在子网的最小可分配 IP 地址和最大可分配 IP 地址分别是（　）。

A. 168.16.80.1，168.16.84.254

B. 168.16.80.1，168.16.95.254

C. 168.16.84.1，168.16.84.254

D. 168.16.84.1，168.16.95.254

解析。`/20` 掩码为 255.255.240.0，第三字节以 16 为块长。84 落在 80～95 块内；网络为 168.16.80.0，广播为 168.16.95.255。排除二者，可分配范围为 168.16.80.1 至 168.16.95.254，选 **B**。84 只是主机地址中的第三个字节，不是网络边界。

定长划分从主机位统一借 s 位作子网号，得到 2^s 个等长子网；本书采用现代允许全 0、全 1 子网号的口径，不再无条件减 2。可变长划分 VLSM 要按需求从大到小分配，所分地址块必须在相应前缀边界对齐、彼此不重叠。需要 100 个普通主机地址，至少分 `/25` 的 128 地址块；若再需 50 个，分 `/26`，较大的块也不能从不对齐的地址开始。

4.8.2 例 4-4　“完整划成 5 个子网”限制了最深分支

例 4.8.2 (配套例题)

配套例题。若将 `101.200.16.0/20` 划分为 5 个子网，则可能的最小子网的可分配 IP 地址数是（　）。

A. 126　　B. 254　　C. 510　　D. 1022

解析。这里按整个原地址块被 5 个子网完整分割理解，可把划分看成二叉前缀树。每把一个叶块对半拆成两个子块，叶子数只增加 1。由 1 个块变成 5 个块共能拆 4 次；连续沿一个分支拆 4 次，最深前缀为 `/24`。可分配地址数为 $2^8-2=254$，选 **B**。

例如保留 `101.200.24.0/21`，把另一半继续拆出 `/22`、`/23` 和两个 `/24`，正好 5 块。若允许只从原块中零散取 5 个小块、其余地址未分配，便不能推出这一最小规模；本题“划分”为完整覆盖条件决定了前缀树模型。〖4.2-040〗也应先检查能否用指定总叶子数完整覆盖，而不能仅看两个候选块是否重叠。

特殊地址还需单独判断。`0.0.0.0` 可用于未取得地址时的本机源地址；`255.255.255.255` 为本地受限广播，路由器不转发，默认路由

不能改变这个限制；127/8 用于回环，可在主机内部作源或目的，不应作为外部链路的正常地址。定向广播是某子网主机位全 1 的地址，其跨路由器转发还受安全策略和配置限制，不能据此认为它一定被转发。〖4.2-018〗〖4.2-042〗

最小反例·更具体的出口会胜出

目的 10.1.2.7 同时落在 10/8 与 10.1.2/24 中，即使/8 记录的距离更小，转发仍选/24。先比较匹配前缀长度，才处理同一前缀候选的度量；表项先后顺序也不决定这个结果。

4.9 路由聚合与最长前缀匹配

路由聚合把连续、对齐且具有共同前缀的地址块合成一个更短的前缀，减少表项。若要求**精确聚合**，不能额外覆盖尚不属于该出口的地址；如果题目允许覆盖空隙并用更具体路由纠正，则需按该附加条件讨论。不能只找到任意共同前缀就忽略多覆盖的地址范围。

4.9.1 例 4-5　四个 /21 应缩短几位

例 4.9.1 (配套例题)

配套例题。某路由表中有转发接口相同的 4 条路由表项，其目的网络地址分别为 `35.230.32.0/21`、`35.230.40.0/21`、`35.230.48.0/21` 和 `35.230.56.0/21`，将该 4 条路由聚合后的目的网络地址为（　）。

A. 35.230.0.0/19　　B. 35.230.0.0/20

C. 35.230.32.0/19　　D. 35.230.32.0/20

解析。四个块覆盖第三字节 32～63，共 32 个 `/24`，合计 $4\times 2^{11}=2^{13}$ 个地址，前缀应为 $32-13=19$ 位。32 是 `/19` 对应的块边界，因此精确聚合为 35.230.32.0/19，选 **C**。只把 21 减 1 只能合并两个块；将网络起点写 0 又会改成另一块地址范围。

转发查表时，目的地址可能同时匹配多个前缀，应选**匹配位数最多**的条目。设表中有 `10.0.0.0/8→ 接口 1`、`10.1.0.0/16→ 接口 2`、`10.1.2.0/24→ 接口 3` 和默认 `/0→ 接口 4`，目的 10.1.2.7 走接口 3；10.1.9.7 走接口 2；11.1.2.7 仅匹配默认。先匹配前缀，再读出口；不同前缀长度的匹配项之间，不直接比较度量值大小。

默认路由 `/0` 能匹配所有普通目的，用于没有更具体条目时；它不改变广播、TTL 等协议限制。主机路由 `/32` 只匹配一个地址，比相应网络路由更具体。下一跳 IP 与本机输出接口是不同字段：前者指向邻居，后者确定从本设备哪个口发出。〖4.7-016〗

4.10 NAT：为什么地址变了，传输层校验也要变

网络地址转换 NAT 在边界处改写分组中的地址，并维护内部与外部地址的映射。常见 NAPT 同时利用端口区分多个内部端点，例如 `192.168.1.5:50000` 与 `192.168.1.6:50000` 可映射为同一公网 IP 的两个不同端口。回包到达时按映射表恢复内部目的地址和端口。

对出方向的分组，普通源 NAT 通常修改源 IP，NAPT 还可能修改源端口；目的 IP 与远端服务端口通常不变。IPv4 首部检验和需随地址改变更新；TCP/UDP 校验覆盖的伪首部也包含 IP 地址，因此其校验和可能必须更新。不能因为“路由器主要是网络层设备”就断言它绝不会检查或改写端口。IPv4 UDP 若明确未启用校验，其零字段语义另行处理。

NAT 保存映射状态，不等于给每条连接预留传输带宽；仅靠私有

地址也不会让内部服务器自动对外可见。外部主动发起访问通常需要静态映射、端口转发或其他穿越机制。私有地址主要有 10/8、172.16/12、192.168/16，不能把所有 172 开头地址都归成私有。

不变量·ARP 先确定当前邻居

源主机的第一帧只需交给本地下一跳，ARP 广播也只到当前广播域。即使 IP 最终目的在远端，当前帧目的 MAC 仍可为网关；普通二层交换机转发这帧时，不把源 MAC 换成自己。

4.11 ARP 与逐跳 MAC：最终目的和当前下一跳分开算

主机先用自己的子网掩码判断目的是否在本地。若本地可直接交付，下一跳为目的主机；否则下一跳通常是默认网关或更具体路由指定的邻居。再使用 ARP 查询**下一跳 IP 对应的 MAC**，封装当前以太网帧。不能先按最终目的 IP 向整个互联网广播 ARP。

ARP 请求通常以以太网广播发出，只在当前广播域传播，内含待查询 IP；拥有该 IP 的节点通常单播回复其 MAC。路由器不把普通 ARP 广播照搬到另一子网，代理 ARP 等特定机制需另给条件。ARP 缓存减少重复查询，但并非永久正确，也不是 DNS 那样把域名解析成 IP。

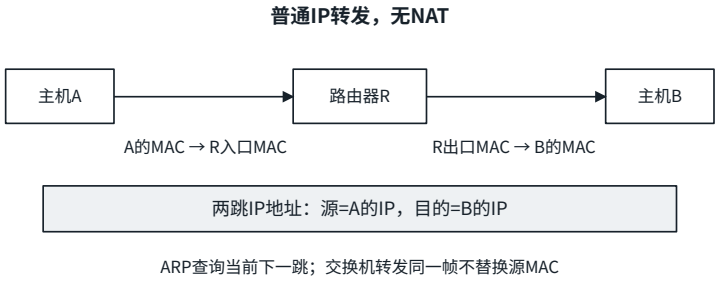


Figure 4.3: 经路由器时 IP 源/目的通常不变，帧的源/目的 MAC 按每跳重封装

在不经过 NAT 的普通转发中，图中 IP 源 A、目的 B 在各跳保持；第一帧目的为路由器入口 MAC，第二帧源为路由器出口 MAC。交换机只转发同一链路层帧时，不把源 MAC 改成交换机的 MAC。

4.11.1 例 4-6　相差几个第三字节，仍可能在同一子网

例 4.11.1 (配套例题)

〖4.2-043〗某以太网中，甲 IP 为 211.71.136.23，子网掩码 255.255.240.0，网关 211.71.136.1。若甲向乙 211.71.130.25 发送 IP 分组，则（　）。

A. 直接发给乙，帧目的 MAC 为网关 MAC

B. 直接发给乙，帧目的 MAC 为乙 MAC

C. 交网关转发，帧目的 MAC 为网关 MAC

D. 交网关转发，帧目的 MAC 为乙 MAC

解析。掩码是 `/20`，136 与 130 按位与 240 均得 128，两地址同属 211.71.128.0/20。应查询乙的 MAC 并直接交付，选 **B**。网关虽然已给出，但本次不需要它。题目给出某个参数，不等于必须使用它。

若同一物理广播域上的两台主机配置了不同掩码，它们对“对方是否本地”的判断可能不一致。接到 ARP 广播仅说明链路广播到达，不能证明双方会按同一 IP 子网路径发回数据；应分别从每个发送方的掩

码重算。这是〖4.2-045〗在网络配置异常条件下增加的判断边界。

错项排除·地址提议尚非地址生效

Offer 提供候选地址，初始选择阶段 Request 表达接受哪份提议，ACK 才确认租约。图中出现候选 IP 并不意味着它已可作客户端 IP 源；题目若问目的、源，还应按设问顺序填写。

4.12 DHCP：请求一个地址不等于已经拥有它

DHCP 自动提供 IP、掩码、默认网关、DNS 服务器和租期等配置。初始获取通常经历 Discover、Offer、Request、ACK 四类交互，承载于 UDP，客户端端口 68、服务器 67。客户端尚无可用地址，初始报文可用源 IP0.0.0.0 与广播目的；处于其他状态的续租请求则可能单播，不能把某一阶段的做法当成全部 DHCP 的固定规则。

Discover 寻找服务器，Offer 提出可用配置；客户端用 Request 表达选定的地址与服务器，收到 ACK 后才按配置使用。Request 在初始选择阶段采用广播，还能让未被选择的服务器知道该地址提议未被采用。跨子网获取配置可由 DHCP 中继帮助转送广播请求，普通路由器默认不会直接转发受限广播。

4.12.1 例 4-7　初始 DHCPREQUEST 的源、目的 IP

例 4.12.1 (配套例题)

配套例题。一台新接入网络的主机 H 通过 DHCP 服务器动态请求 IP 地址过程中，与 DHCP 服务器交换 DHCP 报文的部分过程如下图。封装 DHCPREQUEST 报文的 IP 数据报的目的 IP 地址和源 IP 地址分别是（　）。

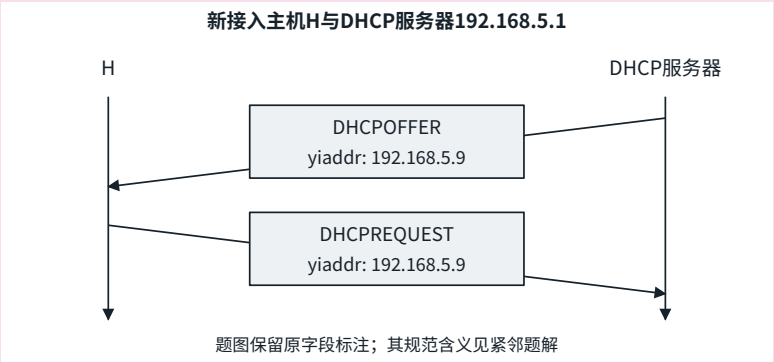


Figure 4.4: 原题的两个报文及字段标记；时间向下

A. 192.168.5.1，0.0.0.0

B. 192.168.5.1，192.168.5.9

C. 255.255.255.255，0.0.0.0

D. 255.255.255.255，192.168.5.9

解析。H 仅收到地址提议，尚未获得 192.168.5.9 的使用确认。初始选择阶段 Request 使用源 0.0.0.0、目的 255.255.255.255，按“目的、源”的设问顺序选 **C**。服务器的单播地址不是这一步的目的 IP 地址。

图中 Request 框沿用原题的 `yiaddr` 标注。按 DHCP 规范，客户端此阶段申请的地址实际由 Requested IP Address 选项携带，客户端发出的 `yiaddr` 为 0。图中的 192.168.5.9 应理解为待申请地址信息，不能据这个标注推出 H 已经能用它作 IP 源地址。Offer 中的 `yiaddr` 才是服务器提出的地址。

4.13 ICMP：报告差错不等于替原分组重传

错项排除·差错报告不是可靠重传

ICMP 把某次网络问题告知原源端，不负责替所有应用重发丢失分组。差错之间还有限制以防反馈放大；历史源点抑制已被废弃，不能把旧类型识别题的答案当成当前拥塞处理建议。

ICMP 封装在 IP 数据报中，IPv4 协议字段为 1。目的不可达、时间超过、参数问题等差错信息帮助源端诊断；回送请求/应答支持连通性探测。ICMP 不把 IP 变成可靠传输协议，不负责代替源端自动重发每份丢失数据。

ICMP 差错一般携带引发问题的 IP 首部和一部分数据，便于源端识别；返回目标是原分组的源端，并不向路径上的每个路由器广播。对 ICMP 差错报文本身通常不再产生 ICMP 差错；对非首片、广播/组播目的及某些特殊源地址等也有限制，防止差错相互放大。首部检验和损坏的 IP 分组通常直接丢弃；其中的源地址已不可靠，不能据以发送差错报告。

直接封装顺序为“链路帧—IP—ICMP”。说“ICMP 最终也由链路帧承载”可以，但不能据此跳过 IP 而把 ICMP 归为直接封装在以太网中的网络邻居协议。〖4.2-048〗

4.13.1 例 4-8　历史 ICMP 题要保留协议年代

例 4.13.1 (配套例题)

配套例题。若路由器 R 因为拥塞丢弃 IP 分组，则此时 R 可向发出该 IP 分组的源主机发送的 ICMP 报文类型是（　）。

A. 路由重定向　B. 目的不可达　C. 源点抑制　D. 超时

解析。依当年协议背景，源点抑制用于通知源端降低速率，选 **C**。它后来已被 RFC 6633 废弃，当前实现不应继续把发送源点抑制作为拥塞控制建议。本题用于辨认历史类型；现代拥塞处理应结合丢包、ECN 等适用机制讨论，不能把旧题答案直接当现行工程操作。

Ping 常用 ICMP Echo，Traceroute 借助逐步增大的 TTL 使沿途返回超时信息；具体探测可使用 UDP、ICMP 或 TCP 等变体。看到超时回复，说明探测分组到达某跳并触发限制，不等于所有数据路径都完全对称。

4.14 IPv6：基本首部固定，不代表没有扩展

IPv6 地址 128 bit，写成 8 组 16 位十六进制数。每组可省前导 0，一处连续的全 0 组可用 `::` 压缩，整串最多出现一次 `::`，否则无法唯一还原。`2001:0db8:0000:0000:0000:0000:0000:0001` 可写成 `2001:db8::1`；省略冒号两侧有效数字或多次使用 `::` 都会改变含义。

IPv6 基本首部固定 40 B，包含版本、流量类别、流标签、有效载荷长度、下一首部、跳数限制和源/目的地址。扩展首部在需要时另接，因此固定的是**基本首部**，不是所有扩展字段的总长。没有 IPv4 式首部检验和，不能由此认定它完全没有链路和端到端检错。

IPv6 路由器转发分组时不分片。分组过大时丢弃，并在适用条件下发 ICMPv6 Packet Too Big，源端据路径 MTU 调整；源端必要时用分片扩展首部处理。换一条任意大 MTU 链路转发不是替代既定路由规则的通用办法。IPv6 无广播地址，用组播等完成相应功能，邻居发现使用 ICMPv6，不能仍以 ARP 作为原生 IPv6 解析机制。

比较维度	IPv4	IPv6
地址长度	32 bit	128 bit
首部长度与长度字段	首部 20～60 B；总长度计首部和数据	基本首部 40 B；有效载荷长度不计这 40 B，计其后的扩展首部和数据
网络层首部检验和	有，只校验 IPv4 首部	基本首部没有该字段
转发时遇到过大分组	允许时可由路由器分片；DF=1 时不能分片	路由器不分片，通知源端调整
本地邻居与广播	ARP 解析下一跳；有受限/定向广播等地址	ICMPv6 邻居发现；无广播地址

表中比较的是两种 IP 机制。双栈、隧道属于过渡方式，NAT 属于地址转换，不能把这些机制理解成 “IPv6 自动把每个 IPv4 字段扩大四倍”。下题分别检查地址空间、基本首部和转发限制。

错项排除·原下一跳变差也得更新

本机原路由经 B 到 N 为 3 跳，B 现在通告 5 跳，经 B 候选就变为 6。继续保留旧 3 会把已失效的信息当成更优路线；只有来自不同下一跳的更差候选，才可按通常规则忽略。

4.14.1 例 4-9　地址位数增加 96，不是地址数乘 96

例 4.14.1 (配套例题)

配套例题。下列关于 IPv4 和 IPv6 的叙述中，正确的是（　）。

I. IPv6 地址空间是 IPv4 地址空间的 96 倍

II. IPv4 首部和 IPv6 基本首部的长度均可变

III. IPv4 向 IPv6 过渡可以采用双协议栈和隧道技术

IV. IPv6 首部的 Hop Limit 字段等价于 IPv4 首部的 TTL 字段

A. 仅 I、II　　B. 仅 I、IV　　C. 仅 II、III　　D. 仅 III、IV

解析。地址空间比例为 $2^{128}/2^{32}=2^{96}$，I 错误；IPv6 基本首部固定，II 错误；双栈让节点同时支持两类协议，隧道把一类分组封装在另一类分组中跨越对应网络，III 正确；Hop Limit 与按跳递减的 TTL 作用相同，IV 正确，选 **D**。双栈和隧道也不意味着普通路由器可随意转换任意网络协议。

4.15 自治系统：域内最短与域间策略分开

自治系统 AS 是在统一管理和路由策略下的一组网络与路由器。域内路由协议 IGP 处理 AS 内部可达性，典型有 RIP、OSPF；域间路由协议处理 AS 之间信息，互联网主要使用 BGP。AS 不是一个 IP 子网，也不要求只有一台出口路由器。

一个分组可以先按本 AS 的域内路径到达出口，再按域间策略进入另一个 AS。IGP 的代价最短与 BGP 的政策选路标准不同，不能拿 RIP 跳数直接和 BGP 的 AS 路径长度混在一张 “最小值表” 里比较。

4.16 RIP：加 1 更新，16 表示不可达

RIP 采用距离向量，度量为跳数，最大可用距离 15，16 表示不可达；消息常通过 UDP 端口 520 传送。收到邻居表项，先把候选距离加 1 并把下一跳改为该邻居，再与本表比较。

若目的未登记，按候选加入；候选更优时替换；若原表下一跳就是该邻居，则即使距离变差，也应采用该邻居新的距离，以反映原路径变化；若不同邻居报告较差路径，通常不替换。具体等价路径保留规则可按题意处理。更新并不需要等所有邻居回复确认后才进行。〖4.4-012〗

例如本机到 N 原为 3 跳、下一跳 B，B 新报 N 为 5，候选 6，应更新到 6 而不是坚持旧 3；若 C 报 2，候选 3，存在可替代的 3 跳路径。邻居报 16 时经它仍不可达，不保留 17 作有效距离。RIP 把无穷取小有助于限制计数时间，也限制可用网络规模；“跳数少” 不保证带宽更大或传播时延更短。

不变量·BGP 策略先于简单距离

AS_PATH 帮助抑制 AS 环并参与选路，真实决策还受本地优先级等属性约束。只有题目明确其余策略属性相同，才可按路径长度及到出口的域内代价进一步比较，不能固定背成 “AS 少就选”。

4.17 OSPF：区域怎样限制拓扑细节

OSPF 采用链路状态方法，基于 IP 直接承载，协议号 89，不使用 TCP 或 UDP 端口。Hello 发现并维持邻居关系；相邻路由器同步链路状态信息，形成数据库，再运行最短路径优先计算。Hello 与 Keepalive 都用于维持邻居或会话关系，但分属 OSPF 与 BGP，不能互换。〖4.4-015〗

OSPF 可把 AS 分成多个区域，区域间通信经骨干区域转发。区域内共享较完整的该区域拓扑，跨区主要使用汇总可达信息，降低每台路由器需要掌握的拓扑规模。区域边界路由器参与相邻区域与骨干的信息处理，不应简单说 “任何路由器只知道一个区域”。

区域划分减少洪泛和计算范围，但增加区域类型、边界通告等机制，不能因此称协议结构一定更简单。〖4.4-006〗中 “局部信息更少” 与 “整个协议更简单” 是两个判断。

4.18 BGP：AS_PATH 是路径向量，不是链路状态

BGP 在自治系统之间交换可达前缀和路径属性，使用 TCP 端口 179。AS_PATH 记录通告所经过的 AS 序列，帮助检测 AS 环路并执行策略；NEXT_HOP 给出通告路由的下一跳。BGP 不把全互联网每条物理链路的代价都洪泛给每个节点。

收到路径含本 AS 号的通告通常应拒绝，以免形成 AS 环；对其余候选还要执行本地策略，不能只按 AS_PATH 长度作唯一决策。对等连接成功也不等于对方通告的一切路径都应被接受。

RIP、OSPF、BGP 分别对应距离向量、链路状态、路径向量；本机邻居是谁、知道哪些信息、按何种规则选择，是比较的共同维度。BGP 虽然有 NEXT_HOP 属性，但其可达信息不能仅概括成 “到某网的下一跳”。〖4.4-019〗〖4.4-020〗

比较维度	RIP	OSPF	BGP
主要管理范围	AS 内部	AS 内部，可再划区域	AS 之间的可达性与策略
交换的核心信息	到目的网络的距离向量	本地链路状态，在规定范围内洪泛	可达前缀及路径属性，含 AS_PATH
接收者如何选路	邻居通告距离加本地一跳，比较候选	根据拓扑数据库计算最短路径树	根据路径属性与本地策略选择可接受路径
更新的主要特点	周期通告，并可触发更新；坏消息可能慢收敛	变化触发链路状态更新，还需刷新与老化	与对等体建立会话后按变化增量通告，不周期发送整张路由表
报文直接承载	UDP，端口 520	IP，协议号 89	TCP，端口 179

“交换了哪些信息” 和 “这些报文如何被传送” 是两个判断维度。BGP 使用 TCP 并没有把选路准则变成 TCP 窗口；OSPF 虽然直接由 IP 承载，但仍要维护邻居关系、管理通告的可靠性。路径算好以后，普通 IP 分组仍按转发表的最长前缀匹配转发，不为每个分组重新运行一次 RIP、Dijkstra 或 BGP 策略选择。

4.19 IP 组播：分支复制放在哪里

课程连接·成员管理与转发树分工

IGMP 像维护当前接收者集合，组播路由机制再按网络结构组织复制分支。知道本地有人加入，并不等于已经知道远端所有路径；TTL 只能限制错误环流寿命，不能替代合理的树结构。

组播把数据交给一个接收者组。源端发送一份，路由器在需要分叉的接口复制，使公共路径上的数据只传输一次。若改用多次单播，则要为每个接收者单独发送，公共链路可能重复承载相同内容。时延是否相同还取决于并发、排队和树结构，不能保证用多个单播替代后时延不变。〖4.5-001〗

IPv4 组播地址为 224.0.0.0/4，不用作普通单播接口地址。映射到以太网组播 MAC 时，只取 IP 低 23 位，放入 01:00:5E 后相应位置，因此多个 IP 组播地址会映射到同一 MAC。链路层收到该帧，不等于本机加入了对应的 IP 组播组，还需上层检查。

IGMP 用于在主机与相邻组播路由器之间管理本地成员关系，不能单独建立跨网络的组播路由树。组播路由协议组织无环转发树、选择正确分支，防止重复和环路；TTL 虽能最终限制环流寿命，却不能代替构造合理转发结构。〖4.5-002〗

4.20 移动 IP：位置改变后，固定地址怎样仍能被找到

移动 IPv4 保留移动节点的归属地址，使用当前访问网络的**转交地址**描述它现在的位置。归属代理维护两者的绑定；有外部代理时可由其提供转交地址和转送服务，也可采用节点自有转交地址。注册的目标是让归属代理知道新的可达位置，不是只让当前外部代理知道本机存在。

传统三角路由中，通信对端按归属地址发送；分组先到归属网络，由归属代理截获，再封装为隧道分组送到转交地址，解封装后交给移动节点。外层目的为转交地址，内层目的仍是归属地址，不能把这两个目的 IP 当成一次普通逐跳改写。

移动节点向外发送时，基础模型可直接经过被访网络路由器发往对端，回程却仍按归属地址经过归属代理。实际部署若存在源地址过滤等限制，可使用反向隧道；因此 “可直接发送” 要限定为该基础模型。〖4.6-001〗〖4.6-003〗分别考查注册与方向，要分两步判断，不能以 “移动后还能发” 为由推断回包也直接到新位置。

4.21 路由器内部：查表后为何仍可能排队

输入端口接收信号、处理链路帧并查转发表，交换结构把分组移向相应输出，输出端口排队并重新封装发送。路由处理器负责控制协议和表项维护，正常高速转发可由专用硬件完成。交换结构可基于内存、总线或互联网络，各自共享资源与并行程度不同。

即使输入查表很快，两个输入端口同时要求同一输出时，也不能都以满速把分组发到这条链路上；可能在输入、交换结构或输出端排队。输入队列若队首分组等待繁忙输出，后面本可去空闲输出的分组也可能被挡住，形成队首阻塞。输出排队则受该出口服务速率约束，缓存满时按规则丢弃。

逐包转发可按以下顺序检查：是否完整收到且链路检错通过；目的是否本机；TTL 等是否允许继续；最长前缀匹配得到哪个出口/下一跳；是否需要分片或报告过大；怎样解析下一跳链路地址并封装；出口什么时候空闲。题目给定的直通等特殊实现可调整开始转发的时刻，但不能省略这些逻辑条件。

把一次转发分成可追踪的状态

下面只表示普通 IPv4 路由转发的检查顺序，题目给定特殊实现时另加条件。

```
接收当前链路帧，检查后取出IP分组
若目的为本机：按协议字段上交
否则检查TTL；查最长匹配前缀，确定出口与下一跳
按出口MTU与DF判断转发、分片或报告差错
查询下一跳链路地址；在出口排队并重新封装
```

NAT 若存在，应按题设在相应位置执行地址/端口与检验和更新。路由选择负责形成表，以上查表没有为每份分组重跑全网最短路。

不变量·内外层各有自己的目的

移动节点的归属地址保持身份，转交地址描述当前位置。隧道外层把分组送到当前访问网络，内层仍保留原目的；将二者画成两个封装层，就不会误以为每一跳都把原目的 IP 改写。

4.21.1 例 4-10　等分子网后，主机余量与分片分别计算

例 4.21.1 (配套例题)

配套例题。某公司的网络如下图所示。IP 地址空间 192.168.1.0/24 被均分给销售部和技术部两个子网，并已分别为部分主机和路由器接口分配了 IP 地址。销售部子网的 MTU=1500 B，技术部子网的 MTU=800 B。

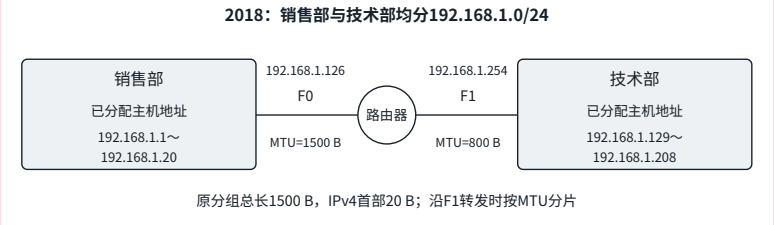


Figure 4.5: 销售部已分配 192.168.1.1～20，技术部已分配 192.168.1.129～208；F0 为.126，F1 为.254

（1）销售部子网的广播地址是什么？技术部子网的子网地址是什么？若每个主机仅分配一个 IP 地址，则技术部子网还可以连接多少台主机?

（2）主机 192.168.1.1 向主机 192.168.1.208 发送一个总长度为 1500 B、首部长度为 20 B 的 IP 分组。路由器通过 F1 转发时进行分片，且尽可能分为最大片。一个最大 IP 分片封装数据的字节数是多少？至少需要几个分片？每个分片的片偏移量是多少?

解析。“均分一个/24”先确定两个/25，每个块有 128 个地址，再利用已经分配的地址识别哪一块属于哪个部门。销售部.1～.20 在 192.168.1.0/25 内，广播地址为 **192.168.1.127**；技术部.129～.208 在 **192.168.1.128/25** 内。技术部可用地址有 $128-2=126$ 个，已分配主机数为 $208-129+1=80$，F1 的.254 还占 1 个，因此还可连接 $126-80-1=$ **45** 台主机。路由器接口不是额外的一台用户主机，但会占用同一子网的可用地址。

分片计算从原 IP 数据部分开始：$1500-20=1480$ B。F1 所在链路每片总长不能超过 800 B，扣首部后最多 780 B；非末片数据长度又必须是 8 B 的整数倍，故最大数据量为 $\lfloor 780/8 \rfloor \times 8 =$ **776** B。余下

$1480 - 776 = 704$ B，一片即可容纳，所以最少 2 片。

分片	数据长度/B	IP 总长度/B	片偏移（8 B 单位）与 MF
第一片	776	796	偏移 0，MF=1
第二片	704	724	偏移 97，MF=0

第二片偏移是 $776/8 = 97$，它累计的是原数据的位置，不加第一片首部。末片不要求数据长度继续凑满 776 B，所有片的数据长度相加仍为 1480 B。

课程连接·成员管理与转发树分工

IGMP 像维护当前接收者集合，组播路由机制再按网络结构组织复制分支。知道本地有人加入，并不等于已经知道远端所有路径；TTL 只能限制错误环流寿命，不能替代合理的树结构。

4.21.2 例 4-11　覆盖性聚合必须保留更具体的直连例外

例 4.21.2 (配套例题)

配套例题。Internet 的两个自治系统构成的网络如下图所示。AS1 由 R1 连接两个子网构成；AS2 由 R2、R3 互联并连接三个子网构成。各子网地址、R2 的接口名以及 R1、R3 的部分接口 IP 地址均在图中标出。

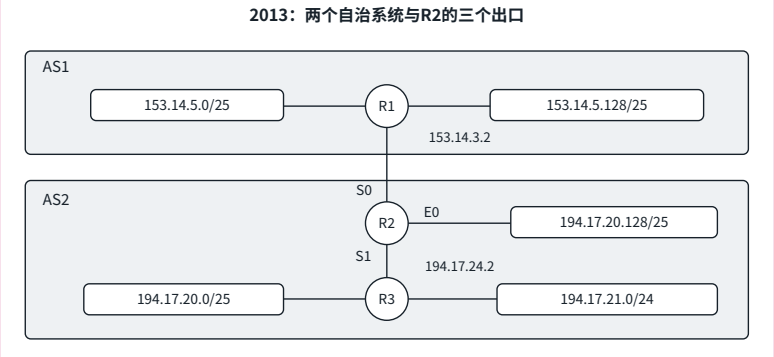


Figure 4.6: R2 经 S0 到 R1、经 S1 到 R3，E0 直连 194.17.20.128/25

（1）路由表结构为“目的网络、下一跳、接口”。利用路由聚合技术给出 R2 的路由表，要求包括到达图中所有子网的路由，且路由项尽可能少。

（2）若 R2 收到目的 IP 地址为 194.17.20.200 的 IP 分组，会通过哪个接口转发?

（3）R1 与 R2 之间利用哪个路由协议交换路由信息？该协议报文封装到哪个协议的数据单元中传输?

解析。解题入手点是先按出口分组，而不是把所有地址不加区别地取共同前缀。经 S0 到达的 153.14.5.0/25、153.14.5.128/25 恰好拼成 153.14.5.0/24，下一跳为 R1 的 153.14.3.2。经 S1 到达的 194.17.20.0/25 与 194.17.21.0/24 可用 194.17.20.0/23 覆盖，但这个/23 也包含 E0 直连的 194.17.20.128/25，必须保留该更具体条目，让最长前缀匹配处理例外。

目的网络	下一跳	接口
153.14.5.0/24	153.14.3.2	S0
194.17.20.0/23	194.17.24.2	S1
194.17.20.128/25	—（直连）	E0

这 3 条分别承担三个不同出口，已不能在保持全部转发结果的前提下继续合并。这里的第二条是“较宽覆盖加更具体例外”，不是声称

S1 后的两个原子网恰好无空隙地拼成一个/23。若删除 E0 例外，原本应直达的分组就会误发给 R3。

194.17.20.200 同时匹配 `194.17.20.0/23` 和 `194.17.20.128/25`；比较前缀长度 25 与 23，选择 **E0**，不依赖表项书写顺序。R1 与 R2 属于不同自治系统，因此交换域间路由使用 **BGP（BGP-4）**，BGP 报文由 **TCP** 承载。自治系统内部使用何种 IGP，不改变这条跨 AS 邻接的协议选择。

不变量·内外层各有自己的目的

移动节点的归属地址保持身份，转交地址描述当前位置。隧道外层把分组送到当前访问网络，内层仍保留原目的；将二者画成两个封装层，就不会误以为每一跳都把原目的 IP 改写。

4.21.3 例 4-12　RIP 逐轮传播与 BGP 出口选择是两条过程

例 4.21.3 (配套例题)

配套例题（改编）。　网络空间是继陆海空天之后的“第五疆域”，路由算法与协议是网络核心技术之一。现有互联网中 4 个自治系统的互连拓扑如下图。AS1 运行 RIP；AS3 内任意两台主机通信经过的路由器不超过 15 个，AS4 内可能超过 20 个。本题采用下图拓扑：**R13 与 R16 不直接相连**。

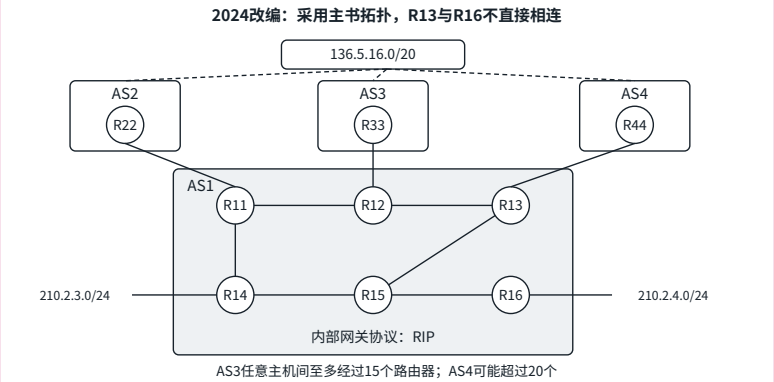


Figure 4.7: AS1 内仅有 R11—R12、R12—R13、R11—R14、R14—R15、R15—R16、R15—R13 六条路由器间链路

（1）若仅有 RIP、OSPF 可选，AS4 应选择哪个内部网关协议？

（2）AS3 中的某主机向本 AS 另一主机发送一个 IP 分组，为保证正常接收，初始 TTL 至少应为多少？

（3）AS1 的路由器同时启动，立即构建并交换初始距离向量，此后每隔 30 s 交换一次最新距离向量。从交换初始向量的时刻算起，R11～R16 全部获得到达 210.2.3.0/24 的正确路由至少需多久？全部获得到达 210.2.4.0/24 的正确路由至少需多久？

（4）R44 向 R13 通告 136.5.16.0/20 的路由，由哪类 BGP 会话完成、通过哪种 BGP 报文通告？R13 又通过哪类会话将该可达性信息通告给 R14、R15？

（5）R14、R15 均收到下表可达性信息。在无策略约束的情况下，更新后两者到 136.5.16.0/20 的下一跳各是谁？

目的网络	AS 路径	下一跳
136.5.16.0/20	AS2、AS8、AS19	R11
136.5.16.0/20	AS3、AS7、AS11、AS19	R12
136.5.16.0/20	AS4、AS10、AS19	R13

解析。第（1）问先检查协议的适用上限。RIP 最大有效跳数为 15，

16 即不可达，AS4 的规模可能超出这一限制，应选 **OSPF**。第（2）问问的是 IP 的 TTL 而非 RIP 度量：每经过一个路由器 TTL 减 1，减至 0 便被丢弃。最多经过 15 个路由器后仍须至少剩 1，故初始 TTL 至少为 **16**。

第（3）问把每次交换视作一轮：本轮只能把交换前已知的路由传给相邻路由器，不能让新收到的信息在同一轮连续穿越全网。又因为首次交换定义为 $t = 0$，直连路由在此刻已经传给一跳邻居，不能额外再计一个 30 s。

距首次交换的时间	已正确获知 210.2.3.0/24 的路由器	已正确获知 210.2.4.0/24 的路由器
0 s	R14、R11、R15	R16、R15
30 s	R11～R16 全部	R16、R15、R14、R13
60 s	R11～R16 全部	R11～R16 全部

210.2.3.0/24 直连 R14，首次传给 R11、R15，下一轮由它们传给 R12、R13、R16，所以需 **30 s**。210.2.4.0/24 直连 R16，首次仅传给 R15，30 s 时到 R14、R13，60 s 时才到 R11、R12，所以需 **60 s**。逐轮集合依赖题图中实际存在的边，不能擅自补上 R13—R16。

第（4）问按会话两端是否同属一个 AS 区分：R44 与 R13 分属 AS4、AS1，用 **eBGP** 会话，通过 **UPDATE** 报文通告；R13 向同属 AS1 的 R14、R15 传播可达性信息，用 **iBGP** 会话。BGP 会话不要求所有对端在物理上直接相连。

第（5）问按本题无策略约束的教材选路模型，先比较 AS_PATH：经 R11、R13 的路径各含 3 个 AS，经 R12 含 4 个，先排除 R12。在其余属性相同的候选中，再比较 AS 内到出口的距离。R14 到 R11 为 1 跳，到 R13 需经 R15 为 2 跳，选 **R11**；R15 到 R13 为 1 跳，到 R11 为 2 跳，选 **R13**。真实 BGP 还受本地优先级等属性影响，本题给定条件并不支持把 “AS 数少、出口近” 当成无条件覆盖所有策略的规则。

课程连接·成员管理与转发树分工

IGMP 像维护当前接收者集合，组播路由机制再按网络结构组织复制分支。知道本地有人加入，并不等于已经知道远端所有路径；TTL 只能限制错误环流寿命，不能替代合理的树结构。

4.21.4 例 4-13　卫星链路：瓶颈速率、窗口与 VLSM 各有约束

例 4.21.4 (配套例题)

配套例题。某公司工程部长期驻扎偏远山区，租用天通一号卫星通信链路连接工程部 R1 和总部 R2，网络如下图。S1、S2 为千兆以太网交换机；TR1、TR2 是卫星地面收发设备，实现全双工调制解调。卫星轨道高度 36000 km，电磁波传播速度 300000 km/s；卫星链路为 R1、R2 提供对称全双工信道，每方向数据传输速率 200 kb/s。

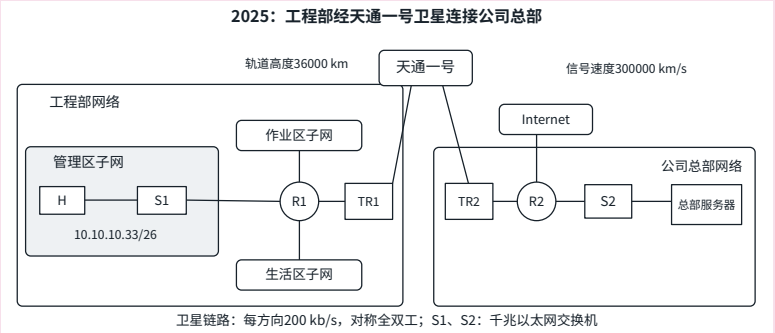


Figure 4.8: H 配置 10.10.10.33/26 并位于管理区；工程部还有作业区、生活区两个子网

（1）忽略卫星中继及 TR1、TR2 调制解调开销，卫星链路单向传播时延是多少？H 向总部服务器传输数据可达到的最大吞吐量是多少？忽略各层首部开销与以太网内传播时延，上传 4000 B 工程进度报告文件至少需要多久？

（2）用 GBN 为卫星链路设计单向可靠的数据链路层协议 SLP，支持 R1 向 R2 发送数据。SLP 数据帧长 1500 B，忽略 ACK 帧长度；要求单向信道利用率不低于 80%，发送窗口至少为多少？帧序号字段至少需要几位？

（3）总部分配 10.10.10.0/24 给工程部，需划给三个子网：生活区可分配地址不少于 120 个，作业区、管理区分别不少于 60 个，H 的地址配置正确。作业区、管理区、生活区的子网地址分别是什么？请用 CIDR 表示。

辨析·分类与 CIDR

分类编址按地址前缀判断类别；CIDR 按明确前缀长度解释，不能混用默认掩码。

解析。单向路径要经过“地面 → 卫星 → 地面”两段。按本题采用的几何模型，每段链路的长度取卫星高度，单向传播时延 $T_p = 2 \times 36000/300000 = \mathbf{0.24\ s}$。路径吞吐量受最慢链路限制，千兆以太网不能抬高 200 kb/s 的卫星瓶颈，因此 H 单方向最大吞吐量为 **200 kb/s**；全双工表示反方向另有同样容量，不能在一次上传中相加成 400 kb/s。

4000 B 经卫星链路发送需 $4000 \times 8/200000 = 0.16$ s。首部忽略时，最后一比特从卫星链路发送端到接收端还要传播 0.24 s，故教材模型给出的最短上传时间为 $0.16 + 0.24 = \mathbf{0.40\ s}$。这个 400 ms 还额外忽略了局域网的发送开销；题干给出的忽略项并不包含千兆链路上的发送与分帧，若要严格计入每段链路上的发送、分帧及存储转发，还需题目进一步指定相应条件。例如 4000 B 在一条 1 Gb/s 链路上仅串行发送也需 32 μs。因此 400 ms 适合作为本题理想下界，不能视为所有实际以太网转发方式下的精确完成时间。

SLP 每帧发送时延 $T_d = 1500 \times 8/200000 = 60$ ms；ACK 长度忽略，从开始发送首帧到收到其确认需 $60 + 2 \times 240 = 540$ ms。设窗口为 W，利用率 $U = \min(1, 60W/540)$；要求 $U \geq 0.8$，得到 $W \geq 7.2$，向上取整为 **8 帧**。GBN 接收窗口为 1，需 $2^k \geq W + 1 = 9$，所以序号字段最少 **4 bit**。3 bit 虽然有 8 个序号，却只能支持最大 7 帧的 GBN 发送窗口；其利用率 $7/9 \approx 77.78\%$，达不到要求。

最后按可用主机数决定块大小。生活区需 $2^7 - 2 = 126$ 个可用地址，至少占/25；另两区各需 $2^6 - 2 = 62$ 个，至少占/26。三块恰好用完 $128 + 64 + 64 = 256$ 个地址。H 的 10.10.10.33/26 锁定管理区为 10.10.10.0/26；生活区的/25 必须按 128 地址边界对齐，且不能与管理区重叠，只能取 10.10.10.128/25，留下 10.10.10.64/26 给作业区。

子网	CIDR 地址	可用地址范围	可用数量
作业区	**10.10.10.64/26**	10.10.10.65～10.10.10.126	62
管理区	**10.10.10.0/26**	10.10.10.1～10.10.10.62	62
生活区	**10.10.10.128/25**	10.10.10.129～10.10.10.254	126

4.22 分类地址、特殊地址与逐级划分

历史分类编址从地址前缀判断网络号长度：A 类首位 0、默认前缀 8 位；B 类首两位 10、默认前缀 16 位；C 类首三位 110、默认前缀 24 位。首字节分别位于 1～126、128～191、192～223 的普通单播类别，0 和 127 另有用途。D 类前四位 1110 即 224.0.0.0/4 为组播；E 类为保留类别。CIDR

按显式前缀长度解释地址，不能再由首字节机械决定掩码。

普通广播子网中，主机部分全 0 表示该网络，全 1 表示定向广播，主机号占 h 位时通常可分配 $2^h - 2$ 个地址。路由器接口也占用其中的地址；若题目另给点对点特殊编址，应按其规则计算。受限广播 255.255.255.255 只用于本地范围；0.0.0.0 可在主机尚未获地址时作源地址；127.0.0.0/8 用于环回，不作为链路上普通主机的可路由地址。

私有 IPv4 地址段为 10.0.0.0/8、172.16.0.0/12、192.168.0.0/16。它们在不同私网中可以重复使用，访问公网通常结合 NAT、代理或其他出口机制。NAT 按映射改写地址，NAPT 还按端口区分多个内部会话；公网端不能只凭一个共享公网地址就唯一确定内网进程。NAT 表项、端口和回程处理须保持同一连接方向，涉及 TCP/UDP 伪首部时也要更新校验和。

方法·先需求后对齐

把主机与网关需求合计，再选择足够大的 2 次幂地址块，并检查边界对齐。

4.22.1 用一个地址块同时满足不同主机需求

现有 128.14.32.128/27 共 32 个地址。网吧需要 12 台主机，再加网关 1 个，普通子网至少需要 14 个可用位置，可分配 128.14.32.128/28：网络.128、广播.143，主机.129～.142。家庭 5 台主机加网关 1 个，正好使用 128.14.32.144/29 的 6 个可用地址.145～.150，广播.151。点对点的主机和路由器采用普通`/30`模型，分配 128.14.32.152/30，可用.153、.154，广播.155。剩余.156～.159 组成另一个`/30`。

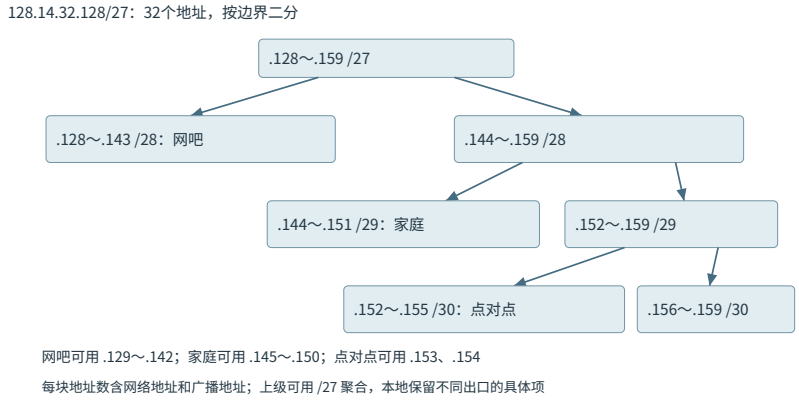


Figure 4.9: 同一/27 逐级二分为/28、/29 与两个/30，块边界与地址范围对应

这些子块都按其大小边界对齐，且互不重叠。上级路由器可以用 128.14.32.128/27 概括这批地址；本地路由器仍须保留指向不同出口的更具体子网项。概括地址包括尚未分配的.156/30，转发时要结合更具体路由、默认路由及丢弃策略避免形成环路。最长前缀匹配匹配的是前缀长度，不是表项顺序或数字大小。

4.23 OSPF 的数据库同步与五种分组

LSDB 是链路状态数据库，LSA 是其中的一份链路状态通告，LSA 头包含用来辨认通告及其新旧程度的信息。OSPF 分组承载这些信息，不能把 “LSA” 与 “LSAck 分组” 混为同一个缩写。

类型值	分组	一次同步中的作用
1	Hello 问候	发现和维持邻居，检查建立关系所需的参数
2	DD 数据库描述	发送 LSDB 摘要，即相应 LSA 的头信息
3	LSR 链路状态请求	对照摘要，请求缺少或更新的 LSA
4	LSU 链路状态更新	发送所需 LSA 的具体内容，也可洪泛变化
5	LSAck 链路状态确认	确认已收到哪些 LSA，支持可靠洪泛

同步顺序可理解为 “认识邻居—交换目录—索取缺项—传送正文—确认收到”。这是一条教学依赖链，不要求每次稳定运行都重新同步完整数据库；收到重复 LSU 时仍可能需要确认，但应避免无限重复洪泛。Hello 间隔和失效时间由网络类型与配置决定；课件示例每 10 s 问候、40 s 超时，是该示例的参数。

变化的链路信息在规定范围内洪泛，使相关路由器更新数据库，再独立运行 Dijkstra 计算最短路径树。区域限制拓扑细节和洪泛范围；区域间、外部路由的通告类型和传播范围按 OSPF 规则处理，不能把 “每个区域有自己的详细数据库” 理解为跨区完全不交换可达信息。

条件·转发表还需递归解析

BGP 的 NEXT_HOP 未必是本地邻居，要通过 IGP 确定实际出口。

4.24 RIP 坏消息与 BGP 会话的区别

RIP 直连网络距离按本章教材计 1，每跨过一个路由器再加 1；周期更新通常 30 s，课件失效判断示例为 180 s。触发更新在发现变化时尽快通告，水平分割和毒性逆转用于减少特定回路，但距离向量仍须警惕计数到无穷。若 A 到网络 N 的旧路径经 B，而 B 又把从 A 学到的路径报回 A，双方可能逐次把距离加大；不能因每次都选择收到的最小值，就断定网络中绝无环路。

BGP 通过 TCP 建立会话。不同 AS 之间的会话为 eBGP，同一 AS 内传播 BGP 信息的会话为 iBGP。基本完整网状 iBGP 用于避免同一 AS 内继续转发 iBGP 学习路由所形成的问题，实际规模化部署还可用路由反射器等机制。iBGP 邻居不必物理直连，AS_PATH 也不是内部逐台路由器的名单。

BGP 基本报文有 OPEN、UPDATE、KEEPALIVE、NOTIFICATION。OPEN 在 TCP 连接成功后交换会话参数；UPDATE 发布或撤销路由；KEEPALIVE 维持会话；NOTIFICATION 报告错误并结束会话。变化通告与保活是不同任务，不能把 KEEPALIVE 理解为周期重发全部路由。

选路先执行题目给定的策略。例如课件简化次序依次比较本地偏好、AS 路径长度、到出口的域内代价、BGP 标识符；真实 BGP 还有其他属性与规则。经 iBGP 得到的 NEXT_HOP 未必等于本机实际转发表中的下一跳，仍须借助 IGP 找到通往该 BGP 下一跳的本地出口。

4.25 IPv6 字段、地址与自动配置

IPv6 基本首部 40 B：版本 4 bit、流量类别 8 bit、流标签 20 bit、有效载荷长度 16 bit、下一首部 8 bit、跳数限制 8 bit、源与目的地址各 128 bit。有效载荷长度计基本首部之后的扩展首部及上层内容；下一首部可指扩展首部，也可指 TCP、UDP 等上层协议。流量类别和流标签为分类处理提供信息，端到端带宽、时延保证仍需网络资源和相应机制配合。

IPv6 可进行无状态地址自动配置，也支持 DHCPv6 等管理方式。接口标识符的生成可以采用多种方案，不能一概由 MAC 地址直接拼接。未指定地址为`::`，环回为`::1`，链路本地单播属于 `fe80::/10`，组播属于 `ff00::/8`。任播给多个接口配置同一地址，数据按路由规则到达其中一个，表示“一组中的一个”，并非每个接收者都获得一份。

压缩 `2001:0db8:85a3:0000:0000:8a2e:0370:7334` 可写为 `2001:db8:85a3::8a2e:370:7334`。每组只可去前导 0，不能删掉 `370` 末尾的 0；`::`出现一次，恢复时补到 8 组。双栈让设备同时运行 IPv4 和 IPv6，隧道将一类 IP 分组封装到另一类 IP 分组的数据区中，内外首部的端点含义分别读取。

辨析·自动配置有多种方式

IPv6 既支持无状态自动配置，也支持 DHCPv6，接口标识符不必来自 MAC。

4.26 主动回忆自测

以下为自拟题，先写当前观察的层与接口，再列必要条件。

1. `192.168.10.131/26` 所在网络、广播和普通可分配地址范围分别是什么？
2. 目的 10.1.2.3 同时匹配`/8`、`/16`、`/24`，距离分别为 1、8、20，应选哪条？
3. IPv4 数据区 1600 B、首部 20 B，经 MTU620 B 链路分片，非末片最大数据区和第二片偏移是多少？
4. 主机加入本地组播组后，IGMP 是否已经算好全网的无环转发树？
5. 路由与转发有什么区别？SDN 逻辑集中是否要求只有一台物理控制器？
6. 向异地目的主机发包时，源主机先 ARP 最终目的 IP 还是下一跳 IP？NAT 改写源 IP 后哪些校验可能受影响？
7. 初次 DHCP 获取地址的 Request 阶段，能否把 Offer 提出的地址立即用作 IP 源地址？
8. 比较 IPv4 与 IPv6 的首部校验、广播和路由器分片行为；IPv6 地址空间是 IPv4 的多少倍？
9. RIP、OSPF、BGP 分别交换什么信息，由什么直接承载？
10. 移动 IPv4 隧道的内外层目的地址分别代表什么？为什么已查到出口后仍可能排队？

4.26.1 核对答案

1. 网络 192.168.10.128，广播 192.168.10.191，可分配.129～.190。 **2.** 选匹配最长的`/24`；同前缀候选如何选优是另一层判断。 **3.** 每个非末片数据 600 B，第二片偏移 $600/8 = 75$；偏移不累计各片首部。

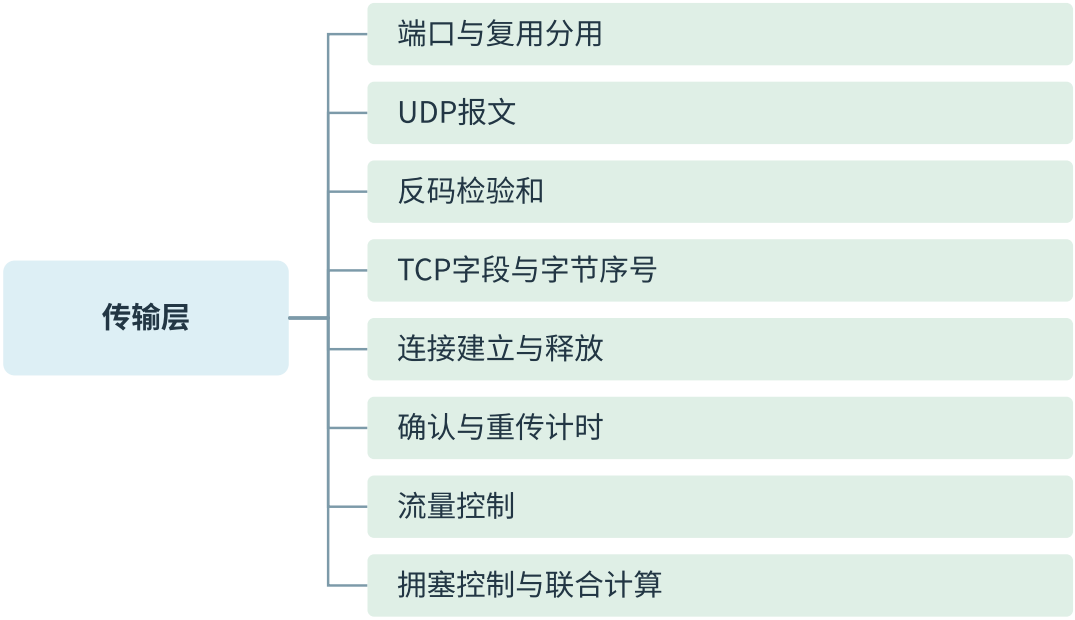
4. 没有，IGMP 管理本地成员，跨网转发树由相应组播路由机制组织。 **5.** 路由计算路径和表；转发查出口。逻辑集中可由多台控制服务器实现。 **6.** ARP 下一跳 IP；更新 IPv4 首部校验，以及启用的 TCP/UDP 伪首部校验。

7. 不能，初始选择阶段通常仍用源 0.0.0.0、广播目的，请求后收到 ACK 才确认使用。 **8.** IPv4 有首部校验且允许路由器分片；IPv6 无首部校验、广播和路由器分片。地址空间之比 2^{96}。 **9.** RIP 交换距离向量，经 UDP；OSPF 交换链路状态，直接经 IP；BGP 交换可达前缀和路径属性，经 TCP。

10. 内层目的为归属地址，外层为转交地址。出口竞争、交换结构或队首阻塞可引起排队。

传输层 5

章节脉络



IP 把数据送向目的主机，传输层再把数据交给相应应用端点。UDP 保留报文边界但不保证可靠交付；TCP 提供有序字节流，并依靠确认、重传、窗口和连接状态来管理这条字节流。计算 TCP 时始终把字节编号、窗口额度与尚可发送量分开记录。

本章 TCP 序号按 32 位模运算；小数值例题不跨回绕点时可直接作整数加减。MSS 是一个 TCP 段可携带的最大应用数据量，不包括 TCP、IP 首部。题目写 KB 时沿用该题的教材单位，窗口与 MSS 相除时保持同一口径。

课程连接·套接字由操作系统管理

应用通过套接字接口使用传输服务，端口参与内核分用，进程号标识操作系统中的进程。一个服务器进程可接受多条连接，故不能把端口、进程和连接按一一对应关系死记。

5.1 端口与套接字：一个 IP 为什么能运行多个服务

定义 5.1.1 (传输层端点标识)

端口号是传输层区分本机应用端点的 16 位标识。熟知端口常给固定服务使用，客户端通常选临时端口。端口号不是操作系统进程号：一个服务进程可以管理多个连接，一个应用也可创建多个套接字。

传输层把多个应用的数据交给 IP，称为复用；收到后交给相应应用端点，称为分用。UDP 在通常教材模型中按目的端口等本地端点信息分用；TCP 连接由源 IP、源端口、目的 IP、目的端口共同区分。实际 UDP 套接字的绑定和连接模式还可能限定远端信息，题目未给出时按基本分用模型判断。

5.1.1 例 5-1　谁决定这个 UDP 数据报交给哪个本机服务

例 5.1.1 (配套例题)

配套例题。UDP 协议实现分用时所依据的头部字段是（　）。

A. 源端口号　B. 目的端口号　C. 长度　D. 校验和

解析。目的端口指出本机接收应用端点，选 **B**。源端口可供接收方回复时使用，长度用于界定 UDP 报文，校验和用于检错；三者都不能替代目的端口的基本分用作用。

两个 TCP 客户可以同时访问服务器的 80 端口，因为它们的源 IP 或源端口不同，四元组不同。不能因为 “服务端端口相同” 就断言只有一条连接。

不变量·UDP 长度包含 8 字节首部

应用数据 12 B 时 UDP 长度为 20 B；普通 IPv4 还要把 IP 首部计入整个数据报上限。UDP 自身保留报文边界，承载它的 IP 分片则必须先在目的 IP 层重组，不能把每个 IP 片都当一个 UDP 报文。

5.2 UDP：8 字节首部保留了哪些边界

UDP 首部只有源端口、目的端口、长度、检验和四个字段，各 16 bit，共 8 B；没有序号、确认号和窗口字段。下图每行 32 bit，两个 16 bit 字段正好占一行。

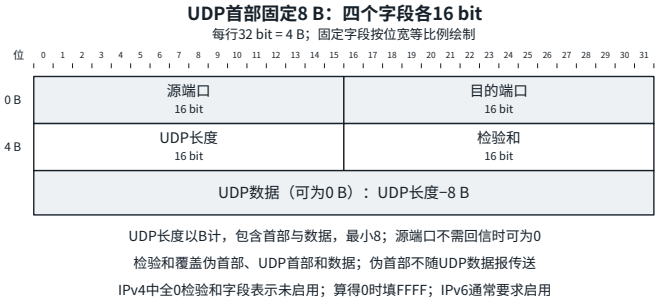


Figure 5.1: UDP 首部两行共 8 B，四个字段各 16 bit；数据从 UDP 起点之后 8 B 开始

源端口供回复时使用，不需回信时可为 0；目的端口用于基本分用。长度字段以 B 为单位，包含 UDP 首部和数据，最小 8，不能把它当成数据区长度。应用提交 12 B 数据，则 UDP 长度为 20。16 bit 长度字段最大可表示 65535 B，但普通 IPv4 还限制整个 IP 数据报不超过 65535 B；若 IPv4 首部 20 B，UDP 数据最多为 $65535-20-8=65507$ B，不能把 UDP 字段上界直接当作应用可提交的载荷上界。

UDP 直接保留应用报文边界，接收操作可区分每个 UDP 数据报；TCP 则可能把多个应用写入合并，或把一次写入拆成多个段。UDP 自身不把过长应用报文拆成多个 UDP 数据报，但 IP 可能将承载它的 IP 数据报分片；待目的主机的 IP 层重组后，才把完整 UDP 数据报交给 UDP 校验并分用。

UDP 无须三次握手，不提供自身的顺序恢复、确认重传和拥塞窗口机制。报文丢失、重复或乱序需要应用按需求处理。无连接既不表示应用完全没有状态，也不表示 UDP 的校验和可以保证可靠交付。音视频、DNS 等可以使用 UDP，但具体应用仍可能同时支持 TCP 或其他可靠机制。

UDP 开销小的直接含义是基本首部较短、没有连接建立所需往返；“一定比任何 TCP 应用快” 还取决于数据量、丢包率和应用处理。实时业务选择不等待迟到的旧报文，不等于连检错也可以省掉。

比较维度	UDP	TCP
交给应用的数据单位	一个个报文，保留边界	有序字节流，不保留应用写入边界
通信准备与端点	无须建立传输连接；底层允许时可单播、组播或广播	先建立一对端点间的全双工连接，不提供广播/组播连接
基本首部与长度	固定 8 B；有 UDP 总长度字段	20～60 B；数据偏移表示首部长，没有独立 TCP 总长度字段
丢失、次序与重复	协议本身不确认重传、不恢复次序	序号、累计确认和重传等保证可靠有序字节流
发送额度	没有自身的接收窗口和拥塞窗口机制	同时受接收窗口、拥塞窗口及在途数据限制

两者都有端口和检错机制，都借助 IP 传送。UDP 应用若需要可靠性和速率管理，这些机制必须由应用自身或其上层承担；“UDP 不提供”不能推出“使用 UDP 的应用不能提供”。TCP 有可靠传输机制，也不承诺固定时延或固定吞吐量。

错项排除·伪首部不是真的多发一层

校验借用源 IP、目的 IP、协议和长度，是为检查交付对象；它只参加计算。奇数字节补 0 也只补计算过程。因此计算线上长度和吞吐量时，不能再给 UDP 额外增加伪首部和求和补位。

5.3 反码校验和：进位为什么要加回低位

UDP 校验覆盖伪首部、UDP 首部和数据。IPv4 伪首部包含源 IP、目的 IP、0、协议号 17 和 UDP 长度，只参与计算，不作为 UDP 数据再发出。有了它，数据报即使被投递到另一地址或另一协议，其中一部分错误也能被检出；TCP 采用相应伪首部，协议号为 6，不能沿用 UDP 的 17。

按 16 位分组求二进制反码和，最高位产生的进位回卷加到最低位，反复处理直至没有进位，最后按位取反。计算前校验和字段置 0；数据总字节数为奇数时补一个 0 字节参与计算，该补齐字节不属于发送的 UDP 数据，也不计入长度字段。

5.3.1 例 5-2　少做一次回卷，会错在哪一位

例 5.3.1 (配套例题)

配套例题。若 UDP 协议在计算校验和过程中，计算得到中间结果为 `1011 1001 1011 0110` 时，还需要加上最后一个 16 位数 `0110 0101 1100 0101`，则最终计算得到的校验和是（　）。

A. `0001 1111 0111 1011`

B. `0001 1111 0111 1100`

C. `1110 0000 1000 0011`

D. `1110 0000 1000 0100`

解析。两数分别为十六进制 B9B6、65C5，普通相加得到 `1 1F7B`。保留低 16 位 1F7B，再加回最高进位 1，得到 1F7C，最后取反为 E083，即 **C**。D 对应 1F7B 直接取反，遗漏回卷；A、B 都还没完成最后取反。

接收端把收到的校验和也加入同样求和，结果应为全 1。若发送方算得的校验和为全 0，UDP 字段应写全 1；在 IPv4 中全 0 字段另表示未使用 UDP 校验。IPv6 通常要求 UDP 校验，不能把 IPv4 的可选零字段推广成 IPv6 通则。〖5.2-008〗

反码检验和与 CRC 一样，检验通过只能说明没有**检测出**差错，不能排除所有抵消或漏检情形。把三种首部中同名的检验和放在一起，必须分别核对覆盖范围与零值语义：

比较项	IPv4 首部检验和	UDP 检验和	TCP 检验和
参加计算的内容	IPv4 首部，含选项与填充	伪首部、UDP 首部、UDP 数据	伪首部、TCP 首部、TCP 数据
IPv4 伪首部	不使用	12 B：源 IP、目的 IP、0、17、UDP 长度	12 B：源 IP、目的 IP、0、6、TCP 长度
字段为全 0 时	没有关闭首部校验的含义	IPv4 表示未启用；IPv6 通常要求启用；算得 0 时传全 1	没有以全 0 关闭校验的约定

伪首部借用网络层地址来检查交付对象，只在计算时构造，不成为线上多出来的 12 B 首部。IPv4 改 TTL 只需相应更新 IP 首部检验和；NAT 若改 IP 地址，启用的 TCP/UDP 校验也可能需更新，因为伪首部变了。UDP 为奇数数据补齐的 0 字节同样只服务于求和，既不发送也不计入 UDP 长度。

最小反例·双向 seq 各有自己的坐标

甲已发 2001～4000，乙回复 seq=4001、ack=3001：乙的 seq 属于反方向，只确认甲到 3000 的是 ack。把乙的 seq 拿来确认甲的数据，会错误释放仍在途的 3001～4000。

5.4 TCP 序号、确认号与控制位怎样一起读

TCP 基本首部 20 B，含选项与填充时可到 60 B。图中沿用主书经典格式：源、目的端口各 16 bit，序号与确认号各 32 bit，数据偏移 4 bit、保留 6 bit、六个控制位各 1 bit，再接 16 bit 接收窗口；最后一行固定字段是检验和与紧急指针，各 16 bit。现代扩展可能重新赋义部分原保留位，须按题图及扩展条件解释，本图保持主书的六控制位口径。

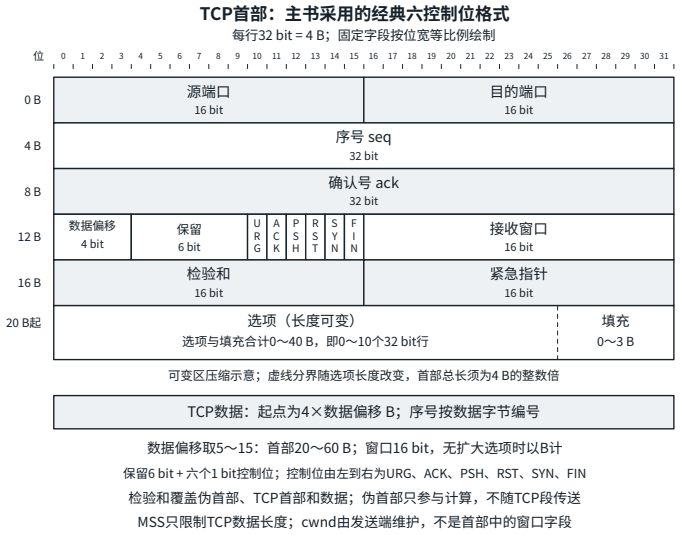


Figure 5.2: TCP 固定首部五行，每行 32 bit；数据偏移 4 B 单位，接收窗口 16 bit，选项与填充另占 0～40 B

数据偏移表示 TCP 数据起点相对 TCP 段起点的距离，以 4 B 为单位，故也就是 TCP 首部长度。取 5 时 20 B，取 15 时 60 B；加入选项后须用必要填充对齐到 4 B。它不是本段数据字节数。对未分片或已重组的普通 IPv4 分组，若其 IP 数据只承载一个 TCP 段，则

$$L_{TCP\text{数据}} = IP\text{总长度} - 4\times IHL - 4\times TCP\text{数据偏移}.$$

例如 IP 总长度 100 B、IHL=6、TCP 数据偏移 =6，TCP 数据为 52 B。不能固定减 40 B，因为两种首部都可能有选项；更不能对每个未重

组 IP 分片分别照减一个 TCP 首部。MSS 限制的是这个 TCP 数据部分，不包括任何 IP 或 TCP 首部。

控制位按条件生效：ACK=1 时确认号有效；SYN 用于连接同步，FIN 表示本方向没有更多数据；RST 用于复位或拒绝不适当的连接/报文；PSH 要求接收方及时向应用交付已收到的数据，不表示必须立即回一个 ACK。URG=1 时 16 bit 紧急指针有效，它相对本段序号指示紧急数据边界；教材把紧急数据放在数据区前部，此时紧急指针可按紧急数据的字节数理解，它不是另一套独立的 32 bit 序号。控制位语义与序号消耗要分别判断，并非任意一个控制位置 1 都“占一个序号”。

接收窗口字段是**本段发送者作为接收方**通告给对端的额度，未协商窗口扩大时以 B 计，最大 65535 B。它与本段 ack 一起描述对端可发送的字节范围；本段 seq 则描述发送者在反方向的字节位置。cwnd 由发送端根据网络反馈维护，首部中并没有名为 cwnd 的字段。下文的窗口题正是把接收窗口、拥塞窗口与在途数据这三个量放在一起计算。

TCP 按**字节**编号。数据段的 seq 是该段第一个数据字节的序号；携带 L B 时，覆盖 seq 至 $seq+L-1$，下一字节为 $seq+L$。确认号 ack 表示下一个期望按序收到的字节，因此它累计确认的是 ack 之前的连续字节，不表示“第 ack 个报文段”。收到 ack=100 表示已按序收到至 99。〖5.3-011〗

课程连接·状态是事件触发的记录

连接状态可像操作系统进程状态一样按事件画转换：发 FIN、收到其 ACK、收到对方 FIN 分别触发不同动作。仅看到报文名字不足以决定两端都已关闭，还要确定报文在何时被哪端收到。

5.4.1 例 5-3　最后收到的字节与确认号相差 1

例 5.4.1 (配套例题)

配套例题。主机甲与主机乙之间已建立一个 TCP 连接，甲向乙发送了两个连续 TCP 段，分别包含 300 B 和 500 B 有效载荷，第一个段的序列号为 200，乙正确接收到两个段后，发送给甲的确认序列号是（　）。

A. 500　　B. 700　　C. 800　　D. 1000

解析。第一段覆盖 200～499，第二段从 500 开始，覆盖 500～999。下一个期望字节为 1000，选 **D**。不能把载荷总长 800 直接当确认号，也不能只给出最后字节 999。

SYN 和 FIN 各占一个序号，纯 ACK 不占；若 SYN 的序号为 x，则正常数据（不带 SYN）的第一个字节序号为 $x+1$。某方向 FIN 的序号等于该方向最后数据字节的下一号；对端确认这个 FIN 时还要再加 1。这一加 1 不代表增加了一个应用数据字节。

5.5 三次握手与连接释放：状态变化由哪个事件触发

三次握手同时确认两个方向的初始序号和收发可达性。客户先发 SYN（seq=x）并进入 SYN-SENT；监听服务器收到后发 SYN+ACK、seq=y、ack=$x+1$，并进入 SYN-RECEIVED；客户收到后发 ACK、seq=$x+1$、ack=$y+1$ 并进入 ESTABLISHED；服务器收到此 ACK 后也进入 ESTABLISHED。第三次 ACK 由客户发出，序号按客户方向编号，也可携带数据；不能因此把两端的初始序号当成同一个值。

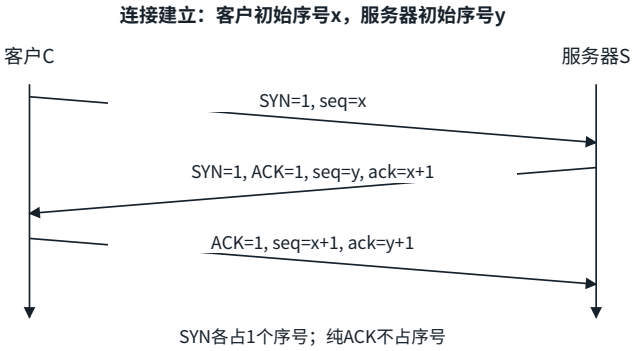


Figure 5.3: 两端各占用一个 SYN 序号；第三次确认不再额外消耗序号

错项排除·半关闭仍允许反向数据

被动端收到 FIN 后进入 CLOSE-WAIT，表示对方不会再发送，本端应用仍可继续向对方发送。把 TCP 全双工看成一根双向同时关闭的管道，会错误断言收到 FIN 后所有方向的数据都不能发。

第三次握手针对的是迟到的旧 SYN：若缺了它，服务器可能仅凭这样一个旧 SYN 就长期保留一条对端并不承认的连接。它同时确认客户已经收到服务器的初始序号。只说 “让双方知道能通信” 过于含糊，应分别指出谁收到哪个序号、确认了哪个方向。

TCP 是全双工，每个方向独立关闭。主动关闭者发送 FIN 后进入 FIN-WAIT-1；收到对自己 FIN 的 ACK 进入 FIN-WAIT-2；收到对方 FIN 并发送 ACK 后进入 TIME-WAIT。被动关闭者收到 FIN、发 ACK 后进入 CLOSE-WAIT，表示对端不再发送，但本方向应用仍可继续发送；本机应用关闭后发 FIN 进入 LAST-ACK，收到确认后才进入 CLOSED。ACK 与 FIN 可以合并，因此报文数不必总为四份，但两方向的关闭语义都必须完成。

TIME-WAIT 通常持续 2MSL，一方面在最后的 ACK 丢失时还能再次响应对方重传的 FIN，另一方面使旧连接的残留报文逐渐消失。它一般出现在完成主动关闭的一端；“每端都要固定等 2MSL 才关” 不成立。同时关闭还可能经过 CLOSING 等状态，应按收到 FIN 和 ACK 的次序判断。

5.5.1 例 5-4　至少关闭时间要从 FIN 开始画

例 5.5.1 (配套例题)

配套例题。假设客户 C 和服务器 S 已建立 TCP 连接，RTT=50 ms，最长报文段寿命 MSL=800 ms，数据传输结束后 C 主动请求断开连接。若从 C 主动向 S 发出 FIN 段时刻算起，则 C 和 S 进入 CLOSED 状态所需的时间至少分别是（　）。

A. 850 ms，50 ms　　B. 1650 ms，50 ms

C. 850 ms，75 ms　　D. 1650 ms，75 ms

解析。求最少时间，可让 S 收到 FIN 后立即合并发送 ACK 与 FIN，忽略处理和段发送时延，单向传播为 25 ms：S 在 25 ms 收到并回复；C 在 50 ms 收到 S 的 FIN，发最后 ACK，同时开始 2MSL 计时；S 在 75 ms 收到最后 ACK 而关闭；C 在 $50 + 2 \times 800 = 1650$ ms 关闭。因此选 **D**。服务器的 CLOSED 须等最后 ACK 真正到达，不能在它发送 FIN 时就提前标记。

5.6 确认、重传与 RTT 估计

TCP 以累计 ACK 报告连续接收进度。若 1000～1999 丢失，而 2000～2999 先到，基本累计确认仍指向 1000；接收方可以缓存失序数据，但不能把累计 ACK 越过缺口。选择确认（SACK）选项能额外报告已收到的分块范围，但它只是补充信息，并不改变累计 ACK 的基本定义。

发送端保留未确认数据并运行重传计时器。超时重传针对未确认内容，不要求把整条连接已经确认过的所有数据重发。连续收到 3 个重复 ACK 可触发快重传，**3 个重复**中不包含最初那份正常 ACK，正是它把确认号推进到了该值；图中应逐次计数。

RTO 需要考虑 RTT 的变化。常见平滑估计为

$$SRTT \leftarrow (1-\alpha)SRTT + \alpha RTT_{\text{sample}},$$

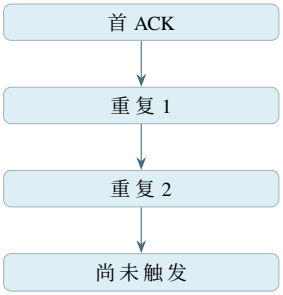
再结合 RTT 偏差设置超时阈值。若题目明确 $RTO = \beta SRTT$，就按该给定简化式计算，不把它推广成所有实现。重传之后收到的 ACK 难以判断确认的是原段还是重传段，因此 Karn 原则不用这种有歧义的样本更新 RTT 估计；超时还会使 RTO 退避。

例如旧 SRTT 为 100 ms、$\alpha = 1/8$、新无重传样本 140 ms，则新 SRTT 为 105 ms，而不是直接改为 140 ms。若同一段曾重传，先判断样本是否可用，再代入公式；只知道 ACK 的到达时间，还不足以确定它对应哪个发送时刻。

可靠传输控制 “同一字节是否按序到齐”，拥塞控制决定 “现在允许多少字节在路上”，流量控制考虑 “接收端还有多少空间”。丢失处理会同时改变确认状态、重传状态和拥塞状态，做综合题时需分别记录。

最小反例·首次加两次还差一次

先收到把确认号推进到 3001 的正常 ACK，再收到两份 ack=3001，重复计数只有 2。第三份重复 ACK 到达才满足传统快重传阈值；同号 ACK 共有 4 份中的第一份不能混算成重复。



三个同号 ACK 仅含两份重复 ACK

5.7 接收窗口：总额度与剩余额度怎样区分

接收方通告的 rwnd 反映当前愿意接收的字节范围，常以累计确认号为左边界。若 ack=3001、rwnd=4000 B，则允许的接收序号区间为 3001～7000，右端的下一号为 7001。发送端已经发出但未确认的字节仍占该区间，不能把 4000 B 全部当成新数据额度。

发送窗口是字节额度，不是 “能发几个段”。窗口 2000 B、MSS1000 B，在没有已发未确认数据时可容纳 2 个满段；若已有 1000 B 在途，最多再发 1000 B。〖5.3-010〗〖5.3-029〗这类题只改动总窗口的数值，两次通告之差并不等于总窗口。

接收方通告零窗口后，发送方暂停普通数据发送，但应启动持续计时器，定时发送零窗口探测，防止 “窗口重新打开” 的通知丢失后双方永久等待。持续计时器不是重传计时器、保活计时器，也不是 TIME-WAIT 的 2MSL。〖5.3-014〗

当窗口尺度选项参与计算时，首部字段与实际字节窗口还需按协商尺度换算；选项只能在连接建立时按规则协商。题目若未给窗口扩大，按字段直接表示的字节额度处理，不自行添加尺度。

5.8 慢开始、拥塞避免与 Reno 的丢包反应

拥塞窗口 cwnd 由发送方根据网络反馈维护，慢开始门限 ssthresh 区分增长阶段。以下数值推演采用教材逐段确认、持续有数据、没有其他瓶

颈的简化模型；用 MSS 计窗口，初值按具体题目给定。图示或题源取 1 MSS 时，不把它当成现代所有 TCP 实现的固定初始窗口。

不变量·按 ACK 事件依次更新四账

先更新累计确认，再核对已发未确认区间，随后按拥塞算法改变 cwnd 并读当前 rwnd，最后取小扣在途。若只把两个窗口数字取小就回答还能发多少，会在所有有在途数据的题目中多算。

慢开始每收到新数据的 ACK 适当增加 cwnd，在一轮中所有段均分别得到确认时，窗口约每 RTT 翻倍；达到门限后转入拥塞避免，平均每 RTT 约增加 1 MSS。实际按 ACK 更新，若有延迟 ACK、应用暂时无数据或接收窗口受限，逐轮翻倍模型未必精确。

以初始 1、门限 8 为例，逐轮发送窗口可为 1、2、4、8、9、10……。若第 8 轮发送时 cwnd=12 并发生超时，按该题模型将门限设为 6，cwnd 退回 1，后续轮次为 1、2、4、6、7……；从初始第 1 轮开始数，第 13 轮为 7。〖5.3-033〗必须先把“轮次记录的是发送前还是确认后”固定，不能在超时处跳过一轮或把 12 继续保留。

超时表明没有及时得到反馈，教材 Reno 模型采用减半门限、cwnd 退回 1 再慢开始。3 个重复 ACK 触发快重传时，说明仍有后续段到达，Reno 进入快恢复：把门限减半，临时把 cwnd 设为门限加 3 MSS，用来反映已经离开网络的重复确认段，后续重复 ACK 还可继续调整；新 ACK 确认缺失数据后，窗口回到门限并进入拥塞避免。课件按传播轮次绘制的简化折线通常记录新 ACK 之后的稳定点，此时 $cwnd = ssthresh$；不要把恢复期间临时的 $+3$ 套进稳定轮次图。Tahoe 收到 3 个重复 ACK 后也退回慢开始，不能把 Reno 的恢复规则无条件套给所有版本。

在规范实现中，门限减半通常还要参考在途数据量等条件；408 教材题经常直接给出“丢包时拥塞窗口”并按其一半设门限。应使用题目明确的算法及数值条件，不以术语相同掩盖模型差异。

5.8.1 例 5-5　没有拥塞，不代表一定指数增长

例 5.8.1 (配套例题)

配套例题。若主机甲与主机乙已建立 TCP 连接，MSS 为 1 KB，RTT 为 2 ms，则在不出现拥塞的前提下，拥塞窗口从 8 KB 增长到 32 KB 所需的最长时间是（　）。

A. 4 ms　B. 8 ms　C. 24 ms　D. 48 ms

解析。“最长”需要考虑门限可能已不大于 8 KB，从起点便处于拥塞避免；每 RTT 增加 1 KB，从 8 到 32 需 24 个 RTT，共 48 ms，选 **D**。若始终处于慢开始，8→16→32 只需 2RTT=4 ms，是较短情形。原题沿用持续有数据、ACK 正常返回的教材增长模型；不能把接收端的任意停顿也算进这个“最长”里。

5.9 两个窗口与在途数据：本次究竟还能发多少

教材常用总发送额度

$$W = \min(cwnd, rwnd).$$

若最新有效确认与窗口左沿一致，已发未确认字节数为 F，那么可新发额度为

$$A = \max(0, W - F).$$

这个式子要求先确定 ACK 确认了哪些字节、cwnd 如何因该 ACK 变化。若题图给出了右边界或不同时间的窗口更新，应直接画出序号区

间，不能拿过期 rwnd 替代当前通告。某个段何时收到 ACK、另一个段是否仍在途，这些中间状态才是窗口题的关键。

比较量	谁维护，依据什么	回答哪个问题
接收窗口 rwnd	接收方按可用接收能力通告，来自 TCP 窗口字段及协商尺度	对端愿意接收多大字节范围
拥塞窗口 cwnd	发送方按确认、丢包等反馈维护，不是首部通告字段	网络反馈允许多少数据在途
已发未确认量 F	发送方按已发送区间与累计确认进度登记	总额度中已经占用了多少
可新发额度 A	以当前两个上限取小，再扣除 F，下限为 0	本时刻还能新发多少字节

不变量·按 ACK 事件依次更新四账

先更新累计确认，再核对已发未确认区间，随后按拥塞算法改变 cwnd 并读当前 rwnd，最后取小扣在途。若只把两个窗口数字取小就回答还能发多少，会在所有有在途数据的题目中多算。

接收窗口缩小不直接证明网络拥塞，cwnd 降低也不证明接收缓存已满。快重传、超时、零窗口探测分别触发不同动作；可靠性、拥塞与流量控制可以相互作用，但不能用一个“窗口变小”替代全部状态判断。

一次 ACK 事件的四本账

第一本记录已按序确认到哪里：由新 ack 减去旧左沿确定释放的字节。第二本记录还有多少数据在途，不能把反方向 seq 当作本方向确认。第三本按题设慢开始、拥塞避免、超时或重复 ACK 事件更新 cwnd。第四本读取当前有效 rwnd，取两上限之小后再扣在途量，才得到还能新发的字节。

可新发额度公式来自集合差：当前总窗口允许的字节区间中，已经发出尚未确认的部分不可再算作新数据；扣除其长度 F 以后才能安排新发送。若总额度小于已在途量，新增额度按 0 处理，而不是要求撤回已进入链路的数据。这个推导默认最新窗口左沿与累计 ACK 一致，复杂窗口更新以实际序号区间为准。

5.9.1 例 5-6　ACK 只移走已经确认的那一段

例 5.9.1 (配套例题)

配套例题。主机甲通过 TCP 向主机乙发送数据的部分过程如下图。seq 为序号，ack_seq 为确认序号，rcvwnd 为接收窗口。甲在 t_0 的拥塞窗口和发送窗口均为 2000 B，拥塞控制阈值为 8000 B，MSS=1000 B，甲始终以 MSS 发送 TCP 段。若甲在 t_1 收到图示确认段，则未收到新的确认之前，还可以继续发送的 TCP 段数是（　）。

Figure 5.4: 原题数据：第二个数据段仍未被 ack_seq=3001 确认

A. 2　B. 3　C. 4　D. 5

不变量·按 ACK 事件依次更新四账

先更新累计确认，再核对已发未确认区间，随后按拥塞算法改变 cwnd 并读当前 rwnd，最后取小扣在途。若只把两个窗口数字取小就回答还能发多少，会在所有有在途数据的题目中多算。

解析。第一段 seq=2001 覆盖 2001～3000，第二段 seq=3001 覆盖 3001～4000。返回 ack_seq=3001 只累计确认第一段，第二段的 1000 B 仍在途。乙段的 seq=4001 属于乙 → 甲方向，不能拿它确认甲发出的字节。

甲原 cwnd 为 2000 B，小于门限 8000 B，收到一份对新数据的确认后，按题设慢开始增加 1 MSS，cwnd 变为 3000 B；rwnd 通告为 4000 B，总发送额度取 3000 B。减去在途 1000 B，尚可新发 2000 B，即 2 段，选 **A**。对应允许的甲方发送范围从 3001 起共 3000 B，其中 3001～4000 已经发出，4001～6000 还可新发。

5.9.2 例 5-7　拥塞窗口变大后，接收窗口仍可成为上限

例 5.9.2 (配套例题)

配套例题。主机甲和乙已建立 TCP 连接，甲始终以 MSS=1 KB 大小的段发送数据，并一直有数据发送；乙每收到一个数据段都会发出一个接收窗口为 10 KB 的确认段。若甲在 t 时刻发生超时时拥塞窗口为 8 KB，则从 t 起不再发生超时的情况下，经过 10 个 RTT 后，甲的发送窗口是（　）。

A. 10 KB　B. 12 KB　C. 14 KB　D. 15 KB

解析。超时后门限为 4 KB，cwnd 退回 1 KB。按本题逐轮教材模型，前 8 个 RTT 后的 cwnd 依次为 2、4、5、6、7、8、9、10 KB，发送额度到达接收方上限 10 KB。之后没有新的丢包，cwnd 不会因超时下降，而固定 rwnd 仍把发送窗口限制在 10 KB，因此第 10 个 RTT 后仍为 10 KB，选 **A**。达到接收方上限后，无须继续假定 cwnd 每轮恰好增长 1 KB 来选答案；窗口受限时的实际 ACK 增长轨迹也未必遵循未受限时的逐轮简化式。

上题 rwnd 固定，故无须再算接收缓存的占用；若题目改问 “还可发送多少”，还必须再减在途数据，不能直接回答 10 KB。窗口题可按 “序号区间—ACK 覆盖—cwnd 事件—rwnd 边界—扣除在途” 的顺序推进。

5.9.3 例 5-8　从完整字节表同时读出握手、填充与已确认数据

例 5.9.3 (配套例题)

配套例题（改编）。　主机 H 通过快速以太网连接 Internet，IP 地址为 192.168.0.8；服务器 S 的 IP 地址为 211.68.71.80。H 与 S 使用 TCP 通信，在 H 上捕获的 5 个 IP 分组及其中一个分组在 S 发出时的前 40 字节如下图，均为十六进制、按网络字节序排列。**第 2 包首字节采用 45，第 3 包 TCP 数据偏移与控制字节采用 50 10**。

2012改编：在H捕获的5个IP分组及S发出的对应分组

每项仅列前40 B；每行20 B，均为网络字节序的十六进制

1	45 00 00 30 01 9b 40 00 80 06 1d e8 c0 a8 00 08 d3 44 47 50 0b d9 13 88 84 6b 41 c5 00 00 00 00 70 02 43 80 5d b0 00 00
2	45 00 00 30 00 00 40 00 31 06 6e 83 d3 44 47 50 c0 a8 00 08 13 88 0b d9 e0 59 9f ef 84 6b 41 c6 70 12 16 d0 37 e1 00 00
3	45 00 00 28 01 9c 40 00 80 06 1d ef c0 a8 00 08 d3 44 47 50 0b d9 13 88 84 6b 41 c6 e0 59 9f f0 50 10 43 80 2b 32 00 00
4	45 00 00 38 01 9d 40 00 80 06 1d de c0 a8 00 08 d3 44 47 50 0b d9 13 88 84 6b 41 c6 e0 59 9f f0 50 18 43 80 e6 55 00 00
5	45 00 00 28 68 11 40 00 31 06 06 7a d3 44 47 50 c0 a8 00 08 13 88 0b d9 e0 59 9f f0 84 6b 41 d6 50 10 16 d0 57 d2 00 00
S	45 00 00 28 68 11 40 00 40 06 ec ad d3 44 47 50 ca 76 01 06 13 88 a1 08 e0 59 9f f0 84 6b 41 d6 50 10 16 d0 b7 d6 00 00

第2包首字节45；第3包TCP控制字节50 10；其余字节按主书表保留

Figure 5.5: 1～5 为 H 处捕获的分组，S 行为在服务器发出时捕获的对应分组；每行 20 B

字段布局参照第 4 章 IPv4 首部格式和本章TCP 首部图：IPv4 源、目的地址分别位于第 13～16、第 17～20 字节，TTL 位于第 9 字节；IHL 与 TCP 数据偏移字段均以 4 B 为单位。

（1）5 个 IP 分组中，哪几个由 H 发送？哪几个完成 TCP 连接建立过程？哪几个在通过快速以太网传输时进行了填充？

（2）根据这 5 个 IP 分组，分析 S 已经收到的应用层数据有多少字节。

（3）若某个 IP 分组在 S 发出时的前 40 字节为图中 S 行，该 IP 分组到达 H 时经过了多少个路由器？

解析。不要先猜“第几次握手”，应按“IP 长度 →TCP 起点 → 控制位 → 序号确认号”顺序解码。所有分组首字节 45 表示 IPv4、IHL=5，即 20 B IP 首部，TCP 从第 21 字节开始。1、3、4 号的源地址 `c0 a8 00 08` 为 192.168.0.8，因此 **H 发送的是 1、3、4 号**；2、5 号的源地址为 S。

编号	IP 总长/B	TCP 首长/B	TCP 标志
1	48	28	SYN
2	48	28	SYN、ACK
3	40	20	ACK
4	56	20	PSH、ACK
5	40	20	ACK

编号	seq（十六进制）	ack（十六进制）	应用数据/B
1	846b41c5	00000000（ACK=0）	0
2	e0599fef	846b41c6	0
3	846b41c6	e0599ff0	0
4	846b41c6	e0599ff0	16
5	e0599ff0	846b41d6	0

1 号的 TCP 数据偏移高半字节为 7，首部长 $7 \times 4 = 28$ B；控制字节 02 置 SYN。2 号的控制字节 12 同时置了 SYN 与 ACK，确认 H 的初始序号加 1；3 号的控制字节 10 只置 ACK，确认 S 的初始序号加 1。因此 **1、2、3 号完成三次握手**，SYN 分别消耗一个序号，第三次纯 ACK

不变量·按 ACK 事件依次更新四账

先更新累计确认，再核对已发未确认区间，随后按拥塞算法改变 cwnd 并读当前 rwnd，最后取小扣在途。若只把两个窗口数字取小就回答还能发多少，会在所有有在途数据的题目中多算。

不额外消耗序号。捕获表只给前 40 B，1、2 号的 8 B TCP 选项没有完整列出，但长度字段足以判定它们没有应用数据。

快速以太网沿用无标签以太网数据字段至少 46 B 的限制，判断填充应比较**整个 IP 分组**与 46 B，不能只比较 TCP 数据。3、5 号 IP 总长 `0028` 为 40 B，均需补 $46-40=6$ B；1、2 号为 48 B、4 号为 56 B，均不需补。因此填充的是 **3、5 号**，每个形成 $14+40+6+4=64$ B 的最短 MAC 帧。填充不计入 IP 总长度，更不占 TCP 字节序号。

第（2）问利用累计确认而不是把 40 字节截取范围当作完整数据。H 首个应用字节的序号为 SYN 序号加 1，即 `846b41c6`；5 号通告下一个期望序号 `846b41d6`，所以 S 已按序收到 `846b41d6` − `846b41c6` = `10` = **16** B 应用数据。这也与 4 号的 $56-20-20=16$ B 相符。

第（3）问先匹配报文身份。S 行 IP 标识为 `6811`，TCP 序号、确认号与 5 号一致，对应 H 处的 5 号；途中的 NAT 可能改变目的地址与端口，不能因此误认为是另一个报文。S 发出时 TTL 为 `40` 即 64，H 处为 `31` 即 49，每经过一个路由器减 1，故经过 $64-49=$ **15** 个路由器。交换机不减 TTL。图中的其余校验和字节按来源保留；题目只截取了部分字节，选项与有效载荷并未给全，不能据此声称已经验证所有报文的完整校验和。

不变量·按 ACK 事件依次更新四账

先更新累计确认，再核对已发未确认区间，随后按拥塞算法改变 cwnd 并读当前 rwnd，最后取小扣在途。若只把两个窗口数字取小就回答还能发多少，会在所有有在途数据的题目中多算。

5.9.4 例 5-9　接收缓存只进不出，窗口最终怎样归零

例 5.9.4 (配套例题)

配套例题（改编）。　下图中 H3 访问 Web 服务器 S。S 为新建 TCP 连接分配 20 KB（K=1024）接收缓存，MSS=1 KB，平均 RTT=200 ms。H3 建立连接时初始序号为 100，持续按 MSS 大小发送数据，拥塞窗口初始阈值 32 KB；S 对收到的每个段确认并通告新接收窗口。连接建立后，S 的 TCP 接收缓存只有数据存入、没有数据取出。图中 R2 的 E0 侧采用 201.1.1.0/24。

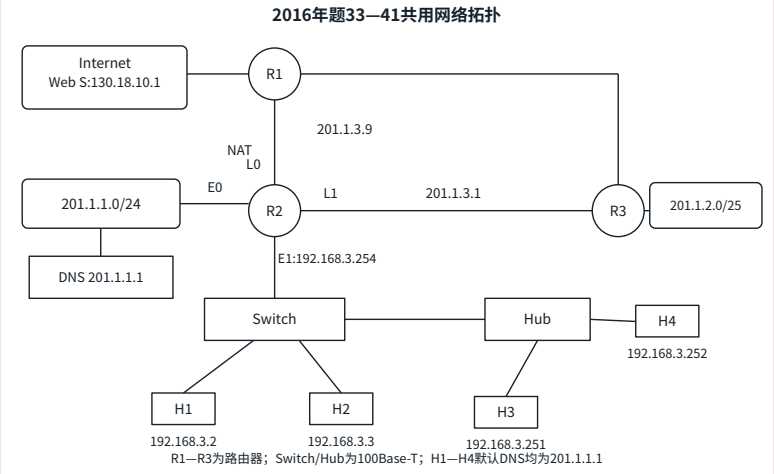


Figure 5.6: H3 为 192.168.3.251，经 Hub 和 R2 访问 130.18.10.1；复用 2016 年共用题图

（1）连接建立时，H3 收到的第二次握手 TCP 段的 SYN、ACK 标志位分别是多少？确认号是多少？

（2）H3 收到的第 8 个确认段所通告的接收窗口是多少？此时拥塞窗口变为多少？发送窗口变为多少？

（3）发送窗口等于 0 时，下一个待发送的数据段序号是多少？

从 H3 发送第 1 个数据段到发送窗口等于 0，平均数据传输速率是多少？忽略段的传输时延。

（4）H3 与 S 通信已结束，H3 在 t 时刻请求断开连接。从 t 起，S 释放该连接的最短时间是多少？

不变量·按 ACK 事件依次更新四账

先更新累计确认，再核对已发未确认区间，随后按拥塞算法改变 cwnd 并读当前 rwnd，最后取小扣在途。若只把两个窗口数字取小就回答还能发多少，会在所有有在途数据的题目中多算。

解析。第（1）问的第二次握手要同时表达 “接受对方 SYN” 和 “发出本方 SYN”，所以 **SYN=1、ACK=1、确认号 =101**。初始序号 100 被 SYN 占用，首个数据字节从 101 开始。

第（2）问必须分别更新两种窗口。沿用教材模型：初始拥塞窗口为 1 MSS，每个新数据段的 ACK 使慢开始窗口增加 1 MSS。第 8 个数据确认到达后，$cwnd = (1+8)\times 1 = \mathbf{9\ KB}$，仍小于 32 KB 阈值；S 已经缓存 8 KB 且未取出，通告 $rwnd = 20 - 8 = \mathbf{12\ KB}$。发送窗口总额度为 $\min(cwnd, rwnd) = \mathbf{9\ KB}$。如果题目问此时能再发送多少，还须扣除已经发出但未确认的数据，不能把 9 KB 直接当成新增发送量。

第（3）问要计到零窗口通告回到 H3 的时刻。每轮受拥塞窗口与剩余接收缓存共同限制，不能把慢开始序列无限写成 1、2、4、8、16。

轮次	发送前 cwnd/MSS	尚余接收缓存/KB	本轮发送/KB
1	1	20	1
2	2	19	2
3	4	17	4
4	8	13	8
5	16	5	5

轮次	累计已接收/KB	本轮确认到达时刻
1	1	1 RTT
2	3	2 RTT
3	7	3 RTT
4	15	4 RTT
5	20	5 RTT

第 5 轮只能再放入 5 KB，S 随后通告零窗口。下一个待发送字节的序号为 $101 + 20\times 1024 = \mathbf{20581}$，这个值不因窗口暂时关闭而重置。计时为 5×200 ms $= 1$ s，平均速率为 $20\times 1024\times 8/1 = \mathbf{163840\ bit/s} = \mathbf{163.84\ kb/s}$。这里 KB 用 1024、kb/s 用 1000；计时起点不包括握手，但终点包括最后一轮确认回传。

零窗口是接收方流量控制的结果，并不说明此前有数据丢失，也不要求按超时规则把 cwnd 重置为 1。发送方可用持续计时器触发零窗口探测，以免窗口更新丢失后永久等待；探测本身不会使接收缓存腾空。在本题 “始终不取数据” 的条件下，不能假设经过一个 RTT 窗口便自动恢复。

第（4）问求 S 的释放时间。为取最短值，S 收到 H3 的 FIN 后立即合并发出 ACK 和自己的 FIN：H3 的 FIN 到 S 用 0.5 RTT，S 的 FIN 到 H3 再用 0.5 RTT，H3 的最后 ACK 返回 S 再用 0.5 RTT。S 收到此 ACK 后进入 CLOSED，故最短为 $1.5\text{RTT} = \mathbf{300\ ms}$。H3 作为主动关闭方的 TIME-WAIT 等待属于另一端的计时，不能再加到 S 的 300 ms 上。

5.10 传输端点、报文边界与内核缓冲

课程连接·两层缓冲分开看

应用缓冲与内核 TCP 收发缓冲不同；对端应用读取速度会影响接收窗口。

同一主机的 TCP 端口空间与 UDP 端口空间相互独立。熟知端口为 0～1023，登记端口为 1024～49151，动态或私有端口为 49152～65535；具体操作系统对临时端口分配和权限的规定应按环境判断。端口字段只有 16 bit，TCP 连接仍可通过四元组区分，同一服务端口可同时对应多条连接。

应用缓冲区与 TCP 发送、接收缓冲区属于不同层次。应用把数据交给发送缓冲区之后，传输层根据窗口、MSS 和协议状态安排分段；已发送而未确认的字节还要保留以便重传。接收方可以缓存失序字节，但按顺序交给应用。应用尚未读取时，已收到的数据继续占用接收缓冲；应用读取并释放空间后，窗口才可相应扩大。

应用先写 1600 B、再写 1400 B，MSS=1000 B 时，TCP 可以划成若干不超过 1000 B 的段，第二段也可能跨越两次应用写入边界；接收应用应依自身报文格式识别边界。MSS 表示单段数据上限，并不要求每段必须装满。普通 IPv4 首部 20 B、TCP 首部 20 B、路径 MTU1500 B 时，简单模型 MSS 可取 1460 B；选项和路径条件改变后须重新核对。

UDP 则保留应用报文边界。课件中源端口 3666、目的端口 9888、应用报文 96 B，对应 UDP 总长度 104 B；若 IPv4 首部 20 B，IP 总长度 124 B。UDP 应用报文与 UDP 总长度、IP 总长度是三层边界，分片时也不能擅自给每片重复增加一套 UDP 首部。

5.11 反码检验的发送端、接收端与漏检

取课件的两个 16 bit 字 `0x5065`、`0x6951`，和为 `0xB9B6`，按位取反得到检验和 `0x4649`。再加入第三个字 `0x65C5`，普通和为 `0x11F7B`，回卷得到 `0x1F7C`，检验和为 `0xE083`。接收方把数据字和检验和一起求反码和，应得到 `0xFFFF`。

通过校验仅表示未检出差错。例如两个字分别增加和减少同一个数，某些情况下会保持相同反码和，因而漏检。协议允许关闭校验的条件也要区分：IPv4 UDP 字段全 0 可表示未使用校验；若实际计算值为 0，应按协议写成全 1，避免与“未校验”混淆。IPv6 的普通 UDP 使用条件不能照搬 IPv4 的可选规则。

5.12 逐字节维护确认号与接收窗口

设服务器接收缓冲为 8 B，应用暂不读取；客户下一数据序号为 600，服务器本方向下一序号为 200。客户先发送 600～602 共 3 B，服务器累计确认号 603，剩余窗口 5 B；再收到 603～604 与 605～606 两个 2 B 段后，累计确认 607，剩余窗口 1 B；收到 607 这一字节后缓冲满，累计确认 608、窗口 0。若应用随后读取全部 8 B，可通告窗口 8 B，确认号仍为 608。

新收到的客户数据	连续已收到至	ACK	应用未读时 rwnd/B
600～602	602	603	5
603～604	604	605	3
605～606	606	607	1
607	607	608	0
应用读取 8 B，无新数据	607	608	8

若接收方采用累计确认，连续两个 2 B 段也可以只发最终 ACK607；表中逐段列出是为了说明各次状态。服务器反方向发送 5 B 数据时序号从 200 开始，客户返回 ACK205；这个 205 只确认服务器字节流，与客户已经发送到 607 的位置相互独立。捎带确认把反方向的新数据与本方向接收进度放进同一段，两个序号空间仍须分开记账。

计数·原 ACK 加三个重复

首次 ACK623 推进确认；再收到三个相同的重复 ACK 才达经典快重传阈值。

5.13 快重传必须逐次计数

客户连续发送序号 622、623、624、625 的四个 1 B 段，623 丢失；假设接收方在 622 之前已经连续收到至 621，则收到 622 后首次返回 ACK623。收到 624 和 625 后各重复 ACK623，这时只有 **2 个重复 ACK**，尚未达到 3 个重复 ACK 阈值。若随后 626 也到达并再次返回 ACK623，才得到第 3 个重复 ACK，触发重传缺失的 623。

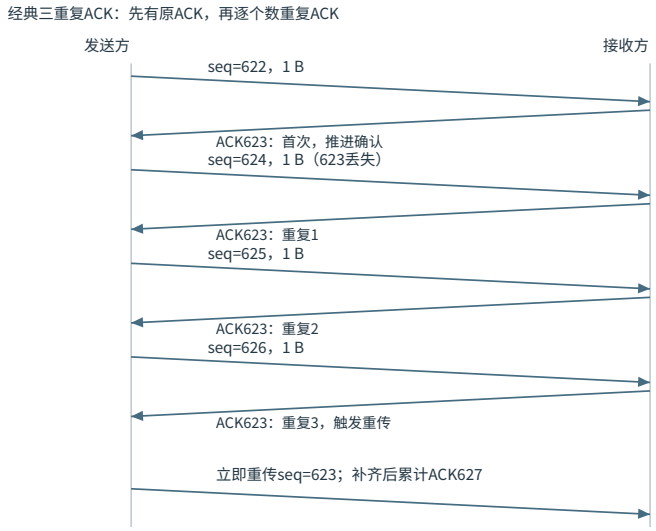


Figure 5.7: 第一个 ACK623 推进确认，后三个相同确认分别计重复 1、2、3

若总共只有 622～625 四段而没有更多到达事件，经典三重复 ACK 规则下应等超时等机制；不能为了让图符合预想结果，把首次正常 ACK 也算入重复次数。收到重传 623 且 624～626 已经缓存时，累计确认可直接推进到 627，无须让发送方重新发送已缓存的数据。

教材按轮次讨论拥塞窗口时，若门限 16 MSS，慢开始发送轮次为 1、2、4、8、16，随后拥塞避免为 17、18……。在 cwnd=24 MSS 时超时，门限取 12 MSS、窗口重置 1 MSS，再经 1、2、4、8、12 后转为 13、14……；在相同点触发快重传并完成课件简化快恢复，则从 12 MSS 进入拥塞避免。窗口增长轨迹必须同时写清初值、门限、丢包事件和记录时刻。

5.14 主动回忆自测

以下为自拟题。每道窗口题都画出本方向字节区间，并说明采用的教材算法。

1. A 发 SYN 时 seq=100，之后发送 500 B 应用数据，再发不带数据的 FIN。FIN 的 seq、对方确认 FIN 的 ack 分别是多少？

2. 最新有效 ack=5001、rwnd=4000 B、cwnd=3000 B；已发未确认区间 5001～6500。还可新发多少？最大新字节序号是多少？
3. 被动关闭端进入 CLOSE-WAIT 后，是否已经不能再发应用数据？
4. 一份最初的正常 ACK 加两份相同的重复 ACK，是否达到 “三个重复 ACK” 的计数？
5. TCP 能否像 UDP 一样把两次应用写入的边界自动交给接收应用？服务器同一个 80 端口能否同时服务多个 TCP 客户？
6. 12 B 应用数据装入 UDP，总长度字段是多少？伪首部和求和补齐的 0 字节会在线上成为额外 UDP 数据吗？
7. IP 总长 100 B、IHL=6、TCP 数据偏移 =6，未分片，应用数据多少字节？
8. 超时重传后收到一个 ACK，为什么不能总用它的到达时刻减第一次发送时刻当 RTT 样本？
9. rwnd 降至 0 是否证明网络拥塞？用于防止零窗口通知丢失后永久等待的计时器是什么？
10. 旧 SRTT 为 100 ms、平滑系数 1/8、新有效样本 140 ms，新 SRTT 是多少？Reno 与 Tahoe 遇到三个重复 ACK 后的行为是否必相同？

5.14.1 核对答案

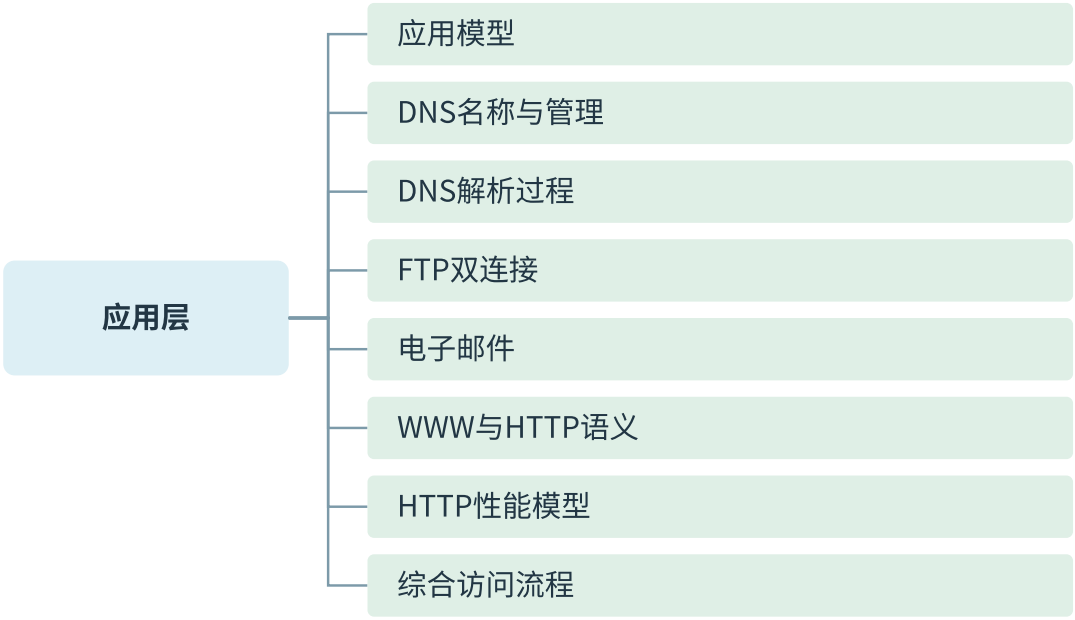
1. 数据 101～600，FIN seq=601，对方 ack=602。
2. 总额度 3000 B，已占 1500 B，尚可发 1500 B；允许新发 6501～8000。
3. 尚可发送，直到本机应用关闭发送方向；收到 FIN 只关闭对端到本端方向。
4. 没有，首次正常 ACK 之后还需 3 份重复 ACK。
5. 不保证，TCP 是字节流；可以，源 IP/端口等使四元组不同。
6. 20 B；两者仅用于校验计算，均不会变成额外应用载荷。
7. $100 - 4 \times 6 - 4 \times 6 = 52$ B。
8. ACK 可能确认原段也可能确认重传段，样本有歧义；Karn 原则应排除相应有歧义样本。
9. 不证明，可能只是接收缓存不足；使用持续计时器触发零窗口探测。
10. 105 ms；不相同，Reno 快恢复与 Tahoe 退回慢开始的处理不同，按具体题设算法判断。

计数·原 ACK 加三个重复

首次 ACK623 推进确认；再收到三个相同的重复 ACK 才达经典快重传阈值。

应用层 6

章节脉络



浏览器中的一个 URL 同时关联资源名称、服务器地址、传输连接和应用报文。计算访问时间时，要按依赖次序展开：先知道服务器地址，才能连接；先收到并解析 HTML，才能知道其中引用哪些资源。缓存命中、连接复用和流水请求改变的是这条依赖链中的具体步骤。

错项排除·角色由这次交互决定

一台机器可向 DNS 发查询，同时给其他客户提供 Web 服务。它在前一次交互中是客户，在后一次交互中是服务器；设备性能、是否长期在线以及机箱外观都不能替代应用角色的判断。

6.1 C/S 与 P2P：角色由谁发起服务请求决定

客户/服务器模型 C/S 中，服务器提供相对稳定的服务入口，客户主动发起请求。服务器可同时服务多个客户，客户不需要一直保持在线。这里“客户”与“服务器”是本次应用交互的角色，不是机器性能等级；同一主机可运行服务器，也可作为其他服务的客户。

P2P 让对等节点既能获取资源，也能向其他节点提供资源。它可以减少单一服务器的分发负担，但节点上下线、查找、安全和资源一致性更难管理。P2P 应用可以使用集中索引服务器辅助发现节点，不能因为存在一个中心组件，就断言所有数据必须经该中心转发。

以把大小为 F 的文件分发给 N 个接收者为例，纯 C/S 至少要求服务器上传 NF，因此时间不小于 NF/u_s；每个接收者还受自身下载速率限制。P2P 允许已收到数据的节点继续上传，能增加总上传能力，但仍至少受服务器上传、最慢接收者下载和全系统总上传量等条件限制。这个比较依赖可用上传率和持续参与等条件，不把“P2P 更快”写成不受网络约束的绝对结论。

文件分发先求不可突破的下界

设服务器上传率 u_s、最慢客户下载率 $d_{\min}$，客户上传率为 u_i，文件大小为 F，共有 N 个客户。纯 C/S 至少受

$$T_{CS} \geq \max\left(\frac{NF}{u_s}, \frac{F}{d_{\min}}\right)$$

限制；理想 P2P 还需服务器先贡献至少一份文件，且全系统总上传量要覆盖 NF，因此

$$T_{P2P} \geq \max\left(\frac{F}{u_s}, \frac{F}{d_{\min}}, \frac{NF}{u_s + \sum_i u_i}\right).$$

这些是持续参与、可用速率等理想条件下的必要下界，不能据此保证任意拓扑与实际调度都恰好达到。

推导只需守住数据守恒：纯 C/S 的 N 份副本都由服务器上传；P2P 则允许已收到数据的客户继续贡献上传，但所有副本的数据仍必须由系统某处发出。任何一个最慢下载者还必须收到完整 F，因此各资源下界应取最大值，不能相加成三个顺序阶段。

6.2 DNS 层次、服务器与区

DNS 把域名与资源记录关联。域名是从右向左逐级具体化的层次名称，完整域名可在末尾带根点。 `www.abc.com` 中 com 为顶级域，abc 是其下的一个域，www 是进一步的名称标签；不能只按点的个数断言每一级都由独立机器负责。

定义 6.2.1 (域与权威区)

域是名称空间中的子树，**区**是由某组权威服务器管理并提供权威数据的范围。一个域可以把某些子域委派出去，因此域与区不必相同。根服务器提供顶级域的委派信息，不存放全世界每台主机的最终地址；权威服务器回答其负责区内的记录。

常见记录有 A（IPv4 地址）、AAAA（IPv6 地址）、NS（权威服务器）、CNAME（规范名称别名）和 MX（邮件交换）。一个名称可以有多个地址，多个名称也可以指向同一个地址，域名、IP 与主机之间都不要求一一对应。〖6.2-001〗

域名书写要区分 “教材普通主机名规则” 与一般 DNS 记录名称。普通主机名标签常用字母、数字和连字符，大小写不敏感，较低层级写在左侧，顶级域在右侧。单标签最长 63 字节；DNS 线格式完整名称最长 255 字节，包含标签长度字节和末尾根标签。常见点分文本不带末尾根点时最多 253 字节，不能把线格式上限不加说明地当成任意显示字符串的字符上限。

顶级域可区分国家/地区类（如 cn）、通用类（如 com、org）与基础设施用途的 arpa 等。管理层级与实际服务器部署不是一一对应，反向名称解析的组织也不能简单写成把 IP 字符串交给普通 A 记录查询。

本地域名服务器即递归解析器，通常由组织或运营商等提供，位置不必与主机处在同一个以太网内。主机先查浏览器或系统缓存，再向配置的解析器发出请求；不能把本地解析器直接等同于根服务器。

最小反例·一个域可以跨多个区

abc.com 把子域 lab.abc.com 委派给另一组权威服务器后，lab 仍在 abc.com 名称子树内，却已不属于父区的直接权威数据范围。这个委派例子足以排除 “域和区永远完全相同” 的说法。

6.3 递归、迭代与缓存：谁又发了一次查询

不变量·每条查询都有发出者

主机向本地解析器的一条请求，不是本地解析器发出的请求。题目问某台服务器发出多少次时，应逐条画箭头并只数它发出的边；缓存命中可使远程次数归零，却不改变递归与迭代的定义。

递归查询要求被请求的服务器继续完成解析，最终返回答案或失败；迭代查询允许服务器返回下一步应询问的服务器信息，由原查询者继续询问。常见流程是主机向本地解析器请求递归服务，本地解析器依次对根、顶级、权威服务器进行迭代查询。

例如解析 `www.abc.com`：本地解析器先问根，取得 com 的服务器信息；再问 com，取得 abc.com 的权威服务器；最后问权威服务器取得 Web 地址。缓存中若已有最终 A/AAAA 记录，可不发远程查询；若只有某级委派，可以从该级继续。缓存受 TTL 有效期限制，不能无条件永久使用。

比较递归与迭代时，固定三件事：当前谁提出查询，收到的答复是最终答案还是下一站信息，由谁发下一条查询。递归把继续解析的工作交给被请求者，迭代让当前查询者沿委派继续。缓存命中只会改变需要走几步，并不改变这两种交互的定义；“用了缓存” 也不能直接推出请求来自根服务器还是本地解析器。

6.3.1 例 6-1　问的是本地服务器发出多少次查询

例 6.3.1 (配套例题)

配套例题。假设所有域名服务器均采用迭代查询方式进行域名解析。当 H4 访问规范域名为 `www.abc.xyz.com` 的网站时，域名服务器 201.1.1.1 完成该域名解析过程中，可能发出 DNS 查询的最少和最多次数分别是（　）。共用网络拓扑如下。

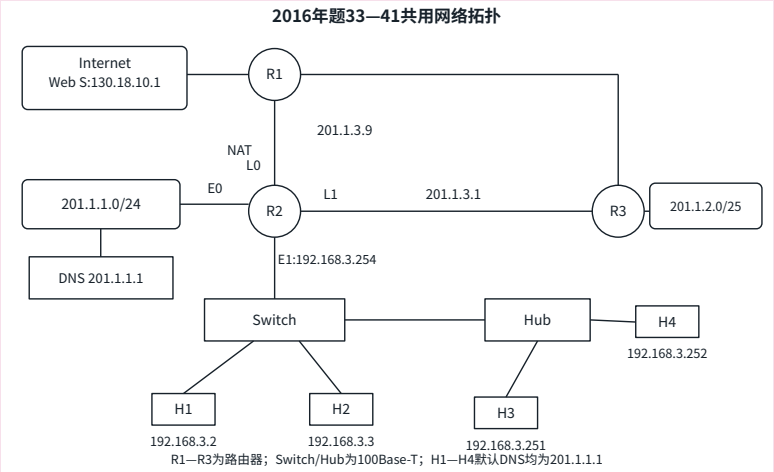


Figure 6.1: 2016 年共用拓扑；本题只使用 H4 及其默认 DNS 服务器 201.1.1.1

A. 0，3　　B. 1，3　　C. 0，4　　D. 1，4

解析。计数主体是 201.1.1.1，不是 H4。最终地址在有效缓存中时，本地服务器不需向外查询，最少 0 次；完全无可用缓存，且相应各级分别委派时，可依次询问根、com、xyz.com、abc.xyz.com，共 4 次，选 **C**。H4 向 201.1.1.1 的最初请求不是这台服务器 “发出” 的请求，不能加入。

“最多 4 次” 采用原题的规范域名及逐级委派模型，不计别名链、失败重试或负载均衡额外查询。若改问主机发出几条请求，或所有层级均以递归方式向下转交，通信主体与计数就要重新列出，而不是照抄 0、

4。

DNS 常用 UDP 端口 53，也支持 TCP，例如区域传送、较大响应或按协议退回 TCP 的场景。不能简单断言 DNS 只用 UDP，更不能将“DNS 也能用 TCP”误读为每次普通查询都必须先三次握手。

最小反例·数据端口不总是 20

FTP 被动模式由服务器通告一个监听数据端口，再由客户连接该端口。控制仍走服务器 21 端口，数据却可走另一个值，因此“服务器所有数据连接端口恒为 20”已被这个合法模式反驳。

6.4 FTP：控制命令与文件数据为什么分两条连接

FTP 运行在 TCP 上，控制连接用于登录、目录、传送命令与状态回复，服务器熟知端口为 21；数据连接用于文件或目录数据。在一次会话中，控制连接可持续存在，多次传送按需要建立、关闭数据连接，控制报文因此不必夹在文件字节流里，也能被识别。

6.4.1 例 6-2　传递 FTP 命令使用哪条连接

例 6.4.1 (配套例题)

配套例题。FTP 客户和服务器间传递 FTP 命令时，使用的连接是（　）。

A. 建立在 TCP 之上的控制连接

B. 建立在 TCP 之上的数据连接

C. 建立在 UDP 之上的控制连接

D. 建立在 UDP 之上的数据连接

解析。先由“FTP 命令”确定控制连接，再由 FTP 的传输协议确定 TCP，选 **A**。文件内容放在数据连接；登录服务器时，第一阶段建立的也是控制连接，不能等文件传送结束后才建立命令通道。〖6.3-005〗

主动模式由服务器向客户指定的地址和端口发起数据 TCP 连接，传统服务器数据源端口为 20；被动模式由服务器通告监听端口，再由客户主动连接该数据端口。不能说“所有 FTP 数据连接的服务器端口永远为 20”，也不能把主动/被动模式误解为控制连接的建立顺序。

教材常用“一个监听主进程，加多个会话控制/数据处理任务”解释 FTP 并发：监听 21 端口的入口接收新客户，每个会话维护自己的控制状态，具体文件传输再使用数据连接。实际服务器可用进程、线程或事件驱动实现，概念模型不要求每条连接都必须创建一个操作系统进程。

6.5 邮件系统、邮件格式与 MIME

邮件系统包括用户代理、邮件服务器和传送协议。用户代理负责编辑、显示和管理邮件，邮件服务器负责队列、邮箱及服务器间传送。一个“发送成功”的界面提示，可能只表示本机或提交服务器接受了邮件，不等于对方用户已经读取。

邮件由首部和正文组成，以空行分隔；首部中的 From、To、Subject 等表达用户可见信息，传输过程中 SMTP 还使用信封发件人与收件人控制实际投递。这两套地址可能不同，不能仅按邮件里显示的地址推断所有投递端点。

基础 SMTP 按 7 位 ASCII 文本传输，不能直接把任意二进制图像当作普通 7 位正文。MIME 定义内容类型、字符集、传送编码和多部分结构，使非 ASCII 文字与附件能表示为可传送内容。例如 Base64 将 3 字节转成 4 个 ASCII 字符，载荷大约膨胀到原来的 4/3，此外还有格式

开销。编码不是加密，收件人仍可按公开规则恢复原内容。现代扩展可协商更丰富的能力，作基础 SMTP 题时先确认题目采用哪一种范围。

不变量·读与发是不同应用动作

SMTP 负责提交和转送，POP3 负责读取邮箱。同一用户的一封邮件可能依次经过两种协议，不能因二者都处理邮件就互换。POP3 会话能读取多封，下载后是否删除还取决于相应操作和更新阶段。

6.6 SMTP 负责发送，POP3 负责读取

SMTP 是推送协议，用于用户代理提交邮件、以及邮件服务器之间转送邮件；服务器间常用 TCP 端口 25，用户提交还常见经认证的提交服务。POP3 是读取协议，用于用户代理从邮件服务器取邮件，基本服务使用 TCP 端口 110。协议名区分的是应用动作，不能因为都处理邮件就认为两者可以相互代替。

典型路径为：发送者用户代理通过 SMTP 提交 → 发送方邮件服务器按队列与 DNS 中的 MX 记录寻找收件域邮件服务器 → 服务器间通过 SMTP 转送 → 收件用户代理用 POP3 等读取。使用 Web 邮箱时，浏览器与 Web 服务还可能使用 HTTP，后端仍另有邮件传送机制。

POP3 依次经过授权、事务处理和更新等阶段。用户可列出、读取多封邮件，删除通常要等到会话的相应更新阶段才落实；不能把 “读取” 无条件等同于立即从服务器删除，也不能说一封邮件必新建一条 TCP 连接。

6.6.1 例 6-3　一次 POP3 连接可以读几封邮件

例 6.6.1 (配套例题)

配套例题。下列关于 POP3 协议的叙述中，正确的是（　）。

I. 支持用户代理从邮件服务器读取邮件

II. 支持用户代理向邮件服务器发送邮件

III. 支持邮件服务器之间发送与接收邮件

IV. 支持通过一条 TCP 连接收取多封邮件

A. 仅 I、IV　　B. 仅 II、III　　C. 仅 I、II、III　　D. 仅 I、III、IV

解析。 POP3 用于用户代理读取，I 正确；发送和服务器间转送由 SMTP 等机制承担，II、III 错误；事务阶段可在同一 TCP 连接内处理多封邮件，IV 正确，选 **A**。决定答案的是协议负责哪个方向，而不是 “都传邮件” 这个共同点。

6.7 WWW、URL 与资源位置

WWW 用超链接把资源组织起来，浏览器按 URL 定位和访问资源。URL 常包含方案、主机、可选端口、路径、查询及片段。例如 `http://www.abc.com:80/index.html?q=408#part1` 中，http 决定访问方案，主机名用于解析服务器地址，路径和查询参与 HTTP 请求定位，`#part1` 由浏览器用于文档内定位，不作为普通 HTTP 请求目标发送给服务器。

网页不是必然只有一份文件。HTML 可再引用图片、脚本、样式表等对象，浏览器通常要先收到 HTML，或在解析过程中逐步发现这些地址。它们可能来自同一服务器，也可能来自不同域名，需要另作 DNS 或连接判断。不能把 “同一个页面” 当成 “只需要一个网络往返”。

HTTP 是应用层协议，通常运行在 TCP 上；HTTPS 在 HTTP 通信中加入 TLS 等安全机制。本章时延题采用题目明确的 HTTP/1.0 或 HTTP/1.1 模型，不把 HTTP/2 多路复用或 HTTP/3 的 QUIC 机制混入旧

题。高版本存在，不会自动修改题干所限定的协议行为。

6.8 HTTP 请求、连接复用、缓存与时延

HTTP 请求行包含方法、请求目标、版本，后接首部、空行和可选消息体；响应包含状态行、首部及消息体。GET 用于获取资源，HEAD 获取相应首部信息，二者在协议语义上属于安全方法；POST 提交数据供资源处理，可能引起状态变化。判断题先按方法的规定语义作答；服务端的错误实现或附带日志等行为不能反过来改变这些方法的协议分类。状态码 200 表示请求成功，301/302 表示重定向，304 用于条件请求中的资源未改动，404 表示未找到等。

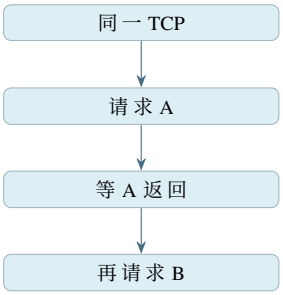
HTTP 被称为无状态协议，是指一次请求的处理不自动要求服务器记住上次会话状态。Cookie 可以让客户随请求携带标识，服务器据此关联状态；持久 TCP 连接则减少反复建立连接的开销。**无状态、Cookie 和连接是否持久是三个不同问题**，不能相互推出。

非持久 HTTP 通常每个对象使用一条 TCP 连接，题目忽略发送时延，一个小对象至少需要 1RTT 建立连接、1RTT 完成请求响应。若允许多个连接并行，可在发现对象后并行获取；若不允许并行，各对象就串行承担这两项。持久 HTTP 在同一连接上取多个对象，非流水方式仍按请求响应逐个等待；HTTP/1.1 流水线允许前一响应尚未完成就发后续请求，但尚不知道的资源地址无法凭空提前请求。

只有条件一致，比较才有可计算的结果：DNS 已完成、TCP 尚未建立，HTML 引用同站的 $m \geq 1$ 个小对象，必须先收到 HTML 才知道它们；不并行建连接，忽略报文传输、处理、拥塞与流量控制，所有往返均为同一个 RTT。

最小反例·持久连接仍可逐个等待

已建立一条 TCP 连接，先请求图片 A，等它完整返回后才请求 B，这是持久但非流水。两对象仍付两次请求响应等待，所以“用了持久连接就所有对象只要 1 RTT”的说法不成立。



持久连接仍可以不使用流水

模型	TCP 连接复用	对象请求能否重叠	从建立首连接到收齐的时间
非持久、串行	每个对象另建，共 $m+1$ 条	每次取完再建立下一条	$2(m+1)RTT$
持久、非流水	所有对象共用一条	HTML 及后续对象逐个等待	$(m+2)RTT$
持久、流水	所有对象共用一条	HTML 后，已知的 m 个请求可连续发出	$3RTT$

三行都包含首次 TCP 建立的 1RTT；若计时从已建连接上的 HTTP 请求发出开始，应各自去掉这一项。流水的 3RTT 仍含“握手—HTML—后续对象”三个依赖阶段；若对象较大、窗口不足或来自不同服务器，这张表的理想时间就不再直接适用。下面例题只改变其中的起点、并行条件或窗口约束。

HTTP 时延按依赖分段

先核对 DNS 是否完成、TCP 是否已建，再问 HTML 何时揭示资源地址；在后续资源阶段才区分是否并行、是否复用、是否流水。之后叠加题目明确给出的发送、拥塞窗口和处理开销。每次减少一个 RTT，都应说明是哪次握手或等待被免去。

例 6.8.1 (请求目标不包含片段标识)

访问 `http://www.abc.com/index.html?q=408#part1`，在题设 HTTP/1.1 模型中，请求可写为：

```
GET /index.html?q=408 HTTP/1.1
Host: www.abc.com
```

错项排除·流水不能猜未知资源

在 HTML 未返回且没有其他先验信息时，浏览器还不知道其中的图片 URL。允许流水只能把发现后的多个请求重叠，不能把 HTML 阶段也凭空抹去；这正是理想模型仍保留三个 RTT 阶段的原因。

路径与查询串交给服务器定位资源，`#part1` 留给浏览器在文档内定位；这段请求文本并未把 DNS、TCP 握手或 TLS 开销包含进去，计时仍须按题设另算。

6.8.1 例 6-4　非持久且不允许并行，要算上 HTML 本身

例 6.8.2 (配套例题)

配套例题。若浏览器不支持并行 TCP 连接，使用非持久的 HTTP/1.0 协议请求浏览 1 个 Web 页，该页中引用同一网站上的 7 个小图像文件，则从浏览器为传输 Web 页请求建立 TCP 连接开始，到接收完所有内容为止，所需要的 RTT 数至少是（　）。

A. 4　B. 9　C. 14　D. 16

解析。一共 8 个对象：HTML 与 7 张图片。每个对象单独建立 TCP 连接并请求响应，各 2RTT，且题目禁止并行，故总计 $8 \times 2 = 16RTT$，选 **D**。只计算 7 张图会得到 14，遗漏页面本身。题干计时从请求建立连接开始，不是从 HTTP 请求已经发出后开始。

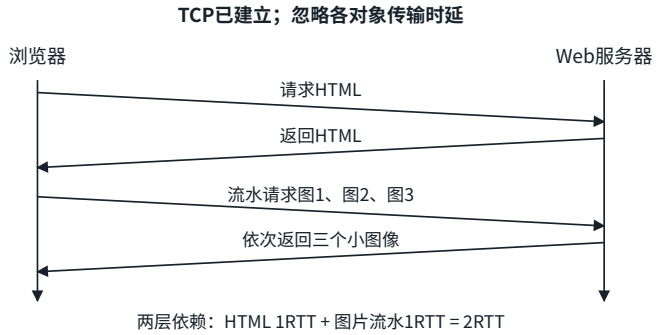
6.8.2 例 6-5　流水线仍不能跨过资源发现依赖

例 6.8.3 (配套例题)

〖6.5-012〗假设主机通过 HTTP/1.1 流水线方式请求 Web 服务器上的 rfc.html，它引用同目录下 3 个 JPEG 小图像，只有收到 rfc.html 后才能发送对应图像请求。一次请求响应为 RTT，忽略其他时延，不考虑拥塞与流量控制。从发出 HTTP 请求报文开始，到收到全部内容，所耗时间是（　）。

A. 2RTT　B. 2.5RTT　C. 4RTT　D. 4.5RTT

解析。起点已经是 HTTP 请求发出，不再加 TCP 握手。第 1RTT 取 HTML，得到 3 个图片地址；3 个请求可流水发送，忽略传输时延时，再 1RTT 收到图片，共 2RTT，选 **A**。若题目不允许流水，应在同一连接逐对象等待，HTML 及 3 图共 4RTT。只有改变请求能否重叠这类条件，计算才会改变；不能把 “持久” 一词直接等同于 1RTT。



不变量·304 仍是一份网络响应

条件请求发到服务器后收到 304，说明发生了验证往返，只免去了完整实体内容。新鲜缓存直接命中则可能完全不发该请求。两种缓存行为都节省流量，却不具有相同的时延表达式。

Figure 6.2: 连接已建立时，HTML 完成后才能发送三个图片请求；流水阶段可重叠

6.8.3 例 6-6　DNS 与 Web 的时间由哪几项组成

例 6.8.4 (配套例题)

配套例题。假设下图网络中的本地域名服务器只提供递归查询服务，其他域名服务器均只提供迭代查询服务；局域网内主机访问 Internet 上各服务器的 RTT 均为 10 ms，忽略其他各种时延。若主机 H 通过超链接 `http://www.abc.com/index.html` 请求浏览纯文本 Web 页 index.html，从单击超链接到浏览器收到页面，所需的最短与最长时间分别是（　）。

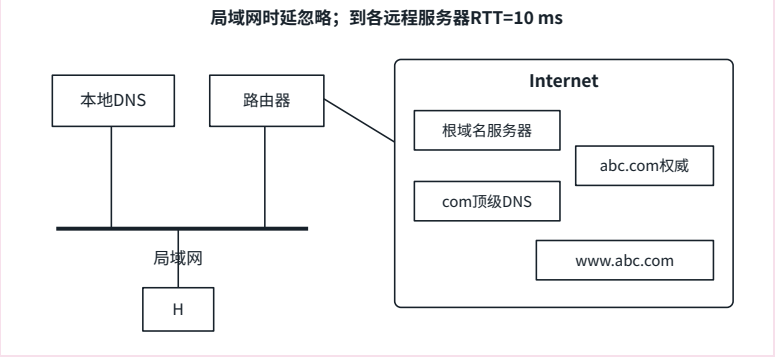


Figure 6.3: 主机、本地 DNS 与路由器共享局域网，远程服务器分别承担名称解析和 Web 响应

A. 10 ms，40 ms　　B. 10 ms，50 ms
C. 20 ms，40 ms　　D. 20 ms，50 ms

解析。采用题目首次取页面、尚未建立目标 TCP 连接的模型。有效 DNS 缓存命中时，不需要远程 DNS 往返，但 TCP 建立与 HTTP 请求响应各 1RTT，最短 20 ms。完全无所需缓存时，本地解析器依次询问根、com、abc.com 权威服务器，增加 3RTT；再取 Web 共 5RTT=50 ms，选 **D**。主机与本地 DNS 之间的局域网时延已忽略，不能额外加一次 10 ms。

如果题目再允许浏览器已缓存新鲜的 HTML 副本，可能无须访问服务器；若 TCP 已经建立，也会少一个握手往返。这些是改变原题模型的附加条件，不能在原题中任意引入后声称存在另一个更短答案。

6.8.4 例 6-7 收到 HTML 后，拥塞窗口还限制图片分几轮

例 6.8.5 (配套例题)

配套例题。假设主机 H 通过 HTTP/1.1 请求 Web 服务器 S 上的 news408.html，它引用同目录下 1 幅图像。HTML 文件大小为 1 MSS，图像 3 MSS，RTT=10 ms，忽略 HTTP 响应首部开销和 TCP 段传输时延。若 H 已完成域名解析，则从 H 请求与 S 建立 TCP 连接时刻起，到收到全部内容，所需的时间至少是（ ）。

A. 30 ms B. 40 ms C. 50 ms D. 60 ms

解析。原题答案采用教材模型：慢开始初始窗口为 1 MSS，使用同一条持久连接，HTML 收到后才请求图像。TCP 建立用 1RTT；HTML 请求与 1 MSS 响应再 1RTT，服务器收到其确认后窗口可增长为 2 MSS；图像有 3 MSS，先发 2 MSS，收到确认后再发余下 1 MSS，图片阶段还需 2RTT。因此共 4RTT=40 ms，选 **B**。

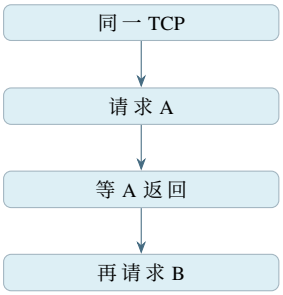
这里题源未在题干单独写初始拥塞窗口，不能把结论推广为所有 HTTP/1.1 实现的固定最短时间。若明确允许较大初始拥塞窗口，3 MSS 图片可在一轮内返回，则题目的数值会改变。遇到这类题，应在计算旁写清采用的 TCP 初态，不能一边沿用 1 MSS 答案、一边声称初态完全无关。

HTTP 缓存还要区分新鲜命中和条件验证。新鲜缓存可直接响应而不向源站发送请求；过期或需重新验证时，可用 If-Modified-Since 或 If-None-Match 等条件请求，服务器返回 304 时不传原完整实体，但验证过程仍有请求响应时延。代理缓存能缩短远程路径、减少上游流量，但并不意味着缓存中的对象永不改变。

回看各应用协议，可按“谁向谁做什么、用何种传输、端口属于哪一方”比较，不能只把名称和端口罗列成表，就以为各协议可以互相替代。

最小反例·持久连接仍可逐个等待

已建立一条 TCP 连接，先请求图片 A，等它完整返回后才请求 B，这是持久但非流水。两对象仍付两次请求响应等待，所以“用了持久连接就所有对象只要 1 RTT”的说法不成立。



持久连接仍可以不使用流水

应用任务与协议	主要交互端点和动作	本章基本承载与端口	需要保留的条件
名称解析 DNS	主机/解析器向 DNS 服务器询问记录	UDP 或 TCP，服务端 53	缓存、递归/迭代决定查询路径；不是每次必经根
自动配置 DHCP	客户向服务器申请地址与其他配置	UDP，服务器 67、客户 68	初始广播与续租单播要按阶段判断
文件传送 FTP	客户和服务器先交换控制命令，再传数据	TCP 控制连接服务端 21；数据另建连接	主动/被动决定数据连接发起方和数据端口
发送邮件 SMTP	用户代理提交、邮件服务器之间转送	TCP，服务器间基本端口 25	推送邮件，不承担用户读取邮箱的角色
读取邮件 POP3	用户代理向邮件服务器读取	TCP，基本服务端 110	一条连接可处理多封邮件，读取不必立即删除
获取 Web 资源 HTTP	浏览器向 Web 服务器请求资源	本章 HTTP/1.x 采用 TCP，默认服务端 80	连接复用、流水、缓存和资源依赖分别判断

这些是本章基础模型的端口与承载，显式 URL 端口、加密服务或其他扩展须按题意另算。MIME 说明邮件内容类型与编码，不取代 SMTP 的投递过程；HTTP 的无状态也不取代 TCP 连接管理。遇到跨协议题，先列应用依赖，再把 DNS 查询、连接建立、请求响应与数据发送逐项接入时间线。

6.9 综合依赖链：名称、连接与逐跳封装

跨章节题先列必须完成的应用动作，再把每个动作接到传输连接和本地链路上。DNS 负责名称，ARP 负责当前邻居 MAC，TCP 负责相应连接的序号与窗口；交换机只从已经实际到达的帧学习，不会提前知道未来哪台主机将参与通信。

不变量·应用数据量与 FIN 分开计

FTP 文件 18000 B，从 SYN 序号 100 开始，则首数据 101，末数据 18100，FIN 为 18101，对方确认 FIN 为 18102。控制连接已有自己的序号空间，不能混进这条数据连接的累计字节数。

6.9.1 例 6-8　FTP 上传时，把连接、字节号与拥塞轮次放到同一时间轴

例 6.9.1 (配套例题)

配套例题。某网络拓扑如下图。H 登录 FTP 服务器后，向服务器上传 18000 B 文件 F。H 为传输 F 建立数据连接时选择初始序号 100，MSS=1000 B，拥塞控制初始阈值 4 MSS，RTT=10 ms，忽略 TCP 段的传输时延。在 F 的传输过程中，H 均按 MSS 大小发送数据，未发生差错、丢包和乱序。

Figure 6.4: H 经交换机、路由器访问 Internet 中的 FTP 服务器，控制连接与数据连接均由 TCP 承载

（1）FTP 控制连接是持久的还是非持久的？数据连接是持久的还是非持久的？H 登录服务器时建立的 TCP 连接是控制连接还是数据连接？

（2）H 通过数据连接发送 F 时，F 第 1 个字节的序号是多少？断开数据连接过程中，FTP 服务器发送的第二次挥手 ACK 段的确认号是多少？

（3）当 H 收到确认号 2101 的确认段时，拥塞窗口调整为多少？收到确认号 7101 的确认段时，拥塞窗口调整为多少？

（4）H 从请求建立数据连接开始，到确认 F 已被服务器全部接收，至少需要多久？期间应用层数据的平均发送速率是多少？

解析。先把 FTP 会话与一次文件传输分开。控制连接在整个 FTP 会话期间保持，承载登录、命令与控制信息，因此是**持久连接**；本题的数据连接服务于本次文件传输，传完后关闭，是**非持久连接**。登录时先建立的是**控制连接**，已经完成的登录过程不再计入第（4）问的数据连接计时。

数据连接的 SYN 占序号 100，F 首字节从 **101** 开始。18000 B 文件占序号 101～18100，发送完后 H 的 FIN 占 18101，服务器第二次挥手的 ACK 应确认下一个期望序号，故为 **18102**。这里确认的是文件数据再加 FIN，不能把登录控制连接的序号混进来。

拥塞控制沿用教材的轮次模型：初始 $cwnd = 1$ MSS，达到 4 MSS 门限后按每 RTT 增加 1 MSS。先看 ACK 落在哪个时刻：确认号 2101 累计确认 $2101 - 101 = 2000$ B，即前 2 个段。第 1 个 ACK 使 cwnd 从 1 增为 2 MSS；第 2 个 ACK 是第二轮中较早的那个，此时仍在慢开始，cwnd 从 2 增为 **3 MSS**。不能把 “第二轮结束时 4 MSS” 提前用于这个 ACK。

确认号 7101 累计确认 $7101-101=7000$ B，即 $1+2+4=7$ 个段，恰好完成第 3 轮。此轮开始时已到 4 MSS 门限，采用拥塞避免，第 3 轮结束后窗口为 **5 MSS**。下表同时列出数据发送、确认时刻与累计量，时间均从数据连接 SYN 发出算起。

阶段	开始时刻/ms	本轮 cwnd/MSS	本轮发送段数
数据连接握手	0	—	—
数据第 1 轮	10	1	1
数据第 2 轮	20	2	2
数据第 3 轮	30	4	4
数据第 4 轮	40	5	5
数据第 5 轮	50	6	6

阶段	累计发送段数	本轮全部确认到达/ms
数据连接握手	0	10
数据第 1 轮	1	20
数据第 2 轮	3	30
数据第 3 轮	7	40
数据第 4 轮	12	50
数据第 5 轮	18	60

建立数据连接用 1 RTT，18 个段按 1、2、4、5、6 分 5 轮发完，并在最后一轮结束时得到全文件确认，所以总计 $6\text{RTT}=$ **60 ms**。期间应用层平均速率为 $18000\times 8/0.060=$ **2.4 Mb/s**，也就是 300000 B/s=0.3 MB/s。分母按题目包含握手、不包含连接释放；不能只用 50 ms 的数据传输阶段，也不能因服务器较早收到最后一比特就省去最后确认的回程。

这里的按轮增长是题目采用的教材近似，阈值前逐 ACK 检查仍有意义；拥塞避免阶段不宜把轮次表误解成每个 ACK 都使 cwnd 增加整整 1 MSS。若要描述某个操作系统的精确逐 ACK 轨迹，还需其拥塞算法和 ACK 策略。

最小反例·缓存命中会减少 ARP 广播

若本次已缓存 Web 服务器 IP，就可直接解析网关 MAC，不必先访问本地 DNS。即使 ARP 表和交换表为空，也不足以证明必出现两次 ARP 广播；该结论还需要一次新的 DNS 查询这一条件。

6.9.2 例 6-9　从 DNS 到 HTTP，交换机只学习实际发送者

例 6.9.2 (配套例题)

配套例题。某网络拓扑如下图，以太网交换机 S 通过路由器 R 与 Internet 互联。路由器接口、本地 DNS、H1、H2 的 IP 地址与 MAC 地址如图。t_0 时 H1 的 ARP 表和 S 的交换表均为空；H1 此时用浏览器通过域名 www.abc.com 请求访问 Web 服务器。$t_1>t_0$ 时，S 第一次收到封装 HTTP 请求报文的以太网帧。其间没有与此次 Web 访问无关的网络通信。

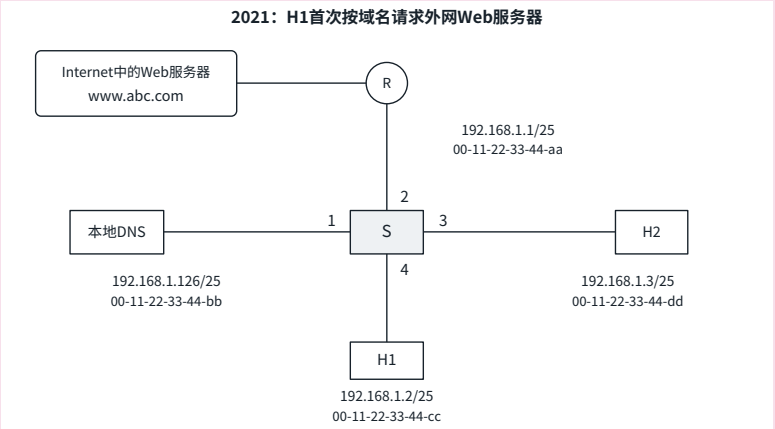


Figure 6.5: 本地 DNS 接 S 的 1 口，R 接 2 口，H2 接 3 口，H1 接 4 口；四者同属 192.168.1.0/25

（1）从 t_0 到 t_1，H1 除 HTTP 外还运行哪个应用层协议？从应用层到数据链路层，该协议报文经过哪些协议逐层封装?

（2）S 交换表结构为 “MAC 地址、端口”，t_1 时交换表内容是什么?

（3）其间 H2 至少接收多少个与此次 Web 访问有关的帧？是什么帧？目的 MAC 地址是什么?

解析。解题入口是按依赖关系列出 “先知道谁的地址，才能发下一帧”。按本题及教材需要新做域名解析的流程，H1 先运行 **DNS**，把域名换成 Web 服务器 IP；常规查询的封装为 **DNS 报文 →UDP 数据报 →IPv4 分组 →Ethernet II 帧**。ARP 用于在本地链路上解析 MAC，不是 DNS 外面的一层，也不是另一个应用层协议；CSMA/CD 是介质访问算法，不是帧格式，在全双工交换链路中通常不运行。

H1 先根据/25 判断本地 DNS 的 192.168.1.126 与自己同属 192.168.1.0/25，先用 ARP 解析本地 DNS 的 MAC 地址，不先问网关。解析出 Web 服务器位于外网后，才 ARP 查询网关 R 的 192.168.1.1。后续 IP 分组的最终目的仍是 Web 服务器，当前以太网帧的目的 MAC 却是 R 的 MAC。

顺序	S 收到的帧	入端口	S 的处理与新增学习
1	H1 广播 ARP，询问本地 DNS 的 MAC	4	学习 H1 的 cc→4；向 1、2、3 口泛洪
2	DNS 单播 ARP 应答给 H1	1	学习 DNS 的 bb→1；已知 cc，只转 4 口
3	H1 单播 DNS 查询给 DNS	4	已知 bb，只转 1 口
4	DNS 单播查询应答给 H1	1	已知 cc，只转 4 口
5	H1 广播 ARP，询问网关 R 的 MAC	4	向 1、2、3 口泛洪
6	R 单播 ARP 应答给 H1	2	学习 R 的 aa→2；只转 4 口
7	H1 向外网服务器发送 TCP SYN，帧发给 R	4	已知 aa，只转 2 口

不变量·应用数据量与 FIN 分开计

FTP 文件 18000 B，从 SYN 序号 100 开始，则首数据 101，末数据 18100，FIN 为 18101，对方确认 FIN 为 18102。控制连接已有自己的序号空间，不能混进这条数据连接的累计字节数。

顺序	S 收到的帧	入端口	S 的处理与新增学习
8	R 转发服务器的 SYN+ACK，帧发给 H1	2	已知 cc，只转 4 口
9	H1 发送 TCP 确认及首个 HTTP 请求	4	帧均发给 R，只转 2 口；到达题设 t_1

课程连接·应用消息和 TCP 段

TCP 按字节流分段，一条应用消息可以跨多个 TCP 段。

表中 cc、bb、aa 分别是下表完整 MAC 地址的末字节缩写。第三次握手 ACK 是否与首个 HTTP 请求合并，不改变学习或 H2 收到的帧数。交换机按**帧的源 MAC 与入端口**学习，不会从广播帧的目的地址、ARP 询问目标、IP 目的地址或题图上 “知道有 H2” 便自行添加表项。因此 t_1 时为：

MAC 地址	端口
00-11-22-33-44-cc（H1）	4
00-11-22-33-44-bb（本地 DNS）	1
00-11-22-33-44-aa（R）	2

H2 只被动收到两次广播 ARP 请求，没有发出自己的帧，故没有 dd→3 条目。两次单播应答到来时，交换机早已从相应请求学到 H1 的位置，不会因初始表为空便把所有单播一直泛洪。第（3）问按上述解析流程，H2 至少收到 **2 个 ARP 请求帧**，分别查询本地 DNS 和网关，目的 MAC 均为 **FF-FF-FF-FF-FF-FF**。

这个 2 帧结论以本题意图中的新 DNS 查询为前提。原题明确为空的是 ARP 表和交换表，并未额外声明 DNS 缓存为空；若允许命中已有 DNS 缓存，就不必先访问本地 DNS，相应学习表与最小广播数都会改变。应保留这一条件边界，不能把 “通过域名访问” 推广成任何情况下都必然重新发送 DNS 查询。

6.10 应用模型与协议服务依赖

C/S 模型由长期运行的服务器等待请求，客户可随时加入或离开。服务器集中管理资源，便于维护和访问控制，也可能形成性能和可用性瓶颈；增加副本和负载均衡是常见扩展。P2P 对等方既获取也提供资源，节点加入会同时带来负载和服务能力；节点可动态变化，定位、访问控制、内容一致性和拥塞仍需协议处理。

应用使用下层服务时，应先问所需的可靠性、时延、吞吐和通信模式，再选择承载。DNS 一般小查询常用 UDP，较大响应、区域传送等可用 TCP；FTP、传统 SMTP/POP3 和本章 HTTP/1.x 使用 TCP。应用层头部与正文也可以被分段，某一个 TCP 段并不天然等于一条完整应用消息。

6.11 DNS 记录与缓存的边界

DNS 资源记录除名称、类型和值之外，还含生存时间 TTL，用于限制缓存使用期限。A 记录给 IPv4 地址，AAAA 给 IPv6 地址，NS 指示权威服务器，MX 指定邮件交换服务器，CNAME 给出别名指向的规范名称。NS 和 MX 中的名称可能还需要解析地址；有了域名不代表已经知道最终 TCP 目的地址。

权威回答负责其管理范围内的数据，本地缓存可以提供尚有效的既有结果。命中目标记录时查询次数可为 0；仅命中某个权威服务器地

址时，可跳过根和顶级步骤，直接问该服务器；完全无相关缓存时按题设从根开始。迭代回答可能给出更接近目标的服务器，而不直接给目标主机地址，因此 “每次收到回答” 与 “域名解析结束” 不同。

辨析·信封与首部

MAIL FROM/RCPT TO 是投递信封，From/To 是邮件内容首部。

6.12 邮件的信封、内容与交互命令

邮件系统由用户代理、邮件服务器和投递/读取协议配合。用户代理负责编辑与呈现，发送服务器将待投递邮件排队，服务器之间由 SMTP 转送；收件用户再通过 POP3、IMAP 或 Web 接口访问邮箱。邮件地址形如`用户标识 @ 域名`，域名部分用于查找邮件服务，不等于邮件服务器 IP 地址的文本形式。

SMTP 基本交互包括建立 TCP 连接、问候、指定发件人、指定收件人、传送邮件内容、结束会话。`MAIL FROM`、`RCPT TO` 属于投递信封，邮件内容中的 `From:`、`To:`、`Subject:`属于邮件首部；多个收件人或转发条件下二者不一定一一相同。DATA 阶段结束标志按 SMTP 协议解释，邮件正文中出现相似内容时由协议透明传输规则处理。

传统 SMTP 以 7 bit ASCII 为基本传输背景。MIME 通过内容类型、传输编码、分段边界等描述非 ASCII 文本、图片和附件，让这些内容可通过相应邮件传送系统。Base64 是编码而不是加密，内容可以被公开解码；MIME 也不代替 SMTP 的实际投递。实际扩展 SMTP 可协商其他能力，应按题目给定的基础模型或扩展条件作答。

POP3 的一次 TCP 连接可处理多封邮件，经历鉴别、事务处理、更新等阶段。下载后删除与下载并保留是不同使用方式；用户端读取不等于 SMTP 服务器间重新投递。Web 邮箱中，浏览器与 Web 服务之间使用 HTTP 类交互，后端仍可能运行 SMTP 等邮件协议，不能根据浏览器界面就把整个邮件系统都归为 HTTP。

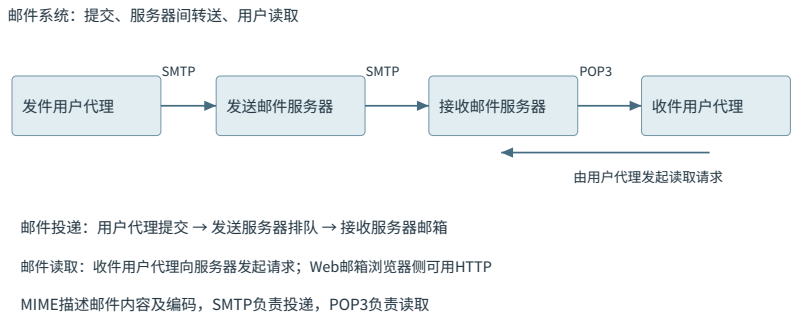


Figure 6.6: 邮件系统的提交、转送和读取路径

6.13 HTTP 报文、Cookie 与缓存验证

HTTP/1.x 请求包含请求行、首部行、空行与可选消息体；响应包含状态行、首部行、空行与可选消息体。请求行给出方法、请求目标与版本，响应状态行给版本、状态码和短语。GET 通常请求资源，HEAD 请求相应首部信息，POST 提交数据；具体语义以方法和资源接口为准。

```
GET /notes/network.html HTTP/1.1
Host: example.test
If-None-Match: "v2"
```

若缓存实体仍有效，服务器可返回 304，验证响应仍消耗一次请求/响应；若实体已变化，通常返回新的完整实体。200 表示成功，301/302 表示重定向类别，404 表示未找到相应资源，5xx 表示服务器错误类别；状态码本身不说明整条 TCP 连接是否已经关闭。

辨析·两种 Cookie 首部

Set-Cookie 来自响应，Cookie 随适用的后续请求发回。

服务器用 `Set-Cookie` 响应首部设置 Cookie，浏览器在后续适用请求中用 `Cookie` 首部带回相应信息。服务器可据标识找到会话状态；HTTP 的每次请求语义与 TCP 连接的持久性分别判断。Cookie 不能自动等同于安全认证，加密、访问范围和会话保护还取决于实际设置。

FTP 也要按消息内容区分连接：`LIST` 命令经控制连接提交，返回的目录列表经数据连接传输。主动模式由服务器发起数据连接，被动模式由服务器先通告监听端点、客户发起连接；控制连接独立维持，所以文件数据流不会把控制命令排在同一字节流后面。

6.14 主动回忆自测

以下为自拟题。时间题先画依赖链，协议题先说谁向谁执行什么动作。

1. HTML 引用 4 个同站小图片，DNS 已完成、TCP 未建立，忽略发送和其他开销。持久非流水与持久流水各需多少 RTT?
2. 本地 DNS 已有目标域权威服务器地址、没有目标主机地址；到权威服务器 RTT20 ms，TCP 建立和 HTTP 请求响应各 30 ms。局域网开销忽略，取纯文本页面至少多久?
3. 收到 304 响应，能否说明本次完全没有网络往返?
4. C/S 中的 “客户” 是否等于性能较低的机器？P2P 使用集中索引后是否就只能由中心转发全部文件数据?
5. 域与区为什么不必相同？A、AAAA、NS、MX 各表达什么?
6. 一次 FTP 登录与后续文件上传分别用哪类连接？所有 FTP 服务器数据端口都固定 20 吗?
7. SMTP、POP3 与 MIME 分别做什么？Base64 编码是否提供保密性?
8. URL 中的片段 `#part1` 是否通常随 HTTP 请求目标发给服务器? HTTP 无状态能否推出不能使用持久 TCP 连接?
9. 在新做 DNS 解析、ARP 表和交换表为空的局域网访问流程中，交换机从目的 MAC 还是源 MAC 学习端口？DNS 缓存命中会改变哪些步骤?

6.14.1 核对答案

1. 非流水 $1+1+4=6$ RTT，流水 $1+1+1=3$ RTT；两者均包括首次 TCP 握手。 **2.** DNS20 ms，加 Web 两次 30 ms，共 80 ms；无需重新查询根与顶级服务器。 **3.** 不能，304 是网络条件验证响应，只省略完整实体传输。

4. 客户主动请求服务；集中索引可仅协助发现，文件仍可在对等节点间传输。 **5.** 子域可委派给不同权威管理范围；四类记录依次指 IPv4、IPv6 地址、权威服务器和邮件交换。 **6.** 登录走控制连接，文件走数据连接；被动模式的数据端口由服务器通告。

7. SMTP 提交/转送、POP3 读取、MIME 描述内容类型与编码；Base64 可公开解码，不是加密。 **8.** 片段留在浏览器侧；无状态与连接复用是不同层次的问题，可以使用持久连接。 **9.** 从源 MAC 和入端口学习；DNS 命中可省去查询及其 ARP，改变交换表和广播计数。